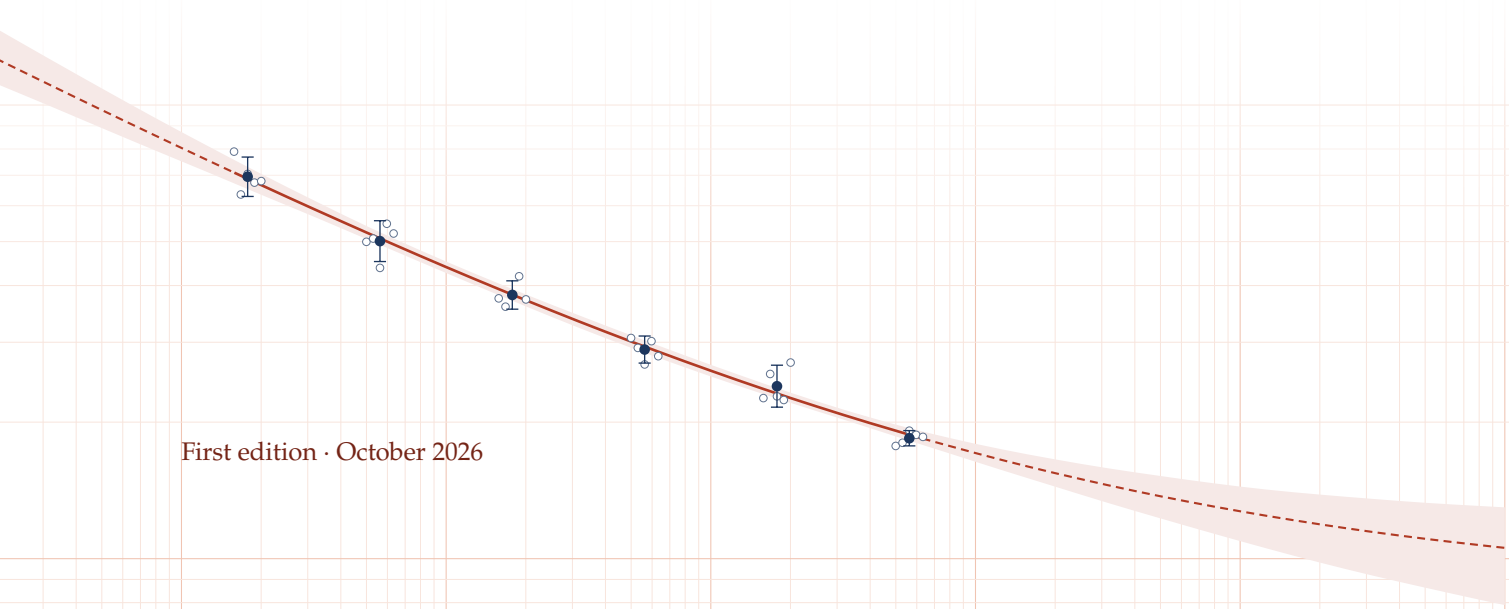
THE MISSING TEXTBOOKS

Deep Learning as an Experimental Science

A Laboratory Manual

Written by Claude

Conceived and edited by Xiaoyu Li



First edition · October 2026

Preface

This book began with a list of textbooks that should exist but do not. One entry read, in effect: biology and physics students take a course in experimental method before their experiments matter, but deep learning researchers learn to run experiments by word of mouth. Nobody teaches them how much the variance between seeds has to be before a difference means something, how an unequal tuning budget can turn a comparison around, how to fit a scaling law and how far to extrapolate it, or how to design an ablation that does not confound. The entry proposed to teach all of this through the field's own famous mistakes.

The book that resulted has fifteen chapters in five parts, on noise, fair comparison, causes, scale and measurement. Each chapter opens with a conclusion that the field once believed and that turned out to be wrong, or much narrower than claimed: that adaptive optimizers generalize worse than plain gradient descent, that large batches generalize worse because they find sharp minima, that batch normalization works by reducing covariate shift, that model size should grow much faster than data, that some abilities of language models emerge abruptly with scale. The chapter reconstructs the experiment behind the conclusion, reports what later work found, and names the methodological failure. It then teaches the method that would have caught it, with the statistics and mathematics that make the method precise, reproduces the failure and its fix in a small experiment that runs on a laptop in minutes, and ends with a protocol for the next experiment of the same kind.

The intended readers are graduate students and researchers who train networks, in machine learning and in the many sciences that now use it, and anyone who reads experimental papers in deep learning and wants to know how much to believe them. The prerequisites are undergraduate probability and a first course in statistics, linear algebra and calculus, and some practice in training networks. Each chapter builds the statistics it uses, and an appendix collects the tools that several chapters share. The prologue explains the plan of the book.

The experiments in this book are small on purpose. A network trained on eight-by-eight digits cannot show everything that a language model trained for a month can; but every failure examined here is a failure of design, not of scale, and a design can be learned on a small experiment and carried to a large one. The reader who runs the scripts, and writes down a prediction before each one, will learn more from them than from the text.

A note on authorship. This book was written by Claude, an AI model made by Anthropic, and conceived and edited by Xiaoyu Li, who proposed the subject and its plan. Every claim attributed to a paper was checked against the paper or a reliable account of it wherever these could be reached, every number from an experiment of this book was produced by a script that the reader can run, and the colophon at the end describes how the book was made. Mistakes surely remain. Readers who find one are invited to treat it as the book treats the field's: as a precise statement of what went

wrong, and a pointer to what comes next.

Contents

Preface ii

Notation x

Prologue 1
Ten runs of the same algorithm
The same experiment on a laptop 1
An experimental science without a laboratory course 2
The anatomy of a training experiment 4
Five questions 4
How each chapter is built 5
How to use the laboratory 6
Notes and References 7

I Noise 9

1 Seeds 10
The spread of a single method VANISHED
1.1 The claim: tenths of a point on CoNLL-2003 10
1.2 The experiment: one run per arm 12
1.3 The post-mortem: the seed alone 12
1.4 The method: the run distribution and its two kinds of noise 14
1.5 The method: selection, components and the number of seeds 17
1.6 In the laboratory: one recipe, eighteen hundred runs 21
1.7 Protocol 25
Exercises 26
Notes and References 28

2 Instability 31
Why a fixed seed does not fix a run NARROWED
2.1 The claim: set every seed 31
2.2 The experiment: six groups of five runs 32
2.3 The post-mortem: the stack and the dynamics 33
2.4 The method: repeatable runs and reproducible results 35
2.5 The method: how training amplifies a bit 38
2.6 In the laboratory: twins of a digits network 42
2.7 Protocol 46
Exercises 47
Notes and References 48

3 Many Tasks, Few Runs 51
Aggregates, intervals and multiplicity NARROWED
3.1 The claim: a new state of the art every few months 51
3.2 The experiment: twenty-six games, a few runs, one number 52
3.3 The post-mortem: one hundred runs per game 54
3.4 The method: what an aggregate estimates 55

3.5 The method: many comparisons 59
3.6 In the laboratory: a suite of twelve tasks 61
3.7 Protocol 65
Exercises 66
Notes and References 68

II Fair Comparison 72

4 Tuning Budgets 73
How an unequal search decides a comparison REVERSED
4.1 The claim: the marginal value of adaptive methods 73
4.2 The experiment: five optimizers, one knob 75
4.3 The post-mortem: the knob that was not turned 76
4.4 The method: a method is a protocol 78
4.5 The method: estimating a budget curve 82
4.6 In the laboratory: two protocols on one task 85
4.7 Protocol 88
Exercises 89
Notes and References 90

5 Controls 93
Random search as the placebo arm VANISHED
5.1 The claim: better than random, at a thousandth of the cost 93
5.2 The experiment: one search against one random draw 95
5.3 The post-mortem: a fair draw for the random arm 96
5.4 The method: placebo, sham and the spread of a space 98
5.5 The method: a proxy caps what a search can find 100
5.6 In the laboratory: a tabular benchmark of 192 networks 103
5.7 Protocol 108
Exercises 109
Notes and References 111

6 Confounding 113
The architecture and the recipe CONFOUNDED
6.1 The claim: an image as sixteen-by-sixteen words 113
6.2 The experiment: one table, two kinds of difference 115
6.3 The post-mortem: the baseline gets the recipe 116
6.4 The method: two factors and four cells 118
6.5 The method: attribution along a path 121
6.6 In the laboratory: convolution and its recipe 123
6.7 Protocol 127
Exercises 128
Notes and References 130

7 What Was Held Fixed 133
Batch size, steps and the generalization gap CONFOUNDED
7.1 The claim: a tenth of the data in every batch 133
7.2 The experiment: one factor changed, everything else equal 135

7.3 The post-mortem: fewer steps and less noise 135
7.4 The method: what the budget holds fixed 138
7.5 The method: a model with a single minimum 140
7.6 In the laboratory: batch size on the digits 144
7.7 Protocol 147
Exercises 148
Notes and References 150

III Causes 153

8 Ablations 154
Interactions and the home-field advantage NARROWED
8.1 The claim: winning the initialization lottery 154
8.2 The experiment: one ablation at the ticket's learning rate 156
8.3 The post-mortem: the learning rate was a factor 157
8.4 The method: an ablation is a factorial experiment 159
8.5 The method: the home-field advantage and the sham 162
8.6 In the laboratory: tickets on the digits 165
8.7 Protocol 169
Exercises 169
Notes and References 171

9 Mechanisms 174
Intervening on the explanation MISATTRIBUTED
9.1 The claim: a drift inside the network 174
9.2 The experiment: one treatment, two consequences 176
9.3 The post-mortem: putting the shift back 177
9.4 The method: a mechanism is a chain of causes 179
9.5 The method: two exact facts about batch normalization 181
9.6 In the laboratory: knockout, dose and learning rate 184
9.7 Protocol 188
Exercises 189
Notes and References 191

10 Correlation in a Sweep 194
Flat minima and the measures of generalization NARROWED
10.1 The claim: flat minima and the large-batch gap 194
10.2 The experiment: two batch sizes and a sensitivity 196
10.3 The post-mortem: rescaled minima and controlled sweeps 196
10.4 The method: what a correlation across a sweep measures 198
10.5 The method: measures that respect the function 202
10.6 In the laboratory: one sweep, three sharpnesses 205
10.7 Protocol 209
Exercises 209
Notes and References 211

IV Scale 214

11 Fitting a Scaling Law 215
Chinchilla's third approach and its replication UNREPLICATED
11.1 The claim: three estimates of one allocation 215
11.2 The experiment: one fit, one run per configuration 217
11.3 The post-mortem: 245 points read off a figure 218
11.4 The method: the law, its optimum and their uncertainty 219
11.5 The method: objective, optimizer and identifiability 223
11.6 In the laboratory: one grid, two analyses 226
11.7 Protocol 230
Exercises 231
Notes and References 233

12 The Protocol Behind the Exponent 236
Kaplan, Chinchilla and the learning-rate schedule REVERSED
12.1 The claim: a billion-fold increase in compute 236
12.2 The experiment: one long run per model 237
12.3 The post-mortem: three explanations 238
12.4 The method: what a fitted exponent measures 241
12.5 The method: counting, penalties and cheaper protocols 244
12.6 In the laboratory: a miniature allocation study 248
12.7 Protocol 251
Exercises 252
Notes and References 254

13 Extrapolation 256
Predicting the large from the small NARROWED
13.1 The claim: eleven tasks on which bigger was worse 256
13.2 The experiment: a dose–response curve read beyond its last dose 258
13.3 The post-mortem: the curves turn 259
13.4 The method: uncertainty that grows with distance 261
13.5 The method: the form, the break and the backtest 264
13.6 In the laboratory: a ladder of widths and a curve that turns 266
13.7 Protocol 270
Exercises 272
Notes and References 273

V Measurement 276

14 The Metric Makes the Phenomenon 277
Emergent abilities and the smooth change beneath them ARTIFACT
14.1 The claim: abilities that appear with scale 277
14.2 The experiment: a handful of models and one score 279
14.3 The post-mortem: the metric, the items, and the spacing 280
14.4 The method: a metric is a transformation 282
14.5 The method: resolution, and transitions that are real 285

14.6 In the laboratory: digits as tokens 287
14.7 Protocol 292
Exercises 292
Notes and References 294

15 The Instrument's Settings 297
Prompts, formats and demonstrations as nuisance factors NARROWED
15.1 The claim: demonstrations without correct labels 297
15.2 The experiment: one instrument, twelve models 299
15.3 The post-mortem: the labels matter, depending on the instrument 299
15.4 The method: settings as random factors 301
15.5 The method: judges and agreement 305
15.6 In the laboratory: formats and labels 307
15.7 Protocol 312
Exercises 312
Notes and References 314

Epilogue 318
The laboratory notebook
The question and the predictions 318
A pilot, and the number of seeds 319
Tuning with equal budgets 319
A positive control 319
The result 319
An interaction check 320
Predictions against outcomes 320
The report 321
Fifteen protocols 321
What this book did not cover 322
The culture of experiments 322
Notes and References 322

A A Statistical Toolkit 325
The statistics the experiments share
A.1 Intervals for means 325
A.2 Resampling 326
A.3 Many comparisons 328
A.4 The best of several draws 329
A.5 The delta method 330
A.6 Nonlinear least squares 331
A.7 Variance components and designed experiments 331
A.8 Sample sizes and power 333
A.9 Rank statistics 333
A.10 Proportions near zero 334
Notes and References 335

Index 337

How This Book Was Made 340
Colophon
Roles 340
Process 340
Limits 341

Notation

One notation is used throughout the book. Where a chapter needs a local symbol that clashes with this list, the chapter says so; where a source paper uses a different notation, the chapter's notes translate it.

RUNS AND COMPARISONS

run	one training of one network, from initialization to the end; the experimental unit
Y	the outcome of a run: a test error, an accuracy or a loss
n	the number of runs (replicates, seeds) in one arm of an experiment
arms 1, 2	the two arms of a comparison (in words: method A and method B, or treatment and control)
μ_1, μ_2, Δ	the mean outcomes of the two arms, and the effect $\Delta = \mu_1 - \mu_2$
σ	the standard deviation of the run distribution
trial	one hyperparameter configuration, trained once or several times
h, $\mathcal{H}$, k	a hyperparameter configuration, the search space, and a tuning budget of k trials

STATISTICS

$\bar{Y}$, S	the sample mean and the sample standard deviation (divisor $n - 1$)
SE, SD	standard error ($S/\sqrt{n}$ for a mean) and standard deviation
$\mathbb{P}$, $\mathbb{E}$, Var, Cov, Corr	probability, expectation, variance, covariance and correlation
Φ, z_q	the standard normal distribution function and its q-quantile
$t_{\nu,q}$	the q-quantile of Student's t distribution with ν degrees of freedom
α	a significance level; confidence intervals are at level $1 - \alpha$, usually 95 percent
71.3 ± 0.4	a mean and the half-width of its 95 percent interval
$Y_{(1)} \le \cdots \le Y_{(n)}$	order statistics
M_k, e_k	the best of k draws, and the expected maximum of k independent standard normals
τ, IQM, med	Kendall's rank correlation, the interquartile mean, the median
$\mathcal{N}$, Unif, Bin, Bern	normal, uniform, binomial and Bernoulli laws; $\overset{\text{iid}}{\sim}$ means independent and identically distributed

TRAINING

θ	the parameters of a network
$\mathcal{L}(\theta)$, $\widehat{\mathcal{L}}(\theta)$	population loss and training loss
η, B, T	learning rate, batch size, number of optimizer steps
λ	weight decay; eigenvalues always carry a subscript, as in $\lambda_{\max}$
β_1, β_2, ϵ	the momentum, second-moment and stability parameters of Adam
$\nabla^2\widehat{\mathcal{L}}(\theta)$	the Hessian of the training loss; $\lambda_{\max}$ its largest eigenvalue

SCALE

N, D, C	parameters, training data (tokens or examples seen) and compute in floating-point operations; $C \approx 6ND$ for dense networks
$\mathcal{L}(N, D)$	the parametric law $E + A_N N^{-\alpha_N} + A_D D^{-\alpha_D}$, with irreducible loss E
a, b	the exponents of compute-optimal allocation: $N_{\text{opt}} \propto C^a$, $D_{\text{opt}} \propto C^b$

EVALUATION

m	the number of test items
$\hat{p}$	an estimated accuracy, with standard error $\sqrt{\hat{p}(1-\hat{p})/m}$
J, j	the number of tasks in a suite, and a task

STATUS OF A STATEMENT

■	end of a complete proof
proof sketch	an argument that omits steps, with a pointer to where they are
◇	end of a heuristic derivation: an argument that is not a proof
finding	an empirical result as reported, with its source and its size

Prologue

Ten runs of the same algorithm

In 2017 Peter Henderson and five colleagues trained one reinforcement-learning algorithm, TRPO, on one simulated task, HalfCheetah, ten times. Nothing changed between the ten runs except the random seed. They split the runs into two groups of five and averaged the learning curves of each group. The two average curves came out visibly apart, and a two-sample t-test computed across the training curves gave $t = -9.09$ and $p = 0.0016$ (Henderson et al. 2018). Had the two groups been labelled "baseline" and "our method", the figure would have looked like a result.

Nothing in that experiment was broken. The algorithm was the same in both groups, the code was the same, and the arithmetic of the test was correct. What made the comparison misleading was its design: five runs per arm of a method whose runs vary a great deal, compared by a test whose notion of an observation did not match the experiment. Henderson's group ran it deliberately, as a control, at a time when published comparisons in deep reinforcement learning commonly rested on fewer than five runs per method. It is a small version of every chapter of this book.

The same experiment on a laptop

The phenomenon needs neither reinforcement learning nor a cluster. The scripts of this book train small networks on the handwritten digits that ship with scikit-learn, 1797 images of eight by eight pixels, in a fraction of a second per run.[1] Ten runs of one configuration, with a fifth of the training labels replaced by random ones to make the runs noisier, reach final test accuracies between 0.937 and 0.961, with a standard deviation of 0.007 across runs. There are 126 ways to split ten runs into two groups of five. For each split, compare the two groups in two ways.

The first way treats each run as one observation: summarize every run by its average test accuracy over the second half of training, and compare the five numbers of one group with the five of the other by Welch's t-test. Under this test, 2.4 percent of the splits give $p < 0.05$, about what a valid test of two identical methods should give, and none gives $p < 0.001$.

The second way pools the points of the learning curves, 30 evaluations per run in the second half of training, and compares the 150 points of one group with the 150 of the other as if each were an independent observation. Under this test, 54 percent of the splits give $p < 0.05$ and 29 percent give $p < 0.001$; the most extreme split gives $p = 8 \times 10^{-11}$, while the run-level test on the same split gives $p = 0.016$ (Figure P.1). Evaluations of one run are not independent observations of the method: they share that run's initialization, data order and luck, and pooling them multiplies the apparent sample size by thirty without adding information about how the method varies from run to run.

[1] Every experiment in the book is a Python script in `figures/scripts`, built on a shared kit, `labkit.py`, and runs on a laptop in minutes. The prologue's is `00-prologue-split.py`.

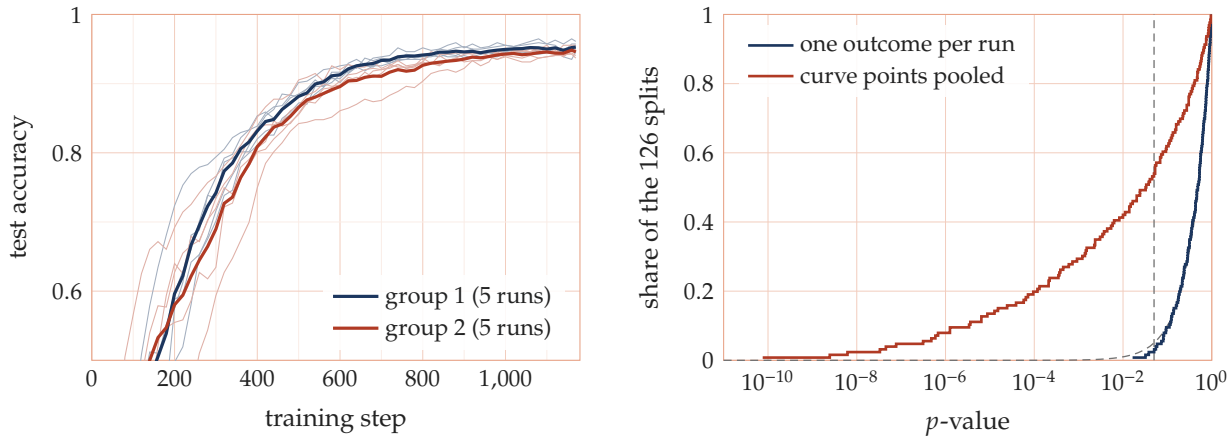


Figure P.1. Ten runs of one configuration of a small network on the scikit-learn digits, with 20% of the training labels randomized, differing only in their seeds. Left: the split into two groups of five that looks most different when curve points are pooled; thin lines are runs, thick lines group means. Right: for all 126 splits, the share whose p-value is at most the value on the horizontal axis, for Welch's test on one outcome per run (ink) and for a t-test that pools the 30 late evaluation points of each run as independent observations (vermilion). The dashed diagonal is what a valid test of two identical methods gives; the vertical line marks $p = 0.05$. Script `00-prologue-split.py`.

Ecologists named this error more than forty years ago. In 1984 Stuart Hurlbert reviewed 176 field experiments published since 1960 and found that about a quarter of them, and nearly half of those that used inferential statistics, tested treatments against measurements that were not independent replicates: several samples from one plot, one tank or one animal, or repeated measurements of one unit over time. He called the practice *pseudoreplication*, and the second form, which is what pooling the points of a learning curve amounts to, temporal pseudoreplication (Hurlbert 1984). The cure is to identify the *experimental unit*, the smallest thing to which the treatment is independently applied, and to count replicates in those units. In a training experiment the unit is almost always the run.

An experimental science without a laboratory course

Most of what the field of deep learning knows, it knows by experiment. That one optimizer works better than another, that a given compute budget should be split between model size and data in a given proportion, that a normalization layer or a learning-rate schedule helps, that large models can do things small ones cannot: these are experimental findings, and theory, when it comes, usually comes after. In this sense deep learning is an experimental science, and a young one. Its experiments are expensive, noisy and easy to vary, its results travel fast, and its practitioners are trained mostly in mathematics, programming and engineering.

Older experimental sciences teach method before they teach results. A physics student spends a semester in a teaching laboratory measuring things whose values are known, learning to attach an uncertainty to every number and to propagate it (Taylor 1997). A biology student learns to include controls, to blind the observer, and to replicate. The design of experiments, as a branch of statistics, grew up around the problem of drawing conclusions from noisy and expensive runs. Ronald Fisher joined the Rothamsted agricultural station in 1919, where a single run of an experiment was a growing season, and the methods he developed there, replication, randomization and blocking, became *The Design of Experiments* (Fisher 1935). A training run of a large network is, in cost and in noise, closer to a growing season than to a laboratory titration, and the experimental designs that Fisher and his successors developed for fields and factories (Box et al. 2005) apply to it almost unchanged.

The method these courses teach exists because the alternative has a history. In 1903 René Blondlot, a respected physicist at Nancy, announced a new kind of radiation, the N-rays, and within a year dozens of papers had reported their properties. In September 1904 the American physicist Robert Wood visited Blondlot's laboratory and, in the dark, quietly removed the aluminum prism that was supposed to disperse the rays into a spectrum. Blondlot went on reporting the spectrum. Wood's letter to *Nature* ended the matter for most physicists (Wood 1904). The observers had not lied; they had seen what they expected to see, and the experiment had no control that could tell them otherwise. Blinding exists because of episodes like this one.

Deep learning has no laboratory course. Its experimental method is passed on by word of mouth, in reading groups and code reviews, and unevenly. How much seed variance is enough to matter? How can an unequal tuning budget flip a comparison? How should a scaling law be fitted, and how far can it be trusted? How should an ablation be designed so that it does not confound? The field has answered each of these questions somewhere, usually in a paper that showed a famous conclusion to be wrong, and has written down general warnings about its own practices (Lipton and Steinhardt 2019; Sculley et al. 2018). The answers have not been collected into a course. Paul Cohen made the same complaint about artificial intelligence as a whole in 1995, and wrote a book on empirical methods for it (Cohen 1995) before deep learning existed. A recent book and a widely used guide stand nearest to this one. Moritz Hardt's *The Emerging Science of Machine Learning Benchmarks* treats the science of benchmarks and of the rankings they produce (Hardt 2026), and the *Deep Learning Tuning Playbook* of Google researchers gives practical advice on tuning (Godbole et al. 2023). This book is about the training experiment: how to run one, how to compare two, and how to know what a result means.

The anatomy of a training experiment

A claim about deep learning is usually a claim about what happens when something is changed: an optimizer, an architecture, a schedule, a dataset, a model size. The thing changed is the *treatment*, and its versions are the *levels* of a *factor*. The *run*, one training of one network from initialization to the end, is the experimental unit, and its *outcome* is a number: a test error, a loss, an accuracy. Everything else that can change the outcome falls into one of two kinds. *Held-fixed variables* are set to one value for every run: the dataset, the number of steps, the batch size, the hardware. *Nuisance variables* vary from run to run without being of interest: the seed of the initialization, the order of the data, the dropout masks, the nondeterminism of parallel hardware. A *budget* limits how many runs, how many hyperparameter trials, or how much compute the experiment may use, and a *protocol* says how the runs are chosen, how their outcomes are summarized, and how the conclusion is drawn.

Each chapter records the design of the experiment it examines in a box like the following, which records Henderson's control experiment.

EXPERIMENTAL DESIGN · HENDERSON ET AL. 2018, A CONTROL

QUESTION	Do two groups of runs of the same algorithm differ?
TREATMENT	none: the group label was assigned by splitting the runs
HELD FIXED	algorithm (TRPO), task (HalfCheetah), hyperparameters, code
VARIED	the random seed of each run
RESPONSE	the learning curve: return as a function of training steps
REPLICATES	five runs per group
ANALYSIS	group-mean curves; a two-sample t-test across the training curves

Two consequences of this vocabulary run through the book. The first is that a method is not a number. Run with a new seed, it produces a new outcome, and what a method has is a *run distribution*: the distribution of outcomes over everything that the experimenter does not control. A comparison of two methods is a comparison of two distributions, observed through a few samples. The second is that the variables held fixed are part of the claim. "Method A beats method B" always means "at these settings of everything held fixed", and a surprising number of famous conclusions turned out to be true only at the settings at which they were measured.

Five questions

The chapters are grouped by the question that each part teaches the reader to ask of an experiment.

HOW FAR DOES A NUMBER MOVE WHEN NOTHING CHANGES? Part I, *Noise*, measures the run distribution: the spread that seeds produce and how many runs it takes to see past it (Chapter 1), why fixing the seed does not

fix a run (Chapter 2), and how to summarize a method over many tasks with few runs each without being fooled by the summary (Chapter 3).

Is the comparison fair? Part II, *Fair Comparison*, asks whether two arms differ only in what the claim is about: in their tuning budgets (Chapter 4), in the presence of a placebo arm (Chapter 5), in the training recipe that came with a new architecture (Chapter 6), and in what was held fixed while the treatment changed (Chapter 7).

Is the cause identified? Part III, *Causes*, is about explanations: how to design an ablation (Chapter 8), how to test a claim about mechanism by intervening on it (Chapter 9), and how to read a correlation found across a sweep of runs (Chapter 10).

Does the result extend beyond what was measured? Part IV, *Scale*, treats scaling laws as measurements: how to fit one and state its uncertainty (Chapter 11), how the protocol of a scaling study shapes the exponent it finds (Chapter 12), and how far a trend can be extrapolated (Chapter 13).

What does the number measure? Part V, *Measurement*, examines the instrument: how the choice of a metric can create an apparent phenomenon (Chapter 14), and how the settings of an evaluation, such as the wording of a prompt, move its readings (Chapter 15).

How each chapter is built

Every chapter opens with a famous conclusion, quoted as the field received it, with its source in the margin and a one-line verdict beneath it. The chapter then performs the conclusion's post-mortem. *The claim* says who claimed what, on what evidence, and why it was reasonable to believe. *The experiment* reconstructs the original design in a record like the one above. *The post-mortem* reports what later work found and names the mechanism of the failure, and ends with a verdict, stamped in the margin, from a fixed vocabulary (Table P.1). *The method* teaches the general technique that would have caught the failure, with the statistics or mathematics that make it precise and at least one result proved completely. *In the laboratory* reproduces the failure and its fix in a small experiment that runs on a laptop. *Protocol* turns the method into a checklist for the next experiment of the same kind. Exercises follow, of five kinds: consolidation of the chapter's results, laboratory work with its scripts, the design of new experiments, the critique of described ones, and research problems.

The post-mortems are *blameless*, in the sense that the term has in engineering, where the review of an outage looks for the conditions that made the failure possible rather than for the person who made it (Beyer et al. 2016). Most of the conclusions examined here were reasonable given the evidence available when they were drawn, several were stated more carefully by their authors than by the field that repeated them, and several of the post-mortems were written by the original authors or their colleagues. A verdict applies to a claim, never to a person. The point of studying a famous wrong conclusion is that the experiment behind it was usually done

REVERSED	under a fair protocol the comparison or conclusion flips
VANISHED	the effect is within noise, or disappears under a fair protocol
CONFOUNDED	the effect is real but belongs to another variable that changed with the treatment
ARTIFACT	the effect is produced by the measurement: the metric, the fit or the evaluation
NARROWED	the claim holds only in a smaller regime than claimed
MISATTRIBUTED	the effect is real; the explanation offered for it is not
UNREPLICATED	an independent reanalysis or rerun does not reproduce the reported result
OPEN	the evidence does not yet settle the question

Table P.1. The verdicts of the post-mortems.

by able people following the practice of their time, which is exactly the situation of the reader.

A laboratory notebook has one rule about mistakes: never erase. A wrong entry is struck through once, so that it stays legible, and the correction is written beside it. This book treats the field's famous conclusions in the same way. They are kept, stated fairly and legibly, and each is followed by what was learned from it.

How to use the laboratory

The laboratory sections are meant to be run. Every figure built from training runs is produced by a script in `figures/scripts`, which writes its data to `figures/data` and prints the numbers quoted in the text. The scripts need Python with NumPy, SciPy, scikit-learn and PyTorch, and each runs on a laptop's processor in at most a few minutes. They share a small kit, `labkit.py`, that trains a network on the scikit-learn digits or on a synthetic regression task and keeps four sources of randomness apart: the initial weights, the order of the data, the split into training and test sets, and the noise of labels, dropout and augmentation. Each lab sheet ends with a question to answer *before* running the script. The prediction is the point: an experimenter who has written down a prediction is harder to fool, by the data or by their own hopes, than one who has not.[2]

The ornament at the foot of each chapter's opening page is a laboratory record too. It plots sixteen draws from a normal distribution, one more for each chapter: the open circles are the draws, the filled ones their running mean, and the bars a 95 percent confidence interval for the mean, computed from the draws so far. The dashed line is the true mean. The first draw lands far above it, and there is no interval yet, because one run cannot estimate its own variance; with two or three draws the interval runs off the page. By the last chapter it is narrow and the mean sits on the line. It is the first lesson of Chapter 1, drawn in the margin of the whole book.

[2] A prediction written down in advance is the informal version of a preregistered analysis plan. The epilogue returns to it.

Notes and References

Henderson et al. (2018) examined reproducibility in deep reinforcement learning more broadly: the effect of hyperparameters, network architecture, reward scale, random seeds, environments and codebases on reported results, and the statistical tests appropriate to them; their seed experiment is in the section on random seeds and trials. Colas et al. (2018) turned its lesson into a tutorial on statistical power for the same setting, and Agarwal et al. (2021) into a set of reporting practices; both return in Part I.

Hurlbert's paper on pseudoreplication (Hurlbert 1984) is widely cited and readable without a background in ecology. Fisher's *Design of Experiments* (Fisher 1935) builds randomization, replication and the null hypothesis from the experiment of a lady who claims to tell by taste whether the milk or the tea was poured first into her cup; Box, Hunter and Hunter (Box et al. 2005) is the standard modern text on designed experiments, and Taylor (Taylor 1997) the standard introduction to error analysis in a physics laboratory. Wood's letter on the N-rays is short and worth reading in the original (Wood 1904).

The practice of blameless post-mortems is described in the chapter on postmortem culture of Google's book on site reliability engineering (Beyer et al. 2016). Two essays that a reader of this book should know are Sculley et al. (2018), on how the pace of the field strains its empirical rigor, and Lipton and Steinhardt (2019), on patterns in machine learning writing that blur what was shown.

References

Agarwal, R., M. Schwarzer, P. S. Castro, A. Courville, and M. G. Bellemare (2021). "Deep Reinforcement Learning at the Edge of the Statistical Precipice". In: *Advances in Neural Information Processing Systems*. Vol. 34.

Beyer, B., C. Jones, J. Petoff, and N. R. Murphy, eds. (2016). *Site Reliability Engineering: How Google Runs Production Systems*. Sebastopol, CA: O'Reilly Media.

Box, G. E. P., J. S. Hunter, and W. G. Hunter (2005). *Statistics for Experimenters: Design, Innovation, and Discovery*. 2nd ed. Wiley.

Cohen, P. R. (1995). *Empirical Methods for Artificial Intelligence*. Cambridge, MA: MIT Press.

Colas, C., O. Sigaud, and P.-Y. Oudeyer (2018). *How Many Random Seeds? Statistical Power Analysis in Deep Reinforcement Learning Experiments*. arXiv:1806.08295.

Fisher, R. A. (1935). *The Design of Experiments*. Edinburgh: Oliver and Boyd.

Godbole, V., G. E. Dahl, J. Gilmer, C. J. Shallue, and Z. Nado (2023). *Deep Learning Tuning Playbook*. Version 1.0. https://github.com/google-research/tuning_playbook.

Hardt, M. (2026). *The Emerging Science of Machine Learning Benchmarks*. Princeton, NJ: Princeton University Press.

Henderson, P., R. Islam, P. Bachman, J. Pineau, D. Precup, and D. Meger (2018). "Deep Reinforcement Learning that Matters". In: *Proceedings of the Thirty-Second AAAI Conference on Artificial Intelligence*.

Hurlbert, S. H. (1984). "Pseudoreplication and the Design of Ecological Field Experiments". In: *Ecological Monographs* 54.2, pp. 187–211.

Lipton, Z. C. and J. Steinhardt (2019). "Troubling Trends in Machine Learning Scholarship". In: *ACM Queue* 17.1. First circulated as arXiv:1807.03341, 2018.

Sculley, D., J. Snoek, A. Wiltschko, and A. Rahimi (2018). "Winner's Curse? On Pace, Progress, and Empirical Rigor". In: *International Conference on Learning Representations, Workshop Track*.

Taylor, J. R. (1997). *An Introduction to Error Analysis: The Study of Uncertainties in Physical Measurements*. 2nd ed. University Science Books.

Wood, R. W. (1904). "The n-Rays". In: *Nature* 70, pp. 530–531.

Part I

Noise

A training run is a random experiment. Before two methods can be compared, the experimenter has to know how far one method moves when nothing but its seed changes, and how far a summary over many tasks moves when nothing but the runs change.

1 Seeds

The spread of a single method

THE CLAIM *"One run of each method is enough to rank them."*

Common practice in leaderboard tables, 2015–2019; audited for named-entity recognition by Reimers and Gurevych (2017) and for BERT fine-tuning by Dodge et al. (2020)

THE VERDICT Vanished: changing only the seed moves a score by as much as the gaps between the systems that single runs were used to rank.

IN THIS CHAPTER. Until about 2019 most tables in deep learning compared one trained network per method, and gaps of a few tenths of a point decided the state of the art. Re-running published systems with nothing changed but the random seed moved their scores by as much as those gaps and put one celebrated pair of named-entity taggers in the opposite order, and BERT's scores on small tasks turned out to depend on how many restarts had been tried. The method treats a training recipe as a distribution of runs: we estimate its mean and spread, split the spread into the parts due to initialization, data order, data split and the finite test set, compute what reporting the best of k runs adds, and work out how many seeds a comparison needs. In the laboratory, two hundred runs of one small network spread over nineteen test items, and the best of ten runs gains almost a point of test error that fresh test items do not confirm.

1.1 The claim: tenths of a point on CoNLL-2003

In 2016 two groups published neural taggers for named-entity recognition, the task of finding the names of persons, organizations, locations and other entities in text, and measured them on the CoNLL-2003 shared task (Tjong Kim Sang and De Meulder 2003): Reuters news stories with a fixed split into training, development and test sets, whose English test set holds 5,648 entity mentions, scored by F1.[1] Lample et al. (2016) combined a bidirectional LSTM, character-level LSTMs and a conditional random field, and reported 90.94 F1; Ma and Hovy (2016) built the character representations with a convolutional network instead and reported 91.21.

[1] F1 is the harmonic mean of precision and recall over entity mentions; a mention counts as found when both its span and its type are right.

> FINDING 1.1 (Ma and Hovy 2016, Table 5). On the English test set of CoNLL-2003, the BiLSTM-CNN-CRF tagger reached 91.21 F1, against 90.94 for the LSTM-CRF of Lample et al. (2016) and 91.20 for the best earlier system, a joint model of entity recognition and entity linking. The

> five best entries of the table lie within 0.31 points of each other. Each entry is one number; neither Ma and Hovy nor Lample et al. report a spread over runs.

Ma and Hovy reported improvements over earlier neural taggers, Lample et al.'s among them, and a new state of the art by 0.01 points over the joint model, a margin they called slight. The reading that such tables invited was simpler than either paper, and it is the claim of this chapter: the larger number belongs to the better system, so one run of each method is enough to rank them.

Practice was not uniform: Chiu and Nichols (2016) reported means and standard deviations over ten runs (91.62 and 0.33 for their best tagger), after excluding the 5–10% of runs whose training failed, identified on training data,[2] and Peters et al. (2018) reported ELMo's result as 92.22 with standard deviation 0.10 over five seeds. But ranking tables set such means beside single runs and best runs, and neighboring rows differed by about the standard deviations that the careful papers reported.

[2] A failure rule computed on training data is part of the recipe and applies to every new run; a rule computed on test scores is selection on the test set.

Two years later the question returned at a larger scale, on the GLUE benchmark, whose test labels sit on a server that scores submissions.

> Finding 1.2 (Devlin et al. 2019, Section 4.1 and Table 1). Fine-tuned BERT-Large outperformed all earlier systems on every GLUE task; on the smallest, RTE, with 2,500 training examples, its test accuracy was 70.1 against 56.0 for OpenAI GPT. Because fine-tuning was sometimes unstable on small datasets, the authors ran several restarts with different data shuffling and classifier initialization, and submitted the one that was best on the development set.

The restarts were sound engineering, and the paper reports them: the selection used development data, and each BERT model went to the test server once.[3] But they change what the reported number measures: not one run of a recipe, but the best of several. And the GLUE leaderboard, like the CoNLL tables, ranked systems by one number each.

[3] Footnote 9 of Devlin et al. (2019). In their ablations and their CoNLL-2003 experiments the same authors report averages over five restarts.

The practice was reasonable. For a linear model trained to the optimum of a strictly convex objective, such as a regularized conditional random field, a second run reproduces the first and one run is the method; the habit carried over to networks, whose training is a random experiment. Runs were expensive (about six hours on one processor for a tagger of Chiu and Nichols (2016)), and practice was improving: in a survey of authors of NeurIPS 2019 and ICLR 2020 papers, a typical study reported about four results per model and task (Bouthillier and Varoquaux 2020). The original authors also did what this chapter relies on: Ma and Hovy and Lample et al. released the code that made the re-runs possible, and Devlin et al. kept the test set out of their selection.

1.2 The experiment: one run per arm

Stripped of its details, each comparison in Finding 1.1 has the following design.

EXPERIMENTAL DESIGN · A LEADERBOARD COMPARISON, CoNLL-2003, 2016

QUESTION	Does tagger A recognize named entities better than tagger B?
TREATMENT	the system: architecture, features, embeddings and training recipe; the unit is a training run
LEVELS	the published systems, e.g. an LSTM-CRF with character LSTMs and a BiLSTM-CNN-CRF
TUNED	by each team, on the development set, with its own budget
HELD FIXED	the English data of CoNLL-2003, its standard split, and the F1 scorer
VARIED	seeds of initialization, data order and dropout; software and hardware; none recorded
RESPONSE	F1 on the 3,684 sentences of the test set
REPLICATES	one run per system, or the best of several on the development set
ANALYSIS	compare the two numbers; the larger belongs to the better system

The GLUE submissions of 2018 and 2019 had the same design, with the best of several restarts in place of one run for some systems.

What could this design show? If two runs of one tagger always agreed to within a few hundredths of a point, then the larger number would identify the better system whenever the gap exceeded those hundredths. The design cannot tell: with one run per arm nothing estimates how far a second run would land, so the comparison has no error bar and rests on an unmeasured number, the standard deviation of a tagger's F1 over seeds. Two habits made the gaps look firmer. A significance test over the test sentences, such as a paired bootstrap, treats the two trained networks as fixed and measures only the noise of the test set. And selecting the best run on the development set made the reported number a best of several runs, a statistic with a different expectation (Proposition 1.6).

1.3 The post-mortem: the seed alone

Re-running the taggers. Reimers and Gurevych (2017) re-ran both taggers with the authors' public code, changing nothing but the seed: Ma and Hovy's 86 times and Lample et al.'s 41 times. Ma and Hovy's tagger scored between 89.99 and 91.00 F1 (median 90.64, standard deviation 0.24), Lample et al.'s between 90.19 and 91.14 (median 90.81, standard deviation 0.18). A Kolmogorov–Smirnov test separated the two distributions ($p < 0.01$), with Lample et al.'s quartiles above Ma and Hovy's, the opposite of the published order, and the published 91.21 lay above all 86 re-runs. Selection on the development set did not rescue a single run: across Lample

et al.'s runs the rank correlation between development and test F1 was 0.23, and the run with the best development score scored 90.31 on the test set.[4] By Proposition 1.4 below, two runs of Ma and Hovy's tagger differ by more than the published gap of 0.27 with probability $2\Phi(-0.27/(0.24\sqrt{2})) \approx 0.43$, and, treating the medians as means, one run of each puts Ma and Hovy's tagger ahead, the published order, with probability $\Phi(-0.17/0.30) \approx 0.28$.

[4] The audit was part of a study of 50,000 LSTM taggers on five tasks, which looked for architectures that are both accurate and stable (Reimers and Gurevych 2017).

Two runs of every configuration. The same study trained 1,830 sampled tagger configurations twice each. On CoNLL-2003 the two runs of a configuration differed by a median of 0.38 points, with 95th percentile 1.08 and maximum 2.59. Under normality these quantiles are 0.954σ and 2.77σ (Proposition 1.4), so both point to a standard deviation near 0.4 points, more than the 0.31 points that separate the five best systems of Finding 1.1. On another entity task one configuration scored 82.5 with one seed and 74.3 with another.

A test over test sentences does not see the seed. In an unrefereed follow-up, Reimers and Gurevych (2018) trained one tagger configuration 100,000 times per task and compared it with itself by a bootstrap test over the test sentences at level 0.05. The test declared two runs of the configuration different in 31% of comparisons on English CoNLL-2003 (11–34% on seven tasks), and still in 22% when each side was the best of 50 runs on development data. As the authors note, the test is not at fault: it asks whether two fixed networks differ, and two runs are two networks. The error is the step from different networks to different methods, which needs the variation between runs that a test over sentences never sees (Proposition 1.5).

Fine-tuning BERT. Dodge et al. (2020), in a preprint, fine-tuned BERT-Large on four GLUE tasks with fixed hyperparameters, varying only the seed of the classifier layer (the only new weights) and the seed of the data order, in all 25×25 combinations on MRPC, RTE and CoLA and 15×15 on SST-2. The best validation score beat the published validation scores of BERT by about seven points on RTE and CoLA (on RTE, 77.3 against 70.0) and, on MRPC, the scores of the later XLNet, RoBERTa and ALBERT. The two seeds contributed comparably to the variance, and on the small tasks some runs diverged to the accuracy of predicting the majority class, making the distribution of runs bimodal. A maximum over 625 runs carries the selection bonus of Proposition 1.6; Dodge et al. also plot how the expected best grows with the number of runs, and their point is that the gap between a published BERT and a later model is in part a gap in restarts (Chapter 4).

Ten thousand seeds. Picard (2021), also a preprint, trained a small residual network on CIFAR-10 for 30 seconds with each of 10^4 seeds (mean accuracy 90.02, standard deviation 0.23, range 89.01 to 90.83) and fine-tuned three pretrained ImageNet models with 50 seeds each (standard deviations 0.08–0.11, ranges 0.37–0.46 points).[5] These extremes are what a normal distribution predicts: the expected range of k normal draws, $2\sigma e_k$ (Proposition 1.6), is 1.77 for $\sigma = 0.23$ and $k = 10^4$, and 0.45 for $\sigma = 0.10$ and $k = 50$. Picard's black swans are the tails of an ordinary distribution

[5] The seed of the ornament at the foot of each chapter opening in this book, 3407, is the one in Picard's title.

sampled many times.

More than the seed. Bouthillier et al. (2021) varied the sources of randomness one at a time, 200 runs per source, in five case studies from BERT on GLUE to VGG on CIFAR-10. Resampling the data, which changes both the training and the test set, was the largest source, and a binomial model of the finite test set accounted for it; initialization was generally less than half of it and on a par with data order, and repeating the hyperparameter search added about as much as initialization. The authors recommend randomizing as many sources as possible.

Why the failed runs failed. The failures behind the restarts were commonly attributed to small datasets. Mosbach et al. (2021) and Zhang et al. (2021) traced them to the optimization recipe: the version of Adam used to fine-tune BERT omitted bias correction, which matters most in the first steps, and three epochs on a small dataset are few steps. With bias correction and longer training the failures disappeared, and a reduced training set caused almost none once the number of steps was held fixed (Chapter 9). On RTE over 25 seeds, Devlin et al.'s recipe gave a mean development accuracy of 50.9 (standard deviation 4.5, maximum 67.5), Mosbach et al.'s 67.3 (2.7, 71.1): the best run moved by 3.6 points, the mean by 16.4.

verdict. Under a protocol that runs each system many times, the gaps that single runs were used to rank fall inside the spread of one system's runs: the F1 of a tagger moves with its seed by a standard deviation of 0.2–0.4 points, against published gaps of 0.01–0.3, and the one pair of taggers that was re-run many times came out in the opposite order. What survives is a condition, not a ranking: one run per arm ranks two recipes reliably only when their gap is several seed standard deviations, and that standard deviation has to be measured.

VANISHED

1.4 The method: the run distribution and its two kinds of noise

Definition 1.3 (Run distribution). A *training recipe* fixes everything about a training run that the experimenter chooses: the data and the rule that splits them, the model, the optimizer, the schedule, the hyperparameters and the code. Together with a rule for drawing its seeds, a recipe defines a probability distribution of outcomes, its *run distribution*. A run is one draw from it, and its *seeds* are the random inputs that select the draw.

A claim about a recipe is a claim about its run distribution, usually about its mean μ. The kit `labkit.py` keeps four sources of randomness apart: the initial weights (`init`), the order of the minibatches (`order`), the split into training and test sets (`split`), and the noise of label corruption, dropout and augmentation (`noise`). Nondeterministic kernels make even a run

with every seed fixed a fresh draw (Chapter 2), and the hyperparameter search has randomness of its own (Chapter 4). Which sources to randomize depends on the claim: a claim about a recipe on a fixed benchmark split randomizes initialization, order and noise, and a claim about a method on a task randomizes the split as well.

Summaries. From n runs with outcomes $Y_1, \dots, Y_n$, the sample mean $\bar{Y}$ estimates μ, the sample standard deviation S estimates the spread σ of single runs, and the *standard error* $\mathrm{SE} = S/\sqrt{n}$ the spread of $\bar{Y}$: S says how far the next run may land from the mean, SE how far the mean of these runs may be from μ. For normal runs $\bar{Y} \pm t_{n-1,0.975}\,\mathrm{SE}$ is a 95% confidence interval for μ (Theorem A.1), and a bootstrap over runs gives an approximate one without normality (Theorem A.3).[6] Plotted against the number j of runs so far, the *running mean* and its interval show the estimate settling, the interval narrowing like $1/\sqrt{j}$. The foot of every chapter opening in this book draws this plot for sixteen standard normal draws, showing as many draws as the chapter's number; this chapter's opening shows one run and no interval, which is all that one run supports.

[6] The t distribution was published by Gosset under the pen name Student (Student 1908), for the mean of a handful of measurements: the regime of a training experiment with five seeds.

Proposition 1.4 (One run of each). *Let $Y_1 \sim \mathcal{N}(\mu_1, \sigma_1^2)$ and $Y_2 \sim \mathcal{N}(\mu_2, \sigma_2^2)$ be the outcomes of one run of each of two recipes, independent, with larger outcomes better and $\Delta = \mu_1 - \mu_2 \geq 0$.*

(i) One run of each ranks the recipes in the wrong order with probability $\mathbb{P}(Y_1 < Y_2) = \Phi\big(-\Delta/\sqrt{\sigma_1^2 + \sigma_2^2}\big)$.

(ii) If the two recipes are the same, with $\sigma_1 = \sigma_2 = \sigma > 0$, then $\mathbb{P}(|Y_1 - Y_2| > \delta) = 2\Phi\big(-\delta/(\sigma\sqrt{2})\big)$ for every $\delta \geq 0$, and the median and the 95th percentile of $|Y_1 - Y_2|$ are $\sqrt{2}\, z_{0.75}\, \sigma \approx 0.954\,\sigma$ and $\sqrt{2}\, z_{0.975}\, \sigma \approx 2.77\,\sigma$.

Proof. The difference $Y_1 - Y_2$ is normal with mean Δ and variance $\sigma_1^2 + \sigma_2^2$, so $\mathbb{P}(Y_1 < Y_2) = \mathbb{P}(Y_1 - Y_2 < 0) = \Phi(-\Delta/\sqrt{\sigma_1^2 + \sigma_2^2})$. In part (ii) the variable $Z = (Y_1 - Y_2)/(\sigma\sqrt{2})$ is standard normal, so $\mathbb{P}(|Z| > x) = 2\Phi(-x)$ by symmetry, which gives the tail probability with $x = \delta/(\sigma\sqrt{2})$. The q-quantile of $|Z|$ is $z_{(1+q)/2}$, and the quantiles of $|Y_1 - Y_2|$ are $\sigma\sqrt{2}$ times those of $|Z|$. ∎

With $\delta = \sigma$ the tail probability is $2\Phi(-1/\sqrt{2}) \approx 0.48$: two runs of one recipe differ by more than a standard deviation about half the time. With $\Delta = \sigma$, one run of each ranks two recipes wrongly with probability $\Phi(-1/\sqrt{2}) \approx 0.24$. A comparison by single runs is informative only when the gap is several times σ. The outcome of a classifier is moreover a measurement on a finite test set, which adds a second source of noise.

Proposition 1.5 (Two sources of noise in a measured error). *Let a run R be drawn from the run distribution and test items $\mathbf{Z} = (Z_1, \dots, Z_m)$ be drawn from the data distribution, independently of each other and of R. Write $e_R(z) \in \{0, 1\}$ for the indicator that the network of run R errs on item*

z, $p(R) = \mathbb{E}[e_R(Z_1) \mid R]$ for its population error, $q(z) = \mathbb{E}\, e_R(z)$ for the probability that a fresh run errs on item z, and $\bar{p} = \mathbb{E}\, p(R) = \mathbb{E}\, q(Z_1)$. The measured error $\hat{p} = \frac{1}{m}\sum_{i=1}^{m} e_R(Z_i)$ satisfies the following.

(i) $\mathbb{E}\hat{p} = \bar{p}$ *and* $\operatorname{Var}\hat{p} = \operatorname{Var} p(R) + \mathbb{E}[p(R)(1-p(R))]/m$.

(ii) For a test set held fixed while the runs vary, the variance over runs has mean

$$\mathbb{E}\big[\operatorname{Var}(\hat{p} \mid \mathbf{Z})\big] = \frac{m-1}{m}\operatorname{Var} p(R) + \frac{1}{m}\mathbb{E}\big[q(Z_1)\big(1-q(Z_1)\big)\big]. \quad (1.1)$$

(iii) $\bar{p}(1-\bar{p}) = \mathbb{E}[q(Z_1)(1-q(Z_1))] + \operatorname{Var} q(Z_1)$*; in particular* $\mathbb{E}[q(Z_1)(1-q(Z_1))] \le \bar{p}(1-\bar{p})$*, with equality if and only if* $q(Z_1) = \bar{p}$ *almost surely.*

Proof. The plan is to condition on the run for part (i), to derive part (iii) by algebra, and to obtain part (ii) from both by conditioning on the test items.

Part (i). Given R, the indicators $e_R(Z_1), \ldots, e_R(Z_m)$ are independent Bernoulli variables with mean $p(R)$, because the items are independent of each other and of R. Hence $\mathbb{E}[\hat{p} \mid R] = p(R)$ and $\operatorname{Var}(\hat{p} \mid R) = p(R)(1-p(R))/m$. Taking expectations gives $\mathbb{E}\hat{p} = \bar{p}$, and the law of total variance gives the formula for $\operatorname{Var}\hat{p}$.

Part (iii). Since $q(1-q) = q - q^2$, we have $\mathbb{E}[q(Z_1)(1-q(Z_1))] = \bar{p} - \mathbb{E}\, q(Z_1)^2 = \bar{p} - \bar{p}^2 - \operatorname{Var} q(Z_1)$, which is the identity; the inequality and its case of equality follow because $\operatorname{Var} q(Z_1) \ge 0$, with equality exactly when $q(Z_1)$ is almost surely constant.

Part (ii). Given the items, $\hat{p}$ has mean $\frac{1}{m}\sum_i q(Z_i)$, an average of m independent copies of $q(Z_1)$, whose variance is $\operatorname{Var} q(Z_1)/m$. The law of total variance, now conditioning on the items, gives $\mathbb{E}[\operatorname{Var}(\hat{p} \mid \mathbf{Z})] = \operatorname{Var}\hat{p} - \operatorname{Var} q(Z_1)/m$. Inserting part (i), in which $\mathbb{E}[p(R)(1-p(R))] = \bar{p} - \mathbb{E}\, p(R)^2 = \bar{p}(1-\bar{p}) - \operatorname{Var} p(R)$, gives

$$\mathbb{E}\big[\operatorname{Var}(\hat{p} \mid \mathbf{Z})\big] = \operatorname{Var} p(R) + \frac{\bar{p}(1-\bar{p}) - \operatorname{Var} p(R) - \operatorname{Var} q(Z_1)}{m}.$$

By part (iii), $\bar{p}(1-\bar{p}) - \operatorname{Var} q(Z_1) = \mathbb{E}[q(Z_1)(1-q(Z_1))]$, and collecting the two terms in $\operatorname{Var} p(R)$ gives (1.1). ∎

Part (i) is the noise of the test set: even a fixed network measures with standard error $\sqrt{\bar{p}(1-\bar{p})/m}$, about 0.7 points for an error of 3% on 539 items. Part (ii) says what a spread over seeds on a fixed benchmark measures: the variance of the population error plus the *item churn* $\mathbb{E}[q(1-q)]/m$, the variance due to runs that err on different items. By part (iii) the churn lies between zero, when every run errs on the same items, and the full binomial variance, when every item is equally hard; the rest of the binomial variance, $\operatorname{Var} q(Z_1)/m$, is *item difficulty*, which moves all runs together when the test set changes. A seed standard deviation measured on a small test set can therefore be mostly a property of the test set, as Lab 1.3 finds.

1.5 The method: selection, components and the number of seeds

The best of k. A reported number is often the best of several runs: of a few restarts, of the seeds tried during development, of a few submissions. Its expectation is not the mean of the recipe.

Proposition 1.6 (The best of k runs). *Let $k \geq 1$, and let $e_k = \mathbb{E} \max_{i \leq k} Z_i$ for independent standard normals $Z_1, \dots, Z_k$.*

(i) If $Y_1, \dots, Y_k \overset{\text{iid}}{\sim} \mathcal{N}(\mu, \sigma^2)$, then $\mathbb{E} \max_i Y_i = \mu + \sigma e_k$ and $\mathbb{E} \min_i Y_i = \mu - \sigma e_k$. Moreover $e_1 = 0$, the sequence e_k increases strictly, and $e_k \leq \sqrt{2 \ln k}$.

(ii) If $(D_1, T_1), \dots, (D_k, T_k)$ are independent pairs, each bivariate normal with means μ_D, μ_T, standard deviations $\sigma_D, \sigma_T > 0$ and correlation ρ, and ι is the index of the largest D_i, then $\mathbb{E} T_\iota = \mu_T + \rho \, \sigma_T e_k$.

Proof. The plan is to reduce part (i) to standard normals and bound e_k through the moment generating function, and then to derive part (ii) from part (i) by conditioning on the D_i.

Part (i). Write $Y_i = \mu + \sigma Z_i$ with Z_i independent standard normals. Since $\sigma \geq 0$, $\max_i Y_i = \mu + \sigma \max_i Z_i$, which gives the first formula; and $\min_i Y_i = \mu - \sigma \max_i(-Z_i)$, where the $-Z_i$ are again independent standard normals, which gives the second. For $k = 1$, $e_1 = \mathbb{E} Z_1 = 0$. The maximum of $Z_1, \dots, Z_{k+1}$ is at least that of $Z_1, \dots, Z_k$, and strictly larger on the event that Z_{k+1} exceeds the others, which has probability $1/(k+1) > 0$; so $e_{k+1} > e_k$. For the bound, let $k \geq 2$ and $t > 0$. Jensen's inequality for the convex exponential, followed by bounding the largest of k positive numbers by their sum, gives

$$e^{t e_k} \leq \mathbb{E} \, e^{t \max_i Z_i} \leq \sum_{i=1}^{k} \mathbb{E} \, e^{t Z_i} = k \, e^{t^2/2}.$$

Taking logarithms, $e_k \leq (\ln k)/t + t/2$, and the choice $t = \sqrt{2 \ln k}$, which minimizes the right side, gives $e_k \leq \sqrt{2 \ln k}$. For $k = 1$ the bound reads $0 \leq 0$.

Part (ii). The pairs are independent, so given $D_1, \dots, D_k$ the variable T_i depends only on D_i, and the regression of a bivariate normal gives $\mathbb{E}[T_i \mid D_1, \dots, D_k] = \mu_T + \rho(\sigma_T/\sigma_D)(D_i - \mu_D)$. The index ι is a function of the D_i, unique with probability one, so $\mathbb{E}[T_\iota \mid D_1, \dots, D_k] = \mu_T + \rho(\sigma_T/\sigma_D)(\max_i D_i - \mu_D)$. Taking expectations and applying part (i) to the D_i gives $\mathbb{E} T_\iota = \mu_T + \rho(\sigma_T/\sigma_D)\sigma_D e_k = \mu_T + \rho \sigma_T e_k$. ∎

Part (ii) returns in Chapter 5, where the development score becomes the cheap proxy that guides a search, and the conditioning argument is extended to every selection rule that looks only at proxies (Lemma 5.5).

Table 1.1 lists e_k, and Exercise 1.1 derives $e_2 = 1/\sqrt{\pi}$ and $e_3 = 3/(2\sqrt{\pi})$. The constant grows slowly, like $\sqrt{2 \ln k}$ (Exercise 1.2), but it grows: the best

k	2	3	4	5	6	7	8	9	10
e_k	0.564	0.846	1.029	1.163	1.267	1.352	1.424	1.485	1.539
$\sqrt{2\ln k}$	1.177	1.482	1.665	1.794	1.893	1.973	2.039	2.096	2.146

Table 1.1. The expected maximum e_k of k independent standard normals (numerical integration in `01-seeds-bestof.py`) and the bound of Proposition 1.6; further, $e_{20} = 1.867$, $e_{50} = 2.249$, $e_{100} = 2.508$, $e_{10^4} = 3.852$.

of 5 runs sits 1.16σ above the mean, the best of 625 runs 3.10σ. Part (i) is the bonus of reporting the best run on the items that score it, and the bonus is luck. Part (ii) is the honest procedure, choosing on development data and scoring once on test data: if the two scores share a real component, then the selected run is truly better, by $\rho\sigma_T e_k$, the fraction ρ of the apparent gain; Reimers and Gurevych's correlation of 0.23 between development and test scores says that most of what their best development run gained was luck. The same order statistic governs the best of k hyperparameter trials (Chapter 4) and selection by a cheap proxy (Chapter 5); its asymptotics are in Proposition A.9.

Failed runs. The normal model breaks when some runs fail. If a run fails with probability π, landing near the score of a trivial predictor, then the mean mixes failures with working runs, the standard deviation is inflated by the gap between them, and the median remains the score of a working run as long as $\pi < 1/2$. The best of k runs fails only with probability π^k, which is why restarts worked for BERT, and why a best-of-k number hides the failure rate, as the numbers for RTE in Section 1.3 show. Report the failure rate with an exact binomial interval (Proposition A.17), summarize the working runs separately, and show every run.

Variance components. Which seeds produce the spread? A *crossed design* answers this: draw I initialization seeds and J data-order seeds, and train one run for every combination. Write Y_{ij} for the outcome of the run with the ith initialization and the jth order seed, $\bar{Y}_{i\cdot} = \frac{1}{J}\sum_j Y_{ij}$ and $\bar{Y}_{\cdot j} = \frac{1}{I}\sum_i Y_{ij}$ for the row and column means, and $\bar{Y}_{\cdot\cdot}$ for the grand mean. The mean squares of the table are

$$\begin{aligned} \mathrm{MS}_A &= \frac{J}{I-1}\sum_{i=1}^{I}(\bar{Y}_{i\cdot} - \bar{Y}_{\cdot\cdot})^2, \qquad \mathrm{MS}_B = \frac{I}{J-1}\sum_{j=1}^{J}(\bar{Y}_{\cdot j} - \bar{Y}_{\cdot\cdot})^2, \\ \mathrm{MS}_E &= \frac{1}{(I-1)(J-1)}\sum_{i=1}^{I}\sum_{j=1}^{J}(Y_{ij} - \bar{Y}_{i\cdot} - \bar{Y}_{\cdot j} + \bar{Y}_{\cdot\cdot})^2. \end{aligned} \tag{1.2}$$

The textbook model behind them, $Y_{ij} = \mu + a_i + b_j + e_{ij}$ with independent normal effects, is an assumption. For seeds it is not needed: a deterministic training pipeline is a function of its seeds, and every such function has an exact decomposition of the same form.

Lemma 1.7 (Centred sums of squares). *If $X_1, \dots, X_n$, with $n \geq 2$, are pairwise uncorrelated with a common mean and a common variance τ^2, then $\mathbb{E}\sum_{i=1}^{n}(X_i - \bar{X})^2 = (n-1)\tau^2$.*

Proof. Let ν be the common mean. Expanding around ν gives $\sum_i (X_i - \bar{X})^2 = \sum_i (X_i - \nu)^2 - n(\bar{X} - \nu)^2$. The first sum has mean $n\tau^2$, and $\bar{X} - \nu$ has variance τ^2/n because the X_i are pairwise uncorrelated, so the expectation is $n\tau^2 - \tau^2$. ∎

Theorem 1.8 (Variance components of a crossed design). *Let $U_1, \dots, U_I$ and $V_1, \dots, V_J$, with $I, J \geq 2$, be independent random seeds, the U_i identically distributed and the V_j identically distributed, and let $Y_{ij} = f(U_i, V_j)$ for a function f with $\mathbb{E} f(U_1, V_1)^2 < \infty$. Let $\mu = \mathbb{E}Y_{11}$, and define the main effects $a(u) = \mathbb{E} f(u, V_1) - \mu$ and $b(v) = \mathbb{E} f(U_1, v) - \mu$, the interaction $c(u, v) = f(u, v) - \mu - a(u) - b(v)$, and*

$$\sigma_a^2 = \operatorname{Var} a(U_1), \qquad \sigma_b^2 = \operatorname{Var} b(V_1), \qquad \sigma_e^2 = \operatorname{Var} c(U_1, V_1).$$

Then $\operatorname{Var} Y_{11} = \sigma_a^2 + \sigma_b^2 + \sigma_e^2$, and the mean squares (1.2) satisfy

$$\mathbb{E}\mathrm{MS}_A = \sigma_e^2 + J\sigma_a^2, \qquad \mathbb{E}\mathrm{MS}_B = \sigma_e^2 + I\sigma_b^2, \qquad \mathbb{E}\mathrm{MS}_E = \sigma_e^2.$$

Consequently $\hat{\sigma}_a^2 = (\mathrm{MS}_A - \mathrm{MS}_E)/J$, $\hat{\sigma}_b^2 = (\mathrm{MS}_B - \mathrm{MS}_E)/I$ and $\hat{\sigma}_e^2 = \mathrm{MS}_E$ are unbiased estimators of σ_a^2, σ_b^2 and σ_e^2.

Proof. The plan is to show that the pieces $a(U_i)$, $b(V_j)$ and $c(U_i, V_j)$ are pairwise uncorrelated, and then to compute each mean square from them with Lemma 1.7. Write $a_i = a(U_i)$, $b_j = b(V_j)$ and $c_{ij} = c(U_i, V_j)$, and use dots for averages over an index, as in $\bar{c}_{i\cdot} = \frac{1}{J}\sum_j c_{ij}$.

The pieces are orthogonal. Because the seeds are independent, fixing $U_i = u$ leaves the law of the other seeds unchanged. Averaging c over either seed gives zero: for each u, $\mathbb{E}\, c(u, V_1) = \mathbb{E} f(u, V_1) - \mu - a(u) - \mathbb{E}\, b(V_1) = 0$, since $\mathbb{E}\, b(V_1) = \mathbb{E} f(U_1, V_1) - \mu = 0$; symmetrically $\mathbb{E}\, c(U_1, v) = 0$ for each v. Hence a_i, b_j and c_{ij} have mean zero, and they are square integrable because conditional expectations do not increase second moments. Two of them that involve disjoint sets of seeds are independent, hence uncorrelated. Otherwise the two share exactly one seed, say U_i (a shared V_j is symmetric), and at least one of them is a c. Given $U_i = u$ they are independent, and the c among them, $c(u, V_j)$, has mean zero; so the conditional mean of their product is zero, and so is its expectation. In particular $\operatorname{Var} Y_{11} = \operatorname{Var}(a_1 + b_1 + c_{11}) = \sigma_a^2 + \sigma_b^2 + \sigma_e^2$.

Rows and columns. The row mean is $\bar{Y}_{i\cdot} = \mu + a_i + \bar{b}_\cdot + \bar{c}_{i\cdot}$, and $\bar{b}_\cdot$ is the same for every row; so $\bar{Y}_{i\cdot} - \bar{Y}_{\cdot\cdot} = X_i - \bar{X}$ with $X_i = a_i + \bar{c}_{i\cdot}$. By orthogonality the X_i are pairwise uncorrelated, with mean zero and variance $\sigma_a^2 + \sigma_e^2/J$, so Lemma 1.7 gives $\mathbb{E}\sum_i (\bar{Y}_{i\cdot} - \bar{Y}_{\cdot\cdot})^2 = (I-1)(\sigma_a^2 + \sigma_e^2/J)$, and multiplying by $J/(I-1)$ gives $\mathbb{E}\mathrm{MS}_A = J\sigma_a^2 + \sigma_e^2$. Exchanging the roles of rows and columns gives $\mathbb{E}\mathrm{MS}_B = I\sigma_b^2 + \sigma_e^2$.

Residuals. Substituting the decomposition into the residual, the terms in μ, a and b cancel:

$$r_{ij} = Y_{ij} - \bar{Y}_{i\cdot} - \bar{Y}_{\cdot j} + \bar{Y}_{\cdot\cdot} = c_{ij} - \bar{c}_{i\cdot} - \bar{c}_{\cdot j} + \bar{c}_{\cdot\cdot}.$$

The table $c_{ij} - \bar{c}_{\cdot\cdot}$ is the sum of three tables, with entries $\bar{c}_{i\cdot} - \bar{c}_{\cdot\cdot}$, $\bar{c}_{\cdot j} - \bar{c}_{\cdot\cdot}$ and r_{ij}, which are orthogonal in $\mathbb{R}^{I\times J}$: the residuals sum to zero along every row and every column, the first table depends only on the row and the second only on the column, and each of these two sums to zero. By Pythagoras,

$$\sum_{i,j}(c_{ij} - \bar{c}_{\cdot\cdot})^2 = J\sum_{i}(\bar{c}_{i\cdot} - \bar{c}_{\cdot\cdot})^2 + I\sum_{j}(\bar{c}_{\cdot j} - \bar{c}_{\cdot\cdot})^2 + \sum_{i,j} r_{ij}^2.$$

We apply Lemma 1.7 three times: to the IJ variables c_{ij}, of variance σ_e^2; to the I row means $\bar{c}_{i\cdot}$, of variance σ_e^2/J; and to the J column means, of variance σ_e^2/I, all pairwise uncorrelated by orthogonality. Taking expectations,

$$\mathbb{E}\sum_{i,j} r_{ij}^2 = (IJ-1)\sigma_e^2 - (I-1)\sigma_e^2 - (J-1)\sigma_e^2 = (I-1)(J-1)\sigma_e^2,$$

which is $\mathbb{E}\mathrm{MS}_E = \sigma_e^2$. The three estimators are linear in the mean squares, so their unbiasedness follows. ∎

Three remarks make the theorem usable. *Nothing about training is assumed* beyond independent seeds; independence and normality of the effects are needed only for F-tests and exact intervals (Theorem A.12). *One run per cell cannot separate* the interaction from noise that is fresh in every run, such as that of nondeterministic kernels (Chapter 2); separating them takes two runs per cell. *Estimates can be negative*, when $\mathrm{MS}_A < \mathrm{MS}_E$; report them as computed, since rounding them up to zero biases them. Fixing a seed leaves a conditional spread: runs that share the initialization seed spread with standard deviation $\sqrt{\sigma_b^2 + \sigma_e^2}$, runs that share the order seed with $\sqrt{\sigma_a^2 + \sigma_e^2}$.[7] With splits as rows and runs within a split as replicates, the same analysis gives the split component.

[7] Dodge et al. summarized their crossed design by such conditional spreads, averaged over seeds: their "standard deviation aggregated over weight initializations" estimates roughly $\sqrt{\sigma_b^2 + \sigma_e^2}$.

How many seeds. The last question is the one an experimenter faces first: how many runs does a comparison need?

Proposition 1.9 (Power of a comparison of means). *Let each of two recipes be run n times, all runs independent, with outcomes normal with means μ_1, μ_2 and a common known standard deviation $\sigma > 0$, and let $\Delta = \mu_1 - \mu_2$. The two-sided test that rejects $\Delta = 0$ when $|\bar{Y}_1 - \bar{Y}_2| > z_{1-\alpha/2}\,\sigma\sqrt{2/n}$ has level α and power*

$$\Phi\Big(\frac{\sqrt{n}\,|\Delta|}{\sigma\sqrt{2}} - z_{1-\alpha/2}\Big) + \Phi\Big(-\frac{\sqrt{n}\,|\Delta|}{\sigma\sqrt{2}} - z_{1-\alpha/2}\Big).$$

If instead the runs are paired, the ith runs of the two recipes sharing their seeds and split, with the pairs independent and bivariate normal and $\sigma_D^2 = \mathrm{Var}(Y_{1i} - Y_{2i}) > 0$, then the same holds with $2\sigma^2$ replaced by σ_D^2, for the test that rejects when $|\bar{Y}_1 - \bar{Y}_2| > z_{1-\alpha/2}\,\sigma_D/\sqrt{n}$.

Proof. Write $s = \sigma\sqrt{2/n}$. The difference $\bar{Y}_1 - \bar{Y}_2$ is normal with mean Δ and standard deviation s, so the rejection probability is

$$\mathbb{P}(\bar{Y}_1 - \bar{Y}_2 > z_{1-\alpha/2}s) + \mathbb{P}(\bar{Y}_1 - \bar{Y}_2 < -z_{1-\alpha/2}s) = \Phi\Big(\frac{\Delta}{s} - z_{1-\alpha/2}\Big) + \Phi\Big(-\frac{\Delta}{s} - z_{1-\alpha/2}\Big),$$

which is symmetric in the sign of Δ and equals the displayed power. At $\Delta = 0$ it is $2\Phi(-z_{1-\alpha/2}) = \alpha$. In the paired case $\bar{Y}_1 - \bar{Y}_2$ is the mean of n independent normal differences of variance σ_D^2, and the same computation applies with $s = \sigma_D/\sqrt{n}$. ∎

Corollary 1.10 (Number of seeds). *In the setting of Proposition 1.9, the power is at least* 0.8 *whenever*

$$n \geq \frac{2\sigma^2\,(z_{1-\alpha/2} + z_{0.8})^2}{\Delta^2},$$

which for $\alpha = 0.05$ *is* $n \geq 15.7\,\sigma^2/\Delta^2$ *runs per arm. For paired runs,* $2\sigma^2$ *is replaced by* σ_D^2.

Proof. The second term of the power is positive, and the first is at least 0.8 exactly when $\sqrt{n}|\Delta|/(\sigma\sqrt{2}) - z_{1-\alpha/2} \geq z_{0.8}$; squaring and rearranging gives the condition on n. With $z_{0.975} = 1.960$ and $z_{0.8} = 0.842$, the constant $2(z_{0.975} + z_{0.8})^2$ is 15.70. ∎

To detect a gain of one seed standard deviation takes 16 runs per arm, of half a standard deviation 63. A gain of 0.3 points at a standard deviation of 0.2 takes 7 runs per arm by the corollary and 9 by an exact t-test (Exercise 1.3, Proposition A.14); the taggers of Section 1.3, with a gap of 0.27 and a standard deviation of 0.24, needed 13 each. *Pairing* helps as much as the arms share their noise, since $\sigma_D^2 = \sigma_1^2 + \sigma_2^2 - 2\,\mathrm{Cov}(Y_1, Y_2)$: a shared split drops out of the difference, and so does the difficulty of shared test items, since only items on which two networks disagree contribute to it (Proposition A.5).

1.6 In the laboratory: one recipe, eighteen hundred runs

All three labs use one recipe: a two-layer perceptron of width 64 (8,970 parameters) on the scikit-learn digits, trained with SGD and momentum 0.9 at learning rate 0.05, batch size 64, for 20 epochs. A split puts 1,258 images in the training set and 539 in the test set, so that one misclassified digit is 0.19 points of test error.

lab 1.1 · Two hundred runs of one recipe

Setup. Two hundred runs; run r uses seed r for initialization, data order and split. About 15 seconds.

script
`01-seeds-spread.py`

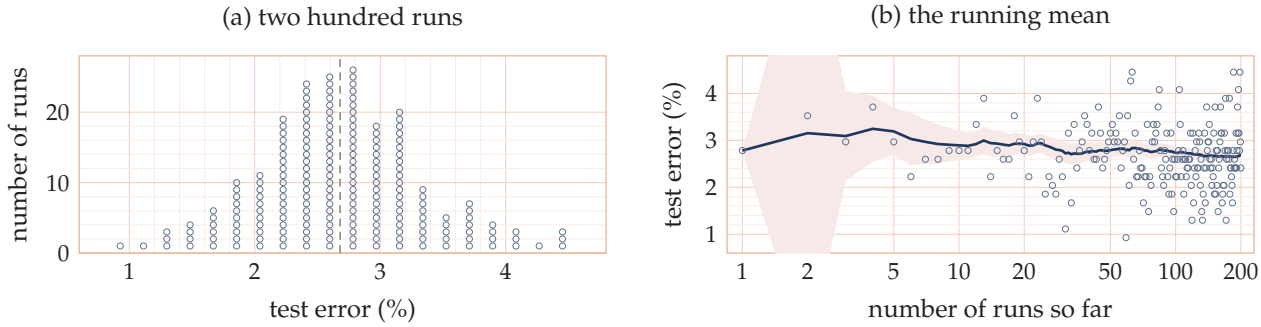


Figure 1.1. Lab 1.1: 200 runs of one recipe (two-layer MLP on the digits, 539 test images), each with new seeds for initialization, data order and split (seeds 0–199). (a) Each hollow mark is one run, runs with equal error stacked; the dashed line is the mean, 2.68%. (b) The runs in the order they were made (hollow marks), the running mean (line) and its 95% t-interval (band); the band for two runs, $[-1.6, 7.9]$, is cut off by the frame.

Predict first. How many test items will separate the best and the worst run? After how many runs will the 95% interval be narrower than one test item?

Look for. The shape of the distribution and its extremes; the running mean inside an interval that narrows like $1/\sqrt{j}$; the fraction of pairs of runs more than one standard deviation apart (0.48 by Proposition 1.4).

What happened. The 200 runs (Figure 1.1a) have mean test error 2.68 ± 0.09 (95% t-interval), standard deviation 0.65 points (95% interval 0.60–0.73) and range 0.93% to 4.45%: nineteen test items separate the best and the worst run of one recipe. These are ordinary extremes; for 200 normal draws, Proposition 1.6 with $e_{200} = 2.75$ predicts 0.88% and 4.48%. Of 100 disjoint pairs of runs, 50 differ by more than one standard deviation (exact 95% interval 40–60) against 48 predicted by Proposition 1.4, with median difference 0.65 points against 0.62: two copies of this recipe, one run each, would show a gap of more than half a point about half the time.

Panel (b) is the ornament of the chapter openings with real data: after two runs the 95% interval is $[-1.6, 7.9]$; after sixteen, all the draws of the ornament, 2.91 ± 0.26; after 43 runs its half-width drops below one test item; after 200, 2.68 ± 0.09, narrower than at sixteen by a factor of 2.8 rather than $\sqrt{200/16} = 3.5$, because the first sixteen runs happened to spread less (standard deviation 0.48). By Proposition 1.5(i), a fixed network scored on a random set of 539 items measures with standard deviation 0.70 points at this error, or 0.58 for test sets drawn without replacement from the 1,797 digits: most of the spread of these runs belongs to the test set.

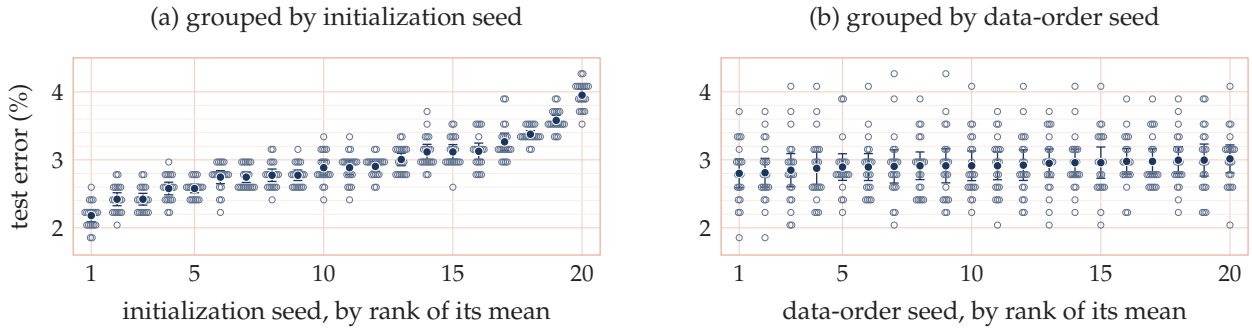


Figure 1.2. Lab 1.2, crossed design: 400 runs on split 0, every combination of 20 initialization seeds and 20 data-order seeds. The same runs (hollow marks; equal errors side by side) grouped (a) by initialization seed and (b) by data-order seed, each group sorted by its mean; filled marks are group means with 95% t-intervals over the 20 runs of the group.

lab 1.2 · Where the spread comes from

script
`01-seeds-components.py`

Setup. Crossed design: split 0, initialization seeds 0–19 crossed with data-order seeds 0–19 (400 runs). Nested design: 50 further splits, 8 runs on each with new initialization and order seeds (400 runs). Estimators of Theorem 1.8 and their one-way analogue for the split; intervals from a parametric bootstrap of the fitted normal random-effects model. About a minute.

Predict first. Which moves the test error more in this network, the initial weights or the order of the minibatches? Will a new split move it more or less than new seeds?

Look for. Whether the means of the initialization seeds spread out more than those of the order seeds; the interaction; the split component against the binomial noise of 539 items and its part due to item difficulty.

What happened. The initialization seed carries 81% of the seed variance on this split: $\hat{\sigma}_a = 0.42$ points (95% interval 0.29–0.55), against $\hat{\sigma}_b = 0.04$ (0.00–0.06) for the data order and $\hat{\sigma}_e = 0.20$ (0.18–0.21) for their interaction. The means of the initialization seeds range from 2.18% to 3.95%, those of the order seeds only from 2.80% to 3.02% (Figure 1.2), and the F-test finds the first effect beyond doubt ($p \approx 10^{-125}$) and the second barely ($p = 0.02$). The order matters only in combination with an initialization: runs that share the initialization seed spread with standard deviation 0.20, runs that share the order seed with 0.46. This differs from BERT fine-tuning, where Dodge et al. (2020) found the two seeds comparable, but there the initialization seed set only the 2,048 weights of a new classifier layer, and here it sets all 8,970; the ordering is also consistent with the finding of Frankle et al. (2020) that small networks on MNIST are stable to the noise of SGD from initialization on, an explanation we have not tested.[8]

Is the initialization effect a property of the network or of these 539 items? On two halves of the test items, the means of the 20 initialization seeds

[8] Stability to SGD noise means that two runs from the same weights with different data orders end in the same linearly connected region of weights; Chapter 2 takes it up.

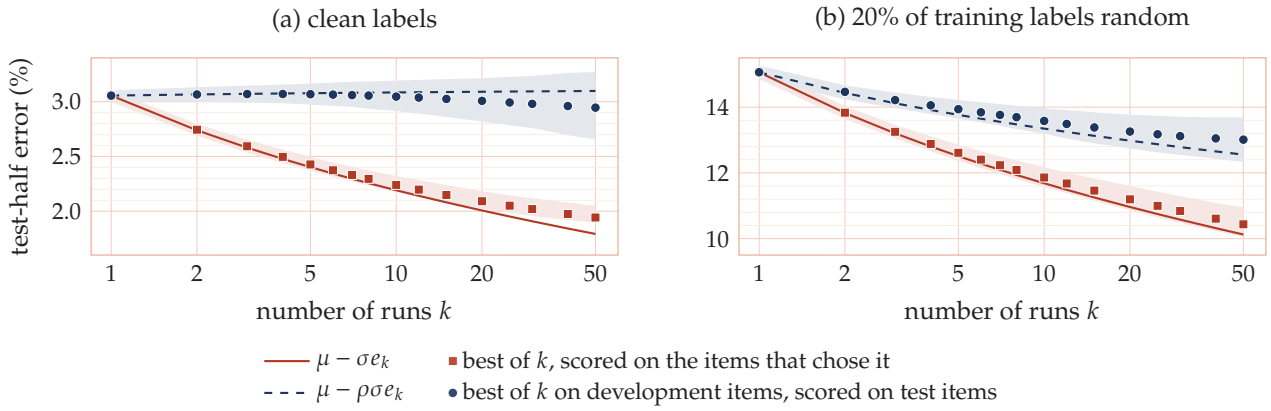


Figure 1.3. Lab 1.3: the best of k runs, 400 runs per panel (split 0, seeds 0–399), errors on a test half of 269 images. Squares: expected lowest test-half error among k runs; circles: expected test-half error of the run with the lowest error on the other 270 images; both exact averages over all k-subsets, with 95% bootstrap bands over runs. Lines: the predictions of Proposition 1.6 from the sample mean μ, standard deviation σ and development–test correlation ρ of each panel.

correlate at only 0.15 (95% interval −0.32 to 0.55); Lab 1.3 settles the question. A new split moves the error about as much as new seeds do: 0.44 points (0.35–0.53) between splits against 0.38 (0.35–0.41) within them. The binomial noise of 539 items is 0.73 points at this error (0.61 for the finite pool), and by Proposition 1.5(iii) its part that a new split moves and new seeds do not, item difficulty, is 0.52 points after the finite-pool correction, inside the interval for the split: a new split acts almost entirely through which digits land in the test set, as Bouthillier et al. (2021) found for data sampling at larger scale.

lab 1.3 · The best of k

script
`01-seeds-bestof.py`

Setup. Split 0, 400 runs with new initialization and order seeds, once on clean labels and once with 20% of the training labels replaced by random labels. The 539 test images are divided at random into a development half (270) and a test half (269). For each $k \le 50$, exact averages over all k-subsets of the 400 runs of the best test-half error, and of the test-half error of the run with the best development-half error (ties broken at random). About 70 seconds.

Predict first. On clean labels, how much lower is the best of 10 runs than a typical run? How much of that survives when the best run is chosen on the development half and scored on the test half? And with noisy labels?

Look for. The best-of-k curve against $\mu - \sigma e_k$, and where it departs; the honest curve against $\mu - \rho\sigma e_k$; the correlation ρ between development and test errors in the two settings.

What happened. On clean labels (Figure 1.3a) the test-half error has mean 3.06 ± 0.06 and standard deviation 0.56 points. The best of k runs scored on the test half follows $\mu - \sigma e_k$: for $k = 10$ it is 2.24% (95% band 2.18–2.32) against 2.19% predicted, 0.82 points below a typical run; at $k = 50$ it falls short of the prediction (1.94% against 1.79%), because the errors are skewed to the right (skewness 0.23) and cannot fall below the few digits every run gets wrong. Chosen on the development half, the best of 10 runs scores 3.05% (2.92–3.19) on the test half: nothing of the 0.82 points survives, because a run's errors on the two halves are uncorrelated ($\rho = -0.03$, 95% interval −0.13 to 0.07). Item churn makes up 91% of the seed variance on all 539 items (80–104%): on clean digits the seed decides which borderline digits a network gets wrong, not how many it would get wrong on new digits, and the initialization effect of Lab 1.2 is an effect on which items fail.

With 20% of the training labels random (Figure 1.3b) the runs differ for real: mean 15.06 ± 0.22, standard deviation 2.20 points, $\rho = 0.51$ (0.43–0.58), and item churn only 37% of the seed variance (32–43%). The best of 10 runs scores 11.86% on the items that chose it (11.69% predicted) and 13.58% (13.19–13.97) when chosen on the development half (13.35% predicted by $\mu - \rho\sigma e_k$). At every k the honest gain is 44–48% of the apparent gain, close to ρ. Restarts selected on held-out data buy a real improvement when runs differ in what they learned, as they did for BERT; scored on the items that selected them, they report that improvement plus a bonus of luck.

The miniature reproduced the post-mortems, and differed from them in one respect: here the initialization seed outweighs the data-order seed.

1.7 Protocol

protocol 1.1 · Measuring one recipe

- ☐ Decide which sources of randomness the claim must hold across (initialization, data order, augmentation and dropout, split, hardware); randomize those, fix the rest, and record every seed.
- ☐ Fix the number of runs before looking, from a pilot standard deviation and the smallest difference worth detecting (Corollary 1.10); never fewer than five for a description.
- ☐ Report the mean with a 95% interval, the standard deviation, the number of runs and every run, in a dot plot or a table; report failed runs and the rule that defines them separately.
- ☐ Never report the best of k runs without k and the mean; when a run must be selected, select it on development data and score it once on test data.
- ☐ Compare recipes on the same splits, seeds and test items, and test the difference across runs, not only across test items.
- ☐ Compute the binomial noise of the test set and, from per-item predictions, its churn; if they are comparable to the effect under

study, then the test set is too small for the claim.

- ☐ When the spread matters, estimate where it comes from with a crossed design before choosing which seeds to vary.

The protocol makes the measurement of one recipe honest; it does not make a comparison fair. Two well-measured recipes can still differ in their tuning budgets (Chapter 4) or in more than the component a claim is about (Chapter 6), be compared on many tasks at once (Chapter 3), or behave differently on other hardware (Chapter 2). And at 16 runs per arm to detect a gain of one standard deviation, the protocol is out of reach for the largest models. Then measure the spread where runs are affordable (a smaller model, fewer steps, a subset of the data) and say so, use paired designs, and when only one run is possible, report it as one run beside the seed standard deviation of a comparable setting, with a provisional conclusion.

Exercises

1.1 CONSOLIDATION. (Expected maxima.) (a) Show that $e_2 = 1/\sqrt{\pi}$, using $\max(a,b) = \frac{1}{2}(a+b) + \frac{1}{2}|a-b|$, and that $e_3 = 3/(2\sqrt{\pi})$. (b) Show that $e_k = \int z k \varphi(z) \Phi(z)^{k-1}\,\mathrm{d}z$, and check Table 1.1 numerically. (c) Show that the expected range of k independent $\mathcal{N}(\mu,\sigma^2)$ draws is $2\sigma e_k$, and check the CIFAR-10 range of Picard (2021). (d) For runs uniform on $[\mu-h, \mu+h]$, show that the best of k has mean $\mu + h(k-1)/(k+1)$, and compare its growth with that of e_k; which run distributions make best-of-k reports most misleading? *(Hint: for e_3, integrate $3\int z\varphi\Phi^2\,\mathrm{d}z$ by parts, then use $\Phi(z)+\Phi(-z)=1$ and $\int \varphi^2 = 1/(2\sqrt{\pi})$.)*

1.2 CONSOLIDATION. ★ (The growth of e_k.) Prove that $e_k/\sqrt{2\ln k} \to 1$. *(Hint: for the lower bound, with $M = \max_i Z_i$ and $t = (1-\varepsilon)\sqrt{2\ln k}$, show $e_k \ge t\,\mathbb{P}(M \ge t) - \mathbb{E}|Z_1|$, and use $1-\Phi(t) \ge \varphi(t)\,t/(1+t^2)$ to show $\Phi(t)^k \to 0$.)*

1.3 CONSOLIDATION. (How many seeds.) A method is expected to gain 0.3 points over a baseline whose seed standard deviation is 0.2 points. (a) By Corollary 1.10, how many runs per arm give power 0.8 at level 0.05? (b) With the noncentral t distribution, show that a two-sample t-test needs 9 runs per arm. (c) Suppose the runs are paired by seed and split, and paired outcomes have correlation 0.6. How many pairs suffice by the corollary?

1.4 LAB. (A smaller test set.) Modify `01-seeds-spread.py` to score every run on only the first 100 test images of its split. Predict the new standard deviation from Proposition 1.5(i) and Lab 1.1 before running it, then compare.

1.5 LAB. (When does the order matter?) Rerun the crossed design of `01-seeds-components.py` with learning rate 0.2, then with batch size 16, then with Adam at learning rate 10^{-3}. Observe how $\hat{\sigma}_b$ and $\hat{\sigma}_e$ change

relative to $\hat{\sigma}_a$, and relate the change to the stability of training to the noise of SGD (Chapter 2).

1.6 LAB. (The size of the development set.) In `01-seeds-bestof.py` with noisy labels, use development sets of $d = 50$, 270 and 489 of the 539 test images and test sets of the rest. Predict how ρ changes with d, check it, and observe the honest best-of-10 error. When is selecting the best of k runs on development data worth its cost?

1.7 DESIGN. (Components of a fine-tuning pipeline.) A team fine-tunes a pretrained language model and can vary the pretraining checkpoint (five are available, differing only in their pretraining seed), the initialization of the new head and the data order, with a budget of 300 runs. Design a study of the three main components: which interactions are estimable and which are confounded with what, how the runs are allocated, and how precisely five checkpoints determine their component.

1.8 DESIGN. (Paired or unpaired.) Two recipes are to be compared with 60 runs in all. A pilot gives a seed standard deviation of 0.5 points for each and a correlation of 0.7 between runs of the two recipes that share split, seeds and test items. Compare the power of paired and unpaired designs to detect a gain of 0.2 points, and list what the code must do for the pairing to be real.

1.9 CRITIQUE. (Best of five against single runs.) A paper's main table lists its method as "best of 5 seeds" next to baseline scores copied from the original papers, each a single run. The method leads by 0.4 points, and its own runs have a standard deviation of 0.3. What does the lead measure? Estimate the part due to selection with Proposition 1.6, and describe the table a reader needs.

1.10 CRITIQUE. (The best and the worst seed.) A study crosses 25 initialization seeds with 25 data orders, picks the initialization seeds with the highest and the lowest mean, and reports a two-sample t-test between their 25 runs each, $p < 0.01$, as evidence that some initializations are better than others. Suppose that all runs are independent with standard deviation σ, as when neither seed has an effect. Show that the expected gap between the best and the worst seed mean is $2e_{25}\sigma/5 \approx 0.79\sigma$ ($e_{25} = 1.965$), while the standard error of the test is $\sigma\sqrt{2/25} \approx 0.28\sigma$. Which test answers the question?

1.11 RESEARCH. (Intervals for bimodal run distributions.) When some runs fail, the mean is a poor summary and the t-interval loses its coverage. Open: a procedure that reports the failure rate and the mean of working runs jointly, with a failure rule that does not look at test data and with guaranteed coverage when failures are rare, at rates near $1/n$. Also open: a theory that predicts which seed dominates the spread (the initialization in Lab 1.2, neither for BERT fine-tuning)

from the width, the learning rate and the fraction of weights a seed initializes.

Notes and References

The problem of this chapter is older than deep learning. Student (1908) derived the t distribution for the mean of a few measurements, and Tippett (1925) studied the extremes and the range of normal samples, whose mean is $2e_k$ in our notation. Theorem 1.8 is the classical result on expected mean squares in the balanced two-way layout (Searle et al. 1992), stated here for a function of independent seeds.

For deep networks, Reimers and Gurevych (2017) put the seed on the agenda of natural language processing, and Card et al. (2020) showed that many comparisons in the field are underpowered; in reinforcement learning, Henderson et al. (2018) (**??**) and Colas et al. (2018) raised the same questions, the latter with a power analysis for the number of seeds. Bouthillier et al. (2019) distinguished reproducing a result from reproducing a finding, and Gundersen et al. (2022) review the sources of irreproducibility in machine learning. Seeds change more than scores: they change the explanations of predictions (Madhyastha and Jain 2019) and generalization outside the training distribution (McCoy et al. 2020). That a best-of-k number depends on k is the starting point of Dodge et al. (2019) and of Chapter 4.

Translating the literature. Reimers and Gurevych (2017) report F1 as a fraction, so their $\sigma = 0.00241$ is our 0.24 points; their score distribution is our run distribution, and the "Evaluation 1" and "Evaluation 2" of their 2018 preprint are our one run per arm and best of k on development data. Dodge et al. (2020) call a run a trial, while in this book a trial is a hyperparameter configuration; their WI and DO seeds are our initialization and order seeds, their expected validation performance of the best of x trials is our $\mathbb{E}[M_x]$ on the validation set, and their standard deviation aggregated over WI seeds is the conditional spread $\sqrt{\sigma_b^2 + \sigma_e^2}$ of Theorem 1.8, averaged over seeds. What Picard (2021) calls the seed is our full set of seeds. In Bouthillier et al. (2021), data sampling is our split component, ξ_O collects our initialization, order and noise seeds, and numerical noise is the nondeterminism of Chapter 2.

References

Bouthillier, X., C. Laurent, and P. Vincent (2019). "Unreproducible Research is Reproducible". In: *Proceedings of the 36th International Conference on Machine Learning*. Vol. 97. Proceedings of Machine Learning Research, pp. 725–734.

Bouthillier, X. and G. Varoquaux (2020). *Survey of Machine-Learning Experimental Methods at NeurIPS2019 and ICLR2020*. Research Report hal-02447823. Inria Saclay Ile de France.

Bouthillier, X. et al. (2021). "Accounting for Variance in Machine Learning Benchmarks". In: *Proceedings of Machine Learning and Systems*. Vol. 3.

Card, D., P. Henderson, U. Khandelwal, R. Jia, K. Mahowald, and D. Jurafsky (2020). "With Little Power Comes Great Responsibility". In: *Proceedings of EMNLP 2020*, pp. 9263–9274.

Chiu, J. P. C. and E. Nichols (2016). "Named Entity Recognition with Bidirectional LSTM-CNNs". In: *Transactions of the Association for Computational Linguistics* 4, pp. 357–370.

Colas, C., O. Sigaud, and P.-Y. Oudeyer (2018). *How Many Random Seeds? Statistical Power Analysis in Deep Reinforcement Learning Experiments*. Preprint, arXiv:1806.08295.

Devlin, J., M.-W. Chang, K. Lee, and K. Toutanova (2019). "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding". In: *Proceedings of NAACL-HLT 2019*, pp. 4171–4186.

Dodge, J., S. Gururangan, D. Card, R. Schwartz, and N. A. Smith (2019). "Show Your Work: Improved Reporting of Experimental Results". In: *Proceedings of EMNLP-IJCNLP 2019*, pp. 2185–2194.

Dodge, J., G. Ilharco, R. Schwartz, A. Farhadi, H. Hajishirzi, and N. Smith (2020). *Fine-Tuning Pretrained Language Models: Weight Initializations, Data Orders, and Early Stopping*. Preprint, arXiv:2002.06305.

Frankle, J., G. K. Dziugaite, D. Roy, and M. Carbin (2020). "Linear Mode Connectivity and the Lottery Ticket Hypothesis". In: *Proceedings of the 37th International Conference on Machine Learning*. Vol. 119. Proceedings of Machine Learning Research, pp. 3259–3269.

Gundersen, O. E., K. Coakley, C. Kirkpatrick, and Y. Gil (2022). *Sources of Irreproducibility in Machine Learning: A Review*. Preprint, arXiv:2204.07610.

Henderson, P., R. Islam, P. Bachman, J. Pineau, D. Precup, and D. Meger (2018). "Deep Reinforcement Learning That Matters". In: *Proceedings of the AAAI Conference on Artificial Intelligence*. Vol. 32. 1.

Lample, G., M. Ballesteros, S. Subramanian, K. Kawakami, and C. Dyer (2016). "Neural Architectures for Named Entity Recognition". In: *Proceedings of NAACL-HLT 2016*, pp. 260–270.

Ma, X. and E. Hovy (2016). "End-to-end Sequence Labeling via Bi-directional LSTM-CNNs-CRF". In: *Proceedings of the 54th Annual Meeting of the ACL*, pp. 1064–1074.

Madhyastha, P. and R. Jain (2019). "On Model Stability as a Function of Random Seed". In: *Proceedings of CoNLL 2019*, pp. 929–939.

McCoy, R. T., J. Min, and T. Linzen (2020). "BERTs of a feather do not generalize together: Large variability in generalization across models with similar test set performance". In: *Proceedings of the Third BlackboxNLP Workshop*, pp. 217–227.

Mosbach, M., M. Andriushchenko, and D. Klakow (2021). "On the Stability of Fine-tuning BERT: Misconceptions, Explanations, and Strong Baselines". In: *International Conference on Learning Representations*.

Peters, M. E., M. Neumann, M. Iyyer, M. Gardner, C. Clark, K. Lee, and L. Zettlemoyer (2018). "Deep Contextualized Word Representations". In: *Proceedings of NAACL-HLT 2018*, pp. 2227–2237.

Picard, D. (2021). *torch.manual_seed(3407) is all you need: On the influence of random seeds in deep learning architectures for computer vision*. Preprint, arXiv:2109.08203.

Reimers, N. and I. Gurevych (2017). "Reporting Score Distributions Makes a Difference: Performance Study of LSTM-networks for Sequence Tagging". In: *Proceedings of EMNLP 2017*, pp. 338–348.

Reimers, N. and I. Gurevych (2018). *Why Comparing Single Performance Scores Does Not Allow to Draw Conclusions About Machine Learning Approaches*. Preprint, arXiv:1803.09578.

Searle, S. R., G. Casella, and C. E. McCulloch (1992). *Variance Components*. Wiley.

Student (1908). “The Probable Error of a Mean”. In: *Biometrika* 6.1, pp. 1–25.

Tippett, L. H. C. (1925). “On the Extreme Individuals and the Range of Samples Taken from a Normal Population”. In: *Biometrika* 17.3/4, pp. 364–387.

Tjong Kim Sang, E. F. and F. De Meulder (2003). “Introduction to the CoNLL-2003 Shared Task: Language-Independent Named Entity Recognition”. In: *Proceedings of CoNLL 2003*, pp. 142–147.

Zhang, T., F. Wu, A. Katiyar, K. Q. Weinberger, and Y. Artzi (2021). “Revisiting Few-sample BERT Fine-tuning”. In: *International Conference on Learning Representations*.

2 Instability

Why a fixed seed does not fix a run

THE CLAIM *"Fixing the random seed makes a training run reproducible."*

Common practice in code and papers; framework documentation promises less

THE VERDICT Narrowed: a seed repeats a run only on an identical stack of code, kernels, and hardware, and where training is unstable, a one-bit change reproduces most of the seed-to-seed variability.

IN THIS CHAPTER. Setting every random seed is the standard recipe for a reproducible training run, and on one machine with deterministic kernels it does repeat a run bit for bit. Another GPU, library version, batch size, or order of summation can change the last bits of the arithmetic, and Summers and Dinneen (2021) found that moving one initial weight of a ResNet by the smallest representable amount produces as much run-to-run variability as all sources of randomness together. We prove how far a reordered floating-point sum can move, that training which contracts forgets such differences while training which expands amplifies them, and what a perturbed twin of a run measures about the run distribution. In the laboratory a fixed seed repeats a digits network exactly; a one-bit twin stays identical in function when the labels are clean and becomes largely a new draw when a fifth of them are noisy.

2.1 The claim: set every seed

Training code begins by seeding every random number generator it touches, and frameworks add switches for deterministic kernels: in PyTorch, one call makes operations with nondeterministic implementations use deterministic ones or raise an error, and a flag stops the convolution library from timing several algorithms and keeping the fastest (PyTorch Contributors 2026).[1] A paper then states that all seeds were fixed, and its reader takes the reported number to be a reproducible fact about the method.

The documentation promises less. PyTorch does not guarantee reproducible results across releases, commits, or platforms, nor between CPU and GPU with identical seeds, and warns that deterministic operations are often slower (PyTorch Contributors 2026). NVIDIA's matrix library cuBLAS repeats its results bit for bit only within one toolkit version, on GPUs of one architecture with the same number of multiprocessors (NVIDIA 2026a), and the convolution library cuDNN lists routines, among them algorithms for the backward pass of convolution and of max pooling, whose atomic

[1] The call is `torch.use_deterministic_algorithms(True)`, the flag `torch.backends.cudnn.benchmark = False`.

operations make the rounding differ from run to run (NVIDIA 2026b).[2]

The strongest version of the case was made carefully. Nagarajan et al. (2018), in a preprint, built a deterministic implementation of deep Q-learning, with a seeded generator for each source of randomness and PyTorch's deterministic GPU mode on, and released one source at a time.

> FINDING 2.1 (Nagarajan et al. 2018). Five runs of deep Q-learning on the game Breakout, 20 million steps each, with every seed fixed and deterministic GPU operations, produced identical learning curves. With one source of randomness released and five runs per source, the standard deviation of the final score relative to its mean was 12.4% for nondeterministic GPU operations, 6.7% for the sampling of minibatches, 16.9% for exploration, 37.7% for a stochastic environment, and 43.6% for the initial weights.

The authors separated *replicability*, repeating an experiment exactly, from *reproducibility*, obtaining the same results with minor differences, and called deterministic implementations necessary but not sufficient for the first: in their supplementary material, the same deterministic implementation produced different learning curves on two machines with different GPUs, operating systems, and library versions, while on each machine it repeated itself exactly.

The field received the shorter version: fix the seeds, and the result is reproducible. In a survey of 901 researchers and practitioners, Pham et al. (2020) found that 83.8% were unaware of, or unsure about, variability introduced by the deep learning libraries themselves, and only 10.4% expected two runs with the same seeds to differ in accuracy by 2 points or more; of 225 recent papers that trained deep networks, 19.5% (margin 3 points) reported several runs of one configuration.

Much of the practice is right. On a fixed stack of code, libraries, and hardware, exact repetition is possible and valuable: a changed number means changed code, which makes debugging possible, and a model or a published number can be regenerated for an audit. The labs of this book rely on it. Two questions remain: does the repetition survive a change of machine, library, or batch layout, and is the repeated number representative of the method? The first is about arithmetic, the second about the dynamics of training.

2.2 The experiment: six groups of five runs

The design of Nagarajan et al. (2018) is the cleanest experiment behind the practice: it builds a pipeline in which nothing varies and then lets one source of randomness vary at a time.

EXPERIMENTAL DESIGN · NAGARAJAN, WARNELL AND STONE 2018	
QUESTION	How much of the variability of a deep Q-learning agent does each source of randomness produce?

[2] An atomic addition lets many threads add into one memory location without losing updates. The additions happen in the order in which the threads arrive, which changes from run to run.

UNITS	training runs of deep Q-learning on Atari Breakout, 20 million steps each
TREATMENT	the one source of randomness left uncontrolled
LEVELS	none; GPU operations; sticky actions in the environment; exploration; initial weights; minibatch sampling
HELD FIXED	all other seeds; one machine (GeForce GTX 1080, PyTorch 0.3.0, CUDA 8.0, cuDNN 7.0); hyperparameters
RESPONSE	mean score over 100 greedy evaluation episodes from fixed start states, at the end of training and at the best checkpoint
REPLICATES	five runs per group
ANALYSIS	standard deviation and relative standard deviation in each group; tests of equal variance between groups at level 0.1

The design establishes that determinism can be reached on one machine and that releasing a single source spreads the outcome. It cannot show that the deterministic numbers recur on another machine, and the authors' own comparison of two machines shows that they do not. It ranks the sources only coarsely: for normal scores, the 95% interval for σ from a sample standard deviation S of five runs extends from $0.60S$ to $2.87S$ (Exercise 2.1), so the GPU and minibatch groups, at 12.4% and 6.7%, cannot be told apart, and the one difference in variance that the authors report at level 0.1 is between the GPU group and the initialization group.

Most important, the design cannot tell a source from the training that amplifies it. Each row of Finding 2.1 varies a source and, with it, the size and the time of its first effect: the initial weights differ at step zero by a full random draw, while the runs of the GPU group start from last-bit differences and, as the authors observed, share identical experiences for tens of thousands of steps. If training amplifies every perturbation, then each row measures the amplification as much as the source. The post-mortem separates the two.

2.3 The post-mortem: the stack and the dynamics

A fixed seed fixes the random numbers. It does not fix the arithmetic, which depends on the stack, and it does not fix the outcome, which depends on what training does with small differences.

THE STACK. Floating-point addition is not associative, so a change in the order in which a reduction adds its terms can change the last bits of the result (Section 2.4.1). On parallel hardware the order changes often: atomic additions complete in the order in which threads arrive, autotuning can choose a different algorithm in each run (Pham et al. 2020), libraries choose the blocking of a matrix product by the shapes of the matrices and hence by the batch size, and data-parallel training adds the gradients of several devices in an order set by their number. Pham et al. (2020) trained a WideResNet-28-10 on CIFAR-100 sixteen times on one GPU, with the same libraries and every algorithmic source of randomness fixed: the test accuracies ranged from 77.3% to 80.2%, and the accuracy on one class,

camel, from 38.1% to 90.5%.

Zhuang et al. (2022) separated the randomness that seeds control from that of the tooling: for ResNet-50 on ImageNet, two runs disagreed on 14.68% of the test predictions with tooling noise alone and on 14.89% with seed noise alone. Tensor processing units, deterministic by design, removed the tooling noise,[3] but a different order of the examples inside a batch still changed the trained models, even when the whole training set formed one batch, because that order is the order in which the gradient is accumulated. Deterministic GPU kernels took up to 7.46 times the GPU time of the default ones.[4] Even a kernel without atomic operations can return different bits for the same input in batches of different sizes. He (2025), in a blog post, traced the nondeterminism of language-model inference servers to this missing *batch invariance*: at temperature zero, 1000 completions of one prompt gave 80 distinct texts, and batch-invariant kernels made all 1000 identical.

[3] A TPU multiplies matrices with systolic arrays under a single-threaded, deterministic model of computation (Zhuang et al. 2022).

[4] On a P100 with 7×7 convolutions; on a V100 and a T4 the largest factors were 2.41 and 1.96.

THE DYNAMICS. Summers and Dinneen (2021) designed the experiment that separates a source from its amplification. They trained a ResNet-14 on CIFAR-10 for 500 epochs (cosine schedule from a learning rate of 0.4, batch size 512, momentum 0.9), 100 runs for each source of randomness with the others held fixed: the initialization, the shuffling of the data, the data augmentation, and the nondeterministic cuDNN kernels, alone and in combinations. Then they added a source of their own: in each run, one randomly chosen initial weight was moved to the next representable number, a change of the order of 10^{-10}.

> FINDING 2.2 (Summers and Dinneen 2021). ResNet-14 on CIFAR-10, 100 runs per source: every source of randomness, alone or combined with others, gave a standard deviation of test accuracy between 0.21 and 0.26 percentage points, and two runs disagreed on 10.5% to 10.7% of the test images. Moving one initial weight to the next representable number gave 0.21 points and 10.6%.

They called the phenomenon the *instability* of training, by analogy with numerical stability: viewed end to end, training is ill-conditioned, so that, as they argued, every source of randomness that changes the weights at all inherits at least the variability of a one-bit change. When they released a source only after N deterministic epochs, the variability of accuracy was nearly as large for $N = 50$ as for $N = 0$ and fell only gradually as the onset moved later.

The paper also narrows its own claim. In a linear model a one-bit change altered no test prediction, and in a network with one convolutional hidden layer and in a small network on MNIST it reproduced about half of the disagreement that all sources produced together (Table 2.1); the authors suggested that the degree of instability depends on the model, the data, and the degree of overfitting. Jordan (2024) replicated the effect by multiplying a single weight by 1.001: in CIFAR-10 trainings of 128 epochs this alone produced an average disagreement of 5.14%, against 5.19% between runs with entirely different seeds, and in short trainings 7.0–7.5% against about 8.5%.

data	model	all sources	one bit	ratio
CIFAR-10	linear	5.5	0.0	0.00
CIFAR-10	one convolutional hidden layer	12.7	6.4	0.50
MNIST	small convolutional network	0.56	0.30	0.54
CIFAR-10	one fully connected hidden layer	24.9	23.4	0.94
CIFAR-10	VGG-11	6.6	6.5	0.98
CIFAR-10	ResNet-14	10.7	10.6	0.99
ImageNet	ResNet-18	20.7	20.6	1.00
Penn Treebank	QRNN language model	17.4	17.7	1.02

Table 2.1. Disagreement between two runs, in percent of test images (of tokens for Penn Treebank), when every source of randomness varies and when only one initial weight moves by one unit in the last place; from Tables 2, 3, 4, 6, and 7 of Summers and Dinneen (2021), 100 runs per row except 20 for ImageNet. The ratio of the two columns is ours.

A related notion of stability concerns where training ends rather than what the network predicts. Frankle et al. (2020) trained two copies of a network from its state at step k with different data orders and asked whether the error rises along the straight line between the end points: only LeNet on MNIST was stable in this sense from initialization, ResNet-20 on CIFAR-10 after 3% of its training, and ResNet-50 on ImageNet after 20%. Fort et al. (2020) found that a chaotic early phase of two to three epochs determines the linearly connected basin in which training ends. Neither finding contradicts the onset experiment of Summers and Dinneen: two networks in one basin can still disagree on many test images. Stability in this sense returns in the lottery-ticket experiments of Chapter 8.

VERDICT. Fixing every seed does repeat a run bit for bit, but only with the same code, library versions, kernels, and hardware, as the documentation of PyTorch, cuBLAS, and cuDNN says. Where it holds, the repetition is fragile. In the deep networks of Summers and Dinneen, moving one initial weight by one unit in the last place reproduced nearly all of the disagreement between independently seeded runs, while in a linear model it changed nothing and in two small models it reproduced about half. A fixed seed selects one draw from the run distribution and makes it repeatable; it does not make it representative.

NARROWED

2.4 The method: repeatable runs and reproducible results

Three different things go by the name of reproducibility, and a seed controls only the first.

Definition 2.3 (Three levels of reproducibility). Fix a training procedure (code, data, and hyperparameters) and an *environment* $\mathcal{E}$: the hardware, the library versions, the kernels selected, the numbers of devices and threads, and the layout of the batches. A run with seeds s in $\mathcal{E}$ produces an outcome $Y(s, \mathcal{E})$.

(i) The procedure is *repeatable* in $\mathcal{E}$ if two runs with the same seeds in $\mathcal{E}$ produce the same bits, for every s.

(ii) It is *distributionally reproducible* between $\mathcal{E}$ and $\mathcal{E}'$ if, with seeds drawn at random, $Y(s, \mathcal{E})$ and $Y(s, \mathcal{E}')$ have the same distribution.

(iii) A conclusion of an experiment, a decision computed from its runs such as "method 1 beats method 2", is *reproducible* if an independent replicate of the experiment, with fresh seeds and possibly in another environment, reaches the same decision with high probability.

Repeatability is a property of a pipeline on a machine, distributional reproducibility a property of the run distribution, and the reproducibility of a conclusion a property of the design: the size of the effect, the number of runs, and the test (Chapter 1). None implies the next: a repeatable pipeline can repeat an unrepresentative run, and a pipeline with nondeterministic kernels is not repeatable but can be distributionally reproducible. The three levels are close to what Goodman et al. (2016) call methods, results, and inferential reproducibility.[5]

[5] The words are used inconsistently across fields, sometimes with the meanings of *reproducible* and *replicable* swapped (Goodman et al. 2016). Nagarajan et al. (2018) use replicability for our repeatability.

Remark 2.4 (What a seed buys). A repeatable pipeline makes three things possible. Debugging: a changed number means a changed input, which bisection over versions of the code finds. Audit: a reported model can be regenerated. Pairing: two arms of a comparison run with the same seeds share their initialization and data order, and if their outcomes are positively correlated, the variance $\mathrm{Var}(Y_1) + \mathrm{Var}(Y_2) - 2\,\mathrm{Cov}(Y_1, Y_2)$ of their difference is smaller than with independent seeds (Chapter 1); the correlation lasts only as long as the two runs stay close, which in unstable training is not long (Section 2.5). A seed does not buy evidence: the number it repeats is one draw from the run distribution.

2.4.1 The arithmetic beneath a run

A binary floating-point format with *precision* p represents a nonzero number as $\pm m\,2^e$, with an integer significand $2^{p-1} \le m < 2^p$ and an integer exponent e in a fixed range. Between 1 and 2 the representable numbers are 2^{1-p} apart; this spacing is one *unit in the last place* (ulp) of 1, and half of it is the *unit roundoff* $u = 2^{-p}$, about 6.0×10^{-8} in binary32 (single precision), where $p = 24$, and 3.9×10^{-3} in the 16-bit format bfloat16. The IEEE 754 standard (IEEE 2019) requires that, in its default mode, a sum be rounded to a nearest representable number, which gives the model from which rounding-error analysis starts (Higham 2002, Chapter 2).

Assumption 2.5 (Standard model of floating-point addition). For floating-point numbers a and b whose sum does not overflow, the computed sum is $\mathrm{fl}(a+b) = (a+b)(1+\delta)$ for some δ with $|\delta| \le u$.

The exact sum of two floating-point numbers is in general not representable, the rounded sum of several depends on which are added first, and so floating-point addition is not associative.

EXAMPLE 2.6 (Three numbers). In binary32 let $x = (1, 2^{-24}, 2^{-24})$. The exact sum $1 + 2^{-24}$ lies halfway between the neighbors 1 and $1 + 2^{-23}$; the tie goes to the neighbor whose last bit is even, which is 1, and adding the second 2^{-24} gives 1 again. Added the other way, $2^{-24} + 2^{-24} = 2^{-23}$ exactly, and $1 + 2^{-23}$ is representable. The two orders give 1 and $1 + 2^{-23}$: they differ in the last bit, by $2^{-23} = 2u$.

Over a chain of operations the standard model compounds into products of factors $1+\delta$. For integers $m \ge 0$ with $mu < 1$ define $\gamma_m = mu/(1-mu)$.

Lemma 2.7 (Products of rounding factors). *Let $m \ge 1$ with $mu < 1$. If $|\delta_k| \le u$ for $k = 1, \dots, m$, then $\prod_{k=1}^{m}(1+\delta_k) = 1 + \theta$ with $|\theta| \le \gamma_m$.*

PROOF. Each factor lies in $[1-u, 1+u]$, an interval of positive numbers because $u \le mu < 1$, so the product lies between $(1-u)^m$ and $(1+u)^m$. For the upper end, $1 + u \le e^u$, and $e^{mu} \le 1/(1-mu)$ because $e^{-x} \ge 1 - x$ for $x = mu \in [0, 1)$; hence $(1+u)^m - 1 \le 1/(1-mu) - 1 = \gamma_m$. For the lower end, Bernoulli's inequality, valid since $-u \ge -1$, gives $(1-u)^m \ge 1 - mu \ge 1 - \gamma_m$. ∎

A *summation tree* for $x_1, \dots, x_n$ is a binary tree whose leaves are the x_i and whose internal nodes add the values of their two children; evaluated in floating point, it computes a sum $\hat{s}_T$. The depth d_i of the leaf x_i is the number of internal nodes on its path to the root, and the depth of the tree is $h = \max_i d_i$. *Recursive summation* in the order $x_1, \dots, x_n$ computes $\hat{s}_1 = x_1$ and $\hat{s}_k = \mathrm{fl}(\hat{s}_{k-1} + x_k)$ for $k = 2, \dots, n$; its tree has depth $n-1$. *Pairwise summation* adds neighbors in a balanced tree of depth $\lceil \log_2 n \rceil$. The reductions of parallel hardware use trees of this kind, whose shape depends on how the work is divided among threads.

Proposition 2.8 (Error of a floating-point sum). *Under Assumption 2.5, let $n \ge 2$, let $s = \sum_i x_i$, and let T be a summation tree of depth h with $hu < 1$.*

(i) The computed sum is $\hat{s}_T = \sum_{i=1}^{n} x_i(1+\theta_i)$ with $|\theta_i| \le \gamma_{d_i}$, so that $|\hat{s}_T - s| \le \gamma_h \sum_i |x_i|$.

(ii) Two summation trees of depths h and h' give sums that differ by at most $(\gamma_h + \gamma_{h'}) \sum_i |x_i|$. For recursive summation in two orders the bound is $2\gamma_{n-1} \sum_i |x_i| = 2(n-1)u \sum_i |x_i| + O(u^2)$, and for pairwise summation in two orders it is $2\gamma_{\lceil \log_2 n \rceil} \sum_i |x_i|$.

(iii) Recursive summation satisfies $|\hat{s}_n - s| \le \frac{u}{1-u} \sum_{k=2}^{n} |\hat{s}_k|$.

PROOF. The plan is an induction over the tree for (i), the triangle inequality for (ii), and a telescoping sum for (iii).

Part (i). For a node v let $\hat{v}$ be its computed value, so that $\hat{v} = x_i$ when v is the leaf x_i. For an internal node v let δ_v be the rounding error of its addition, and for a leaf x_i below v let $P(i, v)$ be the set of internal nodes on the path from x_i up to v, with v included and $P(i, x_i)$ empty. We show, by induction over the nodes with children before parents, that

$$\hat{v} = \sum_{x_i \text{ below } v} x_i \prod_{w \in P(i,v)} (1 + \delta_w). \tag{2.1}$$

For a leaf the product is empty and (2.1) reads $x_i = x_i$. For an internal node v with children a and b, the standard model gives $\hat{v} = (\hat{a} + \hat{b})(1 + \delta_v)$; inserting (2.1) for a and b, and using $P(i, v) = P(i, a) \cup \{v\}$ for the leaves below a and likewise for b, gives (2.1) for v. At the root the product for x_i has d_i factors, with $d_i \geq 1$ because $n \geq 2$ and $d_i u \leq hu < 1$; Lemma 2.7 turns it into $1 + \theta_i$ with $|\theta_i| \leq \gamma_{d_i} \leq \gamma_h$, since γ_m increases with m. Hence $|\hat{s}_T - s| = |\sum_i x_i \theta_i| \leq \gamma_h \sum_i |x_i|$.

Part (ii). The exact sum s is the same for both trees, so the triangle inequality and part (i) give the first bound. The recursive tree has depth $n - 1$, the balanced tree depth $\lceil \log_2 n \rceil$, and $\gamma_{n-1} = (n - 1)u + O(u^2)$.

Part (iii). For $k \geq 2$ the standard model gives $\hat{s}_{k-1} + x_k = \hat{s}_k/(1 + \delta_k)$, that is, $\hat{s}_k - \hat{s}_{k-1} - x_k = \hat{s}_k \, \delta_k/(1 + \delta_k)$. Summing over $k = 2, \ldots, n$ and using $\hat{s}_1 = x_1$, the left side telescopes to $\hat{s}_n - s$, and $|\delta_k/(1 + \delta_k)| \leq u/(1 - u)$. ∎

In Example 2.6 the two orders differ by $2u$, while part (ii) allows $2\gamma_2(1 + 2^{-23}) \approx 4u$. For long sums the worst case is far from typical, and part (iii) says why: the error is controlled by the partial sums, and a sum of terms of mixed sign, such as the sum of per-example gradients over a minibatch, keeps them small; with rounding errors modeled as independent with mean zero, bounds with $\sqrt{n}\, u$ in place of nu hold with high probability (Higham and Mary 2019). Training computes such sums over the examples of a minibatch, the inner dimension of a matrix product, and the devices of data-parallel training, and every change of their order moves the result by an amount that Proposition 2.8 makes tiny. The question is what training does with it.

2.5 The method: how training amplifies a bit

Let θ_t be the state of a run after t steps, its parameters together with the optimizer's buffers, and write one step in exact arithmetic as $\theta \mapsto \Phi_t(\theta)$. The map Φ_t depends on the minibatch of step t, its learning rate, and the random numbers of dropout or augmentation; fixed seeds fix the sequence $\Phi_0, \Phi_1, \ldots$. A *twin* of a run uses the same seeds and differs by a perturbation: another initial state, as when one weight moves by an ulp, or other rounding errors, as when a minibatch is summed in another order. In floating point the run computes $\theta_{t+1} = \Phi_t(\theta_t) + \rho_t$ and its twin $\theta'_{t+1} = \Phi_t(\theta'_t) + \rho'_t$, with rounding errors ρ_t and ρ'_t; with $e_t = \theta'_t - \theta_t$ and $r_t = \rho'_t - \rho_t$,

$$e_{t+1} = \Phi_t(\theta_t + e_t) - \Phi_t(\theta_t) + r_t. \tag{2.2}$$

Lemma 2.9 (Propagation of a difference). *Suppose that* $\|\Phi_t(\theta'_t) - \Phi_t(\theta_t)\| \le L_t\|e_t\|$ *and* $\|r_t\| \le \varepsilon_t$ *for every* $t \ge 0$. *Then, for every* t,

$$\|e_t\| \le \Big(\prod_{k=0}^{t-1} L_k\Big)\|e_0\| + \sum_{j=0}^{t-1}\Big(\prod_{k=j+1}^{t-1} L_k\Big)\varepsilon_j.$$

In particular, if $L_t \le L < 1$ *and* $\varepsilon_t \le \varepsilon$ *for every* t, *then* $\|e_t\| \le L^t\|e_0\| + \varepsilon/(1-L)$.

Proof. By (2.2) and the hypotheses, $\|e_{t+1}\| \le L_t\|e_t\| + \varepsilon_t$. The bound holds for $t = 0$, where the product is empty and the sum has no terms, and inserting it for t into this inequality gives it for $t+1$. In the special case the sum is at most $\varepsilon \sum_{j\ge 0} L^j = \varepsilon/(1-L)$. ∎

If every step contracts, then a perturbation is forgotten and the rounding differences stay bounded. If steps expand, then the product of the factors grows exponentially. Gradient descent on a strongly convex loss is the first case.

Proposition 2.10 (Contraction forgets). *Let* $f : \mathbb{R}^d \to \mathbb{R}$ *be twice continuously differentiable with* $\mu I \preceq \nabla^2 f(\theta) \preceq MI$ *for every* θ, *where* $0 < \mu \le M$, *and let* $0 < \eta \le 2/(\mu + M)$. *Then the step* $\Phi(\theta) = \theta - \eta\nabla f(\theta)$ *satisfies* $\|\Phi(\theta) - \Phi(\theta')\| \le (1 - \eta\mu)\|\theta - \theta'\|$, *and two runs of gradient descent on* f *whose rounding errors differ by at most* ε *per step stay within* $(1-\eta\mu)^t\|e_0\| + \varepsilon/(\eta\mu)$ *of each other at every step* t.

Proof. The plan is to write the difference of two gradients with an averaged Hessian. For $\theta, \theta' \in \mathbb{R}^d$, the fundamental theorem of calculus applied to $s \mapsto \nabla f(\theta' + s(\theta - \theta'))$, whose derivative is continuous because f is twice continuously differentiable, gives $\nabla f(\theta) - \nabla f(\theta') = \bar{H}(\theta - \theta')$ with $\bar{H} = \int_0^1 \nabla^2 f(\theta' + s(\theta - \theta'))\,\mathrm{d}s$. As an average of such matrices, $\bar{H}$ is symmetric with $\mu I \preceq \bar{H} \preceq MI$. Hence $\Phi(\theta) - \Phi(\theta') = (I - \eta\bar{H})(\theta - \theta')$, and every eigenvalue of $I - \eta\bar{H}$ lies in $[1 - \eta M, 1 - \eta\mu]$. The condition $\eta \le 2/(\mu + M)$ gives $\eta M - 1 \le 1 - \eta\mu$, and, since $2/(\mu + M) \le 1/\mu$, also $1 - \eta\mu \ge 0$; so every eigenvalue has absolute value at most $1 - \eta\mu$, which bounds the norm of the symmetric matrix $I - \eta\bar{H}$. The second statement is Lemma 2.9 with $L_t = 1 - \eta\mu < 1$, for which $1 - L_t = \eta\mu$. ∎

Softmax regression with weight decay λ is λ-strongly convex on every minibatch, so stochastic gradient descent on it with a small enough step, and the same minibatches in both runs, forgets a one-ulp change; convexity is the reason Summers and Dinneen (2021) gave for the stability of their linear models. The bound $\varepsilon/(\eta\mu)$ can far exceed ε, but it does not grow with t.

Proposition 2.11 (Expansion amplifies). *Let* $f(\theta) = \frac{1}{2}\theta^{\top}H\theta$ *with* H *symmetric, let* v *be a unit eigenvector of* H *with eigenvalue* λ, *and let* $\eta > 0$. *Two runs of gradient descent on* f *in exact arithmetic, started at* θ_0 *and*

$\theta_0 + e_0$, differ at step t by $e_t = (I - \eta H)^t e_0$, and $\langle v, e_t\rangle = (1-\eta\lambda)^t\langle v, e_0\rangle$. If $|1-\eta\lambda| > 1$, which happens exactly when $\lambda < 0$ or $\eta\lambda > 2$, and $\langle v, e_0\rangle \neq 0$, then $\|e_t\| \geq |1-\eta\lambda|^t|\langle v, e_0\rangle|$; in particular, for every $\Delta > |\langle v, e_0\rangle|$, $\|e_t\| \geq \Delta$ from the step $\lceil \ln(\Delta/|\langle v, e_0\rangle|)/\ln|1-\eta\lambda|\rceil$ on.

PROOF. The step $\Phi(\theta) = (I - \eta H)\theta$ is linear, so $e_{t+1} = (I - \eta H)e_t$ and, by induction, $e_t = (I - \eta H)^t e_0$. Because H is symmetric and $Hv = \lambda v$, we have $\langle v, (I - \eta H)w\rangle = (1 - \eta\lambda)\langle v, w\rangle$ for every w, which gives the formula for $\langle v, e_t\rangle$, and the Cauchy–Schwarz inequality gives $\|e_t\| \geq |\langle v, e_t\rangle|$. The step count follows by taking logarithms in $|1 - \eta\lambda|^t|\langle v, e_0\rangle| \geq \Delta$, which is legitimate because $|1 - \eta\lambda| > 1$. ∎

A one-ulp change of a weight in binary32 has relative size about 2^{-24}, and growing to the size of the weights takes about $24 \ln 2/\ln|1-\eta\lambda| \approx 17/\ln|1-\eta\lambda|$ steps: about 175 when $|1 - \eta\lambda| = 1.1$, and 24 when it is 2. Both causes of expansion occur in deep learning: saddle points have directions of negative curvature, and Cohen et al. (2021) found that full-batch gradient descent on neural networks typically runs at the *edge of stability*, with the largest eigenvalue of the Hessian hovering just above $2/\eta$. A quadratic with an expanding direction diverges outright, however, and a run that stays bounded while separating from its twin needs nonlinearity. Gradient descent on a double well with a large step is exactly chaotic (Exercise 2.3), and Kong and Tao (2020) showed that deterministic gradient descent with a large learning rate on a multiscale objective can behave like a stochastic method. For a network, the linear picture becomes a heuristic.[6]

[6] The butterfly of chaos theory entered in 1972, in the title that the convener of a session chose for a talk by Lorenz, whose 1963 model of convection had shown the effect (Lorenz 1963).

HEURISTIC DERIVATION: THE DIVERGENCE TIME. While the twins are close, (2.2) is linear to first order: $e_{t+1} \approx J_t e_t + r_t$, where $J_t = D\Phi_t(\theta_t)$ is the Jacobian of the step along the run; for gradient descent $J_t = I - \eta\nabla^2\widehat{\mathcal{L}}_t(\theta_t)$, with $\widehat{\mathcal{L}}_t$ the loss on the minibatch of step t. Define the *cumulative exponent* $\Lambda(t) = \ln\big(\|J_{t-1}\cdots J_0 e_0\|/\|e_0\|\big)$. Then $\|e_t\| \approx \|e_0\|e^{\Lambda(t)}$, plus the rounding differences r_j carried forward by the later Jacobians, until the linearization fails at a distance Δ of the order of the distance between runs that share the initialization but not the subsequent noise. If $\Lambda(t) \approx \ell t$ with a positive rate ℓ, a finite-time Lyapunov exponent, then the twins separate exponentially, and the *divergence time* is

$$t^\star \approx \frac{\ln(\Delta/\|e_0\|)}{\ell}.$$

Three predictions follow: the logarithm of the distance grows linearly before it saturates near Δ; each factor of ten in the initial perturbation advances the divergence by $\ln(10)/\ell$ steps; and no perturbation acts as if it were smaller than the rounding differences that every step adds, so that below that floor a one-ulp change and a reordered sum are the same experiment. If the rate varies over training, then $t^\star$ solves $\Lambda(t^\star) = \ln(\Delta/\|e_0\|)$, and the twins separate where Λ rises steeply. If Λ stays bounded, as under Proposition 2.10, then they never separate. ◇

A network with ReLU activations is only piecewise smooth: when a perturbation pushes a pre-activation across zero, the Jacobian changes abruptly, and the difference between the twins can jump. Shamir et al. (2020), in a preprint, found that smooth activations improve the trade-off between accuracy and the reproducibility of predictions.

2.5.1 What a twin measures

The heuristic says when twins separate, not how different they end. The run distribution answers that question.

Proposition 2.12 (What a twin measures). *Let Y and Y' be the outcomes of a run and of its twin, with $\mathbb{E}Y^2 < \infty$, and let $\mathcal{G}$ be a σ-field, the information that the two runs share. Suppose that, conditionally on $\mathcal{G}$, the outcomes Y and Y' are independent and have the same distribution, and that $\operatorname{Var} Y > 0$. Let Y'' be the outcome of an independent run with the distribution of Y. Then*

$$\operatorname{Cov}(Y, Y') = \operatorname{Var}\big(\mathbb{E}[Y \mid \mathcal{G}]\big), \qquad \mathbb{E}(Y - Y')^2 = 2\,\mathbb{E}\operatorname{Var}(Y \mid \mathcal{G}),$$

so that $\rho = \operatorname{Corr}(Y, Y') = \operatorname{Var}(\mathbb{E}[Y \mid \mathcal{G}])/\operatorname{Var} Y$ lies in $[0, 1]$, and

$$\frac{\mathbb{E}(Y - Y')^2}{\mathbb{E}(Y - Y'')^2} = 1 - \rho.$$

Proof. The plan is to condition on $\mathcal{G}$. Write $m = \mathbb{E}[Y \mid \mathcal{G}]$. Because Y' has the conditional distribution of Y, $\mathbb{E}[Y' \mid \mathcal{G}] = m$ and $\mathbb{E}[Y'^2 \mid \mathcal{G}] = \mathbb{E}[Y^2 \mid \mathcal{G}]$; taking expectations, Y' has the mean and the variance of Y. Conditional independence gives $\mathbb{E}[YY' \mid \mathcal{G}] = m^2$, so $\operatorname{Cov}(Y, Y') = \mathbb{E}m^2 - (\mathbb{E}m)^2 = \operatorname{Var} m$. Since the means are equal,

$$\mathbb{E}(Y - Y')^2 = \operatorname{Var} Y + \operatorname{Var} Y' - 2\operatorname{Cov}(Y, Y') = 2(\operatorname{Var} Y - \operatorname{Var} m) = 2\,\mathbb{E}\operatorname{Var}(Y \mid \mathcal{G}),$$

the last step by the law of total variance. For the independent run, $\operatorname{Cov}(Y, Y'') = 0$, and the same computation gives $\mathbb{E}(Y - Y'')^2 = 2\operatorname{Var} Y$. Dividing, and using $\operatorname{Var} Y' = \operatorname{Var} Y$ in the definition of ρ, gives the ratio. Finally $0 \le \operatorname{Var} m \le \operatorname{Var} Y$ by the law of total variance, so $\rho \in [0, 1]$. ∎

For a twin, $\mathcal{G}$ holds the shared initialization, the data order, and the trajectory up to the divergence. If the twin separates at once and keeps nothing that matters, then $\rho = 0$ and the twin is a new draw from the run distribution. If it never separates, then $\rho = 1$ and the twin is a copy. In both cases Y' has the distribution of Y, so a two-sample test of twins against runs with fresh seeds cannot tell a copy from a new draw; the paired quantities can. Applied to the prediction on one test example, the same argument shows that twins disagree on it at most as often as independent runs (Exercise 2.4); the fraction of test examples on which two runs disagree is called *churn* (Milani Fard et al. 2016).

2.6 In the laboratory: twins of a digits network

A GPU is not needed to see the phenomena of this chapter: on a CPU we can create the perturbations that a GPU creates by accident, a moved bit and a reordered sum, on purpose and one at a time. The labs use the digits network of the book's kit, a perceptron with two hidden layers of 64 ReLU units and 8970 parameters.

SCRIPT
`02-instability-sums.py`

LAB 2.1 · WHAT A FIXED SEED REPEATS

SETUP. Binary32 arithmetic on a laptop CPU: the three numbers of Example 2.6; for each n from 10 to 10^5, twenty vectors of n standard normal numbers, each summed in 50 random orders, recursively and pairwise; the logits of a trained network for the 539 test images, computed in batches of 539, 64, and 1; and three runs with the same seeds, one of them on one thread instead of two. A few seconds.

PREDICT FIRST. How many of 50 random orders of 10^5 numbers give the same sum? Do the logits of an image depend on how many other images share its batch?

LOOK FOR. The number of distinct sums and their spread against Proposition 2.8; the images whose logits change in some bit; whether the hashes of the final weights agree.

The two orders of Example 2.6 gave 1 and $1 + 2^{-23}$. In Table 2.2, recursive summation of 10^5 numbers in 50 random orders gave 46 distinct results on average, spread over 1.6 units of $u \sum_i |x_i|$, where part (ii) of Proposition 2.8 allows 2×10^5; the mean error stayed near $0.27\, u \sum_i |x_i|$ for every n, and the largest ratio of an error to the bound of part (iii), over all 5000 recursive sums, was 0.57. The logits of a trained network did not change when the same batch was recomputed, but in batches of 64 those of 5.0% of the test images changed in some bit, and one image at a time those of every image changed, by at most 7.6×10^{-6}; no predicted class changed. The three runs with the same seeds gave weights with one hash: on this machine (PyTorch 2.13 with Apple's Accelerate library) the thread count did not change the order of the sums, which need not hold elsewhere.

SCRIPT
`02-instability-twins.py`

LAB 2.2 · TWIN RUNS

SETUP. The digits network trained by SGD with momentum 0.9, learning rate 0.1, and batch size 64 for 50 epochs (950 steps), in two regimes that differ only in the labels: clean, and with 20% of the training labels replaced by random labels. For each of ten seeds per regime: the run, a repeat, a one-ulp twin (one random initial weight moved to its neighbor by `torch.nextafter`), a reorder twin (the examples inside every minibatch permuted), an order twin (a new data-order seed), and an independent run, with the distance between parameter vectors every 5 steps. With noisy labels, also twins

	recursive			pairwise		
n	distinct	spread	bound	distinct	spread	bound
10	5.5	1.39	18	5.0	1.24	8
100	15.5	1.69	198	7.9	0.65	14
1000	23.6	1.58	1998	7.5	0.25	20
10 000	37.6	1.52	20 010	9.1	0.09	28
100 000	46.2	1.64	201 197	10.7	0.03	34

Table 2.2. Sums of n standard normal binary32 numbers in 50 random orders (Lab 2.1; 20 vectors per n, seed 20261017). Distinct: number of different results among the 50 orders, averaged over the vectors. Spread: largest minus smallest result, in units of $u \sum_i |x_i|$, averaged over the vectors. Bound: $2\gamma_h/u$, the worst case of Proposition 2.8(ii) in the same units, with $h = n-1$ for recursive and $h = \lceil \log_2 n \rceil$ for pairwise summation.

moved in a random direction by 10^{-7} to 10^{-2} of the initial weight norm. About 30 seconds.

PREDICT FIRST. With clean labels, will a one-ulp twin end with a different test error? With noisy labels? At which step will the distance between twins reach 1?

LOOK FOR. The floor at which every twin sits after a few dozen steps; whether and when the distance leaves it; where it saturates; how the step of divergence depends on the size of the initial change.

Figure 2.1 shows the result. A repeat with identical seeds stayed identical at all 950 steps in both regimes, and the script's own training loop, which adds the perturbations, reproduced the kit's runs bit for bit. Within 50 steps every twin, one-ulp or reorder, sat at a distance of about 10^{-6}, a few units of u relative to the norm of the weights: the rounding floor of the heuristic, at which a moved bit and a reordered sum are indistinguishable. In one pair of the ten the moved bit was rounded away within fifteen steps, and that twin stayed bitwise identical to its run in both regimes.[7]

With clean labels nothing more happened. The distance drifted to about 10^{-5}, at most 2.5×10^{-4}, while order twins ended about 3.3 apart; no one-ulp or reorder twin disagreed with its run on a single test image, while order twins disagreed on 1.15% and independent runs on 1.91% of them. The difference stayed near the rounding floor, as it would under Proposition 2.10, although the loss is not convex.

With noisy labels the twins stayed near the floor for several hundred steps and then separated within a few dozen: the logarithm of the distance grew at a median rate of 0.012 per step until the distance reached 10^{-4}, and of 0.093 per step from there to 1. The one-ulp twins crossed a distance of 1 at a median step of 535 (range 345–585) and the reorder twins at 512 (230–590), and both saturated near 11, while order twins ended about 25 apart. At the end, one-ulp twins disagreed with their runs on 20.9% of the test images and reorder twins on 23.4%, against 26.8% for order twins and

[7] A difference can vanish only by rounding: once the perturbed weight rounds to the same number as the original, every later computation is identical.

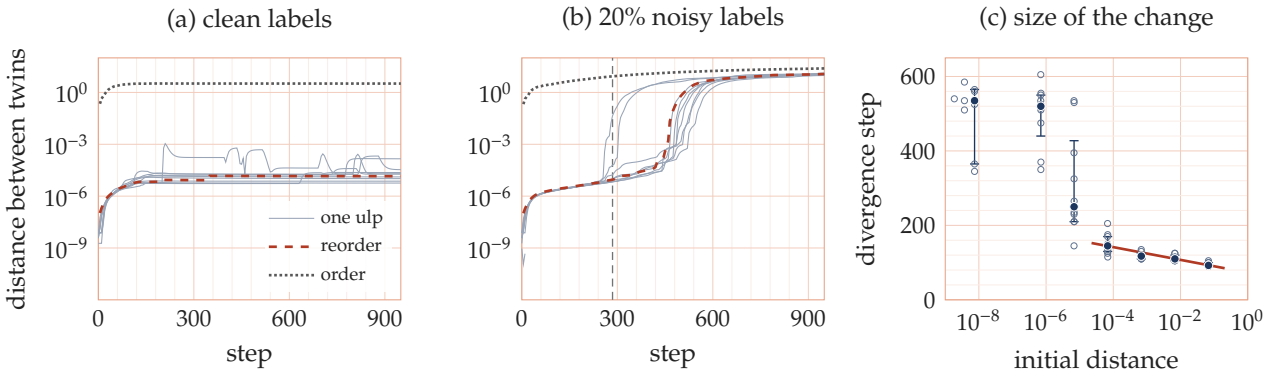


Figure 2.1. Twin runs of the digits network (8970 parameters; SGD with momentum 0.9, learning rate 0.1, batch size 64, 950 steps); seeds $i = 0, \ldots, 9$, run i with initialization and order seeds i. (a) Clean labels and (b) 20% random training labels: Euclidean distance between the parameters of a run and of its twin. Thin lines: one-ulp twins, one per seed; dashed: median over seeds of reorder twins; dotted: median of order twins. Gaps mark a distance of exactly zero; a repeat with identical seeds is at zero throughout and is not drawn. The vertical line in (b) is the median step at which the runs have fitted half of their 237 mislabeled training examples. (c) Noisy labels: first step at which the distance exceeds 1, against the initial distance, for one-ulp twins (left) and for twins moved in a random direction by $10^{-7}, \ldots, 10^{-2}$ of the initial weight norm. Circles: seeds; filled: medians with 95% percentile-bootstrap intervals; line: least-squares fit to the four largest sizes.

28.1% for independent runs. The burst came after a run had fitted half of its 237 mislabeled training examples (median step 285); at the divergence step a run had fitted a median 76% of them (range 59–81%). The instability lives in the phase in which the network memorizes wrong labels. In panel (c), initial changes of 10^{-5} to 10^{-2} of the weight norm diverged in the early phase, at median steps from 145 down to 92, a slope of 7.4 steps per unit of ln or an exponent of 0.135 per step, while changes of one ulp and 10^{-7} waited for the late burst, near steps 535 and 520, and 10^{-6} fell in between (median 250). The exponent is not constant, and the divergence time follows the cumulative exponent Λ rather than a single rate, as the last sentence of the heuristic allows.

LAB 2.3 · TWINS AGAINST NEW SEEDS

SETUP. The network, optimizer, and regimes of Lab 2.2. In each regime, 100 runs with seeds $i = 0, \ldots, 99$, their one-ulp twins, and 100 runs with new seeds $1000 + i$; with noisy labels also 100 early twins, moved by 1% of the initial weight norm, which diverge near step 90. Then 20 pairs for each of five learning rates from 0.0125 to 0.15. About 2.5 minutes.

PREDICT FIRST. Will a two-sample test tell twins from runs with new seeds? Will the correlation between a run and its twin?

LOOK FOR. The p-values of the permutation and Kolmogorov–Smirnov tests; the correlation between each run and its twin; the ratio of disagreements across learning rates.

SCRIPT
`02-instability-draws.py`

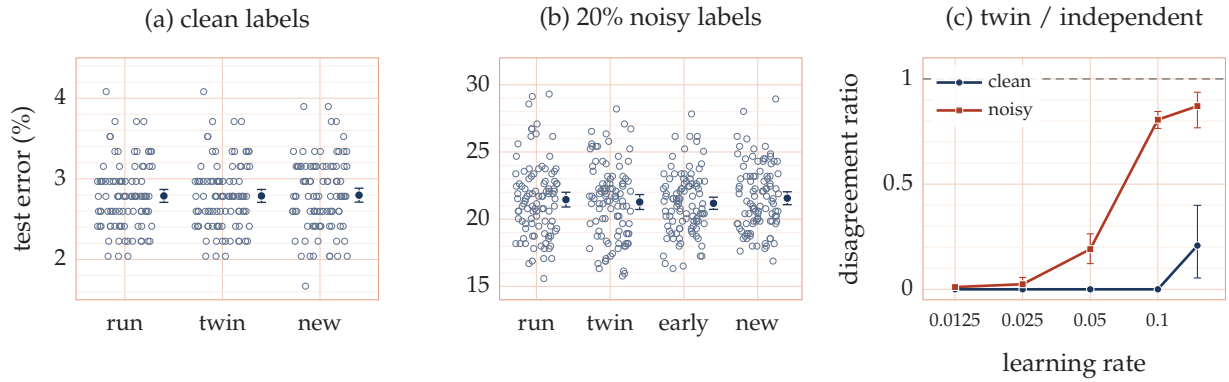


Figure 2.2. Test errors of the network of Figure 2.1. (a) Clean labels and (b) 20% random training labels: 100 runs (seeds $i = 0, \dots, 99$), their one-ulp twins, early twins moved by 1% of the initial weight norm in a random direction (noisy labels only), and 100 runs with new seeds $1000 + i$. Circles: runs, spread horizontally; filled, to the right of each group: means with 95% t-intervals. (c) Disagreement on the 539 test images between a run and its one-ulp twin, divided by the mean disagreement between a run and an independently seeded one, against the learning rate; 20 pairs per point, 95% percentile-bootstrap intervals over pairs. The dashed line is the ratio for two independent runs.

In both regimes the twins passed the tests of distributional reproducibility against new seeds (Figure 2.2).[8] With clean labels the mean test errors were $2.79 \pm 0.08\%$ for the runs, $2.79 \pm 0.08\%$ for their twins, and $2.80 \pm 0.09\%$ for new seeds (means with 95% t-intervals, 100 runs each), and twins against new seeds gave $p = 0.90$ in a two-sided permutation test of the means and $p = 0.82$ in a Kolmogorov–Smirnov test. With noisy labels the means were $21.45 \pm 0.55\%$, $21.28 \pm 0.56\%$, $21.18 \pm 0.45\%$ for the early twins, and $21.56 \pm 0.48\%$, and the tests gave $p = 0.44$ and $p = 0.47$ for the one-ulp twins. As Proposition 2.12 predicts, these tests cannot see what separates the regimes; the paired statistics can. With clean labels the correlation between the test errors of a run and its twin was 1.00, and no twin disagreed with its run on a single image: the twins were copies, and three of the 100 ended bitwise identical. With noisy labels the correlation was 0.25 (95% Fisher interval 0.06 to 0.43), against 0.12 (−0.07 to 0.31) between runs and new seeds, and the twins disagreed with their runs on 20.7% of the test images, against 27.4% for new seeds: a ratio of 0.75 (bootstrap interval 0.72 to 0.78). The early twins, which separate near step 90 instead of 535, came closer to new draws, with correlation 0.21 (0.01 to 0.39) and ratio 0.85 (0.83 to 0.87): the earlier twins separate, the less they share. The direct ratio of mean squared differences, 0.98 (0.70 to 1.37), is too noisy to use.

In panel (c) the disagreement ratio stayed at 0 with clean labels for every learning rate up to 0.1 and reached 0.21 (0.05 to 0.40) at 0.15. With noisy labels it rose from 0.01 at 0.0125, where the runs did not memorize the wrong labels in 50 epochs, through 0.19 at 0.05, to 0.81 at 0.1 and 0.87 (0.77 to 0.94) at 0.15. The miniature reproduces both halves of the verdict: a fixed seed repeats a run exactly, and a one-bit change is erased in the stable regime and becomes most of a new draw in the unstable one. It does not reach the ratios near 1 of Table 2.1, because its twins diverge late, after

[8] One test image is 0.19 percentage points of test error, which is why the runs in Figure 2.2 fall on a grid.

sharing half of their training. We chose the learning rate, the number of epochs, and the label noise after a short pilot scan, which the regime map reports in full; the network, data, and optimizer are the defaults of the kit.

2.7 Protocol

PROTOCOL 2.1 · REPEATABLE RUNS, REPRODUCIBLE RESULTS

- ☐ Give each source of randomness its own seed (initialization, data order, augmentation and dropout, data split) and record all of them with the result.
- ☐ For debugging and audit runs, turn on deterministic algorithms and turn off autotuning; record the code version, library versions, hardware, numbers of devices and threads, batch size, and determinism flags, and store a hash of the final weights.
- ☐ Check repeatability before relying on it: run one configuration twice and compare the hashes. If they differ, then find the nondeterministic operation before going on.
- ☐ Check stability: train a twin with one weight moved by one ulp, or with the examples of each minibatch reordered, and compare its predictions with the run's. If they differ, then treat every seed as a random draw and expect bitwise repetition to fail on other hardware.
- ☐ Draw seeds before training and never choose them by their results; report every run, with n runs per arm sized as in Chapter 1.
- ☐ When the hardware or a library changes, test distributional reproducibility: n runs with fresh seeds in each environment, a two-sample test of the outcomes, and the comparison of interest recomputed in both.
- ☐ State which level of reproducibility a reported number has: repeatable on a stated stack, or a summary of a run distribution with its interval.

The protocol does not make a result portable: no seeding makes two GPU architectures add in the same order, and only order-independent arithmetic, such as the reproducible summation of Demmel and Nguyen (2015), would. Determinism costs time, up to several times the GPU time (Zhuang et al. 2022), so it belongs in debugging and audit runs rather than in every run of a sweep. A twin test with a few pairs detects instability that matters but not a weak correlation between twins, which needs about a hundred pairs (Lab 2.3), and a two-sample test with few runs per environment has little power (Chapter 1). When a run is too expensive to repeat, as in large-scale pretraining, run the twin test on a small version of the model, report whether training was stable there, and treat differences between single runs smaller than the spread of seeds as unresolved.

Exercises

2.1 CONSOLIDATION. (Five runs and a standard deviation.) For normal outcomes, $(n-1)S^2/\sigma^2$ has the χ^2 distribution with $n-1$ degrees of freedom. (a) Derive the 95% interval for σ from S, and show that it runs from $0.60S$ to $2.87S$ for $n=5$ and from $0.76S$ to $1.46S$ for $n=20$. (b) Which rows of Finding 2.1 could be told apart with five runs per group?

2.2 CONSOLIDATION. (Momentum.) For heavy-ball momentum on $f(\theta)=\frac{1}{2}\lambda\theta^2$, the twin difference obeys $e_{t+1}=(1+\beta-\eta\lambda)e_t-\beta e_{t-1}$. (a) For $0\le\beta<1$, show that $e_t\to 0$ for every start exactly when $0<\eta\lambda<2(1+\beta)$, and that it grows exponentially when $\lambda<0$ or $\eta\lambda>2(1+\beta)$. (b) For $\eta=0.1$ and $\beta=0.9$, the settings of Lab 2.2, find the curvature above which a direction expands. *(Hint: a polynomial z^2+az+b with real coefficients has both roots inside the unit circle exactly when $|b|<1$ and $|a|<1+b$.)*

2.3 CONSOLIDATION. ★ (Chaotic gradient descent.) Gradient descent with step $\eta=2$ on the double well $f(x)=\frac{1}{4}(x^2-1)^2$ is the map $x\mapsto 3x-2x^3$. (a) Show that the minima $x=\pm1$ are unstable fixed points. (b) Substitute $x=\sqrt{2}\cos\varphi$ and show that the map becomes $\varphi\mapsto 3\varphi+\pi$ modulo 2π, and that $[-\sqrt{2},\sqrt{2}]$ is invariant. (c) Deduce that two orbits that start 2^{-24} apart in φ are of order one apart after about 15 steps, and check numerically that the average of $\ln|\Phi'(x_t)|$ along an orbit is $\ln 3\approx 1.099$. *(Hint:* $\cos 3\varphi=4\cos^3\varphi-3\cos\varphi$.*)*

2.4 CONSOLIDATION. (Twins disagree less.) Under the hypotheses of Proposition 2.12, let the outcome be the class that a run predicts for one test example, and let $p_k=\mathbb{P}(\text{the run predicts } k\mid\mathcal{G})$. Show that two twins disagree on the example with probability $1-\mathbb{E}\sum_k p_k^2$ and two independent runs with probability $1-\sum_k(\mathbb{E}p_k)^2$, that the first is at most the second, and that they are equal exactly when every p_k is almost surely constant.

2.5 LAB. (A smaller step.) Run `02-instability-twins.py` with the learning rate 0.025 instead of 0.1. Predict from Figure 2.2(c) whether the noisy-label twins will leave the rounding floor, then check, and relate the outcome to the fraction of mislabeled examples that the runs fit.

2.6 LAB. (Smooth activations.) Repeat the noisy-label part of Lab 2.3 with `activation="tanh"` in the configuration. Does the disagreement ratio change? Compare with the claim of Shamir et al. (2020) that smooth activations make predictions more reproducible, and say what your experiment can and cannot show about it.

2.7 LAB. (Where the exponent lives.) From the distances recorded in Lab 2.2, compute the growth rate of the logarithm of the distance over windows of 25 steps for the noisy-label one-ulp twins, and plot it against the step. Where is it largest, and what does that say about $\Lambda(t)$?

2.8 DESIGN. (Two GPU types.) A team wants to know whether moving a fine-tuning pipeline to another type of GPU changes the distribution of test accuracy. Seeds move the accuracy with a standard deviation of 0.4 points, and the budget is 40 runs. Give the runs per type, the test, the smallest shift detected with power 0.8 at level 0.05, and what must be held fixed and recorded.

2.9 DESIGN. (When does a run commit?) In the noisy regime of Lab 2.2, design an experiment with 200 runs that finds the step after which releasing a new data-order seed changes the test predictions less than half as much as releasing it at step zero, in the spirit of the onset experiment of Summers and Dinneen (2021). State the treatment and its levels, the response, the replicates, and the analysis.

2.10 CRITIQUE. A paper reports: "We fixed all random seeds (seed 42) for reproducibility. Our method improves test accuracy over the baseline by 0.4 points." Each method was trained once, on eight GPUs. Which level of Definition 2.3 does the statement establish? What would you ask the authors for, and what does Remark 2.4 say about whether the shared seed makes the comparison more precise?

2.11 CRITIQUE. (An invented report.) A team moves training to another type of GPU, sees the accuracy of a single run drop by 0.6 points, and concludes that the new GPU's arithmetic is less accurate. What else changed with the hardware, what should the drop be compared with, and which experiment would test the claim?

2.12 RESEARCH. (When does a run forget?) Proposition 2.10 covers strongly convex losses, the edge of stability of Cohen et al. (2021) places full-batch training where the sharpest direction neither contracts nor expands, and Figure 2.2(c) shows the instability switching on with the learning rate and the label noise. Open: predict the cumulative exponent $\Lambda(t)$ of a network from quantities measurable early in training, such as curvature relative to $2/\eta$, gradient noise, and the fraction of examples not yet fitted, and hence when a one-bit twin becomes a new draw.

Notes and References

Goldberg (1991) is the classic introduction to floating-point arithmetic, and Higham (2002) the reference for the rounding errors of summation, compensated summation included; Higham and Mary (2019) gave the probabilistic analysis behind the $\sqrt{n}\,u$ behavior of typical sums. Summation whose result does not depend on the order of the terms has been designed for parallel machines (Demmel and Nguyen 2015).

Sensitive dependence on initial conditions was found by Lorenz (1963) in a model of atmospheric convection, and Strogatz (2015) explains Lyapunov exponents and why the horizon of prediction grows only with the logarithm of the precision of the initial state. For training, Summers and Dinneen (2021)

gave the decisive experiment, Jordan (2024) replicated it while connecting run-to-run variance to the finite test set, and Kong and Tao (2020) and Cohen et al. (2021) studied gradient descent at learning rates where it does not converge in the classical sense.

On the reproducibility of machine learning research, Gundersen and Kjensmo (2018) surveyed 400 papers from two AI conferences and found that none documented every variable needed to reproduce it, Pineau et al. (2021) report on the code policy, the reproducibility challenge, and the checklist of NeurIPS 2019, and Bouthillier et al. (2019) showed that a result can be repeated exactly and still fail to hold under a slightly different experiment. Churn was named as a problem of shipping successive versions of a model (Milani Fard et al. 2016).

TRANSLATING THE LITERATURE. Summers and Dinneen (2021) call a source of randomness a *nondeterminism source*, our one-ulp twin a *random bit change*, and our churn *pairwise disagreement*; their *instability* is our amplification. Frankle et al. (2020) use *instability* for the rise of the error along the line between two runs spawned from one state, so that a run can be stable in their sense and unstable in ours. Zhuang et al. (2022) write ALGO for the randomness that seeds control and IMPL for the nondeterminism of the environment $\mathcal{E}$. Pham et al. (2020) call sources of randomness *NI-factors* and our repeats *fixed-seed identical training runs*; Nagarajan et al. (2018) call our repeatability *replicability*; and Jordan (2024) calls a twin made by multiplying one weight by 1.001 a *poke*.

References

Bouthillier, X., C. Laurent, and P. Vincent (2019). "Unreproducible Research is Reproducible". In: *Proceedings of the 36th International Conference on Machine Learning*. Vol. 97. Proceedings of Machine Learning Research, pp. 725–734.

Cohen, J. M., S. Kaur, Y. Li, J. Z. Kolter, and A. Talwalkar (2021). "Gradient Descent on Neural Networks Typically Occurs at the Edge of Stability". In: *International Conference on Learning Representations*.

Demmel, J. and H. D. Nguyen (2015). "Parallel Reproducible Summation". In: *IEEE Transactions on Computers* 64.7, pp. 2060–2070.

Fort, S., G. K. Dziugaite, M. Paul, S. Kharaghani, D. M. Roy, and S. Ganguli (2020). "Deep Learning versus Kernel Learning: An Empirical Study of Loss Landscape Geometry and the Time Evolution of the Neural Tangent Kernel". In: *Advances in Neural Information Processing Systems*. Vol. 33.

Frankle, J., G. K. Dziugaite, D. M. Roy, and M. Carbin (2020). "Linear Mode Connectivity and the Lottery Ticket Hypothesis". In: *Proceedings of the 37th International Conference on Machine Learning*. Vol. 119. Proceedings of Machine Learning Research, pp. 3259–3269.

Goldberg, D. (1991). "What Every Computer Scientist Should Know about Floating-Point Arithmetic". In: *ACM Computing Surveys* 23.1, pp. 5–48.

Goodman, S. N., D. Fanelli, and J. P. A. Ioannidis (2016). "What Does Research Reproducibility Mean?" In: *Science Translational Medicine* 8.341, 341ps12.

Gundersen, O. E. and S. Kjensmo (2018). "State of the Art: Reproducibility in Artificial Intelligence". In: *Proceedings of the Thirty-Second AAAI Conference on Artificial Intelligence*, pp. 1644–1651.

He, H. (2025). *Defeating Nondeterminism in LLM Inference*. Connectionism (blog), Thinking Machines Lab. URL: https://thinkingmachines.ai/blog/defeating-nondeterminism-in-llm-inference/.

Higham, N. J. (2002). *Accuracy and Stability of Numerical Algorithms*. 2nd ed. Philadelphia: SIAM.

Higham, N. J. and T. Mary (2019). "A New Approach to Probabilistic Rounding Error Analysis". In: *SIAM Journal on Scientific Computing* 41.5, A2815–A2835.

IEEE (2019). *IEEE Standard for Floating-Point Arithmetic*. IEEE Std 754-2019, revision of IEEE Std 754-2008. IEEE Computer Society.

Jordan, K. (2024). "On the Variance of Neural Network Training with respect to Test Sets and Distributions". In: *International Conference on Learning Representations*.

Kong, L. and M. Tao (2020). "Stochasticity of Deterministic Gradient Descent: Large Learning Rate for Multiscale Objective Function". In: *Advances in Neural Information Processing Systems*. Vol. 33.

Lorenz, E. N. (1963). "Deterministic Nonperiodic Flow". In: *Journal of the Atmospheric Sciences* 20, pp. 130–141.

Milani Fard, M., Q. Cormier, K. Canini, and M. Gupta (2016). "Launch and Iterate: Reducing Prediction Churn". In: *Advances in Neural Information Processing Systems*. Vol. 29.

Nagarajan, P., G. Warnell, and P. Stone (2018). *Deterministic Implementations for Reproducibility in Deep Reinforcement Learning*. arXiv preprint 1809.05676.

NVIDIA (2026a). *cuBLAS Library Documentation: Results Reproducibility*. URL: https://docs.nvidia.com/cuda/cublas/ (visited on 10/17/2026).

NVIDIA (2026b). *cuDNN Developer Guide: Reproducibility (Determinism)*. URL: https://docs.nvidia.com/deeplearning/cudnn/latest/developer/misc.html (visited on 10/17/2026).

Pham, H. V., S. Qian, J. Wang, T. Lutellier, J. Rosenthal, L. Tan, Y. Yu, and N. Nagappan (2020). "Problems and Opportunities in Training Deep Learning Software Systems: An Analysis of Variance". In: *Proceedings of the 35th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, pp. 771–783.

Pineau, J., P. Vincent-Lamarre, K. Sinha, V. Larivière, A. Beygelzimer, F. d'Alché-Buc, E. Fox, and H. Larochelle (2021). "Improving Reproducibility in Machine Learning Research (A Report from the NeurIPS 2019 Reproducibility Program)". In: *Journal of Machine Learning Research* 22.164, pp. 1–20.

PyTorch Contributors (2026). *Reproducibility*. PyTorch 2.14 documentation, notes on randomness. URL: https://docs.pytorch.org/docs/2.14/notes/randomness.html (visited on 10/17/2026).

Shamir, G. I., D. Lin, and L. Coviello (2020). *Smooth Activations and Reproducibility in Deep Networks*. arXiv preprint 2010.09931.

Strogatz, S. H. (2015). *Nonlinear Dynamics and Chaos: With Applications to Physics, Biology, Chemistry, and Engineering*. 2nd ed. Westview Press.

Summers, C. and M. J. Dinneen (2021). "Nondeterminism and Instability in Neural Network Optimization". In: *Proceedings of the 38th International Conference on Machine Learning*. Vol. 139. Proceedings of Machine Learning Research, pp. 9913–9922.

Zhuang, D., X. Zhang, S. L. Song, and S. Hooker (2022). "Randomness in Neural Network Training: Characterizing the Impact of Tooling". In: *Proceedings of Machine Learning and Systems*. Vol. 4.

3 Many Tasks, Few Runs

Aggregates, intervals and multiplicity

THE CLAIM *"A higher median score over a few runs per game identifies the better algorithm."*

THE VERDICT Narrowed: rerun 100 times per game under one evaluation protocol, some claimed gains hold, while others are within noise or reverse.

The practice of the Atari 100k benchmark, 2019–2021, as audited by Agarwal et al. (2021)

IN THIS CHAPTER. Between 2019 and 2021, a succession of papers on the Atari 100k benchmark reported gains over earlier agents, mostly in the median human-normalized score over 3 to 20 runs per game, and the best were read as the new state of the art. Agarwal et al. (2021) trained five of these agents 100 times on each game: medians from a few runs scattered as widely as the gaps between agents, the sample median was biased, and some agents had been scored by a maximum. The method is to know what an aggregate over tasks estimates and how precisely: two levels of variation, the effective number of tasks behind the mean, the median and the interquartile mean, the stratified bootstrap and its shortfall with three runs, and the comparisons hidden in per-task tests, in maxima and in the choice of aggregate. In a twelve-task miniature built from digits, a baseline scored by its best checkpoint overtakes a better method, the aggregate decides between two others, and two hundred three-run replications show the median's interval missing a real gain in half of them.

3.1 The claim: a new state of the art every few months

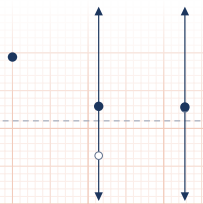
Reinforcement learning on Atari games is evaluated on a suite. The Arcade Learning Environment of Bellemare et al. (2013) made dozens of Atari 2600 games a common testbed, and Mnih et al. (2015) put the games on one scale: an agent's score on a game is rescaled so that a random policy scores 0 and a human player scores 1,

$$\text{human-normalized score} = \frac{\text{agent score} - \text{random score}}{\text{human score} - \text{random score}},$$

and the scores are summarized across games by their mean or their median. One number per agent made agents comparable at a glance, and the number became the result.

Kaiser et al. (2020) asked how well an agent can play after very little experience. Their model-based agent, SimPLe, learned from 100,000 interactions with each game, that is 400,000 frames with the usual frame skip of four, or about two hours of real-time play,[1] on 26 games, and they reported

[1] The exact budget in Kaiser et al. (2020) is $6400 \times 16 = 102{,}400$ interactions, 409,600 frames or 114 minutes at 60 frames per second.



the mean and standard deviation of five runs on each game, comparing with model-free agents game by game. The setting became the *Atari 100k* benchmark,[2] and its aggregate became the median human-normalized score over the 26 games.

A sequence of papers followed, each reporting gains over the agents it compared with. van Hasselt et al. (2019) changed a few hyperparameters of Rainbow (Hessel et al. 2018) (one update per environment step, 20-step returns, learning from replay after 1,600 transitions), and this *data-efficient Rainbow*, DER, beat SimPLe on 17 of the 26 games with five runs each.[3] In a preprint, Kielak (2020) trained Rainbow with eight updates per environment step, called it OTRainbow, for "overtrained", and reported a median of 0.204 against SimPLe's 0.102. CURL (Laskin et al. 2020) added a contrastive loss on image representations and reported a median of 0.175 over 20 runs, against 0.144 for SimPLe and 0.161 for DER: the "1.2x" gain on Atari in its abstract.[4] DrQ (Yarats et al. 2021) augmented the input images and reported a median of 0.268 over five runs, a new state of the art. SPR (Schwarzer et al. 2021) added a self-predictive loss and reported 0.415 over ten runs, "a 55% relative improvement" over the previous state of the art.

[2] The 26 games were those on which SimPLe or Rainbow obtained non-random results at 100k interactions (Kaiser et al. 2020, Section 6): the suite was selected with the help of the methods it was first used to compare.

[3] The SimPLe numbers in that comparison were those of SimPLe's first preprint; the conference version reports revised scores and concedes that tuned Rainbow is on par, better on 13 games and worse on 13 (Kaiser et al. 2020).

[4] The first two arXiv versions of CURL used 3 runs per game and reported a median of 0.24; the conference version moved to another code base and 20 runs.

FINDING 3.1 (Schwarzer et al. 2021, Table 1). On the 26 games of Atari 100k, the mean and median human-normalized scores of six agents were as follows; the runs per game are those of the original papers, as listed by Agarwal et al. (2021) and Kaiser et al. (2020). SPR led by both aggregates.

	SimPLe	DER	OTRainbow	CURL	DrQ	SPR
runs per game	5	5	5	20	5	10
mean	0.443	0.285	0.264	0.381	0.357	0.704
median	0.144	0.161	0.204	0.175	0.268	0.415

DrQ and then SPR were received as the new state of the art. The claim this chapter examines is the practice behind such readings: that the agent with the higher median over a few runs per game is the better agent. Prior results on the benchmark had been computed mostly from three or five runs per game, more rarely from ten or twenty (Agarwal et al. 2021).

The practice was reasonable. Five runs on each of 26 games is already 130 trainings per agent, and the median was chosen on purpose: on a few games an agent can reach several times the human score, and those games dominate a mean (by the mean, SimPLe is ahead of CURL and DrQ; by the median, behind both). The improvements were also, in part, real: DER and OTRainbow corrected a comparison against untuned baselines, the subject of Chapter 4, and SPR's lead was large. What no paper reported was how far its median would move when the runs were repeated.

3.2 The experiment: twenty-six games, a few runs, one number

The comparisons behind these claims share one design.

EXPERIMENTAL DESIGN · ATARI 100K COMPARISONS, 2019–2021

QUESTION	Is the new agent more sample-efficient than earlier agents on Atari?
UNITS	A run: one agent trained from scratch on one game for 100k interactions.
TREATMENT	The agent: the new method against three to six earlier methods.
TUNED	The new method's hyperparameters, sometimes per game; baselines as published.
HELD FIXED	The 26 games and the budget of 100k interactions.
NOT FIXED	The evaluation protocol of the baselines, their code bases, and the human reference scores used to normalize.
RESPONSE	A run's human-normalized score: its mean return over a set of evaluation episodes.
REPLICATES	3 to 20 runs per game, most often 3 or 5.
ANALYSIS	Mean over runs on each game; median and mean over games; point estimates set against numbers copied from earlier papers; win counts over games.

This design answers one question well: did the new agent's runs, on these games, average higher than the numbers printed in earlier papers? It cannot answer the question it was read as answering, because three of its features are invisible in its output.

First, the median of 26 game means is a random quantity, and one set of runs gives one draw of it. Its spread over reruns is not in the table, nor is the spread of the numbers it is compared with, although with five runs per game an interval could have been computed by resampling runs. The gaps between consecutive claims were small: 0.161 to 0.175 from DER to CURL, 0.204 to 0.268 from OTRainbow to DrQ.

Second, the comparison has no common control arm. Each paper ran its own agent and copied the baselines' numbers, so a difference in the table mixes the difference of agents with differences of evaluation protocol, code base and reference scores. Even one baseline's numbers moved between papers: SimPLe's median is 0.102 in Kielak (2020) and 0.144 in later tables, and DER's is 0.147 in early versions of CURL and 0.161 in the final one, through changes in the SimPLe results used and in the human reference scores.[5]

Third, a table over 26 games is a family of comparisons. A paper that bolds the best score on each game and counts the bold entries makes 26 comparisons at once, and a protocol that keeps the best of several evaluations makes several more per run, none of them adjusted for. None of the three is a flaw in the agents; each is a gap between what the design measures and what its single number was taken to mean.

[5] DrQ's own reported median moved between its arXiv versions, from 0.270 to 0.284 to 0.268 (Yarats et al. 2021).

3.3 The post-mortem: one hundred runs per game

Agarwal et al. (2021) measured what the design leaves out by running it a hundred times. For five agents, DER, OTRainbow, CURL, DrQ and SPR, and for DrQ(ϵ), DrQ with the standard exploration settings of the benchmark, they trained 100 independent runs on each of the 26 games and scored each run by its mean return over 100 evaluation episodes after training. DER and OTRainbow, which had no public code, were reimplemented in the Dopamine framework; CURL and SPR were run from their authors' code, and DrQ from code its authors provided. Subsampling these runs gives the distribution of every statistic that a paper with fewer runs could have reported.

> FINDING 3.2 (Agarwal et al. 2021, Section 3 and Appendix A). With 100 runs per game for each agent: (i) medians recomputed from as many runs as each paper had used (5, 10 or 20) vary widely, and the published medians lie far out in these distributions, some too high and some too low; (ii) the expected sample median changes with the number of runs, by an amount and in a direction that depend on the agent: for SPR the change from 5 to 100 runs, +0.03, is about 36% of its improvement over DrQ(ϵ), +0.08; (iii) 95% intervals for the median remain wide with 50 runs, and with no true difference at all two sets of 100 runs can differ by 20% in median, while the interquartile mean needs far fewer runs for the same precision; (iv) CURL's published scores were the maximum over 10 evaluations during training, and DrQ's code evaluated with a 5% chance of random actions instead of the standard 0.1%.

By the reported numbers OTRainbow was ahead of DER; with a hundred runs, DER "may in fact be better" than OTRainbow (Agarwal et al. 2021, Section 3), although part of this may come from implementation, since both were reimplemented. The protocol mattered more than the runs. Scored the way CURL was scored, by the maximum over evaluations during training, DER exceeds CURL's published scores by a wide margin, and scored at the end of training it is ahead of CURL: the gap between the two in the published table belongs to the protocol. SUNRISE (Lee et al. 2021), which reported improvements over DER, had taken the maximum over eight hyperparameter configurations separately for each game, with three runs each, and simulating that protocol on DER explained much of the gain (Agarwal et al. 2021, Appendix A.4). SPR's lead, by contrast, held: with a hundred runs it remained the strongest of the five.[6]

The audit found the same pattern on the 200-million-frame Atari benchmark, where the ranking of agents changed with the aggregate, on the DeepMind Control Suite, and on Procgen, where several claimed improvements had a probability of improvement of only 0.5 to 0.7. Its recommendations were interval estimates of aggregates from a stratified bootstrap, performance profiles, the interquartile mean in place of the median, the optimality gap and the probability of improvement, and publication of every run; the authors released a library, rliable, and all of their runs.

[6] Read from the axes of Figure 3 of Agarwal et al. (2021): the expected medians order the agents SPR, DrQ(ϵ), DER, OTRainbow, CURL at every number of runs from 3 to 100.

NARROWED

VERDICT. The claims that rested on a median of a few runs per game hold only where the gap between agents exceeds the uncertainty of that median and the agents were scored under one protocol. With a hundred runs per game, SPR stays ahead; DER, reported below OTRainbow and CURL, may be ahead of the first and is ahead of the second; and the advantage printed for CURL came from its evaluation protocol. The point estimates were not wrong as measurements; they could not carry the conclusions drawn from them.

The post-mortem did not show that these agents were poor, and its recommendations are not the only defensible ones. It argues for intervals rather than dichotomous tests, whereas earlier audits recommended tests: Henderson et al. (2018), whose seed experiment opens this book (**??**), proposed bootstrap intervals, significance tests and power analysis, and Colas et al. (2018, 2019) recommended Welch's t-test with corrections for multiple comparisons, warning that bootstrap tests have high false-positive rates with few runs. Jordan et al. (2020) built intervals with guaranteed coverage after finding that percentile bootstrap intervals failed 11% of the time with ten runs. These disagree about the form of an uncertainty statement, not about whether to make one; and the interquartile mean is not the true aggregate, only a choice of what to measure.

3.4 The method: what an aggregate estimates

An aggregate over a suite comes from a sample with two levels, tasks and runs within tasks; before computing an interval we need to know which level a claim is about and how the aggregate weighs the tasks.

TWO LEVELS OF VARIATION. A suite has J tasks, indexed by j, and each method is run n times on each task. The normalized score of run i on task j is Y_{ji}. Given the task, the runs are independent draws from the task's run distribution, with mean μ_j and variance σ_j^2, and the task mean $\bar{Y}_j = \frac{1}{n}\sum_i Y_{ji}$ estimates μ_j. A *suite-level* claim is about these J tasks, for instance about $\frac{1}{J}\sum_j \mu_j$: the agent is better on these 26 games. A *population-level* claim treats the tasks as a sample from a population of tasks: the agent is better on games like these. The two have different uncertainties.

Proposition 3.3 (Two levels of variation). *Let the pairs (μ_j, σ_j^2), $j = 1, \dots, J$, be independent and identically distributed, with $\omega^2 = \operatorname{Var}\mu_j$ and $\bar{\sigma}^2 = \mathbb{E}\sigma_j^2$ finite, and let the runs on task j be, given (μ_j, σ_j^2), independent with mean μ_j and variance σ_j^2. The suite mean $\bar{Y} = \frac{1}{J}\sum_j \bar{Y}_j$ satisfies*

$$\operatorname{Var}\left(\bar{Y} \,\middle|\, \mu_{1:J}, \sigma^2_{1:J}\right) = \frac{1}{J^2}\sum_{j=1}^{J}\frac{\sigma_j^2}{n}, \qquad \operatorname{Var}\bar{Y} = \frac{\omega^2}{J} + \frac{\bar{\sigma}^2}{nJ}. \tag{3.1}$$

PROOF. Given the tasks, the task means are independent with variances σ_j^2/n, and the variance of their average is the first formula. The law of

total variance adds the mean of this conditional variance, $\bar{\sigma}^2/(nJ)$, to the variance of the conditional mean $\frac{1}{J}\sum_j \mu_j$, which is ω^2/J because the μ_j are independent with variance ω^2. ■

More runs shrink only the second term; the first, ω^2/J, is fixed by the number of tasks. For two methods compared on the same tasks, apply the proposition to the per-task difference: ω^2 is then the variance of the effect across tasks, small when a method helps everywhere by the same amount and large when it helps on some tasks and hurts on others. The stratified bootstrap resamples runs within fixed tasks, so it estimates the conditional variance and supports suite-level claims. A test across tasks, such as the signed-rank test of Section 3.5, treats the tasks as the sampling units and supports population-level claims, but only to the extent that the tasks are a sample; a suite chosen by hand, like the 26 games, is not.

AGGREGATES. Five summaries of a method's scores are in common use (Agarwal et al. 2021). The *mean* is $\frac{1}{J}\sum_j \bar{Y}_j$, and the *median* is $\mathrm{med}(\bar{Y}_1, \dots, \bar{Y}_J)$. The *interquartile mean*, IQM, pools all nJ scores, discards the lowest and the highest quarter, and averages the middle half: it is the 25% trimmed mean of the pooled runs. The *optimality gap* at a target γ is $\mathrm{OG}_\gamma = \gamma - \frac{1}{nJ}\sum_{j,i} \min(Y_{ji}, \gamma)$, the average shortfall below the target, with $\gamma = 1$ for human level. The *probability of improvement* of method A over method B is the average over tasks of $\mathbb{P}(Y_j^{\mathrm{A}} > Y_j^{\mathrm{B}}) + \frac{1}{2}\mathbb{P}(Y_j^{\mathrm{A}} = Y_j^{\mathrm{B}})$ for independent runs of the two methods, estimated on each task by the Mann–Whitney statistic (Proposition A.16). Behind all of them stands the *score distribution* $\hat{F}(t) = \frac{1}{nJ}\sum_{j,i} \mathbf{1}\{Y_{ji} > t\}$, the fraction of runs above t, an unbiased estimate of $\frac{1}{J}\sum_j \mathbb{P}(Y_j > t)$; plotted against t it is a performance profile in the sense of Dolan and Moré (2002).

Each aggregate ignores something. The median sees the magnitude of the scores only on the one or two tasks in the middle: when it is positive, lowering the scores of every task below the middle, even to zero, leaves it unchanged. The mean is dominated by the tasks with the largest scores and the largest run-to-run variance; the interquartile mean ignores the extremes, which may include the tasks on which a method fails; the probability of improvement ignores the size of the improvement, and the optimality gap everything above its target. Each answers a different question, to be chosen before the scores are seen.

PRECISION: THE EFFECTIVE NUMBER OF TASKS. The median of J task means depends on all of them but moves with the run noise of only one or two: when the tasks are far apart, the task in the middle stays in the middle. The next theorem makes this exact for every aggregate that is a weighted sum of order statistics of the task means.

Theorem 3.4 (Effective number of tasks). *Let $X_j = \mu_j + e_j$ for $j = 1, \dots, J$, where $\mu_1 < \mu_2 < \cdots < \mu_J$ and the errors e_j are independent with mean 0 and variances s_j^2. Let $L(x) = \sum_{k=1}^{J} c_k x_{(k)}$, with weights $c_k \geq 0$ summing to 1, let $V = \sum_k c_k^2 s_k^2$, and let $\mathcal{A}$ be the event $X_1 < X_2 < \cdots < X_J$ that the*

errors do not reorder the tasks.

(i) On $\mathcal{A}$, $L(X)-L(\mu)=\sum_k c_k e_k$; everywhere, $|L(X)-L(\mu)|\le \max_j |e_j|$.

(ii) The root mean squared error $R=\big(\mathbb{E}(L(X)-L(\mu))^2\big)^{1/2}$ satisfies $|R-\sqrt{V}|\le 2\big(\mathbb{E}\big[\max_j e_j^2\,\mathbf{1}_{\mathcal{A}^c}\big]\big)^{1/2}$.

(iii) If the errors are Gaussian with $s_j\le s$ and consecutive means differ by at least $g>0$, then

$$|R-\sqrt{V}|\le 2s\left(3J(J-1)\,\Phi\big(-g/(s\sqrt{2})\big)\right)^{1/4}. \tag{3.2}$$

Proof. The plan is to compare $L(X)$ with the linear statistic $Z=\sum_k c_k e_k$, which it equals when the tasks keep their order, and to bound the difference on the event that they do not.

Part (i). On $\mathcal{A}$ the kth smallest of the X_j is X_k, so $L(X)=\sum_k c_k(\mu_k+e_k)=L(\mu)+Z$, where $L(\mu)=\sum_k c_k\mu_k$ because the μ_k are sorted. For the bound everywhere we use that order statistics are 1-Lipschitz in the maximum norm. Suppose $x_j\le y_j+\delta$ for every j. The k indices that carry the k smallest y_j carry k values x_j, each at most $y_{(k)}+\delta$, so $x_{(k)}\le y_{(k)}+\delta$; exchanging x and y gives $|x_{(k)}-y_{(k)}|\le\max_j|x_j-y_j|$. With $y=\mu$, and the weights nonnegative with sum 1,

$$|L(X)-L(\mu)|\le\sum_k c_k|X_{(k)}-\mu_k|\le\max_j|e_j|.$$

Part (ii). Let $D=L(X)-L(\mu)-Z$. By part (i), $D=0$ on $\mathcal{A}$. Since also $|Z|\le\sum_k c_k|e_k|\le\max_j|e_j|$, the same part gives $|D|\le 2\max_j|e_j|$ on $\mathcal{A}^c$. The errors are independent with mean 0, so $\mathbb{E}Z^2=V$. Minkowski's inequality for the norm $\big(\mathbb{E}(\cdot)^2\big)^{1/2}$ bounds $|R-\sqrt{V}|$ by $\big(\mathbb{E}D^2\big)^{1/2}$, which is at most $2\big(\mathbb{E}[\max_j e_j^2\,\mathbf{1}_{\mathcal{A}^c}]\big)^{1/2}$.

Part (iii). If $X_k<X_{k+1}$ for every k, then the X_j are in order, so $\mathcal{A}^c$ is contained in the union of the $J-1$ events $\{X_k\ge X_{k+1}\}$. Each says that $e_k-e_{k+1}\ge\mu_{k+1}-\mu_k\ge g$, and e_k-e_{k+1} is Gaussian with mean 0 and variance at most $2s^2$, so the union bound gives $\mathbb{P}(\mathcal{A}^c)\le(J-1)\Phi(-g/(s\sqrt{2}))$. Next we bound the expectation in part (ii) by the Cauchy–Schwarz inequality, then the maximum of the fourth powers by their sum, using $\mathbb{E}e_j^4=3s_j^4\le 3s^4$ for Gaussian errors:

$$\mathbb{E}\big[\max_j e_j^2\,\mathbf{1}_{\mathcal{A}^c}\big]\le\Big(\mathbb{E}\max_j e_j^4\Big)^{1/2}\mathbb{P}(\mathcal{A}^c)^{1/2}\le\big(3Js^4\big)^{1/2}\big((J-1)\Phi(-g/(s\sqrt{2}))\big)^{1/2}.$$

Inserting this bound in part (ii) gives (3.2). ∎

When the tasks are well separated, then, each such aggregate behaves like a weighted average of independent task errors; with equal run noise, $s_j=s$, its variance is that of an average of $J_{\mathrm{eff}}=1/\sum_k c_k^2$ independent tasks. The mean has $J_{\mathrm{eff}}=J$; the median has $J_{\mathrm{eff}}=1$ for an odd number of tasks and 2 for an even number; a 25% trimmed mean of the task means has

run distribution	median	interquartile mean	mean
normal	0.64	0.84	1
logistic	0.82	1.04	1
Laplace	2.00	1.63	1
Student's t, 3 degrees of freedom	1.62	1.97	1
normal, 5% contaminated with SD 10	3.45	4.48	1

Table 3.1. Asymptotic efficiency of the median and the interquartile mean relative to the mean, v_{mean}/v from Proposition 3.5, for one sample from five symmetric location families. Computed by numerical integration in `03-aggregates-reruns.py`.

$J_{\text{eff}} = J/2$ when J is a multiple of 4, and so does the interquartile mean of pooled runs when the scores of different tasks do not overlap. On Atari 100k the median carries the run noise of two games and the interquartile mean that of about thirteen. The bound (3.2) is loose: in a simulation with $J = 13$ Gaussian tasks (Lab 3.2), the median's error is already $0.995s$ when consecutive tasks are $g = 4s$ apart, and $0.891s$ at $g = 2s$.

The opposite extreme is a suite in which every task has the same run distribution. The pooled runs are then one sample from a location family, and the classical asymptotics apply.

Proposition 3.5 (One sample). *Let $Y_1, \dots, Y_N$ be independent with density $f(y-\theta)$, where f is symmetric about 0, has variance σ^2, and is continuous and positive at 0 and at its quartiles $\pm q$. As $N \to \infty$, the sample mean, the sample median and the 25% trimmed mean are asymptotically normal around θ, with variances v/N given by*

$$v_{\text{mean}} = \sigma^2, \qquad v_{\text{med}} = \frac{1}{4f(0)^2}, \qquad v_{\text{IQM}} = 4\Big(\int_{-q}^{q} y^2 f(y)\,\mathrm{d}y + \frac{q^2}{2}\Big).$$

Proof sketch. For the median, $\sqrt{N}(\text{med} - \theta) \le t$ holds exactly when at least half of the Y_i lie below $\theta + t/\sqrt{N}$; this count is binomial with success probability $\frac{1}{2} + f(0)t/\sqrt{N} + o(N^{-1/2})$, and the central limit theorem for the binomial gives the limit $\Phi(2f(0)t)$. For the trimmed mean, Stigler (1973) proves asymptotic normality with the variance of the influence function, here the winsorized observation $\max(-q, \min(Y-\theta, q))$ divided by $1 - 2\cdot\frac{1}{4}$; its second moment is the integral plus the two tails, each of mass $\frac{1}{4}$ at $\pm q$, which gives v_{IQM}. Quantiles and trimmed means are treated in general in van der Vaart (1998, Chapters 21–22). ∎

Table 3.1 gives the efficiencies. For normal runs the median wastes a third of the data and the interquartile mean a sixth; for heavy-tailed runs both beat the mean, and the interquartile mean is never far from the better of the other two. Between the two extremes, with normal run noise, the median's effective number of tasks lies between 1 and $2J/\pi \approx 0.64J$, and the interquartile mean's between about $J/2$ and $0.84J$. This is the case for the interquartile mean with few runs: it gives up little when the runs are well

behaved, much less than the median, and it does not let one task with wild runs dominate, as the mean does.

The sample median is also biased: it is an order statistic of noisy values, and its expectation moves away from the median of the true task means. Suppose two tasks are tied at μ, with $r-1$ tasks far below them and r far above, so that $J = 2r+1$ and the true median is μ. Then the sample median is the larger of the two tied estimates, with expectation $\mu + s\,e_2 = \mu + s/\sqrt{\pi}$, where e_2 is the expected maximum of two standard normals (Chapter 1); with the tied pair one place higher the bias is $-s/\sqrt{\pi}$. The bias shrinks with s as runs are added, and its sign depends on the tasks near the middle, which differ between methods: this is the bias that Agarwal et al. (2021) measured.

INTERVALS: THE STRATIFIED BOOTSTRAP. Suite-level uncertainty comes from the runs, so an interval should resample runs and keep the tasks. The *stratified bootstrap* draws n runs with replacement within each task, recomputes the aggregate, and repeats; the 2.5% and 97.5% quantiles of the recomputed values form the percentile interval (Definition A.2). For a difference between methods, both are resampled. With few runs the interval is too narrow, by a factor that is exact for the mean.

Proposition 3.6 (The bootstrap with n runs). *Given the scores, the stratified bootstrap variance of the mean aggregate equals $\frac{n-1}{n}$ times the unbiased estimate $\frac{1}{J^2}\sum_j S_j^2/n$ of its conditional variance (3.1), where S_j^2 is the sample variance of the n runs on task j.*

PROOF. Given the scores, the n resampled runs of task j are independent draws from the empirical distribution of its runs, which has mean $\bar{Y}_j$ and variance $\frac{1}{n}\sum_i (Y_{ji}-\bar{Y}_j)^2 = \frac{n-1}{n}S_j^2$. Their mean therefore has variance $\frac{n-1}{n}S_j^2/n$, the tasks are resampled independently, and the mean aggregate averages the J resampled task means. ■

With three runs the bootstrap standard error is $\sqrt{2/3} \approx 0.82$ times the usual one, and a percentile interval spans about $\pm 1.96 \times 0.82 = \pm 1.60$ standard errors, which covers 89% of the time for normal estimates, before counting the error of estimating each σ_j^2 from three runs; each task contributes at most ten distinct resampled means. Agarwal et al. (2021, Figure 6) found this undercoverage with three runs on Atari, and percentile intervals adequate from about ten runs per task. The median and the interquartile mean share the shortfall to first order. Widening the interval by $\sqrt{n/(n-1)}$ removes the part that Proposition 3.6 identifies, but not the rest.

3.5 The method: many comparisons

A suite multiplies comparisons: one per task, one per aggregate, one per baseline, and, hidden inside a protocol, one per evaluation or configuration over which a maximum is taken.

Proposition 3.7 (Many true nulls). *Suppose that m_0 of m tests concern true null hypotheses and that each of these rejects with probability α. Then the expected number of false rejections is $m_0\alpha$, and the probability of at least one is at most $m_0\alpha$; if the m_0 tests are independent, then it equals $1-(1-\alpha)^{m_0}$.*

PROOF. The number of false rejections is a sum of m_0 indicators with mean α each, so its expectation is $m_0\alpha$; the union bound gives the second statement. If the tests are independent, then the probability that none of them rejects is the product $(1-\alpha)^{m_0}$. ■

For a method no different from its baseline on each of 26 games, two-sided per-game tests at level 0.05 reject 1.3 times on average, and at least once with probability $1-0.95^{26}\approx 0.74$ when the games are independent. The *family-wise error rate*, the probability of at least one false rejection, is controlled by Bonferroni's correction, which tests each hypothesis at level α/m, and more powerfully by the step-down procedure of Holm (1979): order the p-values, compare the kth smallest with $\alpha/(m-k+1)$, and reject until the first comparison fails. Both are valid whatever the dependence between the tests (Proposition A.6 and Theorem A.7): with probability at least $1-\alpha$ every true null has p-value above α/m_0, and then Holm's procedure stops before the first true null. The *false discovery rate*, the expected proportion of false rejections among the rejections, is controlled at α by the step-up procedure of Benjamini and Hochberg (1995) for independent or positively dependent tests (Theorem A.8); under arbitrary dependence the level must be divided by $\sum_{k=1}^{m} 1/k$ (Benjamini and Yekutieli 2001). The choice follows the claim: "B is better on games X, Y and Z" is a list of discoveries, for which the false discovery rate fits; "B is never worse" is a claim about the whole family, which needs the family-wise rate.

TASKS AS THE SAMPLING UNITS. Demšar (2006) studied the same problem for classifiers compared over many data sets. Surveying papers at the International Conference on Machine Learning from 1999 to 2003, he found paired t-tests on each data set to be almost the only test in use, often summarized by counts of wins and losses. He recommended treating the data sets as the sampling units: for two methods, the Wilcoxon signed-rank test on the per-data-set differences (Wilcoxon 1945); for several, Friedman's test on the ranks of the methods within each data set (Friedman 1937), followed by Nemenyi's test for all pairs or Holm's procedure for comparisons with a control, drawn as a critical-difference diagram. These tests answer the population-level question of Proposition 3.3. A caution came later: the tests that follow Friedman's compare two methods through mean ranks that depend on the other methods in the comparison, so adding a third method can change whether the first two differ; pairwise signed-rank tests with a multiplicity correction avoid this (Benavoli et al. 2016).

HIDDEN MAXIMA. A protocol that reports the best of several evaluations makes a multiple comparison inside every run. The cost is easy to compute when the evaluations of one run are noisy measurements around a common value.

Proposition 3.8 (The best of k evaluations). *Let $X_i = \mu + \sigma\big(\sqrt{\rho}\,Z_0 + \sqrt{1-\rho}\,Z_i\big)$ for $i = 1, \dots, k$, with $Z_0, \dots, Z_k$ independent standard normal and $0 \le \rho \le 1$; the X_i have mean μ, standard deviation σ and pairwise correlation ρ. Then $\mathbb{E}\max_i X_i = \mu + \sigma\sqrt{1-\rho}\,e_k$.*

Proof. The shared term $\sigma\sqrt{\rho}\,Z_0$ is the same for every i, and $\sqrt{1-\rho} \ge 0$, so $\max_i X_i = \mu + \sigma\sqrt{\rho}\,Z_0 + \sigma\sqrt{1-\rho}\max_i Z_i$. Taking expectations, with $\mathbb{E}Z_0 = 0$ and $\mathbb{E}\max_i Z_i = e_k$, proves the claim. ∎

The maximum rewards the part of the evaluation noise that is not shared between evaluations: the episode-to-episode noise of a finite evaluation, and the fluctuation of the policy between checkpoints. With ten evaluations, $e_{10} \approx 1.54$, so the best evaluation of a run that has stopped improving lies about $1.54\sqrt{1-\rho}$ standard deviations above its typical evaluation; a run that is still changing also contributes its best moment of training, a form of early stopping on the evaluation data. A maximum over eight hyperparameter configurations per game, the protocol of SUNRISE, is the best of $k = 8$ trials of Chapter 4, with $e_8 \approx 1.42$. In each case the published comparison set a maximum against an average.

The garden of forking paths. Gelman and Loken (2013, 2014) pointed out that a researcher who would have analyzed the data differently had they come out differently has made multiple comparisons, even when only one analysis was run and nothing was hidden. A suite of tasks has many such paths, and papers on Atari 100k took different ones: the aggregate, the normalization and its reference scores, the tasks, the number of runs and which runs to keep, the evaluation protocol, the baselines to rerun. Simmons et al. (2011) showed in psychology how a few such choices, each defensible, inflate the rate of false positives far above its nominal level, and the preprint of Dehghani et al. (2021) showed for machine-learning benchmarks how the choice of tasks alone changes which model ranks first. The remedy is the one experimental sciences use: fix the analysis before the data arrive, report every aggregate a reader might want, and publish the runs so that others can take the paths not taken.

3.6 In the laboratory: a suite of twelve tasks

The miniature of Atari 100k is a suite of twelve tasks built from the digits, with a fixed budget of 400 optimizer steps per task and normalized scores in which a simple reference plays the human. Method A is the volume's standard MLP. Method B adds label smoothing, which in a pilot of 16 runs per task was ahead of A by the mean, the median and the interquartile mean, so that every aggregate has the same correct answer; method C adds dropout and serves to show how aggregates disagree.

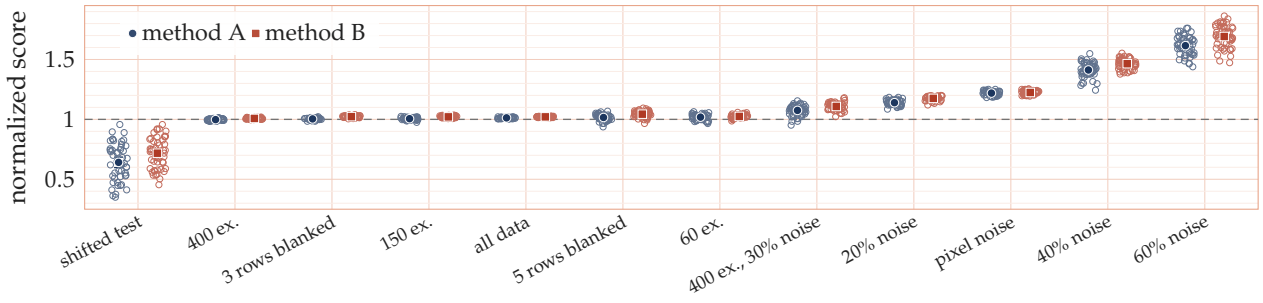


Figure 3.1. The suite of Lab 3.1: normalized scores of 50 runs of method A (ink, left of each task) and method B (vermilion, right) on each of the twelve tasks, ordered by A's mean (hollow marks, jittered horizontally), with the means and 95% t-intervals (filled marks; most intervals are shorter than the marks). Seeds 0–49 for A and 1000–1049 for B. The dashed line is the 1-nearest-neighbor reference.

LAB 3.1 · A SUITE OF TWELVE TASKS

SCRIPT
`03-aggregates-suite.py`

SETUP. Twelve tasks: the digits with all 1258 training examples or 400, 150 or 60 of them; with 20, 40 or 60% of the training labels replaced at random, or 30% on 400 examples; with Gaussian pixel noise; with the bottom 3 or 5 rows blanked; and with the test images shifted one pixel to the right. Each task gets 400 steps of SGD with momentum 0.9, learning rate 0.05 and batch 32. Method A is an MLP with two hidden layers of 64 units; B adds label smoothing 0.1; C adds dropout 0.2. A run's score is (accuracy − 0.1)/(reference − 0.1), the reference being the test accuracy of a 1-nearest-neighbor classifier on the same training set. Fifty runs per task and method (seeds $1000m + i$ for method m), 1800 runs in two to three minutes, each also evaluated every 40 steps for a protocol that keeps the best evaluation. Stratified bootstrap intervals (5000 resamples) from the first 3 runs of each task and from all 50.

PREDICT FIRST. B is ahead of A on every task. With three runs per task, which aggregate gives the narrowest interval for B − A? Scored by its best of ten evaluations, does A overtake B?

LOOK FOR. The spread of the runs on each task against the gap between A and B; how far each aggregate moves between 3 and 50 runs; the aggregate by which C beats A.

Figure 3.1 shows the suite. On every task B's mean is above A's, by between 0.005 (pixel noise) and 0.076 (the shifted test set and 60% label noise). The tasks differ in scale and noise as Atari games do. Without label noise, apart from the shifted test set, the scores lie between 1.00 and 1.22, with run standard deviations between 0.005 and 0.026; with label noise, where the reference copies the wrong labels, they reach 1.1 to 1.7; on the shifted test set A scores 0.641 ± 0.044 (mean and 95% t-interval over 50 runs), with a run standard deviation of 0.155. That one task contributes 62% of the run-noise variance of A's mean aggregate, and the two noisiest

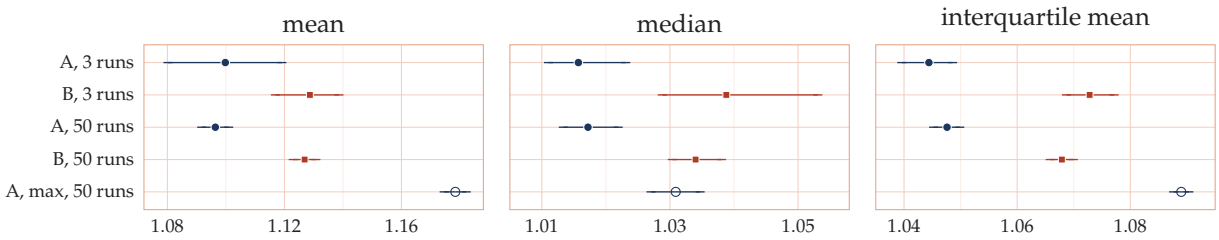


Figure 3.2. Aggregates over the twelve tasks of Lab 3.1, with 95% stratified bootstrap percentile intervals (5000 resamples), from the first 3 runs of each task and from all 50: method A (ink discs), method B (vermilion squares), and method A scored by its best of ten evaluations during training (hollow ink circle). Each panel has its own horizontal scale.

label-noise tasks another 27%.

Figure 3.2 gives the aggregates. With all 50 runs, B − A is +0.0305 by the mean, with 95% stratified bootstrap interval [0.0245, 0.0363], +0.0168 [0.0109, 0.0224] by the median and +0.0203 [0.0172, 0.0235] by the interquartile mean. From the first three runs of each task the same differences are +0.0288 [0.0064, 0.0515], +0.0231 [0.0090, 0.0354] and +0.0285 [0.0225, 0.0352]: every interval excludes zero, but the three-run interval of the interquartile mean is narrow and misses the fifty-run value, as Proposition 3.6 warns. The probability of improvement of B over A is 0.770 [0.744, 0.796] with 50 runs: on a typical task a run of B beats a run of A three times in four.

Scored by the best of its ten evaluations, A gains 0.082 by the mean, 0.014 by the median and 0.041 by the interquartile mean. The gains come from the label-noise tasks, +0.25 on 400 examples with 30% noise and +0.37 with 60%, where test accuracy peaks early and falls as the network fits the wrong labels. Against B scored at the end of training, this A "wins" by the mean, +0.0515 [0.0460, 0.0570], and by the interquartile mean, +0.0211 [0.0187, 0.0236], and ties by the median, −0.0032 [−0.0087, 0.0021]; scored the same way, B is ahead again by every aggregate. This is the comparison of CURL with DER in miniature: the protocol moved the result by more than the method did.

With dropout as the new method, the aggregates disagree even with 50 runs. C − A is +0.0384 [0.0320, 0.0445] by the mean and +0.0145 [0.0110, 0.0178] by the interquartile mean, but −0.0053 [−0.0106, −0.0001] by the median: dropout helps on the label-noise tasks, by up to 0.22, and slightly hurts on the three tasks in the middle of the suite, by 0.004 to 0.009, and C is ahead on 7 of the 12 tasks. Treating the tasks as the sampling units gives the population-level view of Proposition 3.3: the per-task differences B − A have a standard deviation of 0.026 across tasks, a task-level standard error of 0.0074 for their mean, and all twelve are positive (Wilcoxon signed-rank test, $p = 0.0005$), whereas for C − A the standard deviation is 0.071 and $p = 0.23$, so on tasks like these an advantage of C is not established. Friedman's test on the three methods gives $p = 0.009$, with average ranks 2.58, 1.33 and 2.08 for A, B and C, and only B and A differ by more than

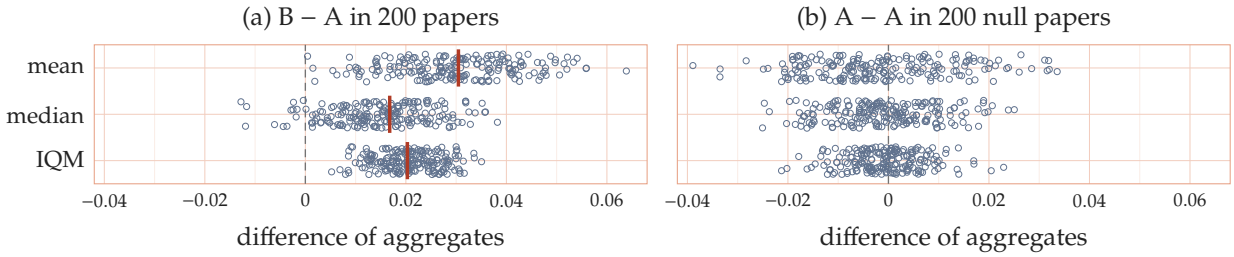


Figure 3.3. Replications of a three-run paper (Lab 3.2). (a) B − A by the mean, the median and the interquartile mean in 200 papers with 3 runs per task and method, drawn without replacement from the 50 runs of Lab 3.1 (seed 2026); hollow marks are papers, jittered vertically, and the vermilion bars are the differences computed from all 50 runs. (b) The same aggregates for 200 null papers that compare two disjoint sets of 3 runs of A.

Nemenyi's critical difference, 0.96 at level 0.05.

LAB 3.2 · TWO HUNDRED THREE-RUN PAPERS

SCRIPT
`03-aggregates-reruns.py`

SETUP. From the 50 runs of Lab 3.1, 200 papers, each with 3 runs per task of A and 3 of B drawn without replacement (seed 2026), and 200 null papers comparing two disjoint sets of 3 runs of A. For each paper: B − A by the mean, the median and the interquartile mean; 95% stratified bootstrap intervals (1000 resamples); Welch's test on each task, with Holm's and the Benjamini–Hochberg adjustments. The papers share runs of a finite bank, which slightly understates their spread. The script also computes Table 3.1 and the simulation after Theorem 3.4, in under a second once the bank exists.

PREDICT FIRST. In how many of the 200 papers does the median name A the winner? In how many null papers does at least one task differ at level 0.05 by Welch's test?

LOOK FOR. The spread of B − A across papers for each aggregate against its fifty-run value; the same spread in the null papers; the coverage of the three-run intervals; what Holm's procedure does to the per-task "wins".

Figure 3.3 shows the 200 papers. The standard deviation of B − A across papers is 0.0097 for the median, 0.0118 for the mean and 0.0060 for the interquartile mean, against fifty-run differences of 0.0168, 0.0305 and 0.0203. The median names A the winner in 12 papers (6.0%); the mean and the interquartile mean never do. The ratio of gap to spread decides it: 3.4 for the interquartile mean, 2.6 for the mean, 1.7 for the median. A paper's interval for B − A excludes zero in 95% of the papers for the interquartile mean, 78.5% for the mean, and 53% for the median: half of the papers that report the median with an honest interval cannot claim the improvement that is there.

The null papers show what three runs produce with nothing to find. The middle 95% of A − A runs from −0.019 to +0.019 for the median, as large as B's real gain by that aggregate, and from −0.015 to +0.014 for the

interquartile mean. Three-run intervals cover the fifty-run values in 82% to 89% of the papers for the mean, 77.5% to 82% for the median and 87% to 88% for the interquartile mean, against a nominal 95%, and the intervals for A − A exclude zero in 18%, 14% and 12% of the null papers. The average three-run median of A is 1.0208, above its fifty-run median 1.0172 by 0.0036: the bias of the sample median, here from the nearly tied tasks in the middle of the suite.

The spread of A's aggregates over the papers is 0.0093 for the mean, exactly the value of Proposition 3.3, 0.0068 for the median and 0.0048 for the interquartile mean, below the separated-task predictions of Theorem 3.4, 0.0098 and 0.0058, because the middle tasks of this suite lie closer together than their run noise. The mean is the noisiest of the three, although it uses every run, because one task with wild runs carries most of its variance.

Per-task tests show multiplicity directly. In the null papers, 33.5% have at least one task that differs at level 0.05 by Welch's test, against $1-0.95^{12} = 46\%$ predicted by Proposition 3.7 for exact independent tests (with three runs per arm Welch's test is conservative); after Holm's or the Benjamini–Hochberg procedure, 2.5% do. For B against A, 92.5% of the papers find at least one task significantly in favor of B by the unadjusted Welch test at level 0.05, and 18.5% after Holm's procedure: with three runs per task, honest per-task claims have little power. Among the 495 subsets of 8 of the 12 tasks, with all 50 runs, the verdict on C against A reverses for 42% of the subsets by the median, 18% by the interquartile mean and none by the mean; B, ahead on every task, is ahead on every subset.

The miniature reproduces the anatomy of the Atari findings: wide spreads of aggregates from three runs, a median less precise than the interquartile mean and biased, intervals that undercover, a maximum that outweighs a real method effect, and aggregates that disagree. It does not reproduce their size. The median picked the wrong winner in 6% of our papers because B's gain is 1.7 spreads of the three-run median, while on Atari 100k the median needed 50 to 100 runs before its intervals became small; Atari runs are noisier relative to the gaps between agents, and a noisier suite would come closer (Exercise 3.7).

3.7 Protocol

PROTOCOL 3.1 · REPORTING RESULTS OVER A SUITE OF TASKS

- ☐ Before running, write down the suite, the score normalization and its reference scores, the evaluation protocol, and one primary aggregate with its interval; say whether the claim is about these tasks or about tasks like them.
- ☐ Rerun every baseline under the same protocol, budget and code base as the new method; never set your runs against numbers copied from papers.
- ☐ Use at least ten runs per task when the budget allows; with fewer,

say so, and widen bootstrap intervals by $\sqrt{n/(n-1)}$.

- ☐ Report the primary aggregate of each method, and of the difference between methods, with a 95% stratified bootstrap interval; report the interquartile mean, the median and the mean together, with the performance profile.
- ☐ For a claim about tasks like these, add a test across tasks: the signed-rank test for two methods, Friedman's test with Holm's procedure against a control for several.
- ☐ Treat per-task claims as a family: Holm's procedure for claims of wins, the Benjamini–Hochberg procedure for lists of discoveries, or the probability of improvement with its interval instead.
- ☐ Never report a maximum over evaluations, configurations or seeds without the number it was taken over, and report the score at the end of training beside it.
- ☐ Publish the score of every run.

The protocol does not protect against a suite chosen, as the 26 games were, with the help of the methods it compares, nor against the slow overfitting of a community that reuses one suite for years; it does not make reimplemented baselines faithful (Chapter 4); and its intervals are conditional on the tasks. Its cost is runs, often more than a project can afford. Then spend the runs where the aggregate is sensitive, on the tasks near the middle for the median and the interquartile mean, rerun the strongest baseline rather than every one, report wider intervals rather than none, and say which claims the budget could not support.

Exercises

3.1 CONSOLIDATION. (Bonferroni and Šidák.) (a) Prove that testing each of m hypotheses at level α/m controls the family-wise error rate at α. (b) Show that for independent tests the level $1-(1-\alpha)^{1/m}$ gives a family-wise error rate of exactly α when every null is true (Šidák 1967), and that it exceeds α/m.

3.2 CONSOLIDATION. (Three procedures on one list.) Twelve per-task tests give the p-values 0.001, 0.004, 0.006, 0.012, 0.019, 0.030, 0.041, 0.060, 0.20, 0.35, 0.60 and 0.90. At $\alpha = 0.05$, find the hypotheses rejected by Bonferroni's correction, by Holm's procedure and by the Benjamini–Hochberg procedure, and say which claim each set of rejections supports.

3.3 CONSOLIDATION. (The median of tied tasks.) (a) Three tasks have means $\mu_1 = \mu_2 = 0$ and $\mu_3 = M$, and independent $\mathcal{N}(0, s^2)$ errors. Show that as $M \to \infty$ the sample median has expectation tending to $s/\sqrt{\pi}$ and variance tending to $s^2(1-1/\pi)$. (b) Use Proposition 3.5

to show that when all J tasks are tied, J times the variance of the median of the task means tends to $\pi s^2/2$. (c) For $J = 26$, with `scipy`'s convention of trimming $\lfloor J/4 \rfloor$ values at each end, find the effective number of tasks of the trimmed mean of well-separated task means.

3.4 CONSOLIDATION. (Interval coverage with few runs.) (a) Use Proposition 3.6 to show that, for normal task means with known variances, the percentile interval of the mean aggregate covers with probability $2\Phi\big(1.96\sqrt{(n-1)/n}\big) - 1$, and evaluate this for $n = 3, 5, 10$. (b) Why is the coverage in Lab 3.2 lower still?

3.5 LAB. (The aggregate decides.) In Lab 3.1, method C is ahead of A by the mean and the interquartile mean and behind by the median. Explain this from the per-task differences C – A that the script prints and the weights of each aggregate. Which aggregate would you have pre-registered for a claim about dropout?

3.6 LAB. (More runs per paper.) Rerun Lab 3.2 with 5 and with 10 runs per paper. Record the wrong-sign rates, how often the interval for B – A excludes zero, and the coverage, and compare the coverage with Exercise 3.4.

3.7 LAB. (A noisier suite.) Halve the budget of every task, or raise its label noise, rerun both labs, and measure how often the median names the wrong winner; relate the change to the ratio of gap to spread of each aggregate.

3.8 DESIGN. (Twenty-six games, three hundred runs.) A new agent is to be compared with two published baselines on the 26 games of Atari 100k, with compute for 300 runs in all. Decide the runs per game of each method, which baselines to rerun, the evaluation protocol, the primary aggregate and its interval, and the multiplicity adjustment, and say which claims the design can support.

3.9 DESIGN. (Four methods, twelve data sets.) Pre-register the analysis for comparing a new method with three baselines on 12 data sets with 5 runs each: the primary aggregate, the comparisons and their adjustment, and the test across data sets. State in advance what result would count as the new method being better.

3.10 CRITIQUE. (A bold column.) An invented paper bolds the best of five agents on each of 26 games, with baseline numbers taken from their papers, and claims to be "best on 17 of 26 games", a new state of the art in median score (0.312 against 0.298), and "significantly better on 9 games (Welch's t-test, $p < 0.05$, 3 seeds)". Audit the three claims.

3.11 CRITIQUE. (The best checkpoint.) An invented methods section reads: "We evaluate every 10k steps on 10 episodes and report, for each seed, the best evaluation; results are averaged over 5 seeds. Baseline results are those reported by the original authors." Use Proposition 3.8 to say

what the reported numbers estimate, and rewrite the paragraph so that the comparison is fair.

3.12 RESEARCH. (Aggregates robust to the choice of tasks.) Zhang and Hardt (2024) show for multi-task benchmarks a trade-off: the more the tasks disagree about the ranking of models, the more an aggregate ranking moves under irrelevant changes. Open: an aggregate whose interval accounts for both the runs and the selection of tasks, with a stability property under adding or removing tasks that it provably satisfies.

3.13 RESEARCH. (Three runs.) With three runs, each task's variance rests on two degrees of freedom. Open: intervals for the interquartile mean with nearly nominal coverage at $n = 3$, pooling variance information across tasks, for instance through a hierarchical model of the σ_j^2, with guarantees that survive skewed, heterogeneous run distributions like those of Figure 3.1.

Notes and References

Comparisons over many data sets were studied in machine learning before deep reinforcement learning made them urgent: Demšar (2006) recommended rank-based tests across data sets, and Benavoli et al. (2016) questioned the post hoc tests based on mean ranks. Performance profiles come from the benchmarking of optimization software (Dolan and Moré 2002), where they plot, for each solver, the fraction of problems solved within a factor of the best solver's time.

In deep reinforcement learning, Henderson et al. (2018) documented how seeds, hyperparameters, code bases and environments move reported results, and Colas et al. (2018, 2019) turned the lesson into tutorials on power analysis and the choice of test. Jordan et al. (2020) aggregated across environments through performance percentiles, with intervals of guaranteed coverage. Agarwal et al. (2021) proposed the stratified bootstrap, the interquartile mean and the score distribution as default tools and released them in the library rliable, whose interquartile mean is `scipy`'s 25% trimmed mean of all runs pooled over tasks and whose intervals are percentile intervals by default.

The statistics are older: Efron and Tibshirani (1993) is the standard introduction to the bootstrap, the asymptotics of trimmed means are due to Stigler (1973), and the statistic behind the probability of improvement to Mann and Whitney (1947). Šidák (1967) and Holm (1979) sharpened Bonferroni's correction, and Benjamini and Hochberg (1995) and Benjamini and Yekutieli (2001) replaced the family-wise error rate by the false discovery rate. The garden of forking paths is the phrase of Gelman and Loken (2013, 2014), and Dehghani et al. (2021) and Zhang and Hardt (2024) studied how the choice and number of tasks shape the rankings of machine-learning benchmarks.

TRANSLATING THE LITERATURE. Agarwal et al. (2021) write M tasks and N runs for our J and n, and $x_{m,n}$ for our Y_{ji}; their "score distribution" $\hat{F}_X(\tau)$ is our $\hat{F}(t)$, and their "true median" is our median of the μ_j. Demšar (2006) writes N data sets and k classifiers for our J tasks and number of methods, and his critical difference $q_\alpha\sqrt{k(k+1)/(6N)}$ is the one used in Lab 3.1. Holm (1979) writes n for our number of hypotheses m, and Benjamini and Hochberg (1995) write q^* for the level α at which we control the false discovery rate.

References

Agarwal, R., M. Schwarzer, P. S. Castro, A. Courville, and M. G. Bellemare (2021). "Deep Reinforcement Learning at the Edge of the Statistical Precipice". In: *Advances in Neural Information Processing Systems 34 (NeurIPS 2021)*, pp. 29304–29320.

Bellemare, M. G., Y. Naddaf, J. Veness, and M. Bowling (2013). "The Arcade Learning Environment: An Evaluation Platform for General Agents". In: *Journal of Artificial Intelligence Research* 47, pp. 253–279.

Benavoli, A., G. Corani, and F. Mangili (2016). "Should We Really Use Post-Hoc Tests Based on Mean-Ranks?" In: *Journal of Machine Learning Research* 17.5, pp. 1–10.

Benjamini, Y. and Y. Hochberg (1995). "Controlling the False Discovery Rate: A Practical and Powerful Approach to Multiple Testing". In: *Journal of the Royal Statistical Society: Series B (Methodological)* 57.1, pp. 289–300.

Benjamini, Y. and D. Yekutieli (2001). "The Control of the False Discovery Rate in Multiple Testing under Dependency". In: *The Annals of Statistics* 29.4, pp. 1165–1188.

Colas, C., O. Sigaud, and P.-Y. Oudeyer (2018). *How Many Random Seeds? Statistical Power Analysis in Deep Reinforcement Learning Experiments*. arXiv:1806.08295.

Colas, C., O. Sigaud, and P.-Y. Oudeyer (2019). "A Hitchhiker's Guide to Statistical Comparisons of Reinforcement Learning Algorithms". In: *ICLR 2019 Workshop on Reproducibility in Machine Learning*. arXiv:1904.06979.

Dehghani, M., Y. Tay, A. A. Gritsenko, Z. Zhao, N. Houlsby, F. Diaz, D. Metzler, and O. Vinyals (2021). *The Benchmark Lottery*. arXiv:2107.07002.

Demšar, J. (2006). "Statistical Comparisons of Classifiers over Multiple Data Sets". In: *Journal of Machine Learning Research* 7, pp. 1–30.

Dolan, E. D. and J. J. Moré (2002). "Benchmarking Optimization Software with Performance Profiles". In: *Mathematical Programming* 91.2, pp. 201–213.

Efron, B. and R. J. Tibshirani (1993). *An Introduction to the Bootstrap*. Monographs on Statistics and Applied Probability 57. New York: Chapman & Hall.

Friedman, M. (1937). "The Use of Ranks to Avoid the Assumption of Normality Implicit in the Analysis of Variance". In: *Journal of the American Statistical Association* 32.200, pp. 675–701.

Gelman, A. and E. Loken (2013). "The Garden of Forking Paths: Why Multiple Comparisons Can Be a Problem, Even When There Is No "Fishing Expedition" or "p-Hacking" and the Research Hypothesis Was Posited Ahead of Time". Unpublished manuscript, 14 November 2013.

Gelman, A. and E. Loken (2014). "The Statistical Crisis in Science". In: *American Scientist* 102.6, pp. 460–465.

Henderson, P., R. Islam, P. Bachman, J. Pineau, D. Precup, and D. Meger (2018). "Deep Reinforcement Learning That Matters". In: *Proceedings of the AAAI Conference on Artificial Intelligence*. Vol. 32. 1, pp. 3207–3214.

Hessel, M. et al. (2018). "Rainbow: Combining Improvements in Deep Reinforcement Learning". In: *Proceedings of the AAAI Conference on Artificial Intelligence*. Vol. 32. 1, pp. 3215–3222.

Holm, S. (1979). "A Simple Sequentially Rejective Multiple Test Procedure". In: *Scandinavian Journal of Statistics* 6.2, pp. 65–70.

Jordan, S. M., Y. Chandak, D. Cohen, M. Zhang, and P. S. Thomas (2020). "Evaluating the Performance of Reinforcement Learning Algorithms". In: *Proceedings of the 37th International Conference on Machine Learning*. Vol. 119. Proceedings of Machine Learning Research, pp. 4962–4973.

Kaiser, Ł. et al. (2020). "Model Based Reinforcement Learning for Atari". In: *International Conference on Learning Representations (ICLR)*.

Kielak, K. (2020). *Importance of Using Appropriate Baselines for Evaluation of Data-Efficiency in Deep Reinforcement Learning for Atari*. arXiv:2003.10181.

Laskin, M., A. Srinivas, and P. Abbeel (2020). "CURL: Contrastive Unsupervised Representations for Reinforcement Learning". In: *Proceedings of the 37th International Conference on Machine Learning*. Vol. 119. Proceedings of Machine Learning Research, pp. 5639–5650.

Lee, K., M. Laskin, A. Srinivas, and P. Abbeel (2021). "SUNRISE: A Simple Unified Framework for Ensemble Learning in Deep Reinforcement Learning". In: *Proceedings of the 38th International Conference on Machine Learning*. Vol. 139. Proceedings of Machine Learning Research, pp. 6131–6141.

Mann, H. B. and D. R. Whitney (1947). "On a Test of Whether One of Two Random Variables Is Stochastically Larger than the Other". In: *The Annals of Mathematical Statistics* 18.1, pp. 50–60.

Mnih, V. et al. (2015). "Human-Level Control through Deep Reinforcement Learning". In: *Nature* 518.7540, pp. 529–533.

Schwarzer, M., A. Anand, R. Goel, R. D. Hjelm, A. Courville, and P. Bachman (2021). "Data-Efficient Reinforcement Learning with Self-Predictive Representations". In: *International Conference on Learning Representations (ICLR)*.

Šidák, Z. (1967). "Rectangular Confidence Regions for the Means of Multivariate Normal Distributions". In: *Journal of the American Statistical Association* 62.318, pp. 626–633.

Simmons, J. P., L. D. Nelson, and U. Simonsohn (2011). "False-Positive Psychology: Undisclosed Flexibility in Data Collection and Analysis Allows Presenting Anything as Significant". In: *Psychological Science* 22.11, pp. 1359–1366.

Stigler, S. M. (1973). "The Asymptotic Distribution of the Trimmed Mean". In: *The Annals of Statistics* 1.3, pp. 472–477.

van der Vaart, A. W. (1998). *Asymptotic Statistics*. Cambridge Series in Statistical and Probabilistic Mathematics 3. Cambridge: Cambridge University Press.

van Hasselt, H., M. Hessel, and J. Aslanides (2019). "When to Use Parametric Models in Reinforcement Learning?" In: *Advances in Neural Information Processing Systems 32 (NeurIPS 2019)*, pp. 14322–14333.

Wilcoxon, F. (1945). "Individual Comparisons by Ranking Methods". In: *Biometrics Bulletin* 1.6, pp. 80–83.

Yarats, D., I. Kostrikov, and R. Fergus (2021). "Image Augmentation Is All You Need: Regularizing Deep Reinforcement Learning from Pixels". In: *International Conference on Learning Representations (ICLR)*.

Zhang, G. and M. Hardt (2024). "Inherent Trade-Offs between Diversity and Stability in Multi-Task Benchmarks". In: *Proceedings of the 41st International Conference on Machine Learning*. Vol. 235. Proceedings of Machine Learning Research, pp. 58984–59002.

Part II

Fair Comparison

A comparison of two methods is a comparison of two procedures, each with its tuning, its pipeline and its budget. An experiment is fair when its arms differ only in what the claim is about.

4 Tuning Budgets

How an unequal search decides a comparison

THE CLAIM *"Adaptive gradient methods generalize worse than SGD."* — *The received reading of Wilson et al. (2017)*

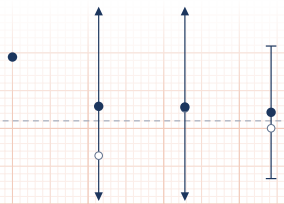
THE VERDICT Reversed: tuned over all their hyperparameters with equal budgets, adaptive methods match or beat SGD with momentum.

IN THIS CHAPTER. In 2017 Wilson and colleagues tuned the step size and its decay for five optimizers on four workloads and found that AdaGrad, RMSProp and Adam generalized worse than SGD with and without momentum; the field read this as a property of adaptive methods. Two years later Choi and colleagues reproduced the gap under the same protocol and removed it by also tuning ϵ and the momentum coefficient: with every hyperparameter searched, adaptive methods were never worse. The method of this chapter treats a training method as an update rule plus a search space plus a budget, compares methods by their budget curves, the expected result of a search of k trials, proves when such curves cross and when a larger search space contains a smaller one, and estimates the curves without bias. In the laboratory, on digits with distractor features, Adam loses to momentum on nine splits of ten under the 2017 protocol and ties it under the 2019 protocol with the same sixteen trials.

4.1 The claim: the marginal value of adaptive methods

By 2017 Adam had become a default choice in much of deep learning. Kingma and Ba (2015) had proposed it as a method that needs little tuning: it keeps exponential moving averages of the gradient and of its square, divides the first by the square root of the second, coordinate by coordinate, and so gives every parameter its own step size. They recommended defaults, a step size $\alpha = 10^{-3}$, decay rates $\beta_1 = 0.9$ and $\beta_2 = 0.999$, and a constant $\epsilon = 10^{-8}$ added to the denominator to keep it away from zero, and frameworks adopted such values as their defaults, with varying choices of ϵ. AdaGrad and RMSProp, the older *adaptive gradient methods*, work the same way with other averages.

Ashia Wilson, Rebecca Roelofs, Mitchell Stern, Nathan Srebro and Benjamin Recht asked what the adaptivity buys (Wilson et al. 2017). Their paper has two parts. The first is a constructed problem: least-squares classification in which the first feature equals the label, two features always equal one, and



every training example has features of its own that no other example shares. Gradient descent and SGD converge to the minimum-norm interpolating solution, which, under a mild condition on the class counts, classifies new points correctly; AdaGrad, RMSProp and Adam, under a condition on the signs of the gradients, converge to a solution proportional to the sign vector of $X^\top y$, which weights the label feature and the two constant features equally and labels every new point positive. When positive labels have probability p slightly above one half, its test error $1 - p$ is close to one half. The example is a theorem, and it stands: adaptive methods can find different solutions, and worse ones.

The second part is an experiment on four workloads: a VGG network on CIFAR-10, a character-level language model (a two-layer LSTM) on Tolstoy's *War and Peace*, and two LSTM parsers on the Penn Treebank, one discriminative and one generative. Five optimizers were compared: SGD, the *heavy ball* method, that is SGD with momentum 0.9, and the three adaptive methods. For each optimizer the authors tuned the initial step size and the schedule by which it decays, and left the other hyperparameters at their defaults.

> FINDING 4.1 (Wilson et al. 2017). With the initial step size and its decay tuned for each optimizer and all other hyperparameters at their defaults, SGD gave the best result on each of the four workloads, ahead of every adaptive method, although the adaptive methods sometimes fit the training set better. On CIFAR-10 SGD had a test error of 7.65% and the best adaptive method, RMSProp, 9.60% (means over five runs; standard deviations 0.14 and 0.19). On the discriminative parser the margin was small, a development F1 of 91.24 for SGD against 91.11 for Adam.

The authors drew two conclusions. Adaptive methods, contrary to their reputation, had needed as much tuning as SGD; and the paper's abstract suggests that practitioners should "reconsider the use of adaptive methods to train neural networks". The paper appeared at NeurIPS 2017, and its observation quickly became a premise. Keskar and Socher (2017) proposed starting with Adam and switching to SGD, citing Wilson et al. for the observation that adaptive methods generalize worse; Loshchilov and Hutter (2019) motivated decoupled weight decay (AdamW) by the same gap; AdaBound (Luo et al. 2019) bounded Adam's step sizes so that it would turn into SGD late in training; and a NeurIPS 2020 paper set out to explain theoretically why SGD generalizes better than Adam (Zhou et al. 2020). In this received reading the protocol had dropped out: adaptive methods generalize worse.

What the original work got right is substantial. The constructed example is correct. The measurements were careful for their time, with five runs per selected setting, means and standard deviations, and learning curves, and they reproduce: as we will see, the gap appears again when the protocol is repeated. The observation that adaptive methods need tuning was right too. And the protocol was a common one. Practitioners tuned the learning rate and its schedule; ϵ, there to keep a denominator away from zero, was seldom tuned, and β_1, β_2 were left at the values that every framework

shipped.[1] The question this chapter asks is what such a protocol can and cannot decide.

[1] Wilson et al. list momentum, ϵ and the initialization of the accumulators among the hyperparameters they could have tuned, and say that a full grid over them was infeasible.

4.2 The experiment: five optimizers, one knob

EXPERIMENTAL DESIGN · WILSON ET AL. 2017

QUESTION	Do adaptive methods find solutions that generalize worse than those of SGD?
UNITS	Training runs on four workloads: VGG on CIFAR-10, an LSTM language model on *War and Peace*, two LSTM parsers on the Penn Treebank.
TREATMENT	The optimizer: SGD, heavy ball (momentum 0.9), AdaGrad, RMSProp, Adam.
TUNED	The initial step size, on a logarithmic grid of five values extended when the best lay at an edge; the decay scheme.
HELD FIXED	Momentum, β_1, β_2, ϵ, accumulator initialization (framework defaults); model, batch size, epochs.
SELECTION	Best development score within the epoch budget; on CIFAR-10, lowest training loss.
RESPONSE	Test error, test loss, development F1, development perplexity.
REPLICATES	Five runs of each selected setting from random initializations.
ANALYSIS	Means and standard deviations over the five runs; learning curves.

Read as a measurement, the record is sound. Every optimizer received the same kind of search, about five step sizes for each decay scheme, the selected settings were rerun from fresh initializations, and the spread over five runs was reported. Under this protocol, on these workloads, the adaptive methods lost.

Read as evidence for the claim, the record has a gap in the row HELD FIXED. The hyperparameters left at their defaults are not neutral background shared by the arms: they are the ones that distinguish the arms. Momentum is the parameter that separates heavy ball from SGD; ϵ decides how adaptive Adam is, and β_1, β_2 set the memory of its two averages. With $\epsilon = 10^{-8}$, Adam divides by $\sqrt{\hat{v}} + \epsilon \approx \sqrt{\hat{v}}$, and every coordinate moves at about the same speed whatever the size of its gradient, the mechanism of the constructed example. With ϵ large compared with $\sqrt{\hat{v}}$, the denominator is nearly constant and Adam moves like momentum SGD with a rescaled learning rate. The experiment compared heavy ball at its tuned learning rate with Adam pinned to one end of its own range. The design could show that Adam with its default ϵ and β's generalizes worse than momentum on these workloads; it could not show that the adaptive update rule does.

A second gap is in what "the same search" meant. Each optimizer got about five trials per decay scheme, but a trial count measures effort, not coverage: five learning rates cover the hyperparameter that matters most for heavy ball, but only a one-dimensional slice of Adam's four-dimensional space. Equal numbers of trials over unequal spaces are an unequal search.[2]

[2] Bergstra and Bengio (2012) made a related point against grids: when only a few hyperparameters matter, a grid spends most of its trials varying the others.

The post-mortem takes up both.

4.3 The post-mortem: the knob that was not turned

Dami Choi, Christopher Shallue, Zachary Nado, Jaehoon Lee, Chris Maddison and George Dahl studied why published optimizer comparisons disagreed (Choi et al. 2019).[3] Their central tool is a definition. One optimizer *includes* another if, for every setting of the second's hyperparameters, some sequence of settings of the first produces trajectories that converge to the second's on the same training history. SGD is momentum with momentum zero, momentum is included in RMSProp and in Adam, and Nesterov's method is included in NAdam. For Adam the construction is the one sketched above: take ϵ large, scale the learning rate with it, and set β_1 to the momentum coefficient. Proposition 4.6 proves the version we need.

Inclusion makes a prediction. If a more general optimizer is tuned over all of its hyperparameters, with a search space that reaches the region where it imitates its special case, then with enough trials it should do at least as well as the special case. A comparison in which Adam loses to momentum is therefore a statement about the search, not about Adam.

[3] The paper has remained an arXiv preprint (first version October 2019, third version June 2020); we cite it as such.

EXPERIMENTAL DESIGN · CHOI ET AL. 2019

UNITS	Seven workloads, from a small CNN on Fashion-MNIST to ResNet-50 on ImageNet and a Transformer on LM1B, with Wilson et al.'s VGG on CIFAR-10 among them.
TREATMENT	SGD, momentum, Nesterov, RMSProp, Adam, NAdam.
TUNED	Every optimizer hyperparameter, ϵ, $1-\beta_1$ and $1-\beta_2$ included, on log scales; a linear learning-rate decay; weight decay or label smoothing on some workloads.
BUDGET	Quasi-random search, 10 to 100 feasible trials per optimizer depending on the workload.
SELECTION	Lowest validation error; the test error of that trial.
ANALYSIS	Bootstrap over trials for the result of tuning; five reruns in the replication of Wilson et al.

The numbers in this section are those reported by the papers. Choi et al. first repeated Wilson et al.'s VGG experiment with its original protocol, a grid over the learning rate only, and reproduced the gap: RMSProp and Adam had worse test error. Tuning momentum and ϵ as well made the four optimizers nearly identical; tuning only the learning-rate schedule also brought them within error bars of each other. Across all seven workloads, with every hyperparameter tuned, the adaptive methods never underperformed momentum or SGD within error bars, and on some they were better. The best values of ϵ spanned ten orders of magnitude across four workloads; the best ResNet-50 result, a validation accuracy of 77.1%, came from NAdam with $\epsilon = 9475$, a value so large that NAdam behaves almost like Nesterov momentum, its special case.[4] They concluded that the search space may be the single most important factor behind conflicting optimizer rankings, and that studies that fix $\epsilon = 10^{-8}$ owe their readers a

[4] Reported by Choi et al.; we could not check it independently. They compare it with a published momentum result of 76.5%, a number from another paper rather than a controlled arm.

justification.

Choi et al. were also candid about their own search. They chose each final search space after an initial round of search over a broader one, checked that the optimum lay away from the boundaries, published both spaces, and noted that the effort that went into choosing them cannot easily be quantified. A search space is itself the result of tuning, and the trial count does not show it.

Benchmarks that charge for tuning. If tuning must be counted, the result of a comparison depends on how much of it is allowed. Sivaprasad et al. (2020) made the budget an axis: for each optimizer they drew hyperparameters from a prior and reported the expected performance of the best of k random-search trials, as a function of k, on image, text and synthetic problems, most of them from the DeepOBS benchmark suite. Adam with only its learning rate tuned was the most reliable choice at every budget, and especially at small ones; tuning more of Adam's hyperparameters paid off only at larger budgets.[5] Schmidt et al. (2021) ran more than fifty thousand runs: fifteen optimizers, eight problems, four schedules, and budgets of one run at the default setting and of 25, 50 and 75 random-search trials. No optimizer dominated, Adam remained a strong contender, rankings moved with the budget and the search space, and trying several optimizers at their defaults did about as well as tuning one.

[5] Their priors were calibrated by fitting distributions to the good configurations of a broad random search on the same tasks, a further instance of a search space shaped by earlier tuning.

The same pattern elsewhere. Layer-wise adaptive optimizers were proposed for very large batches. Nado et al. (2021) (a preprint) tuned Nesterov momentum and Adam as carefully as the published LARS and LAMB pipelines and matched them: on ResNet-50 at batch size 32,768, Nesterov momentum reached a median validation accuracy of 75.97% over 50 runs, the same as LARS, and in BERT pretraining at the same batch size Adam reached a SQuAD F1 of 91.58 against LAMB's 91.48. In language-model pretraining, Wen et al. (2026) tuned AdamW and a set of recent optimizers by coordinate-wise sweeps over all their hyperparameters, at model sizes from 0.1 to 1.2 billion parameters: speedups over AdamW that other papers had claimed, 1.4 to 2×, shrank to at most about 1.4×, and to about 1.1× at 1.2 billion parameters. They also found that rankings read at intermediate checkpoints can flip by the end of training, because the learning-rate decay changes the curves late (Chapter 12 meets the same effect in scaling laws). The AlgoPerf benchmark (Dahl et al. 2023) turned these lessons into rules: an algorithm is scored by its time to reach a target on fixed hardware, and the tuning protocol is part of the submission, either a search space from which a fixed number of trials is drawn or no tuning at all. In its first competition (Kasimbeg et al. 2025), a Distributed Shampoo submission was about 28% faster than the NadamW baseline, by the geometric mean over eight workloads, under five trials per study.[6]

[6] The benchmark paper specified twenty trials per study; the competition used five, repeated in five studies, and scored the median.

The mechanism. Three features of the protocol combined, and the method of this chapter treats each. First, the knob that was held fixed was the one that decides how adaptive Adam is, so "Adam at its defaults" was compared with "momentum tuned". Second, equal numbers of trials

over spaces of different dimension and shape are unequal searches, and which method wins depends on the budget, in a way that can be computed (Proposition 4.3). Third, the result was reported as a property of the update rule, without the budget and the search space that produced it.

The post-mortem literature is not unanimous in its details. Nesterov momentum was a strong alternative on some problems of Schmidt et al. (2021), and SGD variants did best on two problems of Sivaprasad et al. (2020) at large budgets; theoretical work on why the two families find different solutions continues (Zhou et al. 2020); and Wilson et al.'s example remains a correct statement about what an untuned adaptive method can converge to. The verdict concerns the general claim.

VERDICT. Tuned over all of their hyperparameters with equal care, adaptive methods were never worse than momentum or SGD on the workloads of Choi et al., including the one on which the gap was first measured, and a benchmark that charges for tuning found Adam the cheaper method to tune. The gap was produced by the protocol: ϵ and the moment parameters were held at defaults, which fixes Adam in the corner of its search space where it differs most from momentum. Read as a statement about update rules, the received claim does not survive; where tuned methods still differ, the difference more often favors the adaptive ones.



4.4 The method: a method is a protocol

The first lesson is a change of unit. A comparison of training methods compares procedures, and the procedure includes its tuning.

Definition 4.2 (Trial, protocol, training method). A *trial* is one hyperparameter configuration h from a *search space* $\mathcal{H}$, trained with its own seeds and scored. A *tuning protocol* specifies $\mathcal{H}$; how the configurations of the trials are chosen, for instance independently from a sampling distribution ν on $\mathcal{H}$ (*random search*); a *budget*, a number k of trials or the compute they may use; a selection rule, usually the best validation score; and a reporting rule, usually the test score of the selected configuration retrained with fresh seeds. A *training method* is an update rule together with a tuning protocol.

Under random search the trials of a method are independent and identically distributed: trial j yields a selection score V_j, such as a negative validation error, and a reported score Y_j, such as a test accuracy. The result of a search of budget k is $M_k = Y_J$, where J is the trial among the first k with the largest V (ties broken at random). The *budget curve* of the method is $k \mapsto \mathbb{E}M_k$. When the selection score and the reported score coincide, M_k is the best of k draws, the order statistic of Chapter 1; for normal scores with mean μ and standard deviation σ, $\mathbb{E}M_k = \mu + \sigma e_k$ (Proposition A.9). We take scores to be maximized; for an error, apply the results to its negative.[7]

[7] The tuning playbook of Godbole et al. (2023) calls the hyperparameters that must be retuned for each arm of a comparison its nuisance hyperparameters.

The budget curve is the object a comparison should report. Its value at $k = 1$ is the average quality of the search space; its limit is the best the space contains; and its shape says how fast a search climbs from one to the other. Two methods can trade places along the way.

Proposition 4.3 (Budget curves). *Let the trial scores of two arms have distribution functions F_1 and F_2 with finite means, and let $M_k^{(a)}$ be the best of k independent trials of arm a. Then:*

(i) for all budgets $k_1, k_2 \geq 1$,

$$\mathbb{E}M_{k_1}^{(1)} - \mathbb{E}M_{k_2}^{(2)} = \int_{-\infty}^{\infty} \bigl(F_2(x)^{k_2} - F_1(x)^{k_1}\bigr)\,\mathrm{d}x; \tag{4.1}$$

(ii) for each arm, $k \mapsto \mathbb{E}M_k^{(a)}$ is nondecreasing and converges to the upper end $s_a = \sup\{x : F_a(x) < 1\}$ of the distribution, which may be $+\infty$;

(iii) if $s_1 > s_2$, then $\mathbb{E}M_k^{(1)} > \mathbb{E}M_k^{(2)}$ for every sufficiently large k. If in addition $\mathbb{E}M_1^{(1)} < \mathbb{E}M_1^{(2)}$, then the budget curves cross.

Proof. The plan is to write each expected maximum as an integral of its distribution function and then to let k grow.

Part (i). For a random variable Z with finite mean and distribution function G, Fubini's theorem applied to its positive and negative parts gives $\mathbb{E}Z = \int_0^\infty (1 - G(x))\,\mathrm{d}x - \int_{-\infty}^0 G(x)\,\mathrm{d}x$. The best of k independent draws from F is at most x exactly when every draw is, so its distribution function is F^k, and its mean is finite because $|M_k|$ is at most the sum of the k absolute values. Write the formula for $M_{k_1}^{(1)}$ and for $M_{k_2}^{(2)}$ and subtract: all four integrals are finite, and on both half-lines the integrands combine to $F_2^{k_2} - F_1^{k_1}$.

Part (ii). Realize the trials of an arm as one infinite independent sequence $Y_1, Y_2, \ldots$; then $M_k = \max_{j \leq k} Y_j$ is nondecreasing in k, and so is its mean. Let $M_\infty = \sup_j Y_j$. Every Y_j is at most s_a almost surely, so $M_\infty \leq s_a$; and for $x < s_a$, $\mathbb{P}(M_\infty \leq x) \leq \mathbb{P}(M_k \leq x) = F_a(x)^k$ for every k, which tends to 0 because $F_a(x) < 1$. Hence $M_\infty = s_a$ almost surely. The variables $M_k - Y_1$ are nonnegative and increase to $M_\infty - Y_1$, so by monotone convergence $\mathbb{E}M_k - \mathbb{E}Y_1$ increases to $s_a - \mathbb{E}Y_1$, where $\mathbb{E}Y_1$ is finite.

Part (iii). Since $s_2 < s_1$, s_2 is finite, and $M_k^{(2)} \leq s_2$ almost surely gives $\mathbb{E}M_k^{(2)} \leq s_2$ for every k. By part (ii) there is a k_0 with $\mathbb{E}M_{k_0}^{(1)} > s_2$, and by monotonicity $\mathbb{E}M_k^{(1)} > s_2 \geq \mathbb{E}M_k^{(2)}$ for every $k \geq k_0$. If arm 2 is ahead at $k = 1$, then the difference changes sign between $k = 1$ and k_0. ∎

The integrand in (4.1) explains which part of a method the budget sees. For large k, $F(x)^k$ is close to 0 unless $F(x)$ is within about $1/k$ of 1, so at budget k a method is judged by roughly its best $1/k$ fraction of configurations: its average at $k = 1$, its upper tail at large k.

Example 4.4 (Two curves that cross). Let every configuration of arm 2 reach an accuracy of $b = 0.93$, a robust method with a small search space. In arm

1, a fraction $p = 0.1$ of configurations reach $a = 0.95$ and the rest diverge to $c = 0.50$, a broad space with a better corner. The best of k trials of arm 1 is a unless all k trials diverge, so $\mathbb{E}M_k^{(1)} = a - (a - c)(1 - p)^k = 0.95 - 0.45 \cdot 0.9^k$, which is 0.545 at $k = 1$, 0.793 at $k = 10$ and first exceeds 0.93 at $k = 30$. An experiment with 29 trials per arm declares arm 2 the better method; one with 30 declares arm 1. Neither is wrong about its budget, and both are wrong if they omit it.

Corollary 4.5 (Unequal budgets). *Let both arms have the same distribution function F with finite mean, and let arm 1 be tuned with k_1 trials and arm 2 with $k_2 < k_1$. Then*

$$\mathbb{E}M_{k_1}^{(1)} - \mathbb{E}M_{k_2}^{(2)} = \int_{-\infty}^{\infty} F(x)^{k_2}\bigl(1 - F(x)^{k_1 - k_2}\bigr)\,\mathrm{d}x \;\geq\; 0,$$

with equality only if the scores are almost surely constant. For normal scores with standard deviation σ the difference is $\sigma(e_{k_1} - e_{k_2})$.

Proof. Take $F_1 = F_2 = F$ in (4.1) and factor F^{k_2} out of the integrand, which is nonnegative because $0 \leq F \leq 1$. The integral vanishes only if $F(x) \in \{0, 1\}$ for almost every x, which for a right-continuous distribution function means a point mass. For normal scores, $\mathbb{E}M_k = \mu + \sigma e_k$ for each arm. ∎

The corollary is the arithmetic of a common reporting practice. A new method tuned with $k_1 = 50$ trials and a baseline tuned with $k_2 = 5$ differ by $\sigma(e_{50} - e_5) = 1.09\sigma$ when they are the same method, and a baseline given a single trial loses by $\sigma e_{50} = 2.25\sigma$, where σ is the spread of trial scores over the search space. A difference of this size says nothing about the methods.

Inclusion

Wilson et al.'s protocol held Adam's ϵ at 10^{-8}. The next result shows what that excluded: at large ϵ, with a matched learning rate, Adam is momentum SGD. We state it for the update rules as PyTorch implements them, with the momentum coefficient written β_1, as in Adam, and operations on vectors taken coordinate by coordinate.

Proposition 4.6 (Adam contains momentum). *Fix a number of steps $T \geq 1$, a starting point $\theta_0 \in \mathbb{R}^d$, coefficients $\beta_1, \beta_2 \in [0, 1)$ and a learning rate $\eta > 0$. Let $g_1, \ldots, g_T \colon \mathbb{R}^d \to \mathbb{R}^d$ be the minibatch gradients of the T steps, an L2 term $\lambda\theta$ included if there is one. Momentum SGD sets $b_0 = 0$ and, for $t = 1, \ldots, T$, $b_t = \beta_1 b_{t-1} + g_t(\theta_{t-1})$ and $\theta_t = \theta_{t-1} - \eta b_t$. For $\epsilon > 0$, Adam with coefficients $\beta_1, \beta_2, \epsilon$ starts at $\theta_0^\epsilon = \theta_0$ with $m_0 = v_0 = 0$ and sets*

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t, \qquad v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2,$$

$$\theta_t^\epsilon = \theta_{t-1}^\epsilon - \alpha_t \frac{m_t / (1 - \beta_1^t)}{\sqrt{v_t / (1 - \beta_2^t)} + \epsilon},$$

with $g_t = g_t(\theta^\epsilon_{t-1})$ and step sizes

$$\alpha_t = \frac{\epsilon\,\eta\,(1-\beta_1^t)}{1-\beta_1}. \tag{4.2}$$

If each g_t is continuous at the momentum iterate θ_{t-1}, then $\theta^\epsilon_t \to \theta_t$ as $\epsilon \to \infty$, for $t = 1, \dots, T$.

PROOF. The plan is an induction on t with a stronger hypothesis: as $\epsilon \to \infty$, $\theta^\epsilon_t \to \theta_t$, $m_t \to (1-\beta_1)b_t$, and v_t converges to a finite limit $\bar v_t$. At $t = 0$ all three hold, with $\bar v_0 = 0$. Suppose they hold at $t-1$. By the continuity hypothesis, $g_t(\theta^\epsilon_{t-1}) \to g_t(\theta_{t-1})$, so

$$m_t \to \beta_1(1-\beta_1)b_{t-1} + (1-\beta_1)g_t(\theta_{t-1}) = (1-\beta_1)b_t,$$

and $v_t \to \bar v_t = \beta_2 \bar v_{t-1} + (1-\beta_2)g_t(\theta_{t-1})^2$. With the step size (4.2) the bias correction $1-\beta_1^t$ cancels, and Adam's step becomes

$$\frac{\eta}{1-\beta_1}\, m_t \cdot \frac{1}{1 + \sqrt{v_t/(1-\beta_2^t)}\,/\epsilon}.$$

The first factor tends to ηb_t. In the second, $\sqrt{v_t/(1-\beta_2^t)}$ converges to the finite limit $\sqrt{\bar v_t/(1-\beta_2^t)}$, so its ratio to ϵ tends to 0 and the factor tends to 1. Hence $\theta^\epsilon_t \to \theta_{t-1} - \eta b_t = \theta_t$. ∎

For a ReLU network the continuity hypothesis can fail only if some unit sits exactly at its kink, for some example, at a momentum iterate, which one does not expect under a continuous random initialization. Two remarks qualify the result. With a constant step size $\alpha = \epsilon\eta/(1-\beta_1)$ instead of (4.2), the same argument shows that Adam tends to momentum SGD with the learning rate $\eta/(1-\beta_1^t)$ at step t, which starts ten times too large (for $\beta_1 = 0.9$) and settles within a few dozen steps. Adam's bias correction acts as a schedule. And the inclusion is a limit: a search space whose ϵ stops at 10^{-8} never reaches it, and one that reaches it may still spend most of its trials elsewhere. Inclusion says what a search over Adam's space can find; Proposition 4.3 says how many trials finding it costs.

Figure 4.1 shows the limit in a real network. With the matched step sizes, the distance between the final weights falls from 0.20 at $\epsilon = 10^{-2}$ to 3.9×10^{-3} at $\epsilon = 10^2$ and 6.7×10^{-11} at $\epsilon = 10^8$, by exactly a factor of 100 over the last factor of 100 in ϵ, the rate of Exercise 4.4; from $\epsilon = 10^2$ on, Adam's test error equals momentum's. With a constant step size the distance stops at 0.12, and the test error settles at 16.12%: a different run of about the same quality.

THE UNIT OF BUDGET. Counting trials is fair only when trials cost the same. If a trial of method 1 costs c_1 and one of method 2 costs c_2, a comparison at total compute C sets $k_a = C/c_a$ and compares $\mathbb{E}M^{(1)}_{C/c_1}$ with $\mathbb{E}M^{(2)}_{C/c_2}$, which by (4.1) can favor the cheaper method even when the costlier one is better

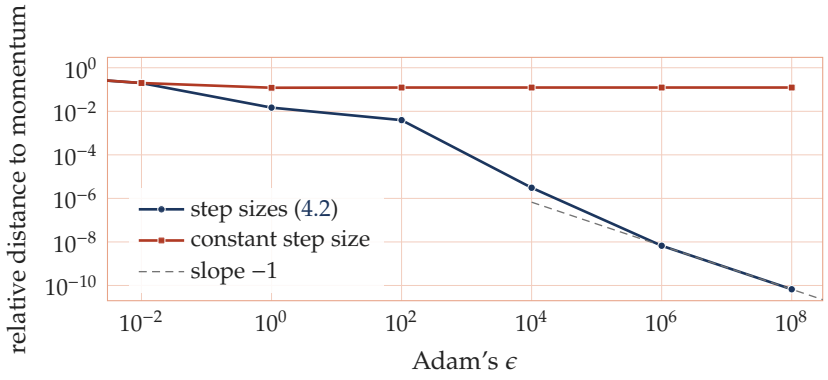


Figure 4.1. A numerical check of Proposition 4.6 (`04-budgets-inclusion.py`). Momentum SGD ($\eta = 0.03$, $\beta_1 = 0.9$) and Adam ($\beta_2 = 0.999$) are trained from the same initial weights on the same minibatches for 900 steps, on split 0 of the task of Lab 4.1, in double precision; the vertical axis is $\|\theta_T^\epsilon - \theta_T\|/\|\theta_T\|$ for a single run of each, so there are no intervals. With the step sizes (4.2) the distance falls to 7×10^{-11}, and from $\epsilon = 10^2$ on the test error equals momentum's 16.46%; with a constant step size the bias correction leaves a distance of 0.12.

trial for trial. The time to reach a target on fixed hardware, the measure of AlgoPerf (Dahl et al. 2023), folds the cost per step, the number of steps and the tuning protocol into one number; its external-tuning rules fix the number of trials and let the submitter choose the search space, and its self-tuning rules allow no external search and give a longer time limit. Which unit is right depends on the claim: "better per trial", "better per GPU-hour" and "better for a practitioner with an afternoon" are three different claims, and Chapter 7 shows how the choice of what to hold fixed decides another famous comparison. The design of the search space belongs in the budget too, although it is rarely counted.

4.5 The method: estimating a budget curve

A budget curve is an expectation over searches, and running many searches is expensive. One pool of trials can stand in for all of them. Suppose we have run K independent trials of a method and want $\mathbb{E}M_k$ for every $k \le K$. Order the pool by its selection scores, $V_{(1)} < \cdots < V_{(K)}$, and let $Y_{[i]}$ be the reported score of the trial with the ith smallest selection score, its *concomitant*.

Theorem 4.7 (An unbiased budget curve). *Let $(V_1, Y_1), \ldots, (V_K, Y_K)$ be independent and identically distributed, with $\mathbb{E}|Y_1| < \infty$ and the V_j almost surely distinct. For $1 \le k \le K$, the statistic*

$$\hat{M}_k = \sum_{i=k}^{K} \frac{\binom{i-1}{k-1}}{\binom{K}{k}} Y_{[i]} \tag{4.3}$$

is the average, over all $\binom{K}{k}$ subsets S of k trials of the pool, of the reported

score of the trial in S with the largest selection score, and $\mathbb{E}\hat{M}_k = \mathbb{E}M_k$.

PROOF. The plan is to count, for each trial, the subsets that select it, and then to use the fact that every subset is a search of budget k.

The average over subsets. A subset S selects its member of largest rank in the ordering by V. The trial of rank i is selected by exactly the subsets that contain it and $k-1$ trials of smaller rank, and there are $\binom{i-1}{k-1}$ of those, none when $i < k$. Summing the selected scores over all subsets therefore gives $\sum_i \binom{i-1}{k-1} Y_{[i]}$, and dividing by the number of subsets gives (4.3).

Unbiasedness. Fix a subset S. Its k pairs are independent with the common law of (V_1, Y_1), and the trial it selects is the same function of them as J is of the first k trials of a search; so its selected score has the law of M_k, and in particular a finite mean, because its absolute value is at most $\sum_{j\in S}|Y_j|$. Each of the $\binom{K}{k}$ terms of the average therefore has expectation $\mathbb{E}M_k$, and so has the average. ∎

Counting the subsets of $\{1, \dots, K\}$ of size k by their largest element shows that the weights sum to one, $\sum_{i=k}^{K} \binom{i-1}{k-1} = \binom{K}{k}$. Ties in the selection score are common, since a validation error on 300 images takes only 301 values; a secondary score such as the validation loss, or an independent random number, breaks them and keeps the hypothesis. The statistic is a *U-statistic* in the sense of Hoeffding (1948), with the selected score of k trials as its kernel; for fixed k, and a kernel with finite variance, it is asymptotically normal as K grows.

Dodge et al. (2019) proposed reporting the *expected validation performance*, the expected best validation score as a function of the budget, and estimated it with the expected maximum of k draws from the empirical distribution of the pool, that is, of k draws *with* replacement. In the notation of (4.3), with $Y = V$, that estimator is

$$\tilde{M}_k = \sum_{i=1}^{K} \left(\left(\frac{i}{K}\right)^k - \left(\frac{i-1}{K}\right)^k \right) Y_{(i)}.$$

Tang et al. (2020) showed that it is biased, always toward a worse best.

Proposition 4.8 (Drawing with replacement is pessimistic). *For real numbers $y_{(1)} \le \cdots \le y_{(K)}$ and $1 \le k \le K$, let U_k and W_k be the right-hand sides of (4.3) and of the display above, with $y_{(i)}$ in place of $Y_{[i]}$ and $Y_{(i)}$. Then $W_k \le U_k$, with strict inequality if $k \ge 2$ and $y_{(1)} < y_{(K)}$. Consequently, for independent scores with a finite mean, $\mathbb{E}\tilde{M}_k \le \mathbb{E}M_k$.*

PROOF. The plan is to write both statistics as $y_{(K)}$ minus a weighted sum of the gaps between consecutive order statistics, and to compare the weights. Both are averages $\sum_i w_i y_{(i)}$ with weights $w_i = G(i) - G(i-1)$ for a function G that increases from $G(0) = 0$ to $G(K) = 1$; summation by parts turns such an average into

$$\sum_{i=1}^{K} w_i y_{(i)} = y_{(K)} - \sum_{i=1}^{K-1} G(i)\big(y_{(i+1)} - y_{(i)}\big). \tag{4.4}$$

For W_k, $G_W(i) = (i/K)^k$, the probability that k draws with replacement all fall among the i smallest values. For U_k, $G_U(i) = \binom{i}{k}/\binom{K}{k}$, the probability that a random subset of size k does, by the identity for the weights applied with i in place of K. By (4.4), $U_k - W_k = \sum_{i=1}^{K-1} \big(G_W(i) - G_U(i)\big)\big(y_{(i+1)} - y_{(i)}\big)$, and every gap is nonnegative, so it suffices to show $G_U(i) \le G_W(i)$. If $i < k$, then $G_U(i) = 0$. If $i \ge k$, then $G_U(i) = \prod_{r=0}^{k-1} \frac{i-r}{K-r}$, a product of k factors in $[0, 1]$, and each is at most i/K because $(i - r)K \le i(K - r)$ amounts to $r(K - i) \ge 0$. For strictness, let $k \ge 2$ and $1 \le i \le K - 1$. For $i < k$ we have $G_U(i) = 0 < G_W(i)$; for $i \ge k$ the factor with $r = 1$ is strictly smaller than i/K, because $K - i > 0$. Every coefficient is then positive, and some gap is positive when $y_{(1)} < y_{(K)}$. For the last claim, take expectations and use Theorem 4.7 with $Y = V$. ■

The bias is not the whole story. Dodge et al. (2021) compared the estimators by variance and mean squared error as well: in their simulations the unbiased estimator had the largest variance, and the two biased estimators led to the fewest wrong conclusions about which of two models was better. The trade-off is real, and the bias is small when k is a small fraction of K; in the main pools of Lab 4.2, up to $k = K/4$, the two estimates of the best validation error differ by less than 0.1 points. What matters more is to report K, the estimator, and how far the curve is pushed.

How far it can be pushed has a simple answer. The probability that the trial selected from a random k-subset is among the m best of the pool is $1 - \binom{K-m}{k}/\binom{K}{k}$, and the same factor-by-factor comparison as in the proof above bounds $\binom{K-m}{k}/\binom{K}{k}$ by $(1 - m/K)^k$. For $m \ge K/k$ this gives at least $1 - (1 - 1/k)^k \ge 1 - e^{-1} \approx 0.63$. At budget k, then, $\hat{M}_k$ rests mostly on the best K/k trials of the pool: at $k = K/4$, on about four. Intervals from the bootstrap, which resamples the pool (Theorem A.3), cannot create trials better than the best one observed, and Tang et al. (2020) found that bootstrap intervals for the expected maximum covered the truth only about 70% of the time at a nominal 95% when $k = 20$ and $K = 50$. We quote comparisons at $k \le K/10$ and draw curves no further than $K/4$.

THE SELECTION SCORE IS OPTIMISTIC. The best validation score of k trials overstates the selected trial's performance on fresh data, increasingly with k: selection on a noisy score picks trials that were lucky on that score, the winner's curse of Chapter 1, and Chapter 5 computes the effect exactly for a Gaussian proxy. With a small validation set, more trials can even make the selected trial worse on the test set,[8] because the search begins to fit the noise of the validation set (Cawley and Talbot 2010). The budget curve to report is therefore the one for the test score of the selected trial, as in (4.3) with $V \ne Y$, and the configuration that a paper finally reports should be retrained with fresh seeds, since its own run was selected for its luck.

[8] With 300 validation images and an error near 15%, the binomial standard error of a validation error is about two points (Chapter 1).

4.6 In the laboratory: two protocols on one task

On the plain digits, momentum and Adam generalize equally well: in the control of Lab 4.2 below, their budget curves agree within a quarter of a point at every budget from six trials on, in both kinds of search space. To reproduce the original gap in miniature we add the ingredient of Wilson et al.'s constructed example, input coordinates that carry no information about the label but let the network fit its small training set. Each image gets 256 distractor features, independent standard normal numbers drawn once per image, appended to its 64 pixels.

LAB 4.1 · TWO PROTOCOLS, ONE COMPARISON

SCRIPT
`04-budgets-protocols.py`

SETUP. Digits with 256 distractor features: 300 training images, 300 validation images, and the remaining 1197 as test set. A two-layer MLP of width 64, 100 epochs, batch size 32, constant learning rate. *Protocol 2017*: for each optimizer, a logarithmic grid of 16 learning rates (momentum 10^{-3} to 1; Adam $10^{-4.5}$ to $10^{-1.5}$), momentum $\beta_1 = 0.9$, Adam's $(\beta_1, \beta_2, \epsilon) = (0.9, 0.999, 10^{-8})$, no weight decay. *Protocol 2019*: for each optimizer, 16 random-search trials, log-uniform in every coordinate, over the learning rate, $1 - \beta_1$ and an L2 weight decay λ for momentum, and over the learning rate, $\epsilon \in [10^{-8}, 10^{2}]$, $1 - \beta_1$, $1 - \beta_2$ and λ for Adam, whose learning rate is drawn relative to $\max(\epsilon, 10^{-2})$. In both, the trial with the lowest validation error is retrained with five fresh seeds and their mean test error is the protocol's result. Everything is replicated on 10 data splits, each with new distractors, search draws and seeds: 840 runs, about three minutes.

PREDICT FIRST. Under the 2017 protocol, which optimizer has the lower test error, and by how much? With the same sixteen trials searched over all hyperparameters, does the gap survive, shrink or reverse? Which optimizer gains more from the larger search?

LOOK FOR. The differences split by split, not only the means; the values of ϵ and λ that the 2019 search selects; retrained runs of a selected configuration that fail.

Figure 4.2 shows the result. Under the 2017 protocol Adam had the higher test error on nine of the ten splits (two-sided sign test $p = 0.021$), by a median of 3.2 points. The mean difference, 1.69 ± 4.10 points (mean and 95% t-interval over splits, Theorem A.1), is diluted by one failure: on split 6 the grid selected momentum's learning rate 0.1, at the edge of stability, on the strength of one good run, and of its five retrained runs one ended at 87% test error and two above 25%.[9] Under the 2019 protocol, with the same sixteen trials per optimizer, the difference was -0.44 ± 0.59 points, with Adam lower on five splits of ten: the gap vanished. Both optimizers improved, Adam by 6.04 ± 0.49 points, to 16.11 ± 0.57%, and momentum to 16.55 ± 0.70%.

The selected hyperparameters show how. The 2019 searches mostly chose a sizable weight decay (median $\lambda = 0.027$ for Adam and 0.017 for

[9] At the edge of stability the seed decides whether a run survives; Chapter 2 studies how small differences grow during training.

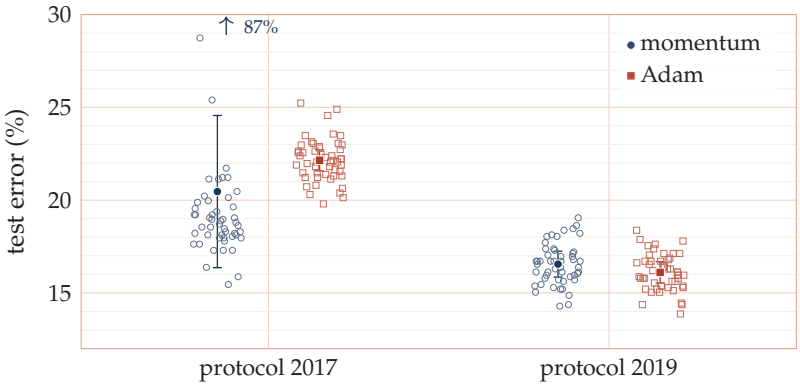


Figure 4.2. Lab 4.1: test error of the configuration selected by each protocol, for momentum (circles) and Adam (squares). Hollow marks: the five retrained runs of the selected configuration on each of 10 data splits (50 per arm and protocol, jittered horizontally); one run, at 87%, lies above the scale. Filled marks: the mean over the 10 splits of the five-run means, with a 95% t-interval over splits. Digits with 256 distractor features; 16 trials per optimizer and protocol.

momentum), a regularizer that the 2017 protocol held at zero for both arms, and half of Adam's selections had $\epsilon \geq 10^{-3}$, three of them above 1, where Adam moves almost like momentum. Which of the two closes the gap is the subject of Exercise 4.5. One more effect is visible in the numbers: the selected trial's own run was better on the test set than its five reruns, by 0.05 to 1.5 points depending on the arm, the optimism of selecting on a single run.

LAB 4.2 · BUDGET CURVES

SCRIPT
`04-budgets-curves.py`

SETUP. Split 0 of the task of Lab 4.1. Four pools of $K = 200$ random-search trials, one run each: momentum and Adam, each over its 2017 space (the learning rate only, log-uniform over the range of the grid) and its 2019 space (ranges in the scripts). For each pool, the expected test error of the trial that a search of k trials selects, for $k = 1, \ldots, 50$, by (4.3), with 95% percentile intervals from 1000 bootstrap resamples of the pool. A control repeats the four pools with $K = 100$ on the plain digits. 1200 runs, about four minutes.

PREDICT FIRST. At $k = 1$, which of the four arms is best? Do the curves of the two 2017 spaces cross, and where? Does more search always lower the test error of the selected trial?

LOOK FOR. Crossings; the budget beyond which the curves are flat; the distance between the expected best validation error and the expected test error of the selected trial; the control.

Figure 4.3 shows the four curves. In the 2017 spaces they cross. A single trial favors Adam, 26.1% (interval [25.1, 27.1]) against 43.2% ([38.8, 48.0]), because a third of momentum's learning-rate range diverges while Adam's

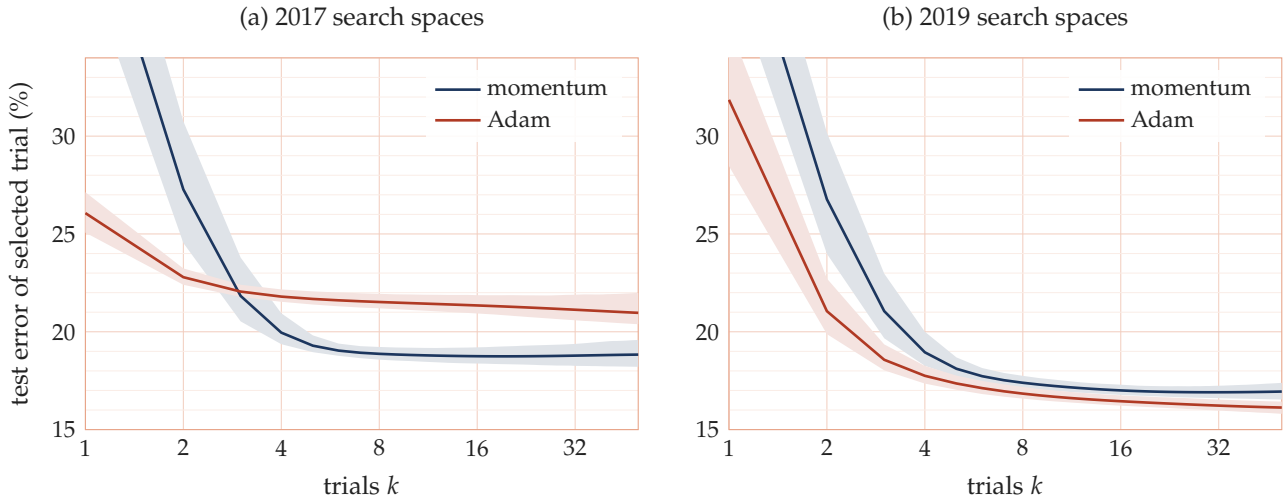


Figure 4.3. Lab 4.2: budget curves, the expected test error of the trial selected by a random search of k trials, estimated by (4.3) from pools of $K = 200$ trials per arm (one run each; split 0 of the task of Lab 4.1). Bands: 95% percentile intervals from 1000 bootstrap resamples of each pool. (a) Learning rate only, other hyperparameters at their defaults. (b) All hyperparameters. At $k = 1$ the momentum curves start at 43%, above the scale, because a third of their learning rates diverge.

range is forgiving; from three trials on momentum is ahead, and at $k = 16$ Adam's curve is higher by 2.6 points (95% bootstrap interval [2.0, 3.2]). This is Proposition 4.3 at work, with momentum the arm whose space has the higher ceiling and the lower floor. In the 2019 spaces Adam is ahead at every budget, by 0.55 points at $k = 16$ (interval [0.17, 0.90]): on this split, the comparison reversed rather than vanished. The same proposition appears within Adam: its 2019 space contains far more diverging trials than its 2017 space (33 of 200 against 6) and also better ones, so the two Adam curves cross between $k = 1$ and $k = 2$, after which the larger space is ahead by 4.9 points at $k = 16$.

Beyond about sixteen trials every curve is nearly flat. The expected best validation error keeps falling, from 15.5% at $k = 16$ to 14.8% at $k = 50$ for momentum's 2017 space, while the test error of the selected trial stays at about 18.8% (point estimates from the same pool): past this point the search mostly fits the noise of 300 validation images. The with-replacement estimate of the best validation error was larger than the unbiased one in all eight pools at every budget from two on, as Proposition 4.8 requires, but by less than 0.1 points in the main pools and 0.2 in the control pools. On the plain digits the control shows no gap: at $k = 10$, Adam minus momentum is -0.02 points (interval $[-0.23, 0.19]$) in the 2017 spaces and 0.01 points ($[-0.30, 0.30]$) in the 2019 spaces.

The miniature reproduces the shape of the story, with one honest qualification. The gap under the 2017 protocol needed the distractor features; without them the two optimizers tie under both protocols. What the lab shows in general is the mechanism: a comparison decided by a protocol that searched one optimizer's distinguishing hyperparameters and not the

other's, and a gap that a search of the same size over all of them removes.

4.7 Protocol

PROTOCOL 4.1 · COMPARING TWO TRAINING METHODS

- ☐ Write down each arm as an update rule plus a search space, a sampling distribution, a budget, a selection rule and a reporting rule, before running anything.
- ☐ Give the arms equal budgets in the unit of the claim: trials, steps, FLOPs or wall-clock time.
- ☐ Search every hyperparameter that distinguishes the arms, ϵ, the momentum coefficients and weight decay among them, over ranges wide enough that no optimum lies at an edge; report the ranges and how they were chosen.
- ☐ Select on validation data; report the test score of the selected configuration retrained with fresh seeds.
- ☐ Repeat the whole tuning protocol with new search draws, seeds and, where possible, data splits; report the paired differences with an interval.
- ☐ Report budget curves with the unbiased estimator (4.3) and bootstrap intervals for k up to about $K/4$, and say whether and where they cross.
- ☐ Treat hyperparameters taken from a paper as tuning done on other data: count them in the budget, or retune.
- ☐ State the conclusion with its budget: "A beats B for searches of k trials or more over these spaces".

The protocol makes a comparison fair to the update rules within the stated budget and spaces, and no further. It does not protect against search spaces designed with hindsight, refined on the very workload by an earlier search, as Choi et al. acknowledge for their own, nor against unequal engineering effort in the implementations. Its estimator assumes independent trials: a Bayesian optimizer or a successive-halving scheme chooses trials adaptively, and its budget curve must be estimated by repeating the whole search (Exercise 4.12). At the scale of large models even one search per arm can be unaffordable; then tune the hyperparameters that distinguish the arms on a smaller proxy and transfer them with stated rules, or fall back to the weakest honest claim, "at the settings we could afford", and report the budget that each arm actually received.

Exercises

4.1 CONSOLIDATION. (The shape of a budget curve.) Let F have a finite mean. (a) Use (4.1) to show that $\mathbb{E}M_{k+1} - \mathbb{E}M_k = \int F(x)^k(1 - F(x))\,\mathrm{d}x$. (b) Deduce that the increments are nonnegative and nonincreasing, so that $k \mapsto \mathbb{E}M_k$ is concave on the integers. (c) Compute the budget curve of the uniform distribution on $[a, b]$, and the number of trials needed to come within 1% of $b - a$ of the top.

4.2 CONSOLIDATION. (Crossing points.) In Example 4.4, show that arm 1 is ahead exactly when $k > \ln\big((a - c)/(a - b)\big)/\ln\big(1/(1 - p)\big)$, and find the crossing budget when the good fraction p is 0.01. What should a paper that compares the two arms with 100 trials each conclude, and how should it say so?

4.3 CONSOLIDATION. (The estimator by hand.) For $K = 4$ trials whose selection scores are 0.3, 0.9, 0.5 and 0.7 and whose reported scores are, in the same order, 0.40, 0.80, 0.70 and 0.60, compute $\hat{M}_2$ from (4.3) and check it against the average over the six pairs. Then compute $\tilde{M}_2$ for the validation scores and verify Proposition 4.8.

4.4 CONSOLIDATION. ★ (Rates and relatives of inclusion.) (a) In Proposition 4.6, assume each g_t is L-Lipschitz. Show that $\|\theta_T^\epsilon - \theta_T\| \le C/\epsilon$ for all large ϵ, with C depending on T, η, β_1, β_2, L and the gradients along the momentum path. (b) Show that with $\beta_1 = 0$ no schedule is needed: Adam with the constant step size $\epsilon\eta$ tends to SGD with learning rate η. (c) RMSProp with momentum, as implemented in PyTorch, divides the gradient by $\sqrt{v_t} + \epsilon$ before the momentum buffer: show that it contains momentum SGD with no schedule at all. *(Hint: in (a), carry a bound of order $1/\epsilon$ through the induction, using that the factor $1/(1 + x)$ differs from 1 by at most x.)*

4.5 LAB. (Which knob closes the gap?) Modify `04-budgets-protocols.py` so that the 2019 protocol searches Adam's learning rate and weight decay with $\epsilon = 10^{-8}$ fixed, and, separately, its learning rate and ϵ with no weight decay. Which part of the larger search closes the gap of Lab 4.1? Is the answer the same for $1 - \beta_1$?

4.6 LAB. (A naive search space.) Sample Adam's learning rate and ϵ independently, log-uniform on $[10^{-5}, 1]$ and $[10^{-8}, 10^2]$, instead of drawing the learning rate relative to $\max(\epsilon, 10^{-2})$. Rerun `04-budgets-curves.py` for this arm. How many trials does Adam now need to match momentum's 2019 curve? Relate your answer to the fraction of the space where Adam trains at all.

4.7 LAB. (Coverage.) Using the 200 trials of one pool of Lab 4.2 as a population, draw 300 subpools of $K = 40$ trials, compute the bootstrap interval for $\mathbb{E}M_k$ in each, and estimate its coverage of the population value for $k = 2, 4, 10, 20$. Compare with the rule of thumb of Section 4.5.

4.8 DESIGN. (Equal compute.) Method A's trials cost twice as much as method B's, and you have 64 GPU-hours, enough for 32 trials of A. Design a comparison that answers "which method should a practitioner with this budget use?" and one that answers "which update rule is better per trial?" Say what each reports, how many trials each arm gets, and how the conclusions can differ, using (4.1).

4.9 DESIGN. (A new optimizer for pretraining.) You may run 96 training runs of a 100-million-parameter language model to compare a new optimizer with AdamW. Decide which hyperparameters to search for each arm, how to split the runs between search and replication, how to choose the search spaces without spending the budget on them, and what to plot. Which finding of Wen et al. (2026) constrains your design?

4.10 CRITIQUE. (Recommended settings.) A paper compares its method, tuned by 100 trials of Bayesian optimization, with three baselines run at "the hyperparameters recommended by their authors". The method wins by 0.8 points; the spread of trial scores over its search space has standard deviation about 1 point. Using Corollary 4.5, say what this comparison can and cannot show, and what the authors should have run.

4.11 CRITIQUE. (A table of best results.) A benchmark table lists, for each of six optimizers, the best test accuracy over "extensive tuning", without the number of trials or the search spaces, and bolds the largest. List what is missing, rank the omissions by how much they could change the ranking, and propose a replacement figure.

4.12 RESEARCH. (Adaptive search.) Bayesian optimization and successive halving choose trials adaptively, so the trials of a search are not independent and Theorem 4.7 does not apply. Open: an estimator of the budget curve of an adaptive search method from fewer runs than repeating whole searches, with a valid interval; or a proof that none can exist without assumptions on the response surface.

Notes and References

The case for random search over grid search was made by Bergstra and Bengio (2012), who showed that it matches or beats grid search at a fraction of the cost, because on a given task few hyperparameters matter, and proposed it as the baseline for adaptive search methods (Chapter 5 makes it the placebo arm of a search). The expected best result as a function of the budget was proposed as a reporting standard by Dodge et al. (2019), with a checklist for reporting hyperparameter searches; Tang et al. (2020) proved that their estimator is biased and derived the unbiased one, and Dodge et al. (2021) weighed bias against variance. Theorem 4.7 with concomitants, which turns the curve of the best validation score into the curve of the selected trial's test score, is the form we found most useful in practice.

Cawley and Talbot (2010) analyzed over-fitting in model selection, the reason that curve can rise.

The pattern of this chapter, a gap that closes when the baseline is tuned as hard as the proposal, recurs across the field. Melis et al. (2018) found that standard LSTMs, properly regularized and tuned by large-scale black-box search, outperformed more recent recurrent architectures in language modeling; Lucic et al. (2018) found that most GAN variants reached similar scores given enough hyperparameter search and random restarts. Chapter 5 adds the placebo arm, Chapter 7 the question of which budget axis is held fixed, and Chapter 12 the effect of the learning-rate schedule on curves read before their end.

For practice, the tuning playbook of Godbole et al. (2023) classifies hyperparameters as scientific, nuisance and fixed; when optimizers are compared, the optimizer is scientific and its own hyperparameters are nuisance hyperparameters to be tuned separately for each arm. It suggests tuning only Adam's learning rate with fewer than about ten trials, adding β_1 at 10 to 25, ϵ beyond 25, and β_2 with substantially more.

TRANSLATING THE LITERATURE. Dodge et al. (2019) write n for our budget k and V_n^* for our M_k with $Y = V$; their expected validation performance $\mathbb{E}[V_n^* \mid n]$ is our $\mathbb{E}M_k$, and their estimator is our $\tilde{M}_k$. Tang et al. (2020) write B for our pool size K and $\hat{U}_n^B$ and $\hat{V}_n^B$ for our $\hat{M}_k$ and $\tilde{M}_k$. Choi et al. (2019) write γ for the momentum coefficient, our β_1, and v for momentum's buffer, our b; their Adam adds ϵ outside the square root and applies the bias corrections to the step, as TensorFlow does, so their ϵ is our ϵ times $\sqrt{1-\beta_2^t}$, the same after a few thousand steps when $\beta_2 = 0.999$. Wilson et al. (2017) write HB for heavy ball, our momentum SGD with $\beta_1 = 0.9$, and use "dev" for validation. Sivaprasad et al. (2020) call our 2017 space for Adam "Adam-LR".

References

Bergstra, J. and Y. Bengio (2012). "Random Search for Hyper-Parameter Optimization". In: *Journal of Machine Learning Research* 13, pp. 281–305.

Cawley, G. C. and N. L. C. Talbot (2010). "On Over-fitting in Model Selection and Subsequent Selection Bias in Performance Evaluation". In: *Journal of Machine Learning Research* 11, pp. 2079–2107.

Choi, D., C. J. Shallue, Z. Nado, J. Lee, C. J. Maddison, and G. E. Dahl (2019). *On Empirical Comparisons of Optimizers for Deep Learning*. Preprint, arXiv:1910.05446 (version 3, 2020).

Dahl, G. E. et al. (2023). *Benchmarking Neural Network Training Algorithms*. Preprint, arXiv:2306.07179 (version 2, 2025).

Dodge, J., S. Gururangan, D. Card, R. Schwartz, and N. A. Smith (2019). "Show Your Work: Improved Reporting of Experimental Results". In: *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*. Association for Computational Linguistics, pp. 2185–2194.

Dodge, J., S. Gururangan, D. Card, R. Schwartz, and N. A. Smith (2021). "Expected Validation Performance and Estimation of a Random Variable's Maximum". In: *Findings of the Association for Computational Linguistics: EMNLP 2021*. Association for Computational Linguistics, pp. 4066–4073.

Godbole, V., G. E. Dahl, J. Gilmer, C. J. Shallue, and Z. Nado (2023). *Deep Learning Tuning Playbook*. https://github.com/google-research/tuning_playbook. Version 1.0.

Hoeffding, W. (1948). "A Class of Statistics with Asymptotically Normal Distribution". In: *The Annals of Mathematical Statistics* 19.3, pp. 293–325.

Kasimbeg, P. et al. (2025). "Accelerating Neural Network Training: An Analysis of the AlgoPerf Competition". In: *International Conference on Learning Representations*.

Keskar, N. S. and R. Socher (2017). *Improving Generalization Performance by Switching from Adam to SGD*. Preprint, arXiv:1712.07628.

Kingma, D. P. and J. Ba (2015). "Adam: A Method for Stochastic Optimization". In: *International Conference on Learning Representations*.

Loshchilov, I. and F. Hutter (2019). "Decoupled Weight Decay Regularization". In: *International Conference on Learning Representations*.

Lucic, M., K. Kurach, M. Michalski, S. Gelly, and O. Bousquet (2018). "Are GANs Created Equal? A Large-Scale Study". In: *Advances in Neural Information Processing Systems 31*, pp. 700–709.

Luo, L., Y. Xiong, Y. Liu, and X. Sun (2019). "Adaptive Gradient Methods with Dynamic Bound of Learning Rate". In: *International Conference on Learning Representations*.

Melis, G., C. Dyer, and P. Blunsom (2018). "On the State of the Art of Evaluation in Neural Language Models". In: *International Conference on Learning Representations*.

Nado, Z., J. M. Gilmer, C. J. Shallue, R. Anil, and G. E. Dahl (2021). *A Large Batch Optimizer Reality Check: Traditional, Generic Optimizers Suffice Across Batch Sizes*. Preprint, arXiv:2102.06356.

Schmidt, R. M., F. Schneider, and P. Hennig (2021). "Descending through a Crowded Valley: Benchmarking Deep Learning Optimizers". In: *Proceedings of the 38th International Conference on Machine Learning*. Vol. 139. Proceedings of Machine Learning Research, pp. 9367–9376.

Sivaprasad, P. T., F. Mai, T. Vogels, M. Jaggi, and F. Fleuret (2020). "Optimizer Benchmarking Needs to Account for Hyperparameter Tuning". In: *Proceedings of the 37th International Conference on Machine Learning*. Vol. 119. Proceedings of Machine Learning Research, pp. 9036–9045.

Tang, R., J. Lee, J. Xin, X. Liu, Y. Yu, and J. Lin (2020). "Showing Your Work Doesn't Always Work". In: *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*. Association for Computational Linguistics, pp. 2766–2772.

Wen, K., D. Hall, T. Ma, and P. Liang (2026). "Fantastic Pretraining Optimizers and Where to Find Them". In: *International Conference on Learning Representations*. arXiv:2509.02046.

Wilson, A. C., R. Roelofs, M. Stern, N. Srebro, and B. Recht (2017). "The Marginal Value of Adaptive Gradient Methods in Machine Learning". In: *Advances in Neural Information Processing Systems 30*, pp. 4148–4158.

Zhou, P., J. Feng, C. Ma, C. Xiong, S. Hoi, and W. E (2020). "Towards Theoretically Understanding Why SGD Generalizes Better Than Adam in Deep Learning". In: *Advances in Neural Information Processing Systems 33*.

5 Controls

Random search as the placebo arm

THE CLAIM *"Neural architecture search finds better architectures than random search."*

As reported for ENAS (Pham et al. 2018) and DARTS (Liu, Simonyan and Yang 2019)

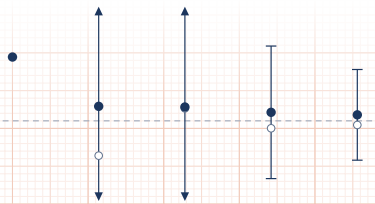
THE VERDICT Vanished: trained with the same pipeline, architectures sampled at random from the same space do about as well.

IN THIS CHAPTER. In 2018 and 2019 weight sharing made neural architecture search cheap, and ENAS and DARTS reported architectures much better than those of random search. When later studies gave the random arm the same space, the same training pipeline and enough draws, the searched architectures were ahead by a few tenths of a point on CIFAR-10 and not at all on Penn Treebank; the original gaps came from tiny random arms, from proxies that barely ranked architectures, and from pipelines that differed between arms. The method is the control arm of the older sciences: a placebo that samples the space, a sham that spends the same budget without the search strategy, and positive and negative controls. We prove that a search guided by a Gaussian proxy gains over the placebo the proxy's correlation with the truth times the gain of a perfect search, and reports a gain inflated by the inverse square of that correlation. On a benchmark of 192 digits networks, random search with a short-training proxy does at least as well as evolution, and a baseline copied from a weaker pipeline makes the search look more than twice as useful as it is.

5.1 The claim: better than random, at a thousandth of the cost

The first architecture searches were expensive. Zoph and Le (2017) trained a recurrent controller with reinforcement learning to write down network descriptions, with the validation accuracy of each trained network as the reward, and found a CIFAR-10 network with 3.65% test error and a recurrent cell that beat the LSTM on Penn Treebank. Searching for a repeatable cell made the result transferable to ImageNet (Zoph et al. 2018), and evolution in the same cell space produced AmoebaNet (Real et al. 2019), at search costs that Liu et al. (2019) list as 2000 and 3150 GPU-days.

WEIGHT SHARING. Pham et al. (2018) cut the cost by three orders of magnitude with *weight sharing*. In ENAS every candidate architecture is a subgraph of one large graph and inherits its weights, so no candidate is trained from scratch: a controller samples subgraphs and is trained by



policy gradient to maximize a reward measured on validation data, while the shared weights are trained on the sampled subgraphs. On one GTX 1080Ti the search took less than sixteen hours, and ENAS found a recurrent cell with test perplexity 56.3 on Penn Treebank and a convolutional cell with 2.89% test error on CIFAR-10 when trained with cutout, 3.54% without.[1] Liu et al. (2019) made the search differentiable. In DARTS each edge of the cell computes a softmax-weighted mixture of the candidate operations; the mixture weights are learned by gradient descent on the validation loss, the network weights on the training loss, and the strongest operations are kept at the end. Second-order DARTS reached 2.76 ± 0.09% test error on CIFAR-10 (mean and standard deviation of ten trainings of the selected cell) after four GPU-days of search, and 55.7 test perplexity on Penn Treebank after one.

[1] The arXiv version of the paper reports a perplexity of 55.8; the proceedings of ICML 2018 report 56.3.

THE RANDOM ARMS. Both papers asked whether the search mattered, which many papers of the period did not (Lindauer and Hutter 2020). ENAS sampled one architecture uniformly from each of its search spaces and trained it like the searched ones; DARTS ran a random search, drawing 24 cells (eight recurrent cells on Penn Treebank) and keeping the one with the best validation error after a short training.

> FINDING 5.1 (Pham et al. 2018; Liu et al. 2019). In the ENAS spaces, one uniformly sampled recurrent cell reached test perplexity 81.2 on Penn Treebank, against 56.3 for the ENAS cell, and one pair of sampled convolutional cells reached 6.77% test error on CIFAR-10, against 3.54% for the ENAS cells, both trained without cutout. In the DARTS space, the best of 24 random cells, kept for its validation error after 100 epochs, reached 3.29 ± 0.15% test error, against 2.76 ± 0.09% for second-order DARTS (means and standard deviations of ten trainings); the best of eight random recurrent cells, kept after 300 epochs, reached perplexity 59.4, against 55.7.

The DARTS paper drew a measured conclusion: random search was competitive on both tasks, which "reflects the importance of the search space design", and DARTS, at comparable or lower cost, improved on it substantially. The field received the stronger half. Weight-sharing searches, many of them variants of DARTS, multiplied, and papers compared themselves in tables of test errors and GPU-days taken from one another. The received claim was that a cheap search finds better architectures than random search does.

WHAT WAS RIGHT. The cost reduction was real, and it brought architecture search within reach of a university laboratory. Both papers ran a random control and published their code, and DARTS reported the variation of its final trainings. The claim was also reasonable. Searches had produced state-of-the-art classifiers for CIFAR-10 and ImageNet (Real et al. 2019; Zoph et al. 2018), and in a controlled comparison in the NASNet space, with the same code for every arm and five repeats, evolution and reinforcement learning both compared favorably against random search (Real et al. 2019).[2] What went wrong was not that search is useless, but that the control arms

[2] Each arm of that comparison trained 20,000 models for 25 epochs on 450 GPUs, about a week per experiment; the random arm was a random search with the same budget, a sham in the language of Section 5.4.

of the cheap searches were too weak to show what the search added.

5.2 The experiment: one search against one random draw

The comparison behind the received claim is clearest in the DARTS paper's CIFAR-10 table.

EXPERIMENTAL DESIGN · LIU, SIMONYAN AND YANG 2019, CIFAR-10

QUESTION	Does gradient-based search find better cells than random search in the same space?
TREATMENT	the search method: second-order DARTS, first-order DARTS, or random search
SPACE	the DARTS cells (convolutions, poolings and skip connections on a fixed cell graph)
SELECTION	DARTS run four times, random search drawing 24 cells; in each arm the cell with the best validation error after 100 epochs of training from scratch is kept
HELD FIXED	the final pipeline: 20 cells, 36 channels, 600 epochs, cutout, path dropout, auxiliary towers
RESPONSE	test error of the kept cell
REPLICATES	ten trainings of the kept cell; each arm's whole procedure (4 GPU-days of search) run once
ANALYSIS	mean and standard deviation of the ten trainings, beside numbers from other papers

The design has a strength that tables of the period often lacked: the random arm is concurrent, with the same space and the same pipeline, so the pipeline cannot explain the gap between 3.29% and 2.76%. Its weakness is the unit of replication. The claim is about two procedures, each of which returns a random architecture; the experiment ran each procedure once and trained the returned architecture ten times. The interval ±0.09 describes the trainings of one cell, not the variation of the search, and whether 3.29% was a typical outcome of a 24-draw random search or an unlucky one, the design cannot say. Nor does it say how good the average architecture of the space is, so the reader cannot split DARTS's 2.76% into what the space and pipeline give every architecture and what the search adds. The other rows of the table came from other papers, some re-trained under DARTS's setup; comparisons with them are historical. The ENAS controls had one architecture per space.

A reader in 2019 could conclude that DARTS had beaten one draw of random search under one pipeline, but not how large the effect of the search was relative to its own variability, nor how close a randomly sampled cell, trained the same way, would come.

5.3 The post-mortem: a fair draw for the random arm

Three studies, published in 2019 and 2020, gave the random arm what it had lacked: more draws, repeated searches, and the same pipeline.

Random search, done properly. Li and Talwalkar (2019) treated architecture search as hyperparameter optimization and ran two strong random baselines in the DARTS spaces. Random search with early stopping (the bandit method ASHA, from one to 300 epochs) evaluated more than 700 cells on CIFAR-10 in nine GPU-days and reached 3.03 ± 0.13% test error (mean and standard deviation of ten trainings), against 3.29% for the DARTS random baseline; on Penn Treebank it reached perplexity 56.4, against 59.4. Random search with weight sharing, which ranks thousands of random cells with weights trained on uniformly sampled subgraphs, reached 55.5 on Penn Treebank, better than DARTS's 55.7, and 2.85 ± 0.08% on CIFAR-10, close to DARTS's 2.76 ± 0.09% and to their own rerun of it, 2.78 ± 0.12%. They found none of twelve recent search papers exactly reproducible from what was released, and their own method, rerun with new seeds, gave 56.5 instead of 55.5.

The search phase against a placebo. Yu et al. (2020) compared each search with one architecture sampled uniformly from the same space and trained like the searched one, and repeated every arm ten times with different seeds.[3] On Penn Treebank the random cells reached validation perplexity 60.13 ± 0.65 (mean and standard deviation over the seeds), against 59.88 ± 1.92 for ENAS, 60.61 ± 2.54 for DARTS and 61.99 ± 1.95 for NAO; Welch's t-tests gave $p = 0.73$, 0.62 and 0.02, the last because NAO was worse. On CIFAR-10 the random cells reached 96.44 ± 0.19% accuracy and the three searches between 96.62% and 96.86%, less than half a point better. They traced the weakness to the proxy: in a reduced recurrent space of 32 cells, each trained ten times to obtain its true rank, the Kendall rank correlation between the ranking given by shared weights and the true one averaged −0.004 over ten runs, and in NAS-Bench-101 spaces of three to seven nodes it fell from 0.44 to 0.20; samplers trained on true accuracies instead beat random sampling with probability 0.90 (ENAS) and 0.92 (NAO). Their reading was that the spaces were constrained so well that a random architecture already did well.

[3] The paper first circulated in 2019 with Christian Sciuto as first author and is often cited as Sciuto et al.; the published version lists Kaicheng Yu and Christian Sciuto as equal first authors.

Eight methods, five datasets, one placebo. Yang et al. (2020) ran eight search methods on five image datasets, eight searches and eight random architectures per method and dataset, all trained with each method's own pipeline. Their score, the relative improvement over the placebo in percent of its accuracy, ranged on CIFAR-10 from −0.48 to 0.74 (DARTS 0.32, ENAS 0.01), and on the other datasets several methods fell below their placebo. The training protocol mattered more: from the plainest pipeline to one with every trick, the same random cells gained more than three points, while the best improvement of a search over random sampling on CIFAR-10 was 0.69 points. Over 214 random cells of the DARTS space, trained with the DARTS pipeline, accuracy was 97.03 ± 0.23% (mean and standard deviation), from 96.18% to 97.56%; a deliberately weaker set of operations moved this

distribution by only 0.18 points, and two trainings of the same 32 cells with different seeds ranked them with Kendall $\tau = 0.48$.

Worse than the placebo, and better. In the tabular benchmark NAS-Bench-201, where all 15,625 cells have been trained, DARTS with its original hyperparameters converged to a cell made only of skip connections, with 54.30% test accuracy on CIFAR-10, while random search reached 93.70 ± 0.36% (mean and standard deviation of 500 runs) and the best cell 94.37% (Dong and Yang 2020); Zela et al. (2020) found such degenerate solutions of DARTS in a wide range of spaces. The same literature marks the limits of the verdict: in NAS-Bench-101, where the proxy is full training, regularized evolution and two Bayesian optimizers overtook random search after about 25 evaluated architectures and reached its final performance about five times faster (Ying et al. 2019). What vanished is the margin of the cheap, weight-sharing searches over a fair random arm in the spaces where they were introduced.

The mechanism. Four things made a small effect look large. The random arms were underpowered: one architecture per space for ENAS, one random search for DARTS, with selection after a short training that Li and Talwalkar (2019) found unstable. The spaces were narrow by design, so the spread between random architectures was about the size of the noise between seeds. The proxies, shared weights, ranked architectures little better than chance, and a search guided by a proxy that does not rank cannot beat the placebo (Section 5.5). And comparisons with published numbers mixed pipelines whose differences were worth more than the search. The same pattern appears wherever new methods are compared with old numbers: given the same tuning and pipeline, the standard LSTM matched or beat newer recurrent cells, the NAS cell among them (Melis et al. 2018), the GAN variants tested did not consistently beat the original (Lucic et al. 2018), and claimed gains over decade-old metric-learning methods shrank to marginal (Musgrave et al. 2020). The missing control was the old method given the new method's budget and pipeline (Chapter 4).

verdict. The large margin of the cheap, weight-sharing searches over random architectures did not survive a fair random arm. With the same space, pipeline and enough draws, random architectures matched the searched cells on Penn Treebank and came within a few tenths of a point on CIFAR-10, a remainder of the order of the seed-to-seed variation that ten searches detected for some methods and not for others. The reported gaps came from random arms of one to twenty-four architectures, from proxies that barely ranked, and, in tables of published numbers, from different pipelines. With a proxy that ranks well and a space with real spread, search does beat random search, by a small margin.



5.4 The method: placebo, sham and the spread of a space

An experiment learns about a treatment only when its arms differ in nothing else, and the older sciences name the arms that make this true. A *negative control* is a condition in which the effect cannot occur, and the assay must show none; epidemiologists borrowed the practice from the laboratory to detect bias (Lipsitch et al. 2010). A *positive control* is a condition in which the effect is known to occur, and the assay must show it; the guideline on control groups in clinical trials calls the ability of a trial to tell an effective treatment from an ineffective one its *assay sensitivity* (International Conference on Harmonisation 2000). A *placebo* is an inert treatment that looks like the real one, and controls for every influence on the course of the disease other than the drug, regression to the mean and expectations included. A *sham* procedure, in surgery and device trials, is the whole procedure without its active step. The guideline treats the external or *historical control*, a group treated earlier or elsewhere, separately, because it can rarely guarantee that the groups are comparable.[4]

A search method has the same anatomy. It consists of a search space $\mathcal{A}$ with a sampling distribution π (uniform, for a finite space); a *proxy*, a cheap and noisy measurement of an architecture's quality such as the validation error after a short training or under shared weights; a budget of k proxy evaluations; a rule that selects the winner; and a final training pipeline that produces the reported number. The active ingredient is the search strategy; everything else is the vehicle.

Definition 5.2 (Controls for a search method). For a search method with space $\mathcal{A}$, distribution π, proxy, budget k, selection rule and final pipeline:

(i) the *placebo arm* draws one architecture from π and trains it with the final pipeline (random sampling);

(ii) the *sham arm* draws k architectures independently from π, evaluates them with the same proxy, selects with the same rule and trains the winner with the final pipeline (random search with the same budget);

(iii) a *positive control* is a run in which a gain is known to exist, such as a perfect proxy (an *oracle*) or a planted architecture of known quality, and the experiment must detect it;

(iv) a *negative control* is a run in which no gain can exist, such as a proxy whose values are permuted among the architectures, and the experiment must report none;

(v) a *historical control* is a number produced earlier by another pipeline, usually in another paper.

The placebo asks whether searching helps at all, and measures the space; the sham asks whether this search strategy helps more than drawing at

[4] Sacks et al. (1982) found six therapies tested both ways: 44 of 56 trials with historical controls (79%) found the new therapy better, but only 10 of 50 randomized trials (20%). The treated patients did about equally well in both; the historical control groups did worse.

random with the same proxy and budget.[5] A method that beats its placebo but not its sham owes its gain to the proxy and the budget, which random search exploits equally well. Both arms are concurrent: they share the space, the proxy and the pipeline with the search. A historical control shares none of them. Chapter 8 uses the same device for ablations, with a shuffled pruning mask as the sham.

[5] Lindauer and Hutter (2020) make the same distinction, between *random sampling*, which returns one random architecture at no search cost, and *random search*, which runs as long as the method it is compared with, and recommend reporting both.

WHAT A HISTORICAL COMPARISON MEASURES. Write $\mu_p(a)$ for the true error of architecture a trained by pipeline p, the mean of its run distribution over seeds, and $\bar{\mu}_p$ for the mean of $\mu_p(A)$ over $A \sim \pi$, the expected error of the placebo. A paper that reports its searched architecture A under a new pipeline against a baseline b measured under an old pipeline reports $\mu_{\text{old}}(b) - \mu_{\text{new}}(A)$. Adding and subtracting $\mu_{\text{new}}(b)$ and $\bar{\mu}_{\text{new}}$ splits it into three parts:

$$\mu_{\text{old}}(b) - \mu_{\text{new}}(A) = \underbrace{\mu_{\text{old}}(b) - \mu_{\text{new}}(b)}_{\text{pipeline}} + \underbrace{\mu_{\text{new}}(b) - \bar{\mu}_{\text{new}}}_{\text{baseline against space}} + \underbrace{\bar{\mu}_{\text{new}} - \mu_{\text{new}}(A)}_{\text{search}}. \tag{5.1}$$

Only the last term belongs to the search, and only a concurrent placebo arm under the new pipeline measures it. The middle term is negative when the baseline is better than the average architecture of the space, as a well-chosen textbook architecture usually is; the first is the co-treatment of Chapter 6.

THE SPREAD OF A SPACE. Let Y be the test error of one run and $\mu(a) = \mathbb{E}[Y \mid a]$ the *true error* of architecture a under the fixed pipeline. For $A \sim \pi$, the mean $\bar{\mu}$ of $\mu(A)$ is the expected error of the placebo, its standard deviation $\sigma_{\mathcal{A}}$ is the *spread* of the space, and $H = \bar{\mu} - \mu^\star$, with $\mu^\star$ the smallest true error in the space, is its *headroom*. A search that returns the architecture $A_{\mathcal{S}}$ has *placebo-adjusted gain* $G = \bar{\mu} - \mathbb{E}\mu(A_{\mathcal{S}})$.

REMARK 5.3 (No search beats the headroom). Every search satisfies $G \le H$, because $\mu(A_{\mathcal{S}}) \ge \mu^\star$ for every outcome of the search. In a space whose headroom is a few tenths of a point, no strategy can gain more than that over the placebo, however expensive.

The spread is measured with noise. If each of m sampled architectures is trained n times and $\bar{Y}_a$ is its mean, then $\operatorname{Var} \bar{Y}_a = \sigma^2_{\mathcal{A}} + \sigma^2/n$, where σ^2 is the run-to-run variance within an architecture, so the standard deviation of the observed means overstates the spread; the one-way random-effects estimator subtracts the within-architecture mean square divided by n (Theorem A.12; the crossed version is in Chapter 1). The best observed mean is biased downward like every best-of-k number (Chapter 1), so the headroom is estimated honestly only by retraining the winner with fresh seeds.

How many random draws does a perfect search need? The answer does not depend on the size of the space.

Proposition 5.4 (Random search reaches a quantile). *Let $A_1, \dots, A_k$ be independent draws from π, and let $Q \subseteq \mathcal{A}$ be a set with $\pi(Q) = q \in (0, 1]$,*

for instance the best q-fraction of the space. Then

$$\mathbb{P}\big(A_i \in \mathcal{Q} \text{ for some } i \le k\big) = 1 - (1-q)^k \ge 1 - e^{-kq},$$

so $k \ge \ln(1/\delta)/q$ draws reach $\mathcal{Q}$ with probability at least $1-\delta$.

Proof. The events $\{A_i \notin \mathcal{Q}\}$ are independent, each with probability $1-q$, so all k occur with probability $(1-q)^k$, and the complement is the stated event. The inequality $1 - q \le e^{-q}$, which says that the convex function e^{-q} lies above its tangent at $q = 0$, gives $(1-q)^k \le e^{-kq}$, and $k \ge \ln(1/\delta)/q$ makes $e^{-kq} \le \delta$. The degenerate case $q = 1$ is covered as well: every draw lies in $\mathcal{Q}$, and the probability is $1 - 0^k = 1$. ∎

With $q = \delta = 0.05$, sixty draws reach the best 5% of a space with probability $1-0.95^{60} = 0.954$, whether the space has a hundred architectures or 10^{18}.[6] But a random search returns a draw from $\mathcal{Q}$ only when its proxy recognizes it, which is the subject of the next section.

[6] Bergstra and Bengio (2012) use the same formula, $1-(1-v/V)^T$ for a target of relative volume v/V and T trials, to compare random search with grids.

5.5 The method: a proxy caps what a search can find

A proxy evaluation of an architecture returns a random number P, such as the validation error of one short training with a fresh seed. For $A \sim \pi$ and one proxy evaluation of it, $T = \mu(A)$ and P form a random pair (T, P). Random search with budget k draws k independent copies $(T_1, P_1), \dots, (T_k, P_k)$ and returns the architecture with the smallest proxy error. What that does to $\mathbb{E}T$ runs through the *regression function* m, defined by $m(P) = \mathbb{E}[T \mid P]$: the expected true error of an architecture given what the proxy says.

Lemma 5.5 (Selection acts through the regression). *Let $(T_1, P_1), \dots, (T_k, P_k)$ be independent copies of a pair (T, P) with $\mathbb{E}|T| < \infty$, and let $m(P) = \mathbb{E}[T \mid P]$. Let the index $J \in \{1, \dots, k\}$ be a function of $P_1, \dots, P_k$ and of a random variable U independent of all the pairs. Then $\mathbb{E}T_J = \mathbb{E}m(P_J)$.*

Proof. The plan is to condition on everything the selection sees. Let $\mathcal{G}$ be the σ-field generated by $P_1, \dots, P_k$ and U; by hypothesis the indicator $\mathbf{1}\{J = i\}$ is $\mathcal{G}$-measurable. The pair (T_i, P_i) is independent of the other proxies and of U, so conditioning on them adds nothing to conditioning on P_i, and $\mathbb{E}[T_i \mid \mathcal{G}] = m(P_i)$ almost surely. In the display below, the first equality is the tower property with the $\mathcal{G}$-measurable indicator taken out of the conditional expectation, legitimate because $|T_i \mathbf{1}\{J = i\}| \le |T_i|$ is integrable, and the second is the identity $\mathbb{E}[T_i \mid \mathcal{G}] = m(P_i)$:

$$\mathbb{E}\big[T_i \mathbf{1}\{J = i\}\big] = \mathbb{E}\big[\mathbf{1}\{J = i\}\, \mathbb{E}[T_i \mid \mathcal{G}]\big] = \mathbb{E}\big[\mathbf{1}\{J = i\}\, m(P_i)\big].$$

Summing over i gives $\mathbb{E}T_J = \sum_i \mathbb{E}[T_i \mathbf{1}\{J = i\}] = \mathbb{E}m(P_J)$. ∎

The lemma is the theory of the negative control: if the proxy carries no information, so that m is constant, then $\mathbb{E}T_J = \bar{\mu}$ for every rule that looks only at proxy values, however clever, and the search is the placebo. With a

Gaussian proxy the gain can be computed. Recall from Chapter 1 that e_k is the expected maximum of k independent standard normal variables.

Theorem 5.6 (Selection on a Gaussian proxy). *Let (T, P) be bivariate normal with means μ_T, μ_P, standard deviations $\sigma_T, \sigma_P > 0$ and correlation ρ, let $(T_1, P_1), \dots, (T_k, P_k)$ be independent copies, and let J be the index of the smallest P_i. Then*

$$\mathbb{E}T_J = \mu_T - \rho\,\sigma_T\,e_k \qquad \text{and} \qquad \mathbb{E}P_J = \mu_P - \sigma_P\,e_k. \tag{5.2}$$

In the additive model $P = T + \xi$, with $\xi \sim \mathcal{N}(0, s^2)$ independent of T, the correlation is $\rho = \sigma_T/\sqrt{\sigma_T^2 + s^2}$.

Proof. The plan is to reduce the statement to the best-of-k computation of Chapter 1, which is the Gaussian case of Lemma 5.5. The P_i have a continuous distribution, so ties have probability zero and J is almost surely unique.

The selected architecture. The index J of the smallest P_i is the index of the largest $D_i = -P_i$, and the pairs (D_i, T_i) are independent and bivariate normal with standard deviations σ_P, σ_T and correlation $-\rho$. Part (ii) of Proposition 1.6, whose proof computes the regression function m and applies the reasoning of Lemma 5.5, gives $\mathbb{E}T_J = \mu_T + (-\rho)\sigma_T e_k = \mu_T - \rho\sigma_T e_k$.

The selected proxy. Part (i) of the same proposition, applied to the P_i, gives $\mathbb{E}P_J = \mathbb{E}\min_i P_i = \mu_P - \sigma_P e_k$.

The additive model. Independence of T and ξ gives $\mathrm{Cov}(T, P) = \sigma_T^2$ and $\sigma_P^2 = \sigma_T^2 + s^2$, so $\rho = \sigma_T^2/(\sigma_T\sigma_P) = \sigma_T/\sqrt{\sigma_T^2 + s^2}$. ∎

With $T = \mu(A)$, so that $\mu_T = \bar{\mu}$ and $\sigma_T = \sigma_{\mathcal{A}}$, random search with a Gaussian proxy has placebo-adjusted gain $G = \rho\,\sigma_{\mathcal{A}}\,e_k$. There are three ways to be no better than the placebo: a narrow space ($\sigma_{\mathcal{A}}$ small), a small budget (e_k grows only like $\sqrt{2\ln k}$), or a proxy that does not rank (ρ near zero).

Corollary 5.7 (Efficiency and the winner's curse). *Under the hypotheses of Theorem 5.6:*

(i) the placebo-adjusted gain of the proxy-guided search is the fraction ρ of the gain $\sigma_T e_k$ of the same search with a perfect proxy;

(ii) in the additive model, the gain that the search reports on its own proxy, $\mu_P - \mathbb{E}P_J$, is the true gain divided by ρ^2.

Proof. A perfect proxy selects the smallest T_i, and the computation of the minimum in the proof of Theorem 5.6, applied to the T_i, gives $\mathbb{E}\min_i T_i = \mu_T - \sigma_T e_k$; dividing $\rho\sigma_T e_k$ by $\sigma_T e_k$ proves (i). In the additive model $\mathbb{E}\xi = 0$ gives $\mu_P = \mu_T$, and $\sigma_P = \sigma_T/\rho$, so the reported gain is $\sigma_P e_k = \sigma_T e_k/\rho$ while the true gain is $\rho\sigma_T e_k$; their ratio is $1/\rho^2$. ∎

Part (ii) is the *winner's curse* of a search: the winner's proxy score is the best of k noisy numbers, and its noise was selected along with its quality.

With $\rho = 0.5$ a search reports four times the gain it achieved. Animal breeders know Theorem 5.6 in the additive case as the breeder's equation $R = h^2 S$: the response to selection is the heritability times the selection differential.[7]

Search papers report the quality of a proxy as a rank correlation, Kendall's τ (Kendall 1938): the probability that two architectures are ordered the same way by the proxy and by the truth, minus the probability that they are ordered oppositely (Proposition A.15). For Gaussian pairs it determines ρ.

[7] The equation goes back to Jay Lush's *Animal Breeding Plans* of 1937; Falconer and Mackay (1996) derive it. Here $S = \sigma_P e_k$, $h^2 = \rho^2$, and $R = \rho \sigma_T e_k$.

Lemma 5.8 (Kendall's τ of a Gaussian pair). *Let (X, Y) be bivariate normal with positive variances and correlation ρ, and let $(X_1, Y_1), (X_2, Y_2)$ be independent copies. Then*

$$\tau = \mathbb{P}\big((X_1 - X_2)(Y_1 - Y_2) > 0\big) - \mathbb{P}\big((X_1 - X_2)(Y_1 - Y_2) < 0\big) = \frac{2}{\pi} \arcsin \rho .$$

Proof. The plan is to reduce to a pair of standard normals and read the answer off an angle. The differences $U = (X_1 - X_2)/\sqrt{2 \operatorname{Var} X}$ and $V = (Y_1 - Y_2)/\sqrt{2 \operatorname{Var} Y}$ are jointly normal with unit variances and, because the copies are independent, correlation ρ; they have the signs of $X_1 - X_2$ and $Y_1 - Y_2$. If $\rho = \pm 1$, then $V = \pm U$ almost surely and $\tau = \pm 1 = \frac{2}{\pi} \arcsin(\pm 1)$. Let $|\rho| < 1$ and $\varphi = \arcsin \rho \in (-\pi/2, \pi/2)$, so that $\sqrt{1 - \rho^2} = \cos \varphi$. The pair (U, V) has the law of $(Z_1, Z_1 \sin \varphi + Z_2 \cos \varphi)$ with independent standard normals Z_1, Z_2; write $(Z_1, Z_2) = \Lambda(\cos \Theta, \sin \Theta)$ in polar coordinates, with Θ uniform on $[0, 2\pi)$ and $\Lambda > 0$. Then $U = \Lambda \cos \Theta$ and $V = \Lambda \sin(\Theta + \varphi)$. Both are positive when $\Theta \in (-\pi/2, \pi/2)$ and $\Theta + \varphi \in (0, \pi)$ modulo 2π, that is, on the arc $(-\varphi, \pi/2)$ of length $\pi/2 + \varphi$, and both are negative on the opposite arc, of the same length. Hence $\mathbb{P}(UV > 0) = \frac{1}{2} + \varphi/\pi$, the event $UV = 0$ has probability zero, and $\tau = 2\mathbb{P}(UV > 0) - 1 = 2\varphi/\pi$. ∎

Inverting, $\rho = \sin(\pi\tau/2)$: $\tau = 0.2$ corresponds to $\rho = 0.31$, and $\tau = 0.5$ to $\rho = 0.71$.

Example 5.9 (Reading published correlations). The weight-sharing ranking of Yu et al. (2020) in their 32-cell recurrent space had $\tau = -0.004$, so $\rho \approx 0$, and Theorem 5.6 predicts no gain over the placebo, which is what they found. In the seven-node NAS-Bench-101 space $\tau = 0.195$ gives $\rho = 0.30$: the search captures 30% of what a perfect search with the same draws would. Two single trainings of the same 32 DARTS cells had $\tau = 0.48$ (Yang et al. 2020), a correlation $\rho_2 = \sin(0.24\pi) = 0.68$. If each training is the true quality plus independent noise, then $\rho_2 = \sigma_{\mathcal{A}}^2/(\sigma_{\mathcal{A}}^2 + \sigma^2)$, and their standard deviation of 0.23 points across cells splits into a spread $\sigma_{\mathcal{A}} \approx 0.19$ and a seed noise $\sigma \approx 0.13$ points. A perfect search of 24 random cells, the size of the DARTS random arm, would gain $\sigma_{\mathcal{A}} e_{24} \approx 0.19 \times 1.95 = 0.37$ points over the average cell; a proxy with $\tau = 0.2$, about 0.11, less than the noise of one training. These are orders of magnitude, resting on a Gaussian model and another group's implementation.

The theorem assumes that the regression of the truth on the proxy is a straight line all the way into the tail where the winners come from. Real

proxies have a second, fixed error, which bends it.

A BIASED PROXY REACHES A PLATEAU. Let a proxy evaluation of a be $\mu(a) + b(a) + \varepsilon$, with a bias $b(a)$ that belongs to the architecture (a short training favors fast learners) and noise ε that is fresh at each evaluation. In a finite space, as the budget grows the draws cover the space and the winner tends to the minimizer of $\mu + b$, perturbed by the selected noise; its expected true error tends to a limit set by how well $\mu + b$ ranks the architectures, not by k, short of the headroom. A strategy that concentrates its evaluations on the proxy's favorites, as evolution does, reaches this plateau sooner, and when the bias is large a better optimizer of the proxy can end with a worse architecture than random search. ◇

REMARK 5.10 (How many searches). The unit of a search experiment is the search. With n searches per arm and r trainings of each returned architecture, an arm's mean has variance $(\sigma_S^2 + \sigma^2/r)/n$, where σ_S is the standard deviation of the true error of what the search returns ($\sigma_S = \sigma_{\mathcal{A}}$ for the placebo). The sample-size formula of Proposition A.14 gives $n \approx 2(z_{0.975} + z_{0.8})^2(\sigma_S^2 + \sigma^2/r)/G^2$ per arm for power 0.8 at level 0.05. With $\sigma_S = 0.2$, $\sigma = 0.13$ and $r = 1$ (Example 5.9), a gain of $G = 0.3$ points needs about ten searches per arm, and a gain of 0.2 points about 22.

5.6 In the laboratory: a tabular benchmark of 192 networks

Architecture search on the digits data is cheap enough to do exhaustively: we train every architecture of a small space several times, store the results in a *tabular benchmark*, and run thousands of searches as table lookups, as NAS-Bench-101 does for CIFAR-10 (Ying et al. 2019). The data are labkit's split 0 of the scikit-learn digits; its 1258 training images are divided once into 958 for training and 300 for validation, and the 539 test images, each worth 0.19 points of test error, are used only to define the truth.

LAB 5.1 · THE SPREAD OF A SEARCH SPACE

SETUP. Every MLP with depth 1 to 4, width 16, 32, 64 or 128, activation ReLU, tanh or GELU, dropout 0 or 0.25, and batch normalization on or off: 192 architectures. One fixed pipeline: Adam with learning rate 10^{-3}, batch 64, 30 epochs at a constant learning rate, on the 958 training images. Each architecture is trained six times (seeds 2000–2005) and scored on the test images; the mean of the six is its true error. 1152 runs, three to four minutes.

PREDICT FIRST. How far is the average architecture from the best? Is the spread between architectures larger or smaller than the spread between two seeds of one architecture? Answer for the full space and for the 96 architectures of width 64 or 128.

LOOK FOR. The shape of the ranked curve, the overlap of the intervals near the top, and where the wide architectures sit.

SCRIPT
`05-controls-space.py`

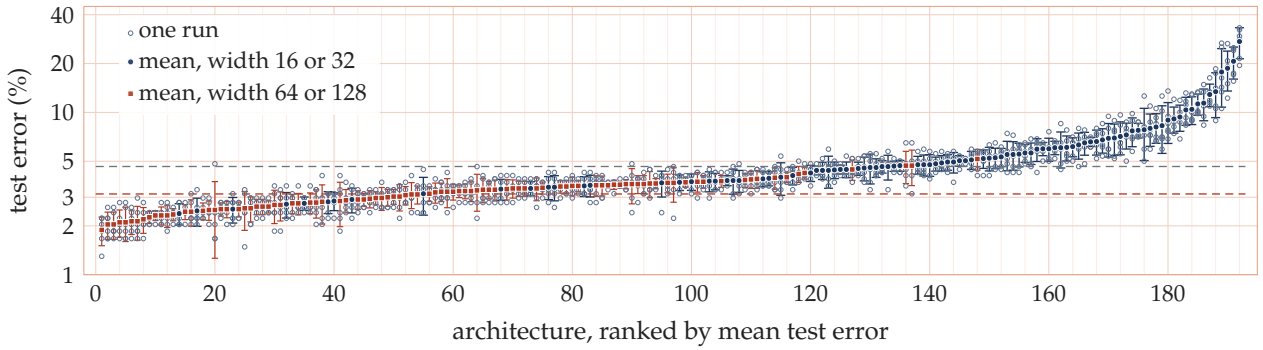


Figure 5.1. The spread of the search space of Lab 5.1: 192 MLP architectures for the digits data, ranked by mean test error over six runs (seeds 2000–2005, Adam, 30 epochs). Hollow marks: the six runs of each architecture. Filled marks: means with 95% t-intervals, vermilion squares for the 96 curated architectures (width 64 or 128) and ink circles for the rest. Dashed lines: the expected error of a randomly sampled architecture, 4.64% in the full space (grey) and 3.14% in the curated space (vermilion). Logarithmic vertical axis.

WHAT THE SPACE LOOKS LIKE. Figure 5.1 shows two spaces in one. The full space has a long tail: its mean true error is 4.64%, the standard deviation of its means 3.18 points, and its worst architecture, four layers of 16 ReLU units with dropout, errs on 27.33% of the test images. Width decides most of it (mean error 7.99% at width 16, 2.91% at width 128), then batch normalization and dropout. The best architecture, two layers of 128 GELU units with batch normalization, reaches 1.89 ± 0.38% (mean and 95% t-interval over six runs), so the headroom of the full space is 2.76 points. The 96 wide architectures form the kind of space a designer writes down first: its mean is 3.14%, its headroom 1.26 points, and the random-effects estimate of its spread, $\sigma_{\mathcal{A}} = 0.62$ points, is about 1.3 times the seed noise within an architecture, $\sigma = 0.47$ points; in the DARTS space the ratio was about $0.19/0.13 \approx 1.5$ (Example 5.9). Two single runs rank the curated architectures with Kendall $\tau = 0.52$ on average (0.39 to 0.66 over the fifteen pairs of seeds), against 0.48 in the DARTS space, and eleven of the 95 other curated architectures cannot be told apart from the best by a Welch test at level 0.05.

LAB 5.2 · A SEARCH, ITS SHAM AND ITS CONTROLS

SCRIPT
`05-controls-search.py`

SETUP. Four more runs of each architecture (seeds 1000–1003) record the validation error after every epoch: the proxy of fidelity f is the error after f epochs, and the main proxy is $f = 2$. An evaluation draws one of the four runs at random, as NAS-Bench-101 does. Five arms, each repeated 2000 times with $k = 1, 2, 4, \ldots, 64$ evaluations: the placebo, random search (the sham), regularized evolution (population 10, tournament 3, one-attribute mutations), the oracle (best of k draws by true error, a positive control) and random search with a shuffled proxy (a negative control). A search is scored by the true error of what it returns. 768 runs, about three minutes.

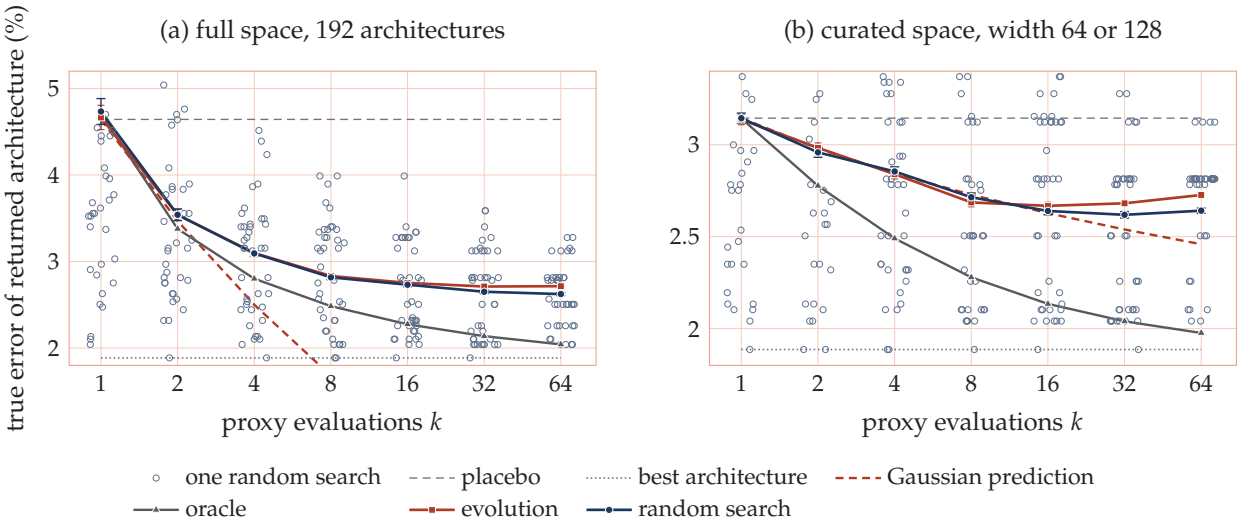


Figure 5.2. Searches on the benchmark of Lab 5.2, guided by the two-epoch proxy (four runs per architecture, seeds 1000–1003). Vertical axis: the true error (mean of six test runs) of the architecture a search returns. Lines with marks: means over 2000 searches with 95% intervals (most smaller than the marks), for random search (ink), regularized evolution (vermilion) and the oracle that selects by true error (grey). Hollow marks: 40 single random searches at each k, spread sideways. Dashed grey: the placebo; dotted: the best architecture; dashed vermilion: Theorem 5.6 for random search, from the space's mean and standard deviation and the proxy's correlation with the truth ($\rho = 0.66$ in the full space, 0.45 in the curated one). The shuffled-proxy search (not drawn) stays at the placebo level.

PREDICT FIRST. In the curated space the two-epoch proxy has Kendall $\tau = 0.32$ with the truth. What fraction of the oracle's gain will random search capture with 16 evaluations? Will evolution beat random search?

LOOK FOR. Where each curve flattens, the gap between random search and the oracle, and what the shuffled search reports about itself.

THE PROXY. In the curated space the two-epoch proxy has Kendall $\tau = 0.32$ with the truth (Pearson $\rho = 0.45$); one epoch gives 0.20, ten epochs 0.46, and the thirty-epoch validation error 0.56, close to the 0.52 at which two full test runs agree. Its bias shows in its favorite: averaged over its four runs, the two-epoch proxy prefers four layers of 128 tanh units with batch normalization, a fast learner that is 33rd of 96 by true error (2.81%), while the thirty-epoch validation error prefers three layers of 128 GELU units with batch normalization, which is second (2.04%). In the full space the proxies rank better ($\tau = 0.57$ at two epochs), because the narrow networks of the tail are bad early and late.

THE SEARCH AGAINST ITS CONTROLS. In the full space (Figure 5.2) every search with an informative proxy beats the placebo by about two points, because a single random draw lands too often in the tail: with 16 evaluations random search returns architectures with true error $2.73 \pm 0.02\%$ (mean

and 95% interval over 2000 searches) against the placebo's 4.64%. In the curated space the gain is about half a point, 2.64 ± 0.02% against 3.14%, the size of one run's seed noise, and single searches vary around that mean with a standard deviation of 0.47 points. In both spaces random search levels off near 2.6%, while the oracle goes on to 2.13% at $k = 16$ and 1.98% at $k = 64$ in the curated space (intervals below 0.01 points): the plateau of the biased-proxy heuristic. Evolution, the more capable optimizer, does not beat the sham. Its architectures are worse than those of random search by 0.03 ± 0.03, 0.06 ± 0.02 and 0.09 ± 0.02 points at $k = 16$, 32 and 64 in the curated space (0.02 ± 0.03, 0.06 ± 0.03 and 0.09 ± 0.02 in the full space): it finds the proxy's favorites faster, and they are fast learners. Given a proxy that ranks better, evolution wins by a little: with the ten- and thirty-epoch proxies it is ahead in seven of the eight comparisons we ran, by 0.02 to 0.07 points (with the thirty-epoch proxy and 64 evaluations, 2.14% against 2.21% in the curated space, a difference of 0.07 ± 0.01 points).

THE THEOREM. In the curated space Theorem 5.6 predicts random search well while the budget is small: 2.98%, 2.84%, 2.73% and 2.63% for $k = 2$, 4, 8 and 16, against 2.96%, 2.85%, 2.71% and 2.64% observed. Beyond that the prediction keeps falling (2.46% at $k = 64$) and the observations do not: the smallest proxy values belong to the fast learners, and the straight regression line fails in the tail. In the full space the Gaussian model fails from the start, predicting 1.68% at $k = 8$, below the best architecture, against 2.82% observed: the long tail inflates the standard deviation to 3.18 points while the headroom is 2.76, and Remark 5.3 is the bound that holds. Figure 5.3 varies the proxy instead of the budget. In the curated space the gain of random search with 16 evaluations, divided by the perfect-proxy gain $\sigma_{\mathcal{A}} e_{16}$, follows the line ρ from the shuffled proxy ($\rho = 0$, gain 0.00 ± 0.03 points, a 95% interval over 2000 searches) through the one-epoch proxy ($\rho = 0.28$, 0.39 ± 0.02 points against a prediction of 0.32) to full training ($\rho = 0.71$, 0.82 ± 0.01 against 0.81); in the full space the points lie at 0.33 to 0.42 of $\sigma_{\mathcal{A}} e_{16}$ whatever the proxy.

WHAT THE SEARCH SAYS ABOUT ITSELF. With the thirty-epoch proxy, whose values are validation errors on the scale of the truth, random search with eight evaluations in the curated space reports a gain of 1.64 points over the average proxy score and achieves 0.68: a ratio of 2.39, somewhat more than the $1/\rho^2 = 2.01$ of Corollary 5.7. The negative control needs no model: with the proxy values shuffled among the architectures, random search with 16 evaluations achieves -0.03 ± 0.03 points (95% interval) and reports 2.09 (in the full space, -0.04 ± 0.14 and 3.44). A search's report is no evidence until it is measured against a control.

LAB 5.3 · A HISTORICAL CONTROL

SCRIPT
`05-controls-pipeline.py`

SETUP. A 2×2 factorial: the textbook MLP (two layers of 64 ReLU units) and the architecture found in Lab 5.2 (the curated architecture with the lowest validation error after full training), crossed with the

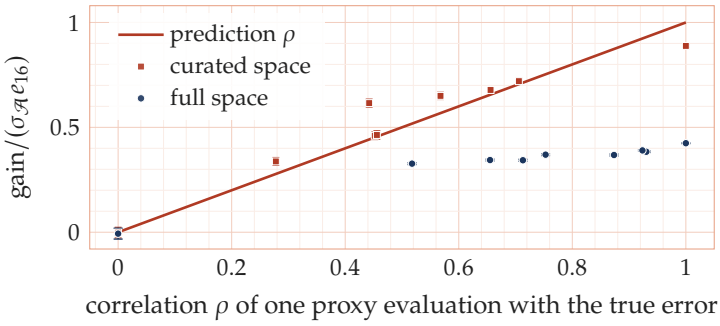


Figure 5.3. The proxy theorem across proxies. Each point is random search with $k = 16$ evaluations on the benchmark of Lab 5.2 (2000 searches, 95% interval): its gain over the placebo divided by $\sigma_{\mathcal{A}} e_{16}$ (1.14 points in the curated space, 5.61 in the full space), against the Pearson correlation of one proxy evaluation with the true error. From left to right: the shuffled proxy, the validation error after 1, 2, 3, 5, 10, 20 and 30 epochs, and the true error itself (the oracle). The line is Theorem 5.6.

Table 5.1. Lab 5.3: test error (%), mean and 95% t-interval; ten runs per architecture and pipeline. The placebo row averages 16 random curated architectures, two runs each, with an interval over the 16.

	old pipeline	new pipeline
textbook MLP (2×64, ReLU)	3.19 ± 0.23	2.00 ± 0.22
found (3×128, GELU, batch norm.)	2.04 ± 0.34	1.69 ± 0.22
placebo (random curated)	2.97 ± 0.29	2.32 ± 0.29

old pipeline of Lab 5.1 and a new one (Adam with learning rate $2{\times}10^{-3}$, cosine schedule, 60 epochs, label smoothing 0.1); ten runs per cell, seeds 3000–3009. A placebo arm: 16 random curated architectures, two runs each, under both pipelines. 104 runs, under a minute.

PREDICT FIRST. A paper reports the found architecture with the new pipeline and copies the textbook MLP's number from an older paper that used the old one. How much of the reported improvement belongs to the search?

LOOK FOR. The pipeline effect on each architecture, and the placebo arm under the new pipeline.

THE HISTORICAL COMPARISON. The paper of the lab sheet would report an improvement of 1.50 ± 0.30 points (Table 5.1; 95% Welch intervals for all differences here). The new pipeline alone improves the textbook MLP by 1.19 ± 0.30 points; under the new pipeline the textbook MLP is better than the average random architecture by $2.32 - 2.00 = 0.32$ points; and the found architecture beats the concurrent placebo by 0.63 ± 0.35 points. These are the three terms of (5.1), $1.19 - 0.32 + 0.63 = 1.50$: the search's own share is less than half of what the historical comparison credits to it. Pipeline and architecture also interact: the new pipeline helps the textbook MLP by 1.19

points but the found architecture by only 0.35 ± 0.38, so the contribution of the architecture depends on the pipeline at which it is measured (1.15 ± 0.38 points under the old one, 0.32 ± 0.29 under the new), the order dependence of Chapter 6. In miniature this is the finding of Yang et al. (2020).

What the miniature reproduces and what it does not. The benchmark reproduces the mechanism: a proxy that ranks poorly holds every search to a plateau, a better optimizer of a biased proxy does not help, a search's report overstates its gain, and a historical control mixes the pipeline into the search. It does not reproduce the near-zero margin over the placebo: our curated space is relatively wider than the DARTS space (headroom 40% of the mean error, against about 18% for the 214 cells of Yang et al. (2020)), and our short-training proxy ranks better than shared weights did, so random search captures about half of the oracle's gain. Weaker proxies move the gain toward the placebo along the line of Figure 5.3; we did not implement weight sharing.

5.7 Protocol

PROTOCOL 5.1 · CONTROLS FOR A SEARCH METHOD

- ☐ Fix the search space, its sampling distribution, the proxy, the budget and the final pipeline before searching, and use them in every arm.
- ☐ Run a placebo arm: train at least ten architectures drawn from the space with the final pipeline, and report their mean, standard deviation and best, the spread and headroom of the space.
- ☐ Run a sham arm: random search with the same proxy, budget and selection rule.
- ☐ Repeat every search with independent seeds, as often as Remark 5.10 requires for the gain you claim, and report the distribution of what the searches return, not the best search.
- ☐ Score every returned architecture by retraining it with fresh seeds on test data never used for selection; never report the winner's proxy score as its quality.
- ☐ Measure the proxy: report Kendall's τ between the proxy and full training on a sample of architectures, beside the τ between two seeds of full training.
- ☐ Run a positive control (an oracle or a planted better architecture, which must be detected) and a negative control (a shuffled proxy, which must show no gain).
- ☐ Use published numbers only as context; claim gains only against concurrent arms, with and without training tricks.

- ☐ Release the space, the pipeline, the seeds and every sampled architecture with its scores.

The protocol measures a search within a space, not the space: a search with no placebo-adjusted gain in an excellent space may still be the right product, and a large gain over a poor placebo is worth little, since a designer can always widen a space with bad options, as our full space shows. Transfer is outside its reach: a cell searched on CIFAR-10 and deployed on ImageNet needs controls at the scale of deployment. And it is expensive when a search is: at a GPU-day per search, ten searches in each of three arms cost a month. Then the comparison of strategies belongs on a tabular or surrogate benchmark, and the full experiment can be reduced to the placebo arm, which has no search cost, the proxy correlation, and enough trainings of the final architecture to state its error with an interval.

Exercises

5.1 CONSOLIDATION. (Expected rank of the best of k.) In a space of S architectures ranked by true error, let ν_k be the rank of the best of k uniform draws with replacement. (a) Show that $\mathbb{P}(\nu_k \geq r) = \big((S-r+1)/S\big)^k$ and $\mathbb{E}\nu_k = \sum_{j=1}^{S}(j/S)^k \leq 1 + S/(k+1)$, and compute $\mathbb{E}\nu_k$ for $S = 96$ and $k = 16$. (b) Compare with the true rank of what random search returned in Lab 5.2, and explain the difference with Lemma 5.5.

5.2 CONSOLIDATION. (What survives without normality.) (a) Show with Lemma 5.5 that if $\mathbb{E}[T \mid P] = \alpha + \beta P$, then every proxy-based selection among independent pairs has $\mathbb{E}T_J = \mathbb{E}T + \beta(\mathbb{E}P_J - \mathbb{E}P)$. (b) Let $P \sim \mathcal{N}(0,1)$ and $T = P^2$. Show that $\mathrm{Corr}(T,P) = 0$, that for $k \geq 3$ keeping the smallest P_i gives $\mathbb{E}T_J > \mathbb{E}T$ (equality for $k = 2$), and that keeping the smallest $|P_i|$ gives $\mathbb{E}T_J < \mathbb{E}T$. What does this say about summarizing a proxy by one correlation?

5.3 CONSOLIDATION. (The winner's curse in a null space.) In a space whose architectures are all equally good ($\sigma_T = 0$), a search with the additive proxy $P = T + \xi$, $\xi \sim \mathcal{N}(0, s^2)$, keeps the smallest of k proxy values. Show that it reports a gain of $s\,e_k$ and achieves none, compute the report for $s = 0.47$ points and $k = 16$, and relate it to the best-of-k reporting of Chapter 1.

5.4 CONSOLIDATION. ★ (Two seeds and one truth.) In the additive model $P_j = T + \xi_j$ with independent $\xi_1, \xi_2 \sim \mathcal{N}(0, s^2)$, show that $\mathrm{Corr}(P_1, P_2) = \mathrm{Corr}(T, P_1)^2$, and check it on the curated space, where two seeds give $\tau = 0.52$ and the thirty-epoch proxy has $\rho = 0.71$. *(Hint: the proxy is a validation error and the truth a six-run test mean, so expect agreement in order of magnitude only.)*

5.5 LAB. (Make the proxy better or worse.) (a) In `05-controls-search.py` set `FID_MAIN` to 1, 5 and 30 and record the gains of random search and

of evolution over the placebo at $k = 64$ in the curated space. At which fidelity does evolution start to win? Explain with the biased-proxy heuristic. (b) At equal compute, compare 60 two-epoch evaluations with 4 thirty-epoch ones in both spaces. Why does the answer differ between the spaces?

5.6 LAB. (A weaker proxy.) Add a proxy computed from one epoch on half the training images to Figure 5.3. Does it fall on the line?

5.7 LAB. (Widen the space.) Add width 8 and dropout 0.5 to the space of Lab 5.1. Predict, then measure, how the placebo, random search and the oracle move. Which control does the new tail affect, and which arm of a comparison does it flatter?

5.8 DESIGN. (Controls for data pruning.) A method keeps 30% of a training set, and its authors claim that training on the kept subset beats training on a random 30%. Design the placebo, the sham when there is one, a positive control and a negative control, with the number of runs per arm, for a budget of 200 training runs.

5.9 DESIGN. (A search experiment under a budget.) A search costs one GPU-day, a final training half a GPU-day, and you have 40 GPU-days. You expect a gain of 0.3 points over random sampling, with single trainings varying by 0.15 points and returned architectures by 0.2. Allocate the budget among searches, placebo draws and final trainings, and state the power of your comparison.

5.10 CRITIQUE. (A table without a random row.) A paper reports its search method's test error ($2.61 \pm 0.05\%$ over five trainings of its best cell) beside the published errors and search costs of six earlier methods, with no random architectures, and says the method "outperforms all baselines at a fraction of the cost". List what the table cannot show, and the fewest extra runs that would make the claim testable. (The description is invented.)

5.11 CRITIQUE. (A historical baseline.) A paper reports 1.8% error for a new architecture with its pipeline (cosine schedule, 300 epochs, augmentation) against 2.6% "reported for the baseline in its original paper" (step schedule, 90 epochs). Write the decomposition (5.1) for this comparison, say which terms the paper has measured, and design the 2×2 experiment that measures the others. (The description is invented.)

5.12 RESEARCH. (When can a search beat random by more than noise?) Theorem 5.6 describes random search; adaptive searches can exploit locality, the similarity of neighbors in the space, which Ying et al. (2019) measured in NAS-Bench-101. Open: a model of a space with locality, a proxy with noise and a fixed bias, and an adaptive strategy, in which the advantage over random search with the same proxy can be computed, with conditions under which it exceeds the noise of one

final training.

Notes and References

Architecture search grew out of hyperparameter optimization and neuroevolution; the survey of Elsken et al. (2019) organizes it, as this chapter does, by search space, search strategy and performance estimation, our proxy, and Zela et al. (2020) explained and repaired the failures of DARTS.

Lindauer and Hutter (2020) turned the lessons of the reality checks into a checklist, and the tabular benchmarks NAS-Bench-101 (Ying et al. 2019) and NAS-Bench-201 (Dong and Yang 2020) made the controls nearly free. Measuring the space itself is the idea of Radosavovic et al. (2019), who compare design spaces by the error distributions of models sampled from them: the placebo arm promoted from a baseline to an instrument.

The vocabulary of controls is borrowed from clinical trials (International Conference on Harmonisation 2000; Sacks et al. 1982) and epidemiology (Lipsitch et al. 2010), and selection on a noisy measurement is old in quantitative genetics (Falconer and Mackay 1996).

TRANSLATING THE LITERATURE. What Lindauer and Hutter (2020) call random sampling and random search are our placebo and sham arms. The "random policy" of Yu et al. (2020) is our placebo repeated over seeds, and their $p(>\text{random})$ is the probability that a searched architecture beats one random architecture. The relative improvement $100(\text{Acc}_m - \text{Acc}_r)/\text{Acc}_r$ of Yang et al. (2020) is our gain G in accuracy, in percent of the placebo's accuracy. The test regret of Ying et al. (2019) is our $\mu(A_S) - \mu^\star$ in accuracy, and their epoch budget is our proxy fidelity f. In the breeder's equation $R = h^2 S$ of Falconer and Mackay (1996), R is our true gain, S the selection differential of the proxy ($\sigma_P e_k$ for the best of k) and h^2 our ρ^2 in the additive model; their selection intensity i is e_k when one of k is kept.

References

Bergstra, J. and Y. Bengio (2012). "Random Search for Hyper-Parameter Optimization". In: *Journal of Machine Learning Research* 13, pp. 281–305.

Dong, X. and Y. Yang (2020). "NAS-Bench-201: Extending the Scope of Reproducible Neural Architecture Search". In: *International Conference on Learning Representations*.

Elsken, T., J. H. Metzen, and F. Hutter (2019). "Neural Architecture Search: A Survey". In: *Journal of Machine Learning Research* 20.55, pp. 1–21.

Falconer, D. S. and T. F. C. Mackay (1996). *Introduction to Quantitative Genetics*. 4th ed. Harlow: Longman.

International Conference on Harmonisation (2000). *Choice of Control Group and Related Issues in Clinical Trials (E10)*. ICH harmonised tripartite guideline, Step 4, July 2000.

Kendall, M. G. (1938). "A New Measure of Rank Correlation". In: *Biometrika* 30.1–2, pp. 81–93.

Li, L. and A. Talwalkar (2019). "Random Search and Reproducibility for Neural Architecture Search". In: *Proceedings of the 35th Conference on Uncertainty in Artificial Intelligence (UAI 2019)*. Vol. 115. Proceedings of Machine Learning Research, pp. 367–377.

Lindauer, M. and F. Hutter (2020). "Best Practices for Scientific Research on Neural Architecture Search". In: *Journal of Machine Learning Research* 21.243, pp. 1–18.

Lipsitch, M., E. Tchetgen Tchetgen, and T. Cohen (2010). "Negative Controls: A Tool for Detecting Confounding and Bias in Observational Studies". In: *Epidemiology* 21.3, pp. 383–388.

Liu, H., K. Simonyan, and Y. Yang (2019). "DARTS: Differentiable Architecture Search". In: *International Conference on Learning Representations*.

Lucic, M., K. Kurach, M. Michalski, S. Gelly, and O. Bousquet (2018). "Are GANs Created Equal? A Large-Scale Study". In: *Advances in Neural Information Processing Systems 31*.

Melis, G., C. Dyer, and P. Blunsom (2018). "On the State of the Art of Evaluation in Neural Language Models". In: *International Conference on Learning Representations*.

Musgrave, K., S. Belongie, and S.-N. Lim (2020). "A Metric Learning Reality Check". In: *Computer Vision – ECCV 2020*. Vol. 12370. Lecture Notes in Computer Science. Springer, pp. 681–699.

Pham, H., M. Y. Guan, B. Zoph, Q. V. Le, and J. Dean (2018). "Efficient Neural Architecture Search via Parameter Sharing". In: *Proceedings of the 35th International Conference on Machine Learning*. Vol. 80. Proceedings of Machine Learning Research, pp. 4095–4104.

Radosavovic, I., J. Johnson, S. Xie, W.-Y. Lo, and P. Dollár (2019). "On Network Design Spaces for Visual Recognition". In: *IEEE/CVF International Conference on Computer Vision*, pp. 1882–1890.

Real, E., A. Aggarwal, Y. Huang, and Q. V. Le (2019). "Regularized Evolution for Image Classifier Architecture Search". In: *Proceedings of the AAAI Conference on Artificial Intelligence*. Vol. 33, pp. 4780–4789.

Sacks, H., T. C. Chalmers, and H. Smith (1982). "Randomized versus Historical Controls for Clinical Trials". In: *The American Journal of Medicine* 72.2, pp. 233–240.

Yang, A., P. M. Esperança, and F. M. Carlucci (2020). "NAS Evaluation is Frustratingly Hard". In: *International Conference on Learning Representations*.

Ying, C., A. Klein, E. Christiansen, E. Real, K. Murphy, and F. Hutter (2019). "NAS-Bench-101: Towards Reproducible Neural Architecture Search". In: *Proceedings of the 36th International Conference on Machine Learning*. Vol. 97. Proceedings of Machine Learning Research, pp. 7105–7114.

Yu, K., C. Sciuto, M. Jaggi, C. Musat, and M. Salzmann (2020). "Evaluating the Search Phase of Neural Architecture Search". In: *International Conference on Learning Representations*.

Zela, A., T. Elsken, T. Saikia, Y. Marrakchi, T. Brox, and F. Hutter (2020). "Understanding and Robustifying Differentiable Architecture Search". In: *International Conference on Learning Representations*.

Zoph, B. and Q. V. Le (2017). "Neural Architecture Search with Reinforcement Learning". In: *International Conference on Learning Representations*.

Zoph, B., V. Vasudevan, J. Shlens, and Q. V. Le (2018). "Learning Transferable Architectures for Scalable Image Recognition". In: *IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 8697–8710.

6 Confounding

The architecture and the recipe

THE CLAIM *"Vision transformers are better image classifiers than convolutional networks."*

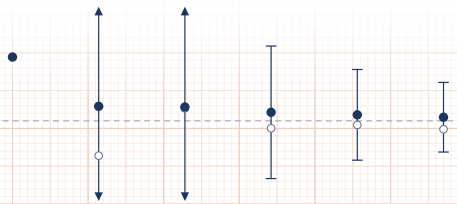
THE VERDICT Confounded: trained with the same modern recipe, convolutional networks come within about a point of vision transformers of similar size, and which one leads depends on whose recipe it is.

The received reading of Dosovitskiy et al. (2021) and Touvron et al. (2021)

IN THIS CHAPTER. In 2020 the vision transformer matched the best convolutional networks after pretraining on 300 million images, and DeiT trained it on ImageNet alone and placed it above the ResNets in a table whose rows each carried the recipe of their own paper. Given the newer recipes, ResNet-50 rose from 76.1% to 80.4%, within about a point of a transformer of its size, ahead or behind according to whose recipe both used. The method is the factorial design: crossing architecture with recipe splits the naive gain into an architecture effect, a recipe effect and their interaction, and we prove that a sequence of changes credits each change independently of the order exactly when the interactions involving it vanish, while the Shapley attribution averages over the orders. In the laboratory a small convolutional network beats an MLP on the digits by 2.0 points when it also gets the newer recipe, half of it the recipe's; with noisy labels the same comparison credits it with 4.3 points although it is the worse network under the old recipe and no better under the new.

6.1 The claim: an image as sixteen-by-sixteen words

In October 2020 twelve researchers at Google Brain posted a paper whose title stated its method: an image is worth sixteen by sixteen words (Dosovitskiy et al. 2021). Their *vision transformer* (ViT) cuts an image into patches of 16×16 pixels, maps each patch linearly to a vector, adds a learned position embedding and feeds the sequence to a standard Transformer encoder; apart from the patches, nothing in it knows that its input is an image. A convolutional network builds in two assumptions, that nearby pixels belong together and that a pattern means the same wherever it appears, and convolutional networks held the state of the art in large-scale recognition, among them BiT, large ResNets pretrained on hundreds of millions of images (Kolesnikov et al. 2020).



FINDING 6.1 (Dosovitskiy et al. 2021, Table 2 and Section 4.4). Pretrained on JFT-300M, an internal set of 300 million labeled images, and fine-tuned on ImageNet, ViT-H/14 reached 88.55% top-1 accuracy and ViT-L/16 87.76%, against 87.54% for BiT-L, a ResNet pretrained on the same data, and 88.4–88.5% for Noisy Student, an EfficientNet; pretraining took 2.5 and 0.68 thousand TPUv3-core-days for the two transformers against 9.9 and 12.3 thousand. In a scaling study that trained ResNets, transformers and hybrids with one procedure on JFT-300M, the transformers needed two to four times less compute than the ResNets for the same transfer accuracy.

The paper qualified its result in both directions.[1] Trained on ImageNet alone, without strong regularization, its transformers were a few points less accurate than ResNets of comparable size, which the authors attributed to the inductive biases that transformers lack; with 14 to 300 million images, they concluded, "large scale training trumps inductive bias". And they noted that pretraining efficiency may depend on the training schedule, the optimizer and the weight decay as well as on the architecture, and pointed to their controlled scaling study.[2]

Two months later Hugo Touvron and colleagues at Facebook AI trained the same architecture on the 1.28 million images of ImageNet-1k alone, on one eight-GPU machine in two to three days (Touvron et al. 2021). Their training procedure built on the timm library: 300 epochs of AdamW with a cosine schedule, the augmentations RandAugment, Mixup, CutMix, random erasing and repeated augmentation, and the regularizers stochastic depth and label smoothing. Trained this way, the ViT-B architecture reached 81.8% at a resolution of 224 pixels and 83.1% at 384, where the ViT paper had reported 77.9% for the same architecture trained on ImageNet alone at 384 pixels.[3]

FINDING 6.2 (Touvron et al. 2021, Table 5). Trained on ImageNet-1k only, DeiT-S (22 million parameters) reached 79.8% top-1 accuracy and DeiT-B (86 million) 81.8%, against 76.2% for ResNet-50 (25 million) and 78.3% for ResNet-152 (60 million) in the same table. With an extra distillation token and a convolutional network as teacher, DeiT-B reached up to 85.2%.

The field read these results as a verdict on the architectures. Within a year Swin, a transformer with attention inside shifted local windows, reached 81.3% at about the size of a ResNet-50 and received the best-paper prize of ICCV 2021 (Liu et al. 2021). Looking back in 2022, the authors of ConvNeXt described vision transformers as having quickly superseded convolutional networks, and the success of the hierarchical transformers, which had brought back convolution-like locality, as still "largely credited to the intrinsic superiority of Transformers" (Liu et al. 2022). A note of 2023 described the belief of many researchers that convolutional networks do well on moderate data but cannot compete with transformers at web scale (Smith et al. 2023). The received claim was that attention makes a better image classifier than convolution, whose built-in assumptions help only

[1] JFT-300M is internal to Google. On the public ImageNet-21k, with 14 million images, ViT-L/16 reached 85.30% after fine-tuning on ImageNet.

[2] The study trained every model with Adam and a weight decay of 0.1, for 7 or 14 epochs of JFT-300M; the paper reports that Adam worked slightly better than SGD for its ResNets in this setting.

[3] DeiT's distillation token is an extra input whose output is trained to match a teacher's predictions. With a RegNetY-16GF teacher at 82.9%, DeiT-B reached 83.4% at 224 pixels, more than its teacher.

when data are scarce.

The original work deserves most of its credit. That a standard transformer, with almost nothing designed for images, is a strong image classifier had not been shown before, and an architecture shared by text and images has practical advantages of its own (Smith et al. 2023). The ViT paper hedged its compute comparison and ran a controlled study, a design this chapter recommends. DeiT published its recipe and code, marked the convolutional networks it had trained with its own procedure, distilled its best models from a convolutional teacher, and claimed that its transformers were competitive, not that attention beat convolution. The stronger reading was reasonable in 2021: transformers had replaced recurrent networks in language, ViT's scaling curves showed no saturation, and tables of published systems were how architectures had always been compared.[4]

[4] Bello et al. (2021) give an example of the practice: new architectures with modern training compared with a ResNet-50 at 76.5%, the accuracy of its 2015 training.

6.2 The experiment: one table, two kinds of difference

The comparisons behind the received claim had the form of the DeiT table: each row a system, that is, an architecture trained by the recipe of its own paper.

EXPERIMENTAL DESIGN · A TABLE OF SYSTEMS, AS IN TOUVRON ET AL. 2021

QUESTION	Is a convolution-free transformer a better image classifier than a convolutional network of similar size?
UNITS	Training runs on ImageNet-1k (1.28 million images, 1,000 classes)
TREATMENT	The architecture: DeiT, a ViT, against ResNets, RegNetYs and EfficientNets
CO-TREATMENTS	Each row's recipe: for DeiT, 300 epochs of AdamW with strong augmentation and regularization; for the ResNets, a standard training of the period (ResNet-50 at 76.2%, close to the 76.1% of the 90-epoch SGD recipe of torchvision); for the RegNetYs, a procedure like DeiT's; for the EfficientNets, their own, at larger resolutions
HELD FIXED	The training data; top-1 accuracy on the validation set; throughput measured in one code base on one GPU
RESPONSE	Top-1 accuracy and images per second
REPLICATES	One run per row
ANALYSIS	Rows compared at similar size or throughput, for instance DeiT-S at 79.8% against ResNet-50 at 76.2%

What could this design show? Each row measures a system, and as a measurement it is sound: DeiT-S trained as DeiT trains is more accurate than ResNet-50 trained with a recipe five years older. The design cannot say why. The two rows differ in the architecture and in everything the recipe sets, the optimizer, the number of epochs (300 against 90), the augmentation and the regularization; the ResNet-50 row is a historical control in the sense of Chapter 5. The table contains one concurrent control, and its authors

marked it: the RegNetY rows were trained with a procedure like DeiT's, and with it RegNetY-4GF (21 million parameters) reached 80.0%, ahead of DeiT-S, and RegNetY-16GF (84 million) 82.9%, ahead of DeiT-B in accuracy and in throughput. The DeiT paper rested its case on distillation and on the trade-off between accuracy and throughput, and said so; the received reading took the comparison with the ResNet rows.

The controlled study of the ViT paper is the better design for its own question: one procedure for every architecture, the same data, and compute on the horizontal axis. Its limit is that one procedure is one level of the recipe. If the best recipe for a ResNet differs from the best for a transformer, then a comparison at one fixed recipe measures the effect of the architecture at that recipe and at no other. Whether a difference between two rows belongs to the architecture or to the recipe, and whether it changes with the recipe at which it is measured, can be decided only by a design in which both factors vary independently. Neither paper ran one.

6.3 The post-mortem: the baseline gets the recipe

THE RECIPE MOVES THE NUMBERS. From 2019 on, several groups retrained old architectures with new recipes. He et al. (2019) raised ResNet-50 from 75.3% to 79.29% with training refinements (a cosine schedule, label smoothing, Mixup), a modified downsampling block and distillation from a larger ResNet. Wightman et al. (2021), in a workshop paper, found the accuracy reported for the plain ResNet-50 ranging from 75.2% to 79.5% across papers, and trained it with three procedures of their own, built on the LAMB optimizer, a binary cross-entropy loss, Mixup, CutMix and RandAugment: 80.4% after 600 epochs, 79.8% after 300 and 78.1% after 100, against 76.1% for the weights distributed with torchvision. Over 100 seeds the 300-epoch procedure had a standard deviation of 0.10 points. Bello et al. (2021) raised ResNet-200 from 79.0% to 82.2% by training and regularization alone and to 83.4% with two small architectural changes, three quarters of the gain from training. The recipe moved the transformers as well: DeiT's moved ViT-B from 77.9% to 83.1% (Finding 6.2); a note by three authors of ViT moved ViT-S/16 from 66.8% with the original setup to 76.5% in 90 epochs and 80.0% in 300, with standard augmentation and minor changes (Beyer et al. 2022); and augmentation and regularization, given more compute, did as well as an order of magnitude more pretraining data (Steiner et al. 2022).

THE SAME RECIPE FOR BOTH. Trained with the 600-epoch procedure that Wightman et al. tuned for ResNet-50, ResNet-50 reached 80.4%, a ViT of DeiT-S's size 80.6% and one of DeiT-B's size 80.4%, below the 81.8% of DeiT's own recipe; the authors note that their procedure was not adapted to the larger models. They also ran the comparison that the tables had not.

> FINDING 6.3 (Wightman et al. 2021, Section 4.4). ResNet-50 and DeiT-S were each trained with two 300-epoch procedures with the same batch size: A2, optimized for ResNet-50, and T2, optimized for DeiT-S. On the

> ImageNet validation set ResNet-50 reached 79.9% with A2 and 79.2% with T2, DeiT-S 79.6% and 80.4%: each architecture won under the procedure designed for it. On ImageNet-V2, a second test set, DeiT-S led under both, by 0.2 and 1.3 points.

Each of these numbers is one run, and the seed study puts their standard deviation near 0.1 points on the validation set, 0.23 on ImageNet-V2. The authors drew the lesson that training two architectures with the same procedure is not enough to compare them, because a procedure tuned for one favors it.

A ResNet modernized one step at a time. Liu et al. (2022) trained ResNet-50 with a fixed recipe close to DeiT's and Swin's, which by itself raised it from 76.1% to 78.8%, and then moved the architecture one step at a time toward Swin's design at about Swin-T's compute, averaging each accuracy over three seeds: Swin's stage ratio (79.4%), a patchify stem (79.5%), wider depthwise convolutions (80.5%), an inverted bottleneck (80.6%), the depthwise convolution moved up in the block (79.9%, a loss that prepared the next step), 7×7 kernels (80.6%), GELU (no change), fewer activations (81.3%) and normalizations (81.4%), layer normalization (81.5%) and separate downsampling layers (82.0%), against 81.3% for Swin-T. The authors set out to identify the confounding variables of the comparison, and concluded that much of the difference between the families may come from the training techniques.[5]

[5] The step to separate downsampling layers diverged until normalization layers were added where the resolution changes: one step's value depended on another, an interaction of Section 6.5.

At scale. Smith et al. (2023), in an unrefereed note, pretrained NFNets, convolutional networks without normalization layers, on JFT-4B, about four billion labeled images, with budgets from 0.4 to 110 thousand TPU-v4 core hours. The held-out loss fell as a power of the compute, and after fine-tuning the largest model reached 90.3% on ImageNet (90.4% with repeated augmentation in fine-tuning), against 90.2% reported for ViT-g/14 (Zhai et al. 2022), whose pretraining they estimated at 120 thousand TPU-v4 core hours. The comparison is with published numbers obtained on JFT-3B, a different dataset, at estimated compute: matched in spirit, not a controlled experiment.

The same pattern in language. Narang et al. (2021) reimplemented a wide range of proposed modifications of the Transformer in one code base and evaluated them on common tasks. Most did not improve performance meaningfully, and those that did had mostly been developed in the same code base or were minor changes; gains, they conjectured, may depend strongly on implementation details. The code base is a co-treatment.

What survives. The narrower original claims survive: a pure transformer is a competitive image classifier, it can be trained on ImageNet alone, and with large data it needs no convolutions. The later work does not overturn ViT's controlled study, since it changed the convolutional family and the data, and some differences survive a matched recipe. Bai et al. (2021) found that convolutional networks trained with transformer recipes matched transformers in adversarial robustness, a gap that earlier comparisons owed

to training, while transformers kept an advantage in generalization outside the training distribution, which their ablations attributed to the architecture. And the confound ran in both directions: the belief that transformers need large datasets also rested partly on a recipe (Beyer et al. 2022).

THE MECHANISM. Four features combined. A new architecture arrived with a new recipe, and tables compared systems, so the architecture and the recipe changed together. Baselines were historical controls, copied from papers with older recipes. Recipes were developed on the new architectures, so that even one recipe for both arms favored one of them. And sequential studies credited each change with its gain at its position in the sequence, although, as Bello et al. (2021) noted, longer training helped their ResNet only once regularization had been added, and dropout hurt until the weight decay was lowered: the credit of each change depended on what came before it.

VERDICT. The gaps that placed transformers above convolutional networks on ImageNet were real gaps between systems, but they belonged largely to the recipes that came with the new architectures. With a modern recipe ResNet-50 gained more than four points and came within about a point of a transformer of its size, ahead or behind according to whose recipe both used; a modernized convolutional network beat Swin at equal compute under a fixed recipe, and at web scale a convolutional family matched the best reported transformers at comparable compute. The narrower original claims hold.

CONFOUNDED

6.4 The method: two factors and four cells

A table of systems measures two cells of a design with four. Its language is that of factorial designs (Box et al. 2005; Fisher 1935).

Definition 6.4 (Factors, cells and co-treatments). A *factor* is a choice that the experimenter can set independently of the others, such as the architecture or the training recipe, and its *levels* are the settings compared. A *cell* is a combination of levels, and its *cell mean* is the mean outcome of its run distribution (Definition 1.3). A comparison of two cells is *confounded* with respect to a factor when the cells differ in that factor and in at least one other; the other differences are its *co-treatments*.

In the language of causal inference, a cell mean is an average potential outcome: the mean outcome that a run would have under that combination of levels (Imbens and Rubin 2015). An observational study must adjust for confounders that it cannot assign (Pearl 2009); an experimenter who trains networks assigns every factor and can cross them, so in a training experiment confounding is avoidable by design. Take two factors with levels 0 (old) and 1 (new), the architecture a and the recipe r, cell means μ_{ar}, and outcomes for which larger is better; for an error, read every effect as a reduction. The *simple effects* are the effects of one factor with the other

held at a level,

$$\delta_A(r) = \mu_{1r} - \mu_{0r}, \qquad \delta_R(a) = \mu_{a1} - \mu_{a0},$$

and the *interaction contrast* is $I = \mu_{11} - \mu_{10} - \mu_{01} + \mu_{00}$. A table of systems measures the *naive contrast* $\Delta = \mu_{11} - \mu_{00}$.

Proposition 6.5 (Two factors). *Let* $\theta_A = \frac{1}{2}\big(\delta_A(0) + \delta_A(1)\big)$ *and* $\theta_R = \frac{1}{2}\big(\delta_R(0) + \delta_R(1)\big)$ *be the main effects. Then:*

(i) $\Delta = \delta_A(0) + \delta_R(1) = \delta_R(0) + \delta_A(1)$*;*

(ii) $\delta_A(1) - \delta_A(0) = \delta_R(1) - \delta_R(0) = I$*, so that* $\delta_A(0) = \theta_A - I/2$ *and* $\delta_A(1) = \theta_A + I/2$*;*

(iii) $\Delta = \theta_A + \theta_R$*;*

(iv) if each cell mean is estimated by the average of n *runs, all* $4n$ *runs independent with common variance* σ^2*, then the estimates of* Δ *and of the four simple effects have variance* $2\sigma^2/n$*, those of* θ_A *and* θ_R *variance* σ^2/n*, and that of* I *variance* $4\sigma^2/n$*.*

Proof. Adding and subtracting μ_{10} gives $\Delta = (\mu_{10} - \mu_{00}) + (\mu_{11} - \mu_{10}) = \delta_A(0) + \delta_R(1)$, and adding and subtracting μ_{01} gives $\Delta = \delta_R(0) + \delta_A(1)$, which is (i). Expanding the differences in (ii), both $\delta_A(1) - \delta_A(0)$ and $\delta_R(1) - \delta_R(0)$ equal $\mu_{11} - \mu_{01} - \mu_{10} + \mu_{00} = I$; with $\delta_A(0) + \delta_A(1) = 2\theta_A$, solving for the two simple effects gives the rest of (ii). Averaging the two expressions of (i) gives (iii). For (iv), each estimate is a combination $\sum c_{ar}\bar{Y}_{ar}$ of four independent cell averages, each of variance σ^2/n, so its variance is $(\sigma^2/n)\sum c_{ar}^2$: coefficients ± 1 on two cells give $2\sigma^2/n$, coefficients $\pm\frac{1}{2}$ on all four give σ^2/n, and coefficients ± 1 on all four give $4\sigma^2/n$. ∎

Part (i) is the confound in one line: the naive contrast is the architecture effect at one recipe plus the recipe effect at the other architecture. It equals an architecture effect only if the recipe does nothing for the new architecture. Part (ii) is the dependence on the path. A study that changes the architecture first and the recipe second credits the architecture with $\delta_A(0)$; one that changes the recipe first credits it with $\delta_A(1)$; the two credits differ by exactly the interaction. Part (iii) splits Δ symmetrically, and θ_A, θ_R are the main effects of Proposition A.13 (mean at the new level minus mean at the old), whose interaction effect is $I/2$. Part (iv) prices the questions: a main effect, which uses all four cells, has half the variance of a simple effect, and the interaction needs four times as many runs as a main effect to be estimated as precisely.

Example 6.6 (Whose recipe). Read Finding 6.3 as a 2×2 design with DeiT-S as the new architecture and T2 as the new recipe: $\mu_{00} = 79.9$, $\mu_{01} = 79.2$, $\mu_{10} = 79.6$ and $\mu_{11} = 80.4$. The architecture effect is $\delta_A(0) = -0.3$ points under A2 and $\delta_A(1) = 1.2$ under T2, so $I = 1.5$, more than three times the main effect $\theta_A = 0.45$; the recipe effect is -0.7 for ResNet-50 and 0.8 for DeiT-S, with main effect $\theta_R = 0.05$. A study that first adopted T2 and then switched to DeiT-S would credit the architecture with 1.2 points; one that

switched architectures under A2 would charge it 0.3. An interaction that reverses the sign of a simple effect is a *crossover*. With one run per cell and a seed standard deviation near 0.1, I has a standard error near $\sqrt{4} \times 0.1 = 0.2$ by part (iv), so the crossover is not seed noise; whether it would survive fresh test images is another matter, since on ImageNet-V2 the architecture effect is 0.2 and 1.3, an interaction of 1.1 without a crossover.

EXAMPLE 6.7 (Adding and removing the same change). A *leave-one-out ablation* removes one change from the full recipe; an *add-one* study adds one change to the baseline. They measure simple effects at opposite corners. In Beyer et al. (2022), ViT-S/16 trained for 90 epochs reached 66.8% with the original setup, 71.6% with RandAugment and Mixup added, 73.6% with all their other changes but without these two augmentations, and 76.5% with everything. Added to the original setup, the augmentations are worth 4.8 points; removed from the improved one, 2.9. The difference, −1.9 points, is the interaction between the augmentations and the other changes, and the main effects split the 9.7 points into 3.85 for the augmentations and 5.85 for the rest.

TWO LEGITIMATE QUESTIONS. The first is the effect of the architecture at a stated recipe, the simple effect $\delta_A(r)$, or its average over a stated set of recipes; when the interaction is not negligible, report the effect at each recipe. The second is the comparison that a practitioner who will tune either architecture needs, each architecture with its own best recipe from a set $\mathcal{R}$ searched with equal budgets (Chapter 4),

$$\Delta_{\text{best}} = \max_{r \in \mathcal{R}} \mu_{1r} - \max_{r \in \mathcal{R}} \mu_{0r}.$$

The naive contrast answers neither unless the old recipe was the old architecture's best. With two recipes, if the new architecture does better with the new recipe ($\mu_{11} \geq \mu_{10}$), then $\max_r \mu_{1r} = \mu_{11}$, and

$$\Delta - \Delta_{\text{best}} = \max(\mu_{00}, \mu_{01}) - \mu_{00} = \max\big(0, \delta_R(0)\big) :$$

the naive contrast overstates the tuned comparison by the gain that the old architecture would have had from the new recipe. For ResNet-50 and the 600-epoch procedure of Wightman et al. that gain was 4.3 points. This is the arithmetic of the home-field advantage, which Chapter 8 proves for hyperparameters tuned on one arm of an ablation.

MATCHED ON WHAT? A comparison also fixes the size at which the architectures meet, and size has several axes. DeiT-S and ResNet-50 have 22.0 and 25.6 million parameters and 4.6 and 4.1 billion floating-point operations per image, yet on one V100 GPU ResNet-50 processes 2,537 images per second against 1,762 (Wightman et al. 2021);[6] matched in parameters and operations, they are not matched in speed. Each matching answers a different question (memory, arithmetic, latency, training cost), and Chapter 7 treats the choice of what to hold fixed.

[6] Measured with batch size 128 in half precision; the authors call such throughputs indicative, since they depend on hardware and software.

6.5 The method: attribution along a path

The roadmap of ConvNeXt and the additive study of Bello et al. (2021) change more than two things, one at a time, and credit each change with the gain observed when it is made. We now treat k factors. Code the levels as $x_j = -1$ (old) and $x_j = +1$ (new), write $\mu(x)$ for the cell mean of the configuration $x \in \{-1,1\}^k$, and let $\mathbf{1} = (1,\dots,1)$ and $-\mathbf{1}$ be the new and the old configuration, so that the total gain is $\Delta = \mu(\mathbf{1}) - \mu(-\mathbf{1})$. For a set S of factors put $c_S(x) = \prod_{j\in S} x_j$, with $c_\emptyset = 1$. As in Proposition A.13, the *factorial effect* of a nonempty set S is the mean of μ over the cells with $c_S = +1$ minus its mean over the cells with $c_S = -1$; written as a sum over all 2^k cells,

$$\theta_S = 2^{1-k} \sum_x c_S(x)\,\mu(x).$$

For one factor, $\theta_j = \theta_{\{j\}}$ is its main effect; for larger sets θ_S is an interaction. For a factor j and a setting z of the others, the simple effect of j is $\delta_j(z) = \mu(x_j{=}1, z) - \mu(x_j{=}{-1}, z)$.

Lemma 6.8 (Expansion). *For every configuration x, with $\bar\mu$ the mean of the 2^k cell means,*

$$\mu(x) = \bar\mu + \frac{1}{2}\sum_{S\neq\emptyset} \theta_S\, c_S(x), \qquad \text{and} \qquad \delta_j(z) = \sum_{S\ni j} \theta_S\, c_{S\setminus\{j\}}(z)$$

for every factor j and every setting z of the other factors.

Proof. Regard functions on $\{-1,1\}^k$ as vectors in $\mathbb{R}^{2^k}$ with the inner product $\langle f, g\rangle = \sum_x f(x)g(x)$. The proof of Proposition A.13 shows that $c_S c_T = c_{S\triangle T}$ and that each sign c_U with U nonempty equals $+1$ on exactly half of the cells; hence $\langle c_S, c_T\rangle = 0$ for $S \neq T$, while $\langle c_S, c_S\rangle = 2^k$. The 2^k signs are therefore an orthogonal basis, and $\mu = \sum_S 2^{-k}\langle \mu, c_S\rangle\, c_S$. The coefficient of $c_\emptyset$ is $\bar\mu$, and for nonempty S it is $2^{-k}\langle \mu, c_S\rangle = \theta_S/2$ by the definition of θ_S. This is the first identity. In the difference that defines $\delta_j(z)$, the signs c_S with $j \notin S$ do not depend on x_j and cancel, while for $j \in S$ we have $c_S(x) = x_j\, c_{S\setminus\{j\}}(z)$, whose difference between $x_j = 1$ and $x_j = -1$ is $2c_{S\setminus\{j\}}(z)$; multiplying by $\theta_S/2$ gives the second identity. ∎

Definition 6.9 (Path and Shapley attribution). An order π of the k factors defines a *path* from $-\mathbf{1}$ to $\mathbf{1}$ that switches the factors to their new levels one at a time. The *attribution* of factor j along π is its simple effect at the configuration where it is switched, $\varphi_j^\pi = \delta_j(z_j^\pi)$, where z_j^π puts the factors before j in π at $+1$ and the others at -1. The *Shapley attribution* of j is the average over all $k!$ orders, $\varphi_j = \frac{1}{k!}\sum_\pi \varphi_j^\pi$.

Theorem 6.10 (Attribution along a path). *Let $k \geq 2$.*

(i) For every order π, the attributions sum to the total gain: $\sum_{j=1}^k \varphi_j^\pi = \Delta$.

(ii) For each factor j the following are equivalent: the attribution φ_j^π is

the same for every order π; the simple effect $\delta_j(z)$ is the same for every setting z of the other factors; $\theta_S = 0$ for every set S that contains j and at least one other factor. When they hold, $\varphi_j^\pi = \theta_j$ for every π. In particular, every attribution is independent of the order if and only if every interaction vanishes.

(iii) The Shapley attributions sum to Δ, and

$$\varphi_j = \sum_{S \ni j,\ |S| \text{ odd}} \frac{\theta_S}{|S|}. \tag{6.1}$$

In particular $\varphi_j = \theta_j$ when every interaction of odd order that contains j vanishes, and always when $k = 2$.

PROOF. The plan is to read each attribution as a simple effect, to settle parts (ii) and (iii) with the expansion of Lemma 6.8, and to compute the Shapley average from the symmetry of a random order.

Part (i). Along π the attributions are the increments of μ between consecutive configurations of a path from $-\mathbf{1}$ to $\mathbf{1}$, so their sum telescopes to Δ.

Part (ii). As π runs over all orders, the set of factors before j runs over all subsets of the other factors: put a given subset first, in some order, then j, then the rest. So the attributions of j over all orders are exactly its simple effects over all settings of the other factors, and the first two conditions are equivalent. By Lemma 6.8, $\delta_j = \theta_j + \sum_S \theta_S\, c_{S\setminus\{j\}}$, the sum running over the sets S that contain j and another factor; as a function of the other $k-1$ factors, this is a combination of their signs in which each nonempty set $U = S \setminus \{j\}$ appears exactly once. These signs are orthogonal, by the argument of the lemma with $k-1$ factors, hence linearly independent, so δ_j is constant if and only if all the coefficients θ_S of the nonempty U vanish; the constant is then θ_j. Applying this to every factor gives the last statement.

Part (iii). Averaging part (i) over the $k!$ orders shows that the Shapley attributions sum to Δ. For the formula, let Π be an order drawn uniformly at random, so that $\varphi_j = \mathbb{E}\,\delta_j(z_j^\Pi)$, and insert the expansion of δ_j:

$$\varphi_j = \sum_{S \ni j} \theta_S\, \mathbb{E}\, c_{S\setminus\{j\}}(z_j^\Pi).$$

Fix $S \ni j$ with $|S| = s$. The sign $c_{S\setminus\{j\}}(z_j^\Pi)$ multiplies a $+1$ for each member of $S \setminus \{j\}$ that comes before j in Π and a -1 for each that comes after, so it equals $(-1)^{s-1-M}$, where M counts the members before j. In a uniformly random order the relative order of the s members of S is uniform, so j is equally likely to hold each of the s positions among them and M is uniform on $\{0, \dots, s-1\}$. Hence the expectation of the sign is $\frac{1}{s}\sum_{m=0}^{s-1}(-1)^{s-1-m}$, which is $1/s$ when s is odd and 0 when s is even, and substituting gives (6.1). When $k = 2$ the only odd set that contains j is $\{j\}$. ∎

The theorem turns a worry into a test. A roadmap, an additive study and a leave-one-out table each report one simple effect per change, at the setting

where the study happened to make it; whether another order would agree is decided by the interactions, which such a study does not measure. In a full factorial each θ_S is estimated from all the runs, with variance $4\sigma^2/(n2^k)$ for n runs per cell (Proposition A.13), more precisely than a single path attribution, a difference of two cells.

The Shapley attribution is the only rule that sums to the total, treats interchangeable factors alike, gives nothing to a factor that never changes the outcome, and adds over sums of games (Shapley 1953); here the game gives each set P of factors the gain $\mu(x^P) - \mu(-\mathbf{1})$, where x^P switches exactly the factors in P. By (6.1) it shares each interaction of odd order equally among its members and ignores those of even order, which add nothing to the total gain. It is a convention, not a discovery: when factors interact, "how much of the gain is due to the architecture" has no answer free of convention, as Lewontin (1974) argued for genes and environment. The convention depends on the endpoints and on how the changes are grouped into factors: splitting the recipe into two factors R_1, R_2 changes the architecture's Shapley attribution by $-\frac{2}{3}\theta_{AR_1R_2}$ (Exercise 6.3). A full factorial over k changes costs 2^k cells, affordable for five or six changes in a laboratory; beyond that, fractional designs estimate the main effects at the price of aliasing some interactions (Chapter 8), and random orders estimate the Shapley attributions at $k+1$ cells per order.

6.6 In the laboratory: convolution and its recipe

The laboratory needs two architectures that differ in a built-in assumption, two recipes a few years apart, and a task small enough to cross them many times. On the digits of Chapter 1 the roles go to the MLP of that chapter (two hidden layers of 64 units, 8,970 parameters) and to a small convolutional network defined in the script (two 3×3 convolutional layers with 16 and 32 channels, a 2×2 max-pooling and a linear classifier, 9,930 parameters); matched in parameters, the CNN performs 35 times as many multiply-accumulates per image and takes 12 to 20 times as long per run. The old recipe is that of Chapter 1: SGD with momentum 0.9 at a constant learning rate of 0.05, batch size 64, 20 epochs. The new recipe makes three changes at once: AdamW (learning rate 2×10^{-3}, weight decay 0.05) with 50 warm-up steps and a cosine schedule, label smoothing 0.1, and 60 epochs. Both were fixed before the labs, after a pilot on other splits had checked that neither learning rate sat at an edge of stability for either network; shift augmentation hurt both networks in a pilot and was left out (Exercise 6.5). The labs run in blocks: in block b every cell uses split b and seeds b, so that each contrast is computed within a block, on the same images, with a t-interval over blocks (Theorem A.1).

LAB 6.1 · THE ARCHITECTURE AND THE RECIPE

SETUP. The 2×2 design: MLP or convolutional network (CNN), old or new recipe, in 20 blocks (splits and seeds 0–19), 80 runs. About

SCRIPT
06-confounding-factorial.py

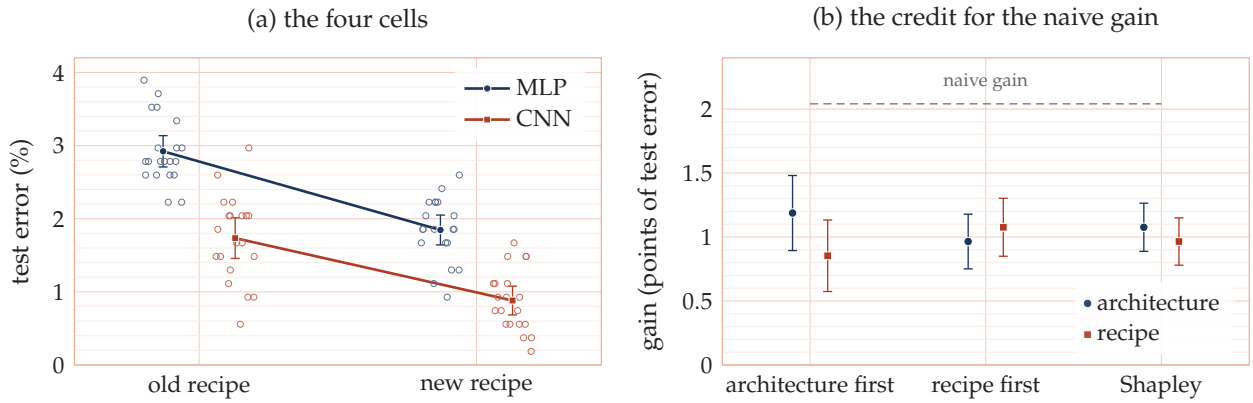


Figure 6.1. Lab 6.1: an MLP and a CNN of about 9,000 parameters on the digits, each trained with the old and the new recipe, in 20 blocks (splits and seeds 0–19). (a) Test error of every run (hollow marks: circles MLP, vermilion CNN, spread sideways) and cell means with 95% t-intervals; nearly parallel lines mean a small interaction. (b) The naive gain of the CNN with the new recipe over the MLP with the old one (dashed, 2.04 points) split into the credit of the architecture and of the recipe, along the two paths and by the Shapley attribution, with 95% t-intervals over blocks.

four minutes on a shared laptop.

PREDICT FIRST. A paper introduces the CNN with the new recipe and compares it with the MLP's number under the old recipe. What gain does it report, and how much of it belongs to the convolution? A second paper introduces the MLP with the new recipe and compares it with the CNN's old number. What does it conclude?

LOOK FOR. The two path attributions and their difference; the interval of the interaction; how often the CNN wins within a block under each recipe.

WHAT HAPPENED. With the old recipe the MLP errs on 2.92 ± 0.21% of the test images (mean and 95% t-interval over blocks) and the CNN on 1.73 ± 0.28%; with the new recipe on 1.85 ± 0.20% and 0.88 ± 0.20% (Figure 6.1a). The first paper of the lab sheet reports a gain of 2.04 ± 0.21 points for the CNN. The convolution is worth 1.19 ± 0.29 points under the old recipe and 0.96 ± 0.21 under the new one, and the recipe is worth 1.08 ± 0.23 points to the MLP and 0.85 ± 0.28 to the CNN. The interaction contrast, -0.22 ± 0.35 points, cannot be told from zero, and the two paths credit the architecture with 58% and 47% of the naive gain, the Shapley attribution with 1.08 ± 0.19 points, or 53% (Figure 6.1b). About half of the reported gain belongs to the recipe. The convolution's advantage itself is real and survives a matched recipe: the CNN is better within 19 of the 20 blocks under the old recipe and within all 20 under the new.

The second paper finds the MLP with the new recipe 0.11 ± 0.31 points behind the CNN with the old recipe, better within 6 blocks, tied within 5 and worse within 9, and concludes that the MLP matches the convolutional network: an architecture that loses by about a point under either recipe

appears its equal because it alone received the newer recipe. This is the structure of the comparison of DeiT-S with ResNet-50 in Finding 6.2. Blocking paid for itself: the standard error of the interaction is 0.17 points within blocks, 0.22 without them. Both networks do better with the new recipe, so the tuned comparison of Section 6.4 is the comparison under the new recipe, 0.96 points; the naive gain exceeds it by 1.08 points, the MLP's gain from the new recipe, as it must.

SCRIPT
`06-confounding-paths.py`

LAB 6.2 · OPENING THE RECIPE

SETUP. The new recipe of Lab 6.1 split into its three changes, the optimizer (with its schedule and weight decay), label smoothing and the duration, crossed with the architecture: a 2^4 design of 16 cells in 6 blocks (splits and seeds 20–25), 96 runs. For each factor, its simple effects at the 8 settings of the others, which are its attributions along the 24 orders of the four changes, its Shapley attribution, and all factorial effects, with t-intervals over blocks. About five minutes on a shared laptop.

PREDICT FIRST. Which of the three changes does most of the recipe's work? How far apart are the largest and the smallest credit that some order gives the architecture?

LOOK FOR. Which interactions differ from zero; the spread of each factor's eight attributions; the Shapley attributions against the main effects.

WHAT HAPPENED. On these blocks the gain from the MLP with the old recipe to the CNN with the new is 1.92 ± 0.60 points. Of the eleven interactions only one has an interval that excludes zero, optimizer by duration, 0.15 ± 0.13 points: AdamW on its cosine schedule profits more from longer training than SGD at a constant rate does. Accordingly the attributions barely depend on the order (Figure 6.2a). Over the 24 orders the architecture receives between 0.93 and 1.18 points, label smoothing between 0.62 and 0.93, the duration between 0.06 and 0.49, and the optimizer between −0.59 and 0.06. The Shapley attributions agree with the main effects to within 0.02 points: 1.04 ± 0.48 for the architecture, 0.83 ± 0.37 for label smoothing, 0.30 ± 0.27 for the duration and -0.25 ± 0.23 for the optimizer. The recipe's gain belongs to label smoothing and the longer schedule; AdamW, after which one would name the new recipe, costs a quarter of a point. The CNN wins under all eight recipes, by 0.93 to 1.18 points (Table 6.1), although the two networks prefer different optimizers: the MLP does best with all three changes, the CNN with label smoothing and long training under SGD. Given each network's own best recipe, chosen on three blocks and scored on the other three, the CNN is ahead by 1.08 ± 0.50 points. On clean digits the changes are nearly additive, and, as part (ii) of Theorem 6.10 says, every order then tells about the same story.

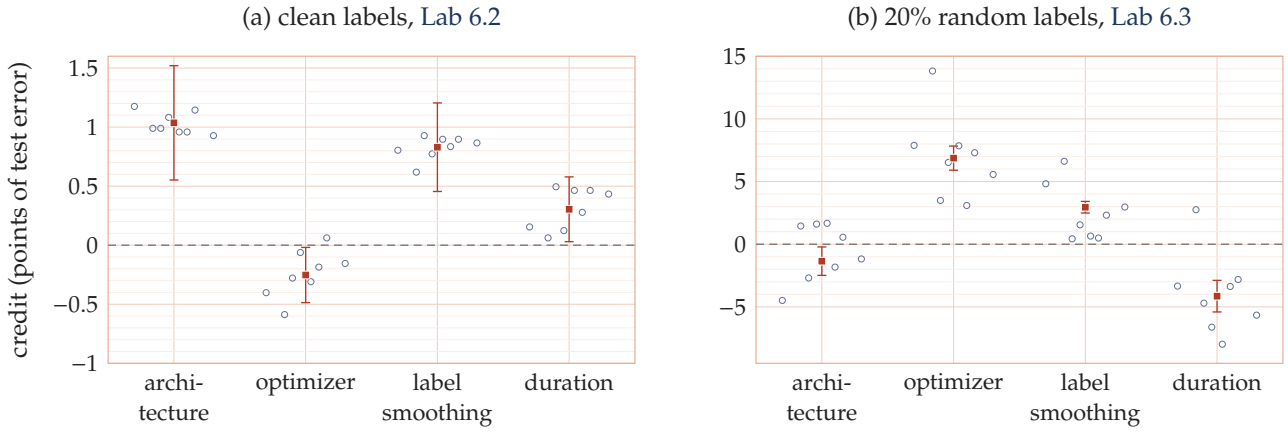


Figure 6.2. Attribution along a path in the 2^4 designs of Labs 6.2 and 6.3 (6 blocks each). For each factor, hollow marks are its eight simple effects, its credit when it is switched after a given set of the other three, placed from left to right by the number already switched (first, second, third, last); each is the credit in some of the 24 orders. Vermilion squares are the Shapley attributions with 95% t-intervals over blocks. Positive credit is a reduction of test error. Note the different vertical scales.

LAB 6.3 · OPENING THE RECIPE, WITH NOISY LABELS

SCRIPT
`06-confounding-noisy.py`

SETUP. Lab 6.2 with 20% of the training labels of every split replaced by random labels, a regime in which training longer can fit the noise; 6 blocks (splits and seeds 40–45), 96 runs. About five minutes on a shared laptop.

PREDICT FIRST. Which of the three changes of the recipe will now help and which hurt? Will the CNN still win under every recipe?

LOOK FOR. The sign of the architecture's credit across the orders; the interactions; the difference between the Shapley attribution and the main effect of the architecture.

WHAT HAPPENED WITH NOISY LABELS. The recipe now matters more than the architecture, and every change interacts with the others.[7] AdamW on its schedule is worth 6.94 ± 1.11 points as a main effect and label smoothing 2.48 ± 0.39, while three times longer training costs 3.97 ± 1.19 points, which we read as the cost of fitting random labels; eight of the eleven interactions have intervals that exclude zero, the largest the three-factor interaction of architecture, optimizer and duration, -1.93 ± 0.92. The credit of each change depends on the order (Figure 6.2b): the optimizer receives between 3.1 and 13.8 points, label smoothing between 0.4 and 6.6, the duration between −8.0 and 2.8, helping in 2 of the 24 orders, and the architecture between −4.48 and 1.67, helping in 8 of them. In the order that changes the recipe first, as the ConvNeXt roadmap did, the architecture is charged 1.18 points; in the order that changes it first, 4.48. Its Shapley attribution, -1.35 ± 1.13 points, differs from its main effect, -0.61 ± 0.94, by a third of its three-factor interactions, as (6.1) requires.

[7] The same 20% of random labels made the differences between seeds real in Lab 1.3.

Table 6.1. The architecture effect under each of the eight recipes of Labs 6.2 and 6.3: mean test error (%) of the MLP and the CNN over 6 blocks, and the CNN's gain with its 95% t-interval over blocks. Half-widths of the cell means: 0.2–0.9 points (clean) and 0.3–3.4 (noisy). The first and last rows are the old and new recipes of Lab 6.1.

			clean labels			20% random labels		
optimizer	smoothing	epochs	MLP	CNN	CNN gain	MLP	CNN	CNN gain
SGD	0	20	2.69	1.52	1.18 ± 0.73	12.96	17.44	-4.48 ± 2.77
SGD	0	60	2.54	1.45	1.08 ± 1.11	16.30	14.69	1.61 ± 1.20
SGD	0.1	20	1.89	0.90	0.99 ± 0.44	8.13	10.82	-2.69 ± 2.72
SGD	0.1	60	1.76	0.62	1.14 ± 0.50	14.75	14.19	0.56 ± 1.43
AdamW	0	20	3.09	2.10	0.99 ± 0.90	5.07	3.62	1.45 ± 0.51
AdamW	0	60	2.60	1.64	0.96 ± 0.64	9.77	11.60	-1.82 ± 1.89
AdamW	0.1	20	2.16	1.21	0.96 ± 0.63	4.64	2.97	1.67 ± 0.35
AdamW	0.1	60	1.70	0.77	0.93 ± 0.33	7.45	8.63	-1.18 ± 2.10

The naive comparison of Lab 6.1 now points the wrong way for the two recipes it involves. The CNN with the new recipe beats the MLP with the old by 4.33 ± 1.58 points, yet the CNN is behind under both of those recipes, by 4.48 ± 2.77 points under the old and by 1.18 ± 2.10, within noise, under the new (Table 6.1); the reverse comparison credits the MLP with 9.99 ± 2.99 points. Neither recipe of the 2×2 is good for this task: both networks do best with AdamW and label smoothing for 20 epochs, and there the CNN is better by 1.67 ± 0.35 points, the same number whether the best recipe is chosen on all six blocks or on three and scored on the other three. Asked how much of the naive gain of 4.33 points the convolution earned, the experiment can give a convention (the Shapley attribution, −1.35) or a table (Table 6.1), not a single answer.

WHAT THE MINIATURE REPRODUCES AND WHAT IT DOES NOT. It reproduces the confound, half of an apparent architecture gain belonging to the recipe, and the false parity of a worse architecture given a newer recipe, as in the DeiT table. With noisy labels it reproduces a crossover like that of Finding 6.3, exaggerated, and the order dependence that Bello et al. (2021) flagged for longer training. It does not reproduce the near-tie of the ImageNet comparisons: on clean digits the convolutional network wins under every recipe, which says that its assumptions suit small centered digits, not that convolution beats attention. Questions of scale are out of its reach, and the crossover needed label noise.

6.7 Protocol

PROTOCOL 6.1 · ATTRIBUTING AN IMPROVEMENT

- ☐ List every difference between the new system and the baseline (architecture, optimizer, schedule, epochs, augmentation, regularization, resolution, data, compute, code base), and treat each as a factor.

- ☐ Run the baseline architecture with the new recipe, and where possible the new architecture with the old one; never let a published number stand in for a cell.
- ☐ State the question: the effect at a named recipe, or each architecture with its own best recipe from searches of equal budget (Chapter 4); take recipes from both sides when they were developed on one architecture.
- ☐ Choose the axis of matching from the claim (parameters, operations, throughput, training compute) and report the others.
- ☐ Run the cells in blocks that share splits, seeds and test items, and compute every contrast within blocks.
- ☐ Report the interaction contrast with its interval; unless it is small against the main effect, report the effect of the architecture at each recipe instead of one number.
- ☐ For a sequence of changes, run a full or fractional factorial and report main effects, interactions and Shapley attributions with intervals, with the range of each change's credit over the orders; never present one path as the contribution of each change.

The protocol separates an architecture from the recipes it was crossed with, not from all recipes: a recipe outside the set can still reverse a conclusion, and a recipe tuned on one architecture favors it (Chapter 8). It costs at least twice the runs of the naive comparison, and a precise interaction four times as many as a precise main effect. At the scale of large pretraining runs, cross the factors at a smaller scale where runs are cheap and say so, run at least the cell that the naive comparison lacks, the baseline with the new recipe, and report an unresolved comparison as one of systems, not of architectures. When data and compute change with the architecture, as at web scale, the matched axis is part of the claim and must be named.

Exercises

6.1 CONSOLIDATION. (Four cells by hand.) Using the ImageNet-V2 accuracies of Finding 6.3, compute the four simple effects, the interaction contrast, the main effects and the architecture's credit along both paths. Which conclusions of Example 6.6 survive on the second test set? Show that the interaction effect of Proposition A.13 equals $I/2$.

6.2 CONSOLIDATION. (Three changes.) A study changes a baseline in three steps, A, B and C, with mean accuracies (%) $\mu(---) = 70.0$, $\mu(+--) = 71.5$, $\mu(-+-) = 71.0$, $\mu(--+) = 69.5$, $\mu(++-) = 72.0$, $\mu(+-+) = 71.0$, $\mu(-++) = 72.5$ and $\mu(+++) = 74.0$ (invented numbers). Compute the attributions along the six orders, the factorial effects and the Shapley attributions, and check (6.1). Which change has an attribution that changes sign with the order, and which interaction is responsible?

6.3 CONSOLIDATION. (Splitting a factor.) In a 2×2 design with factors A and R, split R into two factors R_1 and R_2 whose old and new levels together make the old and new recipe, giving a 2^3 design whose cells with $R_1 = R_2$ are the cells of the $2{\times}2$. Show that the Shapley attribution of A changes by $-\frac{2}{3}\theta_{AR_1R_2}$, where $\theta_{AR_1R_2}$ is the three-factor effect of the 2^3 design. When does splitting the recipe leave the architecture's share unchanged? *(Hint: write both attributions as weighted averages of the simple effects of A: weights $\frac{1}{2}, \frac{1}{2}$ for two factors, and $\frac{1}{3}, \frac{1}{6}, \frac{1}{6}, \frac{1}{3}$ for three, by the number of other factors already switched.)*

6.4 CONSOLIDATION. ★ (Precision of a Shapley attribution.) In a full 2^k factorial with n independent runs per cell of common variance σ^2, show that the estimated Shapley attribution has variance $\frac{4\sigma^2}{n2^k} \sum_{s \text{ odd}} \binom{k-1}{s-1} s^{-2}$, which is σ^2/n for $k = 2$ and $\sigma^2/(3n)$ for $k = 4$, against $2\sigma^2/n$ for the attribution along one path. *(Hint: use* (6.1) *and Proposition A.13.)*

6.5 LAB. (A third factor.) Add labkit's shift augmentation, which moves each training image by up to one pixel, as a third factor to the script of Lab 6.1. Before running, predict from the assumptions built into each network whether it will help the MLP or the CNN more; then run, report the three two-factor interactions with intervals, and explain what you find from how the digits were prepared.

6.6 LAB. (Matched compute.) In the script of Lab 6.1, replace the MLP by one whose multiply-accumulates per image match the CNN's (width about 520). Predict how $\delta_A(0)$, $\delta_A(1)$ and I change, then run. Which matching does a claim about the value of convolution need?

6.7 LAB. (Whose recipe?) On 300 training images split off for validation, tune the learning rate and weight decay of the new recipe separately for each network, with eight random-search trials each. Rerun Lab 6.1 with the MLP's tuned recipe as the new recipe, then with the CNN's. Does the architecture effect depend on whose recipe is used, as in Finding 6.3?

6.8 DESIGN. (A fair comparison on a budget.) A group has 400 GPU-hours to compare a new architecture with ResNet-50 on a dataset of 100,000 images; at 100 epochs a run costs 4 GPU-hours for ResNet-50 and 6 for the new architecture, and each has a recipe from its own paper. Design the study: which cells, how many seeds per cell, which matching, which questions it can answer, and what it reports when the interaction turns out to be as large as the main effect.

6.9 DESIGN. (Eight steps in 64 runs.) A roadmap modernizes a network in eight steps. With 64 runs, design an experiment that estimates the main effect of every step and the interactions you consider most likely, say which interactions are aliased with which, and how you would report a step whose value depends on another. *(Hint: a 2^{8-2} fractional design; Chapter 8.)*

6.10 CRITIQUE. (A roadmap with one seed.) A paper modernizes a baseline in eight steps, one run per step, and keeps a step when it raises accuracy by at least 0.1 points; one step lowers accuracy by 0.5 points and is kept "because it enables the next". The abstract states that the largest contribution comes from the fifth step, with 1.2 points. List what the table can and cannot show, and the fewest extra runs that would test the abstract's statement. (The description is invented.)

6.11 CRITIQUE. (Published numbers as baselines.) A paper trains a new architecture for 400 epochs with modern augmentation and lists baselines with the accuracies of their original papers (90 to 200 epochs, one run each). It claims that its architecture improves accuracy by 2.1 points at equal parameters. List the co-treatments, the cells that are missing, and how much of the gain the recipe could account for, using the numbers of Section 6.3; then rewrite the claim so that it is true. (The description is invented.)

6.12 RESEARCH. (Recipes tuned on one architecture.) When a recipe is tuned on architecture 1 with a budget of k trials and then applied to both architectures, the comparison favors architecture 1 by an amount that depends on k and on how the best hyperparameters differ between the architectures. Open: a model of this bias, for instance a quadratic response surface whose optimum depends on the architecture, with a design that removes most of it at a cost well below tuning both arms fully; or a test of whether a published comparison is consistent with no architecture effect, given only the cells it reports and bounds on the recipe effects.

Notes and References

Crossing the factors of an experiment instead of varying them one at a time is the central recommendation of Fisher (1935), who also made the interaction a quantity to be estimated rather than a nuisance; Box et al. (2005) teach effects and interactions in two-level designs in the convention of this chapter. That apportioning an outcome among interacting causes has no answer free of convention is the argument of Lewontin (1974) against reading an analysis of variance as an analysis of causes, made for genes and environment.

The Shapley value comes from cooperative game theory (Shapley 1953). Economists met the order dependence of sequential decompositions when splitting a change in inequality or poverty among its causes one cause at a time, and Shorrocks (2013) proposed the Shapley average over the orders as a unified answer. In machine learning the same value attributes predictions to input features (Lundberg and Lee 2017), and the functional analysis of variance of Hutter et al. (2014) splits the variation of performance over a hyperparameter space into main effects and interactions, the analogue for variances of the factorial effects used here.

Confounding is the central problem of observational studies, and adjusting for it is the subject of Pearl (2009) and Imbens and Rubin (2015). Lipton

and Steinhardt (2019) listed the failure to identify the sources of empirical gains among four troubling trends in machine learning scholarship, with architectural changes credited for gains that came from tuning as their example, and Melis et al. (2018) had found a carefully tuned standard LSTM ahead of newer recurrent architectures. In vision, recipes became objects of study in their own right on both sides (Bello et al. 2021; Beyer et al. 2022; He et al. 2019; Steiner et al. 2022; Wightman et al. 2021); an all-MLP architecture was reported competitive given large training sets or modern regularization (Tolstikhin et al. 2021), the role our MLP plays in the laboratory; and Bai et al. (2021) compare the robustness of the two families under matched training.

TRANSLATING THE LITERATURE. Wightman et al. (2021) write accuracy as $f(A, T, N)$ for an architecture A, a training procedure T and measurement noise N: their T is our recipe, their f averaged over N is our cell mean, and their $T^\star(A)$, the best procedure for A, defines our tuned comparison Δ_{best}. The roadmap of Liu et al. (2022) and the additive study of Bello et al. (2021) report attributions φ_j^π along one order π; the ablation tables of Touvron et al. (2021) and Beyer et al. (2022) mostly report attributions of ingredients that come last. Box et al. (2005) call our θ_S the effects, written A, B, AB and so on; their interaction AB of a 2×2 design is our $I/2$. In Shapley (1953) our factors are the players and the gain $\mu(x^P) - \mu(-\mathbf{1})$ of switching the factors in P is the characteristic function $v(P)$. A cell mean is the average potential outcome of Imbens and Rubin (2015) for one combination of treatments.

References

Bai, Y., J. Mei, A. Yuille, and C. Xie (2021). "Are Transformers More Robust Than CNNs?" In: *Advances in Neural Information Processing Systems*. Vol. 34.

Bello, I., W. Fedus, X. Du, E. D. Cubuk, A. Srinivas, T.-Y. Lin, J. Shlens, and B. Zoph (2021). "Revisiting ResNets: Improved Training and Scaling Strategies". In: *Advances in Neural Information Processing Systems*. Vol. 34.

Beyer, L., X. Zhai, and A. Kolesnikov (2022). *Better Plain ViT Baselines for ImageNet-1k*.

Box, G. E. P., J. S. Hunter, and W. G. Hunter (2005). *Statistics for Experimenters: Design, Innovation, and Discovery*. 2nd ed. Wiley.

Dosovitskiy, A. et al. (2021). "An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale". In: *International Conference on Learning Representations*.

Fisher, R. A. (1935). *The Design of Experiments*. Edinburgh: Oliver and Boyd.

He, T., Z. Zhang, H. Zhang, Z. Zhang, J. Xie, and M. Li (2019). "Bag of Tricks for Image Classification with Convolutional Neural Networks". In: *IEEE/CVF Conference on Computer Vision and Pattern Recognition*.

Hutter, F., H. Hoos, and K. Leyton-Brown (2014). "An Efficient Approach for Assessing Hyperparameter Importance". In: *Proceedings of the 31st International Conference on Machine Learning*. Vol. 32. Proceedings of Machine Learning Research, pp. 754–762.

Imbens, G. W. and D. B. Rubin (2015). *Causal Inference for Statistics, Social, and Biomedical Sciences: An Introduction*. Cambridge University Press.

Kolesnikov, A., L. Beyer, X. Zhai, J. Puigcerver, J. Yung, S. Gelly, and N. Houlsby (2020). "Big Transfer (BiT): General Visual Representation Learning". In: *European Conference on Computer Vision*.

Lewontin, R. C. (1974). "The Analysis of Variance and the Analysis of Causes". In: *American Journal of Human Genetics* 26.3, pp. 400–411.

Lipton, Z. C. and J. Steinhardt (2019). "Troubling Trends in Machine Learning Scholarship". In: *ACM Queue* 17.1. First circulated as arXiv:1807.03341, 2018.

Liu, Z., Y. Lin, Y. Cao, H. Hu, Y. Wei, Z. Zhang, S. Lin, and B. Guo (2021). "Swin Transformer: Hierarchical Vision Transformer using Shifted Windows". In: *IEEE/CVF International Conference on Computer Vision*, pp. 10012–10022.

Liu, Z., H. Mao, C.-Y. Wu, C. Feichtenhofer, T. Darrell, and S. Xie (2022). "A ConvNet for the 2020s". In: *IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 11976–11986.

Lundberg, S. M. and S.-I. Lee (2017). "A Unified Approach to Interpreting Model Predictions". In: *Advances in Neural Information Processing Systems*. Vol. 30, pp. 4765–4774.

Melis, G., C. Dyer, and P. Blunsom (2018). "On the State of the Art of Evaluation in Neural Language Models". In: *International Conference on Learning Representations*.

Narang, S. et al. (2021). "Do Transformer Modifications Transfer Across Implementations and Applications?" In: *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, pp. 5758–5773.

Pearl, J. (2009). *Causality: Models, Reasoning, and Inference*. 2nd ed. Cambridge University Press.

Shapley, L. S. (1953). "A Value for n-Person Games". In: *Contributions to the Theory of Games, Volume II*. Ed. by H. W. Kuhn and A. W. Tucker. Annals of Mathematics Studies 28. Princeton University Press, pp. 307–317.

Shorrocks, A. F. (2013). "Decomposition Procedures for Distributional Analysis: A Unified Framework Based on the Shapley Value". In: *Journal of Economic Inequality* 11.1, pp. 99–126.

Smith, S. L., A. Brock, L. Berrada, and S. De (2023). *ConvNets Match Vision Transformers at Scale*.

Steiner, A., A. Kolesnikov, X. Zhai, R. Wightman, J. Uszkoreit, and L. Beyer (2022). "How to Train Your ViT? Data, Augmentation, and Regularization in Vision Transformers". In: *Transactions on Machine Learning Research*.

Tolstikhin, I. et al. (2021). "MLP-Mixer: An All-MLP Architecture for Vision". In: *Advances in Neural Information Processing Systems*. Vol. 34.

Touvron, H., M. Cord, M. Douze, F. Massa, A. Sablayrolles, and H. Jégou (2021). "Training Data-Efficient Image Transformers & Distillation through Attention". In: *Proceedings of the 38th International Conference on Machine Learning*. Vol. 139. Proceedings of Machine Learning Research, pp. 10347–10357.

Wightman, R., H. Touvron, and H. Jégou (2021). *ResNet Strikes Back: An Improved Training Procedure in timm*. Presented at the NeurIPS 2021 workshop ImageNet: Past, Present, and Future.

Zhai, X., A. Kolesnikov, N. Houlsby, and L. Beyer (2022). "Scaling Vision Transformers". In: *IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 12104–12113.

7

What Was Held Fixed

Batch size, steps and the generalization gap

THE CLAIM *"Large-batch training generalizes worse because it converges to sharp minima."*

Keskar et al. (2017), the main observation of the paper

THE VERDICT Confounded: at a fixed number of epochs a larger batch also takes fewer steps; matched in steps, with the learning rate tuned, most of the gap disappears.

IN THIS CHAPTER. In 2016 Keskar and colleagues trained six networks with batches of 256 examples and with batches of a tenth of the training set, found the large batches up to about five points worse on test data at similar training accuracy, and attributed the gap to sharp minima. A tenth of the training set per batch means ten steps per epoch, and later work traced most of the gap to what changed with the batch: with the learning rate adapted and the steps matched, it closed, up to a critical batch size beyond which no learning rate buys back the missing steps. The method is an audit of what an experiment holds fixed: the budget that defines the question, curves of the steps needed to reach a target, and a noisy quadratic model, solved exactly, in which large batches lose at a fixed number of epochs although the model has a single minimum of fixed curvature. In the laboratory a small network on the digits loses 64 points of test error at full batch under the original kind of protocol, 12 points with the learning rate tuned, and nothing measurable once the steps are matched.

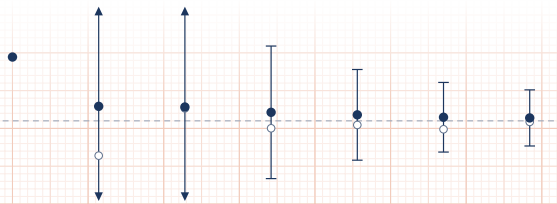
7.1 The claim: a tenth of the data in every batch

Stochastic gradient descent estimates the gradient of the training loss from a batch of B examples and takes one step per batch. The per-example gradients of a batch can be computed in parallel, so a larger batch uses more hardware at once and needs fewer steps per pass over the data. In 2016 networks were usually trained with batches of 32 to 512 examples, and the hope of training with much larger ones met an old observation. LeCun et al. (1998) had written that stochastic learning often finds better solutions than batch learning, because its noise can carry the weights into the basin of a deeper minimum, and Krizhevsky (2014) had noted that very big batches slow convergence and degrade the final solution.[1]

Nitish Shirish Keskar, Dheevatsa Mudigere, Jorge Nocedal, Mikhail Smelyanskiy, and Ping Tak Peter Tang, at Northwestern University and Intel, set out to explain the drop (Keskar et al. 2017).[2] They trained six classification networks with batch normalization: fully connected networks on MNIST and on the TIMIT speech corpus, and shallow and deep convolutional

[1] Krizhevsky also found that a learning rate proportional to the batch size worked better than the square-root rule that theory suggested, a discrepancy he could not explain.

[2] The paper was posted in September 2016 and published at ICLR 2017.



networks, in the styles of AlexNet and VGG, on CIFAR-10 and CIFAR-100. Each was trained with a small batch of 256 examples and with a large batch of a tenth of its training set, by Adam in both regimes, five times from random initializations, without a budget, until the loss stopped improving; SGD, AdaGrad, and adaQN gave similar results for the large batch.

> FINDING 7.1 (Keskar et al. 2017, Table 2). With batches of a tenth of the training set, every network reached a lower test accuracy than with batches of 256 (means and standard deviations over five runs): 97.81% against 98.03% on MNIST, 59.45% against 64.02% on TIMIT, 77.26% and 87.26% against 80.04% and 89.24% for the shallow and deep networks on CIFAR-10, and 46.45% and 57.81% against 49.58% and 63.08% on CIFAR-100. Training accuracies lay between 98.35% and 99.99% in both regimes, and the test accuracy did not peak and then decline during training, so over-fitting in the usual sense was ruled out.

The paper calls this drop the generalization gap.[3] The explanation is in the title: large-batch methods converge to sharp minimizers of the training loss, at which the Hessian has many large positive eigenvalues and which generalize less well, while the noise of small-batch gradients carries small-batch methods to flat ones. A measure of sharpness, the largest relative increase of the training loss within a small box around the solution, was one to two orders of magnitude larger for the large-batch solutions; the loss along the line between the two solutions rose more steeply at the large-batch end; in a sweep over batch sizes at 100 epochs, the test accuracy dropped sharply beyond a threshold, about 15,000 examples on TIMIT and 500 on CIFAR-10; and the small-batch solutions lay three to ten times farther from their initial weights.

[3] The term more often means the difference between a network's training and test error; here it is the difference between the test accuracies of the two regimes.

The authors presented this as numerical evidence that supports the view, and listed as open whether large-batch methods can be proved to converge to sharp minimizers. They tried remedies: data augmentation brought the large-batch test accuracy close to the small-batch one while its solutions stayed sharp, and adversarial training did not help. The reading that traveled was shorter: large batches generalize worse, because of sharp minima. The paper became a standard reference for a large-batch generalization gap and for the link between flat minima and generalization, the subject of Chapter 10. Much of it was right. The gap was real under the protocol, measured with five runs per arm; checking that the test accuracy did not peak and decline was the right test for over-fitting; the threshold in the sweep was an early sighting of what was later named the critical batch size; and the closeness of large-batch solutions to their initialization pointed toward the mechanism.

7.2 The experiment: one factor changed, everything else equal

EXPERIMENTAL DESIGN · KESKAR ET AL. 2017	
QUESTION	Do large batches find solutions that generalize worse than small batches?
UNITS	Training runs of six networks with batch normalization: two fully connected (MNIST, TIMIT) and four convolutional (CIFAR-10, CIFAR-100).
TREATMENT	The batch size: 256 examples, or a tenth of the training set (6,000 on MNIST, 5,000 on CIFAR).
TUNED	Nothing for each batch size: Adam in both regimes, with settings the paper does not report.
HELD FIXED	Optimizer and its settings, network, data, and the stopping rule (train until the loss stops improving; 100 epochs in the batch-size sweep).
RESPONSE	Training and test accuracy; a sharpness measure; the loss along the line between two solutions.
REPLICATES	Five runs per network and batch size, from random initializations.
ANALYSIS	Means and standard deviations over the five runs.

The record shows one factor at two levels with everything else equal. But holding a recipe equal does not hold a run equal, because several properties of a run are functions of its batch size.[4] A batch of a tenth of the training set gives ten steps per epoch on every data set, against 195 for a batch of 256 on CIFAR-10 and 2,817 on TIMIT: epoch for epoch, the large batch takes a twentieth of the steps on CIFAR-10 and less than a 280th on TIMIT. At one learning rate each large-batch step averages away most of the gradient noise. And batch normalization computed its statistics over 5,000 examples instead of 256, which changes the function being trained. The stopping rule looks like a way around the step count, but it is a rule about the training loss, and the training loss can flatten while the weights keep moving and the test error keeps falling.

The design could show that this recipe, with only its batch size changed, generalizes worse at the larger batch on these networks. It could not show that the batch size causes the gap, because the steps, the gradient noise, and the normalization statistics changed with it; the sharpness of the end points is a correlate that the same changes could produce; and two batch sizes give two points, not a curve.

[4] The authors' public script for the parametric plots trains both batch sizes with Keras's default Adam for the same 20 epochs; it reproduces figures, not the experiments of the tables.

7.3 The post-mortem: fewer steps and less noise

TRAIN LONGER. Elad Hoffer, Itay Hubara, and Daniel Soudry found that the distance of the weights from their initialization grows like the logarithm of the number of updates, along similar curves for all batch sizes (Hoffer et al. 2017). A run with a fixed number of epochs and a larger batch makes fewer updates, so it ends closer to its initialization, as Keskar et al. had observed.

Hoffer et al. adapted the large-batch run in three steps: the learning rate multiplied by the square root of the ratio of batch sizes; normalization statistics computed over "ghost" batches of 128 examples within the large batch; and regime adaptation, multiplying the epochs by the ratio of batch sizes so that both runs take the same number of steps. For a ResNet-44 on CIFAR-10, batches of 4,096 instead of 128 lowered the validation accuracy from 92.83% to 86.10%, and the three steps raised it to 89.30%, 90.50%, and finally 93.07%; on five other networks, Keskar et al.'s among them, the adapted large batch exceeded the small one or came within 0.13 points of it. The test error also kept improving at the initial learning rate long after the training and validation errors had stopped changing visibly.[5]

[5] Hoffer et al. report one number per cell, without spreads over runs; their effects are several points, against seed spreads of tenths of a point for such networks (Chapter 1).

SCALE THE LEARNING RATE. Priya Goyal and eight colleagues at Facebook trained ResNet-50 on ImageNet with batches of up to 65,536 images, all for 90 epochs, with the learning rate multiplied by the factor by which the batch grew, the *linear scaling rule*, and raised gradually over the first five epochs (Goyal et al. 2017). A batch of 8,192 reached a top-1 validation error of 23.74% (standard deviation 0.09 over five runs), against 23.60% (0.12) for batches of 256, and 24.84% without the warm-up. Beyond 8,192 the error rose, to 24.79% at 16,384, 27.55% at 32,768, and 33.96% at 65,536, with the epoch budget fixed throughout.[6]

[6] Goyal et al. report means and standard deviations over five runs and warn that results from a single trial, or from the best of many, can mislead, the lesson of Chapter 1.

THE NOISE SCALE. Samuel Smith and Quoc Le read SGD as the discretization of a stochastic differential equation whose noise scale is about $\eta n_{\text{tr}}/B$, where n_{tr} is the number of training examples (Smith and Le 2018). At a fixed learning rate the test accuracy peaked at an intermediate batch size, which grew in proportion to the learning rate and to n_{tr}; Keskar et al.'s two regimes, at one learning rate with batches twenty times apart, sat on this reading at very different noise scales. Smith, Kindermans, Ying, and Le then showed that growing the batch during training can replace decaying the learning rate, reaching the same test accuracy in the same number of epochs with fewer updates (Smith et al. 2018).

STEPS TO A RESULT. Christopher Shallue, Jaehoon Lee, Joseph Antognini, Jascha Sohl-Dickstein, Roy Frostig, and George Dahl measured the steps needed to reach a goal validation error as a function of the batch size, for six families of networks, three optimizers, and seven data sets, tuning the learning rate, momentum, and schedule anew at every batch size (Shallue et al. 2019). Every workload showed three regimes: *perfect scaling*, in which doubling the batch halves the steps; diminishing returns; and a maximal useful batch size. Where perfect scaling ended varied by orders of magnitude between workloads, momentum extended it beyond plain SGD, and no simple rule related the best learning rate to the batch size across problems. Sorting earlier studies by their budgets, they found that a budget of steps favors large batches, which see more examples, a budget of epochs favors small ones, which take more steps, and tuning at one batch size and transferring the learning rate by a rule favors the batch size where the tuning was done. They concluded that such differences explain much of the disagreement, wrote that "we find no evidence that larger batch sizes

degrade out-of-sample performance", and added that regularization may matter more at large batches: for ResNet-50 on ImageNet, label smoothing lowered the error by up to a point at large batches and had no apparent effect at small ones.

THE CRITICAL BATCH SIZE. In a preprint, Sam McCandlish, Jared Kaplan, Dario Amodei, and the OpenAI Dota team defined the *critical batch size* as the ratio of the fewest examples to the fewest steps that reach a given performance, and found that the *gradient noise scale*, essentially the variance of the per-example gradients divided by the squared norm of their mean (Section 7.5.1), predicts it within an order of magnitude on tasks from MNIST to the game Dota 2, over which it ranged from about 20 to millions (McCandlish et al. 2018). The noise scale grew as the loss fell, so the critical batch size depends on the target: for their MNIST network it was about 20 early in training and about 200 averaged over a run. Guodong Zhang and seven colleagues reproduced the three regimes in a noisy quadratic model, the model this chapter solves, and confirmed its predictions with Adam and K-FAC: preconditioning, like momentum, extends perfect scaling to larger batches (Zhang et al. 2019).

WHAT REMAINS. Not all of the gap is about steps. Samuel Smith, Erich Elsen, and Soham De trained a 16-4 Wide-ResNet on CIFAR-10 for a fixed 9,765 updates at every batch size, with the learning rate tuned at each (Smith et al. 2020). The test accuracy rose from 93.5% at a batch of 256 to 94.9% at 2,048, stayed within 0.3 points of that up to 8,192, and fell to 92.5% at 16,384 and 89.9% at 32,768, a third and two thirds of the training set, while the training losses at these batches stayed below the one at 2,048: a benefit of gradient noise that survives matched steps, at batches far larger than Keskar et al.'s. Geiping et al. (2022) supplied such a benefit explicitly: full-batch gradient descent for a ResNet-18 on CIFAR-10, with 3,000 steps, gradient clipping, and a penalty on the gradient norm, reached 95.68%, against 95.70% for SGD with batches of 128. And the advocacy of small batches by Dominic Masters and Carlo Luschi, who found the best test performance at batches of 2 to 32 in a preprint, used a fixed number of epochs (Masters and Luschi 2018), the budget that favors small batches.

THE MECHANISM. Under a fixed number of epochs, or a stopping rule on the training loss, Keskar et al.'s large batch took far fewer steps; at one learning rate it trained with far less gradient noise per epoch; and its normalization statistics came from larger batches. Each moves the test error, and each can be undone: by a learning rate that grows with the batch, by matched steps, by ghost batches. Sharpness moved with them and need not be a separate cause, because the sharpness at which gradient descent can stay is set by its step. On a quadratic, a step η diverges along every direction of curvature above $2/\eta$ (Proposition 2.11); full-batch training of networks typically runs with the largest Hessian eigenvalue near $2/\eta$ (Cohen et al. 2021); and in a preprint, a larger ratio of learning rate to batch size was found to favor wider minima (Jastrzębski et al. 2017). Whether sharpness explains the residual gap of Smith et al. is a separate question, about a

measure that a reparametrization can change without changing the function (Dinh et al. 2017); Chapter 10 takes it up.

VERDICT. Keskar et al.'s comparison changed the batch size at one learning rate and under a stopping rule that, like a fixed number of epochs, gave the large batch far fewer steps; it compared two numbers of steps and two noise levels as much as two batch sizes. With the learning rate adapted and the steps matched, the gap closes at batch sizes like theirs, and the largest systematic study found no evidence that large batches degrade out-of-sample performance. Two narrower statements survive: beyond a critical batch size the steps cannot be bought back, so at a fixed number of epochs large batches lose; and at a third of the training set or more a residual gap can remain under matched steps, which explicit regularization can replace and whose explanation is the question of Chapter 10.

CONFOUNDED

7.4 The method: what the budget holds fixed

An experiment that changes one factor of a recipe holds the others fixed, and which others is part of the question. The batch size makes the point sharply, because the common budgets are tied to it by identities.

Definition 7.2 (Budget and derived quantities). The *budget* of a comparison is the quantity held equal across its arms: the number of optimizer steps T; the number of examples processed $D = BT$, or equivalently the number of epochs, D divided by the size of the training set; the floating-point operations; the wall-clock time on given hardware; or the number of tuning trials. A *derived quantity* of a run is one that the budget and the treatment determine together.

At a fixed number of epochs the steps fall as $T = D/B$; at a fixed number of steps the examples grow as $D = BT$; at a fixed learning rate the variance of one step's noise falls as η^2/B. For a given network the operations follow the examples, and with perfect data parallelism the wall-clock time follows the steps. Each budget therefore asks its own question: a budget of steps or of time asks whether a larger batch trains faster on parallel hardware, and a budget of examples or operations whether it is as economical. Keskar et al.'s question, whether a larger batch finds a worse solution, has no budget at all; it asks about runs trained until nothing changes, a state that Shallue et al. (2019) needed millions of steps to approach even on MNIST, and could not certify.[7]

STEPS TO A TARGET. The informative object is therefore a curve. Fix a target, a test error or a loss, and let $T(B)$, the *steps to the target*, be the fewest steps that reach it with batch B, minimized over the hyperparameters that interact with B, and $D(B) = BT(B)$ the examples. In the perfect-scaling regime $T(B)$ falls like $1/B$ and $D(B)$ is flat; at large batches $T(B)$ approaches a floor $T_{\min}$ and $D(B)$ grows like B. With $D_{\min}$ the small-batch limit of $D(B)$, the

[7] For MNIST and Fashion MNIST they trained 100 configurations per batch size for up to 25 hours each, and were still not certain that performance had saturated at the smallest and largest batches.

critical batch size $B_{\text{crit}} = D_{\min}/T_{\min}$ is where the two asymptotes, $D_{\min}/B$ and $T_{\min}$, cross (McCandlish et al. 2018). McCandlish et al. fit the curve $T(B) = T_{\min}(1 + B_{\text{crit}}/B)$, which takes twice the fewest steps and twice the fewest examples at $B = B_{\text{crit}}$ (Exercise 7.1). Below B_{crit} a larger batch saves time at almost no extra computation; above it, it buys little time at a cost that grows with the batch.

Scaling rules. A *scaling rule* says how to change the other hyperparameters with the batch so that runs stay comparable in some derived quantity. For SGD in the perfect-scaling regime the linear rule, a learning rate proportional to B, keeps the path of the run as a function of the examples processed (Goyal et al. 2017; Krizhevsky 2014). For RMSprop and Adam, Malladi et al. (2022) derived a square-root rule from stochastic differential equations: when the batch is multiplied by s, multiply the learning rate by $\sqrt{s}$, multiply $1 - \beta_1$ and $1 - \beta_2$ by s, and divide ϵ by $\sqrt{s}$. A rule fails where the quantity it holds cannot be held: beyond the critical batch size the learning rate cannot keep growing with the batch, and only more steps make up the difference.

Schedules matched to the budget. A schedule has a horizon, and a run evaluated before its schedule has decayed is not finished. Kaddour et al. (2023) compared efficient-training methods for BERT and T5, among them layer stacking, batch selection, and the optimizers Lion and Sophia, under fixed budgets of reference-system time, with a baseline whose learning rate decayed fully within the same budget, and the reported gains vanished. Their rule is general: train every arm to a fixed budget, decay every schedule in the unit of that budget, and measure at its end. Chapter 4 met the same issue in optimizer comparisons read at intermediate checkpoints, Chapter 12 meets it in scaling laws, and different measures of cost can rank the same models differently (Dehghani et al. 2022).

Crossing the budget with the treatment. When the question does not fix the budget, make the budget a factor. Write $\bar{Y}(B, T)$ for the mean outcome with batch B and T steps, other hyperparameters held at their values for the smaller batch B_1. A comparison with B_2 at a fixed number D of examples splits as

$$\begin{aligned}\bar{Y}(B_2, D/B_2) - \bar{Y}(B_1, D/B_1) = &\left[\bar{Y}(B_2, D/B_1) - \bar{Y}(B_1, D/B_1)\right] \\ &+ \left[\bar{Y}(B_2, D/B_2) - \bar{Y}(B_2, D/B_1)\right],\end{aligned}$$

the effect of the batch at matched steps plus the effect of the steps that the larger batch did not take. A 2×2 design over the budget (epochs or steps) and the hyperparameters (held or retuned) measures each part and their interaction; when the interaction is large, as Lab 7.1 finds, the attribution depends on the order in which the parts are removed (Chapter 6).

7.5 The method: a model with a single minimum

Is a gap at a fixed number of epochs evidence about the minima that the runs find? A model can answer by example. The *noisy quadratic model*, in the form used by Zhang et al. (2019), is a quadratic loss with noisy gradients. It has a single minimum, of fixed curvature, so no run can find a sharper or a flatter one; if large batches lose in it, then losing is not evidence of sharp minima. We solve it exactly in one dimension.

Assumption 7.3 (One noisy curvature). The loss is $f(\theta) = \frac{1}{2}\kappa\theta^2$ with curvature $\kappa > 0$. Starting from a fixed $\theta_0 \neq 0$, with $\ell_0 = f(\theta_0)$, stochastic gradient descent with batch size $B > 0$ and learning rate $\eta > 0$ sets $\theta_{t+1} = \theta_t - \eta(\kappa\theta_t + \xi_{t+1})$ for $t = 0, 1, 2, \dots$, where $\xi_1, \xi_2, \dots$ are independent, with mean 0 and variance c/B for a constant $c > 0$.

The noise ξ_t models the average of B independent per-example deviations of variance c. A batch drawn without replacement from n examples has variance $(c/B)(n - B)/(n - 1)$ instead, so every statement below holds for such batches with B replaced by the effective batch $B_{\text{eff}} = B(n - 1)/(n - B)$.

Theorem 7.4 (Steps to a target in the noisy quadratic model). *Under Assumption 7.3 the following hold.*

(i) For every $t \geq 0$, $\mathbb{E}\theta_{t+1}^2 = (1 - \eta\kappa)^2\,\mathbb{E}\theta_t^2 + \eta^2 c/B$. If $0 < \eta\kappa < 2$, then for every $t \geq 0$

$$\mathbb{E}\,f(\theta_t) = f_\infty + (1 - \eta\kappa)^{2t}\big(\ell_0 - f_\infty\big), \qquad f_\infty = \frac{\eta c}{2B(2 - \eta\kappa)}, \tag{7.1}$$

so that $\mathbb{E}\theta_t^2$ tends to the stationary value $\eta c/(B\kappa(2 - \eta\kappa))$. If $\eta\kappa \geq 2$, then $\mathbb{E}\theta_t^2 \to \infty$.

(ii) Fix a target ℓ with $0 < \ell < \ell_0$ and a largest learning rate $\eta_{\max}$ with $0 < \eta_{\max}\kappa < 1$, and write $R = \ell_0/\ell$, $B_{\text{noise}} = c/(2\kappa\ell)$, and $T_{\min} = \ln R/\big(-2\ln(1 - \eta_{\max}\kappa)\big)$. For $0 < \eta \leq \eta_{\max}$ let $\tau(\eta, B)$ be the time at which the right side of (7.1), as a function of a real $t \geq 0$, equals ℓ, or ∞ when it never does, so that $\mathbb{E}\,f(\theta_t) \leq \ell$ at a step t exactly when $t \geq \tau(\eta, B)$; and let $T^\star(B) = \inf_{0<\eta\leq\eta_{\max}} \tau(\eta, B)$. Then $T^\star$ is nonincreasing in B, and $T^\star(B) \geq T_{\min}$ for every B. Write $u_B = (B_{\text{noise}}/B)\,\eta_{\max}\kappa/(2 - \eta_{\max}\kappa)$. If $u_B < 1$, then

$$T^\star(B) \leq T_{\min}\Big(1 + \frac{\ln\big(1/(1 - u_B)\big)}{\ln R}\Big);$$

in particular $T^\star(B) \to T_{\min}$ as $B \to \infty$.

(iii) Let $\psi(R) = \min_{0<u<1} u^{-1}\ln\big((R - u)/(1 - u)\big)$ and $D_{\min} = B_{\text{noise}}\,\psi(R)/4$. If $2B/(B + B_{\text{noise}}) \leq \eta_{\max}\kappa$, then

$$\Big(1 - \frac{B}{B_{\text{noise}}}\Big)D_{\min} \leq B\,T^\star(B) \leq \Big(1 + \frac{B}{B_{\text{noise}}}\Big)D_{\min};$$

in particular $B\,T^\star(B) \to D_{\min}$ as $B \to 0$.

Part (i) is the formula of Zhang et al. for one coordinate; parts (ii) and (iii) solve it for the steps. Part (iii) is perfect scaling: for small batches the examples needed are nearly constant, so the steps halve when the batch doubles. Part (ii) is the floor: no batch reaches the target in fewer than $T_{\min}$ steps, and large ones come arbitrarily close. The model's critical batch size is $B_{\text{crit}} = D_{\min}/T_{\min}$.

Proof. The plan is to obtain part (i) by squaring the recursion, then to write τ in terms of the dimensionless step $x = \eta\kappa$ and the ratio u of the stationary loss to the target; part (ii) follows by monotonicity, part (iii) by a change of variables from x to u.

Part (i). The recursion is $\theta_{t+1} = (1-\eta\kappa)\theta_t - \eta\xi_{t+1}$. By induction θ_t is a linear combination of $\theta_0, \xi_1, \dots, \xi_t$, so it has a finite second moment and is independent of ξ_{t+1}; hence $\mathbb{E}[\theta_t \xi_{t+1}] = \mathbb{E}\theta_t \, \mathbb{E}\xi_{t+1} = 0$. Squaring the recursion and taking expectations, the cross term vanishes and $\mathbb{E}\theta_{t+1}^2 = q\,\mathbb{E}\theta_t^2 + \eta^2 c/B$ with $q = (1-\eta\kappa)^2$. If $0 < \eta\kappa < 2$, then $1 - q = \eta\kappa(2-\eta\kappa) > 0$, and $v = \eta^2 c/(B(1-q)) = \eta c/(B\kappa(2-\eta\kappa))$ satisfies $v = qv + \eta^2 c/B$. Subtracting, $\mathbb{E}\theta_{t+1}^2 - v = q(\mathbb{E}\theta_t^2 - v)$, so $\mathbb{E}\theta_t^2 = v + q^t(\theta_0^2 - v)$, which tends to v; multiplying by $\kappa/2$ gives (7.1) with $f_\infty = \kappa v/2$. If $\eta\kappa \geq 2$, then $q \geq 1$, so $\mathbb{E}\theta_{t+1}^2 \geq \mathbb{E}\theta_t^2 + \eta^2 c/B$ and $\mathbb{E}\theta_t^2 \geq \theta_0^2 + t\eta^2 c/B \to \infty$.

The time to the target. Let $0 < \eta \leq \eta_{\max}$, so that $x = \eta\kappa$ lies in $(0, x_{\max}]$ with $x_{\max} = \eta_{\max}\kappa < 1$, and $0 < 1 - x < 1$. Dividing f_∞ by ℓ,

$$u = \frac{f_\infty}{\ell} = \frac{c}{2\kappa\ell B} \cdot \frac{x}{2-x} = \frac{B_{\text{noise}}}{B} \cdot \frac{x}{2-x}.$$

By (7.1), $\mathbb{E}f(\theta_t) = \ell\big(u + (1-x)^{2t}(R-u)\big)$. If $u \geq 1$, then this exceeds ℓ for every t, because $R > 1$, and $\tau = \infty$. If $u < 1$, then it decreases strictly in t from ℓ_0 toward $\ell u < \ell$ and is at most ℓ exactly when $(1-x)^{2t} \leq (1-u)/(R-u)$; taking logarithms, with $\ln(1-x) < 0$,

$$\tau(\eta, B) = \frac{L(u)}{-2\ln(1-x)}, \qquad L(u) = \ln\frac{R-u}{1-u}. \tag{7.2}$$

Two facts are used below. First, L is increasing on $[0,1)$ with $L(0) = \ln R > 0$, because $(R-u)/(1-u) = 1 + (R-1)/(1-u)$ and $R > 1$. Second, $x \leq -\ln(1-x) \leq x/(1-x)$ for $0 \leq x < 1$, from the series $-\ln(1-x) = \sum_{k\geq 1} x^k/k$ and from $-\ln(1-x) = \ln\big(1 + x/(1-x)\big)$.

Part (ii). For fixed η, the ratio u decreases as B grows and L is increasing, so $\tau(\eta, B)$ is nonincreasing in B, and so is the infimum $T^\star(B)$. Since $L(u) \geq \ln R$ and $-\ln(1-x) \leq -\ln(1-x_{\max})$ for $x \leq x_{\max}$, every finite $\tau(\eta, B)$ is at least $\ln R/(-2\ln(1-x_{\max})) = T_{\min}$. For the upper bound take $\eta = \eta_{\max}$, for which $u = u_B$. If $u_B < 1$, then $L(u_B) \leq \ln R + \ln(1/(1-u_B))$ because $R - u_B < R$, and dividing by $-2\ln(1-x_{\max}) = \ln R/T_{\min}$ bounds $\tau(\eta_{\max}, B)$, hence $T^\star(B)$, as stated. As $B \to \infty$, $u_B \to 0$ and the bound tends to $T_{\min}$.

Part (iii). Write $x_B = 2B/(B + B_{\text{noise}})$. The condition $u < 1$ means $x/(2-x) < B/B_{\text{noise}}$, that is $x < x_B$; by hypothesis $x_B \leq x_{\max}$, so every step with a finite time is admissible. The map $x \mapsto u$ is an increasing bijection from $(0, x_B)$ onto $(0,1)$ with inverse $x(u) = 2uB/(B_{\text{noise}} + uB)$, and therefore

$T^\star(B) = \inf_{0<u<1} \tau(u)$, where $\tau(u)$ denotes (7.2) at $x = x(u)$. We bound $\tau(u)$ above and below. By the second fact,

$$\frac{1 - x(u)}{2x(u)} \le \frac{1}{-2\ln(1 - x(u))} \le \frac{1}{2x(u)},$$

and the two outer expressions are $(B_{\text{noise}} - uB)/(4uB)$ and $(B_{\text{noise}} + uB)/(4uB)$, because $1/(2x(u)) = (B_{\text{noise}} + uB)/(4uB)$ and $(1 - x(u))(B_{\text{noise}} + uB) = B_{\text{noise}} - uB$. Multiplying by $B\,L(u)$, and using $0 < u < 1$ together with $B < B_{\text{noise}}$, which holds because $x_B \le x_{\max} < 1$, gives

$$(B_{\text{noise}} - B)\,\frac{L(u)}{4u} \le B\,\tau(u) \le (B_{\text{noise}} + B)\,\frac{L(u)}{4u}.$$

The function $L(u)/u$ is continuous on $(0, 1)$ and tends to $+\infty$ at both ends, since $L(0) = \ln R > 0$ and $L(u) \to \infty$ as $u \to 1$, so it attains its minimum $\psi(R)$. Taking the infimum over u in the last display gives $(B_{\text{noise}} - B)\psi(R)/4 \le B\,T^\star(B) \le (B_{\text{noise}} + B)\psi(R)/4$, which is part (iii). As $B \to 0$, eventually $x_B \le x_{\max}$, and both bounds tend to $D_{\min}$. ∎

Corollary 7.5 (A fixed number of epochs). *In the setting of Theorem 7.4, let every run process the same number D of examples, so that a run with batch B takes $T = D/B$ steps, assumed to be an integer.*

(i) For every $\eta \le \eta_{\max}$, $\mathbb{E}\,f(\theta_T) \ge (1 - \eta_{\max}\kappa)^{2D/B}\ell_0$, a bound that increases to ℓ_0 as $B \to \infty$. If $B > D/T_{\min}$, then $\mathbb{E}\,f(\theta_T) > \ell$ for every $\eta \le \eta_{\max}$: no learning rate reaches the target.

(ii) If $2B/(B + B_{\text{noise}}) \le \eta_{\max}\kappa$ and $D > (1 + B/B_{\text{noise}})D_{\min}$, then some $\eta \le \eta_{\max}$ gives $\mathbb{E}\,f(\theta_T) < \ell$.

Proof. For part (i), (7.1) can be written $\mathbb{E}\,f(\theta_T) = (1-x)^{2T}\ell_0 + \big(1-(1-x)^{2T}\big)f_\infty$ with $x = \eta\kappa \in (0, x_{\max}]$; the second term is nonnegative and $(1 - x)^{2T} \ge (1 - x_{\max})^{2T}$, which gives the bound, and the bound increases to ℓ_0 because $T = D/B$ decreases to 0. It exceeds ℓ exactly when $2T\ln(1 - x_{\max}) > -\ln R$, that is when $T < T_{\min}$, which is the condition $B > D/T_{\min}$. For part (ii), part (iii) of the theorem gives $T^\star(B) \le (1 + B/B_{\text{noise}})D_{\min}/B < D/B = T$, so some η has $\tau(\eta, B) < T$; since $\mathbb{E}\,f$ is strictly decreasing in t for that η and equals ℓ at τ, it is below ℓ at the step T. ∎

Give every run $D = \lambda D_{\min}$ examples, $\lambda > 1$ times the fewest that suffice. Small batches reach the target, and every batch above $\lambda D_{\min}/T_{\min} = \lambda B_{\text{crit}}$ misses it at every learning rate, with a final loss that approaches the initial one as the batch grows. The model has one minimum and one curvature, so a large-batch gap at a fixed number of epochs needs no sharp minima: it is what too few steps look like.

What limits each regime. The small-batch limit $D_{\min} = c\,\psi(R)/(8\kappa\ell)$ does not involve $\eta_{\max}$: small batches are limited by noise. The floor $T_{\min}$ does not involve c or B: large batches are limited by the largest step that the curvature allows. Hence

$$B_{\text{crit}} = \frac{D_{\min}}{T_{\min}} = B_{\text{noise}}\,\frac{\psi(R)}{\ln R}\cdot\frac{-\ln(1 - \eta_{\max}\kappa)}{2} \approx \frac{\psi(R)}{\ln R}\cdot\frac{\eta_{\max}c}{4\ell}$$

for small $\eta_{\max}\kappa$. The last factor is the batch size at which the stationary loss of SGD at its largest step, about $\eta_{\max}c/(4B)$, equals the target, and $\psi(R)/\ln R$ lies between 1.4 and 2.1 for $10 \le R \le 10^4$ (`07-heldfixed-nqm.py`). The critical batch size thus belongs to a target, not to a network, and grows as the target falls. Anything that allows larger steps along slowly converging directions lowers $T_{\min}$ and raises B_{crit} without helping small batches, which is what Zhang et al. (2019) found for momentum and preconditioning and Shallue et al. (2019) for momentum. The best step at small batches, $x(u) = 2uB/(B_{\text{noise}} + uB)$ in the proof, grows in proportion to B: the linear scaling rule, and its saturation at $\eta_{\max}$ above the critical batch size (Exercise 7.2).

7.5.1 Many curvatures

Take $f(\theta) = \frac{1}{2}\sum_i \kappa_i \theta_i^2$ with independent noise of variance c_i/B in coordinate i. Part (i) of the theorem applies to each coordinate, so $\mathbb{E}\, f(\theta_t)$ is a sum of terms (7.1); one learning rate serves all of them, and the stiffest coordinate caps it below $2/\max_i \kappa_i$. Each coordinate has its own corner, at its own batch size, and their sum bends smoothly. Figure 7.1 shows both cases, computed exactly: a single curvature turns from $D_{\min}/B$ to $T_{\min}$ within a factor of about two in B, and a thousand curvatures $\kappa_i = 1/i$ with $c_i = \kappa_i$, the choice of Zhang et al. (2019) with fewer coordinates, follow the hyperbola $T_{\min}(1 + B_{\text{crit}}/B)$ to within 14%. McCandlish et al. derived the hyperbola from one step.

The noise scale. Near θ, approximate the loss by its second-order expansion with gradient G and Hessian H, and let the minibatch gradient $\hat{G}$ have mean G and covariance Σ/B, where Σ is the covariance of one example's gradient. A step $-\eta\hat{G}$ changes the loss in expectation by about

$$-\eta|G|^2 + \tfrac{1}{2}\eta^2\Big(G^\top H G + \frac{\operatorname{tr}(H\Sigma)}{B}\Big),$$

which is smallest at $\eta_{\text{opt}}(B) = \eta_{\text{gr}}/(1 + B_{\text{noise}}/B)$, with $\eta_{\text{gr}} = |G|^2/G^\top H G$ and the noise scale $B_{\text{noise}} = \operatorname{tr}(H\Sigma)/G^\top H G$, where the decrease is $\Delta_{\max}/(1 + B_{\text{noise}}/B)$ with $\Delta_{\max} = |G|^4/(2G^\top H G)$. The best learning rate and the progress per step both grow in proportion to B while $B \ll B_{\text{noise}}$ and saturate when $B \gg B_{\text{noise}}$. If B_{noise} changes little along the way to the target, then the steps scale as $T(B) \approx T_{\min}(1 + B_{\text{noise}}/B)$, the hyperbola with $B_{\text{crit}} \approx B_{\text{noise}}$. When H is a multiple of the identity, B_{noise} equals the *simple noise scale* $B_{\text{simple}} = \operatorname{tr}\Sigma/|G|^2$, which needs only per-example gradients. ◇

In one dimension $B_{\text{noise}} = c/(2\kappa f(\theta))$, which at the target is the B_{noise} of the theorem, and the exact result shows how far the one-step argument goes: the critical batch size is the noise scale times $\psi(R)/\ln R$, a correction for the whole path, and times $-\ln(1 - \eta_{\max}\kappa)/2$, a correction for the cap that other directions put on the step, which one step cannot see. In ill-conditioned problems the second factor is small: for the thousand curvatures of Figure 7.1 the noise scales at the target, $B_{\text{noise}} \approx 7{,}100$ and $B_{\text{simple}} \approx 16{,}000$, exceed the critical batch size of 117 by factors of 60 and 140. The noise scale

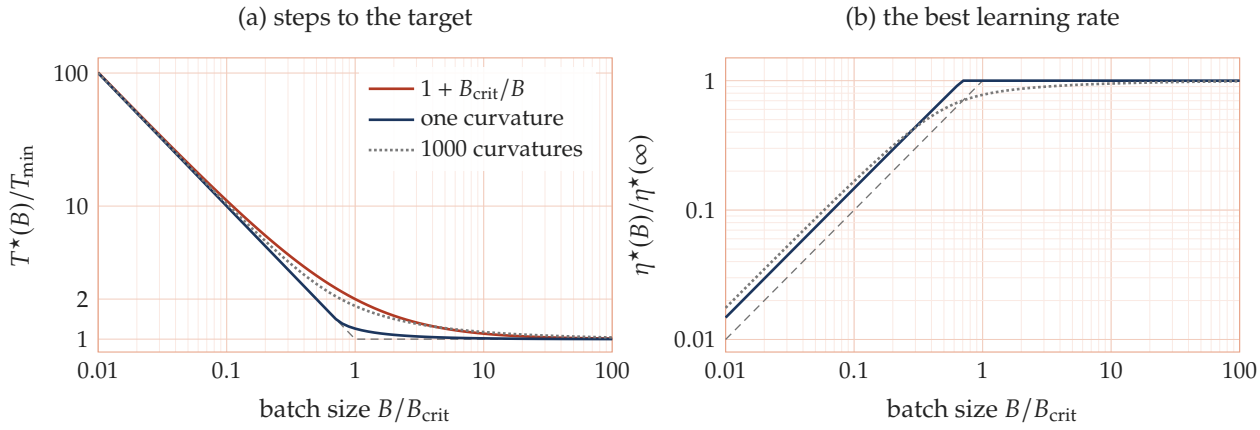


Figure 7.1. The noisy quadratic model, computed exactly (`07-heldfixed-nqm.py`; no training runs, hence no intervals). One curvature: Theorem 7.4 with target $R = \ell_0/\ell = 100$ and $\eta_{\max}\kappa = 0.05$. A thousand curvatures: $\kappa_i = 1/i$, noise variances $c_i = \kappa_i$, $\theta_i(0) = 1$, target 2% of the initial loss, learning rates up to $2/\kappa_1$. Batch sizes are in units of each model's $B_{\text{crit}} = D_{\min}/T_{\min}$. (a) The fewest steps to the target, over learning rates, against the asymptotes $\max(1, B_{\text{crit}}/B)$ (dashed) and the hyperbola of McCandlish et al. (b) The learning rate that achieves them, relative to its large-batch value; the dashed line has slope one, the linear scaling rule.

is cheap to measure and moves in the right direction, but it guides the order of magnitude at best; when the decision matters, measure B_{crit} from steps-to-target curves.[8]

[8] McCandlish et al. also describe a way to measure B_{simple} with negligible overhead during data-parallel training.

7.6 In the laboratory: batch size on the digits

Both labs train the two-layer perceptron of width 64 of Chapter 1 with plain SGD, the optimizer of the theorem, on split 0 of the digits: 1,258 training images, so that $B = 1{,}258$ is full-batch gradient descent, and 539 test images. Selection is honest by two-fold *cross-fitting*: the test images are divided at random into halves of 270 and 269, the learning rate is chosen on one half and scored on the other with fresh seeds, and then the halves swap roles, so that every reported number comes from images that played no part in choosing its learning rate.[9]

[9] A reported error counts mistakes out of all 539 images, so one image is 0.19 points of test error; the runs of Figure 7.2b fall on that grid.

LAB 7.1 · FOUR PROTOCOLS FOR ONE COMPARISON

SETUP. Batch sizes $8, 16, \ldots, 1024$ and 1,258; in every run a linear warm-up over the first 5% of the steps and cosine decay to zero. A 2×2 design: a budget of 10 epochs ($T = 10\lfloor 1258/B \rfloor$ steps) or of 1,570 steps for every batch (those of $B = 8$ in 10 epochs), crossed with a learning rate held at the value selected for $B = 8$ or selected for each B on a doubling grid, with two selection seeds. Protocols: (i) epochs and rate fixed, as in the batch-size sweep of Keskar et al.; (ii) epochs fixed, rate tuned; (iii) steps fixed, rate tuned; (iv) steps fixed, rate fixed. Ten fresh seeds per cell. About two minutes.

SCRIPT `07-heldfixed-gap.py`

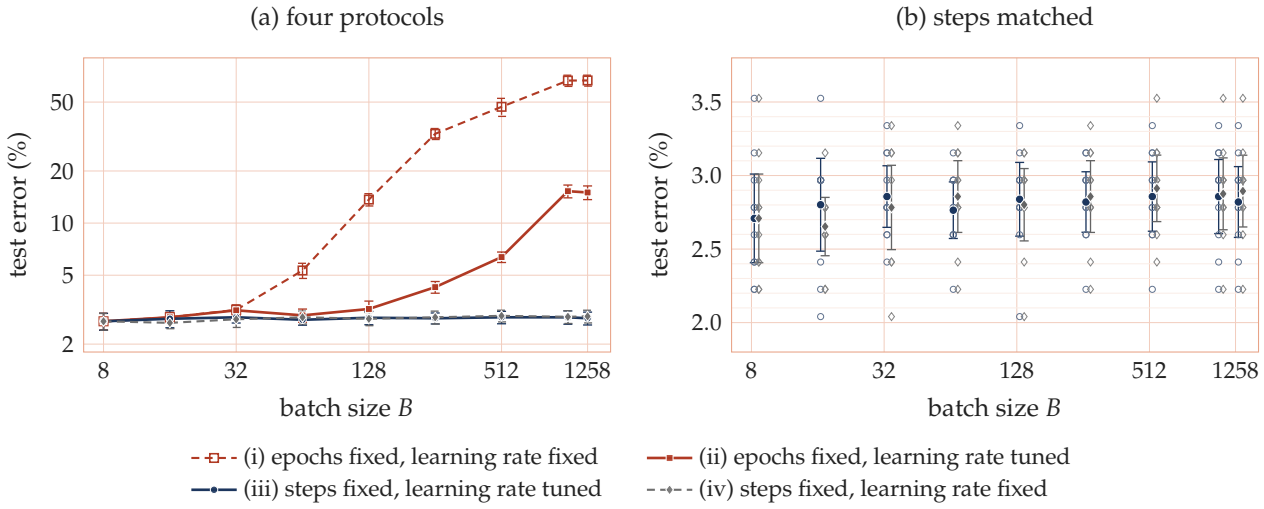


Figure 7.2. Lab 7.1: test error against batch size under the four protocols, two-layer MLP on the digits (split 0; 1,258 training and 539 test images; $B = 1{,}258$ is full batch). Each point is the mean over ten fresh seeds (100–109) of errors on the half of the test set not used to select the learning rate, with a 95% t-interval over seeds. (a) All four protocols, logarithmic scale. (b) The two protocols with matched steps on a linear scale: hollow marks are single runs (equal errors coincide; the protocols are offset horizontally), filled marks the means.

PREDICT FIRST. Under (i), how much worse is full batch than $B = 8$? At $B = 128$, a tenth of the training set as in Keskar et al., which protocols show a gap? Does tuning the rate close it at a fixed number of epochs? Does matching the steps?

LOOK FOR. Where each protocol's error starts to rise; the selected learning rates; the training loss at the end; the paired differences from $B = 8$ under (iii) and (iv).

WHAT HAPPENED. Under protocol (i), the design of Keskar et al.'s batch-size sweep in miniature, the test error grew from $2.71 \pm 0.30\%$ at $B = 8$ (mean over ten seeds, 95% t-interval) to $5.3 \pm 0.5\%$ at $B = 64$, $13.7 \pm 1.1\%$ at 128, and $66.7 \pm 4.9\%$ at full batch (Figure 7.2a). These runs had not learned the training set either: their final training loss was 0.54 at $B = 128$ and 2.21 at full batch, where a uniform guess scores 2.30. Tuning the learning rate (ii) removed the gap at moderate batches, to $3.19 \pm 0.35\%$ at $B = 128$, with the selected rate growing from 0.2 at $B \le 16$ to 1.6 at $B \ge 128$, first about in proportion to the batch and then not at all. But at a fixed number of epochs the full batch takes ten steps, and no rate made up for that: $15.0 \pm 1.4\%$, 12.3 ± 1.2 points worse than $B = 8$ in the paired comparison over seeds. With the steps matched (iii), the error stayed between 2.71% and 2.86% at every batch size, full batch included; the paired differences from $B = 8$ lay between +0.06 and +0.15 points, with half-widths from 0.33 to 0.39 (Figure 7.2b). Matching the steps alone (iv), with the rate left at its small-batch value, gave 2.65% to 2.91%. At $B = 128$ either remedy alone removed nearly all of the 11-point gap, so their interaction was nearly as

large as either effect; at full batch the steps alone removed 63.8 points and the learning rate alone 51.6.

In this miniature the fixed-epoch gap is not a gap in generalization alone: the large batches did not finish training, on the training set or the test set, as Corollary 7.5 predicts. With the steps matched, no residual gap appeared even at full batch, to within about 0.4 points; the residual that Smith et al. (2020) found for a Wide-ResNet at a third of the training set is beyond the reach of a network this small and a test set this size.

LAB 7.2 · STEPS TO A TARGET

SCRIPT
07-heldfixed-steps.py

SETUP. The network of Lab 7.1 with a constant learning rate and no warm-up, the setting of the theorem. For each batch size and learning rate $0.1 \cdot 2^{k/2}$, $k = 0, \ldots, 9$, the first step at which the test error reaches 10%, 5%, and 3.5%, evaluated after every step up to 64 and then every 3%, up to 1,500 steps. For each target and batch size the learning rate with the smallest median over four selection seeds is scored with eight fresh seeds on the other half: 16 hitting times per point. The hyperbola $T = T_{\min}(1 + B_{\text{crit}}/B_{\text{eff}})$ is fitted in log steps, with bootstrap intervals over seeds; the noise scales come from per-example gradients on the training set along a reference run. About two minutes.

PREDICT FIRST. Between $B = 8$ and $B = 16$, how much do the steps fall? Between 512 and 1,024? Which target has the largest critical batch size? How should the best learning rate change with the batch?

LOOK FOR. The slope -1 at small batches and the floor at large ones; the examples BT at small batches; the selected learning rates; the noise scales against the fitted B_{crit}.

WHAT HAPPENED. The curves have the shape of Theorem 7.4 (Figure 7.3a). For the 5% target the median steps fell from 155 at $B = 8$ (95% bootstrap interval 147 to 209) to 87.5, 48.5, and 32 as the batch doubled up to 64, while the examples BT grew only from 1,240 to 2,048; from $B = 256$ on they stayed between 18 and 20 steps, so the full batch, 157 times larger than $B = 8$, was eight times faster. The fitted critical batch sizes were 27 (25 to 34) for the 10% target and 59 (52 to 74) for 5%, with floors of 14.5 and 17.9 steps. For 3.5% the curve turned more sharply than the hyperbola, like the single curvature of Figure 7.1: the examples stayed between 2,300 and 2,800 up to $B = 64$ and the steps near 28 to 30 from 128 to 512, and the fit, 59 (41 to 66), compromises between the two. Read directly as the crossing of the asymptotes, the examples at $B = 8$ divided by the fewest steps, the critical batch sizes are 40 (35 to 50), 69 (61 to 93), and 92 (74 to 108): the more demanding the target, the larger the critical batch size, as the model predicts. The selected learning rate grew in proportion to the batch up to about $B = 64$ and then stayed between about 1 and 1.6 (Figure 7.3b), the two regimes of the theorem.

Two observations depart from the model. At the two largest batches, 81% and 100% of the training set, the steps to the 3.5% target rose again, from

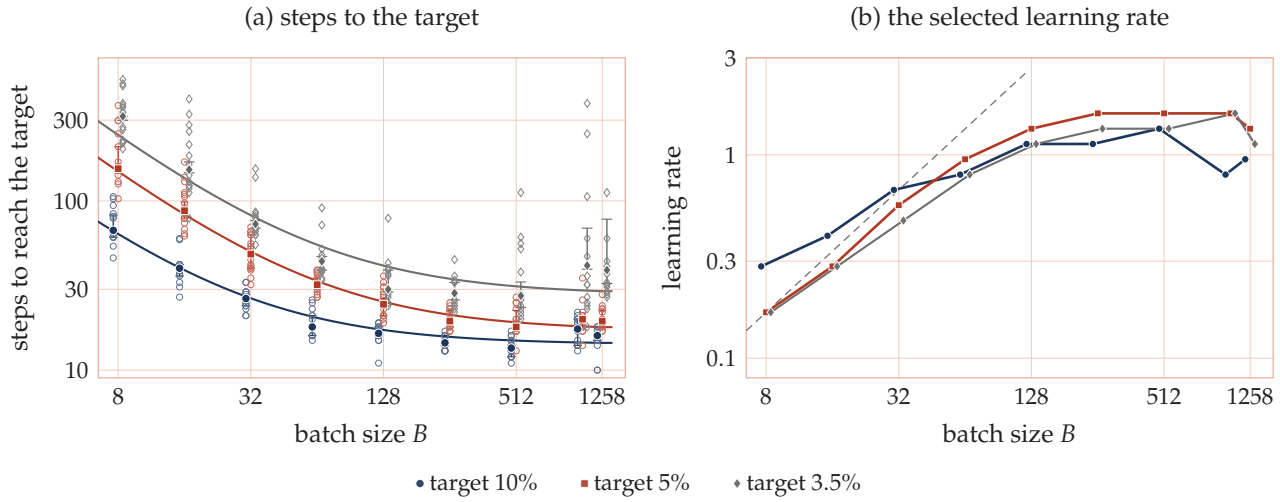


Figure 7.3. Lab 7.2: steps to a target test error against batch size, constant learning rate, the network and data of Figure 7.2. (a) Hollow marks: the 16 first-hitting steps per batch size and target (eight fresh seeds, 100–107, on each test half; the targets are offset horizontally); filled marks: their medians, with 95% bootstrap intervals over seeds; lines: the hyperbola $T_{\min}(1 + B_{\text{crit}}/B_{\text{eff}})$ fitted to the medians. (b) The learning rate selected for each batch size and target (geometric mean of the two halves' choices); the dashed line is proportional to B, the linear scaling rule.

27.5 at $B = 512$ to about 40, which the model, whose $T^\star$ is nonincreasing, cannot produce, and which Shallue et al. (2019) never saw; a possible reason is that the first passage of a noisy curve below a level comes earlier than that of a smooth one, so that moderate batches gain in the measurement as well as in the training. And the noise scales, measured along a reference run at $B = 64$, were $B_{\text{simple}} = 184$ and 284 when the run first reached 10% and 5%, five to seven times the fitted critical batch sizes, as for McCandlish et al.'s MNIST network; at 3.5% the simple noise scale fell, to 250, as the per-example gradients shrank with the training loss.

7.7 Protocol

PROTOCOL 7.1 · CHOOSING WHAT TO HOLD FIXED

- ☐ Write the claim with its budget: equal epochs, equal steps, equal operations, equal time on stated hardware, or a curve over the budget; a claim without a budget is a claim about saturated runs and needs evidence that they saturated.
- ☐ List the derived quantities that move with the treatment under that budget (steps, examples, the ratio of learning rate to batch, normalization statistics, the schedule's length) and decide for each whether it belongs to the treatment or confounds it.
- ☐ Retune, at every level of the treatment and with the same search, the hyperparameters that interact with it, such as learning rate, warm-up, momentum, schedule, and regularization; check that

no selected value sits at the edge of its grid.

- ☐ Decay every schedule within its own budget and compare final results, not intermediate checkpoints.
- ☐ For a claim about scale or parallelism, measure steps to a stated target over a range of batch sizes and report $T_{\min}$, $D_{\min}$ and B_{crit} with intervals and with their target.
- ☐ When the question does not fix the budget, cross the budget with the treatment in a factorial design and report the parts of the effect and their interaction.
- ☐ Before attributing a residual gap to a mechanism, show that it survives matched budgets and retuning, then test the mechanism by intervention (Chapters 9 and 10).

The protocol makes a comparison fair to the batch size within a stated budget; it does not make the budget the right one. It does not protect against layers whose behavior depends on the batch, such as batch normalization, whose statistics should be computed over a fixed number of examples, against optimizer settings whose best values change with scale (Chapter 12), or against a target chosen after seeing the curves. Its cost is real: matching steps multiplies the examples by the ratio of batch sizes, retuning multiplies the runs by the number of batch sizes, and first-hitting times are noisy enough to need several seeds per point. When that is unaffordable, measure the curve on a smaller proxy and transfer it with stated rules, use the noise scale as an order-of-magnitude guide, and at the least report results at two budgets, epochs and steps, so that a reader can see which one decided the comparison.

Exercises

7.1 CONSOLIDATION. (The hyperbola.) Suppose $T(B) = T_{\min}(1 + B_{\text{crit}}/B)$. (a) Show that $D(B) = BT(B) = D_{\min}(1 + B/B_{\text{crit}})$ with $D_{\min} = T_{\min}B_{\text{crit}}$, and that $(T/T_{\min} - 1)(D/D_{\min} - 1) = 1$. (b) At which batch size are the steps and the examples both twice their minimum? (c) For the single curvature of Figure 7.1, $T^\star(B_{\text{crit}}) = 1.20\,T_{\min}$; explain from the proof of Theorem 7.4 why one curvature turns more sharply than the hyperbola.

7.2 CONSOLIDATION. (The linear scaling rule.) In part (iii) of Theorem 7.4, let u_R minimize $L(u)/u$ on $(0, 1)$. (a) Show that the minimizer is unique, by differentiating $uL'(u) - L(u)$. (b) Show that as $B \to 0$ the best step satisfies $\eta^\star(B)B_{\text{noise}}\kappa/B \to 2u_R$, so that the best learning rate grows in proportion to the batch. (c) Show that the best learning rate is $\eta_{\max}$ for all large B, and compare with Figure 7.1(b).

7.3 CONSOLIDATION. ★ (Momentum in the model.) Heavy-ball momentum on Assumption 7.3 sets $\theta_{t+1} = \theta_t - \eta(\kappa\theta_t + \xi_{t+1}) + \beta(\theta_t - \theta_{t-1})$.

(a) Show that the stationary variance, when the iteration is stable, is $(1+\beta)\eta c/\big(B\kappa(1-\beta)(2(1+\beta)-\eta\kappa)\big)$. (b) Deduce that for small $\eta\kappa$ momentum acts like SGD with learning rate $\eta/(1-\beta)$, so it cannot lower $D_{\min}$. (c) Argue that it can lower $T_{\min}$ when other directions cap the step, and so raise B_{crit}. *(Hint: the recursion is an autoregression of order two; use the Yule–Walker equations.)*

7.4 CONSOLIDATION. (A fixed number of epochs.) In the single-curvature model of Figure 7.1, every run processes $D = 3D_{\min}$ examples. Using Corollary 7.5, find a batch size beyond which no learning rate reaches the target, and the limit of the best final loss as $B \to \infty$. Which observation of Keskar et al. does the model reproduce, and which does it not?

7.5 LAB. (Adam and the square-root rule.) Rerun protocol (ii) of Lab 7.1 with Adam, on a grid of learning rates around 10^{-3}. Compare the slope of the selected learning rate against B with the linear rule and with the square-root rule of Malladi et al. (2022), then repeat with β_1, β_2 and ϵ scaled as that rule prescribes.

7.6 LAB. (Momentum extends perfect scaling.) Rerun Lab 7.2 with momentum 0.9 and the learning-rate grid divided by ten. Predict from Exercise 7.3 which part of the curve moves, then check whether B_{crit} grows and whether $D_{\min}$ changes.

7.7 LAB. (Train until the loss stops improving.) Add Keskar et al.'s stopping rule to Lab 7.1: at the learning rate of protocol (i), train $B = 8$ and $B = 128$ until the training loss has not improved for ten epochs. Compare the test errors with those of protocols (i) and (iv), and count the steps each run took. What did the stopping rule hold fixed?

7.8 DESIGN. (A batch-size study on a budget.) You have 200 GPU-hours to choose the batch size for a model whose run at batch 256 takes two hours on eight GPUs, and whose throughput grows almost linearly with the batch up to 4,096. Design a study that estimates B_{crit} for your target with an interval: which batch sizes, how many learning rates and seeds at each, whether to use a proxy, and what to report.

7.9 DESIGN. (An efficiency claim.) A method is claimed to reach a baseline's accuracy in half the epochs while doubling the batch size. Design the experiment that tests whether it saves wall-clock time, and the one that tests whether it saves computation, saying what each holds fixed and which schedule each arm uses.

7.10 CRITIQUE. (Half the epochs.) A paper reports that its method reaches the baseline's test accuracy in half the epochs, comparing its curve with the curve of the baseline's standard 90-epoch run read at 45 epochs. List what the comparison holds fixed and what it does not, describe the baseline run it should have used, and say in which direction each omission biases the claim.

7.11 CRITIQUE. (Tuned at the base batch.) A study tunes the learning rate at batch 128, scales it linearly up to 32,768 at a fixed number of epochs, and reports the rising error beyond 4,096 as evidence that large batches generalize worse. What can the study show and what can it not, and which two additional runs per batch size would most change its conclusion?

7.12 RESEARCH. (The residual gap.) Under matched steps and tuned learning rates, Smith et al. (2020) found large batches worse at a third of the training set and more, and Lab 7.1 found nothing at full batch. Open: a criterion, computable early in training, that predicts whether and at which batch size a residual gap appears under matched steps, and an experiment that separates its candidate causes, such as gradient noise, normalization statistics, and the curvature reached, by intervening on each. Also open: what replaces the critical batch size when the gradient noise is far from isotropic or changes as fast as the loss.

Notes and References

The idea that small batches are better is older than deep learning: LeCun et al. (1998) argued it for on-line learning, and Wilson and Martinez (2003) found that full-batch training needed many more passes through the data than on-line training, with no apparent difference in accuracy. Keskar et al. (2017) made it a claim about generalization and gave it the explanation of Hochreiter and Schmidhuber (1997), flat minima, which Dinh et al. (2017) and Chapter 10 examine. The response came within three years: regime adaptation and ghost batches (Hoffer et al. 2017), linear scaling with warm-up (Goyal et al. 2017), the noise scale (Smith et al. 2018; Smith and Le 2018), the ratio of learning rate to batch size and the width of minima (Jastrzębski et al. 2017), steps to a result (Shallue et al. 2019), the gradient noise scale (McCandlish et al. 2018), and the noisy quadratic model (Zhang et al. 2019). Ma et al. (2018) derived a critical batch size for SGD in the interpolation regime: below it one step with a batch of m is nearly worth m steps with a batch of one, above it nearly one step of full-batch gradient descent, and it is nearly independent of the size of the data.

The critical batch size became a planning quantity for large models. Kaplan et al. (2020) found that it grows as a power of the loss reached, approximately doubling for every 13% decrease in loss, and does not depend directly on model size; Zhang et al. (2025) found it to scale with the size of the data rather than of the model. Whether a generalization benefit of small batches survives matched steps remains debated: Smith et al. (2020) found one at very large batches, and Geiping et al. (2022) replaced it by explicit regularization. Unfair budgets are not special to batch sizes: Kaddour et al. (2023) and Chapters 4 and 12 meet them in efficient training, optimizer comparisons, and scaling laws.

TRANSLATING THE LITERATURE. Keskar et al. (2017) write x for our θ, f for the training loss $\widehat{\mathcal{L}}$, α_k for the learning rate, and $|B_k|$ for the batch size, call

the regimes SB and LB, and use ϵ for the size of the box of their sharpness measure, unrelated to Adam's ϵ. McCandlish et al. (2018) write ϵ for the learning rate, S and E for our steps T and examples D, $S_{\min}$ and $E_{\min}$ for $T_{\min}$ and $D_{\min}$, and $\epsilon_{\max}$ for our η_{gr}; B_{noise}, B_{simple}, and B_{crit} keep their names. Zhang et al. (2019) write α for the learning rate and h_i for our curvatures κ_i, and their steady-state risk is our f_∞. Smith and Le (2018) write ϵ for the learning rate and N for our n_{tr}. Shallue et al. (2019) say metaparameters for hyperparameters, workload for a model, data set, and optimizer together, and steps to result for our $T(B)$. Hoffer et al. (2017) write M for the batch size and call matched steps regime adaptation; Goyal et al. (2017) write kn for a batch of k workers with n examples each.

References

Cohen, J. M., S. Kaur, Y. Li, J. Z. Kolter, and A. Talwalkar (2021). "Gradient Descent on Neural Networks Typically Occurs at the Edge of Stability". In: *International Conference on Learning Representations (ICLR)*.

Dehghani, M., Y. Tay, A. Arnab, L. Beyer, and A. Vaswani (2022). "The Efficiency Misnomer". In: *International Conference on Learning Representations (ICLR)*.

Dinh, L., R. Pascanu, S. Bengio, and Y. Bengio (2017). "Sharp Minima Can Generalize for Deep Nets". In: *Proceedings of the 34th International Conference on Machine Learning*. Vol. 70. Proceedings of Machine Learning Research, pp. 1019–1028.

Geiping, J., M. Goldblum, P. E. Pope, M. Moeller, and T. Goldstein (2022). "Stochastic Training Is Not Necessary for Generalization". In: *International Conference on Learning Representations (ICLR)*.

Goyal, P. et al. (2017). *Accurate, Large Minibatch SGD: Training ImageNet in 1 Hour*. Preprint, arXiv:1706.02677.

Hochreiter, S. and J. Schmidhuber (1997). "Flat Minima". In: *Neural Computation* 9.1, pp. 1–42.

Hoffer, E., I. Hubara, and D. Soudry (2017). "Train Longer, Generalize Better: Closing the Generalization Gap in Large Batch Training of Neural Networks". In: *Advances in Neural Information Processing Systems 30*.

Jastrzębski, S., Z. Kenton, D. Arpit, N. Ballas, A. Fischer, Y. Bengio, and A. Storkey (2017). *Three Factors Influencing Minima in SGD*. Preprint, arXiv:1711.04623.

Kaddour, J., O. Key, P. Nawrot, P. Minervini, and M. J. Kusner (2023). "No Train No Gain: Revisiting Efficient Training Algorithms for Transformer-Based Language Models". In: *Advances in Neural Information Processing Systems 36*.

Kaplan, J. et al. (2020). *Scaling Laws for Neural Language Models*. Preprint, arXiv:2001.08361.

Keskar, N. S., D. Mudigere, J. Nocedal, M. Smelyanskiy, and P. T. P. Tang (2017). "On Large-Batch Training for Deep Learning: Generalization Gap and Sharp Minima". In: *International Conference on Learning Representations (ICLR)*. arXiv:1609.04836.

Krizhevsky, A. (2014). *One Weird Trick for Parallelizing Convolutional Neural Networks*. Preprint, arXiv:1404.5997.

LeCun, Y., L. Bottou, G. B. Orr, and K.-R. Müller (1998). "Efficient BackProp". In: *Neural Networks: Tricks of the Trade*. Ed. by G. B. Orr and K.-R. Müller. Vol. 1524. Lecture Notes in Computer Science. Springer.

Ma, S., R. Bassily, and M. Belkin (2018). "The Power of Interpolation: Understanding the Effectiveness of SGD in Modern Over-Parametrized Learning". In: *Proceedings*

of the 35th International Conference on Machine Learning. Vol. 80. Proceedings of Machine Learning Research, pp. 3325–3334.

Malladi, S., K. Lyu, A. Panigrahi, and S. Arora (2022). "On the SDEs and Scaling Rules for Adaptive Gradient Algorithms". In: *Advances in Neural Information Processing Systems 35*.

Masters, D. and C. Luschi (2018). *Revisiting Small Batch Training for Deep Neural Networks*. Preprint, arXiv:1804.07612.

McCandlish, S., J. Kaplan, D. Amodei, and OpenAI Dota Team (2018). *An Empirical Model of Large-Batch Training*. Preprint, arXiv:1812.06162.

Shallue, C. J., J. Lee, J. Antognini, J. Sohl-Dickstein, R. Frostig, and G. E. Dahl (2019). "Measuring the Effects of Data Parallelism on Neural Network Training". In: *Journal of Machine Learning Research* 20.112, pp. 1–49.

Smith, S. L., E. Elsen, and S. De (2020). "On the Generalization Benefit of Noise in Stochastic Gradient Descent". In: *Proceedings of the 37th International Conference on Machine Learning*. Vol. 119. Proceedings of Machine Learning Research, pp. 9058–9067.

Smith, S. L., P.-J. Kindermans, C. Ying, and Q. V. Le (2018). "Don't Decay the Learning Rate, Increase the Batch Size". In: *International Conference on Learning Representations (ICLR)*.

Smith, S. L. and Q. V. Le (2018). "A Bayesian Perspective on Generalization and Stochastic Gradient Descent". In: *International Conference on Learning Representations (ICLR)*.

Wilson, D. R. and T. R. Martinez (2003). "The General Inefficiency of Batch Training for Gradient Descent Learning". In: *Neural Networks* 16.10, pp. 1429–1451.

Zhang, G., L. Li, Z. Nado, J. Martens, S. Sachdeva, G. E. Dahl, C. J. Shallue, and R. Grosse (2019). "Which Algorithmic Choices Matter at Which Batch Sizes? Insights from a Noisy Quadratic Model". In: *Advances in Neural Information Processing Systems 32*.

Zhang, H., D. Morwani, N. Vyas, J. Wu, D. Zou, U. Ghai, D. Foster, and S. Kakade (2025). "How Does Critical Batch Size Scale in Pre-training?" In: *International Conference on Learning Representations (ICLR)*.

Part III

Causes

Knowing that a component helps is not knowing why. An ablation removes a part and watches what changes, a mechanism claim names what the part does, and a correlation across many runs suggests a cause without isolating it. Each has a design that makes it informative and a way of going wrong.

8 Ablations

Interactions and the home-field advantage

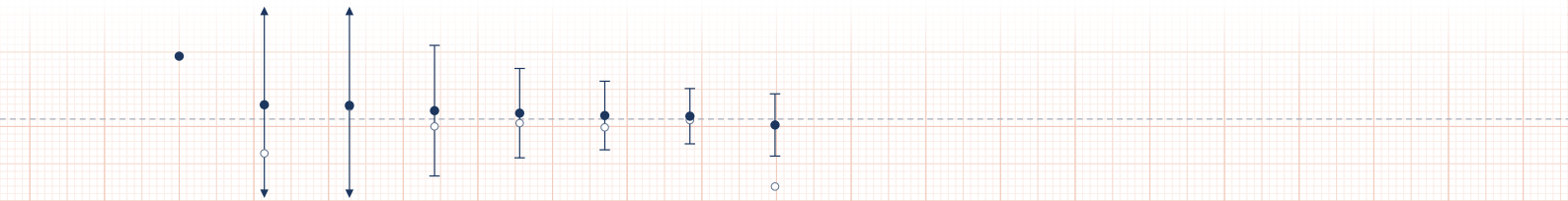
THE CLAIM *"A winning ticket trains well because of its original initialization."* — *Frankle and Carbin (2019), for the networks in which they found winning tickets; received as a property of pruned networks in general*

THE VERDICT Narrowed: the original initialization matters in small networks and at small learning rates; at the standard learning rates of deeper networks, only weights from early in training do.

IN THIS CHAPTER. In 2018 Frankle and Carbin found that iterative magnitude pruning uncovers sparse subnetworks which, reset to their original initial weights, train to the accuracy of the full network, while the same subnetworks with fresh initial weights do not: the winning tickets had won the initialization lottery. The ablation behind this conclusion ran at learning rates chosen for the tickets, and later work found its effect to depend on the learning rate: at the standard rates of deeper networks, reinitialized subnetworks did as well, and only weights rewound to early in training restored the advantage. The method treats an ablation as a factorial experiment. We prove that a leave-one-out ablation measures a main effect plus every interaction of the removed component, that a two-level factorial design estimates main effects with $(k+1)/2$ times less variance than one factor at a time, and that an ablation run at the full system's tuned hyperparameters overstates the re-tuned effect by exactly the ablated system's regret, a bias to which tuning on noisy scores adds its own. In the laboratory, the original initialization of a pruned network on the digits lowers its test error by 41 points at learning rate 0.02 and by 1.2 points at 0.2, two rates at which the dense network does equally well; with each arm at its own best rate, the effect is 0.9 points.

8.1 The claim: winning the initialization lottery

Pruning a trained network is an old idea. Optimal Brain Damage removed the weights whose deletion an estimate of the loss's curvature predicted to matter least (LeCun et al. 1990), and Han et al. (2015) removed the weights of smallest magnitude and retrained the rest, shrinking AlexNet from 61 to 6.7 million parameters without loss of accuracy. But, as they also reported, the pruned layers did worse when reinitialized and retrained than when they kept their trained weights. If a network can be that small after training, why can it not be trained small?



Jonathan Frankle and Michael Carbin proposed an answer in a preprint of March 2018, published at ICLR the following year (Frankle and Carbin 2019). Their procedure, *iterative magnitude pruning*, trains a network, removes a fixed fraction of the weights of smallest magnitude in each layer, resets the surviving weights to the values they had at initialization, and repeats; one-shot pruning removes the whole fraction after a single training. The surviving connections form a binary *mask* m on the weights, and the subnetwork with mask m and the original initial values is what they called a *winning ticket* when it trains, in isolation and in at most the same number of iterations, to the test accuracy of the full network. The *lottery ticket hypothesis* states that dense, randomly initialized networks contain such subnetworks.[1] The decisive experiment was an ablation: keep the mask, draw new initial values from the same distribution, and train again.

[1] In the authors' metaphor a dense network holds many subnetworks, as a player holds many tickets, and its random initialization is the draw.

> FINDING 8.1 (Frankle and Carbin 2019, Sections 2 and 3). For a fully connected network on MNIST (Lenet-300-100, 266,000 weights, trained with Adam), iteratively pruned subnetworks reset to their initial values were at least as accurate as the full network down to 3.6% of its weights, and more accurate at intermediate sizes; the same masks with fresh initial values fell below the full network from 21% of the weights on. At 21%, the ticket reached its minimum validation loss 2.51 times faster than its reinitialized copy and was half a point more accurate. Convolutional networks on CIFAR-10 behaved alike, with winning tickets at 10–20% of the weights or fewer. Points are means of five trials (fifteen for reinitialized copies), with the minimum and maximum shown.

The authors' reading was in their abstract: the winning tickets had won the initialization lottery, their connections having initial weights that make training particularly effective. A conjecture, labeled as untested, went further: that stochastic gradient descent finds and trains a well-initialized subnetwork, and that large networks are easier to train because they hold more candidate tickets.

The paper also reported where the pattern failed. For two deeper networks on CIFAR-10, VGG-19 and a residual network,[2] trained with SGD and momentum at the standard learning rate of 0.1, iterative pruning found no winning tickets, and the pruned subnetworks did no better than their reinitialized copies. At 0.01 the familiar pattern returned, at accuracies below what the full network reached at 0.1. A linear warm-up of the learning rate made winning tickets appear for VGG-19 at rate 0.1, down to 1.5% of the weights, and for ResNet-20 at rate 0.03. The limitations section says plainly that on these networks winning tickets were found only with warm-up, and leaves the reason to future work.

[2] The paper calls it Resnet-18; Frankle et al. (2020) note that it is the 20-layer ResNet of He et al., and we call it ResNet-20.

RECEPTION. The paper shared the best-paper award of ICLR 2019, and a literature followed on how winning tickets are found, transferred and explained. The phrase that traveled was the lottery: a pruned network is trainable because of its original initial values. The qualification that came with the deeper networks, that this role of the initialization had been observed at learning rates chosen for it to appear, traveled less well.

WHAT THE WORK GOT RIGHT. The experiment had the controls that most pruning papers lacked (Blalock et al. 2020): a reinitialized copy of every ticket, randomly sparse networks of the same size, and in the appendices a reinitialization drawn from the distribution of the ticket's own initial values (little better than a plain one) and pruning by initial magnitude (worse). The appendices report more than 3,000 runs of the MNIST network alone, every point is a mean over trials with its range, and the sensitivity to the learning rate was reported, not hidden. The effect is real where it was found, as the laboratory of this chapter confirms. And when Liu et al. (2019) reported that reinitialized subnetworks of VGG-19 matched the full network, Frankle and Carbin replied in their final version that this held up to the 80% pruning for which Liu and colleagues had then reported data, but that pruned further the original initialization mattered: an interaction with sparsity, recognized as one.

8.2 The experiment: one ablation at the ticket's learning rate

EXPERIMENTAL DESIGN · FRANKLE AND CARBIN 2019, MNIST AND CIFAR-10

QUESTION	Does a pruned subnetwork owe its trainability to its original initial values?
UNITS	training runs of Lenet-300-100 on MNIST and of Conv-2, -4 and -6 on CIFAR-10
TREATMENT	the initial values of the surviving weights: the original ones (ticket) or a fresh draw from the same distribution (reinitialized copy)
LEVELS	fraction of weights remaining, from 100% to below 1%, by iterative or one-shot pruning
TUNED	the learning rate (Adam) of each network, to make tickets match the full network at small sizes
HELD FIXED	optimizer, learning rate, iterations, batch size and pruning rates, the same for both arms
RESPONSE	iteration of minimum validation loss; test accuracy at that iteration
REPLICATES	five pruning trials per point; three reinitializations of each ticket
ANALYSIS	means over trials, with minimum and maximum

The record has two features that this chapter is about. The first is in the row TUNED. The appendices choose the learning rate by three criteria: it should be good for the unpruned network; it should let the lottery-ticket experiment match the full network with as few weights as possible; and among such rates it should be on the slow side.[3] The second criterion selects the hyperparameter for the success of the treatment arm, and the reinitialized copies were then trained with the same hyperparameters: the ablation was measured on the ticket's home field.

[3] The appendix gives the reason for the third criterion: a slower rate is more likely to train heavily pruned networks under a variety of conditions.

The second feature is in the rows TREATMENT and HELD FIXED. The design varies one factor, the initial values, at one level of everything else, and its result is a *simple effect*: the effect of the initialization at that learning rate, optimizer, number of iterations and sparsity. Whether the effect belongs to the subnetwork or to the subnetwork together with these settings, a design with one level of each setting cannot say. For the deeper networks the authors did vary the learning rate, in effect running a two-by-two design of initialization by learning rate, and the effect of the initialization changed with the rate. That observation is an interaction, and the design that saw it was the right one; it was reported as a sensitivity of the procedure for finding tickets rather than as a limit on what the tickets showed.

The design could show that at the hyperparameters chosen these masks trained better and faster with their original values than with new ones, by margins far beyond the trial-to-trial range. It could not show whether a reinitialized subnetwork given its own learning rate or more iterations would close the gap, or how the gap depends on the settings held fixed.

8.3 The post-mortem: the learning rate was a factor

RETHINKING THE VALUE OF PRUNING. Liu et al. (2019), published at the same conference,[4] found that for structured pruning methods, which remove whole channels, training the pruned architecture from a random initialization did as well as fine-tuning its inherited weights. In a section on the lottery ticket hypothesis they ran the ablation on VGG-16 and ResNet-50 on CIFAR-10, pruning the convolutional layers by magnitude, with SGD and momentum at two initial learning rates and five runs per point. Of the ways in which the two studies differed (network size, optimizer, learning rate), they identified the learning rate as the one that reconciled them.

[4] The first arXiv version of October 2018 was substantially revised for the conference; we cite the published version.

> FINDING 8.2 (Liu et al. 2019, Section 6 and Table 9). At initial learning rate 0.1, subnetworks with the original initial values and reinitialized ones reached the same accuracy within 0.2 points, at every pruning ratio from 20% to 95%. At 0.01 the original values helped (on VGG-16 pruned to 5% of its convolutional weights, by 0.75 points), but at each pruning ratio every network trained at 0.01 was less accurate than its counterpart at 0.1, the original-initialization subnetworks included. For structured pruning, the original values did not help at either rate.

They concluded that the winning ticket improves on a random initialization only for unstructured pruning at a small learning rate, which yields lower accuracy than the standard one, and suggested a reason: with small steps the trained weights stay close to their initial values, so pruning keeps the weights that started large. Gale et al. (2019), in a preprint benchmarking sparsification at scale, repeated the ablation for a Transformer on WMT English–German and ResNet-50 on ImageNet and could not reproduce the effect; they attributed the difference to the complexity of the tasks and the scale of the models.[5]

[5] With the preprint they released code, their best checkpoints and the results of every hyperparameter configuration they ran.

network (data)	learning rate	kept	full	ticket	reinit.	effect
VGG-16 (CIFAR-10)[a]	0.1	20%	93.76	93.74	93.72	+0.02
	0.01	20%	92.69	92.75	92.07	+0.68
	0.1	5%	93.76	93.18	93.05	+0.13
	0.01	5%	92.69	92.58	91.83	+0.75
Lenet-300-100 (MNIST)[b]	Adam, 1.2×10^{-3}	3.5%	98.3	98.2	97.5	+0.7
ResNet-20 (CIFAR-10)[b]	0.1	16.8%	91.7	88.5	88.8	−0.3
	0.01	8.6%	88.8	89.0	84.7	+4.3
	0.03, warm-up	8.6%	89.7	89.6	85.6	+4.0
VGG-16 (CIFAR-10)[b]	0.1	1.5%	93.7	90.9	91.0	−0.1
	0.01	5.5%	91.7	91.6	90.2	+1.4
	0.1, warm-up	1.5%	93.4	93.2	90.7	+2.5
ResNet-50 (ImageNet)[b]	0.4, warm-up	30%	76.1	73.7	73.4	+0.3

Table 8.1. The effect of the original initialization as reported. Test accuracy (%) of the full network, of the pruned subnetwork with its original initial values (ticket) and with fresh ones; the effect is ticket minus reinitialized. "Kept" is the fraction of weights remaining. [a] Liu et al. (2019), Table 9: one-shot pruning of the convolutional layers, means of five runs (standard deviations 0.07–0.25). [b] Frankle et al. (2020), Tables 1 and 2: iterative pruning (one-shot for ResNet-50), at the sparsest level at which some rewinding point gives a matching subnetwork; means of three runs (standard deviations below 0.2).

STABILITY AND REWINDING. The original authors and two colleagues located the mechanism in the early phase of training (Frankle et al. 2020). A network is *stable to SGD noise* from a point in training if two copies trained on from there with different data orders end in minima joined by a straight line of low error (Chapter 2). Only the small MNIST network was stable from initialization; ResNet-20 became stable 3% of the way through training, ResNet-50 on ImageNet after 20%. At the sparsest levels studied, pruned subnetworks matched the full network when they were stable and not otherwise. Rewinding the surviving weights not to their initial values but to their values at an early step k, an idea first circulated in a preprint (Frankle et al. 2019), produced matching subnetworks at the standard learning rate: for ResNet-20 and VGG-16 at iterations 500 and 1000 (0.8% and 1.6% of training), for ResNet-50 and Inception-v3 on ImageNet at epochs 5 and 6. Reinitialized and randomly pruned subnetworks matched at no rewinding point. The paper notes that the low-rate and warm-up variants had been selected by Frankle and Carbin specifically for the procedure to find matching subnetworks.

Table 8.1 collects the numbers. Read as a whole it is a factorial experiment that nobody designed: the effect of the original initialization ranges from under one point to four points at small learning rates or with warm-up, and is within a few tenths of zero at the standard rates of the larger networks, whose full networks are also the most accurate.

A SHAM FOR THE MASK. Does a mask contribute which weights survive, or only how many survive in each layer? Frankle et al. (2021) answered

with a sham ablation. For methods that prune at initialization, shuffling each layer's mask at random, keeping the number of weights per layer, maintained or improved accuracy, and reinitializing did not hurt either: the per-weight decisions could be replaced by a per-layer fraction. Magnitude pruning after training lost accuracy under both ablations. Shuffling within layers is the random baseline with the same layerwise proportions that Blalock et al. (2020) recommended for every pruning paper; in their survey of 81 papers, a quarter compared with no other pruning method at all.

WHAT IN THE INITIALIZATION? Zhou et al. (2019) decomposed the original values: reinitializing, permuting the surviving values within each layer, or setting them to a constant all trained worse than the original values, and most of the loss disappeared when the new values kept the original signs. Ma et al. (2021) traced the conflicting reports to the training recipe: winning tickets appeared at small learning rates or with too few epochs, whose dense networks are weaker and easier to match, and often not at a learning rate of 0.1 with full training. Their definition of a ticket requires that at some learning rate it beat the reinitialized subnetwork at every learning rate by a clear margin: the ablated arm gets its own best setting.

THE MECHANISM OF THE FAILURE. The claim generalized a simple effect measured at settings chosen so that the treatment would succeed. The effect of the initial values interacts with the learning rate and warm-up, the sparsity, the size of the network, the number of epochs and the point to which weights are rewound; the original experiments sat at corners of this space where the effect is large. Nothing in the original measurements was wrong; what did not survive is the step from "the initialization matters here" to "the initialization is why pruned networks train".

VERDICT. The original initial values do make a pruned subnetwork train better in small networks, and in deeper ones at small learning rates or with warm-up, as Frankle and Carbin reported. At the standard learning rates of deeper networks, reinitialized subnetworks match the original ones, and only weights rewound to an early point of training restore an advantage, which coincides with stability to the noise of SGD. The effect is an interaction of the initialization with the training recipe, and the ablation that established it ran at settings chosen for the ticket.

NARROWED

8.4 The method: an ablation is a factorial experiment

An *ablation* removes or replaces a component of a system and measures what changes. Consider a system with k components: modules of an architecture, terms of a loss, steps of a training recipe, or, in the lottery-ticket experiment, the learned mask and the original initial values. Code the two levels of component j as $x_j \in \{-1, +1\}$, with $+1$ the level of the full system, and write $\mu(x)$ for the expected outcome of a run at levels $x = (x_1, \ldots, x_k)$: the mean of the run distribution of Chapter 1, with everything else held fixed. A *leave-one-out ablation* runs the full system $\mathbf{1} = (1, \ldots, 1)$ and, for each j, the

system $\mathbf{1}^{(j)}$ with component j alone at -1. It is the *one-factor-at-a-time* design of the older sciences, with the full system as its baseline. Chapter 6 codes a sequence of changes the same way, and we use its notation: for a set S of components, $c_S(x) = \prod_{j \in S} x_j$, with $c_\emptyset = 1$.

Definition 8.3 (Factorial and simple effects). For nonempty S, the *effect* of S is

$$\theta_S = 2^{1-k} \sum_{x \in \{-1,1\}^k} c_S(x)\, \mu(x),$$

the mean of μ over the 2^{k-1} level combinations with $c_S = +1$ minus its mean over those with $c_S = -1$: the *main effect* θ_j of component j when $S = \{j\}$, an *interaction* when S has two or more elements. For a setting z of the components other than j, the *simple effect* of j is $\delta_j(z) = \mu(x_j{=}1, z) - \mu(x_j{=}{-1}, z)$, and its *leave-one-out effect* is the simple effect at the full system, $\delta_j = \mu(\mathbf{1}) - \mu(\mathbf{1}^{(j)})$.

A main effect averages the effect of a component over all settings of the others; a simple effect fixes them. The expansion of Lemma 6.8 relates the two: every cell mean is $\mu(x) = \bar{\mu} + \frac{1}{2}\sum_{S \neq \emptyset} \theta_S\, c_S(x)$, with $\bar{\mu}$ the mean of the 2^k cell means, and every simple effect is $\delta_j(z) = \sum_{S \ni j} \theta_S\, c_{S \setminus \{j\}}(z)$.[6] At the full system every sign is $+1$, so

$$\delta_j = \theta_j + \sum_{S \ni j,\ |S| \geq 2} \theta_S, \tag{8.1}$$

the main effect plus every interaction that involves j. Averaged over the settings z of the other components, each sign $c_{S \setminus \{j\}}$ with $S \neq \{j\}$ averages to zero, and the simple effects average to the main effect.

A row of an ablation table is therefore not "the contribution of component j" unless the interactions that involve j vanish. When they do not, the rows can sum to more or less than the gain of the full system over the system with every component removed, and they change with the baseline. Chapter 6 shows how attributions along a path of changes depend on its order through the same interaction terms, and how the Shapley value averages over orders. Here we compare designs.

Theorem 8.4 (Factorial against one at a time). *Let the runs be independent, a run at levels x having mean $\mu(x)$ and variance $\sigma^2 > 0$, and let N runs be available.*

(i) In the full factorial design with $r \geq 1$ runs at each of the 2^k level combinations, so that $N = r2^k$, the estimate $\hat{\theta}_S$, the mean of the runs with $c_S = +1$ minus the mean of the runs with $c_S = -1$, is unbiased for θ_S with variance $4\sigma^2/N$, and the $2^k - 1$ estimates are uncorrelated.

(ii) In the leave-one-out design with r runs at $\mathbf{1}$ and at each $\mathbf{1}^{(j)}$, so that $N = r(k+1)$, the difference $\hat{\delta}_j$ of the two cell means is unbiased for δ_j with variance $2(k+1)\sigma^2/N$, and $\mathrm{Corr}(\hat{\delta}_i, \hat{\delta}_j) = 1/2$ for $i \neq j$.

(iii) Suppose further that $k \geq 2$ and that a run at x has outcome $\mu(x) + \varepsilon$,

[6] In the analysis of Boolean functions the c_S are the parity, or Walsh, functions of the cube, and this expansion is the Fourier expansion of μ.

where the joint distribution of the errors does not depend on μ. Then in the leave-one-out design no statistic is unbiased for an interaction θ_S for every μ, and none is unbiased for a main effect θ_j for every μ.

Consequently, when the interactions vanish, both designs estimate the main effects, and the leave-one-out design needs $(k+1)/2$ times as many runs as the factorial design for the same variance.

PROOF. The plan is to compute the first two parts directly and to prove the third by exhibiting a change of the mean function that the leave-one-out design cannot see. If $k = 1$, then the two designs coincide and the third part is vacuous, so the argument is needed only for $k \geq 2$.

Part (i). Each of the two means in $\hat{\theta}_S$ averages 2^{k-1} cell means of r runs each, so its expectation is the mean of μ over the corresponding level combinations, and $\mathbb{E}\hat{\theta}_S = \theta_S$ by Definition 8.3. The variance $4\sigma^2/N$ and the absence of correlation are Proposition A.13, whose hypotheses, a full two-level design with independent runs of common variance, are those of this part.

Part (ii). The $k+1$ cell means are independent, each with variance σ^2/r and with mean μ at its cell. Hence $\hat{\delta}_j$ is unbiased for δ_j, and its variance is $2\sigma^2/r = 2(k+1)\sigma^2/N$. For $i \neq j$, $\hat{\delta}_i$ and $\hat{\delta}_j$ share the cell mean at $\mathbf{1}$ and nothing else, so their covariance is its variance σ^2/r, and the correlation is $(\sigma^2/r)/(2\sigma^2/r) = 1/2$.

Part (iii). The design points $\mathbf{1}$ and $\mathbf{1}^{(j)}$ are the level combinations with at most one coordinate equal to -1. For a set S with at least two elements let

$$\psi_S(x) = \prod_{i \in S} \tfrac{1}{2}(1 - x_i),$$

which equals 1 when $x_i = -1$ for every $i \in S$ and 0 otherwise, and so vanishes at every design point. Expanding the product gives $\psi_S = 2^{-|S|} \sum_{T \subseteq S} (-1)^{|T|} c_T$. The signs c_T are an orthogonal basis of the functions on the cube (proof of Lemma 6.8), so the expansion of a function in them is unique, and comparing with Lemma 6.8 the effects of ψ_S are $\theta_T = 2^{1-|S|}(-1)^{|T|}$ for nonempty $T \subseteq S$ and zero for other T; in particular θ_S and, for $j \in S$, θ_j are nonzero. The mean functions μ and $\mu + \psi_S$ agree at every design point, so by the hypothesis on the errors every statistic has the same expectation under both, while θ_S and θ_j, which are linear in the mean function, differ between them. No statistic is unbiased for either under both. For a main effect θ_j, take $S = \{i, j\}$ with some $i \neq j$, which exists because $k \geq 2$.

The comparison. If the interactions vanish, then $\delta_j = \theta_j$ by (8.1), and the ratio of the variances in (ii) and (i) is $(k+1)/2$; since both variances are inversely proportional to N, this is the ratio of the run counts needed for equal variance. ∎

With three components the factorial design gives every main effect the precision of a leave-one-out design with twice the runs, with seven components four times, and it estimates every interaction as precisely: each of its runs serves every effect, while a leave-one-out run serves one comparison, and the shared baseline correlates the rows.

Two remarks keep the theorem in proportion. First, the leave-one-out design is the efficient design for its own estimand. If the question is what the shipped system loses without component j, the estimand is δ_j, which a factorial design estimates only as a difference of two of its 2^k cell means, with variance $2^{k+1}\sigma^2/N$, larger than $2(k+1)\sigma^2/N$ when $k \geq 2$; and one factor at a time can be right when runs are few, effects large and the aim is to improve a system step by step (Daniel 1973). Second, the factorial design needs 2^k cells. A *fractional factorial* design runs a fraction of them, chosen so that main effects are confounded only with interactions believed small (Exercise 8.3); with eight components, sixteen cells estimate all main effects clear of two-factor interactions (Box et al. 2005).

8.5 The method: the home-field advantage and the sham

The factorial argument assumes that the levels of every other variable are fixed in advance. In practice the hyperparameters of a system are tuned, and they are tuned for the full system. The ablated system then runs at hyperparameters that were chosen for a system it is not.

Definition 8.5 (Fixed and re-tuned effects). Let $\mathcal{H}$ be a finite set of hyperparameter settings, and let $f(h)$ and $g(h)$ be the expected test errors of the full system and of the ablated system trained with $h \in \mathcal{H}$. Write $f^\star = \min_{\mathcal{H}} f$ and $g^\star = \min_{\mathcal{H}} g$. The *effect of the ablation at* h is $\Delta(h) = g(h) - f(h)$, the *re-tuned effect* is $\Delta^\star = g^\star - f^\star$, and the *regrets* of the two systems at h are $r_f(h) = f(h) - f^\star$ and $r_g(h) = g(h) - g^\star$.

Errors are to be minimized, so a positive effect means that the component helps. The effect $\Delta(h)$ asks whether this recipe, at these settings, needs the component; $\Delta^\star$ asks whether the component improves the best that can be done with the search space $\mathcal{H}$. The lottery ticket hypothesis asks the second question: a claim that a subnetwork trains well *because* of its initial values is refuted by a reinitialized copy that trains as well with a learning rate of its own.

Proposition 8.6 (The home-field advantage). *For every* $h \in \mathcal{H}$,

$$\Delta(h) = \Delta^\star + r_g(h) - r_f(h).$$

In particular, if $h^\star$ *minimizes* f, *then* $\Delta(h^\star) = \Delta^\star + r_g(h^\star) \geq \Delta^\star$, *with equality if and only if* $h^\star$ *also minimizes* g.

Proof. Substituting $f(h) = f^\star + r_f(h)$ and $g(h) = g^\star + r_g(h)$ into $\Delta(h) = g(h) - f(h)$ gives the identity. At a minimizer $h^\star$ of f the regret $r_f(h^\star)$ is zero, and $r_g(h^\star) \geq 0$ because $g^\star$ is the minimum of g, with equality exactly when $g(h^\star) = g^\star$. ∎

The proposition is an identity, and its content is in what it makes visible. An ablation run at the full system's tuned hyperparameters overstates the

re-tuned effect by the ablated system's regret, the *home-field advantage*,[7] and more seeds do not reduce it: it is a bias of the design, whose size is set by an interaction between the component and the hyperparameters. The identity also covers settings chosen by other rules. If the full system's error is flat over a range of settings, its regret is near zero over the whole range, and the measured effect is the re-tuned effect plus the ablated system's regret wherever in that range the experimenter happened to land. Choosing settings so that the treatment succeeds, as in Section 8.2, is a rule of this kind.

[7] The name is from sport, in which teams win more often at home; in an ablation table tuned for the full system, the ablated system always plays away.

In practice $h^\star$ is chosen on noisy estimates, and a second bias joins the first.

Corollary 8.7 (Tuning on noisy scores). *Let $\mathcal{H} = \{h_1, \dots, h_K\}$, let $V_1, \dots, V_K$ be estimates of the full system's errors with finite means $\mathbb{E}V_i = f(h_i)$, let $\hat{\imath}$ be an index at which $V_{\hat{\imath}} = \min_i V_i$, chosen by a rule that depends only on $V_1, \dots, V_K$, and let W be an estimate of the ablated system's error at $h_{\hat{\imath}}$ with finite mean and $\mathbb{E}[W \mid V_1, \dots, V_K] = g(h_{\hat{\imath}})$, as for the mean of fresh runs. Then the reported effect $D = W - V_{\hat{\imath}}$ satisfies*

$$\mathbb{E}D = \Delta^\star + \big(\mathbb{E}\, g(h_{\hat{\imath}}) - g^\star\big) + \big(f^\star - \mathbb{E}\min_i V_i\big), \tag{8.2}$$

and both brackets are nonnegative. If the V_i are independent and normal with common standard deviation τ, and f is constant on $\mathcal{H}$, then the last bracket equals τe_K.

Proof. The plan is to compute $\mathbb{E}D$ by conditioning on the estimates and then to add and subtract the two minima. By the tower property, $\mathbb{E}W = \mathbb{E}\,\mathbb{E}[W \mid V_1, \dots, V_K] = \mathbb{E}\, g(h_{\hat{\imath}})$, and $V_{\hat{\imath}} = \min_i V_i$, so

$$\mathbb{E}D = \mathbb{E}\, g(h_{\hat{\imath}}) - \mathbb{E}\min_i V_i = (g^\star - f^\star) + \big(\mathbb{E}\, g(h_{\hat{\imath}}) - g^\star\big) + \big(f^\star - \mathbb{E}\min_i V_i\big),$$

which is (8.2). The term $\mathbb{E}\, g(h_{\hat{\imath}}) - g^\star$ is nonnegative because $g(h) \geq g^\star$ for every h. For the term $f^\star - \mathbb{E}\min_i V_i$, note that $\min_i V_i \leq V_l$ for each l, so $\mathbb{E}\min_i V_i \leq f(h_l)$ for each l, and therefore $\mathbb{E}\min_i V_i \leq f^\star$. In the normal case with $f \equiv f^\star$, $V_i = f^\star + \tau Z_i$ with independent standard normal Z_i, and $\mathbb{E}\min_i Z_i = -e_K$ by Proposition 1.6, so the last bracket is τe_K. ∎

The first bracket in (8.2) is the expected home-field advantage at a randomly chosen h, the second the *selection bonus* of the full system's reported score, the winner's curse of Chapters 1 and 5. The bonus disappears if the full system is scored again with fresh runs at the chosen setting, the home-field term only if the ablated system is re-tuned with the same search. (If the ablated runs share seeds and test items with the runs that chose h, correlated noise can carry part of the selection over to W.) Re-tuning has a cost, and Chapter 4 shows that systems tuned with unequal budgets compare the budgets. A fair re-tuned ablation gives every ablated variant the search space and number of trials of the full system, at least over the hyperparameters that plausibly interact with the removed component: the learning rate and its schedule, the length of training, the regularization.

THE SHAM. An ablation that replaces a component must say what the replacement keeps. Chapter 5 distinguished the placebo, an inert treatment, from the *sham*, the whole procedure without its active step. For a pruning mask the active step is the selection of particular weights; the sham keeps the number of surviving weights in each layer and the values they start from, and destroys the selection by shuffling each layer's mask at random. It has a symmetry that saves an arm of the experiment.

Proposition 8.8 (A shuffled mask forgets the initialization). *Let the initial weights θ_0 have independent, identically distributed entries within each layer, and let the data, including split and data order, be independent of θ_0. Let m be a mask that may depend on θ_0 and on the data but keeps a fixed number of weights in each layer, and let π permute the positions of each layer uniformly at random, independently of everything else. Then $(\pi(m), \theta_0, \text{data})$ has the same joint distribution as $(\pi(m), \theta_0', \text{data})$, where θ_0' is an independent copy of θ_0. Consequently a run started from the shuffled mask with the original initial values has the same distribution of outcomes as one started from the shuffled mask with fresh initial values.*

PROOF. Condition on θ_0 and the data. Then m is fixed, with its prescribed number c_l of weights in each layer l, and a uniformly random permutation of a layer's positions carries its c_l positions to a uniformly random set of c_l positions. So, conditionally on θ_0 and the data, $\pi(m)$ is uniform over the masks with the prescribed counts, a distribution that does not depend on the condition; hence $\pi(m)$ is independent of (θ_0, data). With the independence of θ_0 and the data, the three components are mutually independent. So are those of $(\pi(m), \theta_0', \text{data})$, because θ_0' is independent of $(\theta_0, \pi, \text{data})$, of which $\pi(m)$ and the data are functions; and θ_0' has the law of θ_0, so the two triples have the same law. A training run is a function of the mask, the initial values and the data, which gives the last statement. ■

The identical distribution of entries within a layer is what the usual initialization schemes provide; the fixed counts are what layerwise pruning at prescribed rates provides. With the shuffled mask as one level of a factor M (learned mask $+1$, shuffled -1) and the initial values as a factor I (original $+1$, fresh -1), Proposition 8.8 says that $\mu(-1,+1) = \mu(-1,-1)$: the initialization does nothing under a shuffled mask.

EXAMPLE 8.9 (An effect that exists only as an interaction). In the two-by-two design $M \times I$, Definition 8.3 and $\mu(-1,+1) = \mu(-1,-1)$ give

$$\theta_I = \tfrac{1}{2}\big[\mu(1,1) - \mu(1,-1)\big] = \theta_{MI},$$

and the leave-one-out effect of the initialization, $\delta_I = \mu(1,1) - \mu(1,-1)$, is their sum, twice the main effect, as (8.1) requires. The original values have no effect of their own, only together with the weights that pruning selected, and how much of the ticket's advantage is due to its initialization and how much to its mask has no answer independent of the order of the questions.

THE ABLATION TABLE. The reporting rules of Chapters 1 and 3 apply to an ablation table. Run every arm with the same seeds, splits and test items, so that the design is blocked by seed (Section A.7), and estimate each effect from the per-seed contrasts, which removes the variation the arms share; with n seeds a contrast's t-interval has $n - 1$ degrees of freedom (Theorem A.1). Many rows are a family of comparisons (Chapter 3). And every row should say at which hyperparameters it was measured and whether the ablated system was re-tuned, because, by Proposition 8.6, these are part of what the number means.[8]

[8] Lipton and Steinhardt (2019) list the failure to identify the sources of empirical gains among the troubling trends of machine learning scholarship, and point to gains credited to architectures that came from hyperparameter tuning (Melis et al. 2018). A re-tuned ablation is the experiment that separates the two.

8.6 In the laboratory: tickets on the digits

Both labs prune the same network: a two-layer perceptron of width 128 on the scikit-learn digits (26,122 parameters, 25,856 of them weights), trained with SGD and momentum 0.9, batch size 64, a cosine schedule and 30 epochs, on a split of 1,258 training and 539 test images. Iterative pruning follows Frankle and Carbin: seven rounds, each removing after training the smallest 40% of the surviving weights of each hidden layer and the smallest 20% of the output layer, and resetting the survivors to their initial values. The final mask keeps 229 of 8,192 weights in the first layer, 458 of 16,384 in the second and 268 of 1,280 in the output layer, 3.7% of all weights; one-shot pruning keeps as many per layer, chosen from the trained dense network. The arms are the *ticket* (the mask with the original initial values), the *reinitialized* copy (the mask with a fresh draw from the same distribution) and the *sham* (each layer's mask shuffled at random, with the original values).[9] Compared with the original studies the network and data are much smaller, the optimizer is SGD with momentum rather than Adam, and the density is fixed by the number of rounds, for a reason the first lab shows.

[9] Within a seed, all arms share the split and the data order, and the ticket and the sham share the initial values: the blocking of Section 8.5.

SCRIPT
`08-ablations-tickets.py`

LAB 8.1 · TICKETS AT TWO LEARNING RATES

SETUP. For each of 20 seeds (split, initialization, data order) and each learning rate, 0.02 or 0.2, used for the dense training, every pruning round and the final training: the dense network, iterative and one-shot pruning, and the three arms of each, all with the same data order; the reinitialized copy and the sham also at 23% and 9% density. A $2 \times 2 \times 2$ factorial in initialization, learning rate and schedule, blocked by seed, with the sham as a control. 680 runs, about two minutes.

PREDICT FIRST. The dense network reaches the same test error at both rates. By how much will the original initial values lower the error of the pruned network at each rate? Will the sham, which keeps the number of weights per layer, do better or worse than the reinitialized copy?

LOOK FOR. Whether the lines joining an arm's means are parallel; the training error of the reinitialized copies; the initialization's main effect next to its interaction with the learning rate.

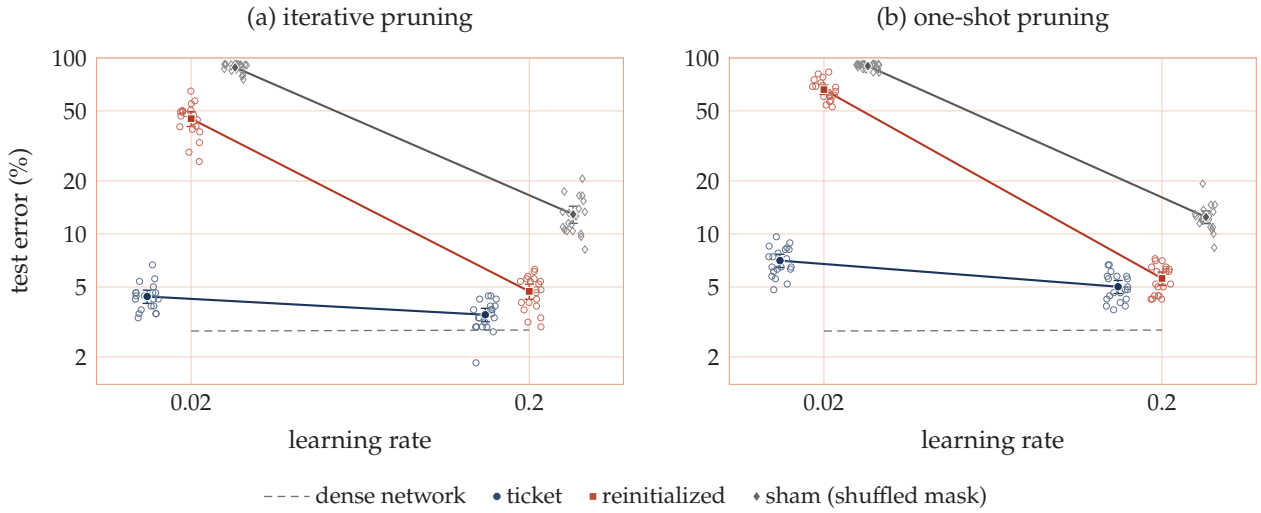


Figure 8.1. Lab 8.1: a two-layer perceptron on the digits pruned to 3.7% of its weights, (a) iteratively and (b) in one shot, at two learning rates; 20 seeds per cell (seeds 0–19). Hollow marks are single runs, spread sideways around each rate; filled marks are means with 95% t-intervals, joined across the two rates for each arm; the dashed line joins the means of the dense network. Both axes are logarithmic.

WHAT HAPPENED. The dense network reached the same test error at the two rates, 2.81 ± 0.22 at 0.02 and 2.85 ± 0.36 at 0.2 (means with 95% t-intervals over 20 seeds). Pruned iteratively to 3.7% of its weights, the ticket came within 1.60 ± 0.31 points of it at 0.02 and 0.63 ± 0.42 at 0.2, paired by seed: close, but not matching. The ablation gave two answers (Figure 8.1a). At 0.02 the reinitialized copies reached 45.2 ± 4.4% error against the ticket's 4.42 ± 0.38%, failing to fit even their training data (43% training error, against 1.8%); the effect, 40.7 ± 4.4 points, appeared for every seed. At 0.2 it was 1.24 ± 0.42 points, positive for 18 seeds of 20, with both arms fitting their training data. One-shot pruning repeated the pattern (Figure 8.1b): 59.0 ± 4.2 points at 0.02 and 0.58 ± 0.49 at 0.2. Only the ticket is nearly flat across the two rates; the lines are far from parallel. At milder sparsity the effect at 0.2 vanishes: 0.04 ± 0.32 points at 23% of the weights and 0.45 ± 0.41 at 9%, against 2.3 and 8.8 points at 0.02. This interaction with sparsity, the one Frankle and Carbin pointed out to Liu and colleagues, is why the labs prune to 3.7%.

In the factorial analysis, with each factor coded +1 at the ablated, larger or one-shot level and effects estimated from per-seed contrasts, the main effect of the initialization is 25.4 ± 1.4 points and its interaction with the learning rate -24.5 ± 1.5, almost as large; neither describes what an experimenter at either rate would see. The interactions with the schedule are smaller, $\theta_{IS} = 4.4 \pm 1.6$ and $\theta_{ILS} = -4.7 \pm 1.6$, and the four leave-one-out effects, 40.7, 1.2, 59.0 and 0.6 points, are the main effect plus the signed interactions (Lemma 6.8).

Why does the learning rate matter so much? At the small rate the trained weights remember their initial values: the rank correlation between the magnitudes of the initial and trained weights of the dense network's first

layer is 0.73 at 0.02 and 0.19 at 0.2. Pruning after training then nearly keeps the weights that started large, as Liu et al. (2019) suggested; the ticket's initial weights are 1.31 times the root mean square of their layers, and 99% keep their sign in training. A fresh draw is smaller, so sparse a network's signal is weak, and at a small rate 30 epochs may not suffice. We have not tested this account (Exercise 8.6).

The sham was the worst arm at both rates: 12.9 ± 1.5% at 0.2, 8.2 points worse than the reinitialized copy, which shares its initial distribution but has the learned positions. The learned mask concentrates its weights: on average 142 to 146 of the 256 hidden units have both an incoming and an outgoing weight in the ticket, against 216 to 219 in the sham. Here the positions matter far more than the initial values, as Frankle and Carbin (2019) found for their MNIST network and Frankle et al. (2021) for magnitude pruning after training.

LAB 8.2 · THE HOME-FIELD ADVANTAGE

SCRIPT
`08-ablations-homefield.py`

SETUP. For each of 20 seeds: iterative pruning at learning rate 0.1 as in Lab 8.1; then, with the final mask fixed, the ticket, the reinitialized copy, the sham and the dense network, each trained at ten learning rates from 0.01 to 0.5. Effects at each rate, re-tuned effects and regrets (Definition 8.5); 10,000 simulated small studies that tune the ticket on three seeds and run the ablation on three others. 940 runs, about two and a half minutes.

PREDICT FIRST. Will the arms have the same best learning rate? At the rate that is best for the ticket, will the measured effect of the initialization be larger or smaller than the re-tuned effect, and by how much?

LOOK FOR. The flat part of the ticket's curve and the slopes of the others over it; the identity of Proposition 8.6 in the printed table; the spread of the small studies' reported effects.

WHAT HAPPENED. The arms did not share a best learning rate (Figure 8.2a). The ticket's error was nearly flat from 0.05 to 0.3, from 3.76% at 0.05 to 3.47 ± 0.38% at its best rate, 0.2; the reinitialized copy was best at 0.3 (4.34%) and the sham at 0.4 (10.24%), both falling steeply over the ticket's flat range. The dense network, best at 0.15, diverged in 9 of 20 runs at 0.3. With each arm at its own best rate, the original initial values lower the error by 0.87 points (95% bootstrap interval 0.50 to 1.28 over seeds), and the learned mask with its original values lowers it by 6.77 points (5.95 to 7.65) against the sham.

At the ticket's best rate the measured effect of the initialization is 1.23 ± 0.44 points, the re-tuned 0.87 plus the reinitialized copy's regret there, 0.36: the home-field advantage of Proposition 8.6, here within the noise. For the sham it is not: at 0.2 its regret is 2.32 ± 0.92 points, a third of the re-tuned effect. And the ticket's flat curve makes the choice of rate nearly arbitrary for the ticket and decisive for the ablation (Figure 8.2b). At

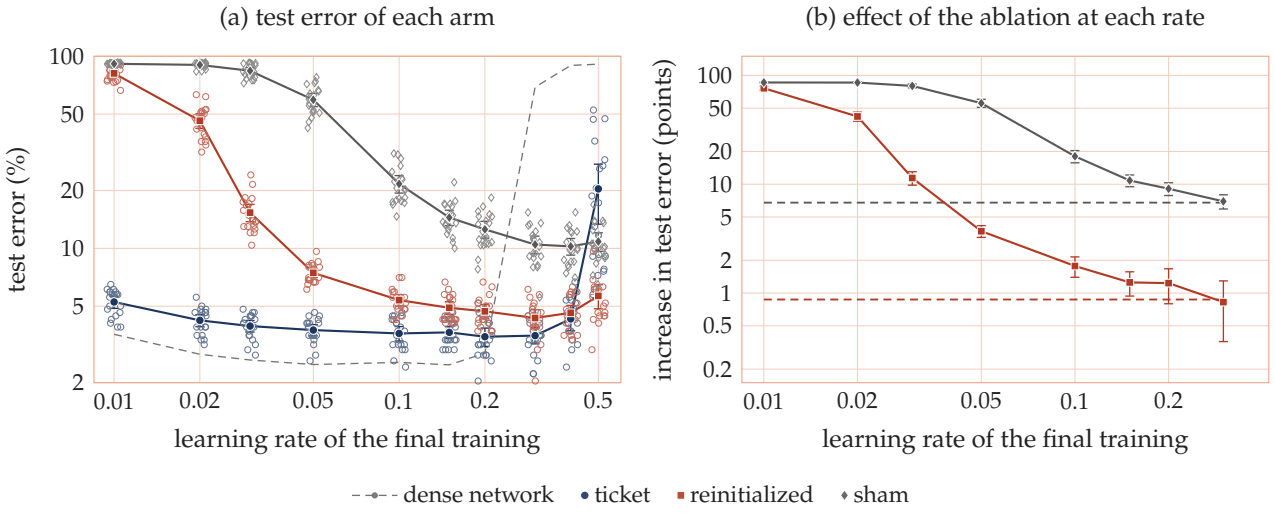


Figure 8.2. Lab 8.2: masks found by iterative pruning at learning rate 0.1, then each arm trained at ten learning rates; 20 seeds (0–19). (a) Test error: hollow marks are single runs, filled marks means with 95% t-intervals; the dense network (dashed) diverges in some runs from 0.3 on. (b) The effect of each ablation at each rate, ablated error minus ticket error paired by seed, with 95% t-intervals, up to 0.3 (at higher rates the ticket itself begins to fail). Dashed lines: the re-tuned effects, each arm's best mean error minus the ticket's, 0.87 points for the reinitialized copy and 6.77 for the sham. By Proposition 8.6, the height of a point above its line is the regret of the ablated arm at that rate minus the regret of the ticket.

0.1, the rate at which the masks were found, the ticket's regret is 0.14 ± 0.33 points while the effect of the initialization is 1.77 ± 0.38, twice the re-tuned value; at 0.05, where the ticket loses 0.29 ± 0.32, it is 3.70 ± 0.45, four times; at 0.02, where the dense network is within 0.33 ± 0.25 points of its best, 41.9 points.

The simulated small studies show the corollary at work. Each chose the ticket's rate as the one with the lowest mean error over three seeds, as a paper with three seeds would, and ran the reinitialized copy at that rate with three other seeds; the chosen rate was 0.1, 0.2 or 0.3 in 89% of them. The reported effect averaged 1.67 points, against the re-tuned 0.87: by (8.2), 0.17 of the excess is the selection bonus of the ticket's reported score and 0.63 the expected home-field advantage, and 37% of the studies reported more than twice the re-tuned effect. For the sham the average was 12.8 points against a re-tuned 6.8, nearly all of the excess home-field advantage.

The miniature reproduced the interaction of the post-mortem and its direction: the original initialization matters enormously at a small learning rate and little at a large one. It did not reproduce the end point: at 3.7% density, with or without re-tuning, the original values were worth about a point at every rate. On this small network, as on Lenet in Table 8.1, the effect is narrowed but not gone.

8.7 Protocol

PROTOCOL 8.1 · DESIGNING AN ABLATION

- ☐ Write down the estimand: what the shipped system loses without the component (a leave-one-out effect at fixed settings), or what the component adds to the best achievable (a re-tuned effect). They are different numbers.
- ☐ List the components and the hyperparameters that may interact with each. If interactions are plausible, use a two-level factorial or fractional factorial design rather than one factor at a time, and report interactions with intervals.
- ☐ Re-tune every ablated variant with the search space and budget of the full system, at least over the learning rate, its schedule and the length of training; report the fixed-setting and the re-tuned effects side by side.
- ☐ Choose settings on validation data and report test results from fresh runs at the chosen settings, never the scores of the runs that chose them.
- ☐ Add a sham that removes only the ingredient the claim is about, such as a shuffled mask or permuted values, and a positive control that must show an effect.
- ☐ Block on seeds: give every arm the same seeds, splits and test items, estimate effects from per-seed contrasts with t-intervals, show every run, and correct for multiplicity across the rows.
- ☐ Measure the effect at more than one level of the settings it may depend on (learning rate, scale, training length), or state those levels as part of the claim.

The protocol makes an ablation measure what it says it measures; it does not make a component's contribution well defined when interactions are strong, nor say how a component works, the subject of Chapter 9. It is expensive: a full factorial design costs 2^k cells, and re-tuning multiplies the cost of every ablated arm by the size of the search. When that is out of reach, re-tune on a smaller model or a shorter schedule and say so, re-tune at least the learning rate, which interacts with almost everything, use a fractional design, and report at which hyperparameters every row was measured and for which arm they were tuned.

Exercises

8.1 CONSOLIDATION. (Simple effects.) (a) Show that the leave-one-out effect $\delta_j(\mathbf{1})$ and the add-one effect $\delta_j(-\mathbf{1})$, measured at the baseline, differ by twice the sum of the interactions of even order that contain j. (b) Show that the sum of the k leave-one-out effects equals the gain

of the full system over the system with every component removed, $\mu(\mathbf{1}) - \mu(-\mathbf{1})$, for every μ if $k = 1$, and give a function with $k = 2$ for which it does not. (c) Express the sum of the leave-one-out effects in terms of the θ_S.

8.2 CONSOLIDATION. (A better one-at-a-time design.) In the leave-one-out design, put r_0 runs at the full system and r_1 at each ablated system, with $r_0 + kr_1 = N$. Minimize $\operatorname{Var} \hat{\delta}_j$ over the allocation, show that the optimum has $r_0 = \sqrt{k}\, r_1$ and variance $(1 + \sqrt{k})^2\sigma^2/N$, and compare with $4\sigma^2/N$ for $k = 3$ and $k = 8$. *(Hint: use $\sigma^2(1/r_0 + 1/r_1)$ and a Lagrange multiplier.)*

8.3 CONSOLIDATION. ★ (Fractional factorials and aliasing.) Run the half of a 2^3 design with $x_1x_2x_3 = +1$ (four cells) and r runs per cell. (a) Show that the contrast for θ_1 estimates $\theta_1 + \theta_{23}$, and find the other aliases. (b) Show that each contrast has variance $4\sigma^2/N$. (c) For four training tricks and eight cells, construct a design in which no two main effects are aliased, and list what it cannot separate. *(Hint: the defining relation $x_1x_2x_3 = 1$ gives $c_{\{1\}} = c_{\{2,3\}}$ on the design.)*

8.4 CONSOLIDATION. (A quadratic home field.) On a logarithmic scale of learning rates $u = \log_{10}\eta$, let $f(u) = f^\star + a(u - u_f)^2$ and $g(u) = g^\star + b(u - u_g)^2$ with $a, b > 0$. (a) Compute $\Delta(u_f) - \Delta^\star$. (b) If u is chosen as $\hat{u} = u_f + \xi$ with $\mathbb{E}\xi = 0$ and $\operatorname{Var}\xi = s^2$, independent of everything else, show that the expected home-field term of Corollary 8.7 is $b\big((u_f - u_g)^2 + s^2\big)$. (c) Estimate a, b, u_f and u_g from the means printed by `08-ablations-homefield.py` near their minima.

8.5 LAB. (Rewinding.) In `08-ablations-tickets.py`, rewind the surviving weights not to their initial values but to their values after one epoch of the dense training, and give the reinitialized copy a fresh draw as before. Predict first how the effect at learning rate 0.2 changes, then observe it, and relate the result to Frankle et al. (2020).

8.6 LAB. (Scale or values?) At learning rate 0.02 the reinitialized copies did not fit their training data. In `08-ablations-tickets.py` add two arms: the fresh draw rescaled in each layer to the root mean square of the ticket's initial weights, and the ticket's own initial values permuted at random among its surviving positions within each layer. Which part of the 41-point effect does each recover, and which hypothesis of the lab does that test?

8.7 LAB. (Longer training as re-tuning.) Every arm trains for 30 epochs. In `08-ablations-homefield.py`, train the reinitialized copy at learning rate 0.02 for 60 and 120 epochs. Is the 42-point effect on where training ends or on how fast it gets there? Which hyperparameters belong in the search of a re-tuned ablation?

8.8 DESIGN. (Four tricks, 64 runs.) A training recipe adds four components to a baseline: label smoothing, a warm-up, an augmentation and

weight averaging. You have 64 runs. Design an ablation that estimates every main effect and every two-factor interaction with intervals, decide whether to spend runs on re-tuning the learning rate, and say what you would report if the warm-up turned out to interact with the learning rate.

8.9 DESIGN. (A sham for a data filter.) A learned filter removes low-quality training examples. Design its ablation for 40 runs: the sham, positive and negative controls, interacting factors and re-tuning.

8.10 CRITIQUE. (An ablation table.) A table lists the full model at 81.4% accuracy and four rows each removing one component, at 79.9, 80.6, 80.1 and 81.0%; the text says the components contribute 1.5, 0.8, 1.3 and 0.4 points, 4.0 in all, while the full model beats its baseline by 3.1. Every number is one run at hyperparameters tuned for the full model. What can the table support, and what experiment would replace it?

8.11 CRITIQUE. (A ticket on its home field.) A study reports that a method for finding sparse subnetworks produces tickets that beat their reinitialized copies by four points, at the learning rate chosen to maximize the tickets' accuracy, and with three seeds. Using Corollary 8.7 and Lab 8.2, say what the four points may contain besides the effect of the initialization, and what you would ask the authors to run.

8.12 RESEARCH. (When is a contribution defined?) Under strong interactions, "the contribution of a component" depends on the baseline, the order of the ablations and the hyperparameters; the Shapley value of Chapter 6 averages over orders at fixed hyperparameters. Open: a definition of a component's contribution that is invariant to re-tuning, with a design that estimates it in fewer than 2^k cells when interactions are sparse; and how to choose the search space of a re-tuned ablation so that its conclusions do not depend on it.

Notes and References

The factorial design is Fisher's. In a paper on field trials he wrote that no aphorism was repeated more often than that Nature should be asked one question at a time, and that this view was "wholly mistaken" (Fisher 1926, p. 511); Fisher (1935) gave the full argument, that a factorial experiment answers several questions with the same runs and tests each over a range of conditions. Box et al. (2005) and Montgomery (2017) teach factorial and fractional designs, Czitrom (1999) compares one-factor-at-a-time studies with designed experiments on industrial examples, and Daniel (1973) says when one-at-a-time plans are the better choice. Theorem 8.4 is the classical efficiency comparison, with the full system as the baseline.

The literature on pruning is older than deep learning (LeCun et al. 1990) and has grown faster than its standards: Blalock et al. (2020) found that results were rarely comparable across papers and proposed a checklist that includes the random baseline with layerwise proportions used as the sham

here, and Gale et al. (2019) found that simple magnitude pruning matched more complex methods at scale. The lottery ticket experiments continued after the papers discussed in this chapter, on what masks encode and on finding tickets cheaply; Ma et al. (2021) review the conflicting reports and propose a stricter definition of a ticket.

That a baseline must be tuned as carefully as the method it is compared with is the lesson of Chapter 4, of which Melis et al. (2018) gave a famous instance; the sham is one of the controls of Chapter 5, and path attributions are in Chapter 6.

TRANSLATING THE LITERATURE. Frankle and Carbin (2019) write P_m for the fraction of weights remaining, our density, and $f(x; m \odot \theta_0)$ for the ticket; their random reinitialization $m \odot \theta_0'$ is our reinitialized copy, and their "random sparsity" is a random mask of the same size. Liu et al. (2019) report a prune ratio, one minus our density, and call the reinitialized copy "random init". Frankle et al. (2020) call a subnetwork *matching* when it reaches the full network's accuracy (within 0.2 points, in their summary across sparsities), and *stable* when the error barrier between two of its runs is below 2%; their rewinding to iteration k replaces our initial values by the weights at step k. The *shuffling* ablation of Frankle et al. (2021) is our sham; the *reshuffling* of Zhou et al. (2019) permutes the initial values among the surviving positions instead, keeping the mask. In Ma et al. (2021), LT and RR are our ticket and reinitialized copy, and their third condition asks for a positive re-tuned effect.

References

Blalock, D., J. J. Gonzalez Ortiz, J. Frankle, and J. Guttag (2020). "What is the State of Neural Network Pruning?" In: *Proceedings of Machine Learning and Systems*. Vol. 2.

Box, G. E. P., J. S. Hunter, and W. G. Hunter (2005). *Statistics for Experimenters: Design, Innovation, and Discovery*. 2nd ed. Wiley.

Czitrom, V. (1999). "One-Factor-at-a-Time versus Designed Experiments". In: *The American Statistician* 53.2, pp. 126–131.

Daniel, C. (1973). "One-at-a-Time Plans". In: *Journal of the American Statistical Association* 68.342, pp. 353–360.

Fisher, R. A. (1926). "The Arrangement of Field Experiments". In: *Journal of the Ministry of Agriculture* 33, pp. 503–513.

Fisher, R. A. (1935). *The Design of Experiments*. Edinburgh: Oliver and Boyd.

Frankle, J. and M. Carbin (2019). "The Lottery Ticket Hypothesis: Finding Sparse, Trainable Neural Networks". In: *International Conference on Learning Representations*.

Frankle, J., G. K. Dziugaite, D. M. Roy, and M. Carbin (2019). *Stabilizing the Lottery Ticket Hypothesis*. arXiv preprint 1903.01611; superseded by Frankle et al. (2020).

Frankle, J., G. K. Dziugaite, D. M. Roy, and M. Carbin (2020). "Linear Mode Connectivity and the Lottery Ticket Hypothesis". In: *Proceedings of the 37th International Conference on Machine Learning*. Vol. 119. Proceedings of Machine Learning Research, pp. 3259–3269.

Frankle, J., G. K. Dziugaite, D. M. Roy, and M. Carbin (2021). "Pruning Neural Networks at Initialization: Why Are We Missing the Mark?" In: *International Conference on Learning Representations*.

Gale, T., E. Elsen, and S. Hooker (2019). *The State of Sparsity in Deep Neural Networks*. arXiv preprint 1902.09574.

Han, S., J. Pool, J. Tran, and W. J. Dally (2015). "Learning both Weights and Connections for Efficient Neural Network". In: *Advances in Neural Information Processing Systems 28*, pp. 1135–1143.

LeCun, Y., J. S. Denker, and S. A. Solla (1990). "Optimal Brain Damage". In: *Advances in Neural Information Processing Systems 2*. Morgan Kaufmann, pp. 598–605.

Lipton, Z. C. and J. Steinhardt (2019). "Troubling Trends in Machine Learning Scholarship". In: *ACM Queue* 17.1. First circulated as arXiv:1807.03341, 2018.

Liu, Z., M. Sun, T. Zhou, G. Huang, and T. Darrell (2019). "Rethinking the Value of Network Pruning". In: *International Conference on Learning Representations*.

Ma, X. et al. (2021). "Sanity Checks for Lottery Tickets: Does Your Winning Ticket Really Win the Jackpot?" In: *Advances in Neural Information Processing Systems 34*.

Melis, G., C. Dyer, and P. Blunsom (2018). "On the State of the Art of Evaluation in Neural Language Models". In: *International Conference on Learning Representations*.

Montgomery, D. C. (2017). *Design and Analysis of Experiments*. 9th ed. Wiley.

Zhou, H., J. Lan, R. Liu, and J. Yosinski (2019). "Deconstructing Lottery Tickets: Zeros, Signs, and the Supermask". In: *Advances in Neural Information Processing Systems 32*.

9 Mechanisms

Intervening on the explanation

THE CLAIM *"Batch normalization works by reducing internal covariate shift."* — *Ioffe and Szegedy (2015), whose title names the mechanism*

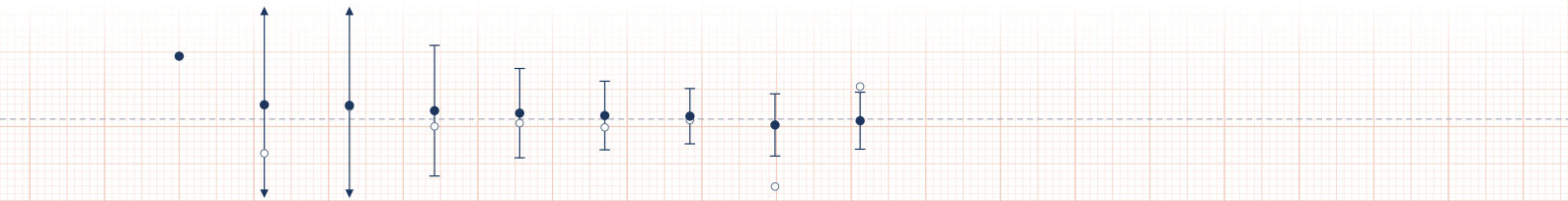
THE VERDICT Misattributed: the benefit survives when covariate shift is injected after every normalization, and which mechanism carries it is still debated.

IN THIS CHAPTER. In 2015 Ioffe and Szegedy introduced batch normalization, which trained an ImageNet classifier to its predecessor's accuracy in a fourteenth of the steps, and explained it by the reduction of internal covariate shift, the drift of each layer's input distribution as the layers below it learn. In 2018 Santurkar and colleagues put the drift back, with fresh random shifts and scalings after every normalization at every step, and the networks trained almost as fast; by a gradient-based measure, normalization did not reduce the shift at all. The method treats a mechanism claim as a chain of causes whose links are tested one at a time, proves what a knockout of the mediator can bound, and proves the two exact facts about batch normalization on which the rival explanations rest: the projection in its Jacobian, and the scale invariance that turns the learning rate into an effective one. In the laboratory, nine times the covariate shift of an unnormalized network costs a normalized one a seventh of its speed-up, normalization whose statistics are hidden from the gradient keeps the distributions steady and trains worse than none, and the benefit of normalization reverses at small learning rates.

9.1 The claim: a drift inside the network

In early 2015 training a deep network took care: small learning rates, carefully scaled initial weights, and rectified linear units in place of saturating sigmoids, whose gradients vanish for large inputs. Sergey Ioffe and Christian Szegedy, at Google, proposed a layer that removed much of this care (Ioffe and Szegedy 2015). For each unit and each minibatch of B examples, *batch normalization* subtracts the batch mean of the unit's pre-activations $y_1, \ldots, y_B$, divides by their standard deviation, multiplies by a learned scale γ and adds a learned shift β; at test time, estimates of the population mean and variance replace the batch statistics. The layer sits between each linear map and its nonlinearity, and the gradient flows through the normalization.

They explained it with a borrowed idea. A learning system whose input distribution changes between training and use suffers *covariate shift* (Shimodaira 2000). Inside a network, each layer's inputs are produced by the layers below it, whose parameters change at every step, so each layer



is trained on a moving distribution and must keep adapting to it; with saturating nonlinearities the moving inputs also drift into the regions where gradients vanish. Ioffe and Szegedy called this *internal covariate shift* and defined it as the change in the distribution of a network's activations caused by the change of its parameters during training; eliminating it, they wrote, "offers a promise of faster training". The precedent was sound: whitening the inputs of a network was known to speed up its training (LeCun et al. 1998),[1] and batch normalization standardizes the inputs of every layer, one unit and one batch at a time: a whitening without the decorrelation.

> FINDING 9.1 (Ioffe and Szegedy 2015, Sections 4.1 and 4.2). On MNIST, a network with three hidden layers of 100 sigmoid units reached a higher test accuracy sooner with batch normalization, and for one typical sigmoid of its last hidden layer the 15th, 50th and 85th percentiles of the input drifted in location and spread during 50,000 steps without normalization and were much more stable with it. On ImageNet, the Inception network reached its best validation accuracy, 72.2%, after 31 million steps; with batch normalization added it reached the same accuracy after 13.3 million steps, and with batch normalization, a five times larger learning rate and six further changes to the recipe after 2.1 million, fourteen times fewer. An ensemble of six normalized networks reached a top-5 validation error of 4.9%.

The paper claimed more than one benefit: much higher learning rates, less careful initialization, a regularizing effect that sometimes made dropout unnecessary, and trainable sigmoid networks (with sigmoids, the normalized Inception reached 69.8% and the original stayed at chance). For the higher learning rates it gave a second argument. Scaling a layer's weights by $a > 0$ leaves the normalized output unchanged and divides the gradient with respect to the weights by a, so larger weights receive smaller gradients and their growth limits itself. The authors also conjectured that normalization keeps the singular values of the layer Jacobians near one, called its effect on gradient propagation a subject for further study, and stated the main claim with care: batch normalization "takes a step towards reducing" internal covariate shift.

The title stated the received version: *Accelerating Deep Network Training by Reducing Internal Covariate Shift*. Batch normalization became a default component of convolutional networks, and its explanation traveled with it; layer normalization, for instance, was motivated by the same covariate-shift problem (Ba et al. 2016). By 2017 the explanation was questioned in public. In the text of their NIPS 2017 test-of-time talk, Ali Rahimi and Ben Recht asked for evidence that batch normalization reduces internal covariate shift and for a definition of the term (Rahimi and Recht 2017),[2] and Lipton and Steinhardt (2019) used it as an example of speculation offered as explanation, asking by which divergence the change in a distribution is to be measured.[3] In 2025 the paper received the ICML Test of Time Award.

The claim was reasonable. Normalized inputs were known to help, the MNIST percentiles showed the stabilization the theory predicted, and the

[1] LeCun et al. advised shifting each input to zero mean, scaling the inputs to about the same covariance and decorrelating them if possible, and added that the zero-mean heuristic "should be applied at all layers", since each layer's outputs are the next layer's inputs.

[2] The award was for their 2007 paper on random features for kernel machines.

[3] They add that covariate shift, in its original sense, changes the distribution of the inputs while the conditional distribution of the labels stays fixed, which is not the situation of a layer inside a network.

speed-ups were large, 2.3-fold even in the comparison that changed nothing but the normalization. The paper also contained two ingredients of later explanations: the scale argument, and the report that in the authors' first experiments the model blew up when the normalization statistics were computed outside the gradient step, which turns out to test what the backward pass of the normalization contributes (Section 9.6). The question of this chapter is not whether batch normalization works, but whether it works for the reason in its title.

9.2 The experiment: one treatment, two consequences

EXPERIMENTAL DESIGN · IOFFE AND SZEGEDY 2015

QUESTION	Does batch normalization speed up training because it reduces internal covariate shift?
UNITS	training runs: a three-layer sigmoid network on MNIST; Inception on ImageNet
TREATMENT	batch normalization of the input to every nonlinearity
LEVELS	without and with (MNIST); Inception, BN-Baseline, BN-x5, BN-x30, BN-x5-Sigmoid (ImageNet)
MEDIATOR	internal covariate shift, shown as three percentiles of the input to one sigmoid during training (MNIST only)
HELD FIXED	architecture and data; BN-x5 and BN-x30 also changed the learning rate and its decay, dropout, weight decay, response normalization, shuffling and distortions
RESPONSE	test accuracy against steps (MNIST); steps to 72.2% validation accuracy (ImageNet)
REPLICATES	one run per network; no spread reported
ANALYSIS	curves and percentiles compared by eye

The record establishes two consequences of one treatment. Batch normalization speeds up training, 2.3-fold on ImageNet with nothing else changed, and it steadies the distribution of each normalized input, which it does by construction, since it fixes the batch mean and variance of every unit it normalizes. A mechanism claim says more: that the second consequence causes the first. The design cannot test this, because in every arm the steadier distributions came with the normalization. Whatever else normalization changes, in the gradients, in the learning rates the network tolerates or in the scale of its weights, would produce the same two observations: the mediator and the treatment are perfectly confounded. Separating them takes an arm in which the treatment acts while the mediator stays at its untreated level, or one in which the mediator changes without the treatment.

Two further gaps made the claim hard to test. The mediator had no measurement: the percentiles of one unit in one network are a picture, and "the change in the distribution" needs a divergence, a time scale and a population of units before it can be compared between arms or given an interval. And the fourteen-fold speed-up bundled the normalization with a five-fold learning rate and six other changes, the problem of Chapter 6;

BN-Baseline is the clean measurement of the treatment. Neither gap is a fault of the method. The paper proposed a mechanism and showed its first link; the post-mortem supplied the missing arm.

9.3 The post-mortem: putting the shift back

THE KNOCKOUT. In 2018 Shibani Santurkar, Dimitris Tsipras, Andrew Ilyas and Aleksander Madry put the shift back (Santurkar et al. 2018).[4] They trained a VGG network on CIFAR-10 with plain SGD, batch size 128 and learning rate 0.1 for 15,000 steps without data augmentation, reaching 83% test accuracy with normalization and 80% without. In a third arm they injected noise after every normalization: at every step each unit drew a shift μ uniform on $[-0.5, 0.5]$ and a scale σ uniform on $[1, 1.25]$, and each example's activation a became $sa + m$, with m uniform within 0.1 of μ and s normal around σ with spread 0.1.[5] Each layer thus received inputs whose distribution changed at every step, by an amount the experimenter set.

> FINDING 9.2 (Santurkar et al. 2018, Section 2 and Figures 2 and 3). With the shift injected after every normalization, the network trained almost exactly as fast as with ordinary batch normalization and much faster than without normalization, although its activations had less stable distributions than those of the unnormalized network; the same noise added to the unnormalized network prevented it from training. Measured instead by how much the gradient of a layer's weights changes when the layers below it are updated, normalized networks had as much internal covariate shift as unnormalized ones or more; in a 25-layer deep linear network the gradients before and after the update were almost uncorrelated with normalization and almost identical without it.

Normalizing by the ℓ_1, ℓ_2 or ℓ_∞ norm of the centered activations instead of their standard deviation also trained about as well, with more distributional shift than the unnormalized networks had. As a replacement the authors proposed that batch normalization makes the loss landscape smoother, so that larger steps become safe, and supported it with measurements along the gradient direction and with theorems about one normalized layer, the first of which we prove in corrected form as Theorem 9.5.

THE LEARNING RATE. Bjorck et al. (2018) trained a residual network of 110 layers on CIFAR-10. Without normalization the learning rate had to be lowered to 10^{-4} and training took about 2,400 epochs; at that learning rate the normalized network reached no better test accuracy than the unnormalized one (five runs per setting). The higher learning rate that normalization permits, they concluded, "mediates the majority of its benefits"; without normalization, a few activations deep in the network explode within the first steps.

SCALE INVARIANCE. Because the normalization divides by the standard deviation, a normalized unit's output does not depend on the length of its incoming weight vector, only on its direction: Ioffe and Szegedy's scale argument. In an unrefereed note, Laarhoven (2017) concluded that weight

[4] The first arXiv version of the paper, in May 2018, carried the subtitle "No, It Is Not About Internal Covariate Shift".

[5] The paper writes this distribution as $N(\sigma, r_\sigma)$ with $r_\sigma = 0.1$; our script reads 0.1 as a standard deviation.

decay on such weights does not constrain the function the network computes and acts only on their length, and through it on the effective learning rate. Arora et al. (2019) proved that gradient descent on scale-invariant weights reaches a stationary point with a fixed learning rate as fast, in their bounds, as with a well-tuned one, because the growth of the weights tunes the step, and Li and Arora (2020) derived from the invariance a schedule that no other account suggests, an exponentially increasing learning rate, proved it equivalent to the standard schedule with weight decay, and trained networks with it.

THE REPLACEMENT CONTESTED. With a mean-field theory of wide fully connected networks, Yang et al. (2019) showed that at initialization batch normalization makes gradients grow exponentially with depth, that no choice of the initial weight variance or of the nonlinearity prevents it, and that deep normalized networks without skip connections are untrainable under common initializations: at the start of training, at least, the normalized landscape is not the smoother one. Cohen et al. (2021) found that under full-batch gradient descent the largest Hessian eigenvalue rises to about $2/\eta$ and stays there with and without normalization, and that the effective smoothness of Santurkar et al., measured at the step actually taken, does the same, while the normalized networks still train faster at every step size.[6] On the other side, Ghorbani et al. (2019) found that the large isolated Hessian eigenvalues that appear early in unnormalized networks, and the concentration of the gradient in their directions, are almost absent with batch normalization.

[6] They read Santurkar et al.'s measurement for VGG, which hovers near $20 = 2/0.1$ in both arms, as consistent with this.

NECESSITY. With a rescaled initialization, Fixup trained residual networks without normalization as stably as with it, even at 10,000 layers, and with suitable regularization matched the state of the art in image classification and machine translation (Zhang et al. 2019). De and Smith (2020) showed that in residual networks batch normalization scales each residual branch down by a factor of the order of the square root of the depth, so that the blocks start close to the identity, that an initialization doing the same trains deep residual networks without normalization, and that the larger learning rates normalization allows help little at small batch sizes. The normalizer-free networks of Brock et al. (2021) matched EfficientNet-B7 on ImageNet while training up to 8.7 times faster; that paper credits normalization with four separate benefits, downscaling the residual branch, removing the mean shift of rectified activations, regularizing, and smoothing the landscape enough for large batches. None of these results says why normalization works; each says which of its consequences can be supplied without it.

VERDICT. The effect is real: batch normalization speeds up training, widens the range of usable learning rates, and on ImageNet cut the steps to the baseline's accuracy 2.3-fold on its own. The explanation is not supported: networks given back more distributional shift after every normalization than unnormalized networks ever had train about as fast, and by a gradient-based measure normalization increases

MISATTRIBUTED

the shift. What carries the effect is contested: larger stable learning rates, scale invariance, a smoother landscape and downscaled residual branches each account for part of the evidence, and the smoothing account conflicts with measurements at initialization and at the edge of stability.

9.4 The method: a mechanism is a chain of causes

A claim that a treatment works through a mechanism names a variable in between, a *mediator*: the treatment changes it, and its change changes the outcome. Baron and Kenny (1986) set out the logic of testing such claims in regression studies;[7] the potential-outcome definitions below are those of Robins and Greenland (1992) and Pearl (2001).

[7] Read causally, their regressions require among other things that nothing unmeasured affects both the mediator and the outcome. In a training run the mediator can often be set by code, which removes that requirement.

Definition 9.3 (Natural direct and indirect effects). A treatment $A \in \{0, 1\}$ is applied to a unit, here a training run with given seeds. A mediator is a real variable measured on the run, with potential values $M(a)$ under treatment a; the outcome has potential values $Y(a, m)$ under treatment a with the mediator set to m, and $Y(a) = Y(a, M(a))$ without intervention on the mediator. For an outcome that is a loss, the *total effect* $\mathbb{E}Y(0) - \mathbb{E}Y(1)$ of the treatment splits into a *direct effect* and an *indirect effect*,

$$\mathbb{E}Y(0) - \mathbb{E}Y(1) = \underbrace{\mathbb{E}Y(0) - \mathbb{E}Y(1, M(0))}_{\text{direct}} + \underbrace{\mathbb{E}Y(1, M(0)) - \mathbb{E}Y(1)}_{\text{indirect}}. \quad (9.1)$$

The split is a telescoping sum. The direct effect compares the two treatments with the mediator held where it would be without treatment; the indirect effect is what a treated run gains when its mediator moves from its untreated to its treated value. That batch normalization works by reducing internal covariate shift says that the indirect effect is most of the total. The quantity $\mathbb{E}Y(1, M(0))$, the loss of a normalized run with the covariate shift of an unnormalized one, is never observed without intervention: an experiment has to construct it, or bound it.

A mechanism claim has four links, and each has a test. *The mediator is a measurement*: it needs an operational definition, naming the quantity, the data, the time scale and the divergence, and different definitions make different claims. *The treatment moves the mediator*: measure the mediator in every arm, with intervals over seeds. *The mediator moves the outcome*: vary the mediator with the treatment fixed, by injecting it into the untreated arm or varying its dose in the treated one. *The treatment acts through the mediator*: a *knockout* applies the treatment and restores the mediator to its untreated level, and if the outcome keeps its benefit, then the benefit did not need the change in the mediator; a *rescue* supplies the treated level of the mediator without the treatment, or with its other actions removed, and if the outcome does not improve, then the change in the mediator was not enough. The names come from genetics, where a gene is knocked out to

see whether a trait needs it. The noisy normalization of Santurkar et al. is a knockout, and the following proposition says what it can establish.

Proposition 9.4 (What a knockout bounds). *In the setting of Definition 9.3, let a knockout arm apply the treatment together with an operation that sets the mediator of each run to M_κ, and let Y_κ be its outcome. Suppose that*

(i) the operation acts on the outcome only through the mediator: $Y_\kappa = Y(1, M_\kappa)$;

(ii) more of the mediator never lowers the loss of a treated run: $m \mapsto Y(1, m)$ is nondecreasing for every run;

(iii) the knockout restores at least the untreated level: $M_\kappa \geq M(0)$ for every run.

Then the indirect effect is at most $\mathbb{E}Y_\kappa - \mathbb{E}Y(1)$, and the direct effect is at least $\mathbb{E}Y(0) - \mathbb{E}Y_\kappa$. If $M_\kappa = M(0)$ for every run, then the first bound holds with equality.

Proof. Fix a run. By hypothesis (iii) and the monotonicity (ii), $Y(1, M(0)) \leq Y(1, M_\kappa)$, and by (i) the right side is Y_κ. Taking expectations, $\mathbb{E}Y(1, M(0)) \leq \mathbb{E}Y_\kappa$, and subtracting $\mathbb{E}Y(1)$ bounds the indirect effect of (9.1) by $\mathbb{E}Y_\kappa - \mathbb{E}Y(1)$. Subtracting the indirect effect from the total effect $\mathbb{E}Y(0) - \mathbb{E}Y(1)$ bounds the direct effect below by $\mathbb{E}Y(0) - \mathbb{E}Y_\kappa$. If $M_\kappa = M(0)$, then $Y_\kappa = Y(1, M(0))$ for every run by (i), and the first inequality is an equality. ∎

The proposition is elementary; its use is to say what each hypothesis costs. Hypothesis (iii) can be checked, by measuring the mediator in the knockout arm and in the untreated arm with the same seeds and comparing them seed by seed, but only for the definition of the mediator that was measured. Hypothesis (ii) is the premise of the claim itself, that the mediator hurts. Hypothesis (i) cannot be checked directly and is the weak point of every knockout: noise injected to restore a shift may also regularize, or slow training, by routes of its own. Applying the same operation in the untreated arm checks it partly.

Three consequences guide the design. *The right dose is the smallest that restores the mediator*: an overshooting knockout gives a valid but looser bound, and a dose that restores the mediator exactly gives an estimate; varying the dose traces the outcome against the measured mediator, the most direct picture of the second link. *A knockout that does not hurt is informative, and one that hurts is not*, since the operation may hurt by other routes. And *the indirect effect depends on the arm in which it is evaluated* when the treatment changes how the mediator acts: if normalization makes a network indifferent to shifts that slow an unnormalized one, then the indirect effect of (9.1) is small, while the gain $\mathbb{E}Y(0) - \mathbb{E}Y(0, M(1))$ of an unnormalized network from the reduced shift may be large. These are the total and the pure indirect effects of Robins and Greenland (1992); a knockout in the treated arm bounds the first, and only a rescue in the untreated arm measures the second.

RIVAL MEDIATORS. The useful experiment is one in which rival mediators predict different things. Failed runs in the fine-tuning of BERT (Chapter 1) had been attributed to catastrophic forgetting and to small datasets. Mosbach et al. (2021) set the second against a rival, the small number of steps that a few epochs on a small dataset provide: they subsampled 1,000 examples from each of three tasks and trained 25 seeds either for three epochs or for as many steps as on the full dataset. With three epochs the failed runs multiplied; with the full number of steps there were none on two tasks and one on the third. Holding the steps fixed while shrinking the data knocked out one mediator and kept the other. For batch normalization the rivals are many, and some are consequences of others, which is one reason the question is still open.

NECESSITY AND PREDICTION. The normalizer-free networks show that the treatment is not necessary for good training, a statement about the treatment rather than its mechanism. A mechanism earns belief by predicting what its rivals do not. Scale invariance predicted that weight decay acts on the effective learning rate rather than on the function, and that a growing learning rate can replace a decaying one (Laarhoven 2017; Li and Arora 2020); the covariate-shift account predicted that putting the shift back would cancel the benefit, and it did not.

9.5 The method: two exact facts about batch normalization

The rival explanations rest on two facts about a single normalized unit. Fix one unit and a batch of $B \geq 2$ examples, and write $y \in \mathbb{R}^B$ for the unit's pre-activations on the batch, $\mathbf{1} \in \mathbb{R}^B$ for the vector of ones and $\langle \cdot, \cdot \rangle$ for the Euclidean inner product. If the entries of y are not all equal, then the batch mean, standard deviation and normalized vector

$$\mu = \frac{1}{B}\langle \mathbf{1}, y \rangle, \qquad \sigma = \left(\frac{1}{B} \| y - \mu \mathbf{1} \|^2 \right)^{1/2} > 0, \qquad \hat{y} = \frac{y - \mu \mathbf{1}}{\sigma}$$

are defined, and the unit's output is $z = \gamma \hat{y} + \beta \mathbf{1}$ for real $\gamma \neq 0$ and β. The rest of the network and the loss, as a function of this unit's outputs on the batch with everything else held fixed, is a function $\ell : \mathbb{R}^B \to \mathbb{R}$; write $L(y) = \ell(z)$ and $g = \nabla \ell(z)$ for the gradient that arrives at the output. We omit the small constant that implementations add to σ^2; Exercise 9.1 restores it.

Theorem 9.5 (The Jacobian of batch normalization). *Let $B \geq 2$, let y have entries that are not all equal, and let ℓ be differentiable at z. Put $P = I - \frac{1}{B}\mathbf{1}\mathbf{1}^\top - \frac{1}{B}\hat{y}\hat{y}^\top$.*

(i) P is the orthogonal projection onto the subspace of $\mathbb{R}^B$ orthogonal to $\mathbf{1}$ and $\hat{y}$, which has dimension $B - 2$.

(ii) $\nabla L(y) = \frac{\gamma}{\sigma} P g$.

(iii) $\| \nabla L(y) \|^2 = \frac{\gamma^2}{\sigma^2} \left(\|g\|^2 - \frac{1}{B}\langle \mathbf{1}, g \rangle^2 - \frac{1}{B}\langle \hat{y}, g \rangle^2 \right)$.

(iv) $\langle \mathbf{1}, \nabla L(y)\rangle = 0$ *and* $\langle y, \nabla L(y)\rangle = 0$.

PROOF. The plan is to show that $\mathbf{1}/\sqrt{B}$ and $\hat{y}/\sqrt{B}$ are orthonormal, which gives part (i), to compute the Jacobian of $\hat{y}$, which turns out to be P/σ and gives part (ii) by the chain rule, and to read off parts (iii) and (iv) from the projection.

Part (i). The entries of $y - \mu\mathbf{1}$ sum to zero, so $\langle \mathbf{1}, \hat{y}\rangle = 0$, and $\|\hat{y}\|^2 = \|y - \mu\mathbf{1}\|^2/\sigma^2 = B$ by the definition of σ. Hence $e_1 = \mathbf{1}/\sqrt{B}$ and $e_2 = \hat{y}/\sqrt{B}$ are orthonormal, and $P = I - e_1 e_1^\top - e_2 e_2^\top$ is the orthogonal projection onto the complement of their span. The span has dimension two because $\hat{y} \neq 0$, since the entries of y are not all equal.

Part (ii). We differentiate μ, then σ, then $\hat{y}$. First, $\partial\mu/\partial y_j = 1/B$. Next, $\sigma^2 = \frac{1}{B}\sum_i (y_i - \mu)^2$, and because $\sum_i (y_i - \mu) = 0$ the dependence of μ on y_j contributes nothing:

$$\frac{\partial \sigma^2}{\partial y_j} = \frac{2}{B}\sum_i (y_i - \mu)\Big(\delta_{ij} - \frac{1}{B}\Big) = \frac{2}{B}(y_j - \mu).$$

Since $\sigma > 0$, the square root is differentiable there, and $\partial\sigma/\partial y_j = (y_j - \mu)/(B\sigma) = \hat{y}_j/B$. The quotient rule then gives

$$\frac{\partial \hat{y}_i}{\partial y_j} = \frac{\delta_{ij} - 1/B}{\sigma} - \frac{(y_i - \mu)\,\hat{y}_j}{B\sigma^2} = \frac{1}{\sigma}\Big(\delta_{ij} - \frac{1}{B} - \frac{\hat{y}_i \hat{y}_j}{B}\Big),$$

so the Jacobian of $y \mapsto \hat{y}$ is P/σ and that of $y \mapsto z$ is $\gamma P/\sigma$. By the chain rule and the symmetry of P, $\nabla L(y) = (\gamma P/\sigma)^\top g = (\gamma/\sigma) P g$.

Part (iii). An orthogonal projection is symmetric and idempotent, so $\|Pg\|^2 = \langle g, Pg\rangle = \|g\|^2 - \langle e_1, g\rangle^2 - \langle e_2, g\rangle^2$, where $\langle e_1, g\rangle^2 = \langle \mathbf{1}, g\rangle^2/B$ and $\langle e_2, g\rangle^2 = \langle \hat{y}, g\rangle^2/B$; multiplying by γ^2/σ^2 and using part (ii) gives the identity.

Part (iv). By part (i), Pg is orthogonal to $\mathbf{1}$ and to $\hat{y}$, hence to $y = \mu\mathbf{1} + \sigma\hat{y}$, and so is $\nabla L(y)$ by part (ii). ∎

Corollary 9.6 (Biases and weight rows). *If* $y = Xw + b\mathbf{1}$ *for a batch of inputs* $X \in \mathbb{R}^{B\times d}$*, a weight row* $w \in \mathbb{R}^d$ *and a bias* b*, then* $\partial L/\partial b = 0$ *and* $\langle \nabla_w L, w\rangle = 0$.

PROOF. By the chain rule $\partial L/\partial b = \langle \mathbf{1}, \nabla L(y)\rangle$ and $\nabla_w L = X^\top \nabla L(y)$, so $\langle \nabla_w L, w\rangle = \langle \nabla L(y), y\rangle - b\langle \nabla L(y), \mathbf{1}\rangle$. All three inner products vanish by part (iv) of the theorem. ∎

The backward pass of a normalization therefore takes the gradient arriving at the unit,[8] removes its components along $\mathbf{1}$ and $\hat{y}$, and rescales the rest by γ/σ. The removed directions are the changes of y that shift its batch or rescale it about its mean, which the forward pass undoes, so the loss is blind to them. In this precise sense batch normalization does not hold the distributions still: it makes the network indifferent to the first two moments of every normalized unit's batch, whatever moves them, a parameter update or injected noise.

[8] The bias before a normalization is idle, which is why implementations usually omit it.

REMARK 9.7 (The printed form of the theorem). Part (iii) is Theorem 4.1 of Santurkar et al. (2018) in our notation. They write m for the batch size, compare a normalized network with an "identical" unnormalized one, and assume that the gradient arriving at the normalized output equals the gradient of the unnormalized network at its pre-activations, our g; under that assumption their theorem is the identity (iii), stated with $\le$. In the NeurIPS proceedings, and in the arXiv version of October 2018, the last term carries $1/\sqrt{m}$ instead of $1/m$; the arXiv version of March 2019 prints $1/m$. The slip is in the last step of their proof, where the squared inner product with the unit vector $\hat{y}/\|\hat{y}\|$, which is $\langle \hat{y}, g\rangle^2/m$ because $\|\hat{y}\| = \sqrt{m}$, becomes $\langle \hat{y}, g\rangle^2/\sqrt{m}$. As printed in the proceedings the inequality is false whenever $m \ge 2$ and $\langle \hat{y}, g\rangle \neq 0$, since its left side equals the right side with $1/m$, which exceeds the right side with $1/\sqrt{m}$. A numerical check in `09-mechanisms-lr.py` confirms the identity with $1/m$ to 2×10^{-15}, while the right side with $1/\sqrt{m}$ is negative in 20 of 200 random draws with $m = 8$.

The identity does not by itself make normalized networks smoother. Removing components never lengthens a vector, but the factor γ/σ can exceed one, and g is computed by different downstream networks in the two cases; Santurkar et al. add that σ tends to be large in practice. The identity also holds one layer at a time, and through many layers the factors multiply: at initialization they can compound into the gradient explosion of Yang et al. (2019).

The second fact is the invariance behind the effective learning rate, of which Corollary 9.6 is the infinitesimal form.

Proposition 9.8 (Scale invariance). *Let $f : \mathbb{R}^d \setminus \{0\} \to \mathbb{R}$ be differentiable and* scale invariant, *$f(cw) = f(w)$ for every $c > 0$ and $w \neq 0$.*

(i) $\langle \nabla f(w), w\rangle = 0$ *for every $w \neq 0$.*

(ii) $\nabla f(cw) = \nabla f(w)/c$ *for every $c > 0$.*

(iii) A gradient step $w' = w - \eta \nabla f(w)$ with $\eta > 0$ satisfies $\|w'\|^2 = \|w\|^2 + \eta^2 \|\nabla f(w)\|^2$; in particular $w' \neq 0$, and along gradient descent the norm never decreases.

(iv) Write $u = w/\|w\|$ and $\eta_{\text{eff}} = \eta/\|w\|^2$. The step moves the direction as a gradient step with learning rate η_{eff} taken from u and projected back onto the unit sphere:

$$\frac{w'}{\|w'\|} = \frac{u - \eta_{\text{eff}} \nabla f(u)}{\|u - \eta_{\text{eff}} \nabla f(u)\|}.$$

PROOF. For part (i), the function $\varphi(c) = f(cw)$ is constant on $c > 0$, so its derivative $\varphi'(c) = \langle \nabla f(cw), w\rangle$ vanishes, in particular at $c = 1$. For part (ii), differentiate the identity $f(cw) = f(w)$ in w: the chain rule gives $c\, \nabla f(cw) = \nabla f(w)$. For part (iii), expand $\|w - \eta \nabla f(w)\|^2 = \|w\|^2 - 2\eta \langle w, \nabla f(w)\rangle + \eta^2 \|\nabla f(w)\|^2$, whose middle term vanishes by part (i); the result is at least $\|w\|^2 > 0$. For part (iv), part (ii) with $c = \|w\|$ gives $\nabla f(w) = \nabla f(u)/\|w\|$, so $w' = \|w\|\big(u - \eta_{\text{eff}} \nabla f(u)\big)$; the vector in parentheses

is nonzero by part (i), and dividing w' by its norm gives the formula. ■

With batch statistics, the loss on a minibatch is scale invariant in every weight row that feeds a normalization, so the proposition applies to each such row at every step of SGD. Four consequences follow. The step that moves a row's direction is $\eta/\|w\|^2$, not η, so the same nominal learning rate is a different step with and without normalization. The norms grow, so the effective learning rate decays by itself, faster when η is large: the automatic rate tuning of Arora et al. (2019). Weight decay with coefficient λ turns part (iii) into $\|w'\|^2 = (1-\eta\lambda)^2\|w\|^2 + \eta^2\|\nabla f(w)\|^2$; it cannot change what a row computes, only its length and hence its effective learning rate (Exercise 9.3). And a shift and scaling added after the normalization leaves all of this intact, since it does not make the output depend on the length of the row: the noisy normalization of Santurkar et al. knocked out one candidate mediator and spared this one.

9.6 In the laboratory: knockout, dose and learning rate

Both labs train a perceptron with six hidden layers of 64 rectified linear units on the digits (1,258 training and 539 test images, split 0 of Chapter 1), with He-normal initial weights, by plain SGD without momentum or weight decay, batch size 64, for 600 steps; with normalization, each hidden layer is a linear map, a batch normalization and a ReLU. Run s uses seed s for the weights, the data order and the injected noise, so the arms of each comparison share their seeds. Every 50 steps we measure the training loss and test error with the noise off and, in normalized networks, with the batch statistics recomputed on the whole training set, since those accumulated with the noise on would mismeasure a noisy network. The speed of training is summarized by the *mean training loss* over these twelve checkpoints. These choices, and the learning rate of Lab 9.1, were fixed after a pilot with three seeds per arm.

LAB 9.1 · KNOCKING OUT COVARIATE SHIFT

SCRIPT
`09-mechanisms-noisybn.py`

SETUP. Learning rate 0.3, at which the unnormalized network reaches its lowest test error in Lab 9.2. A 2×2 design, batch normalization (off, on) crossed with the injected shift of Finding 9.2 (off, on), 20 seeds per cell; the shift follows each normalization or, without normalization, each linear map. A fifth arm detaches the batch mean and variance from the gradient: its forward pass, and so the steadiness of its distributions, is that of batch normalization, but its gradient ignores the normalization. The normalized network also gets the shift at doses $\frac{1}{8}, \frac{1}{2}, 2, 4$ (8 seeds each), a dose scaling the four half-widths of the noise. Covariate shift is measured three ways on 256 training images: the *experienced shift*, the change in one step of the distribution of each unit's input to its nonlinearity (Wasserstein distance in units

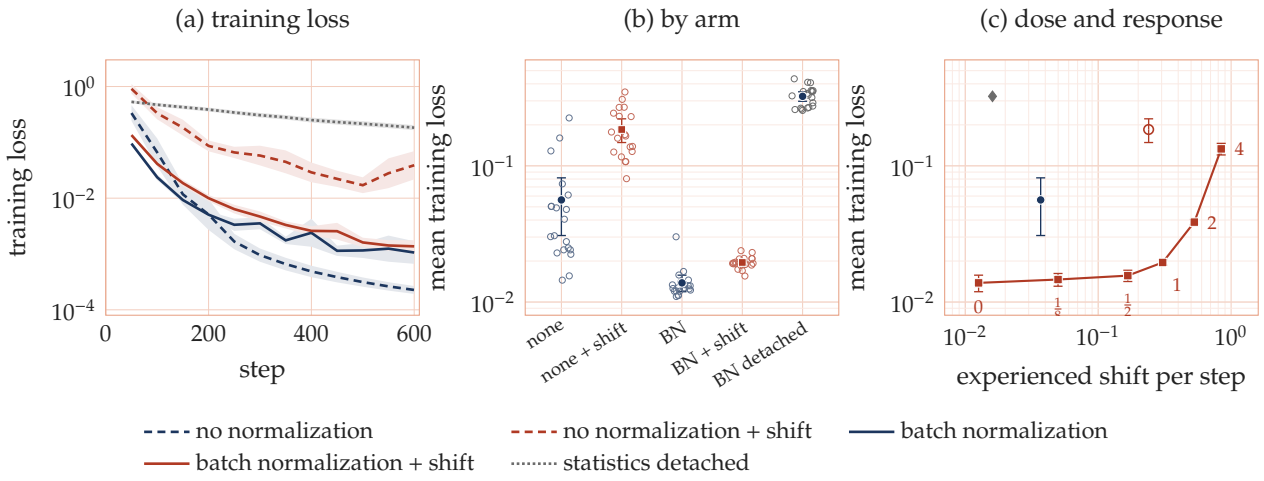


Figure 9.1. Lab 9.1: the six-layer perceptron on the digits at learning rate 0.3, 20 seeds per arm (seeds 0–19, shared across arms). (a) Training loss every 50 steps: geometric means over runs with 95% t-bands on the log scale. (b) Mean training loss over the twelve checkpoints, one hollow mark per run, filled marks the means with 95% t-intervals; vermilion marks the injected shift. (c) Mean training loss against the measured experienced shift: the normalized network at the doses of the shift written beside the squares (8 seeds at doses $\frac{1}{8}, \frac{1}{2}, 2, 4$), the unnormalized network without and with the shift (filled and hollow circles), and batch normalization with detached statistics (diamond).

of the unit's standard deviation, averaged over units and layers), noise included; the *update shift*, the same with the noise held fixed, which counts only the change caused by the parameter update, as Ioffe and Szegedy's definition does; and Santurkar et al.'s *gradient cosine* between the gradients of a layer's weights before and after the layers below it are updated. About four minutes.

PREDICT FIRST. How much of normalization's speed-up survives an injected shift larger than the unnormalized network's own? Does the same shift hurt a network without normalization? Does steadiness without the projection of Theorem 9.5 help?

LOOK FOR. Whether the knockout restores the mediator seed by seed under each definition; the dose at which the shift starts to cost speed; the test error, where the noise has a side effect.

WHAT HAPPENED. Normalization sped up training on every one of the 20 seeds, lowering the mean training loss by 0.042 ± 0.026 (paired differences, mean and 95% t-interval) and the final test error by 0.85 ± 0.30 points (Figure 9.1, Table 9.1), although the unnormalized network ended with the lower training loss, 0.0003 against 0.003. The first link of the claim held as well: by the update shift, the normalized distributions moved a third as much per step as the unnormalized ones, less on every seed. For one unit of the last hidden layer, the spread between the 15th and 85th percentiles of its input grew from 1.27 to 4.79 without normalization and from 1.75 to 2.45 with it, Ioffe and Szegedy's picture in miniature.

	covariate shift			outcome	
arm	experienced	from the update	gradient cosine	mean training loss	test error (%)
no normalization	0.037 ± 0.005	0.037 ± 0.005	0.82 ± 0.03	0.056 ± 0.025	3.84 ± 0.20
with the shift	0.240 ± 0.008	0.081 ± 0.009	0.45 ± 0.06	0.185 ± 0.037	4.71 ± 1.92
batch normalization	0.013 ± 0.001	0.013 ± 0.001	0.68 ± 0.04	0.014 ± 0.002	2.99 ± 0.26
with the shift	0.306 ± 0.001	0.016 ± 0.001	0.59 ± 0.04	0.020 ± 0.001	2.65 ± 0.19
statistics detached	0.016 ± 0.001	0.016 ± 0.001		0.324 ± 0.026	8.82 ± 0.77

Table 9.1. Lab 9.1: three measures of covariate shift and two outcomes in the five arms, means with 95% t-intervals over 20 seeds (over the 19 runs that trained for the unnormalized network with the shift). The shifts are per step, in units of a unit's standard deviation; a gradient cosine of one would mean no shift by Santurkar et al.'s definition.

The knockout restored the mediator, by one definition. With the shift injected, the experienced shift was nine times that of the unnormalized network on average and larger on every seed, so hypothesis (iii) of Proposition 9.4 held. The normalized network still trained almost as fast: its mean training loss rose by 0.0057 ± 0.0021 (on 19 of 20 seeds), against a total effect of 0.042, so that under the proposition's hypotheses at most about a seventh of the speed-up runs through the reduced shift. The dose–response (panel c) sharpens the point. At dose $\frac{1}{2}$, which already exceeds the unnormalized network's shift on all 8 seeds, the loss rose by only 0.0004 ± 0.0049, against a total effect of 0.038 on the same seeds; the cost appears between doses $\frac{1}{2}$ and 1, reaches most of the benefit at dose 2 (0.023 of 0.038), and only at dose 4, 23 times the unnormalized network's shift, does the normalized network train more slowly than the unnormalized one. Test error is not a valid outcome for the bound: the shift lowered it, by 0.33 ± 0.27 points, a side effect of the noise that violates the proposition's hypotheses for that outcome.

The other arms locate the effect. Injected into the unnormalized network, the same shift slowed it, raising the mean training loss by 0.128 ± 0.044 on the 19 seeds that trained; one seed diverged and another ended at 21% test error. The shift that a normalized network ignores hurts a network without normalization, as Theorem 9.5 explains: normalization does not keep the distributions still, it removes their first two moments from the loss. The detached arm is a rescue in the sense of Section 9.4: it keeps the normalized forward pass and removes the normalization from the gradient. Its distributions were nearly as steady as those of batch normalization (an update shift of 0.016 against 0.013) and steadier than the unnormalized network's on 19 of 20 seeds, yet it trained worse than either, with a mean training loss of 0.32 ± 0.03 and a test error of 8.82 ± 0.77%: steadiness without the projection is not enough, as Ioffe and Szegedy found in their first experiments. Finally, the three definitions of the mediator tell different stories. By the gradient cosine, normalization increased the shift (0.82 to 0.68, lower on 19 of 20 seeds), as Santurkar et al. found; and by the update shift, which counts only the change caused by the parameter update, the injected noise did not restore the shift at all (0.016 against 0.037, lower on

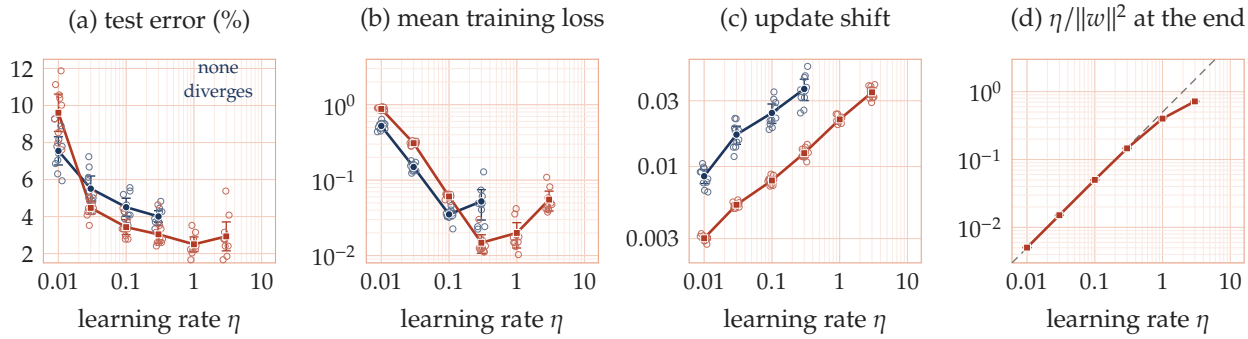


Figure 9.2. Lab 9.2: the network of Lab 9.1 without normalization (ink circles) and with it (vermilion squares), ten seeds per learning rate, paired; hollow marks are runs, jittered horizontally, and filled marks with lines are means with 95% t-intervals over the runs that trained. The unnormalized network diverged on every seed at $\eta \geq 1$, and the normalized one ended at chance on every seed at $\eta = 10$; neither is drawn. (d) Median over rows of the effective learning rate of the weight rows that feed a normalization, at the end of training; the dashed line is its value at the start, $\eta/2$.

every seed). Under Ioffe and Szegedy's own definition, then, the noisy arm is not a knockout and the proposition is silent; Lab 9.2 supplies the comparison.

LAB 9.2 · THE ADVANTAGE AGAINST THE LEARNING RATE

SCRIPT
`09-mechanisms-lr.py`

SETUP. The network of Lab 9.1 without and with batch normalization at learning rates 0.01, 0.03, 0.1, 0.3, 1, 3, 10, ten seeds per cell, paired across arms; mean training loss, final test error and update shift as there. For the normalized network the script checks at every step, for every weight row that feeds a normalization, that the bias before the normalization gets no gradient (Corollary 9.6) and that the squared norm of the row grows by exactly $\eta^2\|\nabla_w \widehat{\mathcal{L}}\|^2$ (Proposition 9.8(iii)), and it reports the median effective learning rate $\eta/\|w\|^2$ over rows at the start and end of training. A numerical check of Theorem 9.5 precedes the runs. About three minutes.

PREDICT FIRST. At which learning rates does normalization help, and does it ever hurt? Up to which learning rate does each network train? Each at its own best learning rate, by how much does normalization win?

LOOK FOR. The sign of the advantage at small learning rates; where the unnormalized network diverges; the update shift of the normalized network where the unnormalized one fails; the growth of the norms.

WHAT HAPPENED. The advantage of normalization depends on the learning rate and changes sign (Figure 9.2). At $\eta = 0.01$ the normalized network is worse on every seed, with a test error higher by 2.06 ± 0.77 points (paired, over ten seeds) and slower training. At $\eta = 0.03$ and 0.1 it reaches a lower test error, by 1.04 ± 0.87 and 1.08 ± 0.54 points, but still trains more slowly

on every seed; only at $\eta = 0.3$ is it both faster, on all ten seeds, and better, by 0.96 ± 0.53 points. Beyond that the comparison ends: the unnormalized network diverges on every seed at $\eta \geq 1$, while the normalized one trains at $\eta = 1$ and 3, reaches its best test error, 2.50%, at $\eta = 1$, and fails at $\eta = 10$, with more than a third of its units dead at the end. Each at its own best learning rate, normalization wins by 1.50 ± 0.47 points. This is the finding of Bjorck et al. (2018) in miniature: the benefit belongs to the large learning rates that only the normalized network survives, and at a small common learning rate, which is not a common step (Proposition 9.8), it reverses.

The update shift grows with the learning rate in both networks, and the normalized network at $\eta = 1$ and 3 moves its distributions about as much per step as the unnormalized one at 0.1 and 0.3 (0.022 and 0.035 against 0.025 and 0.037), while reaching a lower test error. At equal update shift normalization still helps, which is the comparison the injected noise could not provide for Ioffe and Szegedy's definition; it is suggestive rather than decisive, since a learning rate changes more than the shift. Scale invariance held to rounding error. On every step the biases before the normalizations received gradients below 5×10^{-8} (1.4×10^{-6} at $\eta = 10$), and the squared norm of every row grew by $\eta^2 \|\nabla_w \widehat{\mathcal{L}}\|^2$ to a relative error of at most 2×10^{-5} over 600 steps. At $\eta \leq 0.3$ the norms barely grew; at $\eta = 1$ and 3 they grew by factors of 1.22 and 2.09, and the effective learning rate fell to 0.40 and 0.72, so that a threefold range of nominal learning rates became a range of 1.8. The normalized network survives large learning rates in part because it does not take them.

The miniature reproduced the post-mortem in its main lines, the knockout, the gradient-based shift and the learning rate, and differed from it in one respect: in Santurkar et al.'s network the injected shift stopped the unnormalized network from training, while here it slowed it down.

9.7 Protocol

PROTOCOL 9.1 · TESTING A MECHANISM CLAIM

- ☐ Write the claim as a chain from treatment to mediator to outcome, and list the rival mediators, before running anything.
- ☐ Define the mediator by a measurement, naming the quantity, the data, the time scale and the divergence, and measure it in every arm with intervals over seeds.
- ☐ Check the first link: the treatment moves the mediator at the settings of the claim.
- ☐ Knock the mediator out: with the treatment on, restore it to its untreated level and no further, verify the level seed by seed, and report the remaining benefit as a bound on the indirect effect, with the assumptions that make it one.
- ☐ Control the knockout by applying the same operation without

the treatment; if it changes the outcome by a route of its own, then judge the knockout on an outcome that this route does not touch.

- ☐ Rescue: supply the mediator without the treatment, or remove the treatment's other actions; vary the dose and plot the outcome against the measured mediator across all arms.
- ☐ Compare the arms at their own best hyperparameters as well as at shared ones.
- ☐ Name a prediction that the mechanism makes and its rivals do not, and test it.

The protocol tests a mechanism against the mediators someone thought of; it cannot exclude one that nobody named. Every intervention on a mediator risks side effects, which the control of step 5 checks only in the arm where it is run, and the mediators of a training run are high-dimensional processes summarized by scalars, so that a different summary can change the verdict, as the definitions of covariate shift did. The protocol is also expensive: a knockout, its control and a rescue triple the arms of a comparison, and a dose–response multiplies them again. At scale, run the tests on the smallest model in which the effect is present and say so, since the mechanism of a small network may not be that of a large one; and when only the runs of a sweep are available, the evidence is correlational and Chapter 10 applies.

Exercises

9.1 CONSOLIDATION. ★ (The constant in the denominator.) Implementations normalize by $\sigma_\epsilon = (\sigma^2 + \epsilon)^{1/2}$ with a small $\epsilon > 0$. Show that the Jacobian of $y \mapsto \hat{y}$ is then $\sigma_\epsilon^{-1}(I - \frac{1}{B}\mathbf{1}\mathbf{1}^\top - \frac{1}{B}\hat{y}\hat{y}^\top)$ with $\|\hat{y}\|^2 = B\sigma^2/\sigma_\epsilon^2$, that this matrix is not a projection, and that $\|\nabla L(y)\|^2 = (\gamma^2/\sigma_\epsilon^2)\big(\|g\|^2 - \frac{1}{B}\langle\mathbf{1}, g\rangle^2 - a(2-a)\langle\hat{y}, g\rangle^2/\|\hat{y}\|^2\big)$ with $a = \sigma^2/\sigma_\epsilon^2$. Which parts of Theorem 9.5 and Corollary 9.6 survive exactly? *(Hint: the matrix is $I - e_1e_1^\top - a\, e_2e_2^\top$ with $e_2 = \hat{y}/\|\hat{y}\|$, and its square is $I - e_1e_1^\top - (2a-a^2)e_2e_2^\top$.)*

9.2 CONSOLIDATION. (Batches of two.) Show that for $B = 2$ the normalized vector is $(1, -1)$ or $(-1, 1)$, so that near every y with $y_1 \neq y_2$ the unit's output does not depend on y, and check that Theorem 9.5 gives $P = 0$. Show that the form printed in the proceedings (Remark 9.7) would assert that $0 \leq (\gamma^2/\sigma^2)(\frac{1}{2} - \frac{1}{\sqrt{2}})\langle\hat{y}, g\rangle^2$. What does $P = 0$ mean for training with batches of two?

9.3 CONSOLIDATION. (Weight decay and the effective learning rate.) For a scale-invariant f and the step $w' = w - \eta(\nabla f(w) + \lambda w)$, show that $\|w'\|^2 = (1-\eta\lambda)^2\|w\|^2 + \eta^2\|\nabla f(w)\|^2$. Assuming $\|\nabla f(u)\| = G$ for every unit vector u, show that the squared norm has a fixed point N with $N^2(1-(1-\eta\lambda)^2) = \eta^2G^2$, and that the effective learning rate there is close to $\sqrt{2\eta\lambda}/G$ for small $\eta\lambda$: the learning rate and the weight decay

act on the direction only through their product.

9.4 CONSOLIDATION. (Two indirect effects.) Construct potential outcomes for two equally likely kinds of run in which the treatment makes the outcome insensitive to the mediator, $Y(1, m)$ constant in m, while $Y(0, m)$ increases with m and $M(1) < M(0)$. Compute the total effect, the indirect effect of (9.1) and the pure indirect effect $\mathbb{E}Y(0, M(0)) - \mathbb{E}Y(0, M(1))$, and show that a knockout in the treated arm finds no mediation while a rescue in the untreated arm finds a large one. Which of the two does the covariate-shift claim assert?

9.5 LAB. (Shift before the normalization.) In `09-mechanisms-noisybn.py`, inject the shift before each normalization instead of after it. Predict the experienced shift and the training curve before running it: which part of the noise does the normalization remove, and which survives? Why is this design not a knockout?

9.6 LAB. (Which half of the projection.) In the detached arm of Lab 9.1, record the norms of the weight rows and the biases before each normalization during training, and compare them with Corollary 9.6 and Proposition 9.8(iii). Then detach only the batch mean, and then only the standard deviation: which half of the projection carries the benefit in this network?

9.7 LAB. (Weight decay in the sweep.) Repeat Lab 9.2 with weight decay $\lambda = 10^{-3}$. Compare the final effective learning rate with the fixed point of Exercise 9.3, using the measured gradient norm for G, and observe whether the reversal at small learning rates persists.

9.8 DESIGN. (Warm-up.) A linear warm-up of the learning rate is said to help Adam because it reduces the variance of the adaptive learning rate, which is large while the second-moment estimate rests on few gradients (Liu et al. 2020), or because it limits the size of the first updates (Ma and Yarats 2021). With 200 runs of a small model, design a knockout and a rescue that separate the two, with the measurement of each mediator and what each explanation predicts for each arm.

9.9 DESIGN. (Two rivals across batch sizes.) De and Smith (2020) found that the larger learning rates permitted by normalization matter mainly at large batch sizes, and Luo et al. (2019) that the noise of the batch statistics regularizes. Within 300 runs, design an experiment that separates the two across batch sizes, for instance by computing the statistics on sub-batches of fixed size.

9.10 CRITIQUE. (A correlation offered as a mechanism.) A paper proposes a normalization layer. Across 40 trained networks of different depths and widths, the covariate shift measured by the authors' divergence correlates with the number of steps to 95% training accuracy (Kendall's $\tau = 0.55$), and the new layer has both the lowest shift and the fewest steps; the authors conclude that it works by reducing covariate shift.

Which links of the chain does this evidence test, what could produce the correlation without the mechanism, and which two experiments would test the claim?

9.11 CRITIQUE. (A knockout that hurts.) To show that dropout regularizes by preventing co-adaptation, a study adds to a network trained with dropout a penalty that rewards correlated hidden units, finds that test accuracy falls, and concludes that co-adaptation mediates the benefit of dropout. Which hypothesis of Proposition 9.4 carries the conclusion, why does a knockout that hurts prove little, and what control would make the result informative?

9.12 RESEARCH. (Small batches.) The benefit of batch normalization fades at small batch sizes, where batch renormalization (Ioffe 2017) and group normalization (Wu and He 2018) were proposed to replace it. Which mediator explains the loss? Open: a design that separates the noise of the batch statistics, the effective learning rate and the projection of Theorem 9.5 at small batch sizes, and whether some definition of covariate shift predicts training speed across normalization layers at a fixed effective learning rate.

Notes and References

Normalizing a network's inputs to zero mean and unit variance is old advice (LeCun et al. 1998), and covariate shift is a term from statistics, where Shimodaira (2000) corrected maximum likelihood for a change of the input distribution between training and prediction. Batch normalization was followed by normalizations that avoid its dependence on the batch, among them layer normalization (Ba et al. 2016), batch renormalization (Ioffe 2017) and group normalization (Wu and He 2018). The explanations offered after 2018 are those of the post-mortem, to which Luo et al. (2019) add regularization by the noise of the batch statistics; several are compatible, and the field has not agreed on their weights.

Mediation analysis has a long history outside machine learning. Baron and Kenny (1986) gave the regression approach that became standard in psychology, Robins and Greenland (1992) and Pearl (2001) defined direct and indirect effects by potential outcomes and asked when they are identified, and VanderWeele (2015) treats mediation together with interaction. In machine learning the interventions are cheap, since code can often set a mediator directly, as the injected noise sets covariate shift; the main danger is then the side effect of the intervention rather than unmeasured confounding. Lipton and Steinhardt (2019) used internal covariate shift as an example of speculation offered as explanation, and Chapter 10 takes up the other common support for a mechanism, a correlation across a sweep of trained networks.

TRANSLATING THE LITERATURE. Ioffe and Szegedy (2015) write m for the batch size (our B), α for the learning rate (our η), $x = Wu + b$ for the normalized pre-activation (our y) and $\mathrm{BN}(x)$ for the output (our z). Santurkar et al.

(2018) also write m for the batch size; their y_j, $\hat{y}_j$, z_j, γ, β and σ_j are ours for unit j, their $\widehat{\mathcal{L}}$ is our loss of the normalized network, and their $\nabla_{y_j}\mathcal{L}$, the gradient of the unnormalized network, is our g under their identical-network assumption. Their internal covariate shift of Definition 2.1 is the distance between the gradients G and G', whose cosine is our gradient cosine, and their noisy batch normalization with $n_\mu = 0.5$, $n_\sigma = 1.25$, $r_\mu = r_\sigma = 0.1$ is our injected shift at dose one. The scale-invariant parameters of Arora et al. (2019) are our weight rows that feed a normalization, and their auto rate-tuning is the decay of our η_{eff}. Our direct and indirect effects (9.1) are Pearl's natural direct and indirect effects, the pure direct and total indirect effects of Robins and Greenland; the mediation literature, like this chapter, writes A for the treatment and M for the mediator, which is not the best of k draws of the book's notation.

References

Arora, S., Z. Li, and K. Lyu (2019). "Theoretical Analysis of Auto Rate-Tuning by Batch Normalization". In: *International Conference on Learning Representations*.

Ba, J. L., J. R. Kiros, and G. E. Hinton (2016). *Layer Normalization*. arXiv preprint arXiv:1607.06450.

Baron, R. M. and D. A. Kenny (1986). "The Moderator–Mediator Variable Distinction in Social Psychological Research: Conceptual, Strategic, and Statistical Considerations". In: *Journal of Personality and Social Psychology* 51.6, pp. 1173–1182.

Bjorck, J., C. Gomes, B. Selman, and K. Q. Weinberger (2018). "Understanding Batch Normalization". In: *Advances in Neural Information Processing Systems 31 (NeurIPS 2018)*.

Brock, A., S. De, S. L. Smith, and K. Simonyan (2021). "High-Performance Large-Scale Image Recognition Without Normalization". In: *Proceedings of the 38th International Conference on Machine Learning*. Vol. 139. Proceedings of Machine Learning Research, pp. 1059–1071.

Cohen, J., S. Kaur, Y. Li, J. Z. Kolter, and A. Talwalkar (2021). "Gradient Descent on Neural Networks Typically Occurs at the Edge of Stability". In: *International Conference on Learning Representations*.

De, S. and S. L. Smith (2020). "Batch Normalization Biases Residual Blocks Towards the Identity Function in Deep Networks". In: *Advances in Neural Information Processing Systems 33 (NeurIPS 2020)*.

Ghorbani, B., S. Krishnan, and Y. Xiao (2019). "An Investigation into Neural Net Optimization via Hessian Eigenvalue Density". In: *Proceedings of the 36th International Conference on Machine Learning*. Vol. 97. Proceedings of Machine Learning Research, pp. 2232–2241.

Ioffe, S. (2017). "Batch Renormalization: Towards Reducing Minibatch Dependence in Batch-Normalized Models". In: *Advances in Neural Information Processing Systems 30 (NIPS 2017)*.

Ioffe, S. and C. Szegedy (2015). "Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift". In: *Proceedings of the 32nd International Conference on Machine Learning*. Vol. 37. Proceedings of Machine Learning Research, pp. 448–456.

Laarhoven, T. van (2017). *L2 Regularization versus Batch and Weight Normalization*. arXiv preprint arXiv:1706.05350.

LeCun, Y., L. Bottou, G. B. Orr, and K.-R. Müller (1998). "Efficient BackProp". In: *Neural Networks: Tricks of the Trade*. Ed. by G. B. Orr and K.-R. Müller. Vol. 1524. Lecture Notes in Computer Science. Springer.

Li, Z. and S. Arora (2020). "An Exponential Learning Rate Schedule for Deep Learning". In: *International Conference on Learning Representations*.

Lipton, Z. C. and J. Steinhardt (2019). "Troubling Trends in Machine Learning Scholarship". In: *ACM Queue* 17.1. First circulated as arXiv:1807.03341, 2018.

Liu, L., H. Jiang, P. He, W. Chen, X. Liu, J. Gao, and J. Han (2020). "On the Variance of the Adaptive Learning Rate and Beyond". In: *International Conference on Learning Representations*.

Luo, P., X. Wang, W. Shao, and Z. Peng (2019). "Towards Understanding Regularization in Batch Normalization". In: *International Conference on Learning Representations*.

Ma, J. and D. Yarats (2021). "On the Adequacy of Untuned Warmup for Adaptive Optimization". In: *Proceedings of the AAAI Conference on Artificial Intelligence*.

Mosbach, M., M. Andriushchenko, and D. Klakow (2021). "On the Stability of Fine-tuning BERT: Misconceptions, Explanations, and Strong Baselines". In: *International Conference on Learning Representations*.

Pearl, J. (2001). "Direct and Indirect Effects". In: *Proceedings of the Seventeenth Conference on Uncertainty in Artificial Intelligence*.

Rahimi, A. and B. Recht (2017). *Reflections on Random Kitchen Sinks*. argmin blog, 5 December 2017; the text of the NIPS 2017 Test of Time Award talk.

Robins, J. M. and S. Greenland (1992). "Identifiability and Exchangeability for Direct and Indirect Effects". In: *Epidemiology* 3.2, pp. 143–155.

Santurkar, S., D. Tsipras, A. Ilyas, and A. Madry (2018). "How Does Batch Normalization Help Optimization?" In: *Advances in Neural Information Processing Systems 31 (NeurIPS 2018)*. Revised version: arXiv:1805.11604v4, March 2019.

Shimodaira, H. (2000). "Improving Predictive Inference under Covariate Shift by Weighting the Log-Likelihood Function". In: *Journal of Statistical Planning and Inference* 90.2, pp. 227–244.

VanderWeele, T. J. (2015). *Explanation in Causal Inference: Methods for Mediation and Interaction*. Oxford University Press.

Wu, Y. and K. He (2018). "Group Normalization". In: *Proceedings of the European Conference on Computer Vision (ECCV)*, pp. 3–19.

Yang, G., J. Pennington, V. Rao, J. Sohl-Dickstein, and S. S. Schoenholz (2019). "A Mean Field Theory of Batch Normalization". In: *International Conference on Learning Representations*.

Zhang, H., Y. N. Dauphin, and T. Ma (2019). "Fixup Initialization: Residual Learning Without Normalization". In: *International Conference on Learning Representations*.

10 Correlation in a Sweep

Flat minima and the measures of generalization

THE CLAIM *"Flat minima generalize better."*

Hochreiter and Schmidhuber (1997); made the explanation of the large-batch generalization gap by Keskar et al. (2017)

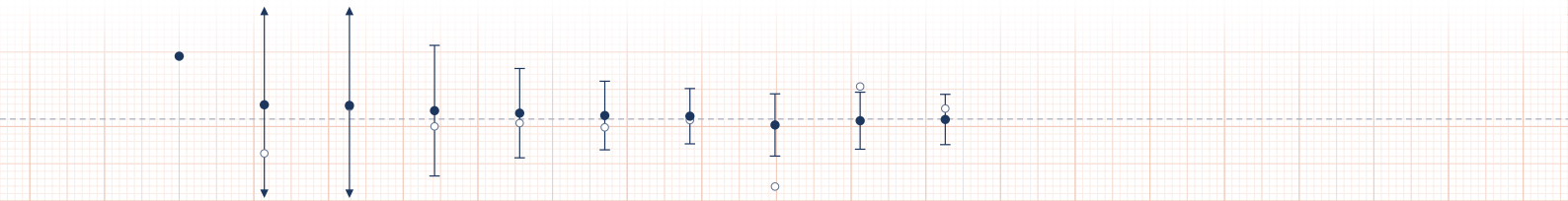
THE VERDICT Narrowed: sharpness changes under rescalings that leave the function intact, and across controlled sweeps it tracks generalization along some hyperparameters and not others, with either sign.

IN THIS CHAPTER. Hochreiter and Schmidhuber argued in 1997 that a flat minimum, a large region of weights with nearly constant training error, should generalize; in 2017 Keskar et al. explained the poorer test accuracy of large-batch training by the sharper minima it reached. But rescaling a ReLU network changes the curvature of its loss without bound and leaves the function unchanged, and in controlled sweeps sharpness tracked generalization along some hyperparameters only, and in modern networks followed the learning rate. The method is the analysis of a correlation in a sweep: pooled correlations split within and between strata, granulated rank correlations, replicate and random controls, invariant measures, and the intervention that a correlation calls for. In the laboratory, three sharpnesses correlate with the gap of 162 networks with both signs, all through the width, and a rescaling moves the top Hessian eigenvalue by a factor of 5,200 while the test error stays fixed.

10.1 The claim: flat minima and the large-batch gap

Hochreiter and Schmidhuber (1997) made the size of the neighborhood of a solution a principle. A *flat minimum* is, in their definition, a large connected region of weight space in which the training error stays approximately constant; at a sharp minimum it rises quickly. Their argument came from the theory of minimum description length: the weights of a flat minimum can be specified with low precision and therefore with few bits, a network with a short description is a simple one, and a simple network should overfit less.[1] They made the principle an algorithm, flat minimum search, which lowers the training error together with a term for the precision that the weights need, at about the cost of backpropagation thanks to the Hessian–vector products of Pearlmutter (1994); a short version had appeared in 1995 (Hochreiter and Schmidhuber 1995).

[1] Limiting the information in the weights by making them noisy is older: Hinton and Camp (1993) trained networks with Gaussian weights and penalized the information the weights carried.



> FINDING 10.1 (Hochreiter and Schmidhuber 1997, Section 5). On a noisy two-class problem with 200 training points, ten networks trained by flat minimum search had lower test error than ten networks trained by backpropagation from the same initializations, in each of the ten pairs. On three tasks of predicting the German stock index, flat minimum search had the best mean performance over 7 or 10 trials, ahead of the next method by 0.13, 2.79 and 8.29 points of the authors' measure; in the first task the trials of flat minimum search and of weight decay each spread over eight points.

Twenty years later the principle found its most famous application. Training with large minibatches, which parallelizes well, produced networks that tested worse, and Keskar et al. (2017) set out to explain why.[2]

[2] The paper first circulated on arXiv in September 2016 (arXiv:1609.04836).

> FINDING 10.2 (Keskar et al. 2017, Tables 2 and 3). Six networks (fully connected on MNIST and TIMIT, convolutional on CIFAR-10 and CIFAR-100) were trained with Adam on batches of 256 examples or of 10% of the training set, five runs each, until the loss stopped improving. Both regimes fit the training data to high accuracy, but the large-batch networks tested worse on all six, by 0.22 to 5.27 points of accuracy. A sensitivity measure, the largest relative increase of the training loss within a small box around the solution, was one to two orders of magnitude larger for the large-batch solutions: 205.14 against 1.23 for the network on MNIST.

Large-batch methods, they concluded, converge to sharp minimizers, with many large positive eigenvalues of the Hessian, while the noise of small batches pushes the iterates out of sharp basins; and the abstract treated the last link as settled: "as is well known, sharp minima lead to poorer generalization." The field received the general claim, that flat minima generalize better. It was received alongside the finding of Zhang et al. (2017) that standard classifiers fit random labels as easily as true ones, so that capacity measures that ignore the training algorithm could not explain generalization; it fitted the PAC-Bayesian bounds of McAllester (1999), which reward solutions that tolerate random perturbations of the weights and which Dziugaite and Roy (2017) made nonvacuous for deep networks; and it became a design target: Entropy-SGD biased training toward wide valleys (Chaudhari et al. 2017), and *sharpness-aware minimization* (SAM), which minimizes the largest training loss within a small ball around the weights, reported state-of-the-art results on several image benchmarks (Foret et al. 2021).

WHAT WAS RIGHT. The observation held where it was made: in all six settings the large-batch solutions were the sharper ones by the authors' metric, and their test accuracy was lower. The authors called their metric imperfect and computed it in the full space and in random subspaces, and the abstract offers numerical evidence that supports a view. The description-length and PAC-Bayesian arguments are sound as far as they go, and later work keeps finding settings in which flatter solutions do generalize better. Why the large-batch gap appeared, with fewer updates

and a learning rate not adapted to the batch size, is the subject of Chapter 7; this chapter examines the explanation, that sharpness causes, or at least measures, generalization.

10.2 The experiment: two batch sizes and a sensitivity

EXPERIMENTAL DESIGN · KESKAR ET AL. 2017

QUESTION	Why do large-batch methods generalize worse? Do they converge to sharp minimizers?
TREATMENT	the batch size: 256 examples, or 10% of the training set
HELD FIXED	network and data set, the Adam optimizer, the initialization scheme; training until the loss stops improving
VARIED	the starting point: five random initializations per arm
RESPONSE	test accuracy; the largest relative increase of the training loss in a box of relative half-width 10^{-3} or $5{\cdot}10^{-4}$, by ten iterations of L-BFGS-B, in the full space and in random subspaces
REPLICATES	five runs per arm, for each of six networks
ANALYSIS	means and standard deviations of both responses in each arm; plots of the loss on the line through the two solutions

What could this design show? It varies one thing, the batch size, and records two outcomes, and both differed between the arms for every network. A causal chain from batch size through sharpness to test accuracy would produce this pattern; so would a batch size that changes test accuracy through another path, such as the number of updates, and changes sharpness as a side effect. With one treatment at two levels, nothing in the data separates the two accounts: there is no variation in sharpness at a fixed batch size, and no intervention on sharpness that leaves the batch size alone. And the metric's box is defined in the coordinates of the weights,[3] while a network has many coordinate systems that compute the same function; nothing in the design shows that the metric belongs to the function. The evidence of 1997 had a similar shape: flat minimum search also pruned units and weights, so its success supports the algorithm more directly than the principle, and in the first stock-market task its margin was small against the spread of the trials (Chapter 1). Later studies replaced the two arms by a sweep of many networks; a sweep has far more variation, but it remains an observational study of the measure.

[3] With A the identity, the box of Keskar et al. (2017) allows each weight x_i to move by $\varepsilon(|x_i| + 1)$: partly in proportion to the weight, partly by a fixed amount.

10.3 The post-mortem: rescaled minima and controlled sweeps

RESCALED MINIMA. Dinh et al. (2017) examined the coordinates. A rectifier is *positively homogeneous*, $\mathrm{ReLU}(cz) = c\,\mathrm{ReLU}(z)$ for $c > 0$, so a network with one hidden layer computes the same function after its first layer is multiplied by c and its second divided by c. With this family of equivalent weights they showed that every minimum of such a network (other than a constant

function) has a region of nearly constant loss of infinite volume; that every minimum with a nonzero Hessian is equivalent to one whose Hessian has arbitrarily large spectral norm; and that the box sharpness of Keskar et al. can be made large in the same way. With other changes of parameters, such as weight normalization, a minimum can be made flatter or sharper at will: flatness cannot be separated from the parametrization. Theorem 10.8 below proves the rescaling result for the networks of our laboratory. Neyshabur et al. (2017) argued that sharpness can control generalization only together with a norm, as in a PAC-Bayesian bound.

Forty measures, two thousand networks. Jiang et al. (2020) built the large controlled sweep that the question needed.

> Finding 10.3 (Jiang et al. 2020, Tables 1–3). Convolutional networks of the network-in-network type were trained on CIFAR-10 under seven hyperparameters at three levels each (batch size, dropout, learning rate, depth, weight decay, width, optimizer), 2,187 networks, each until its training cross-entropy reached 0.01; repetitions and a second data set brought the total above 10,000. Among more than 40 measures, a magnitude-aware sharpness reached Kendall $\tau = 0.484$ with the generalization gap and a granulated coefficient $\Psi = 0.516$, above the $\Psi = 0.487$ of an oracle that sees the gap with Gaussian noise of standard deviation 0.02. A bound built on the spectral norms of the layers reached $\tau = -0.537$, and many norm-based measures correlated negatively. The sharpness of Keskar et al. reached $\tau = 0.400$, but its coefficient for dropout alone was -0.359.

The design guarded against confounding: every run stopped at the same training loss, because after a fixed number of epochs the training loss itself predicts generalization; correlations were averaged within groups of networks that differ in one hyperparameter only; and a conditional mutual information asked whether a measure keeps its information once one or two hyperparameters are known. By their account sharpness passed best, but the conditional information of its magnitude-aware form, 0.054 on their normalized scale, was about that of an oracle that sees the gap with noise of standard deviation 0.05 (0.050).[4]

Robustness. Dziugaite et al. (2020) asked what an average hides. They trained 10,000 networks, 1,000 settings of learning rate, depth, width, training-set size and data set with ten seeds each, and scored 24 of Jiang et al.'s measures on more than 1.6 million pairs of runs by their worst *environment*, a pair of settings that differ in one hyperparameter. For every measure there was an environment in which it predicted the direction of the change in generalization wrongly for every pair of seeds. Noise mattered as much: of 4,000 environments that changed the width, only 328 showed a difference in generalization larger than the variation between seeds.

Modern networks. Kwon et al. (2021) made sharpness invariant to rescaling by measuring each perturbation relative to the size of its weight, their *adaptive* sharpness, which on 625 wide residual networks correlated

[4] The noisy oracles rank the networks by their true gap plus Gaussian noise of a stated size, which calibrates how far a measure could get.

with the gap at $\tau = 0.64$ against 0.17 for the plain version. Andriushchenko et al. (2023) tested it where networks are now trained: vision transformers on ImageNet (56 models), CLIP fine-tuned on ImageNet (72), BERT fine-tuned on MNLI (50), and 200 ResNets and 200 transformers on CIFAR-10. Sharpness did not predict which models generalized best. It correlated instead with the learning rate, whose effect on generalization changed sign from one setting to the next; and for the ResNets it tracked the test error within groups sharing their augmentation and mixup but not across them, where the pooled τ was 0.30 or -0.36, depending on a normalization.

INTERVENTIONS. SAM's success invited the converse reading, that lowering sharpness is why it works. Andriushchenko and Flammarion (2022) found that the variants closest to the theory, random perturbations and SAM on the whole training set, barely improve generalization, while SAM on small batches does, and that a flatter minimum can generalize worse: SAM with batches of 1024 found flatter minima than plain training with batches of 128, which tested better (6.17% against 6.80% error on CIFAR-10). Wen et al. (2023) proved that for two-layer ReLU networks, depending on the architecture, the flattest minimizers may generalize; flattest minimizers may fail to generalize and sharpness minimization fail with them; or they may fail while sharpness minimization still generalizes. And Kaur et al. (2023) found that scaling the batch size and the learning rate together changes the top Hessian eigenvalue without changing generalization.

VERDICT. The claim that flat minima generalize better did not survive as a general law. Sharpness is not a property of the function a network computes, since rescaling a ReLU network changes it without bound, and within a fixed parametrization its correlation with generalization depends on what varies: magnitude-aware sharpness ranked small convolutional networks well on average, every measure failed under some intervention, and in modern networks adaptive sharpness followed the learning rate rather than the test error. What survives is narrower: along some hyperparameters and in some families of networks flatter solutions generalize better, and training that seeks flat regions often helps, for reasons not yet settled.

NARROWED

10.4 The method: what a correlation across a sweep measures

A sweep trains a network for each configuration h of a grid of hyperparameters, usually with a few seeds per configuration, and records for each run a candidate *generalization measure* M, computed from the trained network and its training data, and its *generalization gap* G, the test error minus the training error. Both are outcomes of the run, so a sweep is a designed experiment for the hyperparameters and an observational study for the measure: a correlation between M and G can arise because M influences G, because the configuration moves both, or because the seed moves both.

Definition 10.4 (Three measures of sharpness). Let $\theta \in \mathbb{R}^N$ be the parameters of a trained network, $\widehat{\mathcal{L}}$ its training loss, twice differentiable at θ, and ξ a standard Gaussian vector in $\mathbb{R}^N$.

(i) The *top eigenvalue* $\lambda_{\max}(\theta)$ is the largest eigenvalue of $\nabla^2 \widehat{\mathcal{L}}(\theta)$.

(ii) The *perturbation sharpness* at scale $\sigma > 0$ is $P_\sigma(\theta) = \mathbb{E}\, \widehat{\mathcal{L}}(\theta + \sigma\xi) - \widehat{\mathcal{L}}(\theta)$.

(iii) The *adaptive perturbation sharpness* is $P^{\mathrm{ad}}_\sigma(\theta) = \mathbb{E}\, \widehat{\mathcal{L}}(\theta + \sigma\, \theta \odot \xi) - \widehat{\mathcal{L}}(\theta)$, where $\odot$ multiplies coordinatewise, so that each weight is perturbed in proportion to its size.

The first is the curvature in the steepest direction, which a small box of Keskar et al. approximates; the second averages over random directions, as a PAC-Bayesian bound does; the third is the average-case form of the adaptive sharpness of Kwon et al. (2021). The top eigenvalue follows from *power iteration* $v \mapsto \nabla^2 \widehat{\mathcal{L}}(\theta) v / \|\nabla^2 \widehat{\mathcal{L}}(\theta) v\|$, each step a Hessian–vector product that costs about as much as a gradient (Pearlmutter 1994); the others, from a few dozen Gaussian draws.

POOLED AND WITHIN STRATA. The basic tool splits a pooled correlation into a part within groups of runs, the *strata*, and a part between them: for covariances by the law of total covariance, for Kendall's τ by a split of the pairs of runs.

Proposition 10.5 (Simpson's decomposition). *(i) Let X and Y be random variables with finite variances and V a random variable with finitely many values, each taken with positive probability. Then*

$$\operatorname{Cov}(X, Y) = \mathbb{E}\big[\operatorname{Cov}(X, Y \mid V)\big] + \operatorname{Cov}\big(\mathbb{E}[X \mid V], \mathbb{E}[Y \mid V]\big).$$

(ii) Let $n \geq 2$ runs of a sweep have values (x_j, y_j) and strata v_j; write $s_{jk} = \operatorname{sgn}(x_j - x_k)\operatorname{sgn}(y_j - y_k)$, let a fraction w of the $\binom{n}{2}$ pairs $j < k$ lie in a common stratum, and let τ_{within} and τ_{between} be the averages of s_{jk} over these pairs and over the other pairs (an average over no pairs counts as zero). Then Kendall's $\tau = \binom{n}{2}^{-1} \sum_{j<k} s_{jk}$ of Proposition A.15, with ties counted as zero, satisfies

$$\tau = w\, \tau_{\mathrm{within}} + (1 - w)\, \tau_{\mathrm{between}}. \tag{10.1}$$

If the runs form m strata of $r \geq 2$ runs each, then $w = (r-1)/(mr-1)$ and τ_{within} is the average of the m values of τ within the strata.

(iii) In the sweep of nine runs $(x, y) = (3v+d,\ 3v-d)$ with $v \in \{1, 2, 3\}$ and $d \in \{-1, 0, 1\}$, stratified by v, every stratum has Pearson correlation -1 and $\tau = -1$, while the pooled Pearson correlation is $4/5$ and the pooled τ is $1/2$.

PROOF. The plan is to prove (i) by centering at the conditional means, (ii)

by splitting a sum, and (iii) from both.

Part (i). Write $\bar{X} = \mathbb{E}[X \mid V]$ and $\bar{Y} = \mathbb{E}[Y \mid V]$, the elementary conditional means $\mathbb{E}[X \mid V = v] = \mathbb{E}[X\mathbf{1}\{V = v\}]/\mathbb{P}(V = v)$. For every function g of V, $\mathbb{E}[(X - \bar{X})g(V)] = \sum_v g(v)\,\mathbb{E}[(X - \bar{X})\mathbf{1}\{V = v\}] = 0$, because each term vanishes by the definition of $\mathbb{E}[X \mid V = v]$. Splitting $X - \mathbb{E}X = (X - \bar{X}) + (\bar{X} - \mathbb{E}X)$ and $Y - \mathbb{E}Y$ in the same way and expanding the product, the cross terms have mean zero by this remark, since $\bar{X} - \mathbb{E}X$ and $\bar{Y} - \mathbb{E}Y$ are functions of V. The remaining terms have means $\mathbb{E}[(X - \bar{X})(Y - \bar{Y})] = \mathbb{E}[\mathrm{Cov}(X, Y \mid V)]$ and $\mathbb{E}[(\bar{X} - \mathbb{E}X)(\bar{Y} - \mathbb{E}Y)] = \mathrm{Cov}(\bar{X}, \bar{Y})$, the latter because $\mathbb{E}\bar{X} = \mathbb{E}X$ and $\mathbb{E}\bar{Y} = \mathbb{E}Y$. All products are integrable by the Cauchy–Schwarz inequality.

Part (ii). Splitting the sum over pairs into the $w\binom{n}{2}$ pairs in a common stratum and the others gives $\sum_{j<k} s_{jk} = w\binom{n}{2}\tau_{\text{within}} + (1 - w)\binom{n}{2}\tau_{\text{between}}$, and dividing by $\binom{n}{2}$ gives (10.1). With m strata of r runs, $m\binom{r}{2}$ of the $\binom{mr}{2}$ pairs lie in a common stratum, a fraction $mr(r-1)/\big(mr(mr-1)\big) = (r-1)/(mr-1)$; every stratum contributes the same number $\binom{r}{2}$ of pairs, so the average over these pairs is the average of the strata's values of τ.

Part (iii). Within stratum v we have $y = 6v - x$, a decreasing function of x, so the Pearson correlation is -1 and every pair is discordant. Two runs in strata $v < v'$, with offsets d and d', satisfy $x' - x = 3(v' - v) + (d' - d) \geq 3 - 2 > 0$ and $y' - y = 3(v' - v) - (d' - d) \geq 1$, so every pair in different strata is concordant. Part (ii) with $m = r = 3$ gives $w = 1/4$ and $\tau = -\frac{1}{4} + \frac{3}{4} = \frac{1}{2}$. For the Pearson correlation, let (X, Y, V) be one of the nine runs drawn uniformly, so that V and the offset d are independent and uniform on three values, with variances $2/3$. Given V, $\mathrm{Cov}(X, Y \mid V) = \mathrm{Cov}(d, -d) = -2/3$ and $\mathbb{E}[X \mid V] = \mathbb{E}[Y \mid V] = 3V$, so part (i) gives $\mathrm{Cov}(X, Y) = -\frac{2}{3} + 9 \cdot \frac{2}{3} = \frac{16}{3}$; part (i) with $Y = X$ gives $\mathrm{Var}\, X = \frac{2}{3} + 6 = \frac{20}{3}$, and in the same way $\mathrm{Var}\, Y = \frac{20}{3}$. Hence $\mathrm{Corr}(X, Y) = (16/3)/(20/3) = 4/5$. ■

Part (ii) is the arithmetic of *Simpson's paradox* (Simpson 1951) for rank correlations:[5] the weight $(r - 1)/(mr - 1)$ of the part within strata falls as the number of strata grows, so that in a sweep of many configurations with a few seeds each, the pooled τ is essentially the agreement between the effects of the hyperparameters on the measure and on the gap. That agreement is real, but it is no evidence that the measure has anything to do with generalization beyond the hyperparameters that move both. In the classic example, the graduate admissions at Berkeley in 1973, the pooled admission rate of women was lower than that of men while department by department there was little sign of a bias against them: women had applied more often to the departments that admitted few applicants (Bickel et al. 1975). Whether to stratify is a causal question, answered by the graph of causes and not by the data: stratify by a common cause of measure and gap, not by a consequence of either (Pearl 2009).

[5] Kendall's τ depends only on ranks, so it does not care that the adaptive sharpness of Lab 10.1 spans a factor of 60; a Pearson correlation would be dominated by the extremes.

Definition 10.6 (Granulated coefficient). Let a sweep cross hyperparameters $h = (h_1, \ldots, h_q)$ in a full factorial design, with the replicate index treated as one more coordinate to hold fixed. For each coordinate i, the strata of i are the groups of runs that agree in every coordinate

except h_i. The *granulated coefficient* ψ_i is the average over the strata of i of Kendall's τ between measure and gap within the stratum, and $\Psi = \frac{1}{q}\sum_i \psi_i$ (Jiang et al. 2020).

By Proposition 10.5(ii), ψ_i keeps only the pairs of runs that differ in h_i alone: it asks whether the measure predicts the effect on generalization of changing one hyperparameter while the others stay fixed. In a thought experiment of Jiang et al., a measure that recovers the depth of a network exactly but is random among networks of equal depth reached $\tau = 0.362$ in their sweep but $\Psi = 0.11$. And if the gap and the measure were increasing functions of one common quantity, then every ψ_i would be positive up to noise; coefficients of both signs rule this out.

HYPERPARAMETERS AS COMMON CAUSES. Why would hyperparameters move sharpness? The cleanest mechanism concerns the learning rate. On a quadratic loss with Hessian A, gradient descent with step η multiplies the component of the error along an eigenvector of A by $1 - \eta\lambda$ at each step, so that it grows geometrically when $\eta\lambda > 2$: a run with step η cannot settle where $\lambda_{\max} > 2/\eta$ (Exercise 10.4).[6] On real networks, full-batch gradient descent drives the top eigenvalue up to this bound and holds it just above $2/\eta$, the *edge of stability* of Cohen et al. (2021); with minibatches the noise imposes further conditions that depend on the batch size (Wu et al. 2018). Width, weight decay and the training loss at which a run stops move sharpness too, each with its own path to generalization; weight decay, for instance, shrinks the weights, so that flatness and a small norm arrive together and cannot be told apart (Chapter 9).

[6] With heavy-ball momentum β the cap is $2(1+\beta)/\eta$, or $3.8/\eta$ at $\beta = 0.9$.

CONTROLS AND THE RELIABILITY OF THE GAP. A correlation with the gap can be no stronger than the gap is reproducible.

REMARK 10.7 (The replicate ceiling). Suppose that the gap of a run is $G = g(h) + \varepsilon$, a part $g(h)$ fixed by the configuration and seed noise ε with mean zero and variance σ_ε^2, independent of h, and let $G' = g(h) + \varepsilon'$ be the gap of a replicate run, with ε' an independent copy of ε. Then $\operatorname{Cov}(G, G') = \operatorname{Var} g(h)$ and $\operatorname{Var} G = \operatorname{Var} G' = \operatorname{Var} g(h) + \sigma_\varepsilon^2$, so the two replicates have correlation

$$\rho_I = \frac{\operatorname{Var} g(h)}{\operatorname{Var} g(h) + \sigma_\varepsilon^2},$$

the intraclass correlation of the gap, which a one-way analysis of variance of replicate runs estimates (Theorem A.12). A measure $M = m(h)$ of the configuration has $\operatorname{Cov}(M, G) = \operatorname{Cov}(m(h), g(h))$, so that $\operatorname{Corr}(M, G) = \operatorname{Corr}(m(h), g(h))\sqrt{\rho_I} \le \sqrt{\rho_I}$, with equality for $m = g$. This is the attenuation of Spearman (1904): a noisy gap caps every measure of the configuration, and a measure computed on the run itself can pass the cap only by tracking the seed noise of that run.

Two controls make the ceiling and the noise visible (Chapter 5). The *positive control* uses the gap of a replicate run as if it were a measure: its

correlation with the gap estimates ρ_I, and with it the ceiling. The negative control, a random number per run, shows how far the statistics wander with no signal at all. Stratifying by every hyperparameter leaves only the replicates of one configuration; if a measure predicts which of two such runs generalizes better, then it carries information that no hyperparameter supplies. And Dziugaite et al. (2020) replace the average over strata by the worst stratum, since a measure meant to explain generalization should predict the direction of change in every environment.

10.5 The method: measures that respect the function

A measure meant to describe the function a network computes should not change when the parameters change without changing the function, and for ReLU networks the positive homogeneity of the rectifier supplies a large family of such changes. Let a network with $L \geq 1$ hidden layers of widths $d_1, \dots, d_L$ map an input $x = a_0 \in \mathbb{R}^{d_0}$ through

$$z_l = W_l a_{l-1} + b_l, \qquad a_l = \mathrm{ReLU}(z_l) \qquad (l = 1, \dots, L)$$

to the logits $f_\theta(x) = W_{L+1} a_L + b_{L+1} \in \mathbb{R}^K$, $K \geq 2$, with parameters $\theta = (W_1, b_1, \dots, W_{L+1}, b_{L+1}) \in \mathbb{R}^N$, and let $\widehat{\mathcal{L}}(\theta)$ be the mean over a training set $\mathcal{T}$ of the cross-entropy $\ell(f_\theta(x), y) = \log \sum_k e^{f_\theta(x)_k} - f_\theta(x)_y$. For positive numbers $\alpha_1, \dots, \alpha_{L+1}$ with product 1, put $\beta_l = \alpha_1 \cdots \alpha_l$, and let T_α multiply W_l by α_l and b_l by β_l for every l; the output bias is unchanged, since $\beta_{L+1} = 1$. The map T_α is linear, $T_\alpha \theta = R_\alpha \theta$ for a diagonal matrix R_α with positive diagonal; for $L = 1$ and $\alpha = (c, c^{-1})$ it is the rescaling of Dinh et al. (2017).

Theorem 10.8 (Rescaling a ReLU network). *In this setting:*

(i) The rescaled network computes the same function, $f_{T_\alpha \theta} = f_\theta$, for every θ; the training loss, the test loss and the test error are the same at θ and at $T_\alpha \theta$.

(ii) Let U be the set of parameters at which no hidden pre-activation is zero on a training input. Then U is open, $T_\alpha(U) = U$, $\widehat{\mathcal{L}}$ is twice continuously differentiable on U, and for every $\theta \in U$

$$\nabla^2 \widehat{\mathcal{L}}(T_\alpha \theta) = R_\alpha^{-1} \, \nabla^2 \widehat{\mathcal{L}}(\theta) \, R_\alpha^{-1}. \tag{10.2}$$

(iii) Let $\theta \in U$ be a point at which some unit of the last hidden layer is active on some training input, and for $c > 0$ let T_c be T_α with $\alpha = (c, 1, \dots, 1, c^{-1})$. Then the block of $\nabla^2 \widehat{\mathcal{L}}(\theta)$ belonging to the output weights W_{L+1} has a positive largest eigenvalue μ, and $\lambda_{\max}\big(\nabla^2 \widehat{\mathcal{L}}(T_c \theta)\big) \geq c^2 \mu$ for every $c > 0$. In particular $\lambda_{\max}\big(\nabla^2 \widehat{\mathcal{L}}(T_c \theta)\big) \to \infty$ as $c \to \infty$, while $f_{T_c \theta} = f_\theta$.

(iv) If in addition the block of $\nabla^2 \widehat{\mathcal{L}}(\theta)$ belonging to the coordinates of $W_1, b_1, \dots, b_L$ has a positive largest eigenvalue ν, then $\lambda_{\max}\big(\nabla^2 \widehat{\mathcal{L}}(T_c \theta)\big) \geq \max(c^2 \mu, \nu / c^2) \geq \sqrt{\mu \nu}$ for every $c > 0$.

PROOF. The plan is to prove (i) by induction over the layers, (ii) by differentiating the identity of (i) twice, and (iii) and (iv) by evaluating the quadratic form of the rescaled Hessian at vectors supported on one block, using that the loss is a convex function of the logits.

Part (i). Write $z_l(\theta;x)$ and $a_l(\theta;x)$ for the quantities computed from the input x at the parameters θ, and put $\beta_0 = 1$. We show by induction on l that $z_l(T_\alpha\theta;x) = \beta_l z_l(\theta;x)$ and $a_l(T_\alpha\theta;x) = \beta_l a_l(\theta;x)$; for $l = 0$ the second statement says $a_0 = x$. If it holds for $l-1$, then

$$z_l(T_\alpha\theta;x) = \alpha_l W_l\, \beta_{l-1} a_{l-1}(\theta;x) + \beta_l b_l = \beta_l z_l(\theta;x),$$

because $\alpha_l\beta_{l-1} = \beta_l$, and $a_l(T_\alpha\theta;x) = \beta_l a_l(\theta;x)$ because $\mathrm{ReLU}(\beta z) = \beta\,\mathrm{ReLU}(z)$ for $\beta > 0$, coordinatewise. At the output, $\alpha_{L+1}\beta_L = \alpha_1\cdots\alpha_{L+1} = 1$, so

$$f_{T_\alpha\theta}(x) = \alpha_{L+1} W_{L+1}\, \beta_L a_L(\theta;x) + b_{L+1} = f_\theta(x).$$

Every loss and every error computed from the logits is therefore unchanged.

Part (ii). Each pre-activation $z_{l,j}(\theta;x)$ is continuous in θ, so U, the intersection of the finitely many open sets $\{\theta : z_{l,j}(\theta;x) \neq 0\}$ over units and training inputs, is open. By the induction in (i), T_α multiplies every pre-activation by some $\beta_l > 0$ and so maps U into U; its inverse is $T_{\alpha'}$ with $\alpha'_l = 1/\alpha_l$, which maps U into U as well, so $T_\alpha(U) = U$. Fix $\theta \in U$. By continuity every pre-activation keeps its sign on a neighborhood $V \subseteq U$ of θ; on V each rectifier acts on each training input as the identity or as zero, so every logit is a polynomial in the coordinates, and $\widehat{\mathcal{L}}$, which composes these polynomials with the infinitely differentiable cross-entropy, is twice continuously differentiable on V, hence on U. For $\varphi \in V$, part (i) gives $\widehat{\mathcal{L}}(R_\alpha\varphi) = \widehat{\mathcal{L}}(\varphi)$, with $R_\alpha\varphi \in U$. Differentiating twice in φ by the chain rule, with the constant symmetric matrix R_α, gives

$$R_\alpha\, \nabla^2\widehat{\mathcal{L}}(R_\alpha\varphi)\, R_\alpha = \nabla^2\widehat{\mathcal{L}}(\varphi),$$

and at $\varphi = \theta$, multiplying by R_α^{-1} on both sides gives (10.2).

Part (iii). For $\alpha = (c, 1, \dots, 1, c^{-1})$ the matrix R_α multiplies the coordinates of W_{L+1} by $1/c$, so R_α^{-1} multiplies them by c. Let u be a unit eigenvector of the W_{L+1}-block of $\nabla^2\widehat{\mathcal{L}}(\theta)$ for its largest eigenvalue μ, extended by zeros to all coordinates, so that $R_\alpha^{-1}u = cu$. The largest eigenvalue of a symmetric matrix is at least its quadratic form at every unit vector, so by (10.2)

$$\lambda_{\max}\big(\nabla^2\widehat{\mathcal{L}}(T_c\theta)\big) \geq u^\top R_\alpha^{-1}\nabla^2\widehat{\mathcal{L}}(\theta)R_\alpha^{-1}u = c^2\, u^\top\nabla^2\widehat{\mathcal{L}}(\theta)u = c^2\mu.$$

It remains to show that $\mu > 0$. Write $a_L(x) = a_L(\theta;x)$. Changing only W_{L+1}, to $W_{L+1} + t\Delta$ for a matrix $\Delta \in \mathbb{R}^{K\times d_L}$, changes each logit vector to $f_\theta(x) + t\Delta a_L(x)$. The Hessian of the cross-entropy in the logits is $\mathrm{diag}(p) - pp^\top$ with $p = \mathrm{softmax}(f_\theta(x))$, so the second derivative of $\widehat{\mathcal{L}}$ along this path at $t = 0$, the quadratic form of the W_{L+1}-block at Δ, is

$$\frac{1}{|\mathcal{T}|}\sum_{(x,y)\in\mathcal{T}} \big(\Delta a_L(x)\big)^\top\big(\mathrm{diag}(p_x) - p_x p_x^\top\big)\big(\Delta a_L(x)\big), \qquad p_x = \mathrm{softmax}\big(f_\theta(x)\big).$$

The quadratic form of $\operatorname{diag}(p) - pp^\top$ at $\delta \in \mathbb{R}^K$, $\sum_k p_k \delta_k^2 - \left(\sum_k p_k \delta_k\right)^2$, is the variance of the coordinates of δ under the probabilities p: it is nonnegative, and positive unless δ is constant, because every p_k is positive. By hypothesis some training input x° has $a^\circ = a_L(x^\circ) \neq 0$. Take $\delta = e_1 - e_2$, which is not constant because $K \geq 2$, and $\Delta = \delta a^{\circ\top} / \|a^\circ\|^2$, so that $\Delta a^\circ = \delta$. The term of x° is then positive and the other terms are nonnegative, so the W_{L+1}-block has a positive quadratic form at $\Delta \neq 0$, and its largest eigenvalue μ is positive.

Part (iv). The coordinates of $W_1, b_1, \ldots, b_L$ are those that R_α multiplies by c, since $\alpha_1 = c$ and $\beta_l = c$ for $1 \leq l \leq L$, so R_α^{-1} multiplies them by $1/c$. The argument of (iii), with a unit eigenvector of their block, gives $\lambda_{\max}(\nabla^2 \widehat{\mathcal{L}}(T_c\theta)) \geq \nu/c^2$. The larger of two positive numbers is at least their geometric mean, and $c^2\mu \cdot \nu/c^2 = \mu\nu$. ∎

The theorem needs no minimum, since sharpness is measured where training stops, and its hypotheses are mild:[7] part (iii) needs only that the last hidden layer is not silent on the training set, and part (ii) that no pre-activation sits exactly at a kink, which holds for almost every θ. Within this family the top eigenvalue can be made as large as one likes, though not smaller than $\sqrt{\mu\nu}$; Dinh et al. (2017) reach arbitrarily flat minima with other reparametrizations. The top eigenvalue therefore describes a network together with its coordinates. A measure can be protected by perturbing each weight in proportion to its size.

[7] Dinh et al. (2017) state their result at a minimum whose Hessian is not zero; part (iii) replaces this by a condition that can be checked on the data.

Proposition 10.9 (Invariance of adaptive sharpness). *Let R be a diagonal matrix with positive diagonal such that $\widehat{\mathcal{L}}(R\varphi) = \widehat{\mathcal{L}}(\varphi)$ for every φ; examples are the matrices R_α of Theorem 10.8 and the matrix that multiplies the incoming weights and the bias of one hidden unit by $\gamma > 0$ and its outgoing weights by $1/\gamma$. Then $\widehat{\mathcal{L}}(R\theta + \sigma\,(R\theta) \odot \xi) = \widehat{\mathcal{L}}(\theta + \sigma\,\theta \odot \xi)$ for every θ, σ and ξ. In particular $P_\sigma^{\mathrm{ad}}(R\theta) = P_\sigma^{\mathrm{ad}}(\theta)$, and every other summary of the perturbed loss, such as a worst case over a set of perturbations, is invariant as well.*

Proof. For a diagonal matrix R, $(R\theta) \odot \xi = R(\theta \odot \xi)$, so $R\theta + \sigma(R\theta) \odot \xi = R(\theta + \sigma\,\theta \odot \xi)$, and the hypothesis on R gives the first equality, as well as $\widehat{\mathcal{L}}(R\theta) = \widehat{\mathcal{L}}(\theta)$; taking expectations gives the second. The rescaling of one unit preserves the function by the positive homogeneity used in Theorem 10.8(i): it multiplies the unit's output by γ, and the outgoing weights divide it again. ∎

The perturbation sharpness P_σ has no such protection, since a rescaling moves the weights relative to a perturbation of fixed size; Lab 10.2 finds it changing by three orders of magnitude. Invariance is necessary for a measure to describe the function, but not sufficient for it to describe generalization: an invariant measure still depends on the architecture, the data and σ, and Lab 10.1 shows its dependence on width deciding the sign of a correlation. Other invariant notions normalize the curvature by the size of the weights or of the features (Petzka et al. 2021; Tsuzuku et al. 2020);

the matrix $\mathrm{diag}(|\theta|)\nabla^2\widehat{\mathcal{L}}(\theta)\,\mathrm{diag}(|\theta|)$, whose trace is the small-σ limit of P^{ad}_{σ}, is invariant in the same way (Exercise 10.2).

FROM CORRELATION TO INTERVENTION. A correlation in a sweep suggests a cause; an intervention tests it. SAM intervenes on the training procedure, not on sharpness, and to attribute its effect to flatness an experiment needs what Chapter 9 asks of every mechanism: a *manipulation check*, showing that the procedure lowers an invariant measure of sharpness; a *sham* of the same strength, such as perturbations in random directions, which Andriushchenko and Flammarion (2022) found to help little (Chapter 5); a *dose–response* over the perturbation radius; and an account of *rival mediators*, since SAM also changes the norm and the implicit bias of the solution. The evidence of 1997 calls for the same caution, since flat minimum search also pruned.

10.6 In the laboratory: one sweep, three sharpnesses

Both labs train the digits MLP of Chapter 1, with two hidden layers of ReLU units, by plain stochastic gradient descent without momentum, on the 539 training images of labkit's split 0 (with 30% of the digits for training), keeping 1,258 images for testing. Every run stops when its cross-entropy on the whole training set reaches 0.02, the rule of Jiang et al.[8] Every run also has its own seeds. In a first version every configuration reused the seeds 0 and 1, so that the 27 configurations of one width started from the same two networks; the seed then acted as a block shared by many configurations, and the sweep looked as if the configuration did not matter, with a replicate control of Remark 10.7 at $\tau = -0.07 \pm 0.15$ and 8% of the variance of the gap between configurations (run the script with `--shared-seeds`).

[8] Power iteration used at most 163 Hessian–vector products per run and agreed with the Lanczos method of SciPy to a relative $5 \cdot 10^{-6}$ on six runs.

LAB 10.1 · THREE SHARPNESSES IN ONE SWEEP

SCRIPT `10-sharpness-sweep.py`

SETUP. A full factorial sweep of learning rate $\eta \in \{0.03, 0.1, 0.3\}$, batch size $B \in \{8, 32, 128\}$, width $\in \{32, 128, 512\}$ and weight decay $\lambda \in \{0, 10^{-3}, 3 \cdot 10^{-3}\}$: 81 configurations, two runs each with their own seeds (162 runs). For every run, the gap G in percentage points; $\lambda_{\max}$ by power iteration on Hessian–vector products of the full training loss; $P_{0.05}$ and $P^{\mathrm{ad}}_{0.5}$ from 32 Gaussian draws; the parameter norm. One to four minutes.

PREDICT FIRST. Will the pooled τ between $\lambda_{\max}$ and the gap be positive, as the flat-minima claim predicts? Which hyperparameter moves the gap most, and which moves $\lambda_{\max}$ most?

LOOK FOR. The sign of the pooled τ of each measure and where it comes from: the granulated coefficients, τ within strata of equal width, the split (10.1), and the two controls.

WHAT HAPPENED. All 162 runs reached the target loss, after 111 to 2,974 steps and at losses between 0.0120 and 0.0200; their gaps range from 3.5 to 6.7 points. The positive control shows that the gap depends on the configuration:

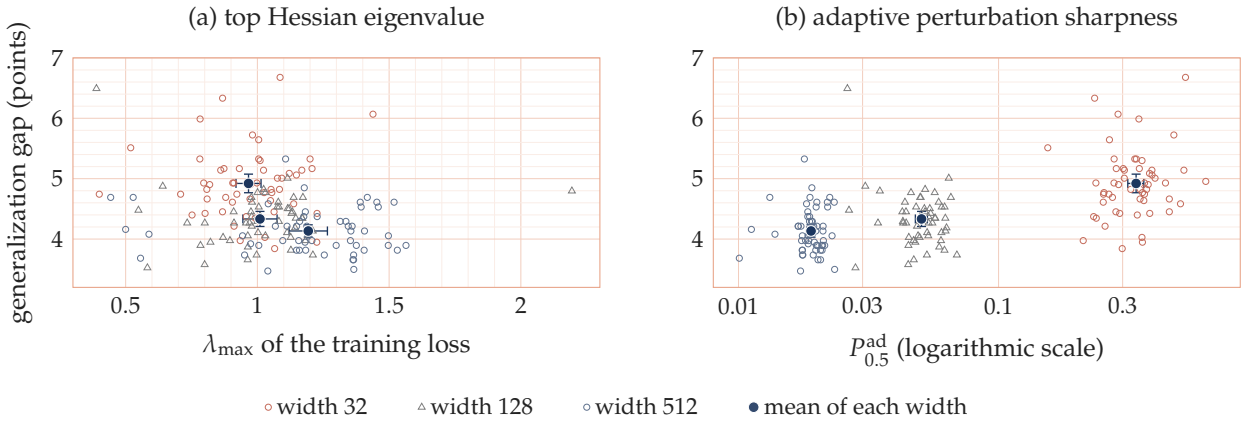


Figure 10.1. The sweep of Lab 10.1: 162 runs of the digits MLP on 539 training images (1,258 test images), 81 configurations of learning rate, batch size, width and weight decay with two runs each, every run with its own seeds and stopped at training cross-entropy 0.02. Hollow marks: single runs, by width; filled marks: the mean of the 54 runs of each width, with 95% t-intervals in both directions. (a) The gap against the top Hessian eigenvalue: pooled $\tau = -0.17$. (b) The gap against the adaptive perturbation sharpness, on a logarithmic scale: pooled $\tau = +0.39$. Within each width neither measure is related to the gap (Table 10.1).

the gaps of a run and of its replicate have $\tau = 0.36 \pm 0.13$ (95% jackknife interval over configurations), and a one-way analysis of variance puts 49% of its variance between configurations (standard deviation 0.41 points between configurations, 0.41 between the runs of one configuration). For normal variables a correlation of 0.49 corresponds to $\tau = 0.33$ (Lemma 5.8), close to the replicate, and by Remark 10.7 the best measure of the configuration would reach $\sqrt{0.49} = 0.70$, or $\tau \approx 0.50$. The negative control gives $\tau = 0.03 \pm 0.11$.

Pooled over the sweep, the three measures of sharpness disagree in sign (Table 10.1). The top eigenvalue correlates negatively with the gap, $\tau = -0.17 \pm 0.13$, so that the sharper networks generalize better; the perturbation sharpness does so more strongly, -0.38 ± 0.10; and the invariant adaptive sharpness correlates positively, 0.39 ± 0.09, as the flat-minima claim predicts. Each sign comes from the width (Figure 10.1). Width moves the gap most, from 4.92 ± 0.15 points at width 32 to 4.33 ± 0.12 at 128 and 4.13 ± 0.10 at 512 (means of 54 runs with 95% t-intervals), and each measure moves with the width in its own direction: λ_{max} rises by almost a quarter, from 0.97 ± 0.05 to 1.19 ± 0.07; the perturbation sharpness, which perturbs more weights in a wider network, rises by a factor of 4.0; and the adaptive one falls by a factor of 18, from 0.34 to 0.019. The split (10.1) makes the attribution exact: a third of the pairs of runs share a width, and for λ_{max} these pairs give $\tau = 0.05$ and the pairs across widths -0.27, for the adaptive sharpness 0.02 and 0.58. And 1/width, which knows nothing about the trained network, reaches a pooled τ of 0.39 ± 0.08, with exactly the 0.58 of the adaptive sharpness across widths.

Within widths the gap still varies reproducibly between configurations, and no measure of sharpness follows it. The replicate keeps $\tau = 0.19 \pm 0.14$

Table 10.1. Kendall's τ between each measure and the generalization gap in Lab 10.1 (162 runs). Pooled: all pairs of runs, with a 95% jackknife interval, which recomputes τ without one configuration (both runs) at a time. Granulated ψ (Definition 10.6): for each hyperparameter, the mean over 54 strata of three runs that differ in it alone, with a 95% t-interval over the strata. Given width: the mean of the three values of τ within widths, with a jackknife interval. The replicate is the gap of the other run of the same configuration (positive control), the random measure a standard normal draw per run (negative control). Dashes: the measure is constant within those strata.

		granulated coefficient ψ along				
measure	pooled τ	learning rate	batch size	width	weight decay	τ given width
top eigenvalue $\lambda_{\max}$	-0.17 ± 0.13	-0.10 ± 0.17	-0.06 ± 0.19	-0.36 ± 0.15	0.13 ± 0.16	0.05 ± 0.13
perturbation $P_{0.05}$	-0.38 ± 0.10	-0.12 ± 0.16	-0.10 ± 0.17	-0.60 ± 0.13	0.10 ± 0.19	-0.04 ± 0.12
adaptive $P^{\mathrm{ad}}_{0.5}$	0.39 ± 0.09	-0.09 ± 0.15	-0.07 ± 0.18	0.60 ± 0.13	0.04 ± 0.18	0.02 ± 0.12
parameter norm $\|\theta\|$	-0.33 ± 0.09	0.21 ± 0.16	0.02 ± 0.18	-0.62 ± 0.13	0.04 ± 0.17	0.11 ± 0.13
1/width	0.39 ± 0.08	—	—	0.60 ± 0.13	—	—
replicate gap	0.36 ± 0.13	0.31 ± 0.17	0.06 ± 0.18	0.46 ± 0.12	0.27 ± 0.14	0.19 ± 0.14
random	0.03 ± 0.11	0.05 ± 0.17	0.16 ± 0.17	-0.09 ± 0.18	-0.03 ± 0.17	0.02 ± 0.11

within widths, and its coefficients along the learning rate (0.31 ± 0.17) and the weight decay (0.27 ± 0.14) show real effects of both; within widths, τ is 0.05 ± 0.13 for $\lambda_{\max}$ and 0.02 ± 0.12 for the adaptive sharpness. The learning rate raises the gap from 4.41 points at $\eta = 0.03$ to 4.60 at $\eta = 0.3$ while $\lambda_{\max}$ falls by 17%, from 1.14 to 0.95 (means of 54 runs), so that the networks that end flatter generalize worse, though $\psi = -0.10 \pm 0.17$ does not separate this from noise; the batch size, which leaves the gap alone, raises $\lambda_{\max}$ by 22%, from 0.94 at $B = 8$ to 1.15 at $B = 128$. Weight decay lowers the gap only at its largest value (4.51, 4.52 and 4.35 points) but lowers $\lambda_{\max}$ (1.14, 1.08, 0.94) and the parameter norm at every step, and the measures reach ψ between 0.04 and 0.13 along it, against 0.27 for the replicate. Comparing the two runs of each configuration, with every hyperparameter fixed, gives $\tau = -0.01 \pm 0.22$ for $\lambda_{\max}$ and 0.23 ± 0.21 for the adaptive sharpness (t-intervals over 81 configurations), against -0.09 ± 0.22 for the random measure; the adaptive sharpness may track some of the seed noise of a run, but among the forty-odd intervals of this section one or two should exclude zero by chance alone (Chapter 3).

LAB 10.2 · RESCALING A TRAINED NETWORK

SCRIPT
`10-sharpness-reparam.py`

SETUP. Five networks of one configuration of Lab 10.1 (width 128, $\eta = 0.1$, $B = 32$, no weight decay, seeds 0–4), rescaled by T_c of Theorem 10.8 for 17 values of c from 0.01 to 100. At each c: the three measures, the training loss, the test error and the largest change of a logit; at $c = 1$, the block eigenvalues μ and ν. Under half a minute.

PREDICT FIRST. By what factor can a rescaling that changes no prediction move $\lambda_{\max}$ between $c = 0.01$ and $c = 100$? Which of the three measures will not move?

LOOK FOR. Slopes of +2 and −2 on logarithmic axes; the bounds $c^2\mu$

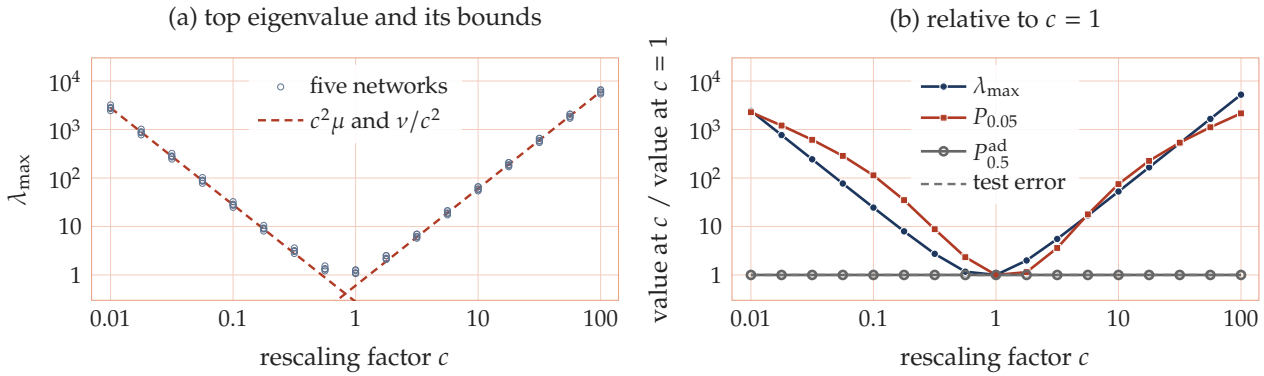


Figure 10.2. Lab 10.2: five networks of one configuration (width 128, learning rate 0.1, batch size 32, seeds 0–4), each rescaled by T_c of Theorem 10.8, which changes no prediction, for 17 values of c. (a) The top Hessian eigenvalue of each network (hollow marks; on this scale the five nearly coincide) and the bounds $c^2\mu$ and ν/c^2 of Theorem 10.8, with μ and ν averaged over the five networks (dashed). (b) Means over the five networks of λ_{max}, the perturbation sharpness, the adaptive perturbation sharpness and the test error, each divided by its value at $c = 1$; the last two stay at 1. Rescaling is deterministic; the spread over the networks is shown in (a).

and ν/c^2; the test error and the adaptive sharpness.

WHAT HAPPENED. Rescaling changes no prediction: the logits move by at most $1.1 \cdot 10^{-5}$, the rounding error of single precision, and each network keeps its test error, between 4.29% and 5.33% across the five. The top eigenvalue moves by a factor of 5,200 (Figure 10.2): averaged over the five networks it is 1.15 at $c = 1$, 6,034 at $c = 100$ and 2,796 at $c = 0.01$. All 85 values exceed the bound $\max(c^2\mu, \nu/c^2)$ of Theorem 10.8, with μ between 0.54 and 0.66 and ν between 0.24 and 0.32 across the networks, and at both ends they exceed it by a relative 0.005% to 0.013%, so that the slopes are +2 and −2. The perturbation sharpness moves by a factor of 2,100 to 2,700 for each network, the adaptive one by a relative $2 \cdot 10^{-7}$, the rounding error, as Proposition 10.9 requires. By choosing c for each network, the five can be put in every order of sharpness; their test errors keep theirs.

WHAT THE MINIATURE REPRODUCES AND WHAT IT DOES NOT. The miniature reproduces the parametrization problem of Dinh et al. (2017) exactly; the finding of Andriushchenko et al. (2023) that sharpness tracks the learning rate rather than generalization; and the lesson of Jiang et al. (2020) and Dziugaite et al. (2020) that the sign of a pooled correlation depends on what the sweep varies and on how the measure is defined. It does not reproduce the batch-size half of Finding 10.2: our larger batches end sharper, but at equal training loss they do not generalize worse (Chapter 7). And our networks stop early, at $\lambda_{max} \approx 1$, far below the cap $2/\eta \geq 6.7$; in networks trained to the edge of stability the learning rate sets λ_{max} more directly.

10.7 Protocol

PROTOCOL 10.1 · READING A CORRELATION IN A SWEEP

- ☐ Sweep the population the claim is about as a factorial design, with at least two runs per configuration, each with its own seeds, all stopped at the same training loss.
- ☐ Fix the measure and its free parameters before looking at the gap; never tune them on the correlation.
- ☐ Rescale one trained network without changing its function, check that the measure does not move, and report how it depends on the architecture.
- ☐ Estimate the reliability of the gap from the replicates, and report a positive control (the replicate gap) and a negative control (a random measure).
- ☐ Report the pooled τ with an interval, the granulated ψ of every hyperparameter, and τ within the strata of each hyperparameter that moves the gap, split as in (10.1); compare with trivial measures built from the hyperparameters and with rivals such as the norm.
- ☐ Judge the measure by its worst stratum as well as by its average.
- ☐ Before calling the measure a cause, intervene on it with everything else fixed, beside a sham of the same strength.

The protocol makes a correlational study of a measure honest; it does not make it causal. Stratifying removes the confounding by the hyperparameters that were varied, not by those held fixed, and a measure that passes every stratum of one sweep may fail in a population that the sweep did not contain: a larger network, another data set, another optimizer. Interventions have their own confound, since a procedure that lowers sharpness changes other things too (Chapter 9). And the protocol is expensive: 162 runs are affordable for our miniature, not for a large model. Then sweep the hyperparameters that matter most at a smaller scale, say so, and treat a correlation found there as a hypothesis about the large model.

Exercises

10.1 CONSOLIDATION. (Rescaling one unit.) Multiply the incoming weights and the bias of one unit of the last hidden layer of the network of Theorem 10.8 by $\gamma > 0$ and its outgoing weights by $1/\gamma$. Show that the function is unchanged and that, if the unit is active on some training input, then $\lambda_{\max}$ tends to infinity as $\gamma \to \infty$. *(Hint: for the limit, repeat part (iii) of the proof with Δ supported on the outgoing weights of the unit.)*

10.2 CONSOLIDATION. (An invariant curvature.) Let $\theta \in U$ have no zero coordinate, and put $D = \operatorname{diag}(|\theta|)$. (a) Show that the matrix $D\nabla^2\widehat{\mathcal{L}}(\theta)D$,

and hence its trace and top eigenvalue, are unchanged when θ is replaced by $R\theta$ for a matrix R as in Proposition 10.9. (b) Assuming that $\widehat{\mathcal{L}}$ has bounded third derivatives, show that $P_\sigma^{\mathrm{ad}}(\theta) = \frac{\sigma^2}{2}\operatorname{tr}\big(D\nabla^2\widehat{\mathcal{L}}(\theta)D\big) + O(\sigma^3)$ whether or not $\nabla\widehat{\mathcal{L}}(\theta) = 0$, as in Andriushchenko et al. (2023).

10.3 CONSOLIDATION. (Simpson in both directions.) (a) For $m \geq 2$ strata of $r \geq 2$ runs that are separated as in Proposition 10.5(iii), with $\tau = -1$ within every stratum, show that the pooled τ is $1 - 2(r-1)/(mr-1)$, and find its limit as $m \to \infty$. (b) Construct nine runs with $\tau = +1$ within every stratum and a negative pooled τ. (c) From Table 10.1 and (10.1), compute the τ of the parameter norm over pairs of runs of different widths, and check it against the sweep script.

10.4 CONSOLIDATION. ★ (The cap of a step size.) (a) For gradient descent with step η on $\frac{1}{2}(\theta-\theta^\star)^\top A(\theta-\theta^\star)$ with A symmetric, show that the iterates converge to $\theta^\star$ from every starting point if and only if every eigenvalue of A lies in $(0, 2/\eta)$, and that they diverge geometrically from every start with a component along the top eigenvector if $\lambda_{\max}(A) > 2/\eta$. (b) For heavy-ball momentum, $\theta_{t+1} = \theta_t - \eta\nabla\widehat{\mathcal{L}}(\theta_t) + \beta(\theta_t - \theta_{t-1})$ with $0 \leq \beta < 1$, show that the cap becomes $2(1+\beta)/\eta$. Which cap applies to stochastic gradient descent with momentum 0.9 in PyTorch? *(Hint: in (b), find when both roots of $z^2 - (1+\beta-\eta\lambda)z + \beta$ lie in the open unit disk.)*

10.5 LAB. (Condition on the learning rate.) Add strata of equal learning rate, and of equal learning rate and batch size, to the analysis of the sweep script. Predict how τ for $\lambda_{\max}$ changes, then check. Why does stratifying by the hyperparameters that move $\lambda_{\max}$ change the correlation so little, while stratifying by width removes it?

10.6 LAB. (Stop by the clock.) Replace the stopping rule of the sweep by a fixed budget of 40 epochs. Predict how the final training losses, $\lambda_{\max}$ and its correlation with the gap will change, then run it, and compare with the reason Jiang et al. give for stopping at a fixed training loss.

10.7 LAB. (Intervene on sharpness.) Implement SAM (Foret et al. 2021) in the training loop of the sweep: at each step move the weights by $\rho\, g/\|g\|$ along the minibatch gradient g, compute the gradient there, and apply it at the unperturbed weights. For one configuration and ten seeds, compare plain training, SAM with $\rho = 0.05$, and a sham that moves the weights by the same distance in a random direction: does SAM lower $P_{0.5}^{\mathrm{ad}}$, does it lower the gap, and does the sham?

10.8 DESIGN. (Does flatness cause generalization?) With a budget of 300 runs of the network of Lab 10.1, design an experiment that tests whether lowering sharpness, with the training procedure otherwise fixed, lowers the gap. Specify two interventions that lower sharpness by different routes, shams, manipulation checks, a dose–response, the runs per arm (Chapter 1), and the result that would refute the claim.

10.9 DESIGN. (Validating a measure.) A team proposes a new measure of generalization and can train 400 networks. Design the sweep, the controls and the analyses that would convince a skeptic who has read Dziugaite et al. (2020). Which factor would you take most care to include?

10.10 CRITIQUE. (A correlation of 0.62.) A paper reports that its new flatness measure has Kendall $\tau = 0.62$ with the generalization gap over 300 networks that differ in width (8 to 512 units), depth and learning rate, one run per configuration, after choosing the perturbation radius from twelve values to maximize τ, and concludes that the measure explains generalization. What can $\tau = 0.62$ mean, and what does the conclusion need? (The description is invented.)

10.11 CRITIQUE. (Flatter and better.) A study trains a network with SAM and with plain stochastic gradient descent, five seeds each, finds that the SAM networks have a lower top Hessian eigenvalue and a test error lower by 0.8 points, and concludes that flat minima cause better generalization. Which explanations does the design leave open, and which experiment of Section 10.5 addresses each? (The description is invented.)

10.12 RESEARCH. (An invariant measure that survives interventions.) Open: a measure, invariant under the function-preserving reparametrizations of a stated family of architectures, whose changes predict the direction of the change in generalization under interventions on width, learning rate, batch size and data size, in the worst case over environments (Dziugaite et al. 2020); or a proof, in the spirit of Wen et al. (2023), that no measure computed from the training loss near the solution can.

Notes and References

The flat-minimum idea has two roots: the description length of the weights, in Hinton and Camp (1993) and Hochreiter and Schmidhuber (1995, 1997), and the PAC-Bayesian bound of McAllester (1999), which Dziugaite and Roy (2017) made nonvacuous for deep networks and Neyshabur et al. (2017) related to sharpness and to norms. Entropy-SGD (Chaudhari et al. 2017) and SAM (Foret et al. 2021) turned flatness into a training objective, and ASAM made SAM's neighborhood adaptive to the size of the weights (Kwon et al. 2021). Scale-invariant notions of flatness (Petzka et al. 2021; Tsuzuku et al. 2020) answer Dinh et al. (2017), and the role of the optimizer in setting the sharpness is the theme of Wu et al. (2018), Cohen et al. (2021) and Kaur et al. (2023).

On the statistics, Simpson (1951) gave the paradox its classical form for contingency tables, Bickel et al. (1975) its most cited example and Pearl (2009) the causal account of when to stratify; the attenuation of a correlation by an unreliable variable is due to Spearman (1904), and the rank correlation to Kendall (1938).

TRANSLATING THE LITERATURE. Keskar et al. (2017) write x for our θ and f for our $\widehat{\mathcal{L}}$; their (C_ε, A)-sharpness is the box measure of Finding 10.2, with A the identity in the full space. Dinh et al. (2017) write θ_1, θ_2 for our W_1, W_2 (they omit biases), T_α for our T_c with $c = \alpha$, and observationally equivalent for function-preserving. Jiang et al. (2020) write θ for a hyperparameter configuration (our h), $\mu(\theta)$ for a measure (our M) and $g(\theta)$ for the generalization gap (our G, as a fraction rather than in points); their ψ_i and Ψ are ours, and their magnitude-aware sharpness perturbs each weight in proportion to its size, as our P_σ^{ad} does, but in the worst case and with a radius calibrated to a fixed increase of the loss. Andriushchenko et al. (2023) write $S_{\text{avg}}^{\rho}(w, |w|)$ for our P_ρ^{ad}, and Kwon et al. (2021) write T_w for our $\text{diag}(|\theta|)$. An environment of Dziugaite et al. (2020) is a pair of configurations that differ in one hyperparameter, a stratum of two in our terms, and their sign-error in an environment is $(1 - \tau)/2$.

References

Andriushchenko, M., F. Croce, M. Müller, M. Hein, and N. Flammarion (2023). "A Modern Look at the Relationship between Sharpness and Generalization". In: *Proceedings of the 40th International Conference on Machine Learning*. Vol. 202. Proceedings of Machine Learning Research, pp. 840–902.

Andriushchenko, M. and N. Flammarion (2022). "Towards Understanding Sharpness-Aware Minimization". In: *Proceedings of the 39th International Conference on Machine Learning*. Vol. 162. Proceedings of Machine Learning Research, pp. 639–668.

Bickel, P. J., E. A. Hammel, and J. W. O'Connell (1975). "Sex Bias in Graduate Admissions: Data from Berkeley". In: *Science* 187.4175, pp. 398–404.

Chaudhari, P. et al. (2017). "Entropy-SGD: Biasing Gradient Descent into Wide Valleys". In: *International Conference on Learning Representations*.

Cohen, J. M., S. Kaur, Y. Li, J. Z. Kolter, and A. Talwalkar (2021). "Gradient Descent on Neural Networks Typically Occurs at the Edge of Stability". In: *International Conference on Learning Representations*.

Dinh, L., R. Pascanu, S. Bengio, and Y. Bengio (2017). "Sharp Minima Can Generalize For Deep Nets". In: *Proceedings of the 34th International Conference on Machine Learning*. Vol. 70. Proceedings of Machine Learning Research, pp. 1019–1028.

Dziugaite, G. K., A. Drouin, B. Neal, N. Rajkumar, E. Caballero, L. Wang, I. Mitliagkas, and D. M. Roy (2020). "In Search of Robust Measures of Generalization". In: *Advances in Neural Information Processing Systems*. Vol. 33.

Dziugaite, G. K. and D. M. Roy (2017). "Computing Nonvacuous Generalization Bounds for Deep (Stochastic) Neural Networks with Many More Parameters than Training Data". In: *Proceedings of the 33rd Conference on Uncertainty in Artificial Intelligence (UAI)*.

Foret, P., A. Kleiner, H. Mobahi, and B. Neyshabur (2021). "Sharpness-Aware Minimization for Efficiently Improving Generalization". In: *International Conference on Learning Representations*.

Hinton, G. E. and D. van Camp (1993). "Keeping the Neural Networks Simple by Minimizing the Description Length of the Weights". In: *Proceedings of the Sixth Annual Conference on Computational Learning Theory (COLT)*, pp. 5–13.

Hochreiter, S. and J. Schmidhuber (1995). "Simplifying Neural Nets by Discovering Flat Minima". In: *Advances in Neural Information Processing Systems 7 (NIPS 1994)*.

Hochreiter, S. and J. Schmidhuber (1997). "Flat Minima". In: *Neural Computation* 9.1, pp. 1–42.

Jiang, Y., B. Neyshabur, H. Mobahi, D. Krishnan, and S. Bengio (2020). "Fantastic Generalization Measures and Where to Find Them". In: *International Conference on Learning Representations*.

Kaur, S., J. Cohen, and Z. C. Lipton (2023). "On the Maximum Hessian Eigenvalue and Generalization". In: *Proceedings on "I Can't Believe It's Not Better! – Understanding Deep Learning Through Empirical Falsification" at NeurIPS 2022 Workshops*. Vol. 187. Proceedings of Machine Learning Research, pp. 51–65.

Kendall, M. G. (1938). "A New Measure of Rank Correlation". In: *Biometrika* 30.1/2, pp. 81–93.

Keskar, N. S., D. Mudigere, J. Nocedal, M. Smelyanskiy, and P. T. P. Tang (2017). "On Large-Batch Training for Deep Learning: Generalization Gap and Sharp Minima". In: *International Conference on Learning Representations*.

Kwon, J., J. Kim, H. Park, and I. K. Choi (2021). "ASAM: Adaptive Sharpness-Aware Minimization for Scale-Invariant Learning of Deep Neural Networks". In: *Proceedings of the 38th International Conference on Machine Learning*. Vol. 139. Proceedings of Machine Learning Research, pp. 5905–5914.

McAllester, D. A. (1999). "PAC-Bayesian Model Averaging". In: *Proceedings of the Twelfth Annual Conference on Computational Learning Theory (COLT)*, pp. 164–170.

Neyshabur, B., S. Bhojanapalli, D. McAllester, and N. Srebro (2017). "Exploring Generalization in Deep Learning". In: *Advances in Neural Information Processing Systems*. Vol. 30, pp. 5947–5956.

Pearl, J. (2009). *Causality: Models, Reasoning, and Inference*. 2nd ed. Cambridge University Press.

Pearlmutter, B. A. (1994). "Fast Exact Multiplication by the Hessian". In: *Neural Computation* 6.1, pp. 147–160.

Petzka, H., M. Kamp, L. Adilova, C. Sminchisescu, and M. Boley (2021). "Relative Flatness and Generalization". In: *Advances in Neural Information Processing Systems*. Vol. 34, pp. 18420–18432.

Simpson, E. H. (1951). "The Interpretation of Interaction in Contingency Tables". In: *Journal of the Royal Statistical Society, Series B (Methodological)* 13.2, pp. 238–241.

Spearman, C. (1904). "The Proof and Measurement of Association between Two Things". In: *The American Journal of Psychology* 15.1, pp. 72–101.

Tsuzuku, Y., I. Sato, and M. Sugiyama (2020). "Normalized Flat Minima: Exploring Scale Invariant Definition of Flat Minima for Neural Networks Using PAC-Bayesian Analysis". In: *Proceedings of the 37th International Conference on Machine Learning*. Vol. 119. Proceedings of Machine Learning Research, pp. 9636–9647.

Wen, K., Z. Li, and T. Ma (2023). "Sharpness Minimization Algorithms Do Not Only Minimize Sharpness To Achieve Better Generalization". In: *Advances in Neural Information Processing Systems*. Vol. 36.

Wu, L., C. Ma, and W. E (2018). "How SGD Selects the Global Minima in Over-parameterized Learning: A Dynamical Stability Perspective". In: *Advances in Neural Information Processing Systems*. Vol. 31.

Zhang, C., S. Bengio, M. Hardt, B. Recht, and O. Vinyals (2017). "Understanding Deep Learning Requires Rethinking Generalization". In: *International Conference on Learning Representations*.

Part IV

Scale

Scaling laws turned the planning of large training runs into an exercise in fitting and extrapolation. A fitted curve is a measurement with a protocol, an uncertainty and a domain of validity, and each of the three has been the source of a famous error.

11 Fitting a Scaling Law

Chinchilla's third approach and its replication

THE CLAIM "$L(N, D) = 1.69 + 406.4/N^{0.34} + 410.7/D^{0.28}$."

Hoffmann et al. (2022), Approach 3: the final training loss of a transformer language model with N parameters trained on D tokens

THE VERDICT Unreplicated: refitting data read off the paper's own figure gives a different law, whose allocation agrees with the paper's other two approaches.

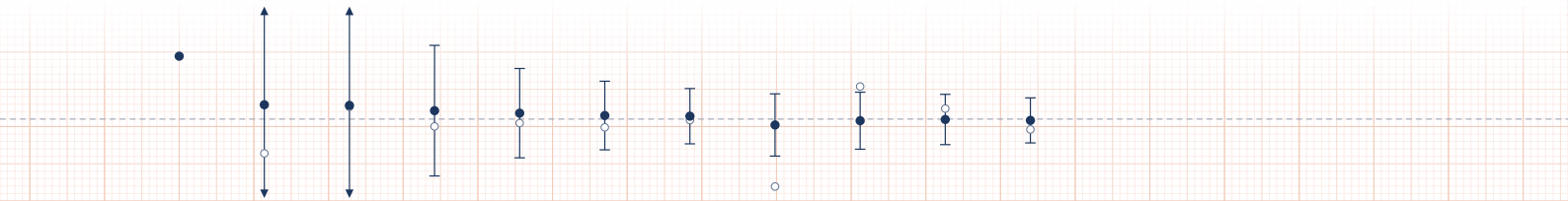
IN THIS CHAPTER. In 2022 Hoffmann and colleagues estimated in three ways how a compute budget should be split between a language model's parameters and its training tokens; the third fitted a five-parameter law to the final losses of their runs and reported its allocation with an interval a thousandth wide, a little off the other two. In 2024 Besiroglu and colleagues read 245 runs off the paper's figure and refitted the law with the paper's own recipe: the exponents changed, the published law fitted the data poorly, intervals that narrow would have needed some 600,000 runs, and the refitted allocation agreed with the other two approaches; by the original authors' account, their optimizer had stopped early. The method treats a fitted law as a measurement: an objective matched to the noise, an optimizer shown to have converged, parameters that a short range of scales cannot separate, and derived quantities with delta-method and bootstrap intervals. In the laboratory, 126 runs of a small network give an allocation exponent of 0.53 by the paper's recipe and 0.52 by isoFLOP parabolas; averaging the loss and stopping early leaves the estimate in place and makes its interval eight times too narrow.

11.1 The claim: three estimates of one allocation

How large a model should be trained with a given amount of compute? A dense network with N parameters costs about $6N$ floating-point operations per training token, two per parameter in the forward pass and four in the backward pass, so a run on D tokens costs $C \approx 6ND$, and every budget C offers a trade between a larger model and more data.[1] In 2020 Kaplan et al. (2020) had answered that most of a growing budget should go to parameters, $N_{\text{opt}} \propto C^{0.73}$, and most of the largest language models of the next two years were trained on about 300 billion tokens, whatever their size (Hoffmann et al. 2022, Table 1). Chapter 12 tells how that answer was overturned; this chapter is about one of the instruments that overturned it.

In March 2022 Jordan Hoffmann and 21 colleagues at DeepMind posted

[1] The count ignores the attention over the context and treats every parameter alike; Chapter 12 shows how such choices of counting move a fitted exponent.



a study, published at NeurIPS that year, of over 400 transformer language models with 70 million to over 16 billion parameters, trained on 5 to 500 billion tokens of their MassiveText corpus (Hoffmann et al. 2022). They estimated the compute-optimal allocation in three ways. In *Approach 1* they trained each of a range of sizes with four schedule lengths, took at each of 1500 FLOP counts the run with the lowest loss, and fitted power laws in C to the sizes and token counts of these winners. In *Approach 2*, the *isoFLOP profile*, they fixed nine budgets from 6×10^{18} to 3×10^{21} FLOPs, trained several sizes at each with the tokens set by the budget, and fitted a parabola to the final loss against the logarithm of the size, whose vertex located the optimal size. In *Approach 3* they fitted a parametric law to the final losses of all these runs.

> FINDING 11.1 (Hoffmann et al. 2022, Section 3.3, Table 2 and Appendix D.2). The law $\hat{L}(N, D) = E + A/N^{\alpha} + B/D^{\beta}$ was fitted by minimizing the sum over runs of a Huber loss with $\delta = 10^{-3}$ between the predicted and the observed log loss, with L-BFGS started from a grid of initializations. The fit gave $E = 1.69$, $A = 406.4$, $B = 410.7$, $\alpha = 0.34$, and $\beta = 0.28$. Minimizing the fitted law under $C = 6ND$ gives $N_{\text{opt}} \propto C^{a}$ with $a = \beta/(\alpha + \beta)$, reported as $a = 0.46$ with 10th and 90th percentiles $(0.454, 0.455)$, estimated by bootstrapping, with 80% of the data sampled 100 times. Approaches 1 and 2 gave $a = 0.50$ $(0.488, 0.502)$ and $a = 0.49$ $(0.462, 0.534)$. For the budget of the authors' Gopher model, 5.76×10^{23} FLOPs, Approach 3 projected an optimal size of 40 billion parameters, Approaches 1 and 2 about 67 and 63 billion.

All three estimates say that parameters and tokens should grow in about equal proportions, against Kaplan et al.'s 0.73. Approach 3 predicts smaller optimal models at large budgets, and the paper offered a reason: runs with less than 10^{21} FLOPs had larger residuals than the rest, the Huber loss treated them as outliers, and the frontier of optimal sizes bends downward at large budgets, so a fit dominated by the larger runs predicts smaller models.[2] The paper put the optimal size for Gopher's budget between 40 and 70 billion parameters and trained Chinchilla at the upper end, 70 billion parameters on 1.4 trillion tokens, 20 tokens per parameter; with the compute of Gopher (280 billion parameters, 300 billion tokens), Chinchilla outperformed it and several larger models on a wide range of downstream evaluations.

[2] The printed estimate $a = 0.46$ lies outside its own printed interval $(0.454, 0.455)$; with the rounded exponents of the finding, $0.28/0.62 = 0.452$ lies outside it on the other side.

The formula of Approach 3 then had a life of its own. Of the three approaches it alone predicts the loss of every pair (N, D), so later work took its constants as given: Sardana et al. (2024) used them to add the cost of inference to the allocation, Muennighoff et al. (2023) built their law for repeated data on the same form fitted by the same Huber objective from a grid of starting points, and Besiroglu et al. (2024) note that its estimates drew the interest of theories of neural scaling. The received reading is the formula of the opening: a measured law of language modeling with exponents known to two digits and an allocation known to a thousandth.

The study deserves credit on four counts. Its main conclusion rested on all three approaches and on a model that was then trained and tested, and

it has held. It described its fitting recipe precisely enough to be followed. It drew its figures as vector graphics, from which the data could later be read. And it reported the disagreement of Approach 3 instead of discarding it. Nothing in 2022 marked the formula as fragile: it was fitted to hundreds of runs, it looked right on the paper's plots, and its interval came from a bootstrap, the standard tool.

11.2 The experiment: one fit, one run per configuration

Approach 3 is an experiment whose measuring instrument is an optimizer.

EXPERIMENTAL DESIGN · HOFFMANN ET AL., APPROACH 3, 2022

QUESTION	How does the final loss depend on parameters N and tokens D, and how should $C \approx 6ND$ be split?
TREATMENT	model size N and number of training tokens D of a decoder-only transformer on MassiveText
LEVELS	over 400 runs, 70M to 16B parameters, 5B to 500B tokens; four schedule lengths per size (Approach 1) and nine budgets of 6×10^{18}–3×10^{21} FLOPs (Approach 2)
TUNED	learning rate and batch size by heuristics per size; cosine schedule decaying tenfold over the run's D
HELD FIXED	data, tokenizer, architecture family, optimizer
RESPONSE	final training loss, smoothed; with no repeated data it estimates the held-out loss
REPLICATES	one run per configuration: no repeated seeds
ANALYSIS	Huber loss ($\delta = 10^{-3}$) on log loss, L-BFGS from a grid, best fit; 10th–90th percentiles of 100 refits on 80% of the data
RELEASED	neither the losses nor the code

The last two fields come from the paper's NeurIPS checklist, which states that the code and data were proprietary, that the paper instead gave instructions for reproducing the scaling methodology, and that the cost of training large models ruled out repeated runs, the clear trends suggesting small noise.

What could this design show? The five numbers are the end point of an optimizer. The paper reports the end point but not the objective there, the gradient, the reason for stopping, or how many starting points reached the same place, so a reader cannot tell a converged fit from one that stopped early. The interval is the spread of 100 more end points; it measures the data only if each refit converges, and whether the 80% were drawn with or without replacement changes what it means (Lemma 11.5). With $\delta = 10^{-3}$ on the log scale, the Huber loss is quadratic only for residuals below a tenth of a percent, while runs differ by more (seed-to-seed differences of up to 4% in loss, by Choshen et al. (2025)), so the fit is in effect a least-absolute-deviation fit (Lemma 11.6). With one run per configuration the noise is not measured at all; it enters only through the misfit. And with neither data nor code released, the only independent check open to others was to read

	E	A_N	A_D	α_N	α_D	a	N_{opt}	$D_{\text{opt}}/N_{\text{opt}}$
printed	1.69	406.4	410.7	0.34	0.28	0.452	32B	93
TEX source	1.6934	406.4	410.7	0.3392	0.2849	0.457	40B	59
refit	1.82	482	2085	0.348	0.366	0.513	72B	18
(standard error)	(0.03)	(125)	(1293)	(0.02)	(0.02)	(0.018)		

Table 11.1. Approach 3 as printed by Hoffmann et al. (2022), with the unrounded values of their TEX source, and as refitted by Besiroglu et al. (2024) to 240 runs read off the paper's Figure 4 (bootstrap standard errors, 4000 resamples). The exponent $a = \alpha_D/(\alpha_N + \alpha_D)$, the optimal size and the tokens per parameter at Gopher's budget, 5.76×10^{23} FLOPs, are computed by Theorem 11.3 in `11-fitting-fits.py`; the refit's own computation gives $a = 0.5126$.

the data back out of a figure, which is what happened.

11.3 The post-mortem: 245 points read off a figure

READING THE DATA. In April 2024 Tamay Besiroglu, Ege Erdil, Matthew Barnett, and Josh You of Epoch AI posted a replication attempt, revised in May (Besiroglu et al. 2024). Figure 4 of Hoffmann et al. plots each run as a point at its training FLOPs and model size, colored by its loss. Converting the arXiv PDF to SVG, they read each point's position and fill color, mapped positions to sizes and FLOPs through the axis labels (the size axis has no tick marks) and colors to losses through the color bar, logarithmic from 2 to 5 in 256 shades, which limits the loss to about 0.01. They recovered 245 runs.[3] Five of them, near 10^{19} FLOPs and with fewer than 0.4 tokens per parameter, had losses up to 70% above runs of similar compute; the main analysis drops them, and an appendix shows that keeping them changes no conclusion.

[3] Besiroglu et al. released the reconstructed data and their code, so that the replication can itself be rerun.

THE REFIT. With the paper's recipe, the 240 runs gave the law in the third row of Table 11.1. The published law fitted the same runs poorly: its residuals were not centered on zero, and the refit attained a lower Huber loss on 90% of the runs. Most of the misfit was rounding. The comments in the paper's TEX source, which arXiv distributes with the paper, give $E = 1.6934$, $\alpha = 0.3392$, and $\beta = 0.2849$, and replacing 0.2849 by the printed 0.28 multiplies the data term by $(10^{11})^{0.0049} \approx 1.13$ at $D = 10^{11}$ tokens. Even unrounded, the published parameters were worse than the refit by 40 to 50 nats of log-likelihood, under a likelihood whose negative logarithm is the Huber loss with a fitted scale; a likelihood-ratio test put the difference at $p \approx 10^{-16}$.

THE INTERVALS. The refit's bootstrap standard error of a is 0.018, so an 80% interval would be about $2 \times 1.28 \times 0.018 \approx 0.05$ wide, fifty times the published width. Standard errors fall like the square root of the number of runs, so a fiftyfold narrower interval needs about 2500 times as many runs: some 600,000, against the 400 to 500 that Besiroglu et al. read into the paper's "over 400".

THE ALLOCATION. The refit puts a at 0.51, with an interval that overlaps those of Approaches 1 and 2, and at Gopher's budget it calls for 72 billion

parameters and 18 tokens per parameter, close to Chinchilla's 70 billion and 20 (Table 11.1). The published law calls for 40 billion parameters and 59 tokens per parameter there, and more tokens per parameter at larger budgets. The disagreement between Approach 3 and the other two, which the paper explained by outliers and curvature, disappears when the fit is redone.

THE CAUSE. In the second version of their preprint, Besiroglu et al. report that one of the lead authors of the original paper had explained the narrow intervals in a thread on Twitter, cited as Borgeaud (2024), which we could not retrieve: the Huber losses had been averaged over the runs instead of summed, and at this scale of the objective the L-BFGS-B optimizer stopped early, in the main fit and in the bootstrap, where the refits barely moved from their starting values. The same account attributes the poor fit of the published parameters to this early stop and to rounding. We report it as Besiroglu et al. give it. As of October 2026, the arXiv record of the original paper has a single version, of March 2022, and the NeurIPS proceedings list no correction.

OTHER CHECKS. Schaeffer et al. (2025), in a preprint, find that the paper's parameter counts admit three readings that differ by up to 15%, none of which changes its main results much. On fitting itself, Li et al. (2025) surveyed 51 papers on scaling laws, found that 23 do not describe how their fits were optimized and 15 do not say how parameters or FLOPs were counted, and showed in their own refits that the initialization, the optimizer, and the loss can each change a fitted law. Choshen et al. (2025), fitting over a thousand laws to 485 published models, found that the five parameters vary across model families along only about three directions, a sign that the form has fewer effective degrees of freedom than parameters (Proposition 11.7).

VERDICT. A refit of the paper's own data, read off its figure and fitted with its own recipe, did not reproduce the published parameters, their interval, or the allocation they imply, and the published law fits those data poorly; by the original authors' account the fit had stopped before converging. What survives is the conclusion the paper drew from all three approaches: parameters and tokens should grow at about the same rate, $a \approx 0.5$, as the refit, Approaches 1 and 2, and Chinchilla itself agree.

UNREPLICATED

11.4 The method: the law, its optimum and their uncertainty

Definition 11.2 (Parametric law and fit). For $\theta = (E, A_N, A_D, \alpha_N, \alpha_D)$ with $E \geq 0$ and $A_N, A_D, \alpha_N, \alpha_D > 0$, the *parametric scaling law* is

$$\mathcal{L}(N, D; \theta) = E + A_N N^{-\alpha_N} + A_D D^{-\alpha_D}.$$

Given runs $i = 1, \ldots, n$ with sizes N_i, token counts D_i, and final

losses L_i, a *fit* minimizes $F(\theta) = \sum_i \rho\big(r_i(\theta)\big)$, where the residual is $r_i(\theta) = \ln \mathcal{L}(N_i, D_i; \theta) - \ln L_i$ (log space) or $\mathcal{L}(N_i, D_i; \theta) - L_i$ (loss space), and ρ is the square $\rho(r) = r^2/2$ or *Huber's loss* with threshold $\delta > 0$,

$$\mathrm{Huber}_\delta(r) = \begin{cases} r^2/2, & |r| \le \delta, \\ \delta\big(|r| - \delta/2\big), & |r| > \delta. \end{cases} \tag{11.1}$$

Hoffmann et al. fit in log space with Huber_δ, $\delta = 10^{-3}$, writing the law as a log-sum-exp, $\ln \mathcal{L} = \mathrm{LSE}(\ln A_N - \alpha_N \ln N,\ \ln A_D - \alpha_D \ln D,\ \ln E)$, which keeps the coefficients positive. The quantity the study was after is the allocation the law implies.

Theorem 11.3 (Compute-optimal allocation). *Let $A_N, A_D, \alpha_N, \alpha_D > 0$, $E \in \mathbb{R}$, and $C > 0$, and put*

$$a = \frac{\alpha_D}{\alpha_N + \alpha_D}, \qquad b = \frac{\alpha_N}{\alpha_N + \alpha_D}, \qquad G = \left(\frac{\alpha_N A_N}{\alpha_D A_D}\right)^{1/(\alpha_N + \alpha_D)}.$$

Among the pairs $N, D > 0$ with $6ND = C$, the law $\mathcal{L}(N, D) = E + A_N N^{-\alpha_N} + A_D D^{-\alpha_D}$ has exactly one minimizer,

$$N_{\mathrm{opt}} = G\left(\frac{C}{6}\right)^{a}, \qquad D_{\mathrm{opt}} = G^{-1}\left(\frac{C}{6}\right)^{b}. \tag{11.2}$$

At the minimizer the two reducible terms balance as $\alpha_N A_N N_{\mathrm{opt}}^{-\alpha_N} = \alpha_D A_D D_{\mathrm{opt}}^{-\alpha_D}$, and the minimal loss is

$$\mathcal{L}_{\mathrm{opt}}(C) = E + \frac{\alpha_N + \alpha_D}{\alpha_D} A_N G^{-\alpha_N} \left(\frac{C}{6}\right)^{-\alpha_N a}.$$

Proof. The plan is to remove D with the constraint, to write the remaining function of N in the variable $x = \ln N$, in which it is strictly convex, and to solve its first-order condition. Write $K = C/6$. On the constraint set $D = K/N$, so

$$\mathcal{L} = E + A_N e^{-\alpha_N x} + A_D K^{-\alpha_D} e^{\alpha_D x} = E + f(x), \qquad x = \ln N \in \mathbb{R},$$

and since $N \mapsto \ln N$ maps $(0, \infty)$ onto $\mathbb{R}$ one to one, we minimize f on $\mathbb{R}$. Both terms of f are positive, and

$$f''(x) = \alpha_N^2 A_N e^{-\alpha_N x} + \alpha_D^2 A_D K^{-\alpha_D} e^{\alpha_D x} > 0,$$

so f is strictly convex. The first term tends to infinity as $x \to -\infty$ because $\alpha_N > 0$, and the second as $x \to \infty$ because $\alpha_D > 0$. A strictly convex function that tends to infinity at both ends has exactly one minimizer, the unique zero of its derivative: $f'(x) = 0$, that is,

$$\alpha_N A_N e^{-\alpha_N x} = \alpha_D A_D K^{-\alpha_D} e^{\alpha_D x}. \tag{11.3}$$

Multiplying (11.3) by $e^{\alpha_N x}/(\alpha_D A_D K^{-\alpha_D})$ gives $e^{(\alpha_N + \alpha_D)x} = G^{\alpha_N + \alpha_D} K^{\alpha_D}$, that is, $N_{\mathrm{opt}} = G K^a$. Then $D_{\mathrm{opt}} = K/N_{\mathrm{opt}} = G^{-1} K^{1-a} = G^{-1} K^b$, because $a + b = 1$.

At the minimizer $K^{-\alpha_D} e^{\alpha_D x} = (K/N_{\text{opt}})^{-\alpha_D} = D_{\text{opt}}^{-\alpha_D}$, so (11.3) is the balance $\alpha_N A_N N_{\text{opt}}^{-\alpha_N} = \alpha_D A_D D_{\text{opt}}^{-\alpha_D}$. Replacing the data term by the size term with this balance gives $\mathcal{L}_{\text{opt}}(C) = E + (1 + \alpha_N/\alpha_D) A_N N_{\text{opt}}^{-\alpha_N}$, which is the stated formula because $N_{\text{opt}}^{-\alpha_N} = G^{-\alpha_N} K^{-\alpha_N a}$. ∎

Three consequences are worth stating. The *allocation exponent* $a = 1/(1 + \alpha_N/\alpha_D)$ depends on the exponents only through their ratio, and E plays no part in the allocation. The *tokens per parameter* at the optimum, $D_{\text{opt}}/N_{\text{opt}} = G^{-2}(C/6)^{b-a}$, is constant only when $\alpha_N = \alpha_D$; otherwise it drifts with the budget. And the prediction is sensitive to the digits: with the printed exponents of Finding 11.1, Theorem 11.3 puts the optimal size at Gopher's budget at 32 billion parameters, with the unrounded ones at 40 billion, the figure the paper reports (Table 11.1). A law meant for extrapolation must be reported with enough digits to reproduce its own predictions.

How uncertain is a? The appendix computes the variance of $\hat{a}$ by the delta method from the covariance of the exponents (Section A.5); the ratio form gives the result a more useful shape, and the same computation covers the optimal size at every budget.

Proposition 11.4 (Uncertainty of the allocation). *Let $\hat{\theta}_n$ be estimators of $\theta = (\ln A_N, \ln A_D, \ln E, \alpha_N, \alpha_D)$, with $\alpha_N, \alpha_D > 0$, such that $\sqrt{n}(\hat{\theta}_n - \theta)$ converges in distribution to $\mathcal{N}(0, \Sigma)$, and write $\Sigma_{NN}, \Sigma_{ND}, \Sigma_{DD}$ for the entries of Σ that belong to the two exponents. Let $\hat{a}_n$ and $\hat{N}_{\text{opt}}(C)$ be computed from $\hat{\theta}_n$ by (11.2).*

(i) $\sqrt{n}(\hat{a}_n - a)$ converges in distribution to $\mathcal{N}(0, a^2 b^2 \tau^2)$, where

$$\tau^2 = \frac{\Sigma_{NN}}{\alpha_N^2} - \frac{2\Sigma_{ND}}{\alpha_N \alpha_D} + \frac{\Sigma_{DD}}{\alpha_D^2}$$

is the asymptotic variance of $\sqrt{n}\,(\ln \hat{\alpha}_N - \ln \hat{\alpha}_D)$.

(ii) For each $C > 0$, with $u = \ln(C/6)$, $\sqrt{n}\big(\ln \hat{N}_{\text{opt}}(C) - \ln N_{\text{opt}}(C)\big)$ converges in distribution to $\mathcal{N}(0, s^2(u))$ with $s^2(u) = (g + uh)^\top \Sigma (g + uh)$, where g and h are the gradients of $\theta \mapsto \ln G$ and $\theta \mapsto a$. The variance $s^2(u)$ is a quadratic polynomial in u with leading coefficient $a^2 b^2 \tau^2$.

(iii) For each $C > 0$, the same holds for $\ln(D_{\text{opt}}/N_{\text{opt}})$ with variance $4s^2(u)$.

Proof. The plan is to write each derived quantity as a differentiable function of θ and to apply the delta method (Theorem A.10) to it. Throughout, $\alpha_N, \alpha_D > 0$ makes every logarithm below differentiable at θ.

Part (i). Put $t = \ln \alpha_N - \ln \alpha_D$ and $\varphi(t) = 1/(1 + e^t)$, so that $a = \alpha_D/(\alpha_N + \alpha_D) = \varphi(t)$. The function φ is differentiable with $\varphi'(t) = -e^t/(1 + e^t)^2 = -\varphi(t)\big(1 - \varphi(t)\big) = -ab$. The map $\theta \mapsto t$ has gradient ∇t with entries $1/\alpha_N$ and $-1/\alpha_D$ in the coordinates of the two exponents and zeros elsewhere, so $\nabla t^\top \Sigma \nabla t = \tau^2$, which by the delta method is the asymptotic variance of $\sqrt{n}(\hat{t}_n - t)$. By the chain rule $h = -ab\,\nabla t$, so $h^\top \Sigma h = a^2 b^2 \tau^2$, and the delta method gives part (i).

Part (ii). By Theorem 11.3, $\ln N_{\text{opt}}(C) = \ln G + au$, with

$$\ln G = \frac{\ln \alpha_N + \ln A_N - \ln \alpha_D - \ln A_D}{\alpha_N + \alpha_D},$$

a differentiable function of θ. For fixed u the gradient of $\theta \mapsto \ln G + au$ is $g + uh$, and the delta method gives the variance $(g+uh)^\top \Sigma (g+uh) = g^\top \Sigma g + 2u\, g^\top \Sigma h + u^2 h^\top \Sigma h$, a quadratic in u whose leading coefficient is $a^2 b^2 \tau^2$ by part (i).

Part (iii). Since $D_{\text{opt}} = (C/6)/N_{\text{opt}}$, we have $\ln(D_{\text{opt}}/N_{\text{opt}}) = u - 2\ln N_{\text{opt}}(C)$, an affine function of $\ln N_{\text{opt}}(C)$ with slope -2 for fixed C, so its asymptotic variance is $4s^2(u)$. ∎

Three readings make the proposition practical. First, $ab \le 1/4$: the allocation exponent is at most a quarter as uncertain as the log-ratio of the exponents. Second, with coefficients of variation κ_N, κ_D of the exponents and correlation ϱ, $\tau^2 = \kappa_N^2 - 2\varrho\kappa_N\kappa_D + \kappa_D^2$, so positively correlated exponents, the usual case since both trade off against E (Proposition 11.7), make $\hat{a}$ more precise than either. With the refit's standard errors of 0.02 for both exponents, $\hat{a}$ has standard error 0.020 for uncorrelated exponents and 0.014 for $\varrho = 0.5$; Besiroglu et al.'s bootstrap gave 0.018. Read as a normal 10–90% interval, the published (0.454, 0.455) means a standard error of about 0.0004, and so a standard deviation of about 0.0016 for $\ln \hat{\alpha}_N - \ln \hat{\alpha}_D$: the ratio of the two exponents known to 0.16%. Such an interval is not impossible in itself, since the log-ratio varies little whenever the exponents move together; the comparison with a bootstrap of comparable data made it implausible. Third, by part (ii) the uncertainty of the optimal size is smallest at one budget, in practice near the middle of those measured, and grows linearly in $|u|$ far from it, as Chapter 13 develops.

A bootstrap estimates Σ, or the distribution of $\hat{a}$ directly, by refitting the law to resampled runs (Theorem A.3). The *bootstrap over runs* draws n runs with replacement; Hoffmann et al.'s scheme drew 80% of them, and the difference is not cosmetic.

Lemma 11.5 (Subsamples). *Let $y_1, \dots, y_n$ be real numbers, $n \ge 2$, with mean $\bar{y}$ and $S^2 = \sum_i (y_i - \bar{y})^2/(n-1)$, and let $\bar{y}_m$ be the mean of m of them, $1 \le m \le n$. If the m are drawn uniformly without replacement, then $\mathbb{E}\bar{y}_m = \bar{y}$ and $\operatorname{Var} \bar{y}_m = (S^2/m)(1 - m/n)$. If they are drawn independently with replacement, then $\mathbb{E}\bar{y}_m = \bar{y}$ and $\operatorname{Var} \bar{y}_m = (n-1)S^2/(nm)$.*

Proof. Without replacement, let Z_i indicate that y_i is drawn. Each Z_i has mean m/n, so $\mathbb{E}\bar{y}_m = \frac{1}{m}\sum_i \frac{m}{n} y_i = \bar{y}$. Subtracting $\bar{y}$ from every y_i changes neither $\operatorname{Var}\bar{y}_m$ nor S^2, so we may assume $\sum_i y_i = 0$. The variance of Z_i is $\frac{m}{n}(1 - \frac{m}{n})$ and, for $i \ne j$, $\operatorname{Cov}(Z_i, Z_j) = \frac{m(m-1)}{n(n-1)} - \frac{m^2}{n^2} = -\frac{m}{n}\big(1 - \frac{m}{n}\big)\frac{1}{n-1}$. Since $\sum_{i \ne j} y_i y_j = (\sum_i y_i)^2 - \sum_i y_i^2 = -\sum_i y_i^2$,

$$\operatorname{Var}\Big(\sum_i Z_i y_i\Big) = \frac{m}{n}\Big(1 - \frac{m}{n}\Big)\Big(\sum_i y_i^2 + \frac{\sum_i y_i^2}{n-1}\Big) = m\Big(1 - \frac{m}{n}\Big)S^2,$$

using $\sum_i y_i^2 = (n-1)S^2$; dividing by m^2 gives the first claim. With replacement, the m draws are independent, each with mean $\bar{y}$ and variance $\frac{1}{n}\sum_i (y_i - \bar{y})^2 = (n-1)S^2/n$, and their mean has $1/m$ of that variance. ∎

The lemma is the finite-population correction of survey sampling (Cochran 1977). Relative to the standard error $S/\sqrt{n}$ of $\bar{y}$, the subsample means spread by $\sqrt{(1-f)/f}$ for a fraction $f = m/n$ drawn without replacement and about $\sqrt{1/f}$ with replacement: 0.50 and 1.12 for $f = 0.8$. A smooth estimator is to first order an average over runs, so the same factors apply to it. Even for converged fits, then, refits on 80% subsamples drawn without replacement show half the standard error, and their 10th–90th percentiles form an 80% interval, not a 95% one; a report should say which scheme it used and rescale it, or use the bootstrap of Definition A.2.

11.5 The method: objective, optimizer and identifiability

LOG SPACE OR LOSS SPACE. A residual in log space is a relative error, $\ln \hat{L} - \ln L \approx (\hat{L} - L)/L$, which is the right scale when the noise of a run is proportional to its loss. Least squares on the loss itself gives the most weight to the largest losses, those of the smallest and often noisiest runs. When the noise varies across runs, as in our laboratory, where the seed-to-seed spread falls 25-fold from the smallest budget to the largest, the textbook covariance $\hat{\sigma}^2(\hat{J}^\top \hat{J})^{-1}$ of Theorem A.11, which assumes one noise level, is too optimistic.

HUBER'S LOSS AND ITS THRESHOLD. Huber (1964) designed the loss (11.1) to estimate a location when a fraction of the data is contaminated: quadratic for small residuals and linear for large ones, so that a few wild runs cannot pull the fit far. The threshold sets what counts as wild, and must be compared with the noise.

Lemma 11.6 (A small threshold is least absolute deviations). *For every $\delta > 0$ and $r \in \mathbb{R}$, $0 \le \delta|r| - \mathrm{Huber}_\delta(r) \le \delta^2/2$. Consequently, if θ_δ minimizes $\sum_i \mathrm{Huber}_\delta(r_i(\theta))$ and θ_1 minimizes $\sum_i |r_i(\theta)|$ over the same set, then $\sum_i |r_i(\theta_\delta)| \le \sum_i |r_i(\theta_1)| + n\delta/2$.*

PROOF. If $|r| > \delta$, then $\delta|r| - \mathrm{Huber}_\delta(r) = \delta^2/2$. If $|r| \le \delta$, then $\delta|r| - \mathrm{Huber}_\delta(r) = \delta|r| - r^2/2$, and $s \mapsto \delta s - s^2/2$ increases on $[0, \delta]$ from 0 to $\delta^2/2$. This proves the inequality. Summing it over the runs, dividing by δ, and using that θ_δ minimizes the Huber objective,

$$\begin{aligned}\sum_i |r_i(\theta_\delta)| &\le \frac{1}{\delta}\sum_i \mathrm{Huber}_\delta\big(r_i(\theta_\delta)\big) + \frac{n\delta}{2} \\ &\le \frac{1}{\delta}\sum_i \mathrm{Huber}_\delta\big(r_i(\theta_1)\big) + \frac{n\delta}{2} \le \sum_i |r_i(\theta_1)| + \frac{n\delta}{2}.\end{aligned}$$ ∎

With $\delta = 10^{-3}$ and residuals of a few percent, the fit is a least-absolute-deviation fit, a median regression, to within $n\delta/2$ in an objective of size

about n times the typical residual. That costs precision. For a location estimated at standard normal noise, the Huber estimate with threshold k has asymptotic efficiency $(2\Phi(k)-1)^2/\mathbb{E}\min(Z^2,k^2)$ relative to the mean (Exercise 11.3): 0.95 at the classical choice $k=1.345$, and $2/\pi\approx 0.64$, that of the median, as $k\to 0$. A threshold of $1.345\hat{\sigma}$, with $\hat{\sigma}$ the noise of the residuals measured from repeated seeds, keeps most of the efficiency of least squares and most of the robustness of the median. A tiny threshold has a second cost: the curvature of the objective sits in the few residuals inside $[-\delta,\delta]$, so neither the textbook covariance nor its sandwich form (Section A.6) is reliable, and the bootstrap is the safe way to an interval.

THE OPTIMIZER AND ITS STOPPING TESTS. The objective is not convex in θ, and a quasi-Newton method such as L-BFGS (Nocedal 1980), in its bounded variant L-BFGS-B (Byrd et al. 1995), finds a local minimum that depends on its start.[4] Hence the grid of starting points, and two questions every fit must answer: did each start converge, and how many reached the best value? In SciPy, which we use, L-BFGS-B stops when the relative decrease $(F_k-F_{k+1})/\max\{|F_k|,|F_{k+1}|,1\}$ falls below ftol $=2.2\times10^{-9}$, when the largest component of the projected gradient falls below gtol $=10^{-5}$, or when it runs out of iterations. Both tests depend on the scale of F: when $|F|<1$ the first compares an absolute decrease with ftol, the second is absolute at every scale, and dividing F by n makes both n times easier to pass. The path does not depend on the scale: in exact arithmetic L-BFGS-B takes the same steps on cF as on F for every $c>0$, because the gradient and the quasi-Newton matrix both scale by c, leaving the search directions unchanged, and the tests that accept a step are homogeneous in F. Averaging the objective over n runs instead of summing it therefore cuts the same path earlier, as the account in Section 11.3 describes; Lab 11.2 checks both halves of this remark numerically. The cure is cheap: tighten the tolerances or scale the objective to order one, and report the termination message, the final objective and gradient, and the number of starts that reached the best value.

[4] L-BFGS approximates the inverse Hessian from the last few steps and changes of gradient, ten pairs by default in SciPy, so its memory and its cost per step stay small.

VARIABLE PROJECTION. For fixed exponents the law is linear in (E,A_N,A_D), so for least squares in loss space the best coefficients solve a nonnegative least-squares problem, and a dense grid over the two exponents finds the global minimum to its resolution, with no starting points to choose. This is *variable projection* (Golub and Pereyra 1973), proposed for scaling laws by Czech et al. (2026) in a preprint; even when the main fit is in log space, it checks the region the optimizer found.

IDENTIFIABILITY. Over a short range of sizes, a curve $E+AN^{-\alpha}$ is nearly straight on log-log axes, and a nearly straight curve does not say where its floor is: a higher floor with a steeper exponent and a lower floor with a shallower one can pass through the same points. The next proposition makes this precise. Write the law near the start N_0 of the range as $\mathcal{L}(N)=E+A_0(N/N_0)^{-\alpha}$, with $A_0=AN_0^{-\alpha}$ the reducible loss at N_0.

Proposition 11.7 (A short range determines two numbers, not three). *Fix $N_0 > 0$, $r > 1$, points $t_1, \dots, t_n \in [0,1]$, and put $N_i = N_0 r^{t_i}$ and $h = \ln r$. For the law $\mathcal{L}(N) = E + A_0(N/N_0)^{-\alpha}$ with $A_0, \alpha > 0$, let J be the $n \times 3$ matrix whose ith row is the gradient of $\mathcal{L}(N_i)$ with respect to (E, A_0, α).*

(i) Up to terms of order h^2, each column of J is an affine function of $x_i = \ln(N_i/N_0)$, and the three affine functions span a space of dimension two.

(ii) For $v = (-1, 1, -\alpha/A_0)$, every entry of Jv lies in $[-\alpha^2 h^2/2, 0]$. Hence, with $w = v/\|v\|$,

$$w^\top J^\top J w \le \frac{n\alpha^4 h^4}{4(2 + \alpha^2/A_0^2)} \le \frac{n\alpha^4 h^4}{8},$$

while the largest eigenvalue of $J^\top J$ is at least n.

(iii) If the observed losses are $\mathcal{L}(N_i) + \varepsilon_i$ with independent errors of mean zero and variance σ^2, and $J^\top J$ is invertible, then the least-squares covariance $\sigma^2 (J^\top J)^{-1}$ of Theorem A.11 gives the component of the estimate along w a variance of at least $8\sigma^2/(n\alpha^4 h^4)$.

Proof. The plan is to compute the three columns as functions of $x = \ln(N/N_0) \in [0, h]$, read off their first-order parts for (i), bound the combination Jv exactly for (ii), and convert the bound into a variance for (iii). With $(N/N_0)^{-\alpha} = e^{-\alpha x}$, the partial derivatives of $\mathcal{L}$ are

$$\frac{\partial \mathcal{L}}{\partial E} = 1, \qquad \frac{\partial \mathcal{L}}{\partial A_0} = e^{-\alpha x}, \qquad \frac{\partial \mathcal{L}}{\partial \alpha} = -A_0 x e^{-\alpha x}.$$

Part (i). Since $e^{-\alpha x} = 1 - \alpha x + O(x^2)$ and $x \le h$, the columns are, up to $O(h^2)$, the values at x_i of the affine functions 1, $1 - \alpha x$, and $-A_0 x$. All three lie in the span of 1 and x, and the first and the third are independent, so the span has dimension two.

Part (ii). The ith entry of Jv is

$$-1 + e^{-\alpha x_i} + \frac{\alpha}{A_0} A_0 x_i e^{-\alpha x_i} = (1 + y)e^{-y} - 1, \qquad y = \alpha x_i \in [0, \alpha h].$$

The function $q(y) = (1 + y)e^{-y} - 1$ has $q(0) = 0$ and $q'(y) = -y e^{-y}$, so $q(y) = -\int_0^y s e^{-s}\, ds$, and since $0 \le s e^{-s} \le s$, we get $-y^2/2 \le q(y) \le 0$. With $y \le \alpha h$ every entry of Jv lies in $[-\alpha^2 h^2/2, 0]$, so $\|Jv\|^2 \le n\alpha^4 h^4/4$. Dividing by $\|v\|^2 = 2 + \alpha^2/A_0^2 \ge 2$ gives the bound on $w^\top J^\top J w$. The largest eigenvalue of $J^\top J$ is at least $e_1^\top J^\top J e_1 = \sum_i (\partial \mathcal{L}/\partial E)^2 = n$, with e_1 the first unit vector.

Part (iii). For a positive definite matrix M and a unit vector w, the Cauchy–Schwarz inequality gives $1 = (w^\top M^{1/2} M^{-1/2} w)^2 \le (w^\top M w)(w^\top M^{-1} w)$. With $M = J^\top J$, the variance $\sigma^2 w^\top (J^\top J)^{-1} w$ is therefore at least $\sigma^2/(w^\top J^\top J w)$, which by part (ii) is at least $8\sigma^2/(n\alpha^4 h^4)$. ∎

The direction v explains the trade. Moving along it lowers E and raises A_0 by the same amount, which keeps the loss $E + A_0$ at the start of the

range, and lowers α by α/A_0 per unit, which keeps the slope $-\alpha A_0$ of the loss against $\ln N$ there. A short range measures a level and a slope; the curvature enters only at second order, so its information is at most of order h^4, and the standard error along v is at least of order $1/(\ln r)^2$: halving the range in log scale quadruples this bound. The bound matters when $\alpha \ln r$ is below about one, a range of $e^{1/\alpha}$: a factor of 19 for $\alpha = 0.34$, of 4 for the exponents near 0.7 of our laboratory. The same holds in log space, where the rows of J are divided by the losses, and in each variable of the two-variable law (Exercise 11.4). Such *sloppy* directions, along which parameters move with almost no effect on the fit, were found in every systems-biology model examined by Gutenkunst et al. (2007), who recommend reporting predictions and their uncertainty rather than individual parameters. For a scaling law, the predictions are the allocation and the loss at given (N, D).[5] The floor E deserves an interval of its own, from a bootstrap or the profile of the objective (Bates and Watts 1988), and a point estimate of E should not be read as the entropy of anything without one.

[5] Sardana et al. (2024) refitted the Chinchilla form with Hoffmann et al.'s recipe to subsets of their 47 runs and obtained floors from 0.17 to 1.45 and size exponents from 0.08 to 0.18, depending on the range of tokens per parameter included.

CROSS-CHECKS. The isoFLOP analysis of Approach 2 shares the runs of a parametric fit but not its assumptions. On the slice $6ND = C$ the law in $x = \ln N$ is $E + A_N e^{-\alpha_N x} + A_D (C/6)^{-\alpha_D} e^{\alpha_D x}$, a valley symmetric about its minimum only when $\alpha_N = \alpha_D$; a parabola estimates the minimum without a global form and without E, but its vertex is biased when the valley is asymmetric or the sizes are not centered on the optimum, even without noise (Czech et al. 2026). The envelope of Approach 1 needs training curves. The decisive cross-check is the simplest: publish N, D, C, seed, and loss for every run, and the fitting code, so that the next reader reruns a script instead of reading a figure.

11.6 In the laboratory: one grid, two analyses

The laboratory uses the teacher task of `labkit.py`: inputs in eight dimensions, targets from a fixed random two-layer network plus noise of variance 0.01. The irreducible loss is known, $E = 0.01$, a luxury no language model offers.[6] A student is a two-layer ReLU network of width w with $N = w^2 + 11w + 1$ parameters. Under the online protocol it is trained for one pass over D fresh examples with Adam, batch size 64, and a cosine schedule over its $D/64$ steps, so that, as in pretraining, the data term of the law is a term for unfinished optimization (Chapter 12).

[6] We measure every run on the same 16,384 noise-free test inputs and add the noise variance, so the reported loss estimates the population loss and cannot fall below 0.01.

LAB 11.1 · A GRID OF ISOFLOP SLICES

SCRIPT
`11-fitting-grid.py`

SETUP. Seven budgets $C = 6ND$ from 6×10^6 to 6×10^9 FLOPs in half-decade steps; on each, the widths whose N lies within a factor 6 of a pilot estimate of the optimum, with $4096 \le D \le 2^{20}$: 42 cells, N from 43 to 17,793 and D from 4096 to 730,304. In each cell the learning rate is the best of $\frac{1}{2}$, 1, and 2 times a rule $\min(0.1, 2/w)(4096/D)^{0.3}$ on one tuning run (seed 0), and three runs (seeds 1–3: initialization, data order, and training sample) are trained with it: 126 runs for the

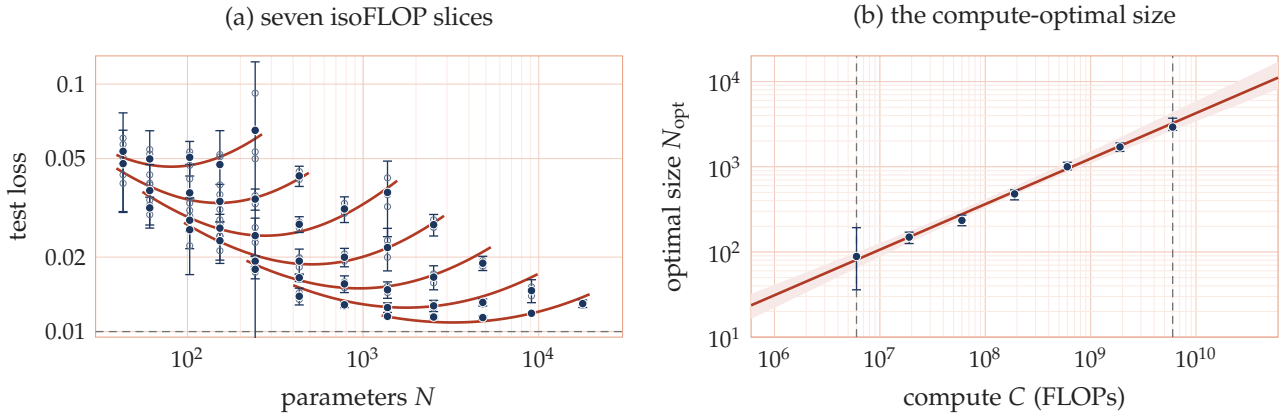


Figure 11.1. Lab 11.1: 126 runs on the teacher task, online protocol, 42 cells of three seeds (seeds 1–3). (a) Test loss against size on the isoFLOP slices $C = 6 \times 10^6$ (top) to 6×10^9 FLOPs (bottom): runs (hollow marks), cell means with 95% t-intervals (filled), and the law fitted by the recipe of Lab 11.2 (curves); dashed: the irreducible loss 0.01. (b) Optimal size against compute: vertices of parabolas fitted to log loss against $\ln N$ on each slice, with 95% bootstrap intervals over runs, and the optimum $G(C/6)^a$ of the fitted law with a 95% bootstrap band; dashed lines mark the measured range.

analysis, 126 for tuning. About three minutes.

PREDICT FIRST. Will the optimal model grow faster or slower than its data as the budget grows? Where will the seeds of one cell disagree most, at small budgets or at large ones?

LOOK FOR. A valley on every slice; the spread of the three seeds against the budget; the law's curves through the cell means; the floor 0.01 below the lowest losses.

WHAT HAPPENED. The 126 runs and the 126 tuning runs took under three minutes, and every slice has a valley (Figure 11.1a). The tuning kept the rule's learning rate in 25 cells, halved it in 13, and doubled it in 4.[7] The seeds of a cell disagree most at small budgets: the median within-cell standard deviation of the log loss falls from 0.16 on the smallest slice, where one run of 64 steps ended at almost twice the loss of its two twins, to 0.006 on the largest, a factor of 25. No run comes within 13% of the floor: the lowest loss is 0.0113.

[7] The three tuning runs of a cell are a tuning budget in the sense of Chapter 4; choosing on seed 0 and measuring on seeds 1–3 keeps the selection out of the measurement, as in Proposition 1.6(ii).

SCRIPT
`11-fitting-fits.py`

LAB 11.2 · ONE LAW, FITTED MANY WAYS

SETUP. The 126 runs of Lab 11.1. The recipe: Huber loss, $\delta = 10^{-3}$, on log loss summed over runs, minimized by SciPy's L-BFGS-B from Hoffmann et al.'s grid of 4500 points ($\ln E$ shifted by -4 and $\ln A_N, \ln A_D$ by -5 to our losses), every start run to convergence (ftol = 10^{-15}, gtol = 10^{-11}). Variants: the objective averaged over runs with SciPy's default tolerances; the recipe stopped after 5 to 50 iterations; $\delta = 1.345\hat{\sigma}$; least squares on log loss and on loss; E fixed

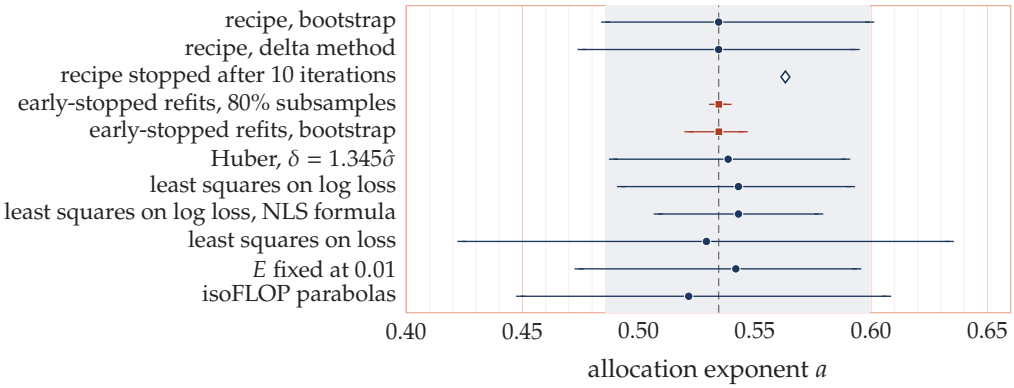


Figure 11.2. Lab 11.2: the allocation exponent a from the 126 runs, fitted many ways. Circles: estimates with 95% intervals (bootstrap over runs unless marked; "NLS formula": $\hat{\sigma}^2(\hat{J}^\top\hat{J})^{-1}$ and the delta method). Vermilion: the averaged objective with SciPy's default tolerances, every refit started at its estimate; the 80% subsamples give 10th–90th percentiles, as in Hoffmann et al.'s Table 2. Diamond: the recipe stopped after ten iterations. Band and dashed line: the recipe's bootstrap interval and estimate.

at 0.01. Intervals: bootstrap over runs (1000 resamples for the recipe, 300 per variant), delta method, and 200 refits on 80% subsamples as in Hoffmann et al. IsoFLOP parabolas with a bootstrap within slices. About two minutes.

PREDICT FIRST. How many of the 4500 starts reach the best fit? Will averaging instead of summing move the estimate, its interval, or both? Will the fitted E be 0.01?

LOOK FOR. Termination messages and iteration counts; where the starts end; the width of every interval for a; the parametric estimate against the isoFLOP estimate; the ridge between E and α_N.

WHAT HAPPENED: THE RECIPE. Run to convergence, the recipe gives $E = 0.0078$, $A_N = 0.42$, $A_D = 30$, $\alpha_N = 0.686$, and $\alpha_D = 0.787$, so $a = 0.534$ with a bootstrap 95% interval of 0.486–0.599; the exponents have intervals 0.513–0.845 and 0.630–0.957, and the coefficients have intervals spanning factors of 5 and 20. Of the 4500 starts, 3677 (82%) reach the best objective; the others stop at local minima with 1.2 to 6.6 times its value and a from 0.05 to 0.94, most of them (591) at $a = 0.88$ (Figure 11.3a): a single start is right four times in five. The bootstrap correlation of the exponents is 0.45, and Proposition 11.4 turns it into a standard error of 0.030 for $\hat{a}$ (interval 0.476–0.593), against 0.040 had they been uncorrelated. The median absolute residual is 0.052, fifty-two times δ, and 95% of the residuals exceed δ: the recipe is a least-absolute-deviation fit (Lemma 11.6).

WHAT HAPPENED: AVERAGING AND STOPPING EARLY. With the objective averaged and SciPy's default tolerances, a start stops after a median of 11 iterations instead of 92, and only 4% of the starts end within 1% of the best value; their end points spread a from 0.12 to 0.84 (5th to 95th percentile). From one start, the averaged and the summed runs agree to

8×10^{-6} in every coordinate for 26 iterations; then the averaged run stops, at a summed objective 6% above the optimum and $a = 0.568$: the same path, cut earlier. Yet the best of the 4500 early-stopped fits lies within 10^{-6} of the optimum, $a = 0.5345$: in our miniature the grid rescued the point estimate. It did not rescue the interval. Started from the estimate, the early-stopped refits of 80% subsamples stop after a median of 6 iterations, and their 10th–90th percentiles span 0.532–0.538, a width of 0.006, against 0.526–0.573, a width of 0.047, when every refit converges (Figure 11.2): the failure of the published interval, reproduced at a factor of eight instead of fifty. A bootstrap of early-stopped refits is also too narrow, 0.522–0.545. Stopping after a fixed number of iterations moves the estimate itself: the best of the grid gives $a = 0.514$ after 5 iterations, 0.563 after 10, 0.549 after 20, and the converged 0.534 after 50, differences of up to one standard error. Converged refits on 80% subsamples spread by 0.58 bootstrap standard errors, or 1.03 when drawn with replacement (Lemma 11.5: 0.50 and 1.12).

WHAT HAPPENED: OTHER OBJECTIVES AND THE ISOFLOP CHECK. The objectives agree within their uncertainty (Figure 11.2): Huber with $\delta = 1.345\hat{\sigma} = 0.12$ gives $a = 0.539$ (0.489–0.589), least squares on log loss 0.543 (0.493–0.591), and least squares on loss 0.529 (0.424–0.634), the widest because the noisiest runs have the largest losses; variable projection on a grid of step 0.01 confirms the last minimum. The textbook interval for least squares on log loss, 0.508–0.577, is 30% narrower than its bootstrap interval, because it assumes one noise level for runs whose spread varies 25-fold. The isoFLOP parabolas give $a = 0.522$ (0.449–0.607, bootstrap within slices), 0.013 from the recipe; parabolas fitted to the recipe's own law at the same sizes put their vertices 0.4% to 2.8% below the law's optima, a bias too small to matter here. In this miniature, as in Hoffmann et al.'s data once refitted, the third approach agrees with the second when its optimizer has converged, and every honestly computed interval in Figure 11.2 contains 1/2: parameters and data grow at about the same rate.

WHAT HAPPENED: THE FLOOR. The fitted floor is 0.0078, with a bootstrap interval of 0.0050–0.0091 that excludes the true 0.01; the profile of least squares on log loss gives 0.0064–0.0086 (twice the log-likelihood within 3.84 of its maximum) and a deviance of 21 at 0.01. Fitted over three decades of compute, the law is not the mechanism of the teacher task, and its floor absorbs the misfit. Fixing E at the truth raises both exponents, to 0.85 and 1.00, and leaves $a = 0.542$; along the profile, as E goes from 0.006 to 0.0104, α_N climbs from 0.60 to 0.85 while a moves only from 0.548 to 0.534, and across the bootstrap $\ln \hat{E}$ and $\hat{\alpha}_N$ have correlation 0.81 (Figure 11.3b). This is Proposition 11.7 at work: the parameters slide along a ridge, and the allocation, a ratio of exponents that slide together, barely moves. Finally, the bootstrap standard deviation of $\ln \hat{N}_{\text{opt}}(C)$ is 0.06 in the middle of the measured range, 0.11 to 0.13 at its ends and 0.26 two decades beyond, the quadratic of Proposition 11.4(ii).

The miniature reproduced the mechanism of the failure and half of its effect: the early stop narrowed the interval eightfold, while the grid of starting points, in our runs, still found the right estimate.

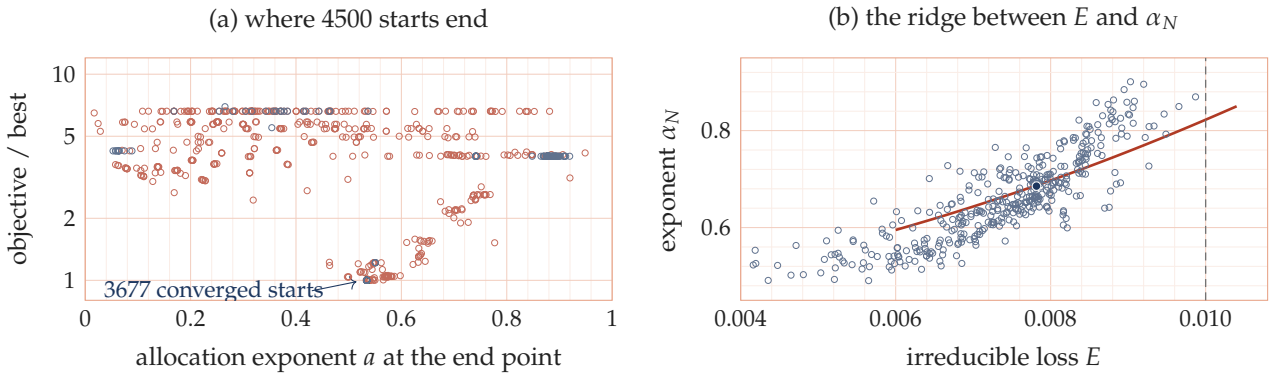


Figure 11.3. Lab 11.2. (a) End points of 500 of the 4500 starts (drawn at random): summed objective divided by its best value, against a, for the recipe run to convergence (ink; the 3677 starts at the optimum coincide) and for the averaged objective with SciPy's default tolerances (vermilion). (b) The recipe's estimate (filled) and 400 of its 1000 bootstrap refits (hollow); the curve is $\hat{\alpha}_N$ with E fixed and the rest refitted by least squares on log loss; dashed: the true floor 0.01.

11.7 Protocol

PROTOCOL 11.1 · FITTING AND REPORTING A SCALING LAW

- ☐ Design the runs before the fit: isoFLOP slices over at least two decades of compute with sizes on both sides of each optimum, schedules matched to each run (Chapter 12), and two or more seeds in some cells to measure the noise.
- ☐ Fix the objective in advance and report it: log space or loss space, the loss function, a threshold set from the measured noise ($\delta \approx 1.345\hat{\sigma}$), and a sum, not a mean, over runs.
- ☐ Minimize from a grid of starts with tolerances suited to the scale of the objective; report the termination message, the final objective and gradient, and how many starts reached the best value; check the region by variable projection.
- ☐ Plot the residuals against compute, size, tokens, and tokens per parameter, and refit without the smallest budget and without the largest models.
- ☐ Get intervals from a bootstrap over runs, refitting every resample to convergence from several starts; give derived quantities such as a and $N_{\text{opt}}(C)$ both delta-method and percentile intervals, and give poorly identified parameters such as E a bootstrap or profile interval.
- ☐ Cross-check the allocation on the same runs with isoFLOP parabolas and, when training curves exist, with envelopes.
- ☐ Report the exponents to three or four digits with their intervals and the number of runs, and release N, D, C, seed, and loss for

every run together with the fitting code.

- ☐ State the range of N, D, and C that the fit covers, and mark every prediction outside it as an extrapolation (Chapter 13).

The protocol makes a fit reproducible and its uncertainty honest; it does not make the functional form true, as the floor of our laboratory shows, and it does not correct a bias shared by all the runs, such as a schedule that does not match the run or a count of parameters that leaves something out (Chapter 12). The bootstrap treats runs as exchangeable; when the noise varies strongly across scales, resampling within slices or weighting by measured variances is more faithful. Fitting costs seconds; the runs are what cost. When seeds are unaffordable at the largest scale, measure the noise on smaller runs and say so, and report every fit with its objective, its convergence diagnostics, and its data.

Exercises

11.1 CONSOLIDATION. (Other cost models.) (a) Show that if a run costs $C = \kappa ND$ for some $\kappa > 0$, then Theorem 11.3 holds with $C/6$ replaced by C/κ. (b) Suppose a run costs $C = 6ND + cD$, a per-token overhead independent of N. Write the first-order condition for N_{opt} and show that $\ln N_{\mathrm{opt}}/\ln C \to a$ as $C \to \infty$.

11.2 CONSOLIDATION. (Tokens per parameter.) Fitted exponents $\hat{\alpha}_N = 0.35$ and $\hat{\alpha}_D = 0.37$ have standard errors 0.02 each and correlation ϱ. (a) By Proposition 11.4(i), compute the standard error of $\hat{a}$ for $\varrho = 0$, 0.5, and 0.9. (b) Show that $\ln(D_{\mathrm{opt}}/N_{\mathrm{opt}})$ changes with $\ln C$ at the rate $b - a$, and use Proposition 11.4(iii) to explain why its interval is narrowest at one budget.

11.3 CONSOLIDATION. (Huber's efficiency.) (a) Using the asymptotic variance $\mathbb{E}\psi(Z)^2/(\mathbb{E}\psi'(Z))^2$ of an M-estimator of location with score ψ at standard normal Z, show that the Huber estimate with threshold k has efficiency $(2\Phi(k) - 1)^2/\mathbb{E}\min(Z^2, k^2)$ relative to the mean. (b) Show that $\mathbb{E}\min(Z^2, k^2) = 2\Phi(k) - 1 - 2k\varphi(k) + 2k^2(1 - \Phi(k))$, and evaluate the efficiency at $k = 1.345$ and as $k \to 0$. (c) With residuals of standard deviation 0.03 in log loss, what is the efficiency of $\delta = 10^{-3}$?

11.4 CONSOLIDATION. ⋆ (Two variables.) Extend Proposition 11.7 to the law $E + A'_N(N/N_0)^{-\alpha_N} + A'_D(D/D_0)^{-\alpha_D}$ in five parameters, on a design inside the rectangle $[N_0, rN_0] \times [D_0, sD_0]$. Find two directions in the five parameters along which J is small, and bound $w^\top J^\top J w$ for each in terms of $\ln r$ and $\ln s$. Which combinations do a short range of N and a long range of D determine? *(Hint: take one direction of the proposition in (E, A'_N, α_N) and the analogous one in (E, A'_D, α_D); their difference does not involve E.)*

11.5 LAB. (Fewer scales.) Modify `11-fitting-fits.py` to fit only the budgets

up to 6×10^8 FLOPs, then only the widths up to 45. Predict first from Proposition 11.4(ii) and Proposition 11.7 which estimate widens most, a, α_N, or E, and compare the parametric and isoFLOP estimates of a in each case.

11.6 LAB. (Rescuing the early stop.) Fit the averaged objective with SciPy's defaults from 1, 32, and 243 starting points drawn at random from the 4500 of the grid. How many starts does it take for the best end point to lie within 1% of the optimum? Then tighten ftol and gtol and observe that the averaged objective converges like the summed one.

11.7 LAB. (Weights.) Fit least squares on log loss with each run weighted by the inverse variance of its cell (with a floor), and compare the estimate of a, its bootstrap interval, and its textbook interval with those of Figure 11.2. Which interval changes, and why?

11.8 DESIGN. (The cheapest grid.) A pilot like Lab 11.1 gives the noise of the log loss at each budget. Using Proposition 11.4 and a bootstrap of the pilot, design the cheapest grid, in total FLOPs, that pins down a to ± 0.05 (95%): how many budgets, how many sizes per slice, how many seeds, and where on each slice. Which choice buys precision most cheaply, a wider range of compute or more seeds?

11.9 DESIGN. (A study of fifty runs.) A team can afford fifty runs to decide whether a new architecture changes the allocation exponent of a baseline, $a \approx 0.5$. Design the comparison: the two grids, the shared seeds, the fit, the difference of the two estimates of a with an interval from a paired bootstrap, and the smallest difference the design can detect.

11.10 CRITIQUE. (Exponents without data.) A paper reports $\mathcal{L}(N, D) = 1.9 + 310/N^{0.31} + 520/D^{0.29}$ for its model family, fitted to "about 200 runs" with "standard methods", with no intervals, no data, and no code, and concludes that its models should be trained on 40 tokens per parameter. List what a reader cannot check, which conclusions follow from Theorem 11.3 alone, and the minimum a report must add.

11.11 CRITIQUE. (A fast bootstrap.) To save time, a study refits each of 50 resamples of 80% of its runs, drawn without replacement, starting from the full-data estimate with at most 20 iterations, and reports the 10th and 90th percentiles as its 95% interval. Name each of the three ways in which this interval is too narrow, estimate the factor contributed by each that Lemma 11.5 and Lab 11.2 allow you to estimate, and repair the procedure.

11.12 RESEARCH. (Unknown floors.) The irreducible loss of language is not known, and Proposition 11.7 shows that a finite range of scales determines it poorly. Open: designs or priors (from compression bounds, from the scaling of the largest models, or from held-out estimates of entropy) that pin down E without biasing the exponents,

and a test of the additive form that can tell a misspecified law, like that of our laboratory, from noise. Also open: when the noise of runs varies strongly across scales, which resampling unit, runs, cells, or slices, gives intervals with the right coverage.

Notes and References

Fitting power laws with an asymptote to learning curves is older than deep learning: Cortes et al. (1993) estimated a classifier's asymptotic error from small training sets this way. Hestness et al. (2017) found power laws in training-set size for translation, language modeling, images, and speech; Rosenfeld et al. (2020) proposed the additive form in model and data size, $an^{-\alpha} + bm^{-\beta} + c_\infty$, inside an envelope for the transition from random guessing; Kaplan et al. (2020) fitted a non-additive joint law; and Caballero et al. (2023) proposed smoothly broken power laws for curves that bend. Li et al. (2025) review the fitting practice of the field, with a reproducibility checklist that overlaps Protocol 11.1.

Approach 3 and its replication are Hoffmann et al. (2022) and Besiroglu et al. (2024). Choshen et al. (2025) draw practical guidance from over a thousand fitted laws, among it to fit intermediate checkpoints as well as final losses, and Muennighoff et al. (2023) extend the form to repeated data. How the protocol of the runs shapes the allocation is the subject of Chapter 12; how far a law can be carried beyond its data, of Chapter 13.

On the statistics: Huber (1964) introduced the loss, Bates and Watts (1988) treat nonlinear regression with its curvature and profile intervals, Golub and Pereyra (1973) variable projection, Efron and Tibshirani (1993) the bootstrap, and Cochran (1977) the finite-population correction; Nocedal (1980) and Byrd et al. (1995) describe L-BFGS and L-BFGS-B, and Gutenkunst et al. (2007) sloppiness, of which Proposition 11.7 is a small instance.

TRANSLATING THE LITERATURE. Hoffmann et al. (2022) and Besiroglu et al. (2024) write A, B, α, β, E for our $A_N, A_D, \alpha_N, \alpha_D, E$, and in the log-sum-exp form write a, b, e for $\ln A_N, \ln A_D, \ln E$, while also using a and b, as we do, for the allocation exponents of $N_{\text{opt}} \propto C^a$ and $D_{\text{opt}} \propto C^b$. Kaplan et al. (2020) count non-embedding parameters as N; their joint law $[(N_c/N)^{\alpha_N/\alpha_D} + D_c/D]^{\alpha_D}$ has no floor, and their α_N, α_D are the exponents of the laws $(N_c/N)^{\alpha_N}$ and $(D_c/D)^{\alpha_D}$, not of our additive terms. Rosenfeld et al. (2020) write n for data and m for model size, with α the exponent of data and β that of the model, the reverse of Hoffmann et al.'s α, β. Choshen et al. (2025) write the law as $e^E + e^A/N^\alpha + e^B/D^\beta$, so that their E, A, B are our $\ln E, \ln A_N, \ln A_D$.

References

Bates, D. M. and D. G. Watts (1988). *Nonlinear Regression Analysis and Its Applications*. New York: Wiley.

Besiroglu, T., E. Erdil, M. Barnett, and J. You (2024). *Chinchilla Scaling: A Replication Attempt*. Preprint, arXiv:2404.10102, version 2 of 15 May 2024.

Byrd, R. H., P. Lu, J. Nocedal, and C. Zhu (1995). "A Limited Memory Algorithm for Bound Constrained Optimization". In: *SIAM Journal on Scientific Computing* 16.5, pp. 1190–1208.

Caballero, E., K. Gupta, I. Rish, and D. Krueger (2023). "Broken Neural Scaling Laws". In: *International Conference on Learning Representations (ICLR 2023)*.

Choshen, L., Y. Zhang, and J. Andreas (2025). "A Hitchhiker's Guide to Scaling Law Estimation". In: *Proceedings of the 42nd International Conference on Machine Learning (ICML 2025)*. Vol. 267. Proceedings of Machine Learning Research, pp. 10683–10699.

Cochran, W. G. (1977). *Sampling Techniques*. 3rd ed. New York: Wiley.

Cortes, C., L. D. Jackel, S. A. Solla, V. Vapnik, and J. S. Denker (1993). "Learning Curves: Asymptotic Values and Rate of Convergence". In: *Advances in Neural Information Processing Systems 6 (NIPS 1993)*, pp. 327–334.

Czech, E., Z. Xu, Y. Elmatad, Y. Wang, and W. Held (2026). *Problems with Chinchilla Approach 2: Systematic Biases in IsoFLOP Parabola Fits*. Preprint, arXiv:2603.22339.

Efron, B. and R. J. Tibshirani (1993). *An Introduction to the Bootstrap*. New York: Chapman & Hall.

Golub, G. H. and V. Pereyra (1973). "The Differentiation of Pseudo-Inverses and Nonlinear Least Squares Problems Whose Variables Separate". In: *SIAM Journal on Numerical Analysis* 10.2, pp. 413–432.

Gutenkunst, R. N., J. J. Waterfall, F. P. Casey, K. S. Brown, C. R. Myers, and J. P. Sethna (2007). "Universally Sloppy Parameter Sensitivities in Systems Biology Models". In: *PLoS Computational Biology* 3.10, e189.

Hestness, J. et al. (2017). *Deep Learning Scaling is Predictable, Empirically*. Preprint, arXiv:1712.00409.

Hoffmann, J. et al. (2022). "An Empirical Analysis of Compute-Optimal Large Language Model Training". In: *Advances in Neural Information Processing Systems 35 (NeurIPS 2022)*. Printed as "Training Compute-Optimal Large Language Models", the title of arXiv:2203.15556, whose appendices are cited here.

Huber, P. J. (1964). "Robust Estimation of a Location Parameter". In: *The Annals of Mathematical Statistics* 35.1, pp. 73–101.

Kaplan, J. et al. (2020). *Scaling Laws for Neural Language Models*. Preprint, arXiv: 2001.08361.

Li, M., S. Kudugunta, and L. Zettlemoyer (2025). "(Mis)Fitting Scaling Laws: A Survey of Scaling Law Fitting Techniques in Deep Learning". In: *International Conference on Learning Representations (ICLR 2025)*. Also arXiv:2502.18969.

Muennighoff, N. et al. (2023). "Scaling Data-Constrained Language Models". In: *Advances in Neural Information Processing Systems 36 (NeurIPS 2023)*.

Nocedal, J. (1980). "Updating Quasi-Newton Matrices with Limited Storage". In: *Mathematics of Computation* 35.151, pp. 773–782.

Rosenfeld, J. S., A. Rosenfeld, Y. Belinkov, and N. Shavit (2020). "A Constructive Prediction of the Generalization Error Across Scales". In: *International Conference on Learning Representations (ICLR 2020)*.

Sardana, N., J. Portes, S. Doubov, and J. Frankle (2024). "Beyond Chinchilla-Optimal: Accounting for Inference in Language Model Scaling Laws". In: *Proceedings of the 41st International Conference on Machine Learning (ICML 2024)*. Vol. 235. Proceedings of Machine Learning Research.

Schaeffer, R., N. Levi, A. Kirsch, T. Guenais, B. Miranda, E. Obbad, and S. Koyejo (2025). *Evaluating the Robustness of Chinchilla Compute-Optimal Scaling*. Preprint, arXiv:2509.23963.

The Protocol Behind the Exponent

12

Kaplan, Chinchilla and the learning-rate schedule

THE CLAIM *"Model size should grow much faster than data: $N_{\text{opt}} \propto C^{0.73}$."*

Kaplan et al. (2020), Section 6; the allocation that GPT-3 and its successors followed

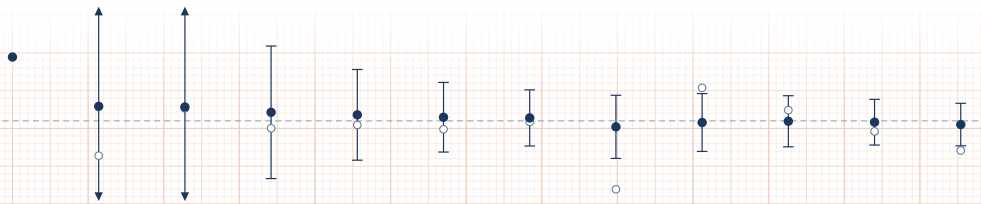
THE VERDICT Reversed: counted in full, with warm-up and hyperparameters set for each scale, parameters and data grow at about the same rate, and the larger, shorter-trained models lose.

IN THIS CHAPTER. Kaplan and colleagues read the compute-optimal size of a language model off the learning curves of models trained once each with one long schedule, and concluded in 2020 that model size should grow almost three times as fast as data; GPT-3 and its successors were trained accordingly. Hoffmann and colleagues trained a separate run for every budget, found parameters and data growing at the same rate, and blamed the unfinished schedule; later reconstructions found the schedule the smallest of four causes, behind what was counted as a parameter, a warm-up too long for small budgets, and hyperparameters held fixed across scales. The method treats a fitted exponent as the reading of an instrument: a power law without a floor measures a local exponent, the allocation exponent is a ratio of two rates of diminishing returns, and a counting convention or a penalty moves it by amounts that can be computed. In a miniature study on the teacher task the exponent is 0.64 with matched schedules, 0.56 read from an unfinished schedule, 0.66 with Kaplan et al.'s fraction of warm-up added, and 0.74 with the input and output layers counted as embeddings.

12.1 The claim: a billion-fold increase in compute

In January 2020 Jared Kaplan, Sam McCandlish and eight colleagues at OpenAI posted a study of how the loss of a language model depends on its scale (Kaplan et al. 2020).[1] They trained decoder-only Transformers on WebText2, about 23 billion tokens of web text, with 768 to 1.5 billion parameters and 22 million to 23 billion tokens, and found that the held-out loss falls as a power law in each of the number of parameters N, the number of training tokens D and the compute C, about $6ND$ operations, whenever the other two do not limit it. Parameters were counted without the embeddings, because the curves of models of different depths then fell onto one line. A model with two layers of width 128 has 393,216 parameters in this count, and about 6.6 million more in its embeddings of

[1] The paper was posted to arXiv once, on 23 January 2020, and has remained a preprint.



50,257 vocabulary tokens and 1,024 positions.

A budget of compute buys a large model trained on few tokens or a smaller one trained on many. To find the *compute-optimal allocation*, the authors took for each budget the model with the lowest loss after the steps that spend it, and fitted a power law to the best size.

> FINDING 12.1 (Kaplan et al. 2020, Section 6, Figure 14 and Table 6). For compute-efficient training the number of non-embedding parameters should grow as $N_{\text{opt}} \propto C^{0.73}$ and the number of training tokens as $D_{\text{opt}} \propto C^{0.27}$, where C is the non-embedding compute, adjusted to the compute that training below the critical batch size would need; the unadjusted trend was $N \propto C^{0.88}$. The best model for a budget is stopped far short of convergence. A fitted model of the loss as a function of size and training steps predicted the exponent 0.71.

A tenfold increase in compute should thus buy a model 5.4 times larger and only 1.9 times more data; in the paper's words, "Big models may be more important than big data." The authors were careful about limits: extrapolated, their laws contradict each other near 10^{12} parameters and 10^{12} tokens, where they expected the predictions to break down, and among their caveats they noted that some hyperparameter with an important effect on scaling might have gone untuned, and that higher learning rates had not been tried for runs that did not proceed to convergence.

The allocation shaped the next models. The eight models of GPT-3 (Brown et al. 2020), from 125 million to 175 billion parameters, were all trained on 300 billion tokens; based on the scaling analysis, their authors wrote, they trained much larger models on many fewer tokens than was typical, and two authors of the scaling study used it to guide the decisions about model and data scale. Jurassic-1 and Gopher, with 178 and 280 billion parameters, were also trained on 300 billion tokens, and Megatron-Turing NLG, with 530 billion, on 270 billion.[2] The received version was a rule of thumb: as the budget grows, grow the model, and let the data follow slowly.

[2] The list of models and tokens is Table 1 of Hoffmann et al. (2022).

Much of the paper has stood: the power laws, the greater sample efficiency of larger models, and the conclusion, shared by the study that reversed the allocation, that the compute-optimal model is not trained to convergence. The method was reasonable too: one run per model, read along its learning curve, gave the whole frontier at the price of one run per size, and the authors corrected it for an artifact they knew of, training above the critical batch size (McCandlish et al. 2018), the subject of Chapter 7.

12.2 The experiment: one long run per model

EXPERIMENTAL DESIGN · KAPLAN ET AL. 2020	
QUESTION	How should a fixed budget of compute be divided between model size and training data?
UNITS	Training runs of decoder-only Transformers on WebText2, with contexts of 1,024 tokens.

TREATMENT	Model size, counted as non-embedding parameters, from 768 to 1.5×10^9; shapes varied as well.
BUDGET	Non-embedding compute $C \approx 6NBS$ after S steps of B tokens; a budget reads every model at the step at which it has spent C.
HELD FIXED	Batch of 2^{19} tokens; 2.5×10^5 steps; linear warm-up over 3,000 steps, then cosine decay to zero at 2.5×10^5 steps; Adam (Adafactor above a billion parameters); a learning rate decreasing slowly in $\log N$.
RESPONSE	Cross-entropy loss on held-out WebText2 text.
REPLICATES	One run per model; the seed-to-seed variation was estimated at about 0.02 in the loss.
ANALYSIS	The best model at each budget; a power law in the budget, after adjusting the compute to training at the critical batch size.

Read as a measurement, the record is sound and reproduces: with these choices Porian et al. (2024) obtained exponents of 0.84 and 0.86 on two datasets, close to the unadjusted 0.88. Read as evidence for an allocation, every entry in the row HELD FIXED is an assumption, and several act on small and large budgets differently. The frontier is made of checkpoints. A budget C reads the model of size N after $C/(6BN)$ steps of a schedule designed for 2.5×10^5, and in Porian et al.'s reconstruction of the setup it was never optimal to train beyond 10 of the schedule's 131 billion tokens, over which a cosine falls by less than 1.5%. Every point of the frontier is thus read from an *unfinished schedule*: a run at nearly constant learning rate, interrupted, not the run a practitioner with that budget would make. The warm-up of 1.57 billion tokens was a large part of the best runs at small budgets,[3] the embeddings left out of both counts held most of the parameters of the small models, the fixed batch gave the small compute-optimal models few steps, and the learning-rate rule had been chosen for runs that converge.

The design could show that, under these choices, the best checkpoint at each budget belongs to a model whose size grows like $C^{0.73}$; it could not separate a property of language models from choices made once for all budgets, nor, with one run per configuration, say how far the exponent would move on a rerun.

[3] Kaplan et al.'s own Figure 14 shows, for many budgets, an optimal number of steps near or below the 3,000 of the warm-up (Porian et al. 2024).

12.3 The post-mortem: three explanations

SEPARATE RUNS FOR EVERY BUDGET. In March 2022 Jordan Hoffmann and 21 colleagues at DeepMind revisited the question with more than 400 models, from 70 million to over 16 billion parameters, trained on 5 to 500 billion tokens (Hoffmann et al. 2022). They counted every parameter and every operation, the embeddings included, let the learning rate fall tenfold along a cosine set to each run's own length, and estimated the allocation in three ways: from the lower envelope of the training curves (Approach 1), from *isoFLOP profiles*, which compare at each of nine budgets the final losses of models whose numbers of tokens exhaust it (Approach 2), and from a

protocol	OpenWebText2	RefinedWeb
Kaplan et al.'s, reproduced	0.864 (0.82, 0.90)	0.835 (0.82, 0.85)
+ last layer counted	0.699 (0.66, 0.72)	0.706 (0.69, 0.72)
+ warm-up of N tokens	0.603 (0.57, 0.63)	0.602 (0.59, 0.62)
+ matched cosine decay	0.574 (0.54, 0.61)	0.571 (0.56, 0.59)
+ B, η, β_2 tuned; constant η	0.518 (0.49, 0.54)	0.497 (0.49, 0.50)

Table 12.1. The allocation exponent a as Kaplan et al.'s protocol is corrected one factor at a time, with 95% intervals, from Table 1 of Porian et al. (2024). Each row keeps the changes above it, except that the last replaces the matched decay by a constant learning rate, with batch size, learning rate and β_2 set for each size.

parametric law fitted to all runs (Approach 3, the subject of Chapter 11).

> FINDING 12.2 (Hoffmann et al. 2022, Table 2, Section 4 and Appendix D.4). The three approaches gave $N_{\text{opt}} \propto C^a$ with $a = 0.50$, 0.49 and 0.46, and $D_{\text{opt}} \propto C^b$ with $b = 0.50$, 0.51 and 0.54: parameters and tokens should grow at about the same rate. At the compute of Gopher, 280 billion parameters trained on 300 billion tokens, the analysis recommended a model about four times smaller trained on four times more data; Chinchilla, with 70 billion parameters and 1.4 trillion tokens, outperformed Gopher on a large range of evaluations. At 10^{21} operations a model of 2.80 billion parameters, the size their analysis recommended, reached a lower loss than one of 4.74 billion, the size Kaplan et al.'s law recommended.

They attributed the difference mainly to the schedule. Kaplan et al. had used one schedule for all models, and a cosine longer than the run by more than 25% degrades the final loss; the intermediate losses of a schedule to 130 billion tokens therefore overestimate those of matched runs, which underestimates training on less data and, Hoffmann et al. wrote, contributes to the conclusion that size should grow faster than data. They added that most of their models had more than 500 million parameters, many of Kaplan et al.'s fewer than 100 million, and that the frontier curves slightly over that range. The schedule became the received explanation, and with it the practice of a separate run, with its own schedule, for every budget.[4]

ONE FACTOR AT A TIME. Two years later Tomer Porian, Mitchell Wortsman, Jenia Jitsev, Ludwig Schmidt and Yair Carmon reproduced Kaplan et al.'s exponent and took its protocol apart one factor at a time (Porian et al. 2024), in more than 900 runs of Transformers with 5 to 901 million parameters on OpenWebText2, close to Kaplan et al.'s WebText2, and RefinedWeb. The reproduction adopted Kaplan et al.'s batch size, warm-up and schedule and, like them, left the final layer, which maps to the vocabulary, out of the counts (Table 12.1).

Three of the four changes moved the exponent by about 0.1 each. Leaving out the last layer misses about 10% of the operations of their largest models and 90% of their smallest, so small models looked cheaper than they were. At small budgets the best number of tokens was close to or below the

[4] Hägele et al. (2024) open their study by recalling this observation of the Chinchilla project: a cosine schedule reaches its best loss only when its length matches the training run.

warm-up; those runs had to use more tokens to escape it, so the optimal number of tokens grew slowly with the budget, which is a large exponent for the model. A fixed batch of 2^{19} tokens leaves the small compute-optimal models a few hundred steps; setting the batch, learning rate and β_2 for each size, the authors found that $\beta_2 = 0.95$ performed poorly at batches of 128 sequences and fewer, where 0.99 or 0.999 was needed. The schedule, the factor that Hoffmann et al. had proposed, moved the exponent by 0.03, and the final agreement with Chinchilla, an exponent within 0.6% of 0.5, was reached with a constant learning rate: careful decay, in their words, "is not essential" for the Chinchilla law, although it lowers the loss substantially, and more so at larger budgets.

COUNTING AT SMALL SCALE. Independently, Tim Pearce and Jinyeop Song explained most of the discrepancy with the published numbers alone (Pearce and Song 2024). They rewrote Hoffmann et al.'s law, fitted in total parameters, in non-embedding parameters and compute, with embeddings growing like the cube root of the other parameters, and simulated a study over Kaplan et al.'s sizes: the simulated frontier had the exponent 0.74 or 0.78, depending on which fit of the Chinchilla law they started from. In small experiments of their own, with 0.8 to 4.6 million parameters, the same runs gave 0.49 in total parameters and 0.74 in non-embedding ones; one run per size at a constant learning rate tuned per size gave 0.49, as did six cosine schedules per size, and one learning rate for all sizes gave 0.58.

THE MECHANISM. The frontier of Finding 12.1 was produced by four choices that act on small and large budgets differently: a count that leaves out a part of the model that is large in small models, a warm-up that is long for small budgets, hyperparameters held fixed across scales, and checkpoints read from schedules far from their end. In the most complete reconstruction the first three moved the exponent by about 0.1 each and the last by 0.03: the explanation adopted first was the weakest of the four, although the observation behind it is right, since a run read before its schedule ends has a higher loss. Two points remain open. The reconstructions differ in detail, Pearce and Song attributing most of the gap to the counting and Porian et al. dividing it among three factors, in setups much smaller than either original study. And an exponent of 0.5 is a property of a protocol and a dataset, not a constant of language models: DeepSeek's study of scaling (Bi et al. 2024), a preprint, which measured size by non-embedding operations per token because both conventional counts are off by up to 50% for small models, found 0.52 on its data and 0.45 to 0.58 across three datasets, larger for data of higher quality.

VERDICT. The claim that compute should go mostly into model size does not survive a protocol that counts the whole model, fits the warm-up and the hyperparameters to each scale, and trains every configuration as it would be trained: parameters and tokens then grow at about the same rate, and the comparison that the claim implied, a large model on few tokens against a smaller one on many at equal

REVERSED

compute, came out the other way when Chinchilla was trained. The exponent 0.73 was a property of the protocol that measured it. The first explanation of the reversal, the unfinished schedule, accounts for the smallest part of it.

12.4 The method: what a fitted exponent measures

A scaling exponent is often reported as if it were a constant of nature. It is better understood as the reading of an instrument: a protocol produces losses for a set of configurations, and an estimator turns them into a number. The method begins by writing the instrument down.

Definition 12.3 (Protocol of a scaling study). A *scaling study* measures the loss of a family of models at configurations (N, D). Its *protocol* fixes (i) the *counting*: which parameters make up N, and how the compute C of a run is computed; (ii) the *source* of each configuration: a run of its own, or a checkpoint of a longer run; (iii) the *schedule*: its warm-up and decay, and how both depend on the run; (iv) the *hyperparameters* at each scale, and how they were chosen; (v) the *range* of N and D; (vi) the *replicates*; and (vii) the *estimator* of the compute-optimal size $N_{\mathrm{opt}}(C)$ and of the *allocation exponent* a in $N_{\mathrm{opt}} \propto C^{a}$, with its interval. The written record of these choices is the study's *protocol card*.

The studies of Kaplan et al. and Hoffmann et al. differ in every item of the card except the replicates, one run per configuration in both. The results below say what the items can do to an exponent.

Local exponents

The loss of every study in this chapter approaches a floor: the entropy of text for language, the noise variance $E = 0.01$ for the teacher task of the laboratory. A power law without a floor cannot follow such a curve; what it measures instead is an average of the curve's local slopes, by a fact about least squares.

Lemma 12.4 (A least-squares slope averages the derivative). *Let $x_1 < \cdots < x_n$, with $n \geq 2$, let φ be continuously differentiable on $[x_1, x_n]$, and let b be the least-squares slope of the points $(x_i, \varphi(x_i))$. Write $\bar{x}$ for the mean of the x_i and $S_{xx} = \sum_i (x_i - \bar{x})^2$. Then*

$$b = \int_{x_1}^{x_n} \varphi'(t)\, K(t)\, \mathrm{d}t, \qquad K(t) = \frac{1}{S_{xx}} \sum_{i:\, x_i > t} (x_i - \bar{x}).$$

The weight K is nonnegative and integrates to one; in particular, b lies between the smallest and the largest value of φ' on $[x_1, x_n]$.

Proof. The plan is to write each $\varphi(x_i)$ as $\varphi(x_1)$ plus an integral of φ' and to exchange the sum and the integral. Put $w_i = (x_i - \bar{x})/S_{xx}$, so

that $b = \sum_i w_i \varphi(x_i)$, $\sum_i w_i = 0$ and $\sum_i w_i x_i = 1$. Since $\varphi(x_i) = \varphi(x_1) + \int_{x_1}^{x_n} \varphi'(t)\mathbf{1}\{t < x_i\}\,\mathrm{d}t$ and the w_i sum to zero,

$$b = \int_{x_1}^{x_n} \varphi'(t) \sum_i w_i \mathbf{1}\{t < x_i\}\,\mathrm{d}t = \int_{x_1}^{x_n} \varphi'(t) K(t)\,\mathrm{d}t.$$

For t in $[x_k, x_{k+1})$, $K(t)$ is the sum of the $n-k$ largest weights. The weights increase with i, so the average of the largest $n-k$ of them is at least the average of all n, which is zero; hence $K \geq 0$. Finally, $\int_{x_1}^{x_n} K(t)\,\mathrm{d}t = \sum_i w_i(x_i - x_1) = \sum_i w_i x_i - x_1 \sum_i w_i = 1$, and a weighted average of values of φ' lies between their extremes. ■

Proposition 12.5 (Local exponents). *Let $\mathcal{L}(N) = E + A_N N^{-\alpha_N}$ with $E \geq 0$, $A_N > 0$ and $\alpha_N > 0$, and call $\lambda(N) = -\,\mathrm{d}\ln\mathcal{L}/\,\mathrm{d}\ln N$ the* local exponent *of $\mathcal{L}$ at N.*

(i) $\lambda(N) = \alpha_N\big(1 - E/\mathcal{L}(N)\big)$. If $E > 0$, then λ decreases strictly in N, from α_N as $N \to 0$ to 0 as $N \to \infty$. For $E = 0$, $\lambda = \alpha_N$ everywhere.

(ii) Let $N_1 < \cdots < N_n$, with $n \geq 2$, and let $\hat{\alpha}$ be minus the least-squares slope of $\ln\mathcal{L}(N_i)$ on $\ln N_i$, the exponent of a power law without floor fitted in log–log coordinates. Then $\lambda(N_n) \leq \hat{\alpha} \leq \lambda(N_1)$; in particular, if $E > 0$, then $\hat{\alpha} \leq \alpha_N\big(1 - E/\mathcal{L}(N_1)\big) < \alpha_N$.

Proof. For part (i), differentiate: $N\mathcal{L}'(N) = -\alpha_N A_N N^{-\alpha_N} = -\alpha_N(\mathcal{L}(N) - E)$, so $\lambda = \alpha_N(\mathcal{L} - E)/\mathcal{L}$. If $E > 0$, then $\mathcal{L}$ decreases strictly from $+\infty$ to E as N runs from 0 to ∞, so $E/\mathcal{L}$ increases strictly from 0 to 1, and λ decreases from α_N to 0. If $E = 0$, then $\lambda = \alpha_N$. For part (ii), apply Lemma 12.4 to $\varphi(x) = \ln\mathcal{L}(e^x)$ at the points $x_i = \ln N_i$: since $\varphi'(x) = -\lambda(e^x)$, the number $\hat{\alpha}$ is a weighted average of λ over $[N_1, N_n]$, and λ is nonincreasing by part (i). ■

A power law fitted without a floor thus reports at best the exponent near the small end of its range. A term that does not depend on N acts as part of the floor: at fixed D the local exponent in N of $E + A_N N^{-\alpha_N} + A_D D^{-\alpha_D}$ is $\alpha_N A_N N^{-\alpha_N}/\mathcal{L}$. For Hoffmann et al.'s fitted law, with $E = 1.69$, $A_N = 406.4$ and $\alpha_N = 0.34$, the local exponent at unlimited data is 0.11 at $N = 10^8$ and 0.06 at 10^9, around Kaplan et al.'s $\alpha_N = 0.076$. The laws were fitted on different data with different counts of N, so the comparison is one of orders of magnitude, but the two exponents measure different things: a local exponent of the loss, and the exponent of the loss above its floor. The same holds for loss against compute, which is part of how Pearce and Song (2024) reconciled Kaplan et al.'s exponent 0.057, fitted without a floor, with Chinchilla's 0.155 for the loss above it (Exercise 12.1).

The local allocation exponent

For the parametric law the compute-optimal size has a closed form, derived with its constants in Chapter 11: $N_{\mathrm{opt}} \propto C^a$ with $a = \alpha_D/(\alpha_N + \alpha_D)$. A protocol changes the law, rarely into another power law, so we need a

local version. For a separable law $\mathcal{L}(N, D) = E + f(N) + h(D)$, write $s_N = -\partial\mathcal{L}/\partial \ln N = -N f'(N)$ and $s_D = -D h'(D)$ for the *marginal values* of size and of data, the loss that one more percent of each buys, and $\kappa_N = -\,\mathrm{d}\ln s_N/\,\mathrm{d}\ln N$ and $\kappa_D = -\,\mathrm{d}\ln s_D/\,\mathrm{d}\ln D$ for the rates at which they fall; for power laws, $\kappa_N = \alpha_N$ and $\kappa_D = \alpha_D$.[5]

[5] A percent more size and a percent more data cost the same compute, since $C \propto ND$; the balance $s_N = s_D$ of Lemma 12.6 is the economist's equal marginal returns per unit of cost.

Lemma 12.6 (The local allocation exponent). *Let $\mathcal{L}(N, D) = E + f(N) + h(D)$, with f and h twice continuously differentiable on $(0, \infty)$ and $s_N, s_D > 0$. Fix $C_0 > 0$, and suppose that N_0 minimizes $N \mapsto \mathcal{L}(N, C_0/(6N))$, with $D_0 = C_0/(6N_0)$, and that $\kappa_N(N_0) + \kappa_D(D_0) > 0$. Then $s_N(N_0) = s_D(D_0)$, and for C near C_0 there is a unique stationary point $N_{\mathrm{opt}}(C)$ near N_0, a strict local minimizer, differentiable in C, with*

$$\frac{\mathrm{d}\ln N_{\mathrm{opt}}}{\mathrm{d}\ln C} = \frac{\kappa_D}{\kappa_N + \kappa_D}, \qquad \frac{\mathrm{d}\ln D_{\mathrm{opt}}}{\mathrm{d}\ln C} = \frac{\kappa_N}{\kappa_N + \kappa_D},$$

the rates evaluated at $N_{\mathrm{opt}}(C)$ and $D_{\mathrm{opt}}(C) = C/(6N_{\mathrm{opt}}(C))$.

Proof. The plan is to write the loss along the isoFLOP line as a function of $u = \ln N$ and to apply the implicit function theorem to its derivative. Put $c = \ln C$ and $v = c - \ln 6 - u$, the logarithm of $D = C/(6N)$, and let $\phi(u, c) = E + f(e^u) + h(e^v)$. Since $\partial v/\partial u = -1$ and $\partial v/\partial c = 1$, the chain rule gives $\partial\phi/\partial u = -s_N(e^u) + s_D(e^v)$, which vanishes at the minimizer $u_0 = \ln N_0$; this is the balance $s_N(N_0) = s_D(D_0)$. Differentiating once more, with $\mathrm{d}s_N/\,\mathrm{d}u = -\kappa_N s_N$ and $\mathrm{d}s_D/\,\mathrm{d}v = -\kappa_D s_D$,

$$\frac{\partial^2\phi}{\partial u^2} = \kappa_N s_N + \kappa_D s_D, \qquad \frac{\partial^2\phi}{\partial u\,\partial c} = -\kappa_D s_D.$$

At $(u_0, \ln C_0)$ the two marginal values are equal, to $s > 0$ say, so $\partial^2\phi/\partial u^2 = s(\kappa_N + \kappa_D) > 0$ by hypothesis. The function $\partial\phi/\partial u$ is continuously differentiable, because f and h are twice so; by the implicit function theorem there is a continuously differentiable $u^\star(c)$ near $\ln C_0$, unique near u_0, with $\partial\phi/\partial u(u^\star(c), c) = 0$, and

$$\frac{\mathrm{d}u^\star}{\mathrm{d}c} = -\frac{\partial^2\phi/\partial u\,\partial c}{\partial^2\phi/\partial u^2} = \frac{\kappa_D s_D}{\kappa_N s_N + \kappa_D s_D} = \frac{\kappa_D}{\kappa_N + \kappa_D};$$

the last equality holds because $s_N = s_D$ at $u^\star(c)$. By continuity the second derivative in u stays positive near $(u_0, \ln C_0)$, so each $u^\star(c)$ is a strict local minimizer. The formula for D_{opt} follows from $\ln D_{\mathrm{opt}} = c - \ln 6 - u^\star(c)$. ∎

For power laws the lemma returns the exponent of Chapter 11 at every budget. The floor E does not appear: unlike the loss exponents, the allocation exponent does not depend on the irreducible loss, only on the balance of two rates of diminishing returns. Whatever in a protocol changes how fast the marginal value of size or of data falls changes the measured exponent; whatever changes the loss without changing those rates does not.

12.5 The method: counting, penalties and cheaper protocols

Counting

Kaplan et al. left the embeddings out of both the size N and the compute $6ND$.[6] A model has a size that sets its loss and a cost per token that sets its compute, and a counting convention can misstate both. Let the counted size $\tilde{N}$ leave out an *overhead* $e(\tilde{N}) \geq 0$, so that the full size is $N = \tilde{N} + e(\tilde{N})$, and write

$$\varepsilon(\tilde{N}) = \frac{\mathrm{d}\ln N}{\mathrm{d}\ln \tilde{N}} = \frac{\tilde{N}\big(1 + e'(\tilde{N})\big)}{N}$$

for the elasticity of the full size with respect to the counted one; for a constant overhead, $\varepsilon = \tilde{N}/N$.

[6] Pearce and Song (2024) recommend counting total parameters and compute, since embeddings learn structure that the model uses.

Proposition 12.7 (Counting). *Let e be twice continuously differentiable on $(0,\infty)$ with $e \geq 0$ and $1 + e' > 0$, let the loss be $\mathcal{L}(N,D) = E + A_N N^{-\alpha_N} + A_D D^{-\alpha_D}$ with positive constants $A_N, A_D, \alpha_N, \alpha_D$, and let $a = \alpha_D/(\alpha_N + \alpha_D)$ be the allocation exponent in full counts.*

(i) Measured against the counted size, the local exponent of the size term is $\alpha_N \varepsilon(\tilde{N})$.

(ii) If compute is counted in full, $C = 6ND$, then the counted size of the compute-optimal model satisfies $\mathrm{d}\ln \tilde{N}_{\mathrm{opt}}/\mathrm{d}\ln C = a/\varepsilon(\tilde{N}_{\mathrm{opt}})$.

(iii) If compute is counted as $\tilde{C} = 6\tilde{N}D$, then at every minimizer of $\mathcal{L}$ at fixed $\tilde{C}$ at which $\tilde{\kappa} = \alpha_N \varepsilon - \mathrm{d}\ln\varepsilon/\mathrm{d}\ln\tilde{N}$ satisfies $\tilde{\kappa} + \alpha_D > 0$, the measured allocation exponent is $\mathrm{d}\ln \tilde{N}_{\mathrm{opt}}/\mathrm{d}\ln\tilde{C} = \alpha_D/(\alpha_D + \tilde{\kappa})$.

(iv) Let $e(\tilde{N}) = \omega \tilde{N}^q$ with $\omega > 0$ and $0 < q < 1$. Then the exponent of part (iii) exceeds a wherever part (iii) applies; as a function of $\tilde{N}$, it tends to a as $\tilde{N} \to \infty$ and to $\alpha_D/(\alpha_D + q\alpha_N)$ as $\tilde{N} \to 0$.

Proof. The plan is to treat the counted size as a change of variable in the size term and to apply Lemma 12.6. Since $1 + e' > 0$, the map $\tilde{N} \mapsto N$ is strictly increasing and continuously differentiable, with $\mathrm{d}\ln N/\mathrm{d}\ln\tilde{N} = \varepsilon > 0$.

Part (i). By the chain rule, $\mathrm{d}\ln(A_N N^{-\alpha_N})/\mathrm{d}\ln\tilde{N} = -\alpha_N\,\mathrm{d}\ln N/\mathrm{d}\ln\tilde{N} = -\alpha_N\varepsilon$.

Part (ii). In full counts the law is separable with $\kappa_N = \alpha_N$ and $\kappa_D = \alpha_D$. Along the isoFLOP line the derivative $-s_N + s_D$ of Lemma 12.6 increases strictly in $\ln N$, from $-\infty$ as $N \to 0$ to $+\infty$ as $N \to \infty$, so the minimizer $N_{\mathrm{opt}}(C)$ is unique, and the lemma gives $\mathrm{d}\ln N_{\mathrm{opt}}/\mathrm{d}\ln C = a$. Its counted size $\tilde{N}_{\mathrm{opt}}(C)$ is, where it exists, the inverse image of $N_{\mathrm{opt}}(C)$ under the increasing map, so $\mathrm{d}\ln\tilde{N}_{\mathrm{opt}}/\mathrm{d}\ln C = (\mathrm{d}\ln\tilde{N}/\mathrm{d}\ln N)\,a = a/\varepsilon$.

Part (iii). In the variables $(\tilde{N}, D)$ the loss is $E + \tilde{f}(\tilde{N}) + A_D D^{-\alpha_D}$ with $\tilde{f}(\tilde{N}) = A_N(\tilde{N} + e(\tilde{N}))^{-\alpha_N}$, twice continuously differentiable because e is, and the constraint $\tilde{C} = 6\tilde{N}D$ has the form of the lemma. The marginal value of counted size is $\tilde{s} = -\tilde{N}\tilde{f}'(\tilde{N}) = \alpha_N A_N N^{-\alpha_N}\varepsilon > 0$, so $\ln\tilde{s} = \mathrm{const} - \alpha_N \ln N + \ln\varepsilon$ and its rate of decrease is $-\mathrm{d}\ln\tilde{s}/\mathrm{d}\ln\tilde{N} = \alpha_N\varepsilon - \mathrm{d}\ln\varepsilon/\mathrm{d}\ln\tilde{N} = \tilde{\kappa}$. The lemma, with $\kappa_D = \alpha_D$, gives the exponent $\alpha_D/(\alpha_D + \tilde{\kappa})$.

counted size $\tilde{N}$	$N/\tilde{N}$	ε	part (ii): $\tilde{N}, 6ND$	part (iii): $\tilde{N}, 6\tilde{N}D$
10^3	767	0.334	1.35	0.71
10^5	36.6	0.352	1.28	0.77
10^6	8.66	0.410	1.10	0.91
10^7	2.65	0.585	0.77	0.93
10^8	1.36	0.825	0.55	0.61
10^9	1.08	0.953	0.47	0.49

Table 12.2. The local allocation exponent of Proposition 12.7 when the embeddings are left out, for Transformers of width 40 times their depth (an assumption) with Kaplan et al.'s 50,257 tokens and 1,024 learned positions, and $\alpha_N = 0.34$, $\alpha_D = 0.28$, so that $a = 0.45$ in full counts. A computation, not an experiment (`12-protocol-counting.py`).

Part (iv). Write $u = \omega \tilde{N}^{q-1}$, which decreases from $+\infty$ to 0 as $\tilde{N}$ runs from 0 to ∞, because $q < 1$. Then $N = \tilde{N}(1+u)$ and

$$\varepsilon = \frac{1+qu}{1+u}, \qquad \frac{\mathrm{d}\ln\varepsilon}{\mathrm{d}\ln\tilde{N}} = \frac{(1-q)^2 u}{(1+u)^2\varepsilon},$$

the second from $\mathrm{d}\varepsilon/\mathrm{d}u = -(1-q)/(1+u)^2$ and $\mathrm{d}u/\mathrm{d}\ln\tilde{N} = -(1-q)u$. Since $q < 1$, $\varepsilon < 1$, and the derivative of $\ln\varepsilon$ is positive; hence $\tilde{\kappa} < \alpha_N$, and where $\tilde{\kappa} + \alpha_D > 0$ the exponent $\alpha_D/(\alpha_D + \tilde{\kappa})$ exceeds $\alpha_D/(\alpha_D + \alpha_N) = a$. As $\tilde{N} \to \infty$, $u \to 0$, so $\varepsilon \to 1$, $\mathrm{d}\ln\varepsilon/\mathrm{d}\ln\tilde{N} \to 0$ and $\tilde{\kappa} \to \alpha_N$. As $\tilde{N} \to 0$, $u \to \infty$, so $\varepsilon \to q$ and $\mathrm{d}\ln\varepsilon/\mathrm{d}\ln\tilde{N} \to 0$, because $u/(1+u)^2 \to 0$; then $\tilde{\kappa} \to q\alpha_N$. ■

For a Transformer whose width d is a fixed multiple r of its depth l, the blocks have $\tilde{N} = 12ld^2 = 12d^3/r$ parameters and the embeddings $(n_{\text{vocab}} + n_{\text{ctx}})d$, so the overhead is $\omega\tilde{N}^{1/3}$ with $\omega = (n_{\text{vocab}} + n_{\text{ctx}})(r/12)^{1/3}$: the case $q = 1/3$, which Pearce and Song (2024) analyzed. With Kaplan et al.'s vocabulary and context and Hoffmann et al.'s $\alpha_N = 0.34$, $\alpha_D = 0.28$, so that $a = 0.45$ in full counts, Table 12.2 lists what the partial conventions report. Over Kaplan et al.'s range of sizes, from 10^3 to 1.5×10^9 counted parameters, the exponent of part (iii) rises from 0.71 to 0.93 near 10^7 and falls to 0.49 at 10^9; a power law fitted across the range averages these local values (Lemma 12.4), and Pearce and Song's simulation of the whole study gave 0.74 to 0.78. The convention alone produces an exponent near the one Kaplan et al. reported, with no change in the models or the data. Counting compute in the same convention as size partly compensates: with the compute counted in full and the size not, part (ii) reports exponents above one at the small end. A constant overhead, $q = 0$, is worse: part (iii) then has no interior minimizer at small sizes, because the overhead's compute is free (Exercise 12.2).

Penalties

A protocol that reads a configuration from a checkpoint, or starts every run with the same warm-up, measures a loss $\mathcal{L}_{\mathrm{P}}$ that differs from the loss $\mathcal{L}$ of the run a practitioner would make with that budget; call $g = \mathcal{L}_{\mathrm{P}} - \mathcal{L}$ the protocol's *penalty*. Whether it moves the exponent depends on its shape, not its size.

Proposition 12.8 (How a penalty moves the exponent). *Let $\mathcal{L}(N, D) = E + A_N N^{-\alpha_N} + A_D D^{-\alpha_D}$ as in Proposition 12.7, and let a protocol measure $\mathcal{L}_{\mathrm{P}} = \mathcal{L} + g(D)$ with g twice continuously differentiable. Write $s_g(D) = -D g'(D)$ for the marginal value that the penalty adds to data, and let a_{P} be the exponent that Lemma 12.6 gives for $\mathcal{L}_{\mathrm{P}}$ at a minimizer where its hypotheses hold and $s_D + s_g > 0$. Then $a_{\mathrm{P}} - a$ has the sign of*

$$-\frac{\mathrm{d}}{\mathrm{d}\ln D}\big(D^{\alpha_D} s_g(D)\big).$$

The measured allocation thus tilts toward larger models where $D^{\alpha_D} s_g$ decreases, toward smaller ones where it increases, and not at all where it is constant; a constant penalty, or one proportional to the data term $A_D D^{-\alpha_D}$, leaves the exponent unchanged. For a penalty $g(N)$ of the model size, with $s_g = -N g'(N)$ and $s_N + s_g > 0$, $a_{\mathrm{P}} - a$ has the sign of $\mathrm{d}(N^{\alpha_N} s_g)/\mathrm{d}\ln N$.

Proof. The plan is to compute the rate of decrease of the penalized marginal value and to note that the exponent of Lemma 12.6 is monotone in it. Under the protocol the size term is unchanged, so $\kappa_N = \alpha_N$, and the marginal value of data is $s_D + s_g$ with $s_D = \alpha_D A_D D^{-\alpha_D}$, whose derivative in $\ln D$ is $-\alpha_D s_D$. Writing s_g' for $\mathrm{d}s_g/\mathrm{d}\ln D$, its rate of decrease is $\kappa_D^{\mathrm{P}} = (\alpha_D s_D - s_g')/(s_D + s_g)$, and therefore

$$\kappa_D^{\mathrm{P}} - \alpha_D = -\frac{s_g' + \alpha_D s_g}{s_D + s_g} = -\frac{D^{-\alpha_D}}{s_D + s_g}\,\frac{\mathrm{d}}{\mathrm{d}\ln D}\big(D^{\alpha_D} s_g\big).$$

By the lemma $a_{\mathrm{P}} = \kappa_D^{\mathrm{P}}/(\alpha_N + \kappa_D^{\mathrm{P}})$, a strictly increasing function of κ_D^{P} where $\alpha_N + \kappa_D^{\mathrm{P}} > 0$, and it equals a when $\kappa_D^{\mathrm{P}} = \alpha_D$; since $s_D + s_g > 0$, the sign of $a_{\mathrm{P}} - a$ is the stated one. A constant g has $s_g = 0$, and $g = \gamma A_D D^{-\alpha_D}$ has $D^{\alpha_D} s_g = \gamma \alpha_D A_D$, constant. For a penalty of the size the same computation gives $\kappa_N^{\mathrm{P}} - \alpha_N = -N^{-\alpha_N}(s_N + s_g)^{-1}\,\mathrm{d}(N^{\alpha_N} s_g)/\mathrm{d}\ln N$, and $a_{\mathrm{P}} = \alpha_D/(\kappa_N^{\mathrm{P}} + \alpha_D)$ decreases in κ_N^{P}. ∎

A protocol can thus raise every loss by a third and leave the exponent alone, while a penalty of one percent concentrated at the smallest budgets moves it. The shapes of real penalties come from a model, not a proof.

Warm-up and unfinished schedules. Model the loss of a run after D examples, under a learning-rate schedule $\eta(\cdot)$ with peak η_0, by two effects: progress, which depends on the area $S = \int_0^D \eta(x)\,\mathrm{d}x$ under the schedule, and a floor of noise proportional to the current rate,

$$\mathcal{L} = E + A_N N^{-\alpha_N} + A_S S^{-\alpha_D} + c\,\eta(D),$$

with constants $A_S, c \geq 0$ that may depend on the model. It is a simplified relative of the laws of Tissue et al. (2025), which combine a power law in the area with a term for annealing, and for SGD on a quadratic it holds in the sense that the stationary excess loss is proportional to the step (Theorem 7.4) while progress is set by the accumulated steps. A run whose cosine schedule decays to zero at D has $\eta(D) = 0$ and $S = \eta_0 D/2$, and the model reduces to the parametric law with $A_D = A_S(\eta_0/2)^{-\alpha_D}$.

An unfinished schedule far from its end. A checkpoint at D of a cosine over $D_{\max} \gg D$ has run at nearly the peak rate, so $\eta(D) \approx \eta_0$ and $S \approx \eta_0 D$, twice the area of the matched run. Its penalty is

$$g(D) \approx -\big(1 - 2^{-\alpha_D}\big) A_D D^{-\alpha_D} + c\,\eta_0 :$$

the doubled area helps, by a fixed fraction of the data term, and the unannealed noise hurts, by a constant, so the checkpoint wins at small D and loses at large D. By Proposition 12.8 neither part moves the exponent; what does is second-order, the dependence of the floor $c\,\eta_0$ on the model through the learning-rate rule, and checkpoints read late enough that the schedule has begun to decay, and its sign depends on details.

A fixed warm-up. A linear warm-up over W examples followed by the peak rate has area $\eta_0(D - W/2)$ for $D \geq W$. Against the same run without warm-up its penalty is

$$g(D) = A_S \eta_0^{-\alpha_D} \left[\left(D - \tfrac{W}{2}\right)^{-\alpha_D} - D^{-\alpha_D} \right] \approx \tfrac{1}{2} \alpha_D A_S \eta_0^{-\alpha_D}\, W D^{-\alpha_D - 1} \qquad (D \gg W),$$

so that $D^{\alpha_D} s_g$ is proportional to W/D and decreases. By the proposition the measured exponent is too large, and more so the closer the compute-optimal runs are to the warm-up. A warm-up proportional to the run, $W = \rho D$, makes the penalty proportional to the data term, and the tilt disappears. ◇

The model accounts for the ordering of Table 12.1. A warm-up of 1.57 billion tokens, comparable to the compute-optimal runs at small budgets, tilts the allocation toward large models, and Porian et al.'s warm-up of N tokens, which their compute-optimal runs exceeded at least fivefold, removes most of the tilt. A schedule read far from its end changes the losses much and the exponent little, in a direction the model leaves open: matched decay lowered the exponent by 0.03 for Porian et al. and raised it by 0.03 for Pearce and Song (2024). Hoffmann et al.'s argument assumed a penalty that grows as the budget shrinks relative to the schedule; at Kaplan et al.'s frontier the schedules had barely begun to decay, and the penalty was close to its constant value.

Hyperparameters and cheaper protocols

Hyperparameters held fixed across scales add a penalty that depends on both N and D: a learning rate tuned for one size is wrong for others, and a batch fixed in tokens gives small budgets few steps. Kaplan et al. corrected the batch with the critical-batch model of McCandlish et al. (2018), which turned 0.88 into 0.73 (Exercise 12.4). The maximal-update parametrization of Yang et al. (2021) keeps the best hyperparameters fixed

as the width grows: tuned on a proxy of 40 million parameters at 7% of the pretraining cost, a 6.7-billion-parameter GPT-3 beat the published one. Alternatively, laws for the hyperparameters can be fitted on small runs first, as DeepSeek did, $\eta_{\text{opt}} = 0.3118\, C^{-0.125}$ and $B_{\text{opt}} = 0.2920\, C^{0.3271}$; either way the hyperparameters become a stated function of scale.

A matched run for every configuration costs about twice as much as one run per model, by Porian et al.'s accounting. A constant learning rate followed by a short cooldown matches a cosine of the same length (Hägele et al. 2024), so one constant-rate run per size, with a cooldown branched from its checkpoint at each budget, serves every budget without an unfinished schedule; the idea was used earlier to scale vision Transformers (Zhai et al. 2022), was named *warmup–stable–decay* in the MiniCPM preprint (Hu et al. 2024), and by Hägele et al.'s estimate would have more than halved the compute of the Chinchilla study. A constant rate without cooldown recovers the exponent, according to Porian et al. and Pearce and Song, but not the loss. Laws for the loss along arbitrary schedules (Tissue et al. 2025) predict every schedule from a few runs, with the cautions of Chapters 11 and 13.

12.6 In the laboratory: a miniature allocation study

Both labs use the teacher task of `labkit.py`:[7] eight-dimensional normal inputs, targets from a fixed random two-layer network plus noise of variance 0.01, the irreducible loss, and one pass over fresh examples per run, the book's online protocol. The models are two-layer ReLU perceptrons of widths 4 to 91, with $N = w^2 + 11w + 1$ parameters, from 61 to 9,283, trained by Adam with batches of 64 and a learning rate $0.032\,(16/w)^{1/2}$ chosen by a small sweep under matched schedules and used in every protocol. A run of D examples costs $C = 6ND$ operations; thirteen compute levels a factor $\sqrt{2}$ apart span 1.2×10^7 to 7.7×10^8.

[7] Runs of protocols K and KW stop after the last cell they are read at; later steps could not change their checkpoints.

LAB 12.1 · THREE PROTOCOLS FOR ONE ALLOCATION

SCRIPT
`12-protocol-schedules.py`

SETUP. Ten widths; runs of 32 to 4,096 steps (D from 2,048 to 262,144) on the cells near each width's compute-optimal budgets. *Protocol C*: a separate run for every cell, its cosine schedule ending with the run. *Protocol K*: one run per width with a cosine over 2^{15} steps, eight times the longest cell, read at the cells as checkpoints. *Protocol KW*: as K, with a linear warm-up over its first 393 steps, the 1.2% of the schedule that Kaplan et al.'s 3,000 warm-up steps were of 250,000. Six seeds, each fixing initial weights, order and examples in all three protocols; one test set of 8,192 examples. Losses on isoFLOP lines by interpolation in $\log D$, the optimum at each level by a parabola in $\log N$ through the five widths nearest the minimum, a by least squares, and 95% intervals from 2,000 bootstrap resamples of the seeds, paired across protocols. 468 runs, 1,044 losses; about three minutes.

PREDICT FIRST. Will a checkpoint of the unfinished schedule have a higher or a lower loss than the matched run on the same data? Will

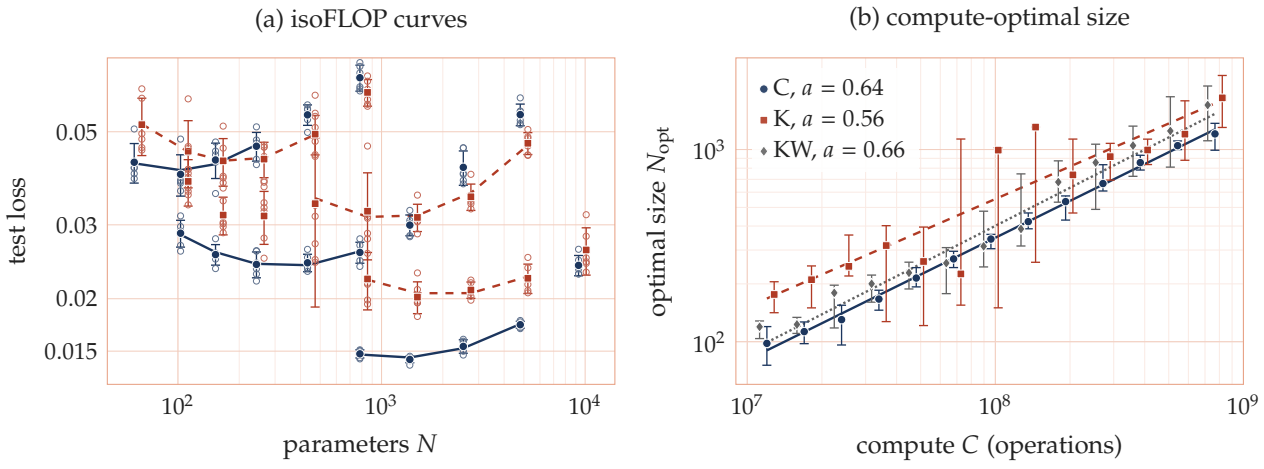


Figure 12.1. Lab 12.1: teacher task, two-layer MLPs of widths 4 to 91, six seeds (0–5) per protocol. (a) IsoFLOP curves at 1.2×10^7, 9.6×10^7 and 7.7×10^8 operations (top to bottom): the loss of each width at the number of examples that spends the budget, for matched schedules (circles, solid parabolas) and checkpoints of an unfinished schedule (squares, dashed, shifted right). Hollow marks: single seeds; filled: means with 95% t-intervals; parabolas in $\log N$ through the five widths nearest each minimum. (b) The compute-optimal size at each level for protocols C (circles), K (squares) and KW (diamonds), with 95% intervals from 2,000 bootstrap resamples of the seeds (offset horizontally), and the fitted power laws.

the optima move, and the exponent? Which way will the warm-up push them?

LOOK FOR. The ratio of checkpoint to matched loss as the data grow; the shift of the optima against the factor 2^a that a doubled area predicts; the paired differences of the exponents; the spread of the checkpoint losses.

WHAT HAPPENED. Under matched schedules the compute-optimal size grew from 98 parameters at the smallest budget to 1,206 at the largest (Figure 12.1b), with exponent $a = 0.637$ (95% bootstrap interval 0.617 to 0.666, Theorem A.3). The isoFLOP curves were shallow near their minima (Figure 12.1a), as Kaplan et al. had noted for theirs, so each optimum carries a wide interval and the exponent rests on the trend across thirteen of them.

Checkpoints of the unfinished schedule beat matched runs on short runs and lost on long ones (Figure 12.2a): the ratio of their losses was 0.85 to 0.94 up to 8,192 examples and 1.14 to 1.36 from 16,384 on, the change of sign that the heuristic predicts, from twice the area early to the unannealed noise later. Checkpoints were also noisier, with a median coefficient of variation over seeds of 10.8% against 6.4%. Along the frontier, protocol K's losses were higher than C's by 5% at the smallest budget and 41% at the largest (point estimates). Its optima were larger than C's by a factor of 1.60 on average over the levels (95% bootstrap interval 1.13 to 1.73), against $2^a = 1.55$ for a doubled area, and its exponent was 0.562 (0.497 to 0.626), lower than C's by 0.074 (paired interval 0.015 to 0.149). Reading checkpoints thus tilted

the allocation toward smaller models, as in the small experiments of Pearce and Song (2024) and against Hoffmann et al.'s argument, by more than the 0.03 that Porian et al. found with the other sign.

With the warm-up, checkpoint losses were 7.7 times those of matched runs at 2,048 examples and 1.8 times at 4,096, and within a few percent of the checkpoints without warm-up from 16,384 on: a penalty concentrated at small budgets. It raised the exponent over protocol K by 0.096 (paired interval 0.016 to 0.195), the direction and about the size of Porian et al.'s warm-up correction, to 0.659 (0.537 to 0.741), within 0.02 of the matched protocol: here the two choices of Kaplan et al.'s protocol nearly cancel, and only the three-way comparison shows that each moved the exponent. The setting was chosen for speed, and the sign of the first effect is fragile: a pilot with batches of 32, half the learning rate, a coarser grid and two seeds gave an exponent 0.03 larger with checkpoints than with matched runs, Porian et al.'s sign.

LAB 12.2 · WHAT COUNTS AS A PARAMETER

SCRIPT
12-protocol-counting.py

SETUP. The matched runs of Lab 12.1, reanalyzed without new training. The input layer ($9w$ parameters) plays the embedding and the output layer ($w+1$) the head, so that the counted size is the hidden layer, $\tilde{N} = w^2 + w$, a third to nine tenths of N. Three conventions: full $(N, 6ND)$, relabeled $(\tilde{N}, 6ND)$ and Kaplan et al.'s $(\tilde{N}, 6\tilde{N}D)$; the estimator and bootstrap of Lab 12.1; predictions of Proposition 12.7 with α_N, α_D estimated on the full-count frontier and the known floor. A few seconds.

PREDICT FIRST. At width 10, what are $\tilde{N}/N$ and ε? By how much will each partial convention raise the exponent, and which will raise it more?

LOOK FOR. The optima in the three conventions; the measured exponents against the proposition; how the predicted local exponent changes along the frontier.

WHAT HAPPENED. The same runs gave three exponents (Figure 12.2b): 0.637 in full counts, 0.797 (0.768 to 0.839) relabeled, and 0.744 (0.714 to 0.795) in Kaplan et al.'s convention, 0.108 above the full count (paired interval 0.062 to 0.161). To check Proposition 12.7 we need the exponents of the law: along the full-count frontier the loss above the floor fell like $C^{-0.451}$, which is $C^{-\alpha_N\alpha_D/(\alpha_N+\alpha_D)}$ for the parametric law, and with a this gives the point estimates $\alpha_N = 0.71$ and $\alpha_D = 1.24$, steeper than in language models because the teacher is smooth. Part (iii) then predicts local exponents falling from 0.74 to 0.69 along the Kaplan-count frontier, 0.72 on average, and part (ii) from 0.89 to 0.73 along the relabeled one, 0.80 on average; the fits gave 0.744 and 0.797. Between the three numbers only the bookkeeping changed, and it moved the exponent more than either choice of schedule did, with an overhead of at most two thirds of a model on the frontier, against 94% in Kaplan et al.'s two-layer model of width 128 (Table 12.2).

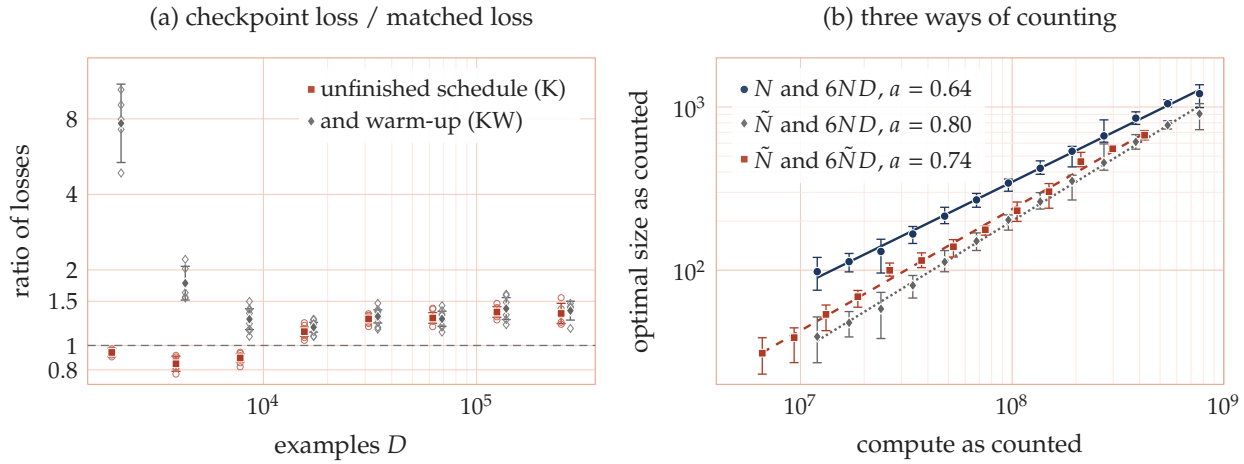


Figure 12.2. (a) Lab 12.1: the loss of a checkpoint over that of the matched run with the same width, seeds and examples, for the unfinished schedule (squares) and with Kaplan et al.'s fraction of warm-up (diamonds). Hollow: geometric means over the six seeds for each width; filled: geometric means over widths, with 95% t-intervals over widths. (b) Lab 12.2: the matched runs under three counting conventions, $\tilde{N} = w^2 + w$ leaving out the input and output layers; optima with 95% bootstrap intervals over seeds, and fitted power laws.

The miniature reproduced the ordering of the post-mortem: counting moved the exponent most, by 0.11, the warm-up next, by 0.10 over the protocol without it, and the unfinished schedule least, by 0.07, in a direction set by details. It differs from the language-model studies in spanning two decades of compute rather than three to eight, in a data exponent about four times larger, and in a learning-rate rule tuned under matched schedules, which may favor protocol C.

12.7 Protocol

PROTOCOL 12.1 · A PROTOCOL CARD FOR A SCALING STUDY

- ☐ Count size and compute as the operations the model performs: every layer that multiplies, the output layer included; if a convention leaves something out, then state what, and how large a share of the smallest model it is.
- ☐ Train every configuration as it would be trained at that budget: a schedule that ends with the run, or a cooldown from a constant-rate checkpoint; if budgets are read off a longer schedule, then say so and give the ratio of each checkpoint to the schedule's length.
- ☐ Make the warm-up a small fraction of every run, and check that the compute-optimal runs end long after it.
- ☐ Set the learning rate, the batch size and the optimizer constants for each scale, by sweeps at small scale, a transfer rule or a fitted law for the hyperparameters, and report the rule.

- ☐ Choose the range of sizes so that the counting convention does not matter, or state the exponent as local to the range.
- ☐ Repeat every configuration with several seeds at the small scales, and report the exponent with an interval from a bootstrap over seeds, together with the loss at the optimum.
- ☐ Estimate the exponent in two ways, such as isoFLOP parabolas and a parametric law, and publish the card, the seven items of Definition 12.3, with the losses of every run.

The card makes an exponent reproducible and comparable, not a law of nature. It does not protect against data that change the exponent, as DeepSeek's range from 0.45 to 0.58 shows, against a law that bends beyond the range studied (Chapter 13), or against a fit that has not converged (Chapter 11); and compute-optimal concerns training alone, while a model that will answer many requests is cheaper to serve when smaller and trained longer (Sardana et al. 2024). The card has a cost: matched schedules double the compute of a study, and per-scale tuning adds sweeps. When that is unaffordable, train one constant-rate run per size with cooldowns from its checkpoints, transfer the hyperparameters with a parametrization designed for it, and report which items of the card were left at defaults.

Exercises

12.1 CONSOLIDATION. (Local exponents along the frontier.) For $\mathcal{L}(N, D) = E + A_N N^{-\alpha_N} + A_D D^{-\alpha_D}$ at the compute-optimal allocation of Chapter 11, (a) show that $\mathcal{L}_{\text{opt}}(C) - E \propto C^{-\gamma}$ with $\gamma = \alpha_N \alpha_D / (\alpha_N + \alpha_D)$, so that the local exponent of $\mathcal{L}_{\text{opt}}$ in C is $\gamma(1 - E/\mathcal{L}_{\text{opt}})$; (b) with Hoffmann et al.'s $E = 1.69$, $A_N = 406.4$, $A_D = 410.7$, $\alpha_N = 0.34$, $\alpha_D = 0.28$, compute it at 10^{19} and 10^{21} operations and compare with Kaplan et al.'s 0.050. *(Hint: $N_{\text{opt}} = G(C/6)^a$ and $D_{\text{opt}} = G^{-1}(C/6)^{1-a}$ with $G = (\alpha_N A_N / (\alpha_D A_D))^{1/(\alpha_N + \alpha_D)}$; the answers are near 0.07 and 0.04.)*

12.2 CONSOLIDATION. (A constant overhead.) In Proposition 12.7 let $e > 0$ be constant. (a) Show that $\varepsilon = \tilde{N}/N$ and $\mathrm{d}\ln\varepsilon / \mathrm{d}\ln\tilde{N} = 1 - \varepsilon$, so part (iii) applies exactly where $\varepsilon > (1 - \alpha_D)/(1 + \alpha_N)$. (b) Show that below this threshold a stationary point of the loss at fixed $\tilde{C}$ is a local maximum, and that the loss tends to $E + A_N e^{-\alpha_N}$ as $\tilde{N} \to 0$; what does the study then conclude about its smallest models?

12.3 CONSOLIDATION. (How long may a warm-up be?) In the heuristic model a run with a linear warm-up over W examples and $D \geq W$ has the data term $A_S \eta_0^{-\alpha_D} (D - W/2)^{-\alpha_D}$. (a) Show that its marginal value falls at the rate $\kappa_D = \alpha_D + (1 + \alpha_D)x/(1 - x)$, $x = W/(2D)$. (b) With $\alpha_N = \alpha_D = 0.3$, find the largest W/D at which the exponent of Lemma 12.6 is within 0.02 of $a = 0.5$. (c) Apply (b) to Kaplan et al.'s warm-up of 1.57 billion tokens at budgets whose compute-optimal runs have at most 10 billion. *(Hint: (b) gives $W/D \approx 0.04$.)*

12.4 CONSOLIDATION. (The batch adjustment.) Kaplan et al. replaced the compute C of a run at batch B by $C_{\min} = C/(1 + B/B_{\text{crit}})$, with $B_{\text{crit}} = B_*\mathcal{L}^{-1/\alpha_B}$, $B_* = 2 \times 10^8$ tokens and $\alpha_B = 0.21$. (a) Show that a constant ratio B/B_{crit} along the frontier would leave the exponent unchanged. (b) With $B = 2^{19}$, compute $C_{\min}/C$ at losses of 4 and 3 nats, convert the losses into compute with the unadjusted law $\mathcal{L} \propto C^{-0.057}$, and estimate $\mathrm{d}\ln C_{\min}/\mathrm{d}\ln C$ between them. (c) Show that the adjusted exponent is the raw one divided by this derivative, and compare 0.88 divided by your estimate with 0.73.

12.5 LAB. (Warmup–stable–decay.) Add to the script of Lab 12.1 a protocol whose cells are constant-rate runs with a linear cooldown to zero over their last 20%, branched from one constant-rate run per width. Predict from the heuristic how its losses and exponent compare with protocols C and K, then run it and count its steps against protocol C. *(Hint:* `labkit.py` *has the schedule* `wsd`*.)*

12.6 LAB. (The learning-rate rule.) Rerun Lab 12.1 with one learning rate, 0.016, for all widths, and then with $0.032\,(16/w)$. How do the ratio of checkpoint to matched loss across widths and the difference between the exponents of K and C change? Explain with the part of Proposition 12.8 about penalties of the size.

12.7 LAB. (Only the head.) In `12-protocol-counting.py` leave out only the output layer, as Porian et al. left out the head. Predict the exponent from Proposition 12.7, then check; why is the effect smaller than for a language model's head?

12.8 DESIGN. (Ten runs.) With ten runs of at most 10^{20} operations, estimate the allocation exponent of a new architecture: choose sizes, budgets and schedules, say which items of the protocol card you can do properly, and how you will report the uncertainty of a.

12.9 DESIGN. (Hungrier for parameters?) A team claims an allocation exponent of 0.58 for its architecture against 0.50 for a standard Transformer. Design the comparison: which items of the card the arms must share and which be tuned for each, and how many budgets and seeds detect a difference of 0.08, by the delta method (Theorem A.10) or a paired bootstrap over seeds.

12.10 CRITIQUE. (The longest run's checkpoints.) A paper on scaling laws for a new optimizer says: "to save compute, we read the loss of smaller budgets from the checkpoints of our longest run, whose cosine schedule spans 100 billion tokens"; its compute-optimal runs use 2 to 20 billion tokens. What does the protocol measure, which way are its losses and exponent likely biased, and what should the authors have run at similar cost?

12.11 CRITIQUE. (A small-scale exponent.) A study of protein language models fits $N_{\text{opt}} \propto C^{0.62}$ on 1 to 50 million non-embedding parameters,

with 33 tokens and 1,024 learned positions, and concludes that data are worth less for proteins than for text. Audit the conclusion with Proposition 12.7, starting from the overhead at width 256.

12.12 RESEARCH. (Allocation with tuned hyperparameters.) Open: an estimator of the allocation exponent that treats the learning rate and batch size at each scale as nuisance parameters, with a valid interval and without a sweep at every scale; and a size variable (total parameters, non-embedding parameters, operations per token) for which the exponent is constant over the widest range.

Notes and References

Power laws for the error of deep networks against data were measured across translation, language modeling, images and speech by Hestness et al. (2017); Kaplan et al. (2020) extended them to model size and compute and turned them into an allocation rule. The parametric fit of Hoffmann et al. (2022) has been refitted by Besiroglu et al. (2024), the subject of Chapter 11. The two reconstructions of the discrepancy, Porian et al. (2024) from new experiments and Pearce and Song (2024) from published numbers, appeared within weeks of each other in June 2024, each citing the other as concurrent work.

The cosine schedule comes from the warm restarts of Loshchilov and Hutter (2017), used in language models as a single cycle; Schaipp et al. (2025) found that the loss curves of cosine and cooldown schedules closely track a bound from convex optimization, a step toward a theory of the unfinished schedule. Read before their end, schedules mislead comparisons of optimizers (Chapter 4) and of efficient training methods (Chapter 7) as well; how far a fitted law may be extended is the subject of Chapter 13.

TRANSLATING THE LITERATURE. Kaplan et al. (2020) write N for our counted size $\tilde{N}$ and C for our $\tilde{C} = 6\tilde{N}D$, and $C_{\min}$ for the compute adjusted to the critical batch size; their α_N, α_D, $\alpha_C^{\min}$ keep their names but are local exponents in the sense of Proposition 12.5, and their law $\mathcal{L}(N, D)$ is not separable. Hoffmann et al. (2022) write A, B, α, β for our $A_N, A_D, \alpha_N, \alpha_D$, and their Approach 2 is our protocol C with an isoFLOP estimator. Porian et al. (2024) count in N every linear layer, the head included, and write $N^\star$ for N_{opt}. Pearce and Song (2024) write N_T and $N_{\setminus E}$ for our N and $\tilde{N}$ and g for the exponent of part (iii) of Proposition 12.7; their schemes 3 and 4 are our protocols K and C. Bi et al. (2024) measure size by M, the non-embedding operations per token, so that $C = MD$. Constant learning rate with cooldown (Hägele et al. 2024) is the warmup–stable–decay schedule.

References

Besiroglu, T., E. Erdil, M. Barnett, and J. You (2024). *Chinchilla Scaling: A Replication Attempt*. Preprint, arXiv:2404.10102.

Bi, X., D. Chen, G. Chen, S. Chen, D. Dai, C. Deng, H. Ding, K. Dong, et al. (2024). *DeepSeek LLM: Scaling Open-Source Language Models with Longtermism*. Preprint, arXiv:2401.02954.

Brown, T. B. et al. (2020). "Language Models are Few-Shot Learners". In: *Advances in Neural Information Processing Systems 33*, pp. 1877–1901.

Hägele, A., E. Bakouch, A. Kosson, L. Ben Allal, L. Von Werra, and M. Jaggi (2024). "Scaling Laws and Compute-Optimal Training Beyond Fixed Training Durations". In: *Advances in Neural Information Processing Systems 37*. arXiv:2405.18392.

Hestness, J. et al. (2017). *Deep Learning Scaling is Predictable, Empirically*. Preprint, arXiv:1712.00409.

Hoffmann, J. et al. (2022). "An Empirical Analysis of Compute-Optimal Large Language Model Training". In: *Advances in Neural Information Processing Systems 35*. First posted as "Training Compute-Optimal Large Language Models", arXiv:2203.15556.

Hu, S. et al. (2024). *MiniCPM: Unveiling the Potential of Small Language Models with Scalable Training Strategies*. Preprint, arXiv:2404.06395.

Kaplan, J. et al. (2020). *Scaling Laws for Neural Language Models*. Preprint, arXiv:2001.08361.

Loshchilov, I. and F. Hutter (2017). "SGDR: Stochastic Gradient Descent with Warm Restarts". In: *International Conference on Learning Representations*.

McCandlish, S., J. Kaplan, D. Amodei, and OpenAI Dota Team (2018). *An Empirical Model of Large-Batch Training*. Preprint, arXiv:1812.06162.

Pearce, T. and J. Song (2024). "Reconciling Kaplan and Chinchilla Scaling Laws". In: *Transactions on Machine Learning Research*. arXiv:2406.12907.

Porian, T., M. Wortsman, J. Jitsev, L. Schmidt, and Y. Carmon (2024). "Resolving Discrepancies in Compute-Optimal Scaling of Language Models". In: *Advances in Neural Information Processing Systems 37*. arXiv:2406.19146.

Sardana, N., J. Portes, S. Doubov, and J. Frankle (2024). "Beyond Chinchilla-Optimal: Accounting for Inference in Language Model Scaling Laws". In: *Proceedings of the 41st International Conference on Machine Learning*. Vol. 235. Proceedings of Machine Learning Research, pp. 43445–43460.

Schaipp, F., A. Hägele, A. Taylor, U. Simsekli, and F. Bach (2025). "The Surprising Agreement Between Convex Optimization Theory and Learning-Rate Scheduling for Large Model Training". In: *Proceedings of the 42nd International Conference on Machine Learning*. Vol. 267. Proceedings of Machine Learning Research, pp. 53267–53294.

Tissue, H., V. Wang, and L. Wang (2025). "Scaling Law with Learning Rate Annealing". In: *Advances in Neural Information Processing Systems (NeurIPS 2025)*. arXiv:2408.11029.

Yang, G. et al. (2021). "Tuning Large Neural Networks via Zero-Shot Hyperparameter Transfer". In: *Advances in Neural Information Processing Systems 34*. Posted in 2022 as "Tensor Programs V", arXiv:2203.03466.

Zhai, X., A. Kolesnikov, N. Houlsby, and L. Beyer (2022). "Scaling Vision Transformers". In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 12104–12113.

13 Extrapolation

Predicting the large from the small

THE CLAIM *"On these tasks, larger language models do worse."*

The received reading of the eleven winning tasks of the Inverse Scaling Prize (McKenzie et al. 2023)

THE VERDICT Narrowed: on a larger family of models only four of the eleven tasks still scaled inversely; a trend over a few decades of scale describes those decades.

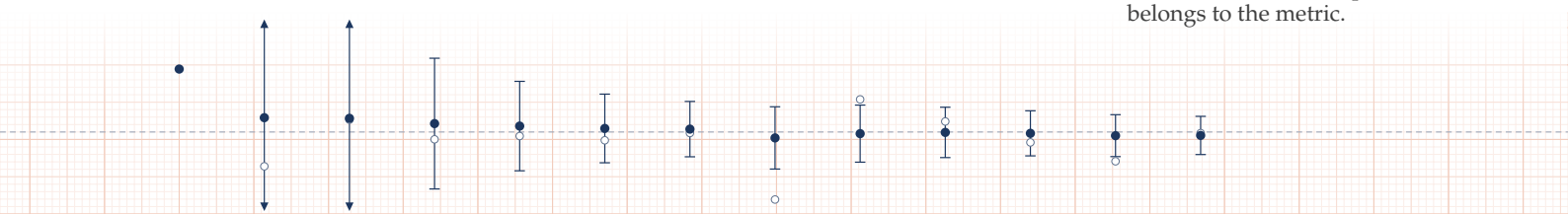
IN THIS CHAPTER. In 2022 a public contest collected eleven tasks on which language models did worse the larger they were, across model families spanning five orders of magnitude of training compute. On a larger family with five times the compute, six of the curves turned upward, one rose throughout, and one solved example in the prompt changed the other four: the trends were real over the range measured and did not hold beyond it. The method treats a forecast at a new scale as a measurement with three sources of error: the uncertainty of the fitted parameters, which grows, far from the data, linearly with the distance; the error of the functional form, which grows quadratically and is nearly invisible inside the data; and a change of regime, which no fit can detect. It checks a forecasting procedure by backtesting it on held-out large scales. In the laboratory, a straight line fitted in log–log coordinates to the six smallest of thirteen widths underestimates the loss of the largest by a factor of 2.8, and lines fitted to five to eight rungs have 95% intervals that cover 21% of held-out runs beyond 1.2 decades, against 88% for a law with a floor; on digits with noisy labels, a miniature inverse-scaling trend reverses within a decade.

13.1 The claim: eleven tasks on which bigger was worse

By 2022 the loss of a language model on held-out text was one of the most predictable numbers in deep learning. Fitted power laws described it over many orders of magnitude of parameters, data and compute (Hoffmann et al. 2022; Kaplan et al. 2020), laboratories chose the size of their largest runs by fitting smaller ones (Chapters 11 and 12), and a lower loss usually came with better scores on downstream tasks. Large models thus combined a predictable loss with abilities that were harder to anticipate, the combination that Ganguli et al. (2022) called predictability and surprise.[1]

Exceptions were known. On TruthfulQA, a set of questions that some people answer falsely because of a common misconception, the largest

[1] The surprise usually meant an ability that appeared abruptly with scale; Chapter 14 examines how much of the abruptness belongs to the metric.



models tested were generally the least truthful (Lin et al. 2022). A task on which performance gets worse as models get larger, or as their loss on the training objective improves, is said to show *inverse scaling*.

In June 2022 a group of researchers, most of them at New York University and FAR AI, opened a public contest for such tasks, the Inverse Scaling Prize, which offered up to $100,000 for "an important task where larger language models do worse" (McKenzie et al. 2022). Participants submitted a dataset of at least 300 examples with plots over the sizes of GPT-3, and the organizers evaluated each submission on further model families, among them private ones from Anthropic and DeepMind, so that a task could not owe its trend to one family. Two rounds, with deadlines on 27 August and 27 October 2022, drew 99 submissions. The grand and second prizes went unawarded, mainly because no task convincingly demonstrated real-world consequences of the failure; eleven tasks received third prizes of $5,000. The organizers and the task authors reported the results together (McKenzie et al. 2023).

> FINDING 13.1 (McKenzie et al. 2023). On the eleven datasets collected by the contest, task performance got worse as training compute grew, over all or part of the range, and for most of the tasks the trend held across most of the model series evaluated: GPT-2, GPT-3, OPT, pretrained Anthropic models, Gopher and Chinchilla, with instruction-tuned variants, from about 10^{18} to 10^{23} training FLOPs. The authors grouped the tasks by four suspected causes: a prior from pretraining that overrides the instruction in the prompt; imitation of undesirable patterns in the training data; an easy distractor task that the model performs instead of the real one; and few-shot demonstrations that are correct but misleading.

The tasks are easy for people. In Memo Trap a model must write a phrase that begins like a famous quotation but ends differently; the larger the model, the more often it completes the quotation. In Hindsight Neglect the demonstrations show bets whose outcomes agree with their expected values, and the question is about a bet whose outcome does not; larger models increasingly judged the bet by its outcome.

The contest asked for tasks on which larger models do worse, and its winners were presented as such tasks. Read as a statement about scale rather than about the models measured, the result became the claim of this chapter. It mattered beyond the eleven tasks: if scale could make some behaviors worse, then scaling could also encourage behavior at odds with human preferences, the implication with which Wei et al. (2023) open their study of the tasks.

The organizers built the safeguards that such a claim needs: held-out model families, labels checked by crowd workers, and results with few-shot prompts and across training checkpoints as well as zero-shot. Their final report, written after the first post-mortem, is more careful than the reception: it describes U-shaped and inverted-U trends and concludes that scaling trends are "less reliable at predicting the behavior of larger-scale models than previously understood." The claim was reasonable when the tasks

were collected. The trends held across families with different data and architectures, several had a plausible mechanism, such as a prior from pretraining that grows stronger with scale, and the field's experience with loss curves suggested that trends in scale are smooth and monotone. The expectation that made the inverse trends alarming also made them look durable.

13.2 The experiment: a dose–response curve read beyond its last dose

Each winning task was a small experiment with scale as its treatment.

EXPERIMENTAL DESIGN · A WINNING TASK OF THE INVERSE SCALING PRIZE, 2022

QUESTION	Does performance on the task get worse as language models get larger?
UNITS	pretrained models, one per size, from each of several families
TREATMENT	scale, measured by training compute (or parameters) within a family
LEVELS	the sizes each family offered, e.g. Gopher 44M–280B parameters; about 10^{18}–10^{23} FLOPs in all
HELD FIXED	the task's prompts, answer options and scoring; zero-shot in the main analysis
VARIED	across families: training data, architecture, tokenizer, ratio of data to parameters
RESPONSE	accuracy, taking the most probable answer option (loss for one task)
REPLICATES	one model per size; at least 300 items per task
ANALYSIS	scaling plots per family, reviewed against a rubric: roughly monotone inverse scaling on public models and inverse scaling on some private ones

This is a dose–response design: the dose is scale and the response is accuracy. It can show that over the range measured the response fell; the held-out families make a quirk of one family's data or code unlikely, and the human validation makes wrong labels unlikely.

It cannot show what happens at a larger dose. A family contributed a handful of models over a few decades of compute, one per size, with no estimate of how far a second model of the same size would have landed, and the trend was judged from the plots. "Larger models do worse" is a statement about models larger than the largest one measured, and the design measured none of them. Two further features would matter. The prompt and the scoring were held fixed, so each trend belonged to the task under one setting of the instrument (Chapter 15). And the families differ in their ratio of data to parameters, so their curves cannot be laid end to end on one axis of compute, as the authors themselves caution.[2]

[2] The axis is itself an estimate: for several families the organizers computed training FLOPs as $6ND$ from the parameters and tokens, and the counts for Gopher and Chinchilla were provided privately.

13.3 The post-mortem: the curves turn

A LARGER FAMILY. In November 2022 Jason Wei, Najoung Kim, Yi Tay and Quoc Le, at Google, ran the four first-round tasks on PaLM, and later all eleven; Kim was also an author of the prize's report (Wei et al. 2023). They used the 8B, 62B and 540B models of the original PaLM paper and a 1B model trained by the same protocol, with the contest's prompts and scoring. The largest used about 2.5×10^{24} training FLOPs, five times as much as Chinchilla 70B, the model trained with the most compute in the contest.

> FINDING 13.2 (Wei et al. 2023). On PaLM models of 1B, 8B, 62B and 540B parameters, six of the eleven tasks scaled in a U shape, performance falling and then rising up to the largest model (Hindsight Neglect, Sig Figs, Resisting Correction, Modus Tollens, Memo Trap, NeQA); one improved throughout (Repetitive Algebra); four still scaled inversely (Pattern Match Suppression, Redefine, Into the Unknown, Prompt Injection). With one solved example added to each prompt, those four became U-shaped or flat, and chain-of-thought prompting changed four of the nine classification tasks that were U-shaped or flat under one-shot prompting to positive scaling, by the authors' count.

Their explanation of this *U-shaped scaling* is the third cause of Finding 13.1, the distractor task: small models can do neither the real task nor the distractor and answer near chance, medium models learn the distractor and do worse, and large models learn to set it aside. The authors call the hypothesis speculative.[3]

Four points limit what the post-mortem shows, and its authors state most of them. First, each curve has four points, one model per size, and in three of the six U shapes the upturn is the 540B model alone. Second, an upturn is not a recovery: on four of the six tasks the 540B model still scored below the 1B model, and with one-shot prompts it was behind or not substantially ahead of the 1B model on six of the ten tasks that were then U-shaped or flat. Third, PaLM is another family, trained on other data, and its curve is not the continuation of Gopher's or GPT-3's. Fourth, the measurement depends on the prompt. The first version of the paper had modified some of the prompts and reported three of the four first-round tasks as U-shaped; the second version, posted eleven days later after feedback from Ethan Perez, one of the prize's organizers, used the contest's prompts unchanged and found two.[4]

ONE MORE MODEL. The GPT-4 technical report added a point (OpenAI 2023): on Hindsight Neglect, accuracy fell across three sizes of GPT-3 and GPT-3.5, then reached 100% with GPT-4, a result the report filed under capabilities that "remain hard to predict." McKenzie et al., who received GPT-4 results on five further tasks, report improvement on three, mixed results on Pattern Match Suppression and poor ones on Memo Trap, and they are uncertain whether GPT-4 should count as a member of the same series as its predecessors.

[3] They also point to earlier U-shaped curves among the tasks of the BIG-bench collection, TruthfulQA among them.

[4] Both versions are on arXiv (2211.02011, versions 1 and 2); the comment on version 2 records the change of prompts. The final version, with all eleven tasks, appeared at EMNLP 2023.

REVERSALS BOTH WAYS. Trends also turned for the worse. On Prompt Injection, McKenzie et al. found performance that improved with scale and then declined, an inverted U. Whether a trend persists beyond the decades measured, and even its direction there, cannot be read from the curve inside them.

HOW COMMON IS AN UNRELIABLE TREND? Lourie et al. (2025) re-examined the 46 downstream tasks of Gadre et al. (2025), whose models span 0.011 to 6.9 billion parameters, and classified how accuracy changed as the validation loss improved: 18 tasks, or 39%, improved smoothly and predictably, and the others scaled inversely, non-monotonically, noisily, without a trend, or by an abrupt breakthrough. The classification is by inspection, as the authors say. Changing the corpus on which the validation loss is measured, or the experimental setup in which the same task is evaluated, could reverse which pretraining corpus looked better or turn a clean scaling law into a non-monotone one.

Caballero et al. (2023) proposed a *smoothly broken power law*, a chain of power laws joined by smooth breaks. On the held-out large scales of their benchmark it extrapolated better than four other forms, with the lowest error on 69% of the downstream vision tasks and 75% of the language tasks, and it can bend into a double-descent curve or a sharp inflection. It cannot anticipate a break that has not happened: on four-digit addition they could extrapolate only from points at or past the break, and they conclude that a sufficiently sharp break well beyond the largest fitted point cannot at present be extrapolated.

THE COUNTERPOINT: FORECASTS THAT HELD. Some forecasts were made in advance and held. For GPT-4, a power law with an irreducible term, $L(C) = aC^b + c$, fitted to models trained by the same method with at most a ten-thousandth of the compute, predicted the final loss on an internal code corpus, shortly after the run started and without its partial results; a forecast of the pass rate on groups of HumanEval problems, from models with at most a thousandth of the compute, was registered before training finished and held except on the easiest group (OpenAI 2023). The Llama 3 team forecast the 405B model's accuracy on the ARC Challenge in two steps, from compute to the loss on the correct answers and from that loss to accuracy, across four orders of magnitude, and slightly underestimated it (Llama Team, AI @ Meta 2024). Gadre et al. (2025) predicted the validation loss of two large runs to within 0.7% from experiments with 300 times less compute, and their average top-1 error on 17 tasks to within 0.05% and 3.6% from 20 times less; individual tasks were much noisier, and without the 1.4B model in the fit one of the errors rose from 0.05% to 10.64%. Ladders with about 1% of the target compute predicted the accuracy of 7B and 13B models to within two points on four multiple-choice tasks (Bhagia et al. 2025), and fits to about a hundred public models predicted several abilities that had looked abrupt (Ruan et al. 2024).

The forecasts that held share three features: the quantity was smooth, a loss or an average over many tasks, or a single task reached through such a quantity; the ladder and the target were trained alike, or their differences

were modeled; and the forecast was checked against the result. The ones that failed predicted one task's metric beyond the range measured, often across a change of family, prompt or regime.

VERDICT. Over the range the contest measured, the inverse trends were real: on most of the eleven tasks, performance fell with scale across most of the families tested. On a larger family with five times the compute, six of the curves turned upward and one rose throughout, a single example in the prompt changed the other four, and GPT-4 solved one task outright. What survives is a description of the decades measured, for the families and prompts used. The step from "falls across this range" to "larger models do worse" was an extrapolation, and nothing in the design could support it.

NARROWED

13.4 The method: uncertainty that grows with distance

A forecast at a new scale is a measurement at a point where nothing was measured. In a scaling study the *design* is the ladder of scales trained, $x_1, \dots, x_n$, where x is the logarithm of the number of parameters, examples or FLOPs, and the response y is the logarithm of a loss or of a transformed metric; a forecast predicts the response of one new run at a scale x_0. It is an *interpolation* when x_0 lies within the range of the design and an *extrapolation* when it lies beyond.

FOUR SOURCES OF ERROR. A forecast misses for four reasons. The new run has its own noise, the spread of runs at one scale (Chapter 1), which no forecast removes. The fitted parameters are uncertain, and their uncertainty, carried to x_0, grows with the distance from the data, linearly once the distance is large (Theorem 13.3). The functional form may be wrong, and its error, almost invisible inside the data, grows quadratically (Proposition 13.5). And the process may change: a mechanism that does not act inside the range measured, such as a distractor that only large models learn to ignore, bends the curve beyond it (Remark 13.6). Prediction intervals account for the first two sources, a test of the form for part of the third, and nothing computed from the data for the fourth.

The first two are a theorem about the straight line, the form that a power law takes in logarithmic coordinates.

Theorem 13.3 (Prediction at a new scale). *Let $x_1, \dots, x_n$ be design points, $n \geq 2$, not all equal, and let $y_i = a + bx_i + \varepsilon_i$ with uncorrelated errors of mean zero and variance σ^2. Write $\bar{x}$ for the mean of the design points, $S_{xx} = \sum_i (x_i - \bar{x})^2$, and $\hat{y}(x_0) = \hat{a} + \hat{b}x_0$ for the least-squares prediction at a point x_0.*

(i) The prediction is unbiased for $a + bx_0$, the slope has $\operatorname{Var} \hat{b} = \sigma^2 / S_{xx}$, and

$$\operatorname{Var} \hat{y}(x_0) = \sigma^2 \Big(\frac{1}{n} + \frac{(x_0 - \bar{x})^2}{S_{xx}} \Big). \tag{13.1}$$

For a new observation $y_0 = a + bx_0 + \varepsilon_0$ whose error has mean zero and variance σ^2 and is uncorrelated with $\varepsilon_1, \dots, \varepsilon_n$, the error $y_0 - \hat{y}(x_0)$ has mean zero and variance $\sigma^2\big(1 + 1/n + (x_0 - \bar{x})^2/S_{xx}\big)$.

(ii) If every x_i lies in an interval $[L, U]$, then $S_{xx} \le n(U - L)^2/4$, so that $\operatorname{Var}\hat{b} \ge 4\sigma^2/\big(n(U - L)^2\big)$, with equality if and only if n is even and half of the x_i equal L and half equal U.

(iii) If moreover $x_0 = L + t(U - L)$ with $t > 1$, then $\operatorname{Var}\hat{y}(x_0) \ge (2t - 1)^2\sigma^2/n$, with equality if and only if every x_i equals L or U and a fraction $t/(2t - 1)$ of them equal U.

Proof. The plan is to write the prediction as the true line plus a linear combination of the errors, which gives part (i), and then to bound S_{xx} for designs inside $[L, U]$ by one inequality, which gives parts (ii) and (iii).

Part (i). Put $w_i = (x_i - \bar{x})/S_{xx}$, so that $\sum_i w_i = 0$, $\sum_i w_i x_i = 1$ and $\sum_i w_i^2 = 1/S_{xx}$. The least-squares line passes through the point of means with slope $\hat{b} = \sum_i w_i y_i$, so $\hat{y}(x_0) = \bar{y} + \hat{b}(x_0 - \bar{x})$. Substituting $y_i = a + bx_i + \varepsilon_i$ and using the two sums of the w_i, we get $\hat{b} = b + \sum_i w_i \varepsilon_i$ and $\bar{y} = a + b\bar{x} + \bar{\varepsilon}$, hence

$$\hat{y}(x_0) = a + bx_0 + \bar{\varepsilon} + (x_0 - \bar{x}) \sum_i w_i \varepsilon_i,$$

whose mean is $a + bx_0$. Because the errors are uncorrelated with variance σ^2, the sum $\sum_i w_i \varepsilon_i$ has variance $\sigma^2 \sum_i w_i^2 = \sigma^2/S_{xx}$, which is $\operatorname{Var}\hat{b}$, the mean $\bar{\varepsilon}$ has variance σ^2/n, and their covariance is $(\sigma^2/n)\sum_i w_i = 0$. Hence $\operatorname{Var}\hat{y}(x_0) = \sigma^2/n + (x_0 - \bar{x})^2\sigma^2/S_{xx}$, which is (13.1). The new error ε_0 is uncorrelated with $\hat{y}(x_0) - a - bx_0$, a combination of $\varepsilon_1, \dots, \varepsilon_n$, so $y_0 - \hat{y}(x_0)$ has mean zero and variance $\sigma^2 + \operatorname{Var}\hat{y}(x_0)$.

The inequality. For each i the product $(U - x_i)(x_i - L)$ is nonnegative because $L \le x_i \le U$, and it is zero exactly when x_i is L or U. Summing over i and expanding gives $\sum_i x_i^2 \le (U + L)\sum_i x_i - nUL$; subtracting $n\bar{x}^2$ from both sides gives

$$S_{xx} \le n(U - \bar{x})(\bar{x} - L), \tag{13.2}$$

with equality if and only if every x_i equals L or U.

Part (ii). The product $(U - \bar{x})(\bar{x} - L)$ equals $(U - L)^2/4 - \big(\bar{x} - (L + U)/2\big)^2$, so it is at most $(U - L)^2/4$, with equality exactly when $\bar{x}$ is the midpoint. Equality throughout therefore holds exactly when all the design points sit at the two ends with their mean at the midpoint, that is, half at each end, which requires n to be even. The bound on $\operatorname{Var}\hat{b}$ follows from part (i).

Part (iii). Write $h = U - L$ and $p = (\bar{x} - L)/h$. The design points are not all equal, so $S_{xx} > 0$, and (13.2) forces $0 < p < 1$. Since $x_0 - \bar{x} = (t - p)h$ and, by (13.2), $S_{xx} \le np(1 - p)h^2$, part (i) gives

$$\frac{n \operatorname{Var}\hat{y}(x_0)}{\sigma^2} = 1 + \frac{n(x_0 - \bar{x})^2}{S_{xx}} \ge 1 + \frac{(t - p)^2}{p(1 - p)}.$$

To compare the right side with $(2t - 1)^2$, expand both sides of the identity $(t - p)^2 - 4t(t - 1)\,p(1 - p) = \big(t - (2t - 1)p\big)^2$; since $(2t - 1)^2 - 1 = 4t(t - 1)$,

it rearranges to

$$1+\frac{(t-p)^2}{p(1-p)}=(2t-1)^2+\frac{\big(t-(2t-1)p\big)^2}{p(1-p)}\geq(2t-1)^2.$$

Equality requires equality in (13.2), so that every design point is L or U, and $p=t/(2t-1)$; for such a design p is the fraction of points at U. Because $t>1$, this fraction lies strictly between $\frac{1}{2}$ and 1, so the design has points at both ends, and it attains the bound. ■

Corollary 13.4 (A prediction interval for one new run). *In the setting of Theorem 13.3(i), suppose in addition that $n\geq 3$ and that $\varepsilon_0,\varepsilon_1,\dots,\varepsilon_n$ are independent and normal, and let S^2 be the residual sum of squares of the fit divided by $n-2$. Then the interval*

$$\hat{y}(x_0)\pm t_{n-2,0.975}\,S\sqrt{1+\frac{1}{n}+\frac{(x_0-\bar{x})^2}{S_{xx}}}$$

contains y_0 with probability exactly 0.95. Without the term 1 under the root it is a 95% confidence interval for the mean response $a+bx_0$.

PROOF. For a linear model with independent normal errors, the residual sum of squares divided by σ^2 has the χ^2 distribution with $n-2$ degrees of freedom and is independent of $(\hat{a},\hat{b})$ (Seber and Lee 2003). The error ε_0 is independent of both, so $y_0-\hat{y}(x_0)$ is normal, with mean zero and the variance of Theorem 13.3(i), and independent of S^2. A standard normal divided by the square root of an independent $\chi^2_{n-2}/(n-2)$ variable has Student's t distribution with $n-2$ degrees of freedom, as in Theorem A.1, and the interval is this statement solved for y_0. The confidence interval follows in the same way without ε_0. ■

READING THE THEOREM. In logarithmic coordinates an interval of half-width h for $\ln L$ is a factor $e^{\pm h}$ on the loss, and far from the ladder the half-width grows in proportion to the distance from its center. Distance is measured in units of the ladder's own spread: what matters is $(x_0-\bar{x})^2/S_{xx}$, not the number of decades.[5] Consider eight runs on a ladder spanning two decades and a forecast one decade beyond its top, so that $t=1.5$. Spread evenly, the runs give $n\operatorname{Var}\hat{y}(x_0)/\sigma^2=10.3$; four at each end give 5; and six at the top with two at the bottom, the fraction $t/(2t-1)=3/4$ of part (iii), give the minimum, 4. The even ladder's forecast is thus 1.6 times as uncertain as the best one in standard deviation, and 3.2 times as uncertain as its own prediction at the center of the ladder. Two rules follow: the variance depends on the design only through n, $\bar{x}$ and S_{xx}, so runs in the middle of a ladder buy little precision; and a forecast upward is most precise when most runs sit at the top of the ladder.[6]

But a design with runs only at the two ends of the ladder cannot test the line it fits: two scales determine a line exactly, whatever the curve between and beyond them. The precision of Theorem 13.3 is the precision of a forecast whose form is right, and the next section asks what happens when it is not.

[5] In the language of regression diagnostics, $1/n+(x_0-\bar{x})^2/S_{xx}$ is the leverage of the point x_0; a forecast beyond the ladder is a prediction at a leverage larger than that of its top rung.

[6] Part (iii) is the simplest case of the optimal allocation of Elfving (1952); Hoel and Levine (1964) found the optimal designs for extrapolating a polynomial of every degree.

13.5 The method: the form, the break and the backtest

A power law with an irreducible term, $L = E + AN^{-\alpha}$, is not a straight line in logarithmic coordinates: its local slope is $-\alpha(1 - E/L)$, which flattens as the loss approaches its floor (Chapter 12). The simplest model of a wrong form is a line fitted to a parabola.

Proposition 13.5 (A line fitted to a curve). *Let the design be as in Theorem 13.3 with $n \geq 3$, and let $y_i = f(x_i) + \varepsilon_i$ with $f(x) = a + bx + cx^2$ and errors as there. Fit a straight line by least squares. Write $m_k = \frac{1}{n}\sum_i (x_i - \bar{x})^k$ for $k = 2, 3, 4$, and*

$$r(x) = (x - \bar{x})^2 - \frac{m_3}{m_2}(x - \bar{x}) - m_2, \qquad \kappa = m_4 - m_2^2 - \frac{m_3^2}{m_2}.$$

(i) For every x_0, $\mathbb{E}\hat{y}(x_0) = f(x_0) - c\, r(x_0)$, and $\operatorname{Var}\hat{y}(x_0)$ is given by (13.1).

(ii) The residuals have means $\mathbb{E}\big(y_i - \hat{y}(x_i)\big) = c\, r(x_i)$, and $\mathbb{E}S^2 = \sigma^2 + c^2 n\kappa/(n-2)$.

(iii) If $c \neq 0$, then for a new observation $y_0 = f(x_0) + \varepsilon_0$, with ε_0 as in Theorem 13.3(i), the probability that the interval of Corollary 13.4 contains y_0 tends to zero as $|x_0| \to \infty$; the same holds for every fixed multiplier in place of $t_{n-2,0.975}$.

Proof. The plan is to use that least squares is linear in the responses, so that the expected fit is the fit of f itself, which we compute; part (ii) then follows, and part (iii) compares a bias that grows quadratically with an interval that grows linearly.

Part (i). By the proof of Theorem 13.3, $\hat{y}(x_0) = \sum_i \ell_i y_i$ with weights $\ell_i = 1/n + (x_0 - \bar{x})w_i$ that depend only on the design. Hence $\mathbb{E}\hat{y}(x_0) = \sum_i \ell_i f(x_i)$, the value at x_0 of the least-squares line through the points $(x_i, f(x_i))$, and the random part of $\hat{y}(x_0)$ is the same combination of the errors as before, so its variance is (13.1). Fitting a line is linear in the responses and reproduces every linear function exactly. Write $u = x - \bar{x}$; then $cx^2 = c\bar{x}^2 + 2c\bar{x}u + cu^2$, and only cu^2 is not reproduced. Since $\sum_i u_i = 0$, the least-squares line through the points (u_i, u_i^2) has intercept $\frac{1}{n}\sum_i u_i^2 = m_2$ and slope $\sum_i u_i^3 / \sum_i u_i^2 = m_3/m_2$. The fit of f at x_0 therefore falls short of $f(x_0)$ by $c\big((x_0 - \bar{x})^2 - m_2 - (m_3/m_2)(x_0 - \bar{x})\big) = c\, r(x_0)$.

Part (ii). At a design point, $\mathbb{E}\big(y_i - \hat{y}(x_i)\big) = f(x_i) - \mathbb{E}\hat{y}(x_i) = c\, r(x_i)$ by part (i). The residual vector is therefore the vector with entries $c\, r(x_i)$ plus the residuals that the errors alone would leave; the cross term has mean zero, and the residual sum of squares of the errors alone is $\varepsilon^{\top}(I - H)\varepsilon$, for the projection $I - H$ onto the orthogonal complement of the constant vector and the vector of design points. Because the errors are uncorrelated with variance σ^2, its mean is σ^2 times the trace of $I - H$, which is $n - 2$. It remains to show $\sum_i r(x_i)^2 = n\kappa$. Expanding the square of $r(x_i) = (u_i^2 - m_2) - (m_3/m_2)u_i$

and using $\sum_i (u_i^2 - m_2)^2 = n(m_4 - m_2^2)$, $\sum_i u_i(u_i^2 - m_2) = n m_3$ and $\sum_i u_i^2 = n m_2$,

$$\sum_i r(x_i)^2 = n(m_4 - m_2^2) - 2\frac{m_3}{m_2} n m_3 + \frac{m_3^2}{m_2^2} n m_2 = n\kappa.$$

Dividing the mean of the residual sum of squares by $n-2$ gives the formula for $\mathbb{E}S^2$.

Part (iii). Write $\delta = x_0 - \bar{x}$ and $q = 1 + 1/n + \delta^2/S_{xx}$, and let $Z = \big(\varepsilon_0 - \hat{y}(x_0) + \mathbb{E}\hat{y}(x_0)\big)/(\sigma\sqrt{q})$, which has mean zero and variance one by part (i) and Theorem 13.3(i). By part (i) the prediction error is $y_0 - \hat{y}(x_0) = c\, r(x_0) + \sigma\sqrt{q}\, Z$, so for a multiplier $\tau > 0$ the interval contains y_0 exactly when

$$\left| c\, r(x_0)/\sqrt{q} + \sigma Z \right| \le \tau S.$$

As $|x_0| \to \infty$, $r(x_0)/\delta^2 \to 1$ and $\sqrt{q}/|\delta| \to 1/\sqrt{S_{xx}}$, so $A = |c\, r(x_0)|/\sqrt{q}$ grows like $|c||\delta|\sqrt{S_{xx}}$ and tends to infinity because $c \neq 0$. The event above requires $\sigma|Z| \ge A/2$ or $\tau S \ge A/2$. By Chebyshev's inequality the first has probability at most $4\sigma^2/A^2$. The second has a probability that tends to zero because S is computed from the design points alone, so its distribution does not change with x_0, and it is finite. Both probabilities tend to zero. ∎

INVISIBLE INSIDE, LARGE OUTSIDE. The function r does two jobs. At the design points, $c\, r(x_i)$ is the pattern that the curvature leaves in the residuals, with mean square $c^2\kappa$ over the design, which adds $c^2 n\kappa/(n-2)$ to the expected residual variance S^2; beyond the design, $c\, r(x_0)$ is the error of the forecast. The two scales are very different. For a design spread evenly over a window of width h, in the limit of many points, $m_2 = h^2/12$, $m_3 = 0$ and $\kappa = h^4/180$, so the curvature shows in the residuals with root mean square $|c|h^2/\sqrt{180} \approx 0.075|c|h^2$, while the forecast half a window past the edge of the design, at $\delta = h$, is off by $|c|(h^2 - h^2/12) \approx 0.92|c|h^2$, twelve times as much, and one window past the edge by 29 times as much. A curvature too small to detect in the residuals is large where the forecast is made. This is the case that Box and Draper (1959) analyzed in their classic paper on design: a line fitted where the truth is quadratic. Their term in the residual sum of squares that makes the curvature detectable is $c^2 n\kappa$, and they concluded that when the line must describe the curve over a region, admitting the error of the form moves the best design from the ends of the region toward its middle, the opposite of the answer when the form is taken as right.

For scaling curves the sign is known. In logarithmic coordinates a law with a floor has second derivative $\alpha^2 E(L-E)/L^2$, which is positive, so a line fitted to it predicts too low a loss beyond the ladder: the error of the form makes the forecast optimistic. The curvature is small among small models, whose loss lies far above its floor, and largest where $L = 2E$; a ladder that approaches its floor is the one whose line misleads most. Fitting the right form helps only if it is the right form, and a different kind of error lies beyond every form.

REMARK 13.6 (A break beyond the data). Data below a scale U cannot rule out a change of slope above it. Take the smoothly broken power law

of Caballero et al. (2023) with one break, $g(x) = bx^{-c_0}\left(1 + (x/d)^{1/f}\right)^{-c_1 f}$, where now $x > 0$ is the scale itself, and let $U < d < x_0$. As the sharpness parameter f tends to zero, the last factor tends to 1 uniformly on $(0, U]$ and to $(x_0/d)^{-c_1}$ at x_0 (Exercise 13.4). So for every $\varepsilon > 0$ there is a broken law that differs from the plain power law bx^{-c_0} by less than a factor $1 + \varepsilon$ at every design point and, at x_0, by a factor within $1 + \varepsilon$ of $(x_0/d)^{-c_1}$, which is as large or as small as c_1 makes it. With normal errors the two laws give distributions of the data whose total variation distance tends to zero, so every test based on the data has power at most its level plus a term that tends to zero. A forecast at x_0 therefore rests on an assumption that the data cannot test: that no break lies between the ladder and the target.

BACKTESTING. The theorem and the proposition bound the two errors that can be computed. Whether a procedure's intervals are honest at a given distance is an empirical question about the curve, and it has an empirical answer.

Definition 13.7 (Backtest and coverage). Let a ladder of rungs $x_1 < \cdots < x_m$ be measured. A *backtest* of a forecasting procedure fits the procedure to the rungs $x_1, \ldots, x_k$ only and predicts each held-out rung x_j, $j > k$, with a nominal 95% interval. The *coverage* at distance $x_j - x_k$ is the probability that the observed outcome at x_j falls in its interval.

Coverage is estimated by counting over independent replicates of the ladder, with an exact binomial interval (Proposition A.17); backtests that share a ladder share its noise and are not independent. Three practices make a backtest informative. Fit several forms, at least a law with a floor and a form that allows a break, and report the spread of their forecasts as a lower bound on the uncertainty that the choice of form adds. Run a simulated control in which the assumed form holds exactly and the noise has the observed size: the positive control of Chapter 5, which separates the error of the interval's approximations, such as the delta method for a nonlinear fit (Theorems A.10 and A.11), from the error of the form. And state the *domain of validity*: the family, the protocol and the prompts of the ladder, the distance of the target in decades, and the coverage measured at that distance. A backtest can only measure coverage at distances shorter than the ladder, so a forecast beyond the backtested distance carries the assumption of Remark 13.6 in full. The forecasts that held in Section 13.3 had these features: smooth quantities, a ladder trained like its target, and, for GPT-4, a forecast written down before the result was known.

13.6 In the laboratory: a ladder of widths and a curve that turns

The first lab backtests forecasts on a smooth scaling curve, where parameter uncertainty and the error of the form are at work; the second fits short windows of a curve that turns, where the regime changes.

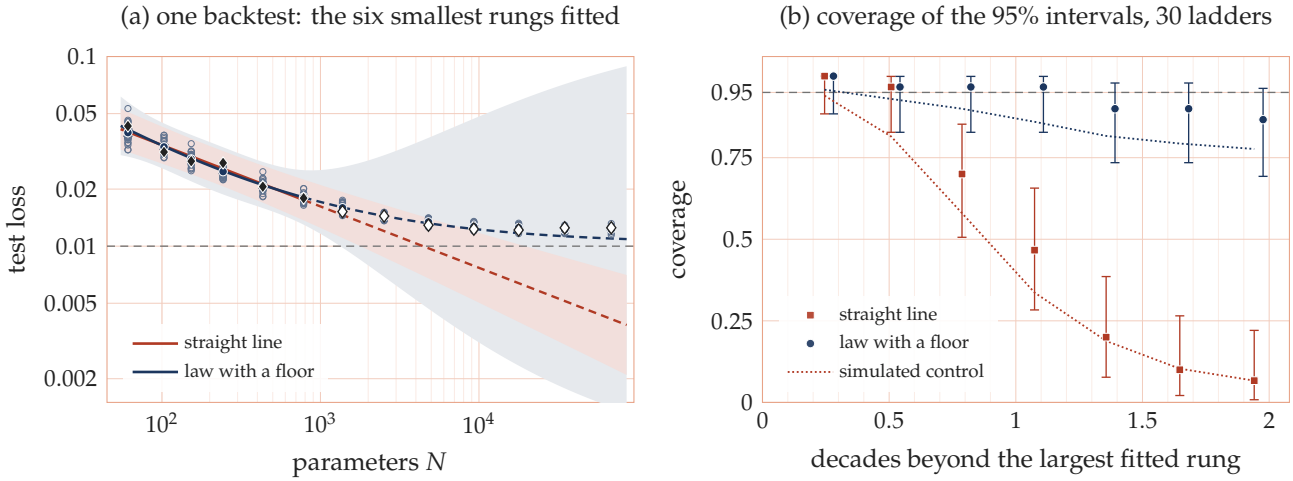


Figure 13.1. Lab 13.1: the teacher task, one pass over 65,536 examples, two hidden layers of 13 widths from 4 to 256, 30 ladders with seeds $1000r + j$ for ladder r and rung j. (a) Hollow marks: all 390 runs; filled circles: means over the ladders with 95% t-intervals; dashed grey line: the noise floor of the teacher, 0.01. The two forms are fitted to the six smallest rungs of ladder 0 (filled diamonds; hollow diamonds are its held-out rungs) and drawn solid over the fitted rungs and dashed beyond, with bands for the 95% prediction interval of one run: vermilion for the straight line, blue for the law with a floor. (b) The same window in all 30 ladders: the fraction of held-out runs inside their intervals, with 95% Clopper–Pearson intervals over ladders (offset for legibility); dotted lines: the same procedures on 600 simulated ladders in which a law with a floor holds exactly.

LAB 13.1 · BACKTESTING A SCALING CURVE

SETUP. The teacher task under the online protocol: one pass over 65,536 fresh examples, Adam at learning rate 0.01 (the best of 0.001, 0.003 and 0.01 at every width in a pilot) with a cosine schedule, batch 64. A ladder of 13 networks with two hidden layers of width 4 to 256 (61 to 68,353 parameters, 3.05 decades); 30 ladders, every run with its own seeds (390 runs). Each backtest fits the 5, 6, 7 or 8 smallest rungs of one ladder, by a straight line in $(\ln N, \ln L)$ with the interval of Corollary 13.4 and by the law $\ln L = \ln(E + AN^{-\alpha})$ with a delta-method interval, and predicts every larger rung: 120 backtests and 780 forecasts per form. A simulated control repeats both on ladders drawn from a law with a floor. About three minutes.

PREDICT FIRST. Fitted to the six smallest rungs, by what factor will the line miss the largest? At what distance will its intervals stop covering? Will the law with a floor do better, and at what price in the width of its intervals?

LOOK FOR. Where the curve bends toward its floor, relative to the fitted rungs; the coverage of each form against the distance; the width of the intervals; the curvature inside the window against the error beyond it.

SCRIPT

`13-extrap-backtest.py`

decades beyond the window	0.25	0.51	0.79	1.07	1.36	1.65	1.94
straight line							
error, observed	−0.08	−0.16	−0.29	−0.44	−0.63	−0.81	−1.03
its 95% half-width	0.03	0.03	0.04	0.04	0.05	0.06	0.07
predicted, $-c\,r(x_0)$	−0.07	−0.15	−0.25	−0.38	−0.53	−0.71	−0.91
interval half-width	0.28	0.32	0.37	0.42	0.47	0.53	0.59
law with a floor							
error, observed	+0.02	+0.02	−0.02	−0.06	−0.13	−0.19	−0.27
interval half-width	0.39	0.51	0.60	0.68	0.77	0.86	0.94

Table 13.1. Lab 13.1, window of the six smallest rungs: errors of the forecast of $\ln L$ (forecast minus observed, in natural logarithms) averaged over the 30 ladders, with the 95% half-width of that average, and median half-widths of the 95% prediction intervals of one run. The row $-c\,r(x_0)$ is the error that Proposition 13.5 predicts from the curvature $c = 0.028$ of a parabola fitted to the mean of $\ln L$ over the six rungs.

WHAT HAPPENED. The mean test loss falls from 0.0397 ± 0.0017 at 61 parameters to 0.0122 ± 0.0001 at 34,753 and 0.0123 ± 0.0001 at 68,353 (95% t-intervals over the 30 ladders), above the teacher's noise floor of 0.01 by the part that 65,536 examples cannot remove.[7] In logarithmic coordinates the local slope is −0.33 to −0.37 over the first four rungs and between −0.04 and +0.01 over the last four: the curve flattens toward its floor between about 10^3 and 10^4 parameters, beyond the six smallest rungs. The spread of $\ln L$ over runs falls from 0.11 at the smallest width to 0.03 at the largest.

In ladder 0 (Figure 13.1a), the line fitted to the six smallest rungs predicts a loss of 0.0041 at 68,353 parameters, with a 95% interval of 0.0023 to 0.0074 that lies wholly below the teacher's noise floor; the run there scored 0.0125. The law with a floor predicts 0.0110 with an interval of 0.0014 to 0.085, a factor of 60 wide: honest, and of little use. Across ladders its fitted floor is barely identified, with median 0.0107 and quartiles 0.0001 and 0.0139 (Chapter 11).

Over all ladders and windows, the line's intervals cover 91% of the held-out runs within 0.55 decades of the window (87–96%, a 95% bootstrap interval over whole ladders), 41% (34–48%) between 0.55 and 1.2 decades, and 21% (13–29%) beyond; the law with a floor covers 99%, 93% and 88% (82–94%). The simulated control reproduces both patterns (Figure 13.1b): for the window of six rungs the line's coverage falls from 94% to 7% there too, so its failure is a failure of the form, and the law with a floor falls from 96% to 78%, further than on the real ladders, so its shortfall at long range comes from its delta-method interval, linear in parameters that six rungs barely determine, and not from a departure of the teacher's curve from the form.

Table 13.1 tests Proposition 13.5. A parabola fitted to the mean of $\ln L$ over the six smallest rungs has curvature $c = 0.028$ per unit of $(\ln N)^2$ (95% interval over ladders 0.009–0.048), and with it the proposition predicts an error of −0.38 at 1.07 decades and −0.91 at 1.94, against -0.44 ± 0.04 and

[7] In the notation of Chapter 11, a curve in N at fixed D has the floor $E + A_D D^{-\alpha_D}$, not E, and the law with a floor fits that sum.

-1.03 ± 0.07 observed. The point estimate accounts for 84–97% of the error at the seven distances, and over the interval of the curvature the prediction brackets the observed error at every distance, here and for the windows of seven and eight rungs. The line's half-width grows from 0.28 to 0.59 while its error grows from 0.08 to 1.03, and the error passes the half-width near one decade, where the coverage falls through one half. Inside the window the same curvature is invisible. It adds $c^2 n\kappa/(n-2) = 0.0005$ to the residual variance, against 0.0056 for the spread of runs; from a single ladder its standard error is 0.047, larger than the curvature itself; and a t-test of a quadratic term rejects the line at level 0.05 in 3 of the 30 ladders. For the window of five rungs the curvature is not determined even by all 30 ladders (interval -0.023 to 0.047), and the proposition predicts nothing useful.

LAB 13.2 · A CURVE THAT TURNS

SCRIPT
`13-extrap-ushape.py`

SETUP. Digits with 20% of the training labels replaced by random labels; a network with one hidden layer of width 2 to 2048 (160 to 153,610 parameters), Adam at learning rate 10^{-3}, batch 64, 200 epochs, no regularization; six runs per width, run s with seed s for initialization, order, split and label noise (108 runs). Two windows are fitted by straight lines in ($\ln N$, ln error) and extrapolated with the intervals of Corollary 13.4: window A on widths 4–12, window B on widths 16–64. About three minutes.

PREDICT FIRST. The training set has 1,258 images, about 226 of them with wrong labels. At what width will the networks fit every training label? What will the test error do as the width grows past that point?

LOOK FOR. The two descents and the peak between them; the training error at the peak; the local slope over short windows, and how its sign depends on where the window sits.

WHAT HAPPENED. The mean test error falls from $50.2 \pm 7.5\%$ at width 2 to $10.4 \pm 2.1\%$ at width 8 and $10.5 \pm 1.2\%$ at width 12, rises to $17.1 \pm 1.6\%$ at width 64, where the networks still miss 1.8% of their training labels, and falls again once they fit nearly all of them, from width 96 on: to $13.0 \pm 1.4\%$ at width 512 and $12.5 \pm 0.9\%$ at 2048, still above the first minimum, as the largest PaLM model was still below the smallest on four of the six U-shaped tasks of Finding 13.2 (Figure 13.2a). This is the *double descent* of Belkin et al. (2019) and Nakkiran et al. (2020), with its peak where the network can just fit the noisy training set.[8]

Window B is a miniature inverse-scaling trend: over widths 16 to 64 the error rises with slope 0.29 per unit of $\ln N$ (95% interval 0.19–0.38), a factor of 1.9 per decade, and no test inside the window would doubt it. Extrapolated, it predicts 20.7% at width 128, with a 95% interval for one run of 15.5–27.6%, and 45.7% (28.7–72.9%) at width 2048; the runs scored 15.0% and 12.5% on average, and of the 42 runs beyond the window only 7 fall inside their intervals, all at the two widths next to it. Window A,

[8] The second volume of this series, *The Hydrogen Atoms of Deep Learning*, derives the double-descent curve of least squares exactly in its first chapter.

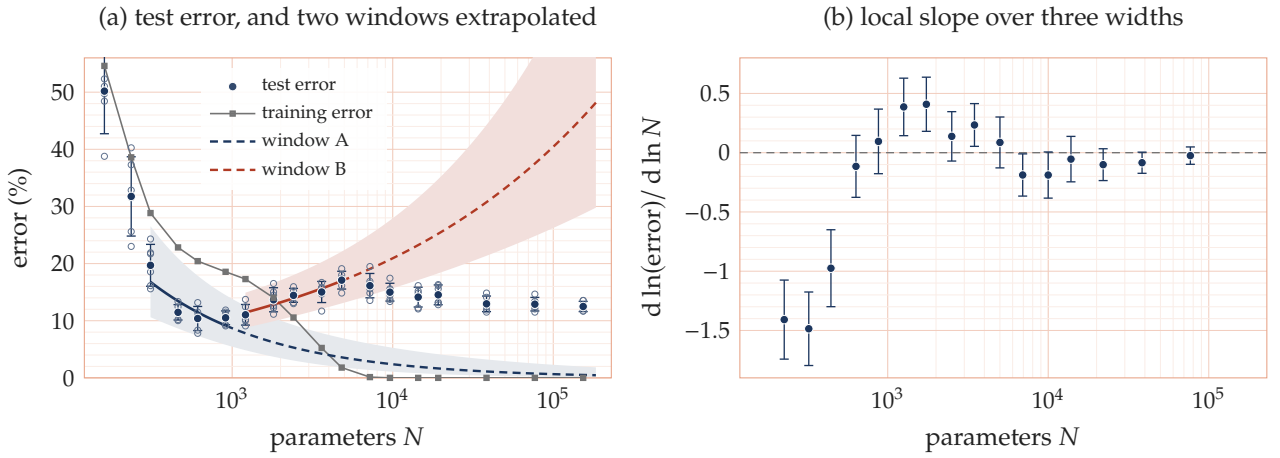


Figure 13.2. Lab 13.2: digits with 20% of the training labels random, one hidden layer of widths 2 to 2048, six runs per width (seeds 0–5). (a) Hollow marks: single runs; filled circles: means with 95% t-intervals; small squares: mean training error on the noisy training labels. Straight lines in ($\ln N$, $\ln$ error) fitted to all runs in window A (widths 4–12, blue) and window B (widths 16–64, vermilion), solid over the window and dashed beyond, with bands for the 95% prediction interval of one run (window B's band is cut by the frame). (b) The slope of a line fitted to the runs of three consecutive widths, plotted at their geometric mean, with 95% intervals.

with slope −0.56 (−0.79 to −0.34), extrapolates the first descent to 3.6% at width 64 and 0.5% at width 2048; 5 of the 72 runs beyond it are covered, all at the next width. A law with a floor fitted to window A does no better: it predicts 10.3% at every larger width, below the peak and below the second descent. The local slope (Figure 13.2b) is −1.41 (95% interval −1.74 to −1.07) near 230 parameters, positive between about 1,250 and 3,500 parameters (+0.41, interval +0.18 to +0.64, near 1,740), negative again near 7,000 (−0.19, interval −0.37 to −0.01), and within 0.11 of zero beyond 13,000. A short window estimates a local slope, which is all that a short window can estimate, and its sign depends on where the window sits.

The two labs reproduce the three errors of a forecast: the line's intervals widen with distance as Theorem 13.3 says, its error from curvature grows as Proposition 13.5 says and escapes its intervals within a decade, and a change of regime defeats every form fitted before it. The second miniature reproduces the shape of the post-mortem, not its mechanism: its peak comes from fitting noisy labels, not from a distractor task.

13.7 Protocol

PROTOCOL 13.1 · BEFORE YOU EXTRAPOLATE

- ☐ State the target: the scale, the quantity and the protocol of the run to be forecast; write the forecast and its interval down, with the date, before the run finishes.
- ☐ Design the ladder: span as many decades as the budget allows, put most runs at the top and the bottom of the range and a few in

between to test the form, and repeat some rungs with new seeds to measure the noise apart from the curvature.

- ☐ Train every rung by the protocol of the target: the same data, schedule matched to each horizon, hyperparameters scaled by a fixed rule (Chapter 12).
- ☐ Forecast a smooth quantity, such as a loss, the log-likelihood of the correct answer or an average over tasks, and reach a thresholded metric through it (Chapter 14).
- ☐ Fit at least two forms, one with a floor and one that allows a break, with intervals for one new run that include the uncertainty of the parameters (Theorem 13.3; Theorems A.3 and A.10); report the spread between forms.
- ☐ Backtest each form on held-out upper rungs over independent ladders, and report coverage and error against distance beside a simulated control in which the form holds.
- ☐ State the domain of validity: family, protocol, prompts, range of the ladder, the distance of the target, and the coverage measured at that distance; say that a target beyond the backtested distance assumes that no break intervenes.
- ☐ After the run, publish the comparison of forecast and result, whichever way it came out.

The protocol bounds the two errors that can be computed and checks the error of the form over the distances a backtest can reach. It does not protect against a change of regime beyond the ladder, which no data below the change can reveal (Remark 13.6); a backtest at one decade says nothing about three; and it does not make a forecast of one task's thresholded metric reliable, because a small change of prompt or harness can change the trend itself (Section 13.3). It is also expensive: replicate ladders multiply the cost of a ladder, whose cost is dominated by its top rungs. When the budget allows one run per rung, report the residual spread of the fit, which mixes the noise with the curvature (Proposition 13.5), as an upper bound on the noise, give the forecast as a range across forms, and prefer decisions that need only a ranking at small scale: in a study of 25 pretraining corpora, the ranking of models at a single small size picked the better corpus at a scale of one billion parameters in about 80% of comparisons, and none of eight scaling-law methods did better for the same compute (Magnusson et al. 2025).

Exercises

13.1 CONSOLIDATION. (Intervals on the loss.) (a) Show that an interval $\hat{y} \pm h$ for $\ln L$ is the interval $[\hat{L}e^{-h}, \hat{L}e^{h}]$ for L, with $\hat{L} = e^{\hat{y}}$, and find the factor that the half-width 0.59 of Table 13.1 allows. (b) Suppose that $\ln L$ is normal with mean μ and variance s^2. Show that e^{μ} is the median of L and $e^{\mu + s^2/2}$ its mean, and say which one $e^{\hat{y}}$ estimates.

13.2 CONSOLIDATION. (Three scales.) Three runs can be placed on a ladder spanning $[L, U]$ in logarithmic scale, to forecast at $x_0 = L + 2(U - L)$. (a) Show that two runs at U and one at L attain the bound of Theorem 13.3(iii), and compare with the design $L, (L+U)/2, U$. (b) A run at U costs ten runs at L, and the budget is 31 runs at L. Which allocation between the two ends minimizes $\operatorname{Var} \hat{y}(x_0)$, and why does it differ from part (a)?

13.3 CONSOLIDATION. (Residuals and bias.) (a) Show that $\kappa \geq 0$ in Proposition 13.5, with equality if and only if the design has at most two distinct points, and conclude that a two-point design cannot reveal curvature. (b) Show that $\kappa = h^4/180$ for a design spread evenly over a window of width h (in the limit of many points), and check the ratios 12 and 29 of Section 13.5. (c) Show that r has two real roots, and interpret them.

13.4 CONSOLIDATION. ⋆ (Breaks are invisible.) For $g_f(x) = bx^{-c_0}\big(1 + (x/d)^{1/f}\big)^{-c_1 f}$ with $0 < U < d < x_0$, prove that $\sup_{0<x\leq U} |\ln g_f(x) - \ln(bx^{-c_0})| \to 0$ and $\ln g_f(x_0) - \ln(bx_0^{-c_0}) \to -c_1 \ln(x_0/d)$ as $f \to 0$. Then show that for normal errors of variance σ^2 in $\ln L$ at n design points, the total variation distance between the distributions of the data under the two laws is at most $\|\mu - \mu'\|/(\sigma\sqrt{2\pi})$, where μ and μ' are the two vectors of means of $\ln L$. *(Hint: use $\ln(1+s) \leq s$ for the first limit, and for the bound on total variation reduce to one dimension along $\mu - \mu'$.)*

13.5 LAB. (The ends of the ladder.) In `13-extrap-backtest.py`, replace the window of six rungs by a design with three runs at each of its two end rungs, drawn from three ladders. Predict from Theorem 13.3(ii)–(iii) how the width of the line's interval changes, then measure its coverage. Why can the law with a floor not be fitted to this design?

13.6 LAB. (Weights and resampling.) The spread of $\ln L$ over runs falls from 0.11 to 0.03 along the ladder. Refit the line by weighted least squares with weights equal to the inverse variance of each rung, and replace the delta-method interval of the law with a floor by a parametric bootstrap (Theorem A.3). Observe the coverage of both in the backtest and in the simulated control, and explain which changes.

13.7 LAB. (Less noise, more noise.) Run `13-extrap-ushape.py` with 10% and with 40% of the training labels random. Predict where the peak moves and how high it rises before running, then check whether window

B, refitted between the first minimum and the peak, still predicts an error that rises without end.

13.8 DESIGN. (A registered forecast.) A team will train a model with 10^{24} FLOPs and wants its validation loss and its accuracy on one multiple-choice benchmark, with 1% of that compute for the ladder. Design the ladder (sizes, tokens, seeds), the forms, the backtest and the registration document: what is written down before the run, and what result would count as a failed forecast.

13.9 DESIGN. (Testing for inverse scaling.) You have one family of six sizes and a task of 500 items. Design an evaluation that decides whether the task scales inversely over the range measured: prompts, items, checkpoints or seeds, the estimate of the slope and its interval, and the sentence that states what the result does and does not say about larger models.

13.10 CRITIQUE. (Two decades past the data.) A plot shows six model sizes spanning one decade, a line fitted in log–log coordinates, and the line continued to a model a hundred times larger, labelled "projected", without an interval. Using Theorem 13.3, compute a lower bound on the standard deviation of the forecast relative to the noise of one run, and list what the plot would need to be an honest forecast.

13.11 CRITIQUE. (Scaling hurts.) A paper evaluates four models of one family, with 1, 3, 7 and 13 billion parameters, on 200 items with one prompt; the accuracies are 61, 58, 57 and 54%, and the paper concludes that scaling hurts the ability measured. Audit the claim: the binomial noise of 200 items, one model per size, one prompt, and the step to larger models.

13.12 RESEARCH. (Detecting a break before it.) Tests of structural change find a break inside the data (Andrews 1993); a forecast needs to know whether a break lies beyond it. Formulate the problem of bounding, from a ladder below U, the probability of a break between U and x_0 under a prior over breaks, and propose observable signals that might precede a break in deep learning, such as a drifting local slope, a growing spread over seeds, or the behavior of a subtask that acts as a distractor. Open: whether such signals predict the U-shaped tasks of Finding 13.2 from the smaller models alone.

Notes and References

Scaling curves were forecasts from the start. Hestness et al. (2017) drew learning curves in four domains with three regions, small-data, power-law and irreducible-error, so that the power law described only the middle. Kaplan et al. (2020) saw no deviation from their power laws, and noted that two of them, extrapolated, would contradict each other near 10^4 petaflop-days, so that at least one had to break before that point. Hoffmann et al. (2022) tested their forecast of the compute-optimal allocation by training

Chinchilla, and the GPT-4 report turned such a forecast into a registered prediction (OpenAI 2023).

The statistics of forecasting with a fitted line is classical: prediction intervals in regression are in every text (Seber and Lee 2003), the design that minimizes the variance of an extrapolated prediction goes back to Elfving (1952) and, for polynomials of every degree, to Hoel and Levine (1964), and Box and Draper (1959) showed that admitting an error of the form changes the best design. Econometrics treats changes of regime inside a series as structural breaks, with tests for a break at an unknown point (Andrews 1993).

In deep learning, Alabdulmohsin et al. (2022) argued that scaling laws should be judged by their error on held-out larger scales rather than by their fit, and released a benchmark of learning curves for doing so; Caballero et al. (2023) used it for the broken power law. For downstream tasks, Lourie et al. (2025) documented how often trends are irregular, while Gadre et al. (2025), Bhagia et al. (2025) and Ruan et al. (2024) showed how forecasts of averages and of losses can succeed; Magnusson et al. (2025) found that for choosing pretraining data a comparison at one small scale is hard to beat. Double descent, the non-monotone curve of Lab 13.2, was named by Belkin et al. (2019) and studied in deep networks, in model size, training time and data, by Nakkiran et al. (2020).

TRANSLATING THE LITERATURE. McKenzie et al.'s inverse scaling is a negative local slope of a task's performance against the logarithm of training compute over the range measured; Wei et al.'s U-shaped and McKenzie et al.'s inverted-U scaling are curves whose local slope changes sign inside it. GPT-4's $L(C) = aC^b + c$ is our $E + AC^{-\gamma}$ with $c = E$, $a = A$ and $b = -\gamma$. In the broken power law of Caballero et al. (2023), $y = a + bx^{-c_0} \prod_i (1 + (x/d_i)^{1/f_i})^{-c_i f_i}$, the constants a, b and c_0 are our E, A_N and α_N when x is N; c_i is the change of slope at the ith break, d_i its location and f_i its sharpness parameter (smaller is sharper), and Remark 13.6 uses one break with $f \to 0$. What Alabdulmohsin et al. (2022) call the extrapolation loss is the error of a backtest; a ladder of Bhagia et al. (2025) is our design; the token multiplier M of Gadre et al. (2025) is D/N; and the irreducible-error region of Hestness et al. (2017) is the floor E.

References

Alabdulmohsin, I., B. Neyshabur, and X. Zhai (2022). "Revisiting Neural Scaling Laws in Language and Vision". In: *Advances in Neural Information Processing Systems 35*. arXiv:2209.06640.

Andrews, D. W. K. (1993). "Tests for Parameter Instability and Structural Change with Unknown Change Point". In: *Econometrica* 61.4, pp. 821–856.

Belkin, M., D. Hsu, S. Ma, and S. Mandal (2019). "Reconciling Modern Machine-Learning Practice and the Classical Bias–Variance Trade-off". In: *Proceedings of the National Academy of Sciences* 116.32, pp. 15849–15854.

Bhagia, A. et al. (2025). "Establishing Task Scaling Laws via Compute-Efficient Model Ladders". In: *Conference on Language Modeling (COLM)*. First posted December 2024 as arXiv:2412.04403.

Box, G. E. P. and N. R. Draper (1959). "A Basis for the Selection of a Response Surface Design". In: *Journal of the American Statistical Association* 54.287, pp. 622–654.

Caballero, E., K. Gupta, I. Rish, and D. Krueger (2023). "Broken Neural Scaling Laws". In: *International Conference on Learning Representations (ICLR)*.

Elfving, G. (1952). "Optimum Allocation in Linear Regression Theory". In: *Annals of Mathematical Statistics* 23.2, pp. 255–262.

Gadre, S. Y. et al. (2025). "Language Models Scale Reliably with Over-Training and on Downstream Tasks". In: *International Conference on Learning Representations (ICLR)*. First posted March 2024 as arXiv:2403.08540.

Ganguli, D., D. Hernandez, L. Lovitt, et al. (2022). "Predictability and Surprise in Large Generative Models". In: *Proceedings of the 2022 ACM Conference on Fairness, Accountability, and Transparency (FAccT)*, pp. 1747–1764.

Hestness, J. et al. (2017). *Deep Learning Scaling is Predictable, Empirically*. Preprint, arXiv:1712.00409.

Hoel, P. G. and A. Levine (1964). "Optimal Spacing and Weighting in Polynomial Prediction". In: *Annals of Mathematical Statistics* 35, pp. 1553–1560.

Hoffmann, J. et al. (2022). "An Empirical Analysis of Compute-Optimal Large Language Model Training". In: *Advances in Neural Information Processing Systems 35*. First posted as "Training Compute-Optimal Large Language Models", arXiv:2203.15556.

Kaplan, J. et al. (2020). *Scaling Laws for Neural Language Models*. Preprint, arXiv:2001.08361.

Lin, S., J. Hilton, and O. Evans (2022). "TruthfulQA: Measuring How Models Mimic Human Falsehoods". In: *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 3214–3252.

Llama Team, AI @ Meta (2024). *The Llama 3 Herd of Models*. Preprint, arXiv:2407.21783.

Lourie, N., M. Y. Hu, and K. Cho (2025). "Scaling Laws Are Unreliable for Downstream Tasks: A Reality Check". In: *Findings of the Association for Computational Linguistics: EMNLP 2025*, pp. 16167–16180.

Magnusson, I. et al. (2025). "DataDecide: How to Predict Best Pretraining Data with Small Experiments". In: *Proceedings of the 42nd International Conference on Machine Learning*. Vol. 267. Proceedings of Machine Learning Research, pp. 42487–42502.

McKenzie, I., A. Lyzhov, A. Parrish, A. Prabhu, A. Mueller, N. Kim, S. Bowman, and E. Perez (2022). *The Inverse Scaling Prize*. Contest announcement and rules; contest opened 27 June 2022. URL: https://github.com/inverse-scaling/prize.

McKenzie, I. R. et al. (2023). "Inverse Scaling: When Bigger Isn't Better". In: *Transactions on Machine Learning Research*. arXiv:2306.09479.

Nakkiran, P., G. Kaplun, Y. Bansal, T. Yang, B. Barak, and I. Sutskever (2020). "Deep Double Descent: Where Bigger Models and More Data Hurt". In: *International Conference on Learning Representations (ICLR)*.

OpenAI (2023). *GPT-4 Technical Report*. Preprint, arXiv:2303.08774.

Ruan, Y., C. J. Maddison, and T. Hashimoto (2024). "Observational Scaling Laws and the Predictability of Language Model Performance". In: *Advances in Neural Information Processing Systems 37*. arXiv:2405.10938.

Seber, G. A. F. and A. J. Lee (2003). *Linear Regression Analysis*. 2nd ed. Hoboken, NJ: Wiley.

Wei, J., N. Kim, Y. Tay, and Q. V. Le (2023). "Inverse Scaling Can Become U-Shaped". In: *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*. First posted November 2022 as arXiv:2211.02011, pp. 15580–15591.

Part V

Measurement

A measurement is a number produced by an instrument. Before a curve can be read as a fact about models, the reader has to know what the instrument does to it: how the metric transforms the quantity beneath it, how many items it rests on, and how its settings move it.

The Metric Makes the Phenomenon

14

Emergent abilities and the smooth change beneath them

THE CLAIM *"Some abilities of large language models emerge abruptly and unpredictably with scale."*

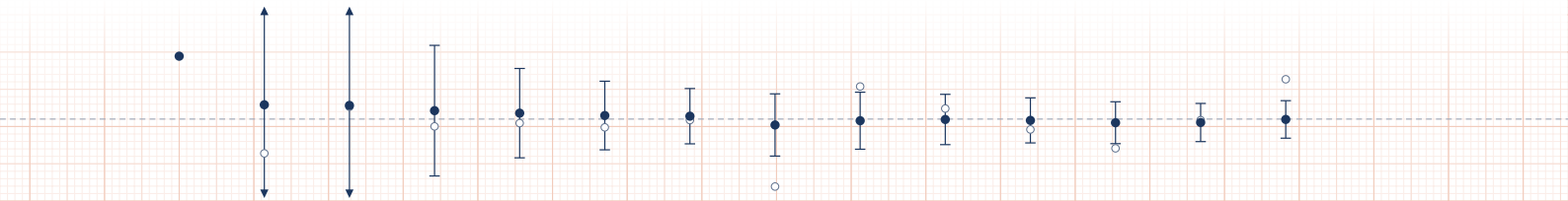
Wei et al. (2022), drawing on Ganguli et al. (2022) and the BIG-bench collaboration; the received reading of their survey

THE VERDICT Artifact, in large part: most abrupt curves were exact-match or multiple-choice readings, on few items, of a quantity that changes smoothly; a few transitions are sharp under every metric.

IN THIS CHAPTER. In 2022 Wei et al. collected scaling curves on which a family of language models scored at chance up to some size and far above it beyond, and called such abilities emergent: present in larger models, absent in smaller ones, and not predictable from them. Most of these curves were exact-match or multiple-choice scores read on tens to thousands of items, and the same outputs scored by a continuous metric, or measured with more resolution, change smoothly; the abruptness came largely from the metric. The method treats a metric as a transformation of a latent quantity: exact match on L tokens turns a power law in the per-token log error into a Gumbel curve in the logarithm of model size, whose location moves with L while its width does not; a count of zero bounds a success rate only below about $3/m$, and an exact-match count keeps a vanishing fraction of the information in per-token scores. In the laboratory, exact match on ten digits emerges in a family of digit classifiers whose per-digit cross-entropy falls steadily, a curve fitted to per-digit accuracy predicts it, fifty items scored digit by digit show what five thousand scored by exact match cannot, and a sparse-parity network shows what a transition looks like when the metric does not make it.

14.1 The claim: abilities that appear with scale

By 2020 the loss of a language model had become predictable: Kaplan et al. (2020) found the test loss falling as a power law in parameters, data, and compute, over more than seven orders of magnitude. Tasks were another matter. GPT-3 was trained in eight sizes, from 125 million to 175 billion parameters, all on 300 billion tokens, and asked to do arithmetic from a few worked examples in its prompt (Brown et al. 2020). Each of ten tests had 2,000 random problems, and an answer counted only if it was exactly right. The largest model added three-digit numbers correctly 80.4% of the time; the 13-billion-parameter model, the second largest, solved two-digit



additions and subtractions about half the time and every other operation less than 10% of the time.

Ganguli et al. (2022) drew the general lesson, under the heading of abrupt, specific capability scaling. On three-digit addition, every GPT-3 model below 6 billion parameters was right less than 1% of the time, the 13-billion model 8% and the largest 80%; Gopher on a multitask examination and a family of program-synthesis models showed similar jumps. Their explanation was that a single task is a tiny slice of the distribution that a model predicts, so that it can change quickly while the loss over the whole distribution changes smoothly, as the weather of one day is harder to predict than the average of a season.[1] The BIG-bench collaboration, 450 authors at 132 institutions, built 204 tasks to probe what models of millions to hundreds of billions of parameters could do (Srivastava et al. 2023). To flag tasks whose score stays flat across several model sizes and then jumps, it defined a statistic, *breakthroughness*,[2] and found that the tasks with breakthrough behavior often involved several steps or brittle metrics.

Wei et al. (2022) gave the phenomenon its name and its definition: an ability is *emergent* if it is "not present in smaller models but is present in larger models." Such abilities, they argued, cannot be predicted by extrapolating the performance of smaller models; on a plot of performance against scale they show performance near random up to a threshold and far above random after it, which the authors compared to a phase transition.

> FINDING 14.1 (Wei et al. 2022, Figure 2, Table 1 and Appendix E). Eight few-shot tasks, evaluated on five model families (LaMDA, GPT-3, Gopher, Chinchilla, and PaLM), had near-random performance up to a scale and well-above-random performance after it. On modified arithmetic from BIG-bench, performance jumped at $2 \cdot 10^{22}$ training FLOPs (13 billion parameters) for GPT-3 and at 10^{23} (68 billion) for LaMDA; on a multitask examination (MMLU), models of about 10^{22} FLOPs were no better than guessing and models of $3\text{–}5 \cdot 10^{23}$ FLOPs (70 to 280 billion parameters) far better; on word-in-context judgments only PaLM at 540 billion parameters rose above chance. Prompting strategies such as chain of thought helped only above a scale, about 10^{23} FLOPs for LaMDA. The authors classified 210 BIG-bench tasks by hand, two of them agreeing on every task labeled emergent: 67 emergent, 58 smoothly increasing, 45 flat, and 40 in none of these classes.

The paper was published in *Transactions on Machine Learning Research* with a survey certification, and the word spread: by November 2022 Wei listed 137 emergent abilities on his blog, 67 of them BIG-bench tasks and 51 subjects of the multitask examination (Wei 2022). Its section on emergent risks argued that harmful behaviors could appear with scale in the same way. The received claim was the one at the head of this chapter: some abilities emerge abruptly and unpredictably as models grow.

WHAT WAS RIGHT. The curves were real readings of real scores, and the authors anticipated the objection this chapter develops. They wrote that the scale of emergence depends on data and training and is not an

[1] Ganguli et al. wrote for policymakers as well as researchers: their paper argues that the combination of predictable loss and unpredictable capabilities shapes how such models are developed and deployed.

[2] For scores $y_1, \ldots, y_n$ at increasing scales, breakthroughness is the overall improvement $\operatorname{sgn}(\arg\max_i y_i - \arg\min_i y_i)(\max_i y_i - \min_i y_i)$ divided by the square root of the median of the squared increments $(y_{i+1} - y_i)^2$: large when most steps are flat and one is not.

immutable property of an ability; that exact match on long targets may disguise compounding incremental improvements as emergence; and, in an appendix, that the cross-entropy of the targets improved at small scales on all six BIG-bench tasks they examined, while the scores stayed at random (Wei et al. 2022). They also found emergence on three generative tasks under every metric BIG-bench provided, including BLEU and ROUGE, which give partial credit, and on classification tasks where the answer is a single choice. And for many tasks the all-or-nothing score is the one that matters: an arithmetic answer with one wrong digit is wrong (Wei 2023).

14.2 The experiment: a handful of models and one score

Stripped of its details, each curve of Finding 14.1 comes from the following design.

EXPERIMENTAL DESIGN · A SCALING CURVE OF A FEW-SHOT TASK; BROWN ET AL. 2020, WEI ET AL. 2022

QUESTION	From what scale on can a model family do the task from a few examples in its prompt?
TREATMENT	model scale: parameters and training compute, within a family trained on the same data
LEVELS	GPT-3: eight sizes from 125M to 175B parameters; Wei et al.: five families spanning about 10^{18} to 10^{24} training FLOPs
TUNED	nothing per task: one prompt with a few worked examples
HELD FIXED	data and recipe within a family; prompt, examples, greedy decoding, and the scoring rule
RESPONSE	exact match of the generated answer, BLEU, or multiple-choice grade, on 2,000 problems per arithmetic test or on the 32 to several thousand items of a BIG-bench task
REPLICATES	one model per size; one evaluation per model and task
ANALYSIS	score against training FLOPs on a logarithmic axis; emergent if near random below a scale and well above it beyond, judged by eye

What could this design show? Within its scope it is sound: the models and prompts are fixed, and the score is a deterministic function of the outputs. It cannot show where the abruptness comes from. A rule that gives credit only when every token of an answer is right, or only when the right option outranks every wrong one, can turn a gradual change in the model into an abrupt change in the score, and with one score per model the design cannot tell the two apart. Three further features matter, and the post-mortem turns on each. The points have no intervals, and a reading of zero on m items is consistent with every success rate below about $3/m$ (Proposition A.17): 0.15% for 2,000 arithmetic problems, 9% for the 32 items of BIG-bench's logical-arguments task, whose noise Wei et al. noted. The sizes are far apart: GPT-3's two largest models differ by a factor of 13.5 in

parameters, and a curve that rises within that factor has no point on its rise.[3] And "near random" describes a floor, the chance level, on which a thresholded score sits until the model has improved enough to leave it.

14.3 The post-mortem: the metric, the items, and the spacing

CHANGING THE METRIC. Schaeffer et al. (2023) proposed that emergent abilities appear only under metrics that scale a model's per-token error rate nonlinearly or discontinuously.[4] In their model, if the per-token cross-entropy falls as a power law in the number of parameters and the tokens of an answer are right independently, then exact match on L tokens is the exponential of $-L$ times a power law, a curve that looks sharp on a logarithmic axis, while the number of wrong tokens, a token edit distance, falls smoothly; Proposition 14.3 makes the model exact. They tested it on the four GPT-3-family models then available to the public, from 350 million to 175 billion parameters, with two-digit multiplication and four-digit addition in two-shot prompts. Scored by exact match, answers of four or five digits showed the familiar jump; scored by token edit distance, the same outputs improved smoothly and predictably with scale. With additional test problems, every model scored above chance even by exact match, on a curve that fell about geometrically with the length of the answer, as their model predicts.

A META-ANALYSIS. Across BIG-bench, Schaeffer et al. computed breakthroughness for every combination of task, metric, and model family: at most 5 of the 39 preferred metrics produced emergence. Among the emergent abilities in Wei's hand-annotated list only 4 metrics occurred, and two of them accounted for more than 92%: exact string match and multiple-choice grade, which counts an item as correct when the correct option receives the highest probability. For LaMDA, whose outputs are published with BIG-bench, they rescored the tasks on which it showed emergence under multiple-choice grade with the Brier score, and the emergence disappeared. The BIG-bench authors had seen the pattern: the log-likelihood of the target improved fairly gradually even on the tasks of highest breakthroughness, and smoother metrics revealed partial progress (Srivastava et al. 2023).

EMERGENCE MADE TO ORDER. If the metric makes the jump, then a metric should be able to make one where nobody has seen one. Schaeffer et al. scored shallow autoencoders on CIFAR-100 by the fraction of images reconstructed with squared error below a cutoff, transformers classifying Omniglot characters by whether all of L characters were right, and LeNet networks on MNIST by whether all of K digits were right. Under the ordinary metrics all three families improved smoothly with size; under the thresholded ones each showed what looked like an emergent ability.[5] Lab 14.1 repeats the last experiment on the digits.

RESOLUTION. Hu et al. (2024) attacked the floor directly. Their evaluation, PassUntil,[6] samples answers to an item until r of them pass and estimates

[3] Wei et al. note that no single measure captures scale: Chinchilla has a quarter of Gopher's parameters and uses similar training compute.

[4] The paper received an outstanding paper award at NeurIPS 2023.

[5] Schaeffer et al. also separate emergence from grokking: grokking is studied in one model over training steps, with separate training and test curves, and emergence across the models of a family.

[6] PassUntil needs a task that random guessing cannot pass; Hu et al. leave multiple choice, with its chance level, to future work.

the pass probability by r divided by the number of samples, the maximum-likelihood estimate under the negative binomial distribution, with up to 10^5 samples per item. On two families of models from 0.03 to 2.4 billion parameters they measured pass rates of order 10^{-5} in the small models and found that $\ln(-\ln \text{pass rate})$ falls almost exactly linearly in $\ln N$, the signature of Proposition 14.3(iii); fitted on the models up to 1.5 billion parameters, the law predicted the 2.4-billion model's code-generation pass rate with a deviation of 0.05%, as they report.[7] Predictability need not even use the scale axis: modeling the scores of about a hundred public models through a low-dimensional measure of capability, Ruan et al. (2024) found that several emergent phenomena follow smooth sigmoidal curves in that measure and can be predicted from smaller models.

[7] The GPT-4 technical report forecast a coding pass rate in the same way, fitting the mean log pass rate as a power law in compute and keeping only problems that every model solved at least once within a large sample budget (OpenAI 2023); Chapter 13 discusses the forecast.

WHAT REMAINS. Not every jump dissolves. On 5 of 8 BIG-bench tasks of unnatural in-context learning, Hu et al. found $\ln(-\ln \text{pass rate})$ concave in $\ln N$: the curve accelerated beyond the scaling law, which they call accelerated emergence, and a law fitted to smaller models does not predict it, however fine the resolution. Du et al. (2024) pre-trained more than 30 models from 300 million to 32 billion parameters on one corpus and found that performance tracks the pre-training loss whatever the size: on four benchmarks, among them the multitask examination and grade-school arithmetic, models of three sizes followed through training stayed at chance until their pre-training loss fell to about 2.2, and two continuous metrics, the probability of the correct choice and the Brier score, showed the same threshold, the Brier score never better than the 0.75 of uniform guessing before it. The probability of the correct option can rise while the model still cannot tell it from the wrong ones: on BIG-bench's periodic-elements task the log-likelihoods of right and wrong answers tracked each other up to 10^{10} parameters and then diverged (Srivastava et al. 2023), so that the quantity that matters, the margin between them, changed abruptly. Wei, replying to the critique, added that smooth surrogates had not yet forecast a jump and that some tasks show a kink even in cross-entropy (Wei 2023).

Genuine transitions are best documented in training time. In transformers of every size with more than one layer, Olsson et al. (2022) found that the ability to use earlier context improves abruptly early in training, between about 2.5 and 5 billion tokens, together with the formation of induction heads and a bump in the training loss, the only place where the loss curve is not convex; the evidence is causal for small attention-only models and correlational for large ones. A network trained on division modulo 97 with half of the table fit its training data within 10^3 steps, showed little sign of generalization until 10^5 and generalized perfectly near 10^6, its validation loss going through a double descent (Power et al. 2022). On sparse parities of k out of n bits, networks learn abruptly after about $n^{O(k)}$ steps, while gradient descent steadily amplifies the relevant features through a Fourier gap in the population gradient, progress that loss and error do not see (Barak et al. 2022); progress measures for grokking tell the same story (Nanda et al. 2023). These transitions are sharp in continuous metrics too, and a verdict must name them.

VERDICT. The claim that abilities appear abruptly and unpredictably with scale did not survive as a general statement. Most of the abrupt curves were readings, by exact match or multiple-choice grade on tens to thousands of items, of quantities that change smoothly, and the same outputs scored by per-token likelihood, edit distance, or Brier score, or measured with enough items, change smoothly and predictably. What survives is narrower: thresholds in continuous quantities exist, such as the pre-training loss at which some benchmarks leave chance, the margin between right and wrong answers, accelerating pass rates, and the induction bump in training, and a claim of emergence must show one in a continuous metric rather than read it off a thresholded one.

ARTIFACT

14.4 The method: a metric is a transformation

Every score is computed from a model's outputs by a rule, and the rule sits between the model and the experimenter like an instrument between a sample and an observer. The first question about a curve of scores is what the rule does to the quantity beneath it.

Definition 14.2 (Latent quantity and metric). An evaluation scores a model on m items. A *latent quantity* of an item is a number computed from the model's predictive distribution that responds to arbitrarily small changes of it, such as the probability q_i that the model assigns to the correct answer of item i or its log-likelihood $\ln q_i$. A *metric* is a rule that maps the model's outputs on the m items to a number; it is a *threshold metric* if each item contributes only whether some latent quantity exceeds a threshold, as in exact match and multiple-choice grade, and a *continuous metric* if it is a continuous function of the latent quantities, as in mean log-likelihood and the Brier score. A metric has a *floor* (a *ceiling*) where it stops responding to decreases (increases) of the latent quantity, and its *resolution* is the smallest change it can register; for a proportion of m items, $1/m$.

WHAT A CHANGE OF METRIC PRESERVES. Stevens (1946) classified scales of measurement by the transformations they admit: every one-to-one relabeling for a nominal scale, every strictly increasing transformation for an ordinal scale, $y \mapsto ay + b$ with $a > 0$ for an interval scale, and $y \mapsto ay$ for a ratio scale.[8] A statement about measurements is meaningful on a scale when no admissible transformation changes its truth. Two metrics that are strictly increasing functions of one latent quantity rank the models of a family identically, so every ordinal statement carries over from one to the other. Statements about the shape of a curve do not. For every strictly increasing curve $\theta(x)$, every x_0, and every $h > 0$, the strictly increasing map $g(u) = \Lambda\big((u - \theta(x_0))/h\big)$, with Λ as in Proposition 14.3, makes $g(\theta(x))$ rise from near 0 to near 1 while $\theta(x)$ crosses an interval of length a few times h, as short as one likes. Abruptness is a property of a curve on a scale, and a

[8] Stevens also listed the statistics each scale permits: the median for ordinal scales, the mean for interval scales, the coefficient of variation for ratio scales.

claim of abruptness is a claim about the metric unless the question fixes the metric.

The metric of most emergent abilities is exact match. Write N for the number of parameters of a model, $p(N)$ for the probability that it gets one token of an answer right, and $\ell(N) = -\ln p(N)$ for the *per-token log error*; for an answer sampled from the model, ℓ is the cross-entropy of the target token. The rate of exact matches on answers of L tokens is $\mathrm{EM}_L(N)$, and $\Lambda(z) = \exp(-e^{-z})$ is the distribution function of the standard *Gumbel distribution*, the limit law of the maximum of many draws (Proposition A.9).

Proposition 14.3 (Exact match is a Gumbel curve). *Suppose that the L tokens of an answer are right independently, each with probability $p(N)$, and that the per-token log error follows the law*

$$\ell(N) = \ell_\infty + cN^{-\gamma}, \qquad c > 0,\ \gamma > 0,\ \ell_\infty \geq 0.$$

Put $\beta = 1/\gamma$ and $\mu_L = \ln(Lc)/\gamma$. Then the following hold for every $N > 0$.

(i) The exact-match rate is $\mathrm{EM}_L(N) = e^{-L\ell_\infty}\,\Lambda\big((\ln N - \mu_L)/\beta\big)$.

(ii) The curves for different lengths are translates of one another in $\ln N$, with $\mu_{L'} - \mu_L = \ln(L'/L)/\gamma$. For $0 < q_1 < q_2 < 1$, the curve rises from the fraction q_1 to the fraction q_2 of its ceiling $e^{-L\ell_\infty}$ over an interval of $\ln N$ of length $\beta \ln\big(\ln(1/q_1)/\ln(1/q_2)\big)$, whatever L; from 10% to 90% this is 3.08β. Its steepest slope, $\mathrm{d}\mathrm{EM}_L/\,\mathrm{d}\ln N = e^{-L\ell_\infty}\gamma/e$, is attained at $\ln N = \mu_L$.

(iii) The double logarithm is affine: $\ln\big(-\ln \mathrm{EM}_L(N) - L\ell_\infty\big) = \ln(Lc) - \gamma \ln N$.

(iv) As $N \to 0$, $\mathrm{EM}_L(N)\, e^{L\ell_\infty} = \exp\big(-e^{\gamma(\mu_L - \ln N)}\big)$ falls to zero doubly exponentially in $\ln N$; as $N \to \infty$, $1 - \mathrm{EM}_L(N)\, e^{L\ell_\infty} \sim LcN^{-\gamma}$.

Proof. The plan is to write the exact-match rate in terms of $x = \ln N$ and to read off each part. By independence, all L tokens are right with probability $p(N)^L = e^{-L\ell(N)}$, and the law for ℓ gives $\mathrm{EM}_L(N) = e^{-L\ell_\infty}\exp(-LcN^{-\gamma})$. Since $Lc > 0$, the second factor's exponent can be written as

$$LcN^{-\gamma} = \exp\big(\ln(Lc) - \gamma x\big) = \exp\big(-(x - \mu_L)/\beta\big),$$

so that $\exp(-LcN^{-\gamma}) = \Lambda\big((x - \mu_L)/\beta\big)$, which is part (i).

Part (ii). The variable $z = (x - \mu_L)/\beta$ depends on L only through μ_L, which proves the first statement. The function Λ increases strictly and $\Lambda(z) = q$ exactly when $z = -\ln\ln(1/q)$, so the fractions q_1 and q_2 of the ceiling are reached at values of z that differ by $\ln\ln(1/q_1) - \ln\ln(1/q_2)$, and at values of x that differ by β times this; with $q_1 = 0.1$ and $q_2 = 0.9$ it is $\ln(2.3026/0.1054) = 3.084$. For the slope, $\Lambda'(z) = u\,e^{-u}$ with $u = e^{-z} > 0$, and $u\,e^{-u}$ is largest, with value $1/e$, at $u = 1$, that is at $z = 0$; since $\mathrm{d}z/\,\mathrm{d}x = 1/\beta = \gamma$, the slope of EM_L in x is at most $e^{-L\ell_\infty}\gamma/e$, with equality at $x = \mu_L$.

Parts (iii) and (iv). Taking logarithms in part (i) gives $-\ln \mathrm{EM}_L - L\ell_\infty = LcN^{-\gamma}$, and taking logarithms once more gives part (iii). As $x \to -\infty$,

part (i) reads $\exp(-e^{\gamma(\mu_L - x)})$ with an exponent that grows exponentially in $-x$. As $x \to \infty$, the quantity $t = LcN^{-\gamma}$ tends to 0 because $\gamma > 0$, and $1 - e^{-t} \sim t$. ∎

The proposition is the model of Schaeffer et al. (2023) made exact, and its part (iii) is the task scaling law of Hu et al. (2024). The per-token log error above its floor is a power law, a straight line on logarithmic axes, and $-\ln \mathrm{EM}_L - L\ell_\infty$ is L times it. Exponentiating creates the S-shape, and the length of the answer moves it: a tenfold longer answer needs a model $10^{1/\gamma}$ times larger to reach the same fraction of its ceiling, and the rise spans the same factor $e^{3.08/\gamma}$ in N at every length. The per-token accuracy, the case $L = 1$, has the same shape, shifted to smaller models: where the long curves rise, it is already near its ceiling and changes slowly, and where it rises, the long curves sit on their floor, the doubly exponential left tail of Λ (Proposition 14.5). Tokens of different difficulty make $\ln(-\ln \mathrm{EM}_L)$ a convex function of $\ln N$ (Exercise 14.2).

SPACING. If the models of a family are spaced by a factor r in N, then about $3.08/(\gamma \ln r)$ of them fall where an exact-match curve rises from 10% to 90% of its ceiling. Between GPT-3's 13-billion and 175-billion models, $r = 13.5$, and a curve with $\gamma > 3.08/\ln 13.5 = 1.19$ can rise from 10% to 90% between them, with no model on its rise: the plot shows a floor, then a jump.

MULTIPLE CHOICE IS A THRESHOLD TOO. An item scored by multiple-choice grade counts as correct when the correct option receives a larger probability than every wrong one, that is when the margin between them is positive. Across items the expected grade is the fraction of items with a positive margin, one minus the distribution function of the margins at zero: smooth when the items differ widely in difficulty, sharp when they are alike, and flat at the chance level $1/K$ of K options until the margins of many items move together (Exercise 14.3). The continuous alternatives are proper scores.

Proposition 14.4 (Proper and improper scores). *An item has options $1, \dots, K$ with $K \geq 2$; the model assigns them the probabilities $q = (q_1, \dots, q_K)$, nonnegative with sum 1; the correct option Y is random with distribution π (for an item with one correct answer, π puts mass 1 on it). Let $k^\star(q)$ be the option with the largest q_k, ties broken by a fixed rule, and consider the* Brier score $B(q, Y) = \sum_k (q_k - \mathbf{1}\{Y = k\})^2$, *the* log score $S(q, Y) = \ln q_Y$ *and the* multiple-choice grade $G(q, Y) = \mathbf{1}\{Y = k^\star(q)\}$.

(i) The expected Brier score is $\mathbb{E}B(q, Y) = \sum_k (q_k - \pi_k)^2 + 1 - \sum_k \pi_k^2$, a continuous function of q whose unique minimum is at $q = \pi$.

(ii) Let $\operatorname{supp} \pi = \{k : \pi_k > 0\}$. If $q_k > 0$ for every $k \in \operatorname{supp} \pi$, then the expected log score is $\mathbb{E}S(q, Y) = \sum_{k \in \operatorname{supp} \pi} \pi_k \ln \pi_k - \mathrm{KL}(\pi \,\|\, q)$ with $\mathrm{KL}(\pi \,\|\, q) = \sum_{k \in \operatorname{supp} \pi} \pi_k \ln(\pi_k / q_k) \geq 0$, and otherwise it is $-\infty$; its unique maximum is at $q = \pi$.

(iii) The expected grade is $\mathbb{E}G(q, Y) = \pi_{k^\star(q)}$. It is maximal at $q = \pi$, and also at every q whose largest coordinate sits at a largest coordinate of π, and it is constant on each region where $k^\star(q)$ is constant.

PROOF. For part (i), expand the square and use $\mathbb{E}\mathbf{1}\{Y = k\} = \mathbb{E}\mathbf{1}\{Y = k\}^2 = \pi_k$:

$$\mathbb{E}B = \sum_k \left(q_k^2 - 2q_k\pi_k + \pi_k\right) = \sum_k (q_k - \pi_k)^2 + \sum_k (\pi_k - \pi_k^2),$$

and $\sum_k \pi_k = 1$. The first sum is a polynomial in q, zero exactly at $q = \pi$ and positive elsewhere.

For part (ii), the options outside $\operatorname{supp}\pi$ have probability zero and do not contribute to the expectation: $\mathbb{E}S = \sum_{k\in\operatorname{supp}\pi} \pi_k \ln q_k$, which is $-\infty$ when $q_k = 0$ for some $k \in \operatorname{supp}\pi$. Otherwise, subtracting $\sum_{k\in\operatorname{supp}\pi} \pi_k \ln \pi_k$ leaves $-\mathrm{KL}(\pi \,\|\, q)$. Jensen's inequality for the concave logarithm, applied to the variable q_Y/π_Y under π, gives

$$-\mathrm{KL}(\pi \,\|\, q) = \sum_{k\in\operatorname{supp}\pi} \pi_k \ln \frac{q_k}{\pi_k} \le \ln \sum_{k\in\operatorname{supp}\pi} q_k,$$

and the right side is at most $\ln 1 = 0$ because the q_k sum to 1. Equality in Jensen's inequality requires q_k/π_k to take one value for all $k \in \operatorname{supp}\pi$, and equality in the bound by $\ln 1$ requires $\sum_{k\in\operatorname{supp}\pi} q_k = 1$; together they force $q = \pi$.

For part (iii), $\mathbb{E}G = \mathbb{P}(Y = k^\star(q)) = \pi_{k^\star(q)}$, which depends on q only through $k^\star(q)$ and is at most $\max_k \pi_k$, with equality whenever $k^\star(q)$ is a largest coordinate of π; in particular at $q = \pi$. ∎

Both the Brier score and the log score are *strictly proper*: a model optimizes its expected score only by reporting the true probabilities (Gneiting and Raftery 2007). For an item with one correct option y, $B = (1 - q_y)^2 + \sum_{k\neq y} q_k^2$ and $S = \ln q_y$ move whenever q_y does; the grade moves only when q_y overtakes the largest wrong option. A continuous metric removes the jumps that a threshold adds, but not a jump in the probabilities themselves, and it can rise without the model learning to discriminate, since raising the probability of every option raises that of the correct one; for multiple choice the latent quantity to watch is the margin, or the probability normalized over the options, as Du et al. (2024) insist. The log score also magnifies changes near the floor, where $\mathrm{d}\ln q/\,\mathrm{d}q = 1/q$ is largest; exact match flattens them.

14.5 The method: resolution, and transitions that are real

A score of zero is not a measurement of zero. The next proposition collects what a finite set of items can and cannot see, building on the exact binomial intervals of the appendix.

Proposition 14.5 (The resolution of a success rate). *Let $X \sim \mathrm{Bin}(m, s)$ count the successes on m independent items, each a success with probability $s \in (0, 1)$.*

(i) No success is reported with probability $\mathbb{P}(X = 0) = (1 - s)^m \ge 1 - ms$. In particular, if $s \le \varepsilon/m$, then the evaluation reports no success with probability at least $1 - \varepsilon$.

(ii) If $X = 0$, then the one-sided 95% upper confidence bound for s is $1 - 0.05^{1/m} < 3/m$.

(iii) Suppose that $s = \mathrm{EM}_L(N)$ as in Proposition 14.3 with $\ell_\infty = 0$, and evaluate it at a model $\Delta \geq 0$ scale units before the midpoint, $\ln N = \mu_L - \Delta\beta$. Then $s = \exp(-e^{\Delta})$, and the evaluation reports at least one success with probability at least $1/2$ if $m \geq \ln 2 \cdot \exp(e^{\Delta})$, and with probability less than $1/2$ if $m < (1-s)\ln 2 \cdot \exp(e^{\Delta})$.

(iv) Suppose that each item has L tokens, right independently with probability $p \in (0,1)$, so that $s = p^L$. The Fisher information about p in the exact-match indicator of one item is the fraction

$$\mathrm{eff}_L(p) = \frac{Lp^{L-1}(1-p)}{1-p^L} = \frac{L}{\sum_{i=0}^{L-1} p^{-i}} \tag{14.1}$$

of the Fisher information in its L token indicators. The fraction increases with p, tends to 1 as $p \to 1$, and satisfies $\mathrm{eff}_L(p) \leq Lp^{L-1}$.

Proof. *Part (i).* By independence $\mathbb{P}(X = 0) = (1-s)^m$. The function $s \mapsto (1-s)^m - (1-ms)$ vanishes at $s = 0$ and has derivative $m\big(1-(1-s)^{m-1}\big) \geq 0$ on $[0,1]$, so it is nonnegative there (Bernoulli's inequality). If $s \leq \varepsilon/m$, then $1 - ms \geq 1 - \varepsilon$.

Part (ii). This is Proposition A.17(ii) at level 95%, which gives $1 - 0.05^{1/m} \leq \ln(20)/m$, and $\ln 20 = 2.996 < 3$.

Part (iii). At $\ln N = \mu_L - \Delta\beta$ the argument of Λ in Proposition 14.3(i) is $-\Delta$, so $s = \exp(-e^{\Delta})$. At least one success has probability $1 - (1-s)^m$, which is at least $1/2$ exactly when $m \ln\big(1/(1-s)\big) \geq \ln 2$. The inequalities $s \leq \ln\big(1/(1-s)\big) \leq s/(1-s)$, which follow from $\ln(1+t) \leq t$ applied to $t = -s$ and to $t = s/(1-s)$, show that $m \geq \ln 2/s$ suffices and that $m < (1-s)\ln 2/s$ makes the probability less than $1/2$; and $1/s = \exp(e^{\Delta})$.

Part (iv). A Bernoulli variable whose success probability $\theta(p)$ depends smoothly on p carries the Fisher information $\theta'(p)^2/\big(\theta(1-\theta)\big)$ about p. For the exact-match indicator $\theta = p^L$, which gives $L^2p^{2L-2}/\big(p^L(1-p^L)\big) = L^2p^{L-2}/(1-p^L)$. For one token $\theta = p$, which gives $1/\big(p(1-p)\big)$, and L independent tokens carry L times as much. The ratio is the first expression in (14.1); dividing its numerator and denominator by $(1-p)p^{L-1}$ and using $1 - p^L = (1-p)\sum_{j=0}^{L-1} p^j$ gives the second. Each p^{-i} with $i \geq 1$ decreases in p, so the sum decreases and the fraction increases; as $p \to 1$ the sum tends to L; and the sum is at least its largest term $p^{-(L-1)}$, which gives the bound. ∎

Part (iii) is the arithmetic of the floor. To see an exact-match curve, in the sense of one success with probability one half, one scale unit β before its midpoint takes about 11 items; two units, 1,100; three units, 3.7×10^8. Whatever its size, a test set shows an exact-match curve only within two or three scale units of its rise, because the number of items needed grows doubly exponentially with the distance; repeated sampling, PassUntil's route, meets the same arithmetic, since it needs about $1/s$ samples per item. Part (iv) prices the alternative: by the delta method (Theorem A.10), p

estimated from exact-match counts as $(X/m)^{1/L}$ has asymptotically $1/\mathrm{eff}_L(p)$ times the variance of the per-token proportion on the same items. At $p = 1/2$ and $L = 10$, $\mathrm{eff}_{10} = 10/1023$: fifty items scored token by token carry as much information about p as 5,100 scored by exact match. The per-token log-likelihood shares the advantage: its standard error is that of a mean of mL terms at every scale, and it does not collapse as the success rate goes to zero.

GENUINE OR METRIC-INDUCED? The verdict of Section 14.3 names transitions that are sharp in continuous metrics. The method needs a test that tells them apart from the ones a metric makes.

Definition 14.6 (Metric-induced and genuine transitions). Let the models of a family be ordered by a scale x, such as $\ln N$, the logarithm of compute, or the number of training steps, let $\theta(x)$ be a latent quantity measured on each, and let a metric be a fixed transformation $M = g(\theta)$. A rise of M between x_1 and x_2 is *metric-induced* relative to θ if a smooth law for θ, fitted to data that do not contain the rise, predicts through g the values of M between x_1 and x_2 within their measurement error. It is *genuine* relative to θ if θ itself departs from the law fitted outside $[x_1, x_2]$, by more than its measurement error, where M rises.

The definition is relative on purpose: a transition can be metric-induced relative to one latent quantity and genuine relative to another. On multiple-choice tasks the likelihood of the correct option may follow its law while the margin jumps; in sparse parity the loss jumps while a progress measure grows steadily (Lab 14.3). The strongest form of the test predicts the larger models from the smaller ones, a backtest in the sense of Chapter 13; Proposition 14.3(iii) supplies the transformation for exact match, and a concave double logarithm, Hu et al.'s accelerated emergence, is a departure that no resolution removes. The prompt, format, and examples of an evaluation move scores too, a separate source of variation (Chapter 15).

14.6 In the laboratory: digits as tokens

The first two labs use the recipe of Chapter 1 (two hidden ReLU layers, SGD with momentum 0.9 at learning rate 0.05, batch size 64, 20 epochs, labkit's split 0 with 1,258 training and 539 test digits) at fifteen widths from 1 to 512, from 87 to 301,066 parameters; width 64 is the network of Chapter 1. A test digit plays the part of a token, and an answer of L tokens is a sequence of L test digits that the network must classify without a mistake.

LAB 14.1 · DIGITS AS TOKENS

SCRIPT
14-emergence-tokens.py

SETUP. Twenty runs per width (seed s for initialization and data order, $s = 0, \ldots, 19$): 300 networks. Each is scored on the 539 test digits by per-digit accuracy p, by exact match on $L = 3$ and $L = 10$ digits, computed exactly over all sequences of L test digits drawn with

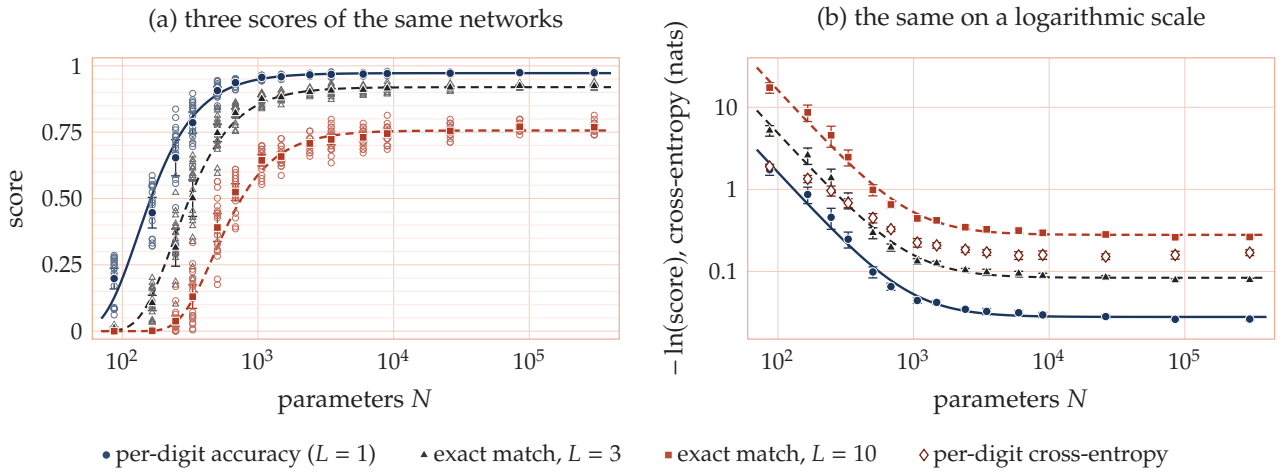


Figure 14.1. Lab 14.1: 300 digit classifiers, 20 runs (seeds 0–19) at each of 15 widths from 1 to 512, scored on the 539 test digits. (a) Per-digit accuracy and exact match on sequences of 3 and 10 digits; hollow marks are single runs, filled marks means with 95% t-intervals over the 20 runs. Solid line: the curve of Proposition 14.3 fitted to per-digit accuracy; dashed lines: its predictions for $L = 3$ and 10, with no further parameters. (b) Means of $-\ln(\text{score})$ for the three lengths, with the same curves, and of the per-digit cross-entropy, with 95% t-intervals.

replacement (for one network, p^L), and by the per-digit cross-entropy. The law $\ell = \ell_\infty + cN^{-\gamma}$ is fitted to the mean per-digit accuracy alone (least squares on the double logarithm; bootstrap over runs). About a minute.

PREDICT FIRST. At which width will exact match on ten digits pass 10%? Will the cross-entropy show anything special there? By what factor in N should the ten-digit curve lag the per-digit one?

LOOK FOR. One S-shape at three lengths, shifted and capped; parallel curves on the logarithmic scale; where the predictions for $L = 3$ and 10 miss, and why.

WHAT HAPPENED. Per-digit accuracy rises from 0.198 ± 0.039 at width 1 (mean and 95% t-interval half-width over 20 runs) to 0.957 at width 12 and levels off at 0.974; exact match on ten digits is 0.0013 ± 0.0007 at width 2, 0.130 ± 0.044 at width 4 and 0.524 at width 8, and levels off at 0.77 (Figure 14.1a). The per-digit cross-entropy falls from 1.90 ± 0.13 nats at width 1 to 0.15 at width 128 and rises slightly at the widest networks (0.17 at width 512), whose confident mistakes the log score penalizes and accuracy does not. On a ladder of factors of two with one run per width (seed 0), exact match on ten digits reads 0.000, 0.003, 0.196, 0.386, and 0.634 at widths 1, 2, 4, 8, and 16, absent in the first two models and present from the third, while the cross-entropy of the same networks, 1.77, 1.11, 0.56, 0.38, and 0.20 nats, shows nothing at width 4.

The law fitted to per-digit accuracy has $\gamma = 1.80$ (bootstrap 95% interval 1.75 to 1.87) and $\ell_\infty = 0.0279$ (0.0273 to 0.0286), with a root-mean-square

residual of 0.13 on the double logarithm. It places the midpoints for $L = 1$, 3, and 10 at $N = 129$, 238, and 464: the ten-digit curve lags the per-digit one by the factor $10^{1/\gamma} = 3.6$, and each rises from 10% to 90% of its ceiling over a factor of 5.5 in N. With no parameter fitted to them, the predicted curves for $L = 3$ and 10 miss the measured means by at most 0.046 and 0.073, and fall inside the 95% intervals at 7 and 8 of the 15 widths. The misses have three sources. Between widths 4 and 8 the per-digit curve is a little steeper than the law, and the tenth power multiplies the error: at width 6 the law gives $p = 0.892$ against 0.907, and exact match 0.32 against 0.39. Means over runs are not powers of means: at width 1, where the runs' accuracies range from 0.06 to 0.29, the mean of p^{10} is about eleven times the tenth power of the mean. And in the tail an error δ in ℓ becomes an error $L\delta$ in $\ln \mathrm{EM}_L$: at width 1 the law predicts 10^{-9} for ten digits, the runs average 10^{-6}. On the logarithmic scale of panel (b) the three lengths run parallel, and the cross-entropy falls with them from the smallest network on.

LAB 14.2 · THE NUMBER OF ITEMS

SCRIPT
14-emergence-items.py

SETUP. The 300 networks of Lab 14.1, scored on a fixed benchmark of 5,000 items, each a sequence of 10 test digits drawn with replacement (generator seed 1414); the first 50 and 500 items form the smaller benchmarks. One network per width makes a model family, so the runs give 20 families. Exact-match counts with Clopper–Pearson intervals, zero counts with the bound of Proposition 14.5(ii); the per-digit cross-entropy over the $10m$ digit slots of the m items. About a minute.

PREDICT FIRST. Which widths will read zero on 50 items? How many items does it take to see the next smaller network? Will the cross-entropy on 50 items show a trend at the widths that read zero?

LOOK FOR. Zero counts and their bounds against the exact value p^{10}; the apparent onset in each family and how it moves with m; the interval of the cross-entropy at the smallest widths.

WHAT HAPPENED. In family 0 (Figure 14.2a), 50 items read zero at widths 1 and 2, bounding the rate only below 5.8%, and one success at width 3; 500 items read zero only at width 1 (bound 0.6%) and one success at width 2; 5,000 items read zero at width 1 (bound 0.06%) and 15 successes at width 2. The exact value at width 1, 1.6×10^{-7}, lies more than three orders of magnitude below every bound.[9] Across the 20 families the apparent onset, the smallest width with a success, was width 3 in 13 families and width 4 in 7 on 50 items; widths 2, 3, and 4 in 8, 8, and 4 families on 500 items, and in 15, 4, and 1 families on 5,000 items. A hundredfold increase in items moved the onset by about one model, as Proposition 14.5(iii) predicts: on the fitted law width 2 sits 1.9 scale units before the ten-digit midpoint and width 1 sits 3.0 units before it. Width 1 read zero in every family at every m; at its median exact value, 5.5×10^{-7}, a success with probability one half would take about 1.3 million items. The shares of zero counts agree with

[9] Drawn with replacement from the 539 test digits, the items are independent given the network, so the counts are exactly binomial; the 539 digits are themselves a sample, whose noise Chapter 1 measures.

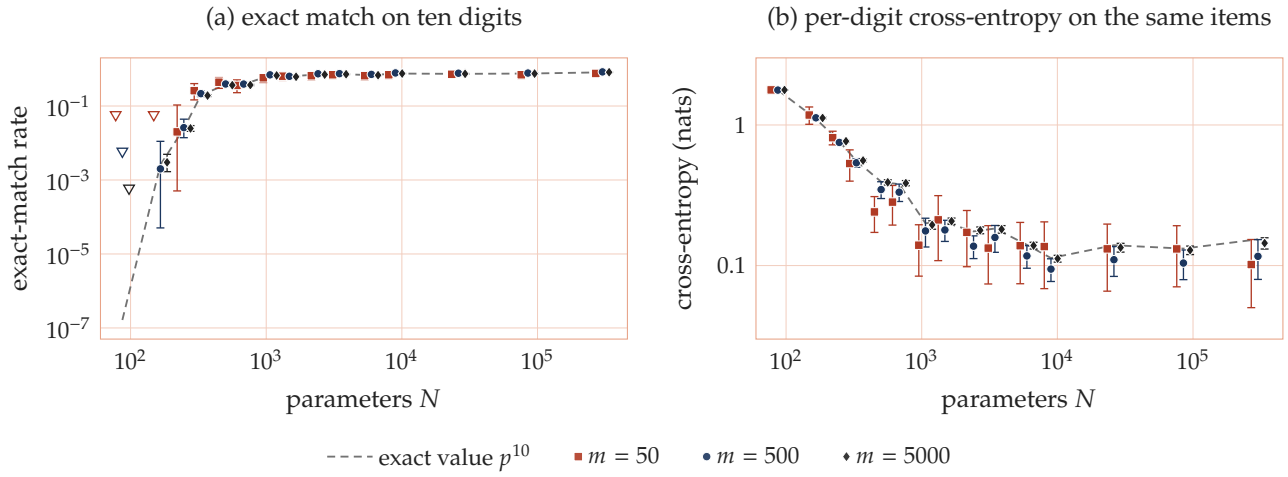


Figure 14.2. Lab 14.2: one model family (seed 0, one network per width) scored on benchmarks of $m = 50$, 500, and 5000 items of ten test digits, nested; marks for the three benchmarks are offset sideways for legibility. (a) Exact-match rates with 95% Clopper–Pearson intervals; hollow downward triangles are zero counts, drawn at their one-sided 95% upper bound $1 - 0.05^{1/m}$; the dashed line is the exact value p^{10} over all sequences. (b) Per-digit cross-entropy over the $10m$ digit slots of the same items, with 95% t-intervals; the dashed line is its value on all 539 test digits.

the binomial prediction, the average of $(1 - p^{10})^m$ over the families: 0.60 against 0.65 at width 2 on 500 items, 0.25 against 0.33 on 5,000, 0.35 against 0.38 at width 3 on 50 items.

The cross-entropy on the same 50 items (Figure 14.2b) is 1.78 ± 0.06 nats at width 1, 1.18 ± 0.17 at width 2 and 0.81 ± 0.09 at width 3 (95% t-intervals over 500 digit slots): the trend is plain from the smallest network on, where exact match reads zero. Per-digit accuracy on the same slots is as informative there: 0.206 at width 1 and 0.540 at width 2 (95% Clopper–Pearson intervals 0.171 to 0.244 and 0.495 to 0.584). At large widths the interval grows to about half the value, since the log loss of an accurate network is dominated by a few confident mistakes; there exact match is well measured. In family 0, exact match keeps the fraction $\mathrm{eff}_{10}(p) = 6{\times}10^{-6}$ of the per-digit information at width 1, 0.02 at width 2, 0.11 at width 3, 0.43 at width 4, and 0.83 at width 12.

Both labs concern transitions that a metric makes. A positive control asks whether the method recognizes one that it does not make.

LAB 14.3 · A TRANSITION THAT THE METRIC DOES NOT MAKE

SCRIPT
`14-emergence-parity.py`

SETUP. Sparse parity after Barak et al. (2022): inputs of 50 random signs ±1, label the product of the first three; one hidden layer of 256 ReLU units, SGD with learning rate 0.1 on fresh batches of 32, logistic loss, 12,000 steps; ten runs (seed s for initialization and data). Every 100 steps, on 4,000 fixed test strings: the test loss, the test error, and a progress measure, the largest share of a hidden unit's squared input weights on the three relevant inputs, divided by the share 3/50 of a

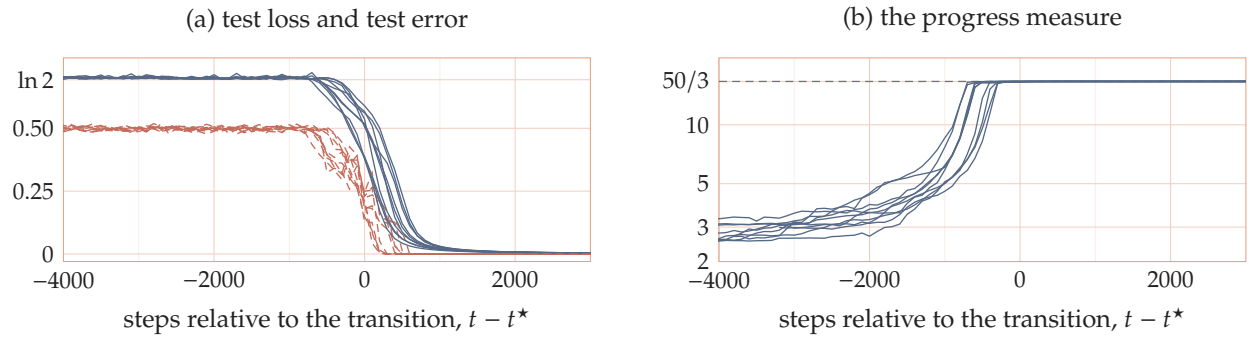


Figure 14.3. Lab 14.3: ten runs (seeds 0–9) of a network with 256 hidden units learning the parity of 3 of 50 input bits, each aligned at its transition time $t^\star$ (from 3,200 to 7,500 steps). (a) Test loss in nats (solid) and test error (dashed) on 4,000 test strings. (b) The largest share of a hidden unit's squared input weights on the three relevant inputs, relative to an even spread; logarithmic scale, maximum 50/3 (dashed).

unit that ignores them (at most 50/3). The transition time $t^\star$ of a run is the first evaluation with error below 0.25. About 30 seconds.

PREDICT FIRST. Will the test loss, a continuous metric, fall gradually while the error stays at one half? Does anything move during the plateau?

LOOK FOR. The loss on the plateau against $\ln 2 = 0.693$, the width of its drop, and the progress measure before the drop.

WHAT HAPPENED. In all ten runs the test loss stayed between 0.693 and 0.720 nats, near the value $\ln 2$ of a constant prediction, and the error above 0.477, for 2,700 to 7,000 steps; then the loss fell from 0.68 to below 0.05 within 1,000 to 1,300 steps, and the error with it (Figure 14.3a). The transition times range from 3,200 to 7,500 steps (median 5,000). The continuous metric jumps with the threshold one: the transition is genuine relative to the loss in the sense of Definition 14.6, and the protocol below must call it so. It is not genuine relative to everything. The progress measure (Figure 14.3b) started at 2.3 to 3.1, the largest of 256 random shares, and stood at 2.7 to 4.5 two thousand steps before $t^\star$ and at 4.6 to 8.6 one thousand steps before, while the loss was still between 0.698 and 0.707: the network was finding the three relevant inputs during the plateau, the hidden progress of Barak et al. (2022), which neither loss nor error could see.

The miniature reproduced the post-mortem: exact match on long answers emerged where per-digit metrics moved smoothly, a law fitted to one metric predicted the other, and the zero readings that make a curve look emergent were limits of resolution. It differs from language models in one respect: its per-digit accuracy is itself steep ($\gamma \approx 1.8$, the whole rise within one decade of N), so the lengths shift one steep curve rather than turning a gradual one into a steep one.

14.7 Protocol

PROTOCOL 14.1 · BEFORE ANNOUNCING AN EMERGENT ABILITY

- ☐ Record for every item the model's output, the log-probability of the reference answer and, for multiple choice, the probability of every option, not only the score.
- ☐ Plot the headline metric beside a continuous one on the same axis of scale: the per-token log-likelihood of the answer, a proper score, or the margin of the correct option over the best wrong one.
- ☐ Give every point an interval; draw zero counts as upper bounds of about $3/m$ on a logarithmic axis, and state m and the chance level.
- ☐ Fit a law to the continuous metric on the smaller models and predict the headline metric for the larger ones through its transformation (for exact match, Proposition 14.3); report how far the measured curve departs from the prediction.
- ☐ Place enough models across the rise: several per scale unit $1/\gamma$ of the fitted law, or intermediate checkpoints.
- ☐ Call a transition genuine only if the continuous metric itself departs from its law where the headline metric rises, beyond its measurement error; then look for a progress measure.
- ☐ State the result as a property of the metric ("exact match rises from below 0.6% to 40% between N_1 and N_2"), not as the appearance of an ability.

The protocol separates a rise that the metric makes from a change in the model; it does not predict genuine transitions in models not yet trained, and it never shows that an ability is absent, only that its rate is below a bound. It needs per-item probabilities; for a model that returns only text, sample each item repeatedly, as PassUntil does, at a cost of about $1/s$ samples per item at success rate s. A continuous metric is not valid because it is continuous: the likelihood of one reference answer penalizes correct answers phrased differently, and the likelihood of the correct option can rise with the wrong ones. Where the headline metric is the one users need, as for an arithmetic answer, the protocol makes its rise predictable, not unimportant; and the prompt and format of the evaluation are a second instrument, the subject of Chapter 15.

Exercises

14.1 CONSOLIDATION. (The Gumbel curve.) (a) For $\ell(N) = cN^{-\gamma}$, show that EM_L has its inflection point in $\ln N$ where $\mathrm{EM}_L = 1/e$. (b) With the law fitted in Lab 14.1 ($\gamma = 1.80$, $c = 6425$, $\ell_\infty = 0.0279$), compute the

midpoints and ceilings for $L = 30$ and $L = 100$. What does the floor ℓ_∞ do to long answers?

14.2 CONSOLIDATION. ★ (Easy and hard tokens.) Suppose token j of an answer is right with probability $\exp(-c_j N^{-\gamma_j})$, independently of the others. Show that $F(x) = \ln(-\ln \mathrm{EM}_L)$, as a function of $x = \ln N$, equals $\ln \sum_j e^{\ln c_j - \gamma_j x}$, that it is convex, and that its slope tends to $-\min_j \gamma_j$ as $x \to \infty$ and to $-\max_j \gamma_j$ as $x \to -\infty$. Which tokens govern the largest models? Compare with the sub-scaling growth of Hu et al. (2024). *(Hint: a log-sum-exp of affine functions is convex; its slope is a weighted average of the $-\gamma_j$.)*

14.3 CONSOLIDATION. (Multiple choice as a threshold.) An item is graded correct when the margin D of the correct option over the best wrong option is positive. (a) Suppose that across items $D \sim \mathcal{N}(\mu(N), \sigma^2)$; show that the expected grade is $\Phi(\mu(N)/\sigma)$; with $\mu(N) = a \ln(N/N_0)$, find the interval of $\ln N$ over which it rises from 10% to 90%, and its limit as $\sigma \to 0$. (b) Show that if the K options' scores are exchangeable, as for a model that cannot discriminate, then $\mathbb{P}(D > 0) = 1/K$. Which feature of the normal model fails at chance?

14.4 CONSOLIDATION. (Counting zeros.) (a) Compute the 95% Clopper–Pearson intervals for $1/500$ and $15/5000$, and the one-sided bound for $0/m$ at $m = 50$, 500, and 5000; compare with $3/m$. (b) What does a score of 0% on GPT-3's 2,000 arithmetic problems establish, and on the 32 items of a BIG-bench task? (c) Show that the two-sided 95% upper limit after zero successes is $1 - 0.025^{1/m} \approx 3.7/m$.

14.5 CONSOLIDATION. (The price of exact match.) (a) Derive (14.1) again by the delta method, comparing the asymptotic variance of $(X/m)^{1/L}$ with that of the per-token proportion. (b) At $p = 0.9$ and $L = 10$, how many items scored by exact match carry the information of 100 items scored per token? At $p = 0.99$?

14.6 LAB. (Longer answers.) Add $L = 30$ to `14-emergence-tokens.py`. Predict from the fitted law, before running, the factor by which its midpoint moves relative to $L = 10$ and its ceiling; then compare, and check that the rise from 10% to 90% of the ceiling spans the same factor in N.

14.7 LAB. (Dependent tokens.) In `14-emergence-items.py`, build each item from ten test digits of one randomly chosen class. Predict whether exact match will lie above or below p^{10}, and why, before running; then measure, and say what happens to the prediction of Lab 14.1. *(Hint: the mean of a product of indicators is the product of their means plus covariance terms; per-class accuracies differ.)*

14.8 LAB. (Longer parities.) Run `14-emergence-parity.py` with 40 and 60 input bits. Observe how the plateau lengthens, whether the loss ever falls gradually, and how long before $t^\star$ the progress measure leaves its starting range.

14.9 DESIGN. (An evaluation that could find a genuine transition.) A lab will train 12 models from 10^8 to 10^{11} parameters on the same data and can afford 10^6 model calls to evaluate a new reasoning task whose answers are about 20 tokens long. Design the evaluation: metrics, items, samples per item, where to add intermediate checkpoints, and the prediction to register before the largest models are scored. What result would you accept as a genuine transition?

14.10 DESIGN. (No log-probabilities.) A model is reachable only through an interface that returns sampled text, and a pass rate near 10^{-4} must be estimated to within a factor of two at the smallest of five sizes. Compare sampling a fixed number of answers per item with sampling until the first pass, in the expected number of calls. *(Hint: the number of samples until the first pass is geometric.)*

14.11 CRITIQUE. (Emergence at ten billion.) A paper scores five models of 0.1, 0.5, 2, 10, and 50 billion parameters on 100 items whose answers are about 30 tokens long, by exact match: 0, 0, 0, 11, and 46 correct. It concludes that the ability emerges abruptly near 10 billion parameters and could not have been predicted. What does the table establish, and what would you ask the authors for? (The description is invented.)

14.12 CRITIQUE. (Smooth, so predictable?) A study of a four-option benchmark reports that the log-probability of the correct option rises smoothly with scale and concludes that the benchmark shows no emergence. What measurement would test the conclusion, and what result would overturn it? (The description is invented.)

14.13 RESEARCH. (When is a transition genuine?) Open: a test, for five to ten models with per-item log-likelihoods, of one smooth law for a latent quantity against a change of slope or a jump, with controlled level and useful power; and an account of when such changes occur, through the quanta of Michaud et al. (2023), competing circuits as Hu et al. (2024) suggest, or thresholds in the pre-training loss (Du et al. 2024). Also open: whether sharp transitions in training time, such as the induction bump, have counterparts across model size at fixed data.

Notes and References

Wei et al. rooted their definition in an essay of Anderson (1972), who argued that at each level of complexity properties appear that the laws of the level below do not predict, and took its wording, quantitative changes producing qualitative ones, from a 2022 essay by Jacob Steinhardt. In language models the observation is older than the word, in the arithmetic of GPT-3 (Brown et al. 2020) and the abrupt capabilities of Ganguli et al. (2022); BIG-bench measured it task by task (Srivastava et al. 2023).

The tools are older still. Stevens (1946) classified scales of measurement by their admissible transformations; Brier (1950) proposed his score to verify probabilistic weather forecasts, and Gneiting and Raftery (2007)

review proper scoring rules; exact binomial intervals are due to Clopper and Pearson (1934), and the rule of three for zero counts is the subject of a short paper in medicine by Hanley and Lippman-Hand (1983). The Gumbel distribution is a limit law of extremes (Gumbel 1958); here it arises without a maximum, from the exponential of a power law.

The case against abruptness was made by Schaeffer et al. (2023) and the case for predictability with resolution by Hu et al. (2024); the GPT-4 report used the same transformation to forecast a coding pass rate (OpenAI 2023), and Ruan et al. (2024) made emergent capabilities predictable through a measure of capability fitted across model families. The replies are Wei (2023) and, with new experiments, Du et al. (2024). Michaud et al. (2023) proposed that a smooth power law in the loss sums many small abrupt steps, discrete skills learned in order of how often they are needed. Sharp transitions in training are documented for induction heads (Olsson et al. 2022), grokking (Nanda et al. 2023; Power et al. 2022), and sparse parities (Barak et al. 2022).

TRANSLATING THE LITERATURE. Schaeffer et al. (2023) write the per-token cross-entropy as $L_{CE}(N) = (N/c)^{\alpha}$ with $\alpha < 0$: their $-\alpha$ is our γ and their $c^{-\alpha}$ our c, with $\ell_\infty = 0$; their "Accuracy" is our exact match EM_L, their target length our L, and their resolution, set by the number of test data, our $1/m$. Hu et al. (2024) write PU for the pass probability of one item, our s, and use α and c as we use γ and c, with N the number of non-embedding parameters; their scaling-law, sub-scaling, and super-scaling growth (accelerated emergence) are a linear, convex, and concave $\ln(-\ln \mathrm{PU})$ in $\ln N$. Wei et al. (2022) plot against training compute C; within a family trained on a fixed number of tokens C is proportional to N, so their curves are ours translated in the logarithm. The emergence score of Schaeffer et al. is BIG-bench's breakthroughness. Du et al. (2024) write L for the pre-training loss, our $\mathcal{L}$, and η for its threshold.

References

Anderson, P. W. (1972). "More Is Different". In: *Science* 177.4047, pp. 393–396.

Barak, B., B. L. Edelman, S. Goel, S. Kakade, E. Malach, and C. Zhang (2022). "Hidden Progress in Deep Learning: SGD Learns Parities Near the Computational Limit". In: *Advances in Neural Information Processing Systems*. Vol. 35.

Brier, G. W. (1950). "Verification of Forecasts Expressed in Terms of Probability". In: *Monthly Weather Review* 78.1, pp. 1–3.

Brown, T. B. et al. (2020). "Language Models are Few-Shot Learners". In: *Advances in Neural Information Processing Systems*. Vol. 33.

Clopper, C. J. and E. S. Pearson (1934). "The Use of Confidence or Fiducial Limits Illustrated in the Case of the Binomial". In: *Biometrika* 26.4, pp. 404–413.

Du, Z., A. Zeng, Y. Dong, and J. Tang (2024). "Understanding Emergent Abilities of Language Models from the Loss Perspective". In: *Advances in Neural Information Processing Systems*. Vol. 37.

Ganguli, D. et al. (2022). "Predictability and Surprise in Large Generative Models". In: *Proceedings of the 2022 ACM Conference on Fairness, Accountability, and Transparency (FAccT '22)*. ACM.

Gneiting, T. and A. E. Raftery (2007). "Strictly Proper Scoring Rules, Prediction, and Estimation". In: *Journal of the American Statistical Association* 102.477, pp. 359–378.

Gumbel, E. J. (1958). *Statistics of Extremes*. New York: Columbia University Press.

Hanley, J. A. and A. Lippman-Hand (1983). "If Nothing Goes Wrong, Is Everything All Right? Interpreting Zero Numerators". In: *JAMA* 249.13, pp. 1743–1745.

Hu, S. et al. (2024). "Predicting Emergent Abilities with Infinite Resolution Evaluation". In: *International Conference on Learning Representations*.

Kaplan, J. et al. (2020). *Scaling Laws for Neural Language Models*. Preprint.

Michaud, E. J., Z. Liu, U. Girit, and M. Tegmark (2023). "The Quantization Model of Neural Scaling". In: *Advances in Neural Information Processing Systems*. Vol. 36.

Nanda, N., L. Chan, T. Lieberum, J. Smith, and J. Steinhardt (2023). "Progress Measures for Grokking via Mechanistic Interpretability". In: *International Conference on Learning Representations*.

Olsson, C. et al. (2022). "In-context Learning and Induction Heads". In: *Transformer Circuits Thread*. Published 8 March 2022; arXiv:2209.11895.

OpenAI (2023). *GPT-4 Technical Report*.

Power, A., Y. Burda, H. Edwards, I. Babuschkin, and V. Misra (2022). *Grokking: Generalization Beyond Overfitting on Small Algorithmic Datasets*. Preprint.

Ruan, Y., C. J. Maddison, and T. Hashimoto (2024). "Observational Scaling Laws and the Predictability of Language Model Performance". In: *Advances in Neural Information Processing Systems*. Vol. 37.

Schaeffer, R., B. Miranda, and S. Koyejo (2023). "Are Emergent Abilities of Large Language Models a Mirage?" In: *Advances in Neural Information Processing Systems*. Vol. 36.

Srivastava, A., A. Rastogi, A. Rao, A. A. M. Shoeb, A. Abid, A. Fisch, A. R. Brown, et al. (2023). "Beyond the Imitation Game: Quantifying and Extrapolating the Capabilities of Language Models". In: *Transactions on Machine Learning Research*. arXiv:2206.04615.

Stevens, S. S. (1946). "On the Theory of Scales of Measurement". In: *Science* 103.2684, pp. 677–680.

Wei, J. (2022). *137 Emergent Abilities of Large Language Models*. Blog post. URL: https://www.jasonwei.net/blog/emergence.

Wei, J. (2023). *Common Arguments Regarding Emergent Abilities*. Blog post. URL: https://www.jasonwei.net/blog/common-arguments-regarding-emergent-abilities.

Wei, J. et al. (2022). "Emergent Abilities of Large Language Models". In: *Transactions on Machine Learning Research*. Survey Certification; arXiv:2206.07682.

15 The Instrument's Settings

Prompts, formats and demonstrations as nuisance factors

THE CLAIM *"Replacing the demonstrations' labels with random labels barely hurts in-context learning."*

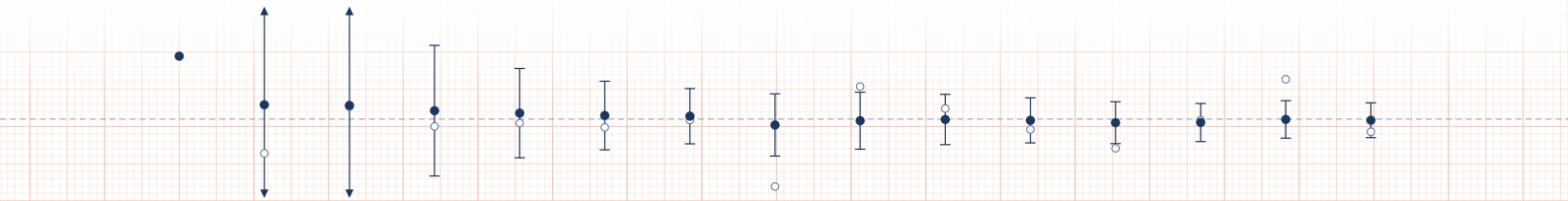
Min et al. (2022), for twelve model variants of up to 175 billion parameters; received as a property of in-context learning

THE VERDICT Narrowed: the cost of wrong labels depends on the model, the task and settings of the evaluation that the original design held fixed, and larger models follow even flipped labels.

IN THIS CHAPTER. In 2022 Min and colleagues replaced the labels of the demonstrations in language-model prompts with random labels and found that twelve model variants, up to GPT-3, lost only a few points; the result was read as evidence that in-context learning recognizes a task rather than learning it. Later work found the cost of wrong labels to depend on the task, the scoring rule, the metric and the model, larger models following even flipped labels, and found that formats, orders of demonstrations and model judges move scores as much as the treatments under study. The method treats the settings of an evaluation as random nuisance factors: we prove how often one random setting reverses a comparison and what sharing and averaging settings buy, how to divide a budget between settings and test items, and how Cohen's κ relates a judge's agreement to its accuracy. In the laboratory, a convolutional network that beats a perceptron on clean digits loses under 15% of 1,620 meaning-preserving formats, and random labels cost a tiny transformer nothing or half its accuracy depending only on how it was trained.

15.1 The claim: demonstrations without correct labels

In 2020 the paper that introduced GPT-3 showed that a language model of 175 billion parameters could carry out a task it had not been trained for when its prompt held a few solved examples before the new input (Brown et al. 2020). To classify the sentiment of a review, the prompt lists a few reviews, each followed by its label, and then the new review; the model's continuation is its answer, and no weight changes. The solved examples are the *demonstrations*, and the procedure is *in-context learning*. It became a standard way to use and to evaluate large language models, and it raised a question: what does the model take from the demonstrations? The obvious answer is the mapping from inputs to labels, what a supervised learner takes from labeled examples.



Min and colleagues tested that answer in a preprint of February 2022, published at EMNLP that December (Min et al. 2022b). They used six language models, GPT-2 Large (774 million parameters), MetaICL (a GPT-2 Large further trained on tasks presented in context), GPT-J, two fairseq models of 6.7 and 13 billion parameters, and GPT-3 (the base davinci model, taken to have 175 billion), each scored in two ways, direct and channel, which made twelve models in the paper's count.[1] Each prompt held 16 demonstrations drawn from a dataset's training data, five draws per dataset, on 16 classification and 10 multiple-choice datasets; the two largest models were run on six of the datasets with three draws.

> FINDING 15.1 (Min et al. 2022b, Section 4). Demonstrations with their correct (gold) labels improved on no demonstrations for nearly every model. Replacing each demonstration's label by one drawn uniformly from the task's label set cost between 0 and 5 points for nearly all models, on average 2.6 points of macro-F1 on classification and 1.7 points of accuracy on multiple choice, and 0.1 to 0.9 points for MetaICL. With 4 to 32 demonstrations the cost stayed between 0.8 and 1.6 points, except 4.4 points for classification with 4; hand-written templates gave the same picture. With every label wrong, GPT-J lost nearly 10 points on classification and still did better than with no demonstrations.

What, then, do demonstrations supply? Inputs from an unrelated corpus cost 3 to 16 points for most variants, random English words as labels 5 to 16 points under direct scoring, and dropping the format of input–label pairs nearly all of the benefit. The authors concluded that demonstrations show the label space, the distribution of inputs and the format, and that models do not rely on the "ground truth input-label mapping" as much as had been thought.

RECEPTION. If the labels barely matter, demonstrations do not teach a task; they help the model recognize one it learned in pretraining. The paper drew this conclusion for a strict definition of learning, and later work took it as the position to test (Wei et al. 2023; Yoo et al. 2022), Pan et al. (2023) naming the two readings task recognition and task learning. The claim of this chapter is the general form: in-context learning barely depends on whether the demonstrations' labels are correct.

WHAT THE WORK GOT RIGHT. The study varied the number of demonstrations, the share of correct labels and the template, released its code, and reported its exceptions; its final version notes that some pairs of dataset and model showed costs up to 14 points and that negated labels had been found to hurt. Its finding that the label space and the format carry much of the benefit has held up. And for its models the claim agrees with later work: GPT-3 davinci, the largest, is among the models that Wei et al. (2023) found unable to push their accuracy below chance by following flipped labels.

[1] Direct scoring picks the label y to which the model gives the highest probability after the prompt and the input x. Channel scoring, introduced by Min et al. (2022a), places the label before the input and picks the y under which x is most probable.

15.2 The experiment: one instrument, twelve models

EXPERIMENTAL DESIGN · MIN ET AL. 2022	
QUESTION	Does in-context learning need the demonstrations' labels to be correct?
UNITS	evaluations of one model on one dataset with one draw of demonstrations
TREATMENT	the labels of the demonstrations
LEVELS	gold labels; labels drawn uniformly from the label set; no demonstrations as a reference
VARIED	six language models (774M to 175B parameters), each with direct and channel scoring; 26 datasets (6 for the two largest models); five draws of demonstrations (three for the two largest)
HELD FIXED	16 demonstrations per prompt; minimal templates; within each model, the scoring rule; macro-F1 for classification, accuracy for multiple choice
RESPONSE	macro-F1 or accuracy on each dataset's evaluation data
ANALYSIS	average over draws, then over datasets; compare gold and random labels model by model

The design treats one nuisance factor as random, the draw of demonstrations, and holds the others fixed: the number of demonstrations, the template, the scoring rule within each variant and the metric; the ablations varied the number and the template for two models on nine datasets. It could show, for each variant, the average cost of random labels over the datasets at these settings. Three gaps separate that from the received claim. A mean over datasets is a suite-level aggregate (Chapter 3) and can hide large costs on some tasks. The models were of one kind, pretrained without instruction tuning, the largest GPT-3. And the cost is a difference of two scores, each measured with a scoring rule, a metric, a template and a number of demonstrations: nothing in the design says that the difference would be the same at other settings. We call such an evaluation pipeline an *instrument*, as a physicist calls a spectrometer one: a procedure that turns a system into a number, with settings that the experimenter chooses. The design measured an effect at one setting of an instrument, and could not say whether the effect belonged to the models or to the models at that setting.

15.3 The post-mortem: the labels matter, depending on the instrument

ANOTHER SETTING OF THE SAME INSTRUMENT. Yoo et al. (2022) started from a counterexample: with GPT-J, accuracy on a hate-speech dataset with gold labels exceeded that with entirely wrong labels by as much as 80 points.[2] With three settings changed, none less valid than the original (direct instead of channel scoring, accuracy instead of macro-F1, 17 classification datasets instead of the five of the ablations),[3] accuracy fell steadily as the share of correct labels fell: the slope was 0.31 for GPT-J over the 17 datasets

[2] The preprint of this paper circulated with Junyeob Kim as first author, and the final version of Min et al. (2022b) cites it as Kim et al. (2022).

[3] What the choice of metric does to a measurement is the subject of Chapter 14.

(R^2 = 0.86), about 31 points between all-correct and all-wrong labels, and from slightly negative to 0.82 for single datasets. The slope was lower with channel scoring, with the calibration of Zhao et al. (2021) and with verbose templates, higher with more demonstrations, and it grew with model size up to GPT-J and levelled off through GPT-3: each an interaction between the treatment and a setting of the instrument.

LARGER MODELS FOLLOW THE LABELS. Wei et al. (2023), in a preprint, flipped the labels of binary tasks, so that following the demonstrations means answering wrongly, for GPT-3, InstructGPT, Codex, PaLM (8 to 540 billion parameters) and the instruction-tuned Flan-PaLM, on seven datasets with 16 demonstrations per class and 100 evaluation items per dataset.[4] Small models stayed at or above chance even with all labels flipped; large ones followed the flipped labels to far below it, as InstructGPT's text-davinci-002 fell from 90.3% to 22.5%. The GPT-3 models, Min et al.'s largest among them, fell at most to chance. With labels replaced by unrelated words such as "foo" and "bar", usable only through the mapping, small models lost much of their accuracy and large ones little. Instruction tuning strengthened both abilities, the use of the prior more: Flan-PaLM never fell below chance with flipped labels, while PaLM fell as low as 31%. With other models, Pan et al. (2023) found that the part of in-context learning that survives wrong labels does not grow with scale while the part that uses the mapping does, and Kossen et al. (2024) that predictions almost always depend on the labels in context.

[4] With 100 items, the accuracy on one dataset has a standard error of up to 5 points; the effects at issue are tens of points.

THE REST OF THE INSTRUMENT. The labels are one choice among many in a prompt, and the others move scores as much. Permuting four demonstrations moved GPT-3 2.7B between 54.3% and 93.4% on the sentiment task SST-2, the model favoring labels frequent among the demonstrations and near the end of the prompt (Zhao et al. 2021), and good orders did not transfer between models (Lu et al. 2022). Sclar et al. (2024) sampled formats from a grammar of meaning-preserving choices, such as separators, casing and spacing, on 53 tasks: LLaMA-2-13B spanned up to 76 points across the formats of one task, and the median spread over ten sampled formats was 7.5 points, not reduced by larger models, more demonstrations or instruction tuning. When one model beat another by at least 2 points under one format, the other won by at least 2 points under another format 14% of the time, for LLaMA-2-13B against LLaMA-2-70B as for LLaMA-2-7B against Falcon-7B; in 76% and 47% of these reversals, both differences were statistically significant at level 0.05 by a one-sided McNemar test on 1,000 items. Over paraphrases of the instructions of 39 tasks, about 6.5 million evaluations of 20 models, rankings agreed only weakly on all ten tasks of one benchmark, and one model ranked first under one paraphrase and ninth under another (Mizrahi et al. 2024).

THE JUDGE. Open-ended answers are often graded by a second model. On MT-bench, shown two similar answers and then the same two in the opposite order, GPT-4 gave the same verdict both times in 65% of cases, GPT-3.5 in 46% and Claude-v1 in 24%, most judges favoring the first position, and

an answer padded with a rephrased copy of its own list won the preference of GPT-3.5 and Claude-v1 in 91% of cases and of GPT-4 in 9% (Zheng et al. 2023). GPT-4 still agreed with human experts on 85% of the votes without ties, more than the experts agreed with each other (81%). With ChatGPT as the judge, the order of the answers alone could make Vicuna-13B beat ChatGPT on 66 of 80 questions (Wang et al. 2024).

VERDICT. The measurement stands: for the twelve model variants of Min et al. (2022b), at their settings and averaged over datasets, random labels cost 0 to 5 points. The general claim does not. The cost depends on the model, rising with scale and falling with instruction tuning; on the task, where wrong labels cost GPT-J up to 80 points on one dataset; and on settings the design held fixed: the scoring rule, the metric, the template and the number of demonstrations. Formats, orders and judges move scores as much as the treatment does, and an effect measured at one setting of an instrument is an effect at that setting.

NARROWED

15.4 The method: settings as random factors

The labels were the treatment of Min et al.'s experiment; the template, the scoring rule and the number of demonstrations were settings of the instrument, nuisance factors with a distribution, to be treated as Chapter 1 treats seeds and Chapter 3 tasks.

Definition 15.2 (Instrument, settings, estimand). An *evaluation instrument* turns a system and a test item into a score. Its *settings* are the choices that fix how: the wording and format of the prompt; the number, choice, labels and order of the demonstrations; the decoding rule; the scoring rule; and, for open-ended answers, the judge. A *setting distribution* $\mathcal{S}$ is a probability distribution over the settings that the experimenter regards as equally legitimate for the claim. The *estimand* of an arm, a system or a system under a treatment, is its mean score μ_a over settings drawn from $\mathcal{S}$ and test items drawn from the task's distribution of items.

The setting distribution is part of the claim: a claim about in-context learning is a claim about a distribution of settings, to be measured on a sample from it. Choosing $\mathcal{S}$ is a decision, like the choice of an aggregate: a grammar of formats (Sclar et al. 2024), the accepted paraphrases of an instruction (Mizrahi et al. 2024), all orders of a set of demonstrations. The best score over settings answers another question, what a tuned prompt can do, and, chosen on the test items, it carries the bias of a best of k (Proposition 1.6).[5]

A RANDOM-EFFECTS MODEL. Let arms $a = 1, 2$ be scored under settings drawn independently from $\mathcal{S}$, and write the score of arm a under setting f as

$$Y_{af} = \mu_a + u_f + v_{af} + \varepsilon_{af}. \tag{15.1}$$

[5] Psycholinguistics met the same problem with words. Clark (1973) called the habit of drawing conclusions about language from a sample of words analyzed as if the words were fixed the language-as-fixed-effect fallacy; Ball et al. (2024), in a preprint, carry the argument to claims about the capabilities of language models.

The setting's *main effect* u_f moves both arms alike, as a format that is hard for every model does; the *interaction* v_{af} is the part of the setting's effect particular to the arm; and ε_{af} is the noise of one evaluation, from a finite sample of items or from sampled decoding. The model is the two-way layout of Theorem 1.8, with the arms, which are fixed, in place of one of the two seeds. The comparison of interest is the sign of $\Delta = \mu_1 - \mu_2$, and a *reversal* occurs when the measured comparison has the opposite sign.

Theorem 15.3 (Ranking under random settings). *In model (15.1), let all the effects u_f, v_{af} and ε_{af}, for both arms and all settings, be independent and normal with mean zero and variances σ_u^2, σ_v^2 and σ_ε^2; let $\Delta \neq 0$ and $s^2 = 2\sigma_v^2 + 2\sigma_\varepsilon^2 > 0$.*

(i) If both arms are scored under the same single setting, the comparison reverses with probability $\Phi(-|\Delta|/s)$, whatever the value of σ_u.

(ii) If both arms are scored under the same F settings and compared by their average scores, the comparison reverses with probability $\Phi(-\sqrt{F}|\Delta|/s)$.

(iii) If each arm is scored under F settings of its own, drawn independently of the other arm's, the comparison reverses with probability $\Phi\big(-\sqrt{F}|\Delta|/\sqrt{s^2 + 2\sigma_u^2}\big)$.

(iv) Without normality, if the effects are independent with mean zero and finite variances, the compared difference has mean Δ and variance s^2/F in (ii) and $(s^2 + 2\sigma_u^2)/F$ in (iii), and the probability of a reversal is at most $s^2/(F\Delta^2)$ and $(s^2 + 2\sigma_u^2)/(F\Delta^2)$.

Proof. The plan is to write each comparison as Δ plus an average of independent terms of mean zero, and to read the probability of a reversal off its distribution. We take $\Delta > 0$; for $\Delta < 0$ exchange the names of the arms.

Shared settings. Under setting f the difference of the two scores is

$$D_f = Y_{1f} - Y_{2f} = \Delta + (v_{1f} - v_{2f}) + (\varepsilon_{1f} - \varepsilon_{2f}),$$

in which the main effect u_f has cancelled. The four effects on the right are independent with mean zero, so D_f has mean Δ and variance $2\sigma_v^2 + 2\sigma_\varepsilon^2 = s^2$, and the differences under distinct settings are independent because they involve disjoint sets of effects. Their average $\bar{D}$ over F settings therefore has mean Δ and variance s^2/F. Under normality $\bar{D}$ is normal, and the comparison reverses when $\bar{D} < 0$, an event of probability $\Phi(-\sqrt{F}\Delta/s)$; the event $\bar{D} = 0$ has probability zero. This proves (ii), and (i) is its case $F = 1$.

Separate settings. Let arm 2 be scored under settings $f_1', \dots, f_F'$ drawn independently of arm 1's settings $f_1, \dots, f_F$. The difference of the two averages is

$$\Delta + \frac{1}{F}\sum_{l=1}^{F}\big(u_{f_l} - u_{f_l'} + v_{1f_l} - v_{2f_l'} + \varepsilon_{1f_l} - \varepsilon_{2f_l'}\big),$$

an average of F independent terms, each a sum of six independent effects

with mean zero and total variance $2\sigma_u^2 + s^2$. The argument for shared settings, with $s^2 + 2\sigma_u^2$ in place of s^2, gives (iii).

Without normality. The means and variances above used only independence and finite variances. A reversal requires the compared difference to fall at least Δ below its mean, and Chebyshev's inequality bounds the probability of that by the variance divided by Δ^2, which gives (iv). ■

Sharing settings is blocking (Section A.7): it removes the main effect, whose variance is the covariance of the two arms' scores across settings (Exercise 15.1), but not the interaction, and which of the two dominates is an empirical matter. In the laboratory of this chapter the scores of two networks across formats have correlation 0.92, and sharing removes most of the variance. In the data of Sclar et al. (2024), two models put a pair of formats in the same order only 56 to 61% of the time; for normal scores this probability is $(1 + \tau)/2$ with Kendall's $\tau = \frac{2}{\pi} \arcsin \rho$ (Lemma 5.8), so the rates correspond to correlations ρ of 0.18 to 0.34, and most of a format's effect is particular to the model. *Averaging works like* $\sqrt{F}$: if $s = 1.5|\Delta|$, one shared setting reverses the comparison with probability $\Phi(-2/3) = 0.25$, and $F \geq (z_{1-\alpha} s/\Delta)^2$ settings, here seven for $\alpha = 0.05$, bring the probability below α, as averaging seeds does for training (Corollary 1.10). *Separate settings add the main effect twice*: two numbers copied from two papers, each measured with its authors' own prompt, are case (iii) with $F = 1$, close to a coin flip when some formats are hard for every model.

Under shared settings, the sample variance S_D^2 of the per-setting differences estimates s^2 without bias, and $\bar{D} \pm t_{F-1,0.975} S_D/\sqrt{F}$ is an exact 95% interval for Δ under the model (Theorem A.1). A test over the items of one setting, such as McNemar's (Proposition A.5), treats the setting as fixed and sees only ε: it can declare a difference that another setting reverses, as it did for most of the reversals of Sclar et al. (2024), just as in Chapter 1 a test over test sentences could not see the seed.

LEADERBOARDS. With more than two arms, the settings reorder a ranking.

Corollary 15.4 (Rankings under random settings). *Let arms $a = 1, \ldots, A$ with distinct means μ_a be scored under shared settings, and suppose that for each pair $a < b$ the per-setting differences $Y_{af} - Y_{bf}$ are independent across settings and normal with mean $\mu_a - \mu_b$ and standard deviation $s_{ab} > 0$. Let $p_{ab} = \Phi\big(-\sqrt{F}|\mu_a - \mu_b|/s_{ab}\big)$. The ranking by the average scores over F settings has expected Kendall's τ with the ranking by the means*

$$\mathbb{E}\hat{\tau} = \binom{A}{2}^{-1} \sum_{a<b} (1 - 2p_{ab}),$$

and two rankings by averages over two independent sets of F settings have expected Kendall's τ equal to $\binom{A}{2}^{-1} \sum_{a<b} (1 - 2p_{ab})^2$.

PROOF. Kendall's $\hat{\tau}$ (Proposition A.15) is the average, over the $\binom{A}{2}$ pairs of arms, of the product of the signs with which the two rankings order

the pair. For the pair $a < b$, the average difference $\bar{D}_{ab}$ over the shared settings has the sign of $\mu_a - \mu_b$ with probability $1 - p_{ab}$ and the opposite sign with probability p_{ab}, by part (ii) of Theorem 15.3, whose proof used only the normality of the per-setting differences and their independence across settings; ties have probability zero. The expected product of the signs is therefore $(1 - p_{ab}) - p_{ab}$, and linearity of expectation gives the first formula, with no assumption on the dependence between pairs. Two independent sets of settings give independent averages $\bar{D}_{ab}$ and $\bar{D}'_{ab}$, each of expected sign $(1 - 2p_{ab})\,\mathrm{sgn}(\mu_a - \mu_b)$, so the expected product of their signs is $(1 - 2p_{ab})^2$. ∎

For three arms with means 0, 1 and 2 points and $s_{ab} = 2$ points, one setting gives $\mathbb{E}\hat{\tau} = 0.48$ and eight settings 0.89. A leaderboard built on one prompt template is a ranking under one setting, and the second formula is the expected agreement of two leaderboards built on different templates. Hardt (2026) argues that rankings are what benchmarks reliably deliver; the corollary says how many settings that takes.

TWO FACETS: SETTINGS AND ITEMS. The noise ε_{af} is mostly that of a finite test set, a second random factor. Let $c_a(f, z) \in \{0, 1\}$ record whether arm a answers item z correctly under setting f, and let $d(f, z) = c_1(f, z) - c_2(f, z)$, whose mean over settings and items is Δ. For a setting f_1 drawn from $\mathcal{S}$ and an independent item Z_1, decompose d as in Theorem 1.8 (Theorem A.12 is the one-way case), with settings and items as the two seeds: the setting effect $d_{\mathrm{s}}(f) = \mathbb{E}\, d(f, Z_1) - \Delta$, the item effect $d_{\mathrm{i}}(z) = \mathbb{E}\, d(f_1, z) - \Delta$ and the interaction $d_{\mathrm{si}}(f, z) = d(f, z) - \Delta - d_{\mathrm{s}}(f) - d_{\mathrm{i}}(z)$, with *variance components* σ^2_{s}, σ^2_{i} and σ^2_{si}. An evaluation may cross the factors, scoring the same m items under each of F settings, or nest the items in the settings, scoring fresh items under each setting.

Proposition 15.5 (Settings and items). *Let $f_1, \dots, f_F$ be settings drawn independently from $\mathcal{S}$ and let the items be drawn independently of them and of each other from the task's distribution, with $\mathbb{E}\, d(f_1, Z_1)^2 < \infty$. Let $\bar{d}$ be the average of d over all evaluated pairs of setting and item.*

(i) In the crossed design, with items $Z_1, \dots, Z_m$ scored under every setting, $\mathrm{Var}\,\bar{d} = \sigma^2_{\mathrm{s}}/F + \sigma^2_{\mathrm{i}}/m + \sigma^2_{\mathrm{si}}/(Fm)$.

(ii) In the nested design, with items $Z_{l1}, \dots, Z_{lm}$ scored under setting f_l only, $\mathrm{Var}\,\bar{d} = \sigma^2_{\mathrm{s}}/F + (\sigma^2_{\mathrm{i}} + \sigma^2_{\mathrm{si}})/(Fm)$.

(iii) For a budget of $N = Fm$ evaluations per arm, the nested variance $\sigma^2_{\mathrm{s}}/F + (\sigma^2_{\mathrm{i}} + \sigma^2_{\mathrm{si}})/N$ decreases in F and is smallest with a fresh setting for every item. If $\sigma_{\mathrm{i}} > 0$, the crossed variance, as a function of a real $F > 0$, is smallest at $F^\star = \sqrt{N}\,\sigma_{\mathrm{s}}/\sigma_{\mathrm{i}}$, where it equals $2\sigma_{\mathrm{s}}\sigma_{\mathrm{i}}/\sqrt{N} + \sigma^2_{\mathrm{si}}/N$.

In the crossed design with $F, m \geq 2$, the mean squares of the $F \times m$ table of differences estimate the three components without bias, by Theorem 1.8.

PROOF. The plan is to split $\bar{d} - \Delta$ into averages of the three kinds of effects, which are pairwise uncorrelated, and to add their variances. The proof

of Theorem 1.8 shows that $d_{\mathrm{s}}(f_l)$, $d_{\mathrm{i}}(Z)$ and $d_{\mathrm{si}}(f_l, Z)$ have mean zero, that each of d_{i} and d_{si} averages to zero over the item for every fixed setting, that d_{si} also averages to zero over the setting for every fixed item, and that two such effects are uncorrelated unless they are the same effect at the same arguments.

Crossed design. Averaging the decomposition over the table gives $\bar{d} - \Delta = \bar{d}_{\mathrm{s}} + \bar{d}_{\mathrm{i}} + \bar{d}_{\mathrm{si}}$, averages of F, m and Fm pairwise uncorrelated effects with variances σ_{s}^2, σ_{i}^2 and σ_{si}^2, and the three averages are uncorrelated with each other. Their variances are σ_{s}^2/F, σ_{i}^2/m and $\sigma_{\mathrm{si}}^2/(Fm)$, and they add.

Nested design. Here $\bar{d} - \Delta = \bar{d}_{\mathrm{s}} + \frac{1}{Fm}\sum_{l,j} w_{lj}$ with $w_{lj} = d_{\mathrm{i}}(Z_{lj}) + d_{\mathrm{si}}(f_l, Z_{lj})$. Given f_l, the term w_{lj} has conditional mean zero, because both of its effects average to zero over the item for a fixed setting. Two terms w_{lj} and $w_{lj'}$ with $j \neq j'$ are independent given f_l, so the conditional mean of their product, and hence its mean, is zero; terms with different settings involve disjoint independent variables and are independent; and $\mathbb{E}[d_{\mathrm{s}}(f_l) w_{lj}] = \mathbb{E}\big[d_{\mathrm{s}}(f_l)\,\mathbb{E}[w_{lj} \mid f_l]\big] = 0$. Each w_{lj} has variance $\sigma_{\mathrm{i}}^2 + \sigma_{\mathrm{si}}^2$, since its two effects share an item and one of them is the interaction, so they are uncorrelated. Adding σ_{s}^2/F for $\bar{d}_{\mathrm{s}}$ and $(\sigma_{\mathrm{i}}^2 + \sigma_{\mathrm{si}}^2)/(Fm)$ for the average of the Fm terms gives (ii).

A budget. With $m = N/F$, the nested variance is σ_{s}^2/F plus a constant, which decreases in F. The crossed variance is $g(F) = \sigma_{\mathrm{s}}^2/F + \sigma_{\mathrm{i}}^2 F/N + \sigma_{\mathrm{si}}^2/N$, a convex function of $F > 0$ with derivative $-\sigma_{\mathrm{s}}^2/F^2 + \sigma_{\mathrm{i}}^2/N$, which vanishes at $F^\star$; substituting $F^\star$ gives two equal terms $\sigma_{\mathrm{s}}\sigma_{\mathrm{i}}/\sqrt{N}$ and the constant. The last statement is Theorem 1.8 applied to the table $d(f_l, Z_j)$, a function of independent settings and independent items. ■

Crossing settings with one fixed test set reduces only the parts of the variance that involve the setting; the item effect leaves a floor of σ_{i}^2/m, the noise of the test set of Chapter 1. A setting drawn at random for each item averages over settings at no cost in evaluations, with variance $(\sigma_{\mathrm{s}}^2 + \sigma_{\mathrm{i}}^2 + \sigma_{\mathrm{si}}^2)/m$ against $\sigma_{\mathrm{s}}^2 + (\sigma_{\mathrm{i}}^2 + \sigma_{\mathrm{si}}^2)/m$ for one setting. Its price is information: it estimates the mean but not the spread over settings, which a user who must pick one prompt needs, and a few crossed settings beside many nested ones buy both.[6] The analysis must match the design: items scored under several settings are not independent observations, a point Miller (2024) makes, in a preprint, for evaluation questions that come in clusters.

[6] In generalizability theory (Cronbach et al. 1972) the random factors of a measurement are its *facets*; estimating their components is a G study, and choosing the numbers of levels for a future measurement, as part (iii) does, a D study.

15.5 The method: judges and agreement

A model that grades the answers of others is an instrument with settings of its own, among them its prompt, the order in which it sees the answers and its sampling temperature, and its quality is measured against a reference, usually human labels. Let the judge's verdict J and the reference's verdict R on a random item be binary, with $\pi_J = \mathbb{P}(J = 1)$ and $\pi_R = \mathbb{P}(R = 1)$ in $(0, 1)$. Their *observed agreement* is $p_o = \mathbb{P}(J = R)$, the agreement expected by chance from independent verdicts with the same rates is $p_e = \pi_J\pi_R + (1-\pi_J)(1-\pi_R)$, and *Cohen's* κ (Cohen 1960) is the share of the agreement possible beyond

chance that is achieved,

$$\kappa = \frac{p_o - p_e}{1 - p_e}.$$

When the reference is the truth, p_o is the judge's accuracy, with sensitivity $\mathrm{se} = \mathbb{P}(J = 1 \mid R = 1)$ and specificity $\mathrm{sp} = \mathbb{P}(J = 0 \mid R = 0)$.

Proposition 15.6 (Agreement, accuracy and κ). *(i) The coefficient is*

$$\kappa = \frac{2\,\mathrm{Cov}(J, R)}{\pi_J(1 - \pi_R) + \pi_R(1 - \pi_J)}.$$

Hence $\kappa = 0$ exactly when J and R are independent, $\kappa = 1$ exactly when $J = R$ almost surely, and $|\kappa| \le |\mathrm{Corr}(J, R)|$, with equality when $\pi_J = \pi_R$.

(ii) The accuracy of the judge is $p_o = \pi_R\,\mathrm{se} + (1 - \pi_R)\mathrm{sp}$, and $\mathrm{Cov}(J, R) = \pi_R(1 - \pi_R)(\mathrm{se} + \mathrm{sp} - 1)$. If the judge errs with the same probability q on both kinds of items, then its accuracy is $1 - q$ and

$$\kappa = \frac{2\pi_R(1 - \pi_R)(1 - 2q)}{2\pi_R(1 - \pi_R) + q(1 - 2\pi_R)^2},$$

which equals $1 - 2q = 2p_o - 1$ when $\pi_R = \frac{1}{2}$ and tends to zero as π_R tends to 0 or 1 with $q > 0$ fixed.

(iii) If J and R are conditionally independent given a binary truth T, then $\mathrm{Corr}(J, R) = \mathrm{Corr}(J, T)\,\mathrm{Corr}(R, T)$. In particular, if T is equally likely to be 0 or 1 and the judge and the reference err with probabilities q_J and q_R whatever the value of T, then $\kappa_{JR} = (1 - 2q_J)(1 - 2q_R) = \kappa_{JT}\,\kappa_{RT}$.

Proof. The plan is to express both agreements through the joint probability $c = \mathbb{P}(J = 1, R = 1)$, which turns κ into a covariance, and then to compute that covariance in the two models of (ii) and (iii).

Part (i). Since $\mathbb{P}(J = 0, R = 0) = 1 - \pi_J - \pi_R + c$, the observed agreement is $p_o = 1 - \pi_J - \pi_R + 2c$, while expanding p_e gives $1 - \pi_J - \pi_R + 2\pi_J\pi_R$. Hence $p_o - p_e = 2(c - \pi_J\pi_R) = 2\,\mathrm{Cov}(J, R)$ and $1 - p_e = \pi_J(1 - \pi_R) + \pi_R(1 - \pi_J) > 0$, which is the formula. Binary variables are independent exactly when their covariance vanishes, which gives the case $\kappa = 0$; and $\kappa = 1$ exactly when $p_o = 1$. The correlation is $\mathrm{Cov}(J, R)$ divided by $\sqrt{\pi_J(1 - \pi_J)\pi_R(1 - \pi_R)}$, the geometric mean of $\pi_J(1 - \pi_R)$ and $\pi_R(1 - \pi_J)$. The arithmetic mean of two positive numbers is at least their geometric mean, with equality when they are equal, which here means $\pi_J = \pi_R$; so the denominator of κ is at least twice that of the correlation, which gives the inequality.

Part (ii). The first formula is the law of total probability. With $c = \pi_R\,\mathrm{se}$ and $\pi_J = \pi_R\,\mathrm{se} + (1 - \pi_R)(1 - \mathrm{sp})$,

$$\mathrm{Cov}(J, R) = \pi_R\,\mathrm{se} - \pi_R\pi_J = \pi_R(1 - \pi_R)\,\mathrm{se} - \pi_R(1 - \pi_R)(1 - \mathrm{sp}),$$

which is $\pi_R(1 - \pi_R)(\mathrm{se} + \mathrm{sp} - 1)$. For $\mathrm{se} = \mathrm{sp} = 1 - q$ the accuracy is $1 - q$, $\pi_J = \pi_R + q(1 - 2\pi_R)$, and the denominator of part (i) is $\pi_J + \pi_R - 2\pi_J\pi_R =$

$2\pi_R(1-\pi_R)+q(1-2\pi_R)^2$, which gives the displayed formula. At $\pi_R=\frac{1}{2}$ it reduces to $1-2q$. As π_R tends to 0 or 1 the numerator tends to zero and the denominator to $q>0$.

Part (iii). A function of a binary T is affine in T, so $\mathbb{E}[J\mid T]=\alpha_J+\beta_J T$ and $\mathbb{E}[R\mid T]=\alpha_R+\beta_R T$ for constants $\alpha_J,\beta_J,\alpha_R,\beta_R$. Conditional independence makes the conditional covariance of J and R vanish, so the law of total covariance gives $\operatorname{Cov}(J,R)=\operatorname{Cov}(\mathbb{E}[J\mid T],\mathbb{E}[R\mid T])=\beta_J\beta_R\operatorname{Var}T$, while $\operatorname{Cov}(J,T)=\beta_J\operatorname{Var}T$ and $\operatorname{Cov}(R,T)=\beta_R\operatorname{Var}T$. Hence $\operatorname{Cov}(J,R)\operatorname{Var}T=\operatorname{Cov}(J,T)\operatorname{Cov}(R,T)$; dividing by the product of the standard deviations of J, R and T gives the identity of correlations. Under the symmetric model with $\mathbb{P}(T=1)=\frac{1}{2}$, each verdict equals 1 with probability $\frac{1}{2}$, so every κ among J, R and T equals the corresponding correlation by part (i), and $\kappa_{JT}=1-2q_J$, $\kappa_{RT}=1-2q_R$ by part (ii). ■

Part (ii) is the reason to report κ with the base rate. A judge right 90% of the time on both kinds of items has $\kappa=0.80$ when half the items are positive, 0.59 when a tenth are and 0.24 when a fiftieth are, the first paradox of Feinstein and Cicchetti (1990); and by part (i), κ penalizes verdicts with the wrong base rate, which accuracy does not.[7]

Part (iii) is Spearman's correction for attenuation (Spearman 1904) for verdicts. If two human annotators err independently with the same probability q_R, their agreement estimates $\kappa_{RR'}=(1-2q_R)^2$, and the judge's κ with the truth is $\kappa_{JR}/\sqrt{\kappa_{RR'}}$. On MT-bench, orienting each pair of answers at random makes chance agreement $\frac{1}{2}$ and $\kappa=2p_o-1$, so the agreements of 85% between GPT-4 and the experts and of 81% between experts are $\kappa=0.70$ and 0.62 (Zheng et al. 2023); under the model an expert errs with probability $(1-\sqrt{0.62})/2=0.11$ and GPT-4 with $(1-0.70/\sqrt{0.62})/2=0.06$. The assumptions are strong, since hard items correlate errors and the votes without ties are a selection, but the model explains the comparison Zheng et al. drew: a judge that agrees with experts as often as they agree with each other is as accurate as one expert.

[7] Landis and Koch (1977) proposed verbal labels for ranges of κ, such as "substantial" for 0.61 to 0.80, which are widely quoted. By part (ii) the same label can fit judges of quite different accuracy.

THE JUDGE'S SETTINGS. The order in which a judge sees two answers is a setting, and a preference for the first position favors whichever system is shown first. Judging every pair in both orders, *counterbalancing* in the language of psychology, shows each system first equally often, and the preference cancels (Exercise 15.4). The share of verdicts that change with the order, 35% for GPT-4 on similar answers, belongs in the report, as does a test–retest κ from two sampled runs.

15.6 In the laboratory: formats and labels

The first lab scores six digit classifiers under test-time *formats*, meaning-preserving changes of the test images in the role of the formats of a prompt; the second trains a tiny transformer to classify digits in context.

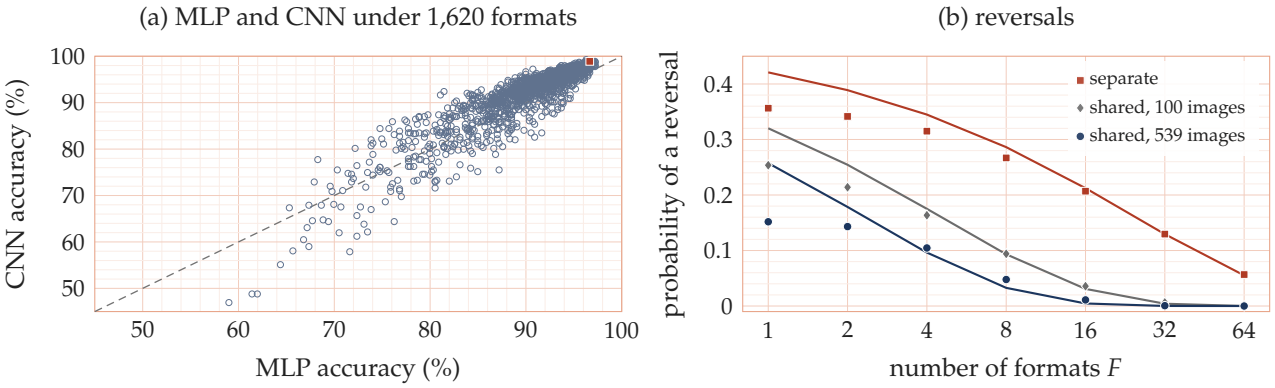


Figure 15.1. Lab 15.1: the MLP of Chapter 1 and the CNN of Chapter 6, each trained once (split 0, seed 0), scored on the 539 test images under each of 1,620 formats. (a) Each hollow mark is one format; the dashed line is equal accuracy, the vermilion square the clean format. (b) The probability that the MLP comes out ahead when the networks are compared by their average over F random formats, the same for both (shared) or drawn separately, on all images or on 100 fresh images per format: observed in 20,000 resamples (marks; binomial 95% intervals are smaller than the marks) and predicted by Theorem 15.3 and Proposition 15.5 from the variance components of the table (lines).

LAB 15.1 · CLASSIFIERS UNDER RANDOM FORMATS

SCRIPT
`15-instruments-formats.py`

SETUP. Six classifiers trained once on split 0 (seed 0), with inputs standardized by one mean and one standard deviation for all pixels: softmax regression, the MLP of Chapter 1, a one-layer MLP of width 128, the MLP with tanh units, an MLP of width 256 trained with Adam, and the CNN of Chapter 6. A grammar of seven atomic changes of the 539 test images (a partial one-pixel shift, contrast, a grey background, blur, pixel noise, thicker strokes, a gamma curve) gives 1,620 formats; every network is scored on every image under every format, and formats and images are resampled from this table 4,000 to 20,000 times per design. About a minute.

PREDICT FIRST. The CNN beats the MLP by 2.2 points on the clean images. Under what share of the formats will the MLP win? How many formats must be averaged before a comparison reverses less than once in twenty, with shared formats and with separate ones?

LOOK FOR. The spread of the formats along the diagonal and across it; the reversal rate against F, against the theorem; where a budget of 1,000 evaluations is best spent; how many formats a six-network leaderboard needs.

WHAT HAPPENED. Numbers for the full table are exact for these images and formats, and resampled rates carry 95% intervals of at most ±0.01. Under the clean format the CNN is ahead by 2.23 points (98.89% against 96.66%). Over the 1,620 formats the six networks average between 90.0% (softmax regression) and 92.1% (the CNN), against 96.5% to 98.9% on clean images, and each spans more than 30 points; the CNN is the most accurate

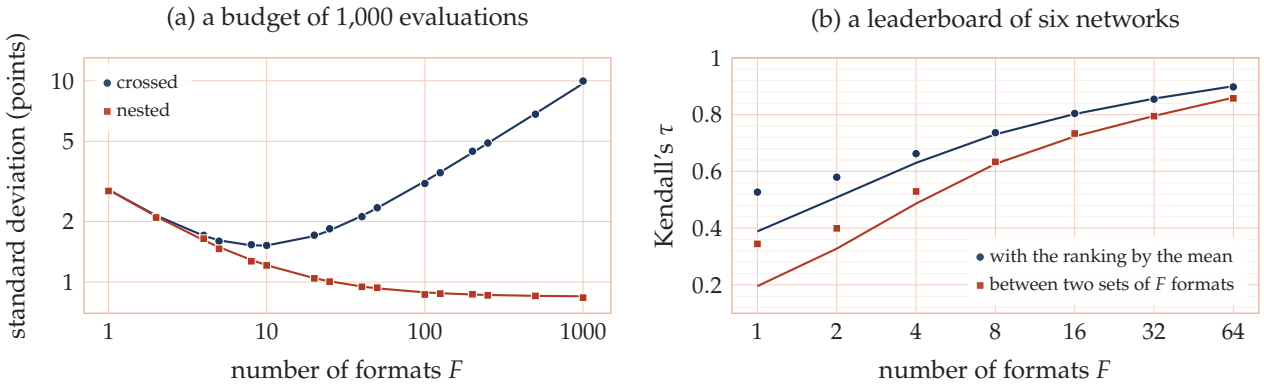


Figure 15.2. Lab 15.1, continued. (a) Standard deviation of the estimated difference between the CNN and the MLP when 1,000 evaluations per network are spent on F formats and $1000/F$ images, the same images for every format (crossed) or fresh images for each (nested): 4,000 resamples (marks) and Proposition 15.5 (lines). (b) Kendall's τ between the ranking of the six networks by their average over F random formats and the ranking by the mean over all formats, and between the rankings by two independent sets of F formats: means of 5,000 resamples (marks; 95% intervals smaller than the marks) and Corollary 15.4 (lines).

on average, by 1.79 points over the MLP, and the least accurate in the worst case, 46.9%. The MLP wins under 15.0% of the formats and ties under 1.0% (Figure 15.1a). The formats move both networks together: their scores have correlation 0.92 across formats, the setting's main effect has standard deviation $\hat{\sigma}_u = 6.0$ points, and the per-format difference only $s = 2.75$. The interaction has a structure, like the atomic perturbations of Sclar et al. (2024): high contrast, a grey background, thicker strokes and a gamma below one, which all brighten the image, cost the CNN more than the MLP, and under the three formats most favorable to the MLP, which combine all four, it wins by 12.6 to 13.7 points. By the criterion of Sclar et al. (2024), a lead of the CNN by 2 points under one format becomes a lead of the MLP by 2 points under an independently drawn format with probability 0.08.

With shared formats (Figure 15.1b), the reversal rate falls from 0.15 for one format to 0.05 for eight and 0.011 for sixteen; the normal model predicts 0.26, 0.03 and 0.005. For one format the model overstates the reversals, because the per-format differences are skewed (skewness −1.5): the CNN wins most formats by one to three points and loses a few by much more. From four shared formats on, the predictions are within 0.015 of the observed rates, though far in the tail the observed rate stays the larger, the skewness of an average decaying only like $1/\sqrt{F}$. Separate formats, which add the main effect twice, reverse the comparison 36% of the time with one format (0.42 predicted) and still 13% with 32: four shared formats (0.10) do better than 32 separate ones. Scoring each format on 100 fresh images, as several studies of prompts did, adds the noise of the items: one format then reverses 25% of the comparisons.

The per-image differences have components $\hat{\sigma}_s = 2.75$, $\hat{\sigma}_i = 9.66$ and $\hat{\sigma}_{si} = 24.9$ points: the networks disagree on 7.3% of the pairs of format and image, and most of the variance of one comparison is the image. With

1,000 evaluations per network (Figure 15.2a), one format gives a standard deviation of 2.83 points, the crossed design is best near $F^\star = 9$ formats of 111 images, at 1.52, and the nested design reaches 0.84 with a fresh format for every image; resampled and predicted values agree to within 3%, since resampling from the table makes the assumptions exact. On the 539 test images, one format for all of them gives a standard deviation of 2.98 points and a reversal probability of 0.27, a fresh random format for each image 1.16 points and 0.06, at the same cost.

The leaderboard (Figure 15.2b), whose closest pairs are 0.04 and 0.26 points apart, needs more formats. On one format its ranking agrees with the ranking by the mean with $\tau = 0.53$, two such leaderboards agree with $\tau = 0.34$, and the CNN tops 65% of them; with 64 formats, 0.90 and 88%. The corollary predicts 0.39 for one format, as the theorem overstates the pair's reversals, and is within 0.01 from eight formats on.

LAB 15.2 · WHAT RANDOM LABELS COST

SCRIPT
15-instruments-icl.py

SETUP. A transformer of two layers, width 64 and no positional encodings (72,458 parameters) reads eight demonstrations, each a digit image of split 0 with its label, and a query image, and predicts the query's label. Each training sequence holds four images of each of two digit classes, the query from one of them; in a fraction ω of the sequences the ten labels are shuffled for the whole sequence. Five values of ω, three seeds each, 1,500 steps of AdamW. Evaluation on 2,000 sequences of test images per condition, with 1, 2, 4 and 8 demonstrations per class. About five minutes on a shared laptop.

PREDICT FIRST. For the network trained with fixed labels ($\omega = 0$), how much do random labels cost? For one trained with 1% shuffled sequences? Does the cost grow or shrink with more demonstrations?

LOOK FOR. The value of ω at which the labels begin to matter; flipped labels; accuracy without demonstrations; the cost of random labels against the number of demonstrations.

The design follows Chan et al. (2022), who found in transformers trained on sequences of Omniglot characters that in-context learning emerges when the data are bursty and hold many rare classes, and is stronger when the meaning of items varies from sequence to sequence, here when labels are shuffled. All conditions are scored against the query's true class: no demonstrations; gold labels; random labels, each demonstration labeled with one of the two classes present by a fair coin, which keeps the label space and destroys the mapping, as in Min et al. (2022b); and flipped labels, as in Wei et al. (2023).

WHAT HAPPENED. The network trained with fixed labels is Min et al.'s finding in miniature (Figure 15.3a): with four demonstrations per class it scores $99.4 \pm 0.1\%$ with gold labels (mean and 95% t-interval over three seeds), $99.1 \pm 0.2\%$ with random labels and $98.9 \pm 0.2\%$ with flipped ones. It has learned the digits in its weights and ignores the labels in context, yet

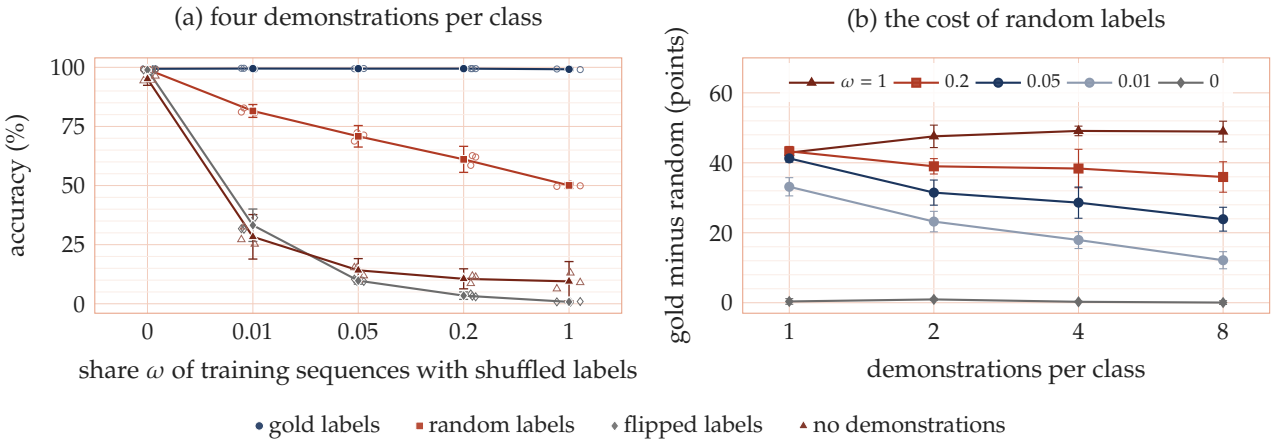


Figure 15.3. Lab 15.2: a two-layer transformer trained for in-context classification of digits with a share ω of training sequences whose labels are shuffled, three seeds per value of ω, evaluated on sequences of test images with the query's true class as the answer. (a) Accuracy with four demonstrations per class: hollow marks are networks, filled marks means with 95% t-intervals over the three seeds. (b) Accuracy with gold labels minus accuracy with random labels, against the number of demonstrations per class: means with 95% t-intervals over seeds.

the demonstrations help (without them it scores $95.1 \pm 2.7\%$), because they show which two classes are in play, the label space, which random labels keep. One shuffled sequence in a hundred changes the picture: random labels then cost 17.9 ± 2.4 points and flipped labels lower the accuracy to $33.2 \pm 6.9\%$, the network following the demonstrations two times in three. With $\omega = 0.05$ it follows flipped labels nine times in ten, and with $\omega = 1$ random labels cost 49.1 ± 1.4 points, leaving $50.1 \pm 1.3\%$, chance between the two classes; without demonstrations these networks score 9% to 28% on average.

The cost also depends on a setting of the instrument (Figure 15.3b). For ω between 0.01 and 0.2 it falls as demonstrations are added, for $\omega = 0.01$ from 33.2 ± 2.6 points with one demonstration per class to 12.1 ± 2.4 with eight, while for $\omega = 1$ it rises slightly, from 42.9 to 48.9; a reading we have not tested is that random labels contradict one another more as they accumulate, so that a network that also knows the digits in its weights falls back on them. Averaged over the fifteen networks, random labels cost 26.9 points, an average of 0.2 to 49.6 that describes none of them. The miniature reproduces both sides of the post-mortem, the insensitive models of Min et al. (2022b) and the label-following ones of Wei et al. (2023), with the training distribution in the role of scale and instruction tuning; it does not show that scale acts the same way.

15.7 Protocol

PROTOCOL 15.1 · EVALUATING UNDER RANDOM SETTINGS

- ☐ Write down the instrument: prompt, format, demonstrations and their order, decoding, scoring rule, judge. Decide, before scoring, the setting distribution the claim must cover.
- ☐ Sample settings from that distribution; never choose them on the test items. Report a tuned setting only as a best of k, with k.
- ☐ Score every arm under the same settings and on the same items, and compute comparisons within settings.
- ☐ Size the design from a pilot's variance components: enough settings for the reversal probability you can accept, and fresh items for each setting when the budget is fixed.
- ☐ Report the mean with an interval over settings and items, the spread over settings, the worst setting, and the share of settings on which the ranking reverses.
- ☐ Report a treatment effect setting by setting and model by model; an average over heterogeneous models is a choice of aggregate, not a property of each.
- ☐ For a judge, report κ against a reference with an interval and the base rate, the reference's own agreement, consistency under swapped order and test–retest agreement; counterbalance positions.
- ☐ Publish an *instrument card*: the instrument with every setting, prompt and demonstration; the setting distribution and the sampling; the design; the results with their variance components; the judge's reliability; and the code.

The protocol makes a measurement honest about its instrument; it does not choose the setting distribution, which is part of the question: a grammar of separators and casing says nothing about paraphrases of the instruction or the prompts users write. It treats settings as nuisance, while for a user who will choose one prompt the best setting is the quantity of interest, and for a deployment the worst. And twenty settings multiply the cost of an evaluation by twenty. When that is too much, draw a fresh setting for each item, which costs nothing; cross a handful of settings with a subset of the items to estimate the spread; and say which settings were held fixed.

Exercises

15.1 CONSOLIDATION. (Unequal arms.) Without model (15.1), let the pairs (Y_{1f}, Y_{2f}) be independent and identically distributed with finite variances. Show that the average difference over F settings has variance $\mathrm{Var}(Y_{11} - Y_{21})/F$ when they are shared and $(\mathrm{Var}\, Y_{11} + \mathrm{Var}\, Y_{21})/F$ when

they are separate. For the networks of Lab 15.1 (standard deviations 5.84 and 6.81 points, correlation 0.92), how many separate formats match one shared format?

15.2 CONSOLIDATION. (The tuned prompt.) Arm 1 is reported at the best of F settings, chosen on the test items, and arm 2 under one setting. Under the model of Theorem 15.3, show with Proposition 1.6 that the expected reported difference is $\Delta + \sqrt{\sigma_u^2 + \sigma_v^2 + \sigma_\varepsilon^2}\, e_F$, and evaluate it for $F = 10$ and a standard deviation of 5.8 points across settings. What does the expected difference become when both arms are reported at their best of the same F settings, and why is that still not the comparison of the means?

15.3 CONSOLIDATION. (A small leaderboard.) Three arms have means 0, 1 and 2 points, and $s_{ab} = 2$ points for every pair. Check that one setting gives $\mathbb{E}\hat{\tau} = 0.48$ and eight give 0.89. How many settings make the expected agreement between two leaderboards built on different settings at least 0.9?

15.4 CONSOLIDATION. ★ (A judge with a position bias.) Pairs of answers are oriented at random, and $R = 1$ when the first answer shown is the better, so that $\mathbb{P}(R = 1) = \frac{1}{2}$. The judge says "first" with probability $1 - q + b$ when $R = 1$ and $q + b$ when $R = 0$, with $0 \le b \le q < \frac{1}{2}$. (a) Show that its accuracy is $1 - q$ and its κ is $1 - 2q$, whatever b. (b) If system A is always shown first and is the better with probability w, show that the judge reports A's win rate as $q + b + w(1 - 2q)$. (c) Show that counterbalancing removes b, and that the remaining shrinkage toward $\frac{1}{2}$ is corrected by $(\hat{w} - q)/(1 - 2q)$. Which measurement in the report would have revealed b? *(Hint: with $\pi_R = \frac{1}{2}$, the denominator in part (i) of Proposition 15.6 is $\frac{1}{2}$ whatever π_J.)*

15.5 LAB. (Full shifts.) In `15-instruments-formats.py`, apply the one-pixel shift at full strength instead of blending 30% of it. Predict how $\hat{\sigma}_u$, s and the correlation between the MLP and the CNN change, and which network gains in the ranking; then compare the reversal rates for shared formats with the theorem.

15.6 LAB. (A pilot study.) Estimate the three components from 20 formats and 100 images of Lab 15.1 with Theorem 1.8, and choose a design for 2,000 evaluations by Proposition 15.5. Repeat the pilot 200 times: how often is the chosen design wrong, and what does the error cost?

15.7 LAB. (Training length.) In `15-instruments-icl.py`, train the networks with $\omega = 0.01$ for 3,000 steps. Do they move toward following the demonstrations or toward their weights, under flipped and under random labels? Relate the answer to the finding of Singh et al. (2023) that in-context learning can fade with longer training.

15.8 DESIGN. (Ten models, a thousand calls each.) Ten language models are to be ranked on a multiple-choice benchmark of 5,000 questions with

1,000 calls per model, and each model's spread over prompts reported. A pilot of two models gives $\sigma_s = 1.5$, $\sigma_i = 12$ and $\sigma_{si} = 30$ points. Design the evaluation: setting distribution, numbers of prompts and questions, crossed or nested, pairing, analysis and report.

15.9 DESIGN. (Validating a judge.) A model judge is to replace human raters for pairwise comparisons of chatbot answers, and you can afford 600 pairs labeled by three humans each. Design the study: what is measured, how pairs are sampled and oriented, how many judge calls are needed, and what result would make you keep the human raters.

15.10 CRITIQUE. (One template.) A leaderboard ranks 40 models on 1,000 reasoning questions, each scored with the template its developers recommend, and bolds differences of half a point. Which comparisons does it support, by Theorem 15.3? Describe the leaderboard a reader needs.

15.11 CRITIQUE. (A new prompting method.) A paper reports that its prompting method gains 4 points over the standard prompt on six tasks. Its method's template was chosen among 30 candidates on the test set; the baseline used the template of an earlier paper. Separate the parts of the gain due to the method, to the choice among templates and to the difference of templates, estimate the second (Exercise 15.2), and design an experiment for the first.

15.12 RESEARCH. (Choosing the setting distribution.) The estimand of Definition 15.2 depends on $\mathcal{S}$, and different grammars of formats, sets of paraphrases and policies for choosing demonstrations give different rankings. Open: a principled choice of $\mathcal{S}$, such as weighting settings by how often users write them or a worst case over a family of distributions, with guarantees on how conclusions transfer between distributions.

Notes and References

In-context learning was named and demonstrated at scale by Brown et al. (2020). Polo et al. (2024) estimate a model's distribution of scores over many prompts from a small budget, and Chan et al. (2022) and Singh et al. (2023) show in small transformers how the training data and the length of training decide between learning in context and in the weights.

The statistics are older than the field. Clark (1973) argued that psycholinguistic experiments must treat their words as a random sample, as they treat their participants; the answer became an analysis with crossed random effects for subjects and items (Baayen et al. 2008), and Yarkoni (2022) argued that stimuli, tasks and research sites deserve the same treatment wherever a claim is to generalize over them. Generalizability theory (Cronbach et al. 1972) decomposes a measurement's variance into facets, such as raters, items and occasions, and designs measurements from the components; Proposition 15.5 is a small part of it. Cohen (1960) introduced κ for two

raters on a nominal scale, Landis and Koch (1977) extended agreement measures to several categories and raters, and Feinstein and Cicchetti (1990) described its paradoxes; Spearman (1904) corrected correlations for the unreliability of their measurements. For evaluations of language models analyzed as experiments see Miller (2024), and for the science of benchmarks and their rankings Hardt (2026).

Translating the literature. Min et al. (2022b) write k for the number of demonstrations, a letter this book reserves for a tuning budget; their twelve models are six models crossed with two scoring rules, a setting of the instrument here; their "Kim et al. (2022)" is the preprint of Yoo et al. (2022), whose label-correctness sensitivity is the slope of the score on the share of correct labels and whose ground-truth label effect ratio is the cost of random labels as a fraction of the gain from gold-labeled demonstrations. In Wei et al. (2023), 50% flipped labels are random labels for a binary task. A "format" of Sclar et al. (2024) is a setting f, and their "performance spread", which their FormatSpread procedure estimates within a budget, is $\max_f Y_{af} - \min_f Y_{af}$ over the sampled formats; the "average performance" and "maximum performance" of Mizrahi et al. (2024) are μ_a under a uniform distribution over paraphrases and the best setting. The "agreement" of Zheng et al. (2023) is our p_o, and their "consistency" the share of verdicts unchanged by a swap of order. In generalizability theory a facet is a random factor, and $\sigma^2(s)$, $\sigma^2(i)$ and $\sigma^2(si)$ are our σ^2_{s}, σ^2_{i} and σ^2_{si}.

References

Baayen, R. H., D. J. Davidson, and D. M. Bates (2008). "Mixed-Effects Modeling with Crossed Random Effects for Subjects and Items". In: *Journal of Memory and Language* 59.4, pp. 390–412.

Ball, T., S. Chen, and C. Herley (2024). *Can We Count on LLMs? The Fixed-Effect Fallacy and Claims of GPT-4 Capabilities*. Preprint, arXiv:2409.07638.

Brown, T. et al. (2020). "Language Models are Few-Shot Learners". In: *Advances in Neural Information Processing Systems 33*, pp. 1877–1901.

Chan, S. C. Y., A. Santoro, A. K. Lampinen, J. X. Wang, A. Singh, P. H. Richemond, J. McClelland, and F. Hill (2022). "Data Distributional Properties Drive Emergent In-Context Learning in Transformers". In: *Advances in Neural Information Processing Systems 35*, pp. 18878–18891.

Clark, H. H. (1973). "The Language-as-Fixed-Effect Fallacy: A Critique of Language Statistics in Psychological Research". In: *Journal of Verbal Learning and Verbal Behavior* 12.4, pp. 335–359.

Cohen, J. (1960). "A Coefficient of Agreement for Nominal Scales". In: *Educational and Psychological Measurement* 20.1, pp. 37–46.

Cronbach, L. J., G. C. Gleser, H. Nanda, and N. Rajaratnam (1972). *The Dependability of Behavioral Measurements: Theory of Generalizability for Scores and Profiles*. New York: Wiley.

Feinstein, A. R. and D. V. Cicchetti (1990). "High Agreement but Low Kappa: I. The Problems of Two Paradoxes". In: *Journal of Clinical Epidemiology* 43.6, pp. 543–549.

Hardt, M. (2026). *The Emerging Science of Machine Learning Benchmarks*. Princeton: Princeton University Press.

Kossen, J., Y. Gal, and T. Rainforth (2024). "In-Context Learning Learns Label Relationships but Is Not Conventional Learning". In: *International Conference on Learning Representations (ICLR)*.

Landis, J. R. and G. G. Koch (1977). "The Measurement of Observer Agreement for Categorical Data". In: *Biometrics* 33.1, pp. 159–174.

Lu, Y., M. Bartolo, A. Moore, S. Riedel, and P. Stenetorp (2022). "Fantastically Ordered Prompts and Where to Find Them: Overcoming Few-Shot Prompt Order Sensitivity". In: *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 8086–8098.

Miller, E. (2024). *Adding Error Bars to Evals: A Statistical Approach to Language Model Evaluations*. Preprint, arXiv:2411.00640.

Min, S., M. Lewis, H. Hajishirzi, and L. Zettlemoyer (2022a). "Noisy Channel Language Model Prompting for Few-Shot Text Classification". In: *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 5316–5330.

Min, S., X. Lyu, A. Holtzman, M. Artetxe, M. Lewis, H. Hajishirzi, and L. Zettlemoyer (2022b). "Rethinking the Role of Demonstrations: What Makes In-Context Learning Work?" In: *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pp. 11048–11064.

Mizrahi, M., G. Kaplan, D. Malkin, R. Dror, D. Shahaf, and G. Stanovsky (2024). "State of What Art? A Call for Multi-Prompt LLM Evaluation". In: *Transactions of the Association for Computational Linguistics* 12, pp. 933–949.

Pan, J., T. Gao, H. Chen, and D. Chen (2023). "What In-Context Learning "Learns" In-Context: Disentangling Task Recognition and Task Learning". In: *Findings of the Association for Computational Linguistics: ACL 2023*, pp. 8298–8319.

Polo, F. M. et al. (2024). "Efficient Multi-Prompt Evaluation of LLMs". In: *Advances in Neural Information Processing Systems 37*, pp. 22483–22512.

Sclar, M., Y. Choi, Y. Tsvetkov, and A. Suhr (2024). "Quantifying Language Models' Sensitivity to Spurious Features in Prompt Design or: How I Learned to Start Worrying about Prompt Formatting". In: *International Conference on Learning Representations (ICLR)*.

Singh, A. K., S. C. Y. Chan, T. Moskovitz, E. Grant, A. M. Saxe, and F. Hill (2023). "The Transient Nature of Emergent In-Context Learning in Transformers". In: *Advances in Neural Information Processing Systems 36*, pp. 27801–27819.

Spearman, C. (1904). "The Proof and Measurement of Association between Two Things". In: *American Journal of Psychology* 15.1, pp. 72–101.

Wang, P. et al. (2024). "Large Language Models are not Fair Evaluators". In: *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 9440–9450. First circulated as arXiv:2305.17926 (2023).

Wei, J. et al. (2023). *Larger Language Models Do In-Context Learning Differently*. Preprint, arXiv:2303.03846.

Yarkoni, T. (2022). "The Generalizability Crisis". In: *Behavioral and Brain Sciences* 45, e1.

Yoo, K. M., J. Kim, H. J. Kim, H. Cho, H. Jo, S.-W. Lee, S.-g. Lee, and T. Kim (2022). "Ground-Truth Labels Matter: A Deeper Look into Input-Label Demonstrations". In: *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pp. 2422–2437.

Zhao, Z., E. Wallace, S. Feng, D. Klein, and S. Singh (2021). "Calibrate Before Use: Improving Few-shot Performance of Language Models". In: *Proceedings*

of the 38th International Conference on Machine Learning. Vol. 139. Proceedings of Machine Learning Research, pp. 12697–12706.

Zheng, L. et al. (2023). "Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena". In: *Advances in Neural Information Processing Systems 36 (Datasets and Benchmarks Track)*, pp. 46595–46623.

Epilogue

The laboratory notebook

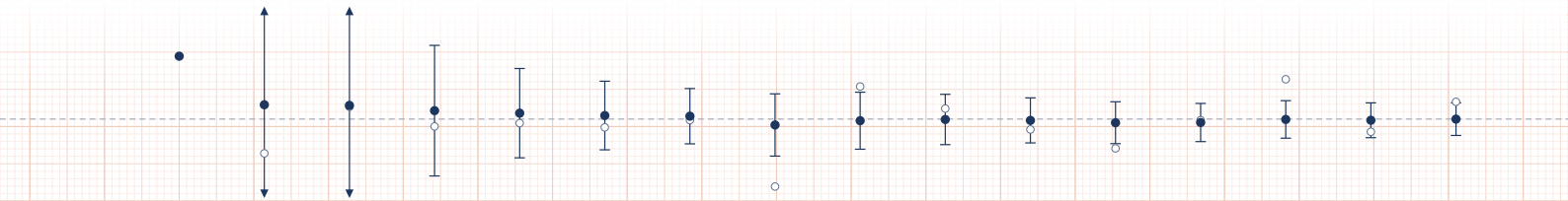
The fifteen chapters of this book examined other people's experiments. The epilogue runs one of its own, from the question to the report, and keeps its notebook in the open. The question is small on purpose: does label smoothing lower the test error of the small network that the laboratory sections have trained throughout? Label smoothing replaces the one-hot training target by a mixture of the one-hot vector and the uniform distribution over classes (Szegedy et al. 2016), and its effects on accuracy, calibration and distillation have been studied at scale (Müller et al. 2019). In Chapter 3 it came out ahead of a baseline on every task of a suite, but there the two arms shared one fixed configuration and nobody asked how much of the gain survived a fair protocol. Every step below follows a protocol of an earlier chapter; the script that ran the whole study is `16-epilogue-study.py`, and it prints every number quoted here.

The question and the predictions

A question becomes an experiment when its unit, treatment and response are fixed. The unit is a run of the two-layer network with 64 units per layer, trained for 30 epochs by stochastic gradient descent with momentum on the scikit-learn digits; the treatment is the smoothing parameter, 0 or 0.1; the response is the error on the 539 test images, measured once per run at the end of training. Before anything was run, four predictions were written into the header of the script, where the version history of the repository dates them (Table E.1, middle column). They are not forecasts to be proud of. They record what the experimenter expected, so that the experiment can show how far the expectation was from the result.

EXPERIMENTAL DESIGN · THE EPILOGUE'S STUDY

QUESTION	Does label smoothing (0.1) lower the test error of the digits network?
TREATMENT	smoothing parameter 0 (baseline) or 0.1
TUNED	learning rate and weight decay, on the same 10-point grid for both arms, 3 seeds per point, scored on a validation split carved from the training set
HELD FIXED	architecture, optimizer, epochs, batch size, data split, test set
VARIED	initialization and data-order seeds, paired across the arms
RESPONSE	test error, read once per run
REPLICATES	chosen by a power calculation on a pilot
CONTROLS	a positive control: the tuned baseline on half of the training set



ANALYSIS	paired t-interval, sign-flip permutation test and bootstrap for the difference; a 2×2 check of the interaction with weight decay

A pilot, and the number of seeds

Chapter 1 asks for the number of runs to be fixed before the comparison is made, and Proposition A.14 turns a pilot into that number. Ten paired seeds per arm at a learning rate of 0.05, scored on a validation split of 300 images taken from the training set, gave validation errors of 3.97 and 2.80 percent and a standard deviation of the paired difference of 0.61 points. To detect a difference of 0.25 points with power 0.8 at level 0.05, the formula asks for $(z_{0.975} + z_{0.8})^2\, 0.61^2/0.25^2 \approx 48$ paired seeds. The pilot's own difference was large, but a pilot is for estimating the spread, not the effect; the effect it shows is a single noisy draw and was not used.

Tuning with equal budgets

Chapter 4 showed how a comparison turns into a comparison of tuning budgets when one arm is searched and the other is not, and Chapter 8 how hyperparameters tuned for one arm favor it. Both arms were therefore searched over the same ten configurations, learning rates 0.01, 0.02, 0.05, 0.1 and 0.2 with weight decay 0 or 10^{-4}, three seeds each, on the validation split only. The searches chose different configurations: a learning rate of 0.05 with weight decay 10^{-4} for the baseline, and 0.1 without weight decay for label smoothing. Had label smoothing been run at the baseline's tuned settings, the comparison would have given the baseline the home-field advantage of Chapter 8; the 2×2 check below, which runs at the baseline's learning rate, shows that the effect would have looked about a sixth smaller.

A positive control

An experiment that cannot detect a known effect cannot be trusted to report the absence of an unknown one (Chapter 5). The positive control trains the tuned baseline on half of the training set. Its test error rose from 2.87 to 4.27 percent, a paired difference of 1.40 ± 0.16 points: the pipeline sees an effect of this size easily. The baseline arm itself is the negative control: it runs the same code with the smoothing parameter set to zero, so that the treatment is the only difference between the arms.

The result

Forty-eight paired seeds per arm, each arm at its own tuned configuration, scored once on the test set, gave test errors of 2.87 ± 0.12 percent for the baseline and 1.91 ± 0.10 percent with label smoothing (means with 95 percent t-intervals). The paired difference is -0.96 points with 95 percent interval $[-1.10, -0.82]$; a bootstrap over seeds gives $[-1.09, -0.83]$, and a sign-flip permutation test of the paired differences gives $p < 10^{-4}$ (no

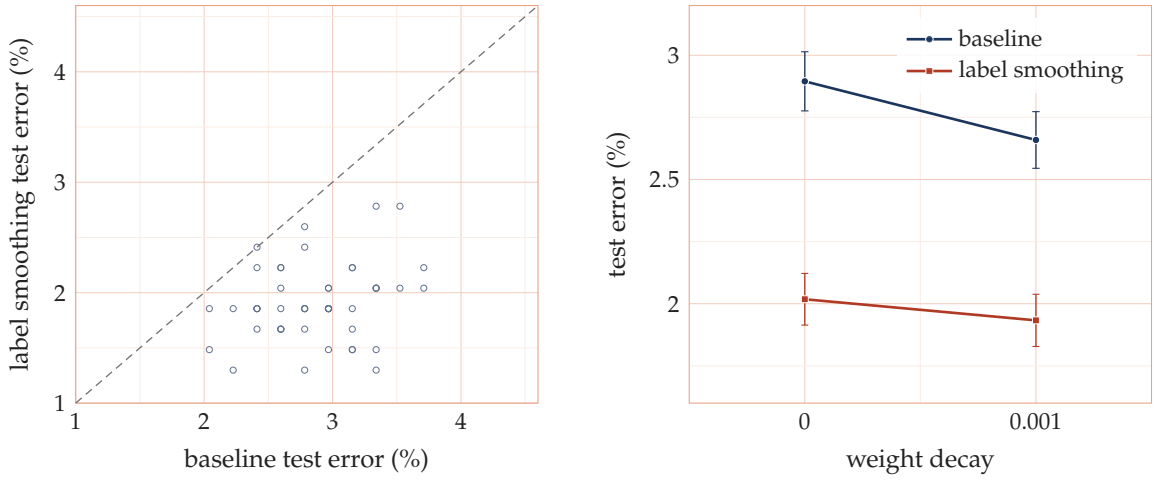


Figure E.1. The epilogue's study. (a) Each mark is one of 48 seeds, trained in both arms at each arm's tuned configuration; points below the diagonal are seeds on which label smoothing had the lower test error. (b) The 2×2 check at the baseline's tuned learning rate (0.05): mean test error over the same 48 seeds with 95 percent t-intervals, for weight decay 0 or 10^{-3}. Script `16-epilogue-study.py`.

permutation out of 20,000 reached the observed mean). Label smoothing was better on 47 of the 48 seeds and tied on the remaining one (Figure E.1a).

Pairing helped less than it often does. The outcomes of the two arms on the same seed correlate only weakly (0.23), because the arms were trained at different learning rates and so follow different trajectories from the same initialization; the paired interval is 0.14 points wide on each side, against 0.16 unpaired. Pairing costs nothing, and here it buys a little.

An interaction check

Chapters 6 and 8 warn that an effect measured at one setting of the other factors may not be the effect at another. A 2×2 design at the baseline's tuned learning rate crossed the smoothing parameter with weight decay 0 or 10^{-3}, again with 48 paired seeds per cell (Figure E.1b). The main effect of label smoothing is -0.80 ± 0.11 points, the main effect of weight decay -0.16 ± 0.04, and the interaction, the difference between the effect of smoothing with and without weight decay, $+0.15 \pm 0.09$: smoothing helps a little less when weight decay is on, and helps in both cells. Two factors at two levels are not a characterization of the method, but they rule out the most common way for a single comparison to mislead, an effect that exists only at the settings where it was measured.

Predictions against outcomes

Table E.1 sets the predictions beside what happened. Two of the four were right. The other two were wrong in the same direction: the effect was about twice the upper end of the predicted range, and the spread of the paired differences was larger than expected, so that 48 seeds were needed where

quantity	predicted before the runs	observed
effect of label smoothing on test error	between −0.5 and −0.1 points	−0.96 [−1.10, −0.82]: larger
half-width of its 95% interval	about 0.15 points with about 20 seeds	0.14 with 48 seeds, 0.22 projected for 20
positive control (half the data)	worse by at least 1 point	worse by 1.40 ± 0.16
interaction with weight decay	smaller than 0.2 points	+0.15 ± 0.09

Table E.1. The predictions written into the study's script before the first run, and the outcomes.

about 20 had been guessed. A prediction that misses is not a failure of the experiment; it is the experiment's report on the experimenter. Had the predictions not been written down, the large effect would have been easy to remember as expected.

The report

A report states the claim with its uncertainty, the protocol that produced it, and what it does not show. For this study:

> Report. On the scikit-learn digits, label smoothing with parameter 0.1 lowered the test error of a two-layer network with 64 units per layer from 2.87 to 1.91 percent: a paired difference of −0.96 points, 95 percent interval [−1.10, −0.82], over 48 paired seeds (better on 47, tied on one). Each arm was tuned over the same grid of ten configurations with three seeds each on a validation split; the test set was read once per run. The number of seeds was fixed beforehand by a power calculation on a pilot. A positive control (half of the training set) raised the error by 1.40 points. At the baseline's learning rate the effect of smoothing changed by 0.15 ± 0.09 points when weight decay was added. The study does not show that the effect extends to other data, architectures or training lengths, and it does not identify how smoothing helps; predictions written before the runs underestimated the effect by about half.

The last sentence of the report is the one that most reports leave out. Its first half is Chapter 13 in one line; its second is Chapter 9.

Fifteen protocols

Each chapter ended with a protocol. Table E.2 lists them with the step that is most often skipped; the study above used five of them, those of Chapters 1, 4, 5, 8 and 13, and the predictions in its script are the first step of the last.

What this book did not cover

The book is about the training experiment, and three neighboring subjects were left to others. The science of benchmarks, of how a shared test set comes to rank models and what those rankings mean, is the subject of Hardt (2026). Causal inference from observational data, where the experimenter cannot assign the treatment, is a field with its own textbooks (Imbens and Rubin 2015; Pearl 2009); Chapter 10 used only its first lesson. Human evaluation, where the instrument is a person, has methods of its own that Chapter 15 touched only through model judges. The documentation of data and models (Gebru et al. 2021; Mitchell et al. 2019) is part of reporting but was not treated here.

The culture of experiments

Every method in this book costs runs, and the incentives of a fast field reward the result that appears first more than the one that was checked (Sculley et al. 2018). Some of the remedies are institutional. Preregistration, the public registration of a study's design and analysis before its data are seen, is standard for clinical trials and spread through psychology after its replication crisis (Nosek et al. 2018); the predictions in the header of a script are its private version. Reproducibility programs and checklists at machine learning conferences ask authors to report seeds, intervals and compute (Pineau et al. 2021). Other remedies are habits. Report negative results and failed replications as findings; a miniature that does not reproduce a famous effect, like several in this book, is information. Review a failed claim as an incident, without blame, as the prologue proposed. Keep the notebook.

The ornament at the foot of this page carries the sixteenth draw. The interval that ran off the page after two draws is now narrow, and the running mean, after a first draw 2.8 standard deviations too high, sits 0.001 below the line. Sixteen is still a small number of runs; but the estimate has an honest interval, which is more than one run could give, and that is the whole difference between a number and a measurement.

Notes and References

Label smoothing was introduced by Szegedy et al. (2016) as a regularizer for the Inception networks; Müller et al. (2019) found that it improves calibration and accuracy in many settings and hurts knowledge distillation, and explained the latter by its effect on the geometry of the penultimate layer. The study in this epilogue says nothing about those settings: it is a worked example of a protocol, not a contribution to the literature on label smoothing.

Nosek et al. (2018) argue the case for preregistration across the sciences and describe its forms. Pineau et al. (2021) report on the reproducibility program of NeurIPS 2019, including its checklist. Datasheets for datasets (Gebru et al. 2021) and model cards (Mitchell et al. 2019) are the standard proposals for documenting data and trained models.

References

Gebru, T., J. Morgenstern, B. Vecchione, J. W. Vaughan, H. Wallach, H. Daumé III, and K. Crawford (2021). "Datasheets for Datasets". In: *Communications of the ACM* 64.12, pp. 86–92.

Hardt, M. (2026). *The Emerging Science of Machine Learning Benchmarks*. Princeton, NJ: Princeton University Press.

Imbens, G. W. and D. B. Rubin (2015). *Causal Inference for Statistics, Social, and Biomedical Sciences: An Introduction*. Cambridge: Cambridge University Press.

Mitchell, M. et al. (2019). "Model Cards for Model Reporting". In: *Proceedings of the Conference on Fairness, Accountability, and Transparency*, pp. 220–229.

Müller, R., S. Kornblith, and G. E. Hinton (2019). "When Does Label Smoothing Help?" In: *Advances in Neural Information Processing Systems*. Vol. 32.

Nosek, B. A., C. R. Ebersole, A. C. DeHaven, and D. T. Mellor (2018). "The Preregistration Revolution". In: *Proceedings of the National Academy of Sciences* 115.11, pp. 2600–2606.

Pearl, J. (2009). *Causality: Models, Reasoning, and Inference*. 2nd ed. Cambridge: Cambridge University Press.

Pineau, J., P. Vincent-Lamarre, K. Sinha, V. Larivière, A. Beygelzimer, F. d'Alché-Buc, E. Fox, and H. Larochelle (2021). "Improving Reproducibility in Machine Learning Research (A Report from the NeurIPS 2019 Reproducibility Program)". In: *Journal of Machine Learning Research* 22.164, pp. 1–20.

Sculley, D., J. Snoek, A. Wiltschko, and A. Rahimi (2018). "Winner's Curse? On Pace, Progress, and Empirical Rigor". In: *International Conference on Learning Representations, Workshop Track*.

Szegedy, C., V. Vanhoucke, S. Ioffe, J. Shlens, and Z. Wojna (2016). "Rethinking the Inception Architecture for Computer Vision". In: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pp. 2818–2826.

protocol	the step most often skipped
Protocol 1.1, Measuring one recipe	Fix the number of runs before looking, and report every run.
Protocol 2.1, Repeatable runs, reproducible results	Record every seed and version; a fixed seed buys an audit trail, not evidence.
Protocol 3.1, Reporting results over a suite of tasks	Choose one aggregate and its interval in advance, and rerun every baseline under the same protocol.
Protocol 4.1, Comparing two training methods	Give the arms equal budgets over every hyperparameter that distinguishes them.
Protocol 5.1, Controls for a search method	Run the placebo: random draws from the same space, through the same pipeline.
Protocol 6.1, Attributing an improvement	List every difference from the baseline, and cross the architecture with the recipe.
Protocol 7.1, Choosing what to hold fixed	State the budget of the claim, and retune what moves with the treatment.
Protocol 8.1, Designing an ablation	Name the estimand, use a factorial design when components interact, and retune the ablated variant.
Protocol 9.1, Testing a mechanism claim	Define the mediator by a measurement, then knock it out and rescue it.
Protocol 10.1, Reading a correlation in a sweep	Give every configuration its own seeds, fix the measure first, and check it under rescaling.
Protocol 11.1, Fitting and reporting a scaling law	Fix the objective in advance, start from a grid, and report convergence and bootstrap intervals.
Protocol 12.1, A protocol card for a scaling study	Count every operation, and train each configuration as it would be trained at its budget.
Protocol 13.1, Before you extrapolate	Write the forecast and its interval down before the run finishes, and backtest the form.
Protocol 14.1, Before announcing an emergent ability	Plot a continuous metric beside the headline one, with intervals and zero counts as bounds.
Protocol 15.1, Evaluating under random settings	Sample the settings, score every arm under the same ones, and compare within settings.

Table E.2. The protocols of the fifteen chapters.

A Statistical Toolkit

A

The statistics the experiments share

IN THIS CHAPTER. A handful of statistical tools recur across the chapters of this book: intervals for a mean and for a difference of means; the bootstrap and the permutation test; paired comparisons of two classifiers on the same test items; corrections for many comparisons; the distribution of the best of several draws; the delta method; nonlinear least squares; variance components and factorial designs; sample sizes; rank statistics; and intervals for proportions near zero. This appendix states what the chapters use and proves what can be proved in a page; for the rest it gives a sketch and a reference.

The appendix is a reference, not a chapter to be read in order. Intervals and tests for means (Section A.1) are used everywhere; resampling (Section A.2) in Chapters 1, 3, 4 and 11; multiple comparisons (Section A.3) in Chapter 3; order statistics (Section A.4) in Chapters 1, 4 and 5; the delta method and nonlinear least squares (Sections A.5 and A.6) in Chapters 11 to 13; variance components and factorial designs (Section A.7) in Chapters 1, 6, 8 and 15; sample sizes (Section A.8) in Chapter 1; rank statistics (Section A.9) in Chapters 3, 5 and 10; and binomial intervals (Section A.10) in Chapter 14.

Throughout, $Y_1, \dots, Y_n$ are outcomes of runs, $\bar{Y}$ is their mean and $S^2 = \frac{1}{n-1}\sum_i (Y_i - \bar{Y})^2$ their sample variance. A p-value for a true null hypothesis is *valid* if $\mathbb{P}(p \le u) \le u$ for every $u \in [0,1]$.

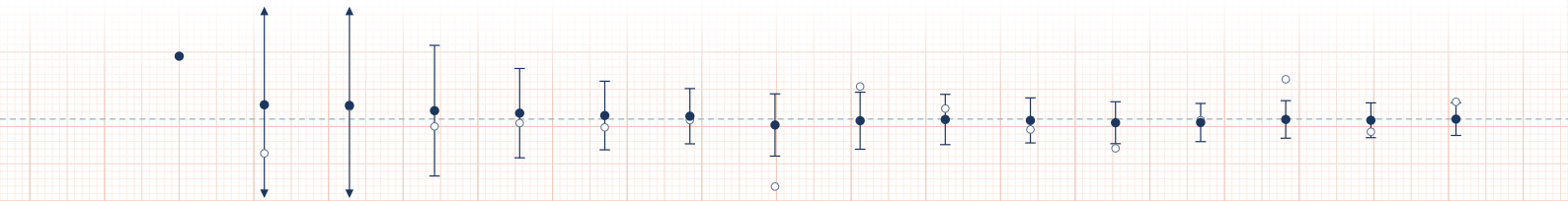
A.1 Intervals for means

If $Y_1, \dots, Y_n$ are independent with mean μ and variance σ^2, then $\mathbb{E}\bar{Y} = \mu$, $\operatorname{Var}\bar{Y} = \sigma^2/n$ and $\mathbb{E}S^2 = \sigma^2$, so that $S/\sqrt{n}$, the *standard error*, estimates the standard deviation of $\bar{Y}$.

Theorem A.1 (t-intervals and Welch's test). *(i) If the Y_i are normal, then $(\bar{Y} - \mu)/(S/\sqrt{n})$ has Student's t distribution with $n-1$ degrees of freedom, and the interval $\bar{Y} \pm t_{n-1,1-\alpha/2}\, S/\sqrt{n}$ contains μ with probability exactly $1-\alpha$.*

(ii) Without normality, if $\mathbb{E}Y_1^2 < \infty$, the coverage of the same interval tends to $1-\alpha$ as $n \to \infty$.

(iii) For two independent samples of sizes n_1, n_2, means μ_1, μ_2 and variances



σ_1^2, σ_2^2, *Welch's statistic*

$$W = \frac{\bar{Y}_1 - \bar{Y}_2 - (\mu_1 - \mu_2)}{\sqrt{S_1^2/n_1 + S_2^2/n_2}}$$

is asymptotically standard normal as $n_1, n_2 \to \infty$, *and for normal samples it is approximately* t*-distributed with* $\nu = \left(S_1^2/n_1 + S_2^2/n_2\right)^2 \big/ \left[(S_1^2/n_1)^2/(n_1-1) + (S_2^2/n_2)^2/(n_2-1)\right]$ *degrees of freedom.*

Proof sketch. Part (i) rests on two facts about normal samples: $\bar{Y}$ and S^2 are independent, and $(n-1)S^2/\sigma^2$ has the χ^2 distribution with $n-1$ degrees of freedom (Casella and Berger 2002, Section 5.3). A standard normal divided by the square root of an independent χ^2_ν/ν has, by definition, Student's distribution with ν degrees of freedom (Student 1908). Part (ii) follows from the central limit theorem for $\sqrt{n}(\bar{Y} - \mu)/\sigma$, the convergence of S to σ in probability, Slutsky's lemma, and $t_{n-1,1-\alpha/2} \to z_{1-\alpha/2}$. Part (iii) is proved the same way for the asymptotic statement; the degrees of freedom come from matching the first two moments of $S_1^2/n_1 + S_2^2/n_2$ to those of a scaled χ^2 variable (Satterthwaite 1946; Welch 1947), which makes the t distribution an approximation rather than an exact law. ■

When the runs of two arms come in pairs that share a seed, a data order or a set of test items, the comparison should use the differences $D_i = Y_{1i} - Y_{2i}$ and the one-sample interval of part (i) for $\mathbb{E}D$. Since $\operatorname{Var} D = \sigma_1^2 + \sigma_2^2 - 2\operatorname{Cov}(Y_1, Y_2)$, pairing narrows the interval whenever the paired outcomes are positively correlated, which is the usual case.

A.2 Resampling

Definition A.2 (Bootstrap intervals). Let $\hat{\theta} = \hat{\theta}(Y_1, \ldots, Y_n)$ be a statistic. A *bootstrap sample* draws n values uniformly with replacement from $Y_1, \ldots, Y_n$; computing $\hat{\theta}$ on B independent bootstrap samples gives $\hat{\theta}_1^*, \ldots, \hat{\theta}_B^*$. The *percentile interval* at level $1-\alpha$ runs from the $\alpha/2$-quantile to the $(1-\alpha/2)$-quantile of the $\hat{\theta}_b^*$. When the data fall into strata, for instance tasks with several runs each, the *stratified bootstrap* resamples with replacement within each stratum and keeps the strata fixed.

Theorem A.3 (Consistency of the bootstrap). *Let* $Y_1, \ldots, Y_n$ *be independent and identically distributed with finite variance, and let* $\hat{\theta} = g(\bar{Y})$ *for a function* g *continuously differentiable at* μ *with* $g'(\mu) \neq 0$. *Then, conditionally on the data and for almost every sequence of data,* $\sqrt{n}(\hat{\theta}^* - \hat{\theta})$ *has the same normal limit as* $\sqrt{n}(\hat{\theta} - g(\mu))$, *and the percentile interval has asymptotic coverage* $1-\alpha$.

Proof sketch. Bickel and Freedman (1981) prove the conditional central limit theorem for the bootstrap mean; the delta method (Theorem A.10) carries it to $g(\bar{Y})$; the symmetry of the normal limit then gives the coverage of the percentile interval (Vaart 1998, Chapter 23). With few runs the percentile interval tends to be too narrow; the bias-corrected and accelerated interval of Efron (1987) corrects for bias and skewness, and Efron and Tibshirani (1993) is the standard introduction. ∎

A permutation test needs no asymptotics. Its null hypothesis is a symmetry: for two arms of runs, that the arm labels are exchangeable, as they are when the two arms have the same run distribution and the runs are independent.

Theorem A.4 (Permutation tests are exact). *Let G be a finite group acting on the data, and suppose that under the null hypothesis gY has the same distribution as Y for every $g \in G$. For a statistic T put $p(Y) = |G|^{-1}\,\#\{h \in G : T(hY) \geq T(Y)\}$. Then, under the null hypothesis, $\mathbb{P}(p(Y) \leq u) \leq u$ for every $u \in [0,1]$.*

Proof. The plan is to show that in every orbit of the group at most a fraction u of the points have a p-value at most u, and then to use the symmetry to turn this count into a probability.

Fix a data set y and let $A = \{g \in G : p(gy) \leq u\}$. Since $h \mapsto hg$ is a bijection of G, the p-value of gy is computed from the same values as that of y:

$$p(gy) = |G|^{-1}\,\#\{h \in G : T(hgy) \geq T(gy)\} = |G|^{-1}\,\#\{h \in G : T(hy) \geq T(gy)\}.$$

If A is empty there is nothing to prove. Otherwise choose $g^\star \in A$ for which $T(g^\star y)$ is smallest among the elements of A. Every $g \in A$ satisfies $T(gy) \geq T(g^\star y)$, so A is contained in $\{h : T(hy) \geq T(g^\star y)\}$, a set with $|G|\,p(g^\star y) \leq u|G|$ elements. Hence $|A| \leq u|G|$ for every y.

Now let the data be random. By the null hypothesis $p(gY)$ has the distribution of $p(Y)$ for each g, so

$$\mathbb{P}(p(Y) \leq u) = \frac{1}{|G|}\sum_{g \in G} \mathbb{P}(p(gY) \leq u) = \mathbb{E}\Big[\frac{\#\{g : p(gY) \leq u\}}{|G|}\Big] \leq u. \qquad \blacksquare$$

For two arms of n runs, G is the group of permutations of the $2n$ pooled outcomes, acting by reassigning arm labels, and T is for instance the difference of the arm means. When $|G|$ is too large to enumerate, M random permutations give the valid p-value $(1 + \#\{b : T_b \geq T_{\mathrm{obs}}\})/(M+1)$; dropping the two ones, as is often done, can produce a p-value of zero and is not valid (Phipson and Smyth 2010).

Two classifiers evaluated on the same test items are a paired design. Its analysis goes back to McNemar (1947); Dietterich (1998) recommended McNemar's test for comparing two learning algorithms when each can be trained only once.

Proposition A.5 (Two classifiers on the same items). *Two classifiers are evaluated on m test items drawn independently. Let b count the items that the first classifies correctly and the second incorrectly, and c the items with the reverse, and let π_b, π_c be the probabilities of these two discordances.*

(i) If $\pi_b = \pi_c$, then conditionally on $b + c = s$ the count b has the binomial distribution $\mathrm{Bin}(s, 1/2)$.

(ii) The difference of the two accuracies is $(b - c)/m$, with variance $\big(\pi_b + \pi_c - (\pi_b - \pi_c)^2\big)/m$.

Proof. Each item falls, independently of the others, into one of four categories: both correct, only the first correct, only the second correct, both wrong. Conditionally on which items are discordant, each discordant item is of the first kind with probability $\pi_b/(\pi_b + \pi_c)$, independently of the others; under $\pi_b = \pi_c$ this probability is $1/2$, which proves (i). For (ii), write the difference of accuracies as $\frac{1}{m}\sum_i X_i$ with $X_i = 1$ for a discordance of the first kind, -1 for one of the second kind and 0 otherwise. Then $\mathbb{E}X_i = \pi_b - \pi_c$ and $\mathbb{E}X_i^2 = \pi_b + \pi_c$, so $\mathrm{Var}\, X_i = \pi_b + \pi_c - (\pi_b - \pi_c)^2$, and the mean of m independent copies has $1/m$ of this variance. ∎

Two classifiers of similar accuracy usually agree on most items, so $\pi_b + \pi_c$ is small and the paired variance in (ii) is far below the unpaired $\big(p_1(1 - p_1) + p_2(1 - p_2)\big)/m$.

A.3 Many comparisons

Let $H_1, \dots, H_m$ be null hypotheses with valid p-values $p_1, \dots, p_m$, let I_0 be the set of true nulls and $m_0 = |I_0|$. A procedure rejects some of the hypotheses; let R be the number of rejections and V the number of true nulls among them. The *family-wise error rate* is $\mathbb{P}(V \geq 1)$, and the *false discovery rate* is $\mathbb{E}[V/\max(R, 1)]$.

Proposition A.6 (Bonferroni). *Rejecting H_i when $p_i \leq \alpha/m$ gives a family-wise error rate at most $m_0\alpha/m \leq \alpha$, whatever the dependence between the p-values.*

Proof. By the union bound and validity, $\mathbb{P}(V \geq 1) \leq \sum_{i \in I_0} \mathbb{P}(p_i \leq \alpha/m) \leq m_0\alpha/m$. ∎

Theorem A.7 (Holm's procedure). *Order the p-values as $p_{(1)} \leq \cdots \leq p_{(m)}$. Let j be the smallest index with $p_{(j)} > \alpha/(m - j + 1)$, reject the hypotheses with the $j - 1$ smallest p-values, and reject all of them if there is no such index. Then the family-wise error rate is at most α, whatever the dependence between the p-values.*

Proof. If $m_0 = 0$ there is no true null to reject. Otherwise consider the event E that every true null has $p_i > \alpha/m_0$; by the union bound its probability is at least $1 - m_0 \cdot \alpha/m_0 = 1 - \alpha$. On E, let j_0 be the first position in the

ordering that holds a true null. The positions before it hold false nulls only, so $j_0 \le m - m_0 + 1$, that is $m - j_0 + 1 \ge m_0$, and therefore

$$p_{(j_0)} > \alpha/m_0 \ge \alpha/(m - j_0 + 1).$$

The procedure stops at or before position j_0 and rejects no true null. The family-wise error rate is at most $\mathbb{P}(E^c) \le \alpha$. ∎

Theorem A.8 (Benjamini and Hochberg). *Let $R = \max\{k \ge 1 : p_{(k)} \le \alpha k/m\}$, or $R = 0$ if no such k exists, and reject the R hypotheses with the smallest p-values. If each true-null p-value is valid and independent of all the other p-values, then the false discovery rate is at most $m_0\alpha/m \le \alpha$.*

Proof. The plan is to write the false discovery rate as a sum over true nulls, and to decouple each true null's p-value from the number of rejections by setting it to zero.

Write $N(k) = \#\{j : p_j \le \alpha k/m\}$. Since $p_{(k)} \le \alpha k/m$ exactly when $N(k) \ge k$, we have $R = \max\{k : N(k) \ge k\}$. A hypothesis H_i is rejected exactly when $p_i \le \alpha R/m$: the rejected p-values are at most $p_{(R)} \le \alpha R/m$, and a p-value $p_i \le \alpha R/m$ outside the R smallest would give $p_{(R+1)} \le \alpha(R+1)/m$, against the maximality of R. Hence

$$\mathbb{E}\Big[\frac{V}{\max(R,1)}\Big] = \sum_{i \in I_0} \sum_{k=1}^{m} \frac{1}{k}\, \mathbb{P}\big(p_i \le \alpha k/m,\ R = k\big). \tag{A.1}$$

Fix $i \in I_0$ and let $R^{(i)}$ and $N^{(i)}$ be computed with p_i replaced by 0; then $N^{(i)}(k) = N(k) + \mathbf{1}\{p_i > \alpha k/m\}$, and $R^{(i)} \ge 1$ always. We claim that on the event $\{p_i \le \alpha k/m\}$ the equalities $R = k$ and $R^{(i)} = k$ are equivalent. On that event $N^{(i)}(k') = N(k')$ for every $k' \ge k$. If $R = k$, then $N^{(i)}(k) = N(k) \ge k$, and for $k' > k$, $N^{(i)}(k') = N(k') < k'$, so $R^{(i)} = k$. If $R^{(i)} = k$, then $N(k) = N^{(i)}(k) \ge k$, and for $k' > k$, $N(k') \le N^{(i)}(k') < k'$, so $R = k$.

The variable $R^{(i)}$ depends only on the p-values other than p_i, which are independent of p_i. By the claim and validity,

$$\mathbb{P}\big(p_i \le \alpha k/m,\ R = k\big) = \mathbb{P}\big(p_i \le \alpha k/m\big)\, \mathbb{P}\big(R^{(i)} = k\big) \le \frac{\alpha k}{m}\, \mathbb{P}\big(R^{(i)} = k\big).$$

Substituting into (A.1), the inner sum is at most $\frac{\alpha}{m} \sum_k \mathbb{P}(R^{(i)} = k) = \alpha/m$, and the outer sum over the m_0 true nulls gives $m_0\alpha/m$. ∎

Under arbitrary dependence the same procedure run at level $\alpha / \sum_{k=1}^{m} k^{-1}$ controls the false discovery rate at α (Benjamini and Yekutieli 2001). The original papers are Holm (1979) and Benjamini and Hochberg (1995).

A.4 The best of several draws

Proposition A.9 (Order statistics). *Let $Y_1, \dots, Y_k$ be independent with distribution function F and $\mathbb{E}|Y_1| < \infty$, and let $M_k = \max_i Y_i$.*

(i) $\mathbb{P}(M_k \le x) = F(x)^k$; when F is continuous, the q-quantile of M_k is

the $q^{1/k}$-quantile of F.

(ii) $\mathbb{E}M_k = \int_0^\infty \bigl(1 - F(x)^k\bigr)\,\mathrm{d}x - \int_{-\infty}^0 F(x)^k\,\mathrm{d}x$.

(iii) For standard normal draws, $e_k = \mathbb{E}M_k$ satisfies $e_k \le \sqrt{2\ln k}$ for every $k \ge 1$.

Proof. Part (i) is independence: $\{M_k \le x\}$ is the intersection of the k events $\{Y_i \le x\}$. If F is continuous, then $F(x)^k = q$ exactly when $F(x) = q^{1/k}$. Part (ii) is the identity $\mathbb{E}Z = \int_0^\infty \mathbb{P}(Z > x)\,\mathrm{d}x - \int_{-\infty}^0 \mathbb{P}(Z \le x)\,\mathrm{d}x$, valid for every integrable Z, applied to $Z = M_k$, which is integrable because $|M_k| \le \sum_i |Y_i|$. For (iii) the case $k = 1$ is $e_1 = 0$. For $k \ge 2$ and $t > 0$, Jensen's inequality and the bound of a maximum by a sum give

$$e^{te_k} \le \mathbb{E}e^{tM_k} \le \sum_{i=1}^{k} \mathbb{E}e^{tY_i} = k\,e^{t^2/2},$$

so $e_k \le (\ln k)/t + t/2$; the choice $t = \sqrt{2\ln k}$ gives $e_k \le \sqrt{2\ln k}$. ∎

The bound is tight to leading order: $e_k/\sqrt{2\ln k} \to 1$, and after centering by b_k and scaling by $a_k = 1/\sqrt{2\ln k}$ the maximum of k standard normals converges to the Gumbel distribution $\exp(-e^{-x})$, with $b_k = \sqrt{2\ln k} - (\ln\ln k + \ln 4\pi)/(2\sqrt{2\ln k})$ (Leadbetter et al. 1983, Theorem 1.5.3). The convergence is slow, and for the small k of practice the exact values of e_k should be computed from (ii) by numerical integration; David and Nagaraja (2003) treat order statistics in general.

A.5 The delta method

Theorem A.10 (Delta method). *Let $\hat\theta_n$ be estimators of $\theta \in \mathbb{R}^p$ with $\sqrt{n}(\hat\theta_n - \theta)$ converging in distribution to $\mathcal{N}(0, \Sigma)$, and let $g : \mathbb{R}^p \to \mathbb{R}$ be differentiable at θ. Then $\sqrt{n}\bigl(g(\hat\theta_n) - g(\theta)\bigr)$ converges in distribution to $\mathcal{N}\bigl(0, \nabla g(\theta)^{\top} \Sigma\, \nabla g(\theta)\bigr)$.*

Proof. The plan is a first-order expansion whose remainder the factor $\sqrt{n}$ cannot rescue. Differentiability at θ means that $g(\theta + h) = g(\theta) + \nabla g(\theta)^{\top} h + \|h\|\,\rho(h)$ for a function ρ with $\rho(0) = 0$ that is continuous at 0. With $h_n = \hat\theta_n - \theta$,

$$\sqrt{n}\bigl(g(\hat\theta_n) - g(\theta)\bigr) = \nabla g(\theta)^{\top} \sqrt{n}\, h_n + \|\sqrt{n}\, h_n\|\,\rho(h_n).$$

The first term converges in distribution to $\mathcal{N}(0, \nabla g^{\top}\Sigma \nabla g)$ by the continuous mapping theorem. In the second, $\|\sqrt{n}h_n\|$ is bounded in probability because $\sqrt{n}h_n$ converges in distribution, and $\rho(h_n) \to 0$ in probability because $h_n = (\sqrt{n}h_n)/\sqrt{n} \to 0$ in probability and ρ is continuous at 0. Their product tends to 0 in probability, and Slutsky's lemma completes the proof. ∎

For example, the compute-optimal allocation exponent of Chapter 11 is $a = \alpha_D/(\alpha_N + \alpha_D)$, with gradient $(-\alpha_D, \alpha_N)/(\alpha_N + \alpha_D)^2$ in (α_N, α_D). If the fitted exponents have covariance Σ, the delta method gives

$$\operatorname{Var} \hat{a} \approx \frac{\alpha_D^2 \Sigma_{NN} - 2\alpha_N \alpha_D \Sigma_{ND} + \alpha_N^2 \Sigma_{DD}}{(\alpha_N + \alpha_D)^4},$$

so a positive correlation between the two fitted exponents makes $\hat{a}$ more precise than either exponent alone.

A.6 Nonlinear least squares

In a nonlinear regression $y_i = f(x_i; \beta) + \varepsilon_i$, $i = 1, \dots, n$, with parameters $\beta \in \mathbb{R}^p$ and independent errors of mean zero and variance σ^2, the least-squares estimate $\hat{\beta}$ minimizes $\sum_i (y_i - f(x_i;\beta))^2$. Let J be the $n \times p$ Jacobian with entries $\partial f(x_i;\beta)/\partial \beta_j$.

Theorem A.11 (Asymptotics of nonlinear least squares). *Suppose that β is identifiable, that f is twice continuously differentiable in β with suitably bounded derivatives, and that $J^\top J/n$ converges to a positive definite matrix V. Then $\hat{\beta}$ is consistent and $\sqrt{n}(\hat{\beta} - \beta)$ converges in distribution to $\mathcal{N}(0, \sigma^2 V^{-1})$. In practice the covariance of $\hat{\beta}$ is estimated by $\hat{\sigma}^2 (\hat{J}^\top \hat{J})^{-1}$, with $\hat{\sigma}^2$ the residual sum of squares divided by $n - p$ and $\hat{J}$ the Jacobian at $\hat{\beta}$.*

Proof sketch. Expanding the normal equations $\hat{J}^\top (y - f(\hat{\beta})) = 0$ to first order around β gives $\hat{\beta} - \beta \approx (J^\top J)^{-1} J^\top \varepsilon$, a linear function of the errors to which the central limit theorem applies; the regularity conditions control the remainder. Jennrich (1969) gives the proof, and Seber and Wild (1989) the practice. ∎

Three practical consequences are used in Chapter 11. First, if $J^\top J$ is nearly singular, some combination of parameters is barely identified and its variance is large; the condition number of $J^\top J$ measures this. Second, the objective is not convex, so a fit should be started from a grid of initial values and checked for convergence. Third, with a robust loss such as Huber's (Huber 1964), which is quadratic for small residuals and linear for large ones, the covariance becomes the sandwich $A^{-1} B A^{-1}$, with A the expected Hessian of the loss and B the covariance of its gradient (Vaart 1998, Chapter 5), and the formula above no longer applies. When in doubt, bootstrap the runs (Section A.2).

A.7 Variance components and designed experiments

In the *one-way random-effects model*, $Y_{ij} = \mu + a_i + e_{ij}$ for I groups and J runs per group, with all a_i and e_{ij} independent, the a_i of mean zero and variance σ_a^2, and the e_{ij} of mean zero and variance σ_e^2. A group may be an initialization seed with J data orders, or a hardware configuration with J

seeds. The mean squares are

$$\mathrm{MS}_A = \frac{J}{I-1}\sum_{i=1}^{I}(\bar{Y}_{i\cdot}-\bar{Y}_{\cdot\cdot})^2, \qquad \mathrm{MS}_E = \frac{1}{I(J-1)}\sum_{i=1}^{I}\sum_{j=1}^{J}(Y_{ij}-\bar{Y}_{i\cdot})^2.$$

Theorem A.12 (Variance components). *In the one-way random-effects model with $I, J \geq 2$, $\mathbb{E}\,\mathrm{MS}_E = \sigma_e^2$ and $\mathbb{E}\,\mathrm{MS}_A = \sigma_e^2 + J\sigma_a^2$. Hence $\hat{\sigma}_e^2 = \mathrm{MS}_E$ and $\hat{\sigma}_a^2 = (\mathrm{MS}_A - \mathrm{MS}_E)/J$ are unbiased.*

Proof. Both statements follow from one fact: for independent $Z_1, \ldots, Z_r$ with common variance v, $\mathbb{E}\sum_l (Z_l - \bar{Z})^2 = (r-1)v$. Within group i, $Y_{ij} - \bar{Y}_{i\cdot} = e_{ij} - \bar{e}_{i\cdot}$, so the inner sum in MS_E is the sum of squared deviations of J independent errors and has expectation $(J-1)\sigma_e^2$; summing over the I groups and dividing by $I(J-1)$ gives σ_e^2. The group means $\bar{Y}_{i\cdot} = \mu + a_i + \bar{e}_{i\cdot}$ are independent with common variance $\sigma_a^2 + \sigma_e^2/J$, so $\mathbb{E}\sum_i (\bar{Y}_{i\cdot} - \bar{Y}_{\cdot\cdot})^2 = (I-1)(\sigma_a^2 + \sigma_e^2/J)$, and multiplying by $J/(I-1)$ gives $J\sigma_a^2 + \sigma_e^2$. ∎

The estimate $\hat{\sigma}_a^2$ can be negative when the true component is small; it is then reported as zero, or the components are estimated by restricted maximum likelihood (Searle et al. 1992). The crossed two-way design used for seeds in Chapter 1 is analyzed in the same way.

In a *two-level factorial design*, each of k factors is set to a low level, coded -1, or a high level, coded $+1$, every one of the 2^k combinations is run r times, and $N = r2^k$. For a nonempty set S of factors and a run with levels $x_1, \ldots, x_k$, put $c_S = \prod_{j \in S} x_j$. The *effect* of S is estimated by the mean outcome over the runs with $c_S = +1$ minus the mean over the runs with $c_S = -1$: a main effect when S has one factor, an interaction when it has more (Box et al. 2005).

Proposition A.13 (Factorial contrasts). *If the runs of a full two-level factorial design are independent with common variance σ^2, the $2^k - 1$ effect estimates are uncorrelated and each has variance $4\sigma^2/N$.*

Proof. Each sign c_S takes the value $+1$ on half of the runs, so the estimate of the effect of S is $\hat{\theta}_S = \frac{2}{N}\sum_n c_S(n) Y_n$, where n runs over all runs. Its variance is $\frac{4}{N^2}\sum_n c_S(n)^2\sigma^2 = 4\sigma^2/N$. For two different sets S and T, $c_S c_T = c_{S \triangle T}$ because $x_j^2 = 1$, and the symmetric difference $S \triangle T$ is nonempty; in a full factorial design every nonempty sign takes each value on exactly half of the runs, so

$$\mathrm{Cov}(\hat{\theta}_S, \hat{\theta}_T) = \frac{4\sigma^2}{N^2}\sum_n c_{S\triangle T}(n) = 0.$$ ∎

Two further principles of design, due to Fisher (1935), complete the toolkit. *Randomize* the order and placement of runs (which machine, which time, which batch of the queue), so that drifts and hidden nuisance variables cannot line up with the treatment. *Block* on known nuisance variables, comparing treatments within blocks: running both arms with the same seeds, or evaluating both on the same test items, is blocking, and its analysis is the paired one of Section A.1. Montgomery (2017) is a standard textbook.

A.8 Sample sizes and power

Proposition A.14 (Runs needed for a comparison). *Two arms have independent normal outcomes with common known standard deviation σ and n runs each. The two-sided level-α test that rejects $\Delta = 0$ when $|Z| > z_{1-\alpha/2}$, with $Z = (\bar{Y}_1 - \bar{Y}_2)/(\sigma\sqrt{2/n})$, has power*

$$\Phi(\delta - z_{1-\alpha/2}) + \Phi(-\delta - z_{1-\alpha/2}), \qquad \delta = |\Delta|\sqrt{n/2}/\sigma,$$

at the effect Δ. In particular, power at least π is guaranteed by $n \geq 2\sigma^2(z_{1-\alpha/2} + z_\pi)^2/\Delta^2$.

Proof. The statistic Z is normal with variance 1 and mean $\Delta\sqrt{n/2}/\sigma$, so $\mathbb{P}(|Z| > z_{1-\alpha/2}) = \Phi(\delta - z_{1-\alpha/2}) + \Phi(-\delta - z_{1-\alpha/2})$, an expression that depends on Δ only through δ. The second term is nonnegative, so the power is at least $\Phi(\delta - z_{1-\alpha/2})$, which is at least π when $\delta \geq z_{1-\alpha/2} + z_\pi$; solving for n gives the stated bound. ∎

For example, to detect a gain of $\Delta = 0.3$ points with a run-to-run standard deviation of $\sigma = 0.2$ points at level 0.05 with power 0.8 ($z_{0.975} \approx 1.960$, $z_{0.8} \approx 0.842$), $n \geq 2 \cdot 0.04 \cdot 2.802^2/0.09 \approx 6.98$, so seven runs per arm suffice. With σ estimated from the runs, the t quantiles replace the normal ones and the answer rises slightly; with paired runs, $2\sigma^2$ is replaced by $\mathrm{Var}(Y_1 - Y_2)$. Cohen (1988) is the classic reference on power.

A.9 Rank statistics

For n pairs (X_i, Y_i) without ties, a pair of indices $i < j$ is *concordant* if $(X_i - X_j)(Y_i - Y_j) > 0$ and discordant otherwise. With C concordant and D discordant pairs, *Kendall's* τ is $\hat{\tau} = (C - D)/\binom{n}{2} \in [-1, 1]$. It depends on the data only through their ranks, so it is unchanged by monotone transformations of either variable.

Proposition A.15 (Kendall's τ under independence). *If X and Y are independent with continuous distributions, then $\mathbb{E}\hat{\tau} = 0$ and $\mathrm{Var}\,\hat{\tau} = 2(2n + 5)/(9n(n - 1))$.*

Proof sketch. For $i \neq j$ the differences $X_i - X_j$ and $Y_i - Y_j$ are independent, continuous and symmetric about zero, so each pair is concordant with probability $1/2$ and $\mathbb{E}\hat{\tau} = 0$. The variance is a computation with the dependence between pairs that share an index; see Kendall (1938) and Kendall (1970). ∎

When the runs of a sweep fall into strata, a correlation computed within each stratum and then averaged, the *granulated* coefficient of Jiang et al. (2020), removes the part of the pooled correlation that the stratifying variable creates (Chapter 10).

The *probability of improvement* of one arm over another is $\theta = \mathbb{P}(X > Y) + \frac{1}{2}\mathbb{P}(X = Y)$ for independent outcomes X and Y of the two arms.

Proposition A.16 (Mann–Whitney estimate). *For independent samples $X_1, \dots, X_{n_1}$ and $Y_1, \dots, Y_{n_2}$ of the two arms,*

$$\hat{\theta} = \frac{1}{n_1 n_2} \sum_{i=1}^{n_1} \sum_{j=1}^{n_2} \Big(\mathbf{1}\{X_i > Y_j\} + \tfrac{1}{2}\mathbf{1}\{X_i = Y_j\} \Big)$$

is unbiased for θ. If the two arms have the same continuous distribution, then $\theta = 1/2$ and $\operatorname{Var}\hat{\theta} = (n_1 + n_2 + 1)/(12 n_1 n_2)$.

Proof. Each pair (X_i, Y_j) has the distribution of (X, Y), so each summand has expectation θ, and so has their average. Under a common continuous distribution, X and Y are exchangeable and tie with probability zero, so $\theta = 1/2$. The variance is that of the Mann–Whitney statistic $U = n_1 n_2 \hat{\theta}$, computed by Mann and Whitney (1947). ∎

A.10 Proportions near zero

Let $X \sim \operatorname{Bin}(m, p)$ count the successes on m independent test items. For $x < m$ let $p_U(x)$ be the solution q of $\mathbb{P}_q(X \le x) = \alpha/2$, and put $p_U(m) = 1$; for $x > 0$ let $p_L(x)$ be the solution q of $\mathbb{P}_q(X \ge x) = \alpha/2$, and put $p_L(0) = 0$.

Proposition A.17 (Binomial intervals near zero). *(i) The interval $[p_L(X), p_U(X)]$ of Clopper and Pearson (1934) contains p with probability at least $1 - \alpha$, for every p.*

(ii) If $X = 0$, then the one-sided upper confidence bound at level $1 - \alpha$, which is the solution q of $(1 - q)^m = \alpha$, equals $1 - \alpha^{1/m}$; it is at most $\ln(1/\alpha)/m$, about $3/m$ at $\alpha = 0.05$.

Proof. For (i), fix p. For $x < m$ the probability $\mathbb{P}_q(X \le x)$ is continuous and strictly decreasing in q, so $p_U(x) < p$ holds exactly when $\mathbb{P}_p(X \le x) < \alpha/2$. Let $x^\star$ be the largest x with $\mathbb{P}_p(X \le x) < \alpha/2$, or -1 if there is none; the cumulative probabilities increase in x, so the event $\{p_U(X) < p\}$ is contained in $\{X \le x^\star\}$, whose probability is less than $\alpha/2$. The same argument with $\mathbb{P}_q(X \ge x)$, which increases in q for $x > 0$, gives $\mathbb{P}_p(p_L(X) > p) < \alpha/2$, and the union bound gives the coverage. For (ii), $(1 - q)^m$ decreases from 1 to 0 on $[0, 1]$ and equals α at $q = 1 - \alpha^{1/m} = 1 - e^{-\ln(1/\alpha)/m}$, which is at most $\ln(1/\alpha)/m$ because $1 - e^{-t} \le t$; and $\ln 20 \approx 3.0$. ∎

The rule of three is the subject of a short paper by Hanley and Lippman-Hand (1983). The Clopper–Pearson interval is conservative; the interval of Wilson (1927) is shorter and has close to nominal coverage, and the usual normal-approximation interval $\hat{p} \pm z_{1-\alpha/2}\sqrt{\hat{p}(1 - \hat{p})/m}$ fails badly near 0 and 1, where it can even have zero width (Brown et al. 2001).

Notes and References

The tools of this appendix are classical, and each has a standard source. For the t distribution and the theory of tests, Casella and Berger (2002) and Lehmann and Romano (2005); for the bootstrap, Efron (1979) and Efron and Tibshirani (1993); for asymptotics, including the delta method and M-estimators, Vaart (1998); for designed experiments, Fisher (1935), Box et al. (2005) and Montgomery (2017); for variance components, Searle et al. (1992); for extremes, Leadbetter et al. (1983).

TRANSLATING THE LITERATURE. Statistics texts write $\bar{X}$, s and n where this book writes $\bar{Y}$, S and n, and call the arms of an experiment treatments or groups. Machine learning papers often report "mean ± standard deviation"; the standard deviation describes the spread of runs, while the half-width of an interval describes the uncertainty of the mean, and the two differ by a factor of about $\sqrt{n}$ for large n. What Dodge et al. (2019) call expected validation performance is the expected maximum $\mathbb{E}M_k$ of Proposition A.9.

References

Benjamini, Y. and Y. Hochberg (1995). "Controlling the False Discovery Rate: A Practical and Powerful Approach to Multiple Testing". In: *Journal of the Royal Statistical Society: Series B* 57.1, pp. 289–300.

Benjamini, Y. and D. Yekutieli (2001). "The Control of the False Discovery Rate in Multiple Testing under Dependency". In: *The Annals of Statistics* 29.4, pp. 1165–1188.

Bickel, P. J. and D. A. Freedman (1981). "Some Asymptotic Theory for the Bootstrap". In: *The Annals of Statistics* 9.6, pp. 1196–1217.

Box, G. E. P., J. S. Hunter, and W. G. Hunter (2005). *Statistics for Experimenters: Design, Innovation, and Discovery*. 2nd ed. Wiley.

Brown, L. D., T. T. Cai, and A. DasGupta (2001). "Interval Estimation for a Binomial Proportion". In: *Statistical Science* 16.2, pp. 101–133.

Casella, G. and R. L. Berger (2002). *Statistical Inference*. 2nd ed. Duxbury.

Clopper, C. J. and E. S. Pearson (1934). "The Use of Confidence or Fiducial Limits Illustrated in the Case of the Binomial". In: *Biometrika* 26.4, pp. 404–413.

Cohen, J. (1988). *Statistical Power Analysis for the Behavioral Sciences*. 2nd ed. Hillsdale, NJ: Lawrence Erlbaum Associates.

David, H. A. and H. N. Nagaraja (2003). *Order Statistics*. 3rd ed. Hoboken, NJ: Wiley.

Dietterich, T. G. (1998). "Approximate Statistical Tests for Comparing Supervised Classification Learning Algorithms". In: *Neural Computation* 10.7, pp. 1895–1923.

Dodge, J., S. Gururangan, D. Card, R. Schwartz, and N. A. Smith (2019). "Show Your Work: Improved Reporting of Experimental Results". In: *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing*, pp. 2185–2194.

Efron, B. (1979). "Bootstrap Methods: Another Look at the Jackknife". In: *The Annals of Statistics* 7.1, pp. 1–26.

Efron, B. (1987). "Better Bootstrap Confidence Intervals". In: *Journal of the American Statistical Association* 82.397, pp. 171–185.

Efron, B. and R. J. Tibshirani (1993). *An Introduction to the Bootstrap*. New York: Chapman & Hall.

Fisher, R. A. (1935). *The Design of Experiments*. Edinburgh: Oliver and Boyd.

Hanley, J. A. and A. Lippman-Hand (1983). "If Nothing Goes Wrong, Is Everything All Right? Interpreting Zero Numerators". In: *JAMA* 249.13, pp. 1743–1745.

Holm, S. (1979). "A Simple Sequentially Rejective Multiple Test Procedure". In: *Scandinavian Journal of Statistics* 6.2, pp. 65–70.

Huber, P. J. (1964). "Robust Estimation of a Location Parameter". In: *The Annals of Mathematical Statistics* 35.1, pp. 73–101.

Jennrich, R. I. (1969). "Asymptotic Properties of Non-Linear Least Squares Estimators". In: *The Annals of Mathematical Statistics* 40.2, pp. 633–643.

Jiang, Y., B. Neyshabur, H. Mobahi, D. Krishnan, and S. Bengio (2020). "Fantastic Generalization Measures and Where to Find Them". In: *International Conference on Learning Representations*.

Kendall, M. G. (1938). "A New Measure of Rank Correlation". In: *Biometrika* 30.1/2, pp. 81–93.

Kendall, M. G. (1970). *Rank Correlation Methods*. 4th ed. London: Griffin.

Leadbetter, M. R., G. Lindgren, and H. Rootzén (1983). *Extremes and Related Properties of Random Sequences and Processes*. New York: Springer.

Lehmann, E. L. and J. P. Romano (2005). *Testing Statistical Hypotheses*. 3rd ed. New York: Springer.

Mann, H. B. and D. R. Whitney (1947). "On a Test of Whether One of Two Random Variables is Stochastically Larger than the Other". In: *The Annals of Mathematical Statistics* 18.1, pp. 50–60.

McNemar, Q. (1947). "Note on the Sampling Error of the Difference Between Correlated Proportions or Percentages". In: *Psychometrika* 12.2, pp. 153–157.

Montgomery, D. C. (2017). *Design and Analysis of Experiments*. 9th ed. Wiley.

Phipson, B. and G. K. Smyth (2010). "Permutation P-values Should Never Be Zero: Calculating Exact P-values When Permutations Are Randomly Drawn". In: *Statistical Applications in Genetics and Molecular Biology* 9.1.

Satterthwaite, F. E. (1946). "An Approximate Distribution of Estimates of Variance Components". In: *Biometrics Bulletin* 2.6, pp. 110–114.

Searle, S. R., G. Casella, and C. E. McCulloch (1992). *Variance Components*. New York: Wiley.

Seber, G. A. F. and C. J. Wild (1989). *Nonlinear Regression*. New York: Wiley.

Student (1908). "The Probable Error of a Mean". In: *Biometrika* 6.1, pp. 1–25.

Vaart, A. W. van der (1998). *Asymptotic Statistics*. Cambridge: Cambridge University Press.

Welch, B. L. (1947). "The Generalization of "Student's" Problem when Several Different Population Variances are Involved". In: *Biometrika* 34.1/2, pp. 28–35.

Wilson, E. B. (1927). "Probable Inference, the Law of Succession, and Statistical Inference". In: *Journal of the American Statistical Association* 22.158, pp. 209–212.

Index

ablation, 159
    fixed-hyperparameter effect, 162
    leave-one-out, 120, 159
    re-tuned effect, 162
adaptive gradient methods, 73
agreement, 305
allocation exponent, 221, 241
assay sensitivity, 98
Atari 100k, 52
attribution
    path, 121
    Shapley, 121

backtest, 266
batch invariance, 34
batch normalization, 174
    Jacobian, 181
Benjamini–Hochberg procedure, 329
best of k, 17
bootstrap, 326
    over runs, 222
    percentile interval, 326
    stratified, 59, 326
breakthroughness, 278
Brier score, 284
broken power law, 260
budget, 4
    axis, 138
budget curve, 78

cell mean, 118
churn, 41
Clopper–Pearson interval, 334
co-treatment, 118
Cohen's kappa, 305
compute-optimal allocation, 220, 237
concomitant, 82
confounding, 118
control
    historical, 98
    negative, 98
    placebo, 98
    positive, 98, 201
    sham, 98
convergence check, 224
counterbalancing, 307
covariate shift, 174
coverage, 266
critical batch size, 137
cross-fitting, 144
crossed design, 18, 304

delta method, 221, 330
demonstration, 297
derived quantity, 138
divergence time, 40
domain of validity, 266
double descent, 269

edge of stability, 201
effect
    direct, 179
    indirect, 179
    total, 179
effective number of tasks, 56
emergent ability, 278
environment of a run, 36
evaluation instrument, 301
exact match, 283
expected maximum, 60, 329
expected validation performance, 83
experimental unit, 2
extrapolation, 261

factor, 4, 118
factorial design, 332
    effect, 160
    fractional, 162
factorial effect, 121
failed run, 18
false discovery rate, 60, 328
family-wise error rate, 60, 328
flat minimum, 194
floating-point arithmetic, 36
floor effect, 282

garden of forking paths, 61

generalization gap, 198
generalization measure, 198
gradient noise scale, 137
    simple, 143
granulated coefficient, 201
Gumbel distribution, 283

held-fixed variable, 4
Holm's procedure, 328
home-field advantage, 163
Huber loss, 220
human-normalized score, 51

identifiability, 225
in-context learning, 297
inclusion of optimizers, 76
instability of training, 34
instrument card, 312
interaction, 119, 160
    crossover, 120
    of arm and setting, 302
internal covariate shift, 175
interpolation, 261
interquartile mean, 56
inverse scaling, 257
isoFLOP profile, 216, 238
item churn, 16
item difficulty, 16
iterative magnitude pruning, 155

Kendall's tau, 333
Kendall's τ, 102
knockout, 179

ladder of scales, 261
latent quantity, 282
least absolute deviations, 223
linear scaling rule, 136
local exponent, 242
log score, 284
lottery ticket hypothesis, 155
Lyapunov exponent, 40

main effect, 160
main effect of a setting, 302
marginal value, 243
mask, 155
McNemar's test, 328
mediator, 179
metric, 282
misspecification, 264
momentum (heavy ball), 74
multiple comparisons, 60
multiple-choice grade, 284

naive contrast, 119
nested design, 304
neural architecture search, 93
noisy quadratic model, 140
nonlinear least squares, 331
nuisance variable, 4

one-factor-at-a-time design, 160
optimality gap, 56
order statistics, 329
outcome, 4
overhead (in parameter counts), 244

p-value
    valid, 325
pairing, 21
PassUntil, 280
penalty of a protocol, 246
per-token log error, 283
perfect scaling, 136
performance profile, 56
permutation test, 327
population-level inference, 55
positive homogeneity, 196
power, 21, 333
power iteration, 199
prediction interval, 261
probability of improvement, 56, 333
profile likelihood, 226
progress measure, 281
proper scoring rule, 284
protocol, 4
protocol card, 241
protocol of a scaling study, 241
proxy, 98
pseudoreplication, 2

random effects, 331
random search, 78, 98
ranking reversal, 302
regret, 162

reproducibility
  bitwise (repeatability), 36
  distributional, 36
  of a conclusion, 36
rescaling, function-preserving, 202
rescue, 179
resolution, 282
rule of three, 285, 334
run, 4
run distribution, 4, 14
running mean, 15

sample size, 333
scale invariance, 183
scale of measurement, 282
scaling law
  parametric, 219
scaling rule, 139
scaling study, 241
search space, 78
  headroom, 99
  spread, 99
seed, 14
selection bonus, 163
setting of an instrument, 301
  distribution, 301
sham ablation, 164
Shapley attribution, 121
sharpness
  adaptive, 199
  perturbation, 199
  top Hessian eigenvalue, 199
sharpness-aware minimization, 195
simple effect, 118, 160
Simpson's paradox, 200
standard error, 15, 325
steps to a target, 138
subsampling, 222
suite-level inference, 55
summation
  pairwise, 37
  recursive, 37
  tree, 37

t-interval, 325
tabular benchmark, 103
token edit distance, 280
tokens per parameter, 221
training recipe, 14
transition
  genuine, 287
  metric-induced, 287
treatment, 4
trial, 78
true error, 99
tuning budget, 78
tuning protocol, 78
twin run, 38

U-shaped scaling, 259
U-statistic, 83
unfinished schedule, 238
unit in the last place (ulp), 36
unit roundoff, 36

variable projection, 224
variance components, 19, 331
  of an evaluation, 304
vision transformer, 113

warmup–stable–decay schedule, 248
weight sharing, 93
Welch's test, 325
winner's curse, 101
winning ticket, 155

How This Book Was Made

Colophon

Roles

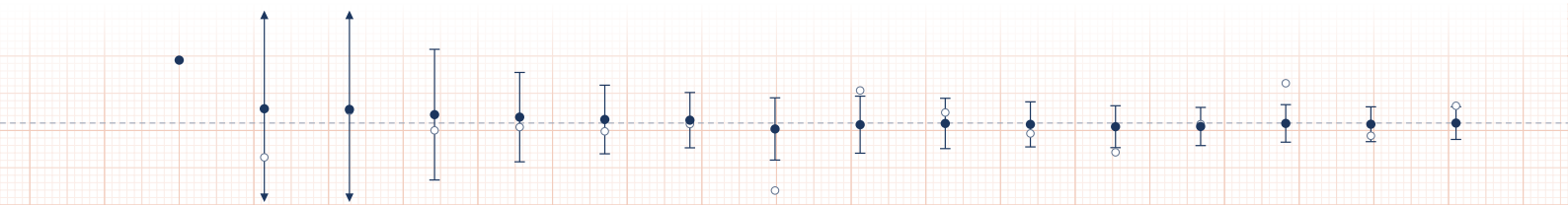
Deep Learning as an Experimental Science was written by Claude Opus 5.5, an AI model made by Anthropic, in October 2026. Xiaoyu Li conceived the series *The Missing Textbooks* and proposed this volume: its subject, its gap (an experimental science whose method is taught by word of mouth), its format (each chapter opening with a famous wrong conclusion and performing its post-mortem), its first examples, and its position relative to existing books. Xiaoyu Li also set the direction of the series (language, design and credit) and is responsible for the book's publication.

Process

A written plan fixed the thesis, the fifteen famous conclusions, the vocabulary of verdicts, the global notation, the conventions of the laboratory, and a specification for each chapter. A style guide fixed the anatomy of a chapter, the voice, a proof style that follows Halmos, and a rule of fairness: each claim is stated as its authors stated it before it is examined, and a verdict applies to a claim, never to a person. A coordinating instance of the model designed the volume's additions to the series typography, wrote the shared laboratory kit, the prologue, the front matter, the statistical appendix and the epilogue, ran the epilogue's study, and integrated the whole.

Separate instances of the same model drafted the fifteen chapters, five at a time, in three batches. Each verified its references and the claims it attributes to papers with web search, wrote and ran the scripts behind its figures, checked its theorems and constants numerically, and built its chapter with the volume's checks for errors, references, layout and style. The second batch was stopped at its start by a usage limit and resumed, with its context, after the limit reset. One author also had its chapter checked by an independent instance, which found that a coverage estimate had treated dependent forecasts as independent and tightened several attributions; the corrections are in the chapter.

The chapters corrected the plan in several places, among them the direction of a parameter-counting effect and the effect of unfinished learning-rate schedules in Chapter 12, the description of proper scoring rules in Chapter 14, and the strength of the verdict in Chapter 6; where the plan and a chapter differ, the chapter is correct. Several laboratory miniatures did not reproduce parts of the results they were built to show, and each chapter says which.



Limits

No human has yet checked every proof or every attribution, and the chapters have not yet received the independent referee pass that the earlier volumes of the series received. Web search was available while the book was written, but the full texts of many papers could not be opened, so some claims about what a paper reports were checked against abstracts, tables and secondary accounts; the chapters say so where it matters. The laboratory experiments are deliberately small, and a miniature that reproduces a failure does not prove that the failure explains the original result. Results from 2024 to 2026 are cited from preprints where no refereed version could be found. Corrections are welcome and will be recorded in the repository of *The Missing Textbooks*.

Typeset with pdfLaTeX in `missingbook`, the series class built on KOMA-Script, with the volume style `laboratory`; Palatino (newpx) for text and mathematics, Source Sans and Source Code Pro for labels and code, biblatex and biber for references, Ti*k*Z and pgfplots for figures, whose data were computed with Python, NumPy, SciPy, scikit-learn and PyTorch. The title page is log-log paper. On it, a scaling law with an irreducible term is fitted, by least squares in the logarithm of the loss, to six measured scales of five runs each (synthetic, with five percent run-to-run noise); the filled marks are means with 95 percent intervals, the open ones single runs, and the band is a bootstrap 95 percent band for the fitted curve, which is dashed where it extrapolates. The foot of each opening page is millimetre paper carrying sixteen draws from a standard normal distribution, made with NumPy's generator and the seed 3407, one more draw for each chapter, with their running mean and a 95 percent t-interval; the first draw lies 2.8 standard deviations above the true mean and the sixteenth running mean lies 0.001 below it. Both are computed by `00-emblem.py`.